

O'REILLY®

RESTful Web API паттерны и практики

Связывание и оркестрация микросервисов
и распределенных данных



SPRINT
book

Майк Амундсен
Предисловие Мэтта Макларти

RESTful Web API Patterns and Practices Cookbook

*Connecting and Orchestrating Microservices
and Distributed Data*

Mike Amundsen

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

RESTful Web API

паттерны и практики

*Связывание и оркестрация микросервисов
и распределенных данных*

Майк Амундсен

SPRINT
book

2025

Выпущено
при поддержке

КРОК

Майк Амундсен
RESTful Web API паттерны и практики

Перевел с английского Д. Брайт

ББК 32.988.02-018
УДК 004.415.2:004.738.5

Амундсен Майк

A62 RESTful Web API. Паттерны и практики. — Астана: «Спринт Бук», 2025. — 464 с.: ил.
ISBN 978-601-08-4867-2

В настоящее время многие организации создают и поддерживают приложения, основанные на сторонних сервисах. Разработчикам программного обеспечения и архитекторам часто приходится заниматься координацией и поддержкой приложений на основе API существующих сервисов, работающих за пределами их экосистемы. Найдите проверенные рецепты, которые помогут объединить множество разрозненных компонентов в вашей сети.

Майк Амундсен предлагает пошаговые решения для поиска, связывания и поддержки приложений, созданных сторонними разработчиками. Независимо, делаете ли вы мобильные приложения, ориентированные на людей, или мощные решения, основанные на межмашинных взаимодействиях, вы найдете правила, процедуры, команды и протоколы объединения отдельных API, способствующие их безопасной, масштабируемой и надежной совместной работе.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др.

ISBN 978-1098106744 англ.	Authorized Russian translation of the English edition of RESTful Web API Patterns and Practices Cookbook ISBN 978-1098106744 © 2023 Mike Amundsen. This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.
ISBN 978-601-08-4867-2	© Перевод на русский язык ТОО «Спринт Бук», 2025 © Издание на русском языке, оформление ТОО «Спринт Бук», 2025

Изготовлено в России. Изготовитель: ТОО «Спринт Бук». Место нахождения и фактический адрес: 010000, Казахстан, город Астана, район Алматы, проспект Рахымжан Кошкарбаев, д. 10/1, н. п. 18.

Дата изготовления: 03.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Подписано в печать 26.02.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 37,410. Тираж 700. Заказ 0000.

Краткое содержание

Предисловие	13
Введение	15
Благодарности	24
От издательства	26

ЧАСТЬ I. ЗНАКОМСТВО С RESTFUL ГИПЕРМЕДИА

Глава 1. Знакомство с RESTful Web API.....	28
Глава 2. Мышление и проектирование в контексте гипермедиа	46

ЧАСТЬ II. РЕЦЕПТЫ ДЛЯ РАБОТЫ С ГИПЕРМЕДИА

Глава 3. Проектирование гипермедиа	84
Глава 4. Клиенты гипермедиа	128
Глава 5. Сервисы на основе гипермедиа.....	191
Глава 6. Распределенные данные.....	274
Глава 7. Рабочий поток на основе гипермедиа	345
Глава 8. Подведение итогов	440

ПРИЛОЖЕНИЯ

Приложение А. Руководящие принципы RESTful Web API	448
Приложение Б. Дополнительные материалы	450
Приложение В. Сопутствующие стандарты.....	452
Приложение Г. Использование HyperCLI	455
Об авторе	462
Иллюстрация на обложке	463

Оглавление

Краткое содержание..... 5

Предисловие 13

Введение 15

 О книге 15

 Условные обозначения..... 22

Благодарности 24

От издательства 26

 О научном редакторе русского издания..... 26

ЧАСТЬ I.
ЗНАКОМСТВО С RESTFUL ГИПЕРМЕДИА

Глава 1. Знакомство с RESTful Web API..... 28

 Что такое RESTful Web API..... 29

 Почему гипермедиа 35

 Общие принципы масштабируемых сервисов в веб-среде 42

Глава 2. Мышление и проектирование в контексте гипермедиа 46

 Закладываем основы с помощью проектирования гипермедиа 49

 Повышение отказоустойчивости клиентов гипермедиа..... 53

 Повышение стабильности и изменяемости сервисов на основе гипермедиа 61

 Поддержка распределенных данных 68

 Увеличение расширяемости с помощью рабочего потока на основе гипермедиа..... 75

ЧАСТЬ II.
РЕЦЕПТЫ ДЛЯ РАБОТЫ С ГИПЕРМЕДИА

Глава 3. Проектирование гипермедиа 84

 3.1. Обеспечение интероперабельности с помощью зарегистрированных медиатипов 86

3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.....	88
3.3. Предоставление информации о предметной области через публикуемые словари	91
3.4. Описание предметной области с помощью семантических профилей	95
3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа	100
3.6. Проектирование последовательных операций записи с помощью идемпотентных действий.....	104
3.7. Обеспечение интероперабельности с помощью передачи состояния между сервисами	106
3.8. Проектирование с учетом возможности повторения действий.....	111
3.9. Проектирование с учетом возможной отмены действий.....	115
3.10. Проектирование с учетом возможности расширения сообщений	119
3.11. Проектирование с учетом возможности изменения интерфейсов	123
Глава 4. Клиенты гипермедиа.....	128
4.1. Ограничение использования жестко прописанных URL.....	130
4.2. Программирование клиентов, понимающих HTTP.....	134
4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений	137
4.4. Программирование эффективных клиентов, понимающих профили словарей	140
4.5. Согласование поддержки профилей во время выполнения.....	144
4.6. Управление форматами представления в среде выполнения	147
4.7. Использование документов схем в качестве источника метаданных сообщений.....	151
4.8. Каждому важному элементу ответа нужен идентификатор.....	156
4.9. Использование средств гипермедиа в ответе.....	159
4.10. Поддержка ссылок и форм для сервисов без поддержки гипермедиа	163
4.11. Проверка свойств данных в среде выполнения.....	167
4.12. Использование документов схем для валидации исходящих сообщений.....	169
4.13. Использование запросов документов для проверки входящих сообщений	175
4.14. Валидация входящих данных.....	178
4.15. Сохранение состояния.....	182
4.16. Наличие цели	185

Глава 5. Сервисы на основе гипермедиа.....	191
5.1. Публикация не менее одного стабильного URL	193
5.2. Предотвращение утечек внутренних моделей.....	196
5.3. Преобразование внутренних моделей во внешние сообщения.....	199
5.4. Выражение внутренних функций в виде внешних действий	205
5.5. Оповещение клиента о поддерживаемых настройках	210
5.6. Поддержка согласования HTTP-содержимого	215
5.7. Публикация полноценных словарей для клиентов	219
5.8. Поддержка общих словарей в стандартных форматах	226
5.9. Публикация документов определений сервисов	232
5.10. Публикация метаданных API	236
5.11. Поддержка мониторинга состояния сервисов.....	240
5.12. Стандартизация отчетов об ошибках.....	246
5.13. Облегчение поиска сервисов с помощью реестра сервисов среды выполнения	250
5.14. Повышение скорости обработки с помощью клиентских идентификаторов.....	253
5.15. Повышение надежности с помощью идемпотентной операции создания.....	258
5.16. Предоставление зависимым сервисам запасного варианта выполнения	262
5.17. Использование семантических посредников для доступа к несовместимым сервисам.....	268
Глава 6. Распределенные данные.....	274
6.1. Соккрытие внутренних деталей хранилища данных.....	276
6.2. Делаем все изменения идемпотентными	280
6.3. Соккрытие связей данных для внешних действий.....	286
6.4. Использование HTTP URL-адресов для поддержки запросов типа Contains и AND.....	290
6.5. Возвращение метаданных в ответах на запросы	294
6.6. Возврат на запросы данных HTTP-кода состояния 200 или 400.....	300
6.7. Использование медиатипов для запросов данных	304
6.8. Игнорирование неизвестных полей данных	310
6.9. Повышение производительности с помощью директив кэширования.....	315

6.10. Изменение моделей данных в промышленной среде	321
6.11. Расширение удаленных хранилищ данных	327
6.12. Установка лимита на объемные ответы	332
6.13. Использование промежуточных сервисов для обмена данными	337
Глава 7. Рабочий поток на основе гипермедиа	345
7.1. Проектирование сервисов, совместимых с рабочими потоками	347
7.2. Поддержка общего состояния в рабочих потоках	352
7.3. Описание рабочего потока через код	356
7.4. Описание рабочего потока через DSL	359
7.5. Описание рабочего потока через документы	362
7.6. Поддержка RESTful-языка управления задачами	365
7.7. Предоставление ресурса прогресса, отражающего выполнение рабочих потоков	370
7.8. Возвращение всех сопутствующих действий	374
7.9. Возвращение последних использованных ресурсов	378
7.10. Паттерн «текущая работа» с сохранением состояния	383
7.11. Внедрение стандартной навигации по спискам	390
7.12. Поддержка частичной отправки форм	395
7.13. Использование наблюдения состояния для реализации рабочего потока под управлением клиента	398
7.14. Оптимизация запросов с помощью сохраненных повторов	408
7.15. Синхронный повтор незавершенной работы с ответом 202 Accepted	414
7.16. Краткосрочные исправления с помощью автоматических повторов	420
7.17. Поддержка локальной отмены или отката	424
7.18. Призыв на помощь	430
7.19. Масштабирование рабочего потока с помощью очередей и кластеров	433
7.20. Использование посредников рабочих потоков для включения несовместимых сервисов	436
Глава 8. Подведение итогов	440
Использование рецептов	440
Трансформация существующих сервисов	444
Дополнительные ресурсы	445
Дальнейшие шаги	446

ПРИЛОЖЕНИЯ

Приложение А. Руководящие принципы RESTful Web API	448
Приложение Б. Дополнительные материалы	450
Приложение В. Сопутствующие стандарты.....	452
Приемлемые зарегистрированные медиатипы для RESTful Web API	452
Форматы определения API	453
Форматы документов семантических профилей	454
Сопутствующие форматы гипермедиа.....	454
Приложение Г. Использование HyperCLI	455
Hello, Hyper!.....	455
Прочая информация.....	456
Команды HyperCLI.....	457
Об авторе	462
Иллюстрация на обложке	463

Непросто создавать и эффективно использовать масштабируемые и надежные веб-API, возможно, даже сложнее, чем вы думаете. Это пособие полно инструкций и принципов, которые помогут вам преодолеть все сопутствующие известные и неизвестные препятствия.

*Арно Лоре (Arnaud Lauret),
эксперт по API, руководитель
в OpenAPI и Postman*

Эта книга понятно объясняет, как создавать грамотные API. Для всех сложностей, с которыми вы сталкиваетесь, здесь найдутся рецепты решения и сопутствующие пояснения.

*Эрик Уайлд (Erik Wilde),
Catalyst, Axway*

Майк Амундсен в очередной раз представил лучшие практики и примеры избранных паттернов API. Это руководство легко читать, усваивать и применять.

*Викки Рейзельман (Vicki Reyzelman),
директор Platform and Governance*

Эта книга посвящена памяти архитектора и специалиста по теории проектирования Кристофера Александра (Christopher Alexander) (1936–2022).

*Спасибо, что помогаешь мне видеть более объемную картину,
пока я смотрю в сторону возможного.*

Предисловие

Какую же запутанную паутину мы сплели!

Моя карьера в сфере технологий началась, когда Всемирную сеть стали использовать в бизнесе. Так получилось, что на протяжении всей карьеры я занимался адаптацией веб-среды к сложному миру корпоративной структуры и стратегии цифрового бизнеса, особенно к тем веб-API, которые находятся в центре всех взаимодействий.

В 2012 году мне посчастливилось работать в команде с Майком Амундсеном. С тех пор мы вместе реализовали множество связанных с API проектов, и я постоянно у него учился. Именно Майк научил меня фундаментальным принципам веб и многому другому. Я не перестаю удивляться тому, как он находит детали, которые я упускаю и которые оказываются столь очевидными. В своей книге он объединил многие из этих принципов и уроков в очень практичный комплекс.

Когда в начале 1990-х Интернет только начинал набирать популярность, я учился на факультете математики. Мои профессора тогда были очень впечатлены этим технологическим прорывом, и это понятно, учитывая академические корни веб-среды. Помню, как мне приходилось выдергивать их из чтения чужих рецензий на фильмы, возвращая к ведению лекций по алгебре. В тех затхлых, заставленных книгами кабинетах кампуса было совсем неочевидно, что Всемирная паутина станет, пожалуй, самым трансформирующим человеческую жизнь изобретением за всю историю.

Как же мы перешли от пиксельных досок объявлений, посвященных личным интересам, к миру, в котором 70 % бизнес-транзакций происходят в цифровом виде (<https://oreil.ly/1Qazf>)? Немалая заслуга здесь того, что веб состоит из очень простого набора компонентов: клиентов, серверов, сетей, открытых протоколов и, конечно же, API. Какой бы сложной и вездесущей ни становилась веб-среда, эти компоненты по-прежнему будут ее основой.

Гиперссылки, или *гипермедиа*, — более обширное понятие. Они оказали на технологию особенно сильное влияние. Изначально созданные по подобию академических ссылок (тут снова профессора), гиперссылки не только стали компонентом, который привнес в веб-среду сетевой эффект, но и сформировали ожидания пользователей в отношении всех технологий. Для веб-среды не существовало руководства, только эпизодически встречающийся синий подчеркнутый текст, который подталкивал пользователей опираться на собственную интуицию. И об этом

принципе чутья постоянно говорит Майк, указывая на то, что он дает важное видение в контексте API и роли гипермедиа.

Так что миру, может, и не нужно руководство для веб, но ему определенно нужно руководство по разработке. На эту тему Майк уже писал или помогал писать книги: *RESTful Web APIs*, *RESTful Web Clients*, *Microservice Architecture*, *Continuous API Management*¹ и др. Издание «RESTful Web API. Паттерны и практики» — продолжение этой его работы. Здесь вы узнаете не только то, что технологии веб-среды являются фундаментом существующего вычислительного мира, но и то, что принципы работы этой среды лежат в основе масштабируемой архитектуры современных программных экосистем. Майк неразрывно связывает принципы веб-разработки с проектированием API, показывая, как можно успешно применять эти принципы. Здесь вы в первую очередь узнаете, как проектировать и создавать устойчивые к изменениям системы, которые пройдут проверку временем.

Любая организация непрерывно усложняет свой программный ландшафт и выстраивает все больше связей с партнерами и поставщиками в цифровой экосистеме. Если вы хотите освоить правила процветания в этой реальности, то это руководство для вас.

*Мэтт Макларти (Matt McLarty),
технический директор (Global Field CTO)
в компании MuleSoft, подразделении Salesforce,
Ванкувер, август 2022 года*

¹ Меджуи М., Уайлд Э., Митра Р., Амундсен М. Непрерывное развитие API. Правильные решения в изменчивом технологическом ландшафте. — СПб.: Питер, 2020.

Введение

Добро пожаловать в мир «RESTful Web API. Паттерны и практики»!

Это довольно оригинальное название, которое следует пояснить, чему и посвящено введение. Я собираюсь немного нарушить правила и поместить в начале книги много сопроводительного текста, который позволит вам сформировать общее представление о включенном в нее материале. Основное же изложение начнется с части I.

О книге

Цель книги — научить проектировщиков, архитекторов, разработчиков и специалистов по сопровождению ПО создавать интерфейсы сервисов (API), которые будут использовать обширные возможности веб-среды, попутно снижая затраты и риски, связанные с созданием надежных высокоуровневых сервисов, зависящих от других API и сервисов, которые доступны им только по сети.

Для этого я собрал более 70 рецептов и паттернов, которые изучал и использовал в течение не одного десятка лет, помогая клиентам проектировать, создавать и развертывать успешные бизнес-сервисы в Интернете. Думаю, вам окажется известен как минимум один из этих рецептов — возможно, под другим названием или в другой форме. Надеюсь, что вы найдете новые для себя подходы к решению известных задач.



За годы работы я понял, что проблемы в проектировании ПО редко меняются. А вот решения для них меняются часто, чему способствуют развитие технологий и мода индустрии. В книге мы будем рассматривать подобные традиционные проблемы, для которых я представляю современные и модные решения.

Поскольку это книга рецептов, исполняемого кода здесь будет немного. Но в ней будет много схем, фрагментов кода и примеров сетевых сообщений, также я дам пояснения к решаемым задачам. Разбираемые вопросы и сама дискуссия не будут

ориентированы на конкретные технологии и платформы. Все рецепты я представлю так, чтобы вы могли перевести их в код и компоненты, которые будут работать в вашей конкретной среде.

Для кого эта книга

Книга больше всего подойдет людям, занимающимся планированием, проектированием и реализацией интерфейсов сервисов, работающих по HTTP. Для одних это будет означать акцент на создании производителей и потребителей сервисов корпоративного уровня. Для других — создание сервисов, способных надежно функционировать в открытой веб-среде, масштабируемых и доступных для потребителей по всему миру. В любом случае это будет означать создание грамотных программных интерфейсов, позволяющих программистам решать стоящие перед ними задачи.

Независимо от того, размещаете вы свои решения локально на собственных аппаратных ресурсах или же создаете ПО, которое будет работать в облаке, приведенные здесь рецепты помогут вам понять сопутствующие сложности и предоставят техники прогнозирования проблем и реализации возможности восстановления в непредвиденных случаях.

Что здесь освещается

Поскольку книга предназначена для широкой аудитории, я разделил ее на главы в соответствии с различными темами. Главы 1 и 2 составляют часть I книги. В них мы изучим историю и основы работы разделяемых сервисов (shared services) в Интернете. Ее можно рассматривать как предысторию технологии гипермедиа, которую мы будем изучать в части II. Как и в любом хорошем руководстве, каждая глава в части II содержит набор самодостаточных рецептов, которые можно использовать для решения конкретных проблем в ходе проектирования, создания и развертывания собственных веб-API.

ОНЛАЙН-РЕСУРСЫ

У этой книги есть несколько сопутствующих онлайн-ресурсов, включая репозиторий GitHub и связанные с ним веб-страницы, некоторые примеры и последние обновления каталога рецептов. Найти все эти ресурсы можно на сайте <http://WebAPICookbook.com>.

Вот краткий список глав с описанием их содержимого.

Часть I. Знакомство с RESTful гипермедиа

Открывающие книгу главы 1 и 2 описывают принципы, лежащие в основе всех включенных в нее рецептов. Они представляют смесь истории, философии и прагматического мышления. Эти идеи и принципы отражают все уроки, которые я усвоил за годы проектирования, создания и обслуживания сетевых приложений, работающих в Интернете.

Глава 1. Знакомство с RESTful Web API

Здесь в общих чертах описывается логическое обоснование выбранных рецептов. Эта глава содержит раздел, отвечающий на вопрос: «Что такое RESTful Web API?» В ней также объясняются причины значимости гипермедиа в создании RWA и приводятся общие базовые принципы, по которым выбирались рецепты для книги. То есть в этой главе мы «сервируем стол» для восприятия дальнейшего материала книги.

Глава 2. Мышление и проектирование в контексте гипермедиа

В этой главе мы изучим историю основанных на гипермедиа распределенных систем, которые формируют фундамент веб-приложений. Каждый набор инструкций в части II (проектирование, клиенты, сервисы, данные и рабочий поток) преподносится в виде смеси истории, философии и прагматического мышления. Чтение этой главы поможет вам понять некоторые фундаментальные идеи проектирования и технические основы для всех паттернов и практик, описанных далее в книге.

Часть II. Рецепты для работы с гипермедиа

Часть II содержит все инструкции, которые я собрал для этого выпуска. Вы заметите, что в названиях большинства глав есть слово «гипермедиа». Это указывает на общий подход, который мы будем использовать в книге.

Глава 3. Проектирование гипермедиа

Надежные и отказоустойчивые сервисы начинаются с продуманного проекта. В этой главе разбираются распространенные препятствия, которые вам встретятся еще до написания кода и релиза своих сервисов. Она окажется особенно полезной для архитекторов и разработчиков сервисов, а также заложит основу для последующих рецептов.

Глава 4. Клиенты гипермедиа

Эта глава акцентируется на сложностях, с которыми вы столкнетесь при создании приложений для потребителей сервисов/API. Я решил разобрать клиентские приложения до того, как говорить о рецептах для самих интерфейсов сервисов. Общий подход к созданию гибких и отказоустойчивых потребителей сервисов необходим для любого проекта, нацеленного на создание стабильной

и надежной платформы для открытых сервисов, которая сможет функционировать как в Интернете, так и внутри корпорации.

Глава 5. Сервисы на основе гипермедиа

Имея твердую основу принципов проектирования и подобающим образом спроектированные клиентские приложения, будет проще создавать и выпускать стабильные сервисные модули, которые можно безопасно обновлять, не ломая существующих потребителей API. Этот набор рецептов акцентируется не только на принципах проектирования устойчивых интерфейсов сервисов, но и на важности поддержки схем восстановления при ошибках и надежности во время выполнения, чтобы ваши решения продолжали работать, даже если в некоторых частях системы возникли сбои.

Глава 6. Распределенные данные

Эта глава акцентируется на проблемах поддержки постоянных данных в распределенной онлайн-среде. Большинство приведенных здесь инструкций нацелены на повышение быстродействия, масштабируемости и надежности ваших сервисов обработки данных путем обеспечения их целостности даже в случае изменения внутренних моделей данных и реализаций во время выполнения.

Глава 7. Рабочий поток на основе гипермедиа

Последний набор рецептов акцентируется на создании рабочего потока сервисов в Интернете и управлении им. Основной сложностью здесь является создание безопасного и надежного набора решений для включения несвязанных сервисов в один отказоустойчивый поток — он будет решать проблему, о которой не знает ни один из сервисов по отдельности. Я оставил эту главу напоследок, так как она опирается на многие ранее приведенные в книге рецепты.

Глава 8. Подведение итогов

Последняя глава представляет собой обобщение пройденного материала и призвана помочь вам определиться со следующими шагами по применению изученных рецептов в своей среде.

Приложения

В книге есть несколько приложений, которые вы можете использовать в качестве дополнительного справочного материала. В процессе повествования на них иногда даются ссылки, но их можно рассматривать и как самостоятельную справку.

Приложение А. Руководящие принципы RESTful Web API

Это приложение является кратким мотивационным обоснованием выбора единого принципа, лежащего в основе всех представленных рецептов, а также раскрывает некоторые вторичные принципы, использованные для оформления пояснений и в конечном счете реализации этих паттернов в целом.

Приложение Б. Дополнительные материалы

В книге я буду рекомендовать дополнительные материалы, приводить цитаты из книг и статей, а также ссылаться на презентации и видео, являющиеся источником многих рекомендаций книги. Это приложение содержит самодостаточный список материалов для чтения и просмотра, который вы можете использовать в качестве справки и ориентира в ходе изучения рецептов.

Приложение В. Сопутствующие стандарты

Поскольку цель книги — научиться создавать сервисы, которые смогут успешно существовать в веб-среде, приведенные в ней примеры будут зависеть от ряда важных стандартов открытого Интернета. Это приложение содержит список актуальных в этом смысле документов.

Приложение Г. Использование HyperCLI

В нескольких местах книги я ссылаюсь на инструмент командной строки под названием HyperCLI. Его можно использовать для взаимодействия с сервисами, ориентированными на гипермедиа. Здесь мы кратко познакомимся с этим инструментом. В приложении также описаны другие онлайн-ресурсы, посвященные использованию HyperCLI и HyperLang.

Чего в книге нет

Поскольку это книга рецептов, ее материал не ориентирован на обучение читателя тому, как реализовывать приведенные паттерны и идеи. Если вы только знакомитесь с основами, на которых построена книга, то вам понадобятся и другие источники информации.

Далее приведены книги, которые я использовал в процессе обучения и консультирования по темам, подробно не разбираемым в этом пособии.

Протокол HTTP

Большинство инструкций этого руководства предназначены для HTTP-протокола. Подробнее узнать о его возможностях вы можете из книги *HTTP Developer's Handbook* Криса Шифлетта (Chris Shiflett) (Sams, 2003), которая очень помогла мне при освоении внутренних особенностей протокола HTTP. Несмотря на то что вышла она в 2003 году, ее материал по-прежнему представляет огромную ценность. Очень рекомендую.

Проектирование API

Изучить детали проектирования API для распределенных сервисов вы можете, прочитав мою книгу *Building Hypermedia APIs with HTML5 and Node* (O'Reilly, 2011). Тем, кто хочет подробнее познакомиться с программированием API, рекомендую свою более свежую книгу *Design and Build Great Web APIs* (Pragmatic Bookshelf, 2020). Это детальное пособие, посвященное всему жизненному циклу API.

Клиенты API

Программирование клиентов API/сервисов — это отдельный навык. Для более глубокого погружения в процесс создания гибких клиентских приложений на основе гипермедиа я рекомендую читателям свою книгу *RESTful Web Clients* (O'Reilly, 2017).

Веб-API

Для знакомства с деталями создания самих веб-API советую изучить книгу *RESTful Web APIs* (O'Reilly, 2013), которую я писал вместе с Леонардом Ричардсоном (Leonard Richardson), а также мою книгу *Design and Build Great Web APIs* (O'Reilly, 2020). Помимо них, я всегда держу под рукой *Principles of Web API Design* Джеймса Хиггинботэма (James Higginbotham) (Addison-Wesley, 2021) и *The Design of Web APIs* Арно Лоре (Arnaud Lauret) (Manning, 2019).

Данные

Для более развернутого изучения темы данных рекомендую книгу *Data Management at Scale*¹ Питхейна Стренгхольта (Piethein Strengholt) (O'Reilly, 2020) и *Data Governance: The Definitive Guid* Эврена Эрюрека (Evren Eryurek) с соавторами (O'Reilly, 2021).

Рабочий поток

Хорошим началом для изучения мира инжиниринга рабочих потоков станут книги *Practical Process Automation* Бернда Рюккера (Bernd Ruecker) (O'Reilly, 2021) и *Service Orchestration as Organization* Малинды Капурудже (Malinda Karuruge) с соавторами (O'Reilly, 2014).

Есть много других мудрых источников по теме проектирования и создания распределенных сервисов, часть которых перечислена в приложении Б.

О рецептах

Притом что рецепты в книге сгруппированы по темам (проектирование, клиент, сервер, данные, реестр и рабочий поток), все они выстроены по одному шаблону.

Задача

Короткое описание проблемы, с которой вы можете столкнуться в ходе проектирования и создания сервисов.

Решение

Описание предполагаемого решения (или решений), которое можно использовать при возникновении обозначенной проблемы.

¹ Стренгхольт П. Масштабируемые данные. Лучшие шаблоны высоконагруженных архитектур. — СПб.: Питер, 2022.

Пример

В некоторых случаях рецепт будет включать пример. Это может быть обмен HTTP-сообщениями (запрос/ответ) или даже короткий фрагмент псевдокода, демонстрирующий внутренний рабочий поток, связанный с решением.

Обсуждение

Раздел с более подробным описанием компромиссов, а также их недостатков и преимуществ. Нередко это самый важный раздел рецепта, поскольку немногие проблемы имеют лишь одно решение.

Читайте также

Многие рецепты будут завершаться списком из одного или более других актуальных рецептов. Некоторые из них опираются на другие или позволяют их реализовать, и именно здесь вы узнаете, как рецепты взаимодействуют друг с другом в реальных рабочих системах.

Как работать с книгой

Настоятельно рекомендую прочесть книгу от начала до конца, чтобы ознакомиться с изложенными в ней принципами и примерами во всей полноте. В то же время я понимаю, что времени не всегда хватает, а для получения пользы от книги вам может не требоваться полноценное погружение. С учетом этого предлагаю несколько способов прочтения, и то, какой вы выберете, будет зависеть от конкретного интереса, целей и количества времени, которое вы готовы уделить тексту.

Для тех, кто спешит

Если вы приобрели книгу недавно и хотите решить насущную проблему, просто найдите в оглавлении созвучный ей рецепт и переходите к нему. Как и любой хороший рецепт, он написан в виде полноценной инструкции. В нем могут присутствовать ссылки на другие рецепты в книге (в частности, в разделе «Читайте также»), которые вы можете при необходимости изучить.

Быстрое знакомство с общей картиной

Если вы хотите быстро ознакомиться с общей картиной, предлагаю прочесть главы 1 и 2, а также главу 8. Часть I сформирует у вас представление о наборе рецептов, познакомит с их историей и лежащими в основе техниками. Исходя из этого, вы сможете решить, нужно ли уделить внимание конкретному набору рецептов из части II или же просмотреть их все.

Тематическая справка для конкретных команд

Если вы являетесь частью команды, задача которой связана с одной или несколькими темами из книги (проектирование, клиентская сторона, сервисы, данные, рабочие потоки и т. д.), предлагаю сначала получить общее представление

(часть I), а затем выбрать конкретную тематическую главу в части II. Таким образом, продвигаясь в решении своей задачи, вы сможете использовать конкретные главы в качестве справки.

Глубокое погружение для архитекторов

Вдумчивое прочтение от корки до корки окажется полезным для тех, чья основная задача — проектирование открытых сервисов производителей и потребителей. Многие рецепты в книге подойдут для реализации серии одобренных корпоративных компонентов, которые можно безопасно объединять с целью создания отказоустойчивой и надежной основы настраиваемого под свои потребности сервиса. В этом случае книга может выступать в качестве набора рекомендаций для совместно используемых внутри корпорации библиотек.

Чек-лист для управления программами на уровне корпораций

Тем, перед кем стоит задача реализации общекорпоративных или других масштабных программ, я предлагаю сначала сформировать для себя общее представление, а затем использовать тематические главы в качестве руководства при создании собственных внутренних чек-листов для разработки и релиза RESTful Web API.

Наконец, книга писалась как полезное справочное руководство. Можете смело использовать наиболее полезные для вас ее части и бегло просматривать разделы, которые в вашей ситуации неактуальны. В будущем вы сможете обратиться к другим разделам для решения новых задач.

Условные обозначения

В этой книге используются следующие условные обозначения.

Курсив

Курсивом выделены новые термины или важные понятия.

Моноширинный шрифт

Применяется для листингов программ, а также используется внутри абзацев для обозначения таких элементов, как переменные и функции, базы данных, типы данных, переменные среды, операторы и ключевые слова, имена файлов и их расширения.

Моноширинный курсив

Показывает текст, который должен быть заменен значениями, введенными пользователем или определяемыми контекстом.

Шрифт без засечек

Применяется для обозначения URL, адресов электронной почты, названий кнопок и других элементов интерфейса, каталогов.



Этот рисунок указывает на совет или предложение.



Этот рисунок указывает на общее примечание.



Этот рисунок указывает на предупреждение.

Благодарности

Никто не достигает успеха без признания помощи других. Мудрые и уверенные люди признают эту помощь с благодарностью.

Альфред Норт Уайтхед
(*Alfred North Whitehead*)

Меня учили, вдохновляли, давали советы и подбадривали очень многие люди, поэтому я даже не знаю, как их перечислить. Но некоторые оказали мне особую помощь в процессе написания книги, и они заслуживают отдельного упоминания.

Как и любой из нас, я стою на плечах гигантов. За годы моего профессионального взросления многие из них вдохновляли меня, а с некоторыми мне посчастливилось встретиться лично. Вот те, чьи размышления и рекомендации помогли составить эту книгу: Суббу Алламараджу (Subbu Allamaraju), Белинда Барнет (Belinda Barnett), Тим Бернерс-Ли (Tim Berners-Lee), Мел Конвей (Mel Conway), Рой Филдинг (Roy Fielding), Джеймс Глик (James Gleick), Тед Нельсон (Ted Nelson), Марк Ноттингем (Mark Nottingham), Хольгер Райнхардт (Holger Reinhardt), Леонард Ричардсон (Leonard Richardson), Ян Робинсон (Ian Robinson) и Джим Веббер (Jim Webber).

Особенно хочу поблагодарить Лоринду Брэндон (Lorinda Brandon), Алианну Инзана (Alianna Inzana), Ронни Митру (Ronnie Mitra), Сэма Ньюмана (Sam Newman), Ираклия Надареишвили (Irakli Nadareishvili), Викки Рейзелмана (Vicki Reyzelman) и Эрика Уайлда (Erik Wilde) за помощь в вычитке фрагментов текста и предоставление прекрасной обратной связи.

Благодарю всех сотрудников издательства O'Reilly за их постоянную поддержку и мудрые советы. В частности, я глубоко признателен Майку Лукидесу (Mike Loukides) и Мелиссе Даффилд (Melissa Duffield), которые верили в этот проект задолго до того, как я отчетливо сформулировал его тематику и формат. Выражаю свою благодарность Кэтрин Тозер (Katherine Tozer), Соне Саруба (Sonia Saruba) и многим другим за всю закулисную работу, которая сделала публикацию книги возможной. Отдельное спасибо Кейт Дулли (Kate Dullea) и Диого Лукасу (Diogo Lucas) за иллюстрации для книги.

Наконец, огромное спасибо всем тем, кого я встречал в течение многих лет: организаторам конференций и руководителям курсов, небольшим и крупным компаниям, которые приглашали меня для выступления и консультирования, участникам курсов и множеству обитателей социальных сетей, которые задавали мне вопросы, позволяли вникать в процессы их организаций и помогали изучать, тестировать и оттачивать изложенные в книге идеи. Все, что вы увидите здесь, было рождено преимущественно благодаря тем, кто трудился на этом поприще до меня и кто продолжает неустанно изо дня в день работать над созданием систем, использующих принципы, перечисленные в приложении А.

От издательства

Мы выражаем огромную благодарность компании «КРОК» за помощь в работе над русскоязычным изданием книги и их вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@sprintbook.kz (издательство «SprintBook», компьютерная редакция).

Мы будем рады узнать ваше мнение!

О научном редакторе русского издания

Артем Маханкин — инженер-разработчик в компании «КРОК». Имеет более пяти лет опыта разработки высоконагруженных систем для государственного и коммерческого сектора. Активный участник-организатор в учебных программах BrainZ by CROC для студентов и школьников.

Часть I

Знакомство с RESTful гипермедиа

Разница между новичком и учителем лишь в том, что новичок еще не научился работать так, чтобы позволять себе небольшие ошибки. Учитель же знает, что при правильной последовательности действий он всегда сможет исправить недочеты на следующих этапах процесса. Это простое, но важное знание придает работе опытного плотника прекрасную, плавную, свободную и почти безмятежную простоту.

Кристофер Александер

ГЛАВА 1

Знакомство с RESTful Web API

Используйте глобальную досягаемость, чтобы решать задачи, о которых даже не думали, для людей, с которыми никогда не встречались.

Принцип RESTful Web API

Во введении я акцентировал внимание на специфическом названии книги. Здесь мы займемся изучением принципов, стоящих за *RESTful Web API*, а также поговорим о том, почему я считаю важным именно такое название и в чем его смысл.

Сначала я немного поясню, что значит фраза «RESTful Web API» и почему я решил использовать столь странное выражение. Затем мы поговорим о том, что я считаю основной движущей технологией, позволяющей создавать отказоустойчивые и надежные сервисы в открытом Интернете, а именно о *гипермедиа*. Завершит главу краткий раздел, посвященный изучению нескольких *общих принципов* реализации и применения интерфейсов сервисов на основе REST. На основе этих принципов выбирались и описывались паттерны и рецепты, собранные в книге.

Реализации на основе гипермедиа опираются на три основных элемента: сообщения, действия и словари (рис. 1.1). В гипермедиа-решениях сообщения передаются с помощью распространенных форматов, таких как HTML, Collection + JSON и SIREN. Эти сообщения содержат информацию, основанную на общем словаре предметной области, например PSD2 для банковской сферы, ACORD для страхования или FHIR для здравоохранения. Эти же сообщения включают четко определенные действия вроде *save*, *share*, *approve* и т. д.

Оперируя этими тремя принципами, я постараюсь сформировать у вас представление о том, как мы создаем и используем сервисы сегодня. Я покажу, как, слегка изменяя точку взгляда и подход, можно обновлять дизайн и реализацию этих сервисов, чтобы повысить их юзабилити, снизить затраты на их создание и доступ к ним, а также расширить для производителей и потребителей сервисов возможности создавать и поддерживать жизнеспособные, использующие API бизнесы, даже когда некоторые сервисы, от которых мы зависим, оказываются ненадежными или недоступными.



Рис. 1.1. Элементы гипермедиа

Для начала разберем смысл названия книги.

Что такое RESTful Web API

Я уже несколько лет использую фразу «RESTful Web API» в статьях, презентациях и обучающих материалах. Мы с Леонардом Ричардсоном в 2013 году выпустили целую книгу (<https://oreil.ly/a5Pwl>). Иногда этот термин вызывает непонимание и даже скептицизм, но почти всегда — любопытство. Что вообще эти три слова делают вместе? Что означает комбинация их сути в целом? Чтобы ответить на эти вопросы, для начала стоит прояснить значение каждого термина по отдельности.

В этом разделе мы разберем следующие определения.

REST по Филдингу

Архитектурный стиль, который подчеркивает масштабируемость взаимодействий компонентов, их независимое развертывание и универсальность интерфейсов.

Веб по Тиму Бернерсу-Ли

Всемирная сеть (World Wide Web) была представлена как универсальная связанная система обработки информации, в которой первостепенное значение имеют универсальность и портативность.

Позднее связывание по Алану Кею

Схема проектирования, позволяющая создавать системы, которые можно безопасно менять во время их работы.

REST по Филдингу

В далеком 1998 году Рой Т. Филдинг (Roy T. Fielding, <https://oreil.ly/cOvy4>) провел в Microsoft презентацию, на которой объяснил свой принцип *передачи состояния представления* (Representational State Transfer, сегодня известен как REST, <https://oreil.ly/DqhFK>). В этой презентации и докторской диссертации, которую он защитил двумя годами позже (*Architectural Styles and the Design of Network-based Software Architectures*, <https://oreil.ly/78U8r>), Филдинг продвигал идею, согласно которой для сетевых реализаций есть уникальный набор программных архитектур и один из шести выделенных им стилей, REST, особенно хорошо подходит для Всемирной паутины.



Спустя несколько лет я услышал распространенную нынче фразу: «Часто цитируют, никогда не читают». Этот острый комментарий хорошо подходит к диссертации Филдинга (2000). Я призываю всех, кто работает над созданием или обслуживанием ПО для веб-среды, уделить время и внимательно прочитать всю диссертацию, а не только ее пресловутую главу 5 под названием Representational State Transfer (<https://oreil.ly/PdLrH>). Описанное Филдингом более 20 лет назад разделение общих стилей на категории оказалось корректным, и многие из этих стилей позднее стали известны как gRPC, GraphQL, событийно-ориентированные, контейнеры и пр.

Использованный Филдингом метод определения желаемых свойств на уровне системы (доступность, производительность, простота, изменяемость и пр.), а также рекомендуемый им набор ограничений (клиент — сервер, отсутствие состояния, кэшируемость и т. д.) для *индуцирования* этих свойств и по прошествии более чем 20 лет оказывается полезным подходом к восприятию проектирования ПО, от которого требуется долгосрочная стабильность.

Хороший вариант обобщения предложенного Филдингом стиля REST содержится в самой диссертации (<https://oreil.ly/tl6U6>):

«REST предоставляет набор архитектурных ограничений, который в случае применения целиком выделяет масштабируемость взаимодействий компонентов, их независимое развертывание, универсальность интерфейсов и промежуточные компоненты для сокращения задержки взаимодействий, повышения безопасности и инкапсуляции устаревших систем».

Для этой книги были отобраны такие рецепты, которые позволят проектировать и создавать сервисы, демонстрирующие многие из *ключевых архитектурных свойств* (<https://oreil.ly/J0Fc2>), выявленных Филдингом. Далее приведены список этих свойств и краткое описание их назначения и способов применения.

Производительность

Производительность основанного на сетевых взаимодействиях решения привязана к физическим ограничениям сети — объему проходящего трафика

(throughput), пропускной способности (bandwidth), издержкам и т. д., а также способам повышения производительности на стороне пользователя, таким как задержка запросов и возможность сократить время завершения с помощью параллельных запросов.

Масштабируемость

Возможность архитектуры поддерживать большое число компонентов или взаимодействий между компонентами внутри действующей конфигурации.

Простота

Для упрощения решения в первую очередь используют принцип разделения обязанностей на этапе распределения функциональности внутри компонентов, а также принцип универсальности интерфейсов.

Изменяемость

Простота, с которой можно вносить изменения в архитектуру в виде развития, расширения, персонализированной настройки и повторного использования.

Видимость

Возможность компонента мониторить взаимодействие между двумя другими компонентами или делать его опосредованным с помощью кэшей, прокси-серверов или прочих посредников.

Портативность

Возможность выполнять одно и то же ПО в разных средах, включая возможность безопасного перемещения кода (например, JavaScript), а также данных между системами среды выполнения (например, Linux, Windows, macOS и т. д.).

Надежность

Степень уязвимости реализации к общесистемным сбоям в результате сбоя одного компонента (машины или сервиса) внутри сети.

Ключевая причина, по которой мы будем использовать в рецептах архитектурные принципы Филдинга, состоит в том, что они помогают получить реализации, которые можно масштабировать и безопасно изменять на больших дистанциях как в пространстве, так и во времени.

Веб по Тиму Бернерсу-Ли

Работа Филдинга опирается на труды другого пионера в онлайн-мире, сэра Тимоти Бернерса-Ли (<https://oreil.ly/GpDkH>). Более чем за десять лет до написания Филдингом диссертации Бернерс-Ли опубликовал 16-страничный документ *Information Management: A Proposal* (1989–1990 годы, <https://oreil.ly/ZE5qk>). В нем он предложил на тот момент уникальное решение для повышения эффективности

хранения и извлечения информации в лаборатории физики CERN, где работал. Эту свою идею Бернерс-Ли назвал Всемирной паутиной (World Wide Web) (рис. 1.2).

CERN DD/OC

Tim Berners-Lee, CERN/DD

Information Management: A Proposal

March 1989

Information Management: A Proposal

Abstract

This proposal concerns the management of general information about accelerators and experiments at CERN. It discusses the problems of loss of information about complex evolving systems and derives a solution based on a distributed hypertext system.

Keywords: Hypertext, Computer conferencing, Document retrieval, Information management, Project control

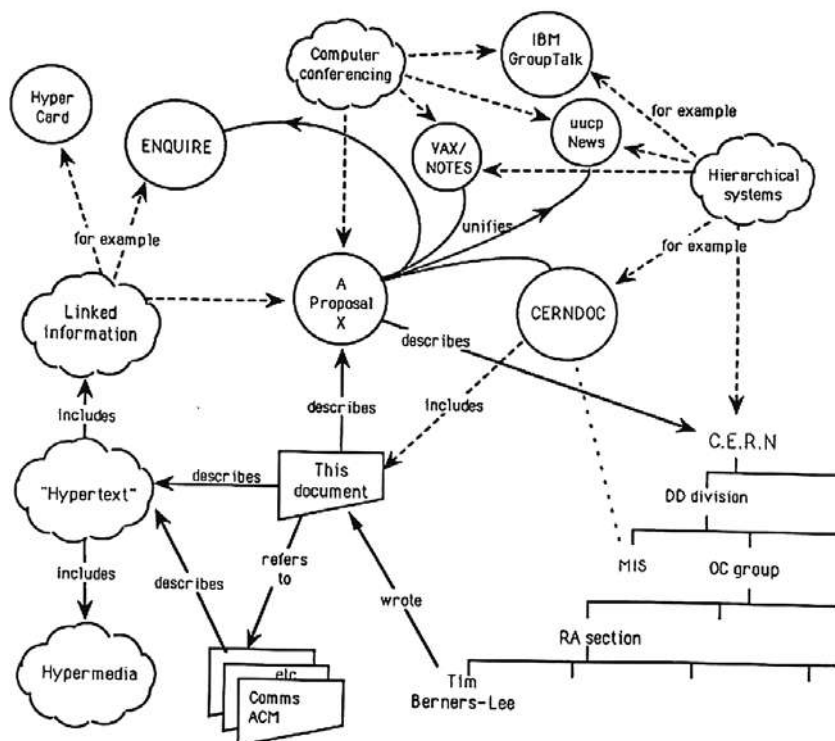


Рис. 1.2. Предложенная Бернерсом-Ли схема Всемирной паутины (1989)

Всемирная паутина (WWW) стала развитием идеи Теда Нельсона (Ted Nelson, <https://oreil.ly/mkyIW>), который придумал термин «*гипертекст*», соединяя связанные документы *ссылками*, а потом и *формами*, в которые пользователям предлагалось вводить данные, отправляемые на серверы по всему миру. Эти серверы легко и быстро настраивались с помощью бесплатного ПО, установленного на совместно используемых настольных компьютерах. Структура WWW соответствовала правилу максимальной простоты, которое гласит, что для решения задачи нужно задействовать самую простую из подходящих технологий. Иными словами, целью было сделать общее решение максимально простым (но не проще). Позже эта концепция была зафиксирована как документ W3C (<https://oreil.ly/h0B2Q>) с тем же именем.

Такой подход установил низкий порог входа для всех, кто хотел присоединиться к сообществу WWW, и помог вызвать его взрывную популярность в 1990-х и начале 2000-х годов.

ЦЕЛЬ ВСЕМИРНОЙ ПАУТИНЫ

В документе, описывавшем систему, которая позднее стала веб-средой (<https://oreil.ly/UHJ7>), Бернерс-Ли написал: «Нам нужно работать над созданием универсальной связанной информационной системы, в которой приоритетом будут универсальность и портативность».

В WWW любой документ можно было отредактировать, чтобы он ссылался (указывал) на другой документ в Сети. При этом никаких дополнительных процессов проделывать ни на одной из сторон этой связи не приходилось. По сути, люди могли свободно создавать собственные соединения, собирать свои избранные документы и создавать личный контент, не получая разрешения от кого бы то ни было. Создание всего этого контента становилось возможным за счет использования ссылок и форм внутри страниц, которые позволяли выстраивать уникальные пути и взаимодействия — такие, о которых авторы исходного документа (того, который подключался) ничего не знали.

Два аспекта WWW — правило максимальной простоты и свободу создавать собственные соединения — мы будем применять во всех приведенных в книге рецептах.

Позднее связывание по Алану Кею

Еще один важный принцип создания надежных и отказоустойчивых сервисов, способных *существовать в веб-среде*, был придуман Аланом Кеем (Alan Kay, <https://oreil.ly/CbNWf>) — ученым из США, работавшим в компьютерной сфере. Именно ему приписывают популяризацию термина «объектно-ориентированное программирование» в 1990-е годы.

АЛАН КЕЙ ПРО ООП

Объясняя свое видение *объектно-ориентированного программирования* (ООП) в сообщении для списка рассылки в 2003 году (<https://oreil.ly/vN8e4>), Кей написал: «Для меня ООП означает только: 1) обмен сообщениями, 2) локальное сохранение, защиту и сокрытие состояния объекта, 3) позднее связывание всего».

В 2019 году Кертис По (Curtis Poe, <https://oreil.ly/pfi4t>) написал статью, в которой разобрал данную Аланом Кеєм трактовку ООП и, помимо прочего, указал: «Позднее связывание важно, так как Кей утверждает, что оно позволяет не прибегать слишком рано к *одному истинному способу* решения проблемы, тем самым упрощая изменение этих решений, но при этом дает возможность создавать системы, которые вы сможете изменять *в процессе их работы!*» (выделение в тексте соответствует оригиналу).



Более непосредственный анализ связей между REST по Филдингу и ООП по Алану Кею изложен в моей статье 2015 года *The Vision of Kay and Fielding: Growable Systems that Last for Decades* (<https://oreil.ly/TLvzq>).

Аналогично тому, как Кей представлял программирование с помощью ООП, веб-среда, то есть сам Интернет, *работает непрерывно*. И сервисы, которые мы устанавливаем на подключенную к Интернету машину, по факту изменяют систему в процессе ее работы. Это нужно иметь в виду при создании сервисов для веб-среды.

Именно тот факт, что позднее связывание позволяет изменять системы во время их работы, мы будем использовать в качестве опорного принципа в рецептах книги.

Итак, в текущем разделе мы рассмотрели:

- использование представлений Филдинга о проектировании систем, допускающих безопасное масштабирование и изменение с течением времени;
- применение предложенного Бернерсом-Ли правила максимальной простоты и принципа снижения барьера входа, чтобы позволить любому подключенному к сети легко соединиться с любым другим ее участником;
- использование позднего связывания по Кею для упрощения изменения частей системы в процессе ее работы.

Важной техникой, которая поможет нам все это реализовать, является *гипермедиа*.

Почему гипермедиа

По моему опыту, концепция гипермедиа находится на пересечении нескольких важных инструментов и технологий, которые положительно повлияли на формирование нашего информационного общества. И я думаю, что она поможет повысить доступность и юзабилити сервисов в Интернете в целом.

В этом разделе мы поговорим:

- об эпохе гипермедиа;
- о ценности сообщений;
- о важности словарей;
- о магических строках Ричардсона.

История гипермедиа насчитывает почти 100 лет и прослеживается в работах XX века по психологии, взаимодействию человека и компьютера, а также в теории информации. Она лежит в основе схемы Всемирной паутины Бернерса-Ли (см. раздел «Веб по Тиму Бернерсу-Ли» ранее в этой главе) и может стать основой нашей сети API. Именно поэтому ее следует изучить подробнее. Для начала давайте определим, что такое гипермедиа и приложения на основе гипермедиа.

Гипермедиа

Термины «гипертекст» и «гипермедиа» придумал Тед Нельсон в начале 1950-х. Он использовал их в своей работе 1965 года, посвященной средствам ACM, *Complex Information Processing: A File Structure for the Complex, the Changing and the Indeterminate* (<https://oreil.ly/bqY3B>). Как писал в 2008 году Томас Исаковиц (Tomas Isakowitz), изначальная структура гипертекстовой системы (<https://oreil.ly/1Ggxv>) «состоит из *узлов*, которые содержат информацию, и *ссылок*, которые представляют связи между этими узлами». Гипермедиа-системы акцентируются на *связях* между их элементами.

По сути, гипермедиа дает возможность связывать между собой отдельные узлы, также называемые *ресурсами*, например документы, изображения, сервисы и даже фрагменты текста в документе. В сети эта связь реализуется с помощью универсальных идентификаторов ресурсов (universal resource identifiers, URI, <https://oreil.ly/ehJpL>). Когда такое соединение подразумевает возможность передачи данных, ссылки выражаются в виде *форм*, которые также можно предлагать пользователям или запрограммированным машинам для ввода информации. К примеру, HTML поддерживает создание ссылок и форм с помощью тегов `<a>`,

`<img>`, `<form>` и др. Существует несколько форматов, которые поддерживают ссылки и формы гипермедиа.

Эти элементы гипермедиа также можно возвращать вместе с результатами запроса. Возможность передавать ссылки и формы в ответах позволяет клиентским приложениям выбирать и активировать эти гипермедиаэлементы, чтобы обеспечить прогресс выполнения. Это дает возможность создавать сетевые решения, состоящие исключительно из серии ссылок и форм (вместе с возвращаемыми данными), которые при переходе по ним обеспечивают решение предусмотренной задачи (например, вычисление результатов, получение обновления и сохранение данных в удаленном расположении и т. д.).

Ссылки и формы обеспечивают универсальность интерфейсов (например, использование гипермедиадокументов по HTTP), которая лежит в основе гипермедиаприложений. Клиентские приложения на основе этой технологии, такие как HTML-браузеры, могут задействовать эту универсальность для поддержания широкого спектра новых приложений даже без изменения или обновления своего исходного кода. Мы просто движемся от одного решения к другому, проходя по ссылкам или вводя их вручную, и применяем одно и то же установленное клиентское приложение для чтения новостей, обновления списков дел, онлайн-игр и т. д.

Рецепты в этой книге основываются на структуре гипермедиа, чтобы выступать решением не только для приложений, управляемых людьми, вроде HTML-браузеров, но и для тех, которыми управляют машины. Это особенно актуально для клиентов, которые для доступа к сервисам сети опираются на API. В главе 4 я познакомлю вас с приложением командной строки, которое позволяет быстро создавать скрипты для клиентских приложений на основе гипермедиа, не меняя кода этих приложений (см. приложение Г).

Эпоха гипермедиа

Идея информационного соединения людей существовала давно. В 1930-х бельгиец Поль Отле (Paul Otlet, <https://oreil.ly/sxkh5>) представлял себе машину, которая позволит людям искать и составлять индивидуальные комбинации аудио-, видео- и текстового контента, а также получать к ним доступ из любой точки мира. Прошло почти 100 лет, и стриминговая революция наконец произошла.

Поль Отле

Схема подключения домашних машин к различным источникам новостей, развлечений и информации была представлена Отле в 1940 году (рис. 1.3). Эта схема, которую он назвал Всемирной сетью (World Wide Network), очень похожа на схемы Теда Нельсона (будет показана чуть позже) и Тима Бернерса-Ли (см. раздел «Веб по Тиму Бернерсу-Ли» ранее).

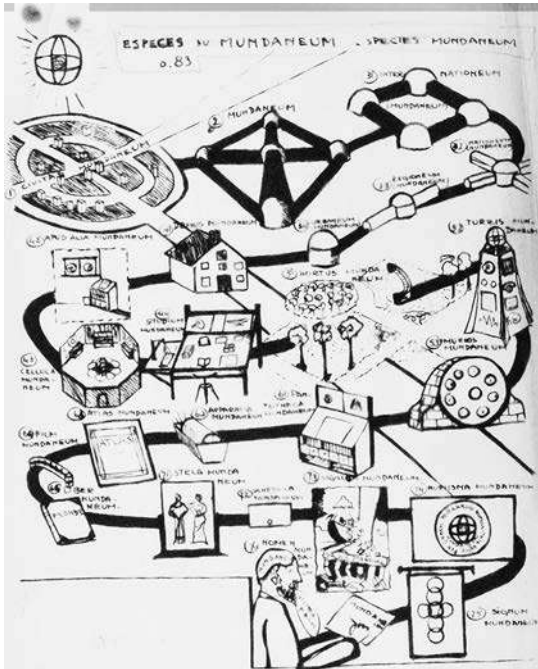


Рис. 1.3. Схема Всемирной сети Отле (1940 год)

Вэнивар Буш

Работая менеджером в проекте «Манхэттен», Вэнивар Буш (Vannevar Bush, <https://oreil.ly/q3xPC>) заметил, что, когда отдельные специалисты объединялись для работы над задачами в творческой обстановке, они зачастую обменивались идеями, перекладываясь с одной на другую и создавая новые связи между научными трудами. В июле 1945 года он опубликовал свои наблюдения в статье *As We May Think* (<https://oreil.ly/V5U0I>), где описал похожую на предложенную Отле рабочую станцию, в которой использовались микрофиша и указывающее устройство, крепившееся на голову читателя.

Дуглас Энгельбарт

Прочитав статью Буша, молодой офицер, служивший тогда в Восточной Азии, задумался, как воплотить описанное им устройство в жизнь. На это ушло почти 20 лет, но в 1968 году тот самый офицер, Дуглас Энгельбарт (Douglas Engelbart, <https://oreil.ly/D5eaV>), провел презентацию, которая сегодня известна как «Мать всех демонстраций» (*The Mother of All Demos*, <https://oreil.ly/a38AX>), представив ряд идей, над которыми работал со своей командой. В этой презентации был показан первый на тот момент «интерактивный компьютер», который позволял оператору

с помощью указывающего устройства выделять текст и щелкать по ссылкам для перехода. Для этой демонстрации Энгельбарту потребовалось изобрести мышь.

МАТЬ ВСЕХ ДЕМОНСТРАЦИЙ

Презентация Энгельбарта, проведенная им более 50 лет назад, в декабре 1968 года, на конференции по мейнфреймам в Сан-Франциско, заложила стандарт для демонстраций из Кремниевой долины, которые мы видим сегодня. Энгельбарт выступал в течение 90 минут, сидя в специально разработанном кресле Имса (<https://oreil.ly/yvkYl>, прототип современных кресел Аэрона), работая с настроенной под себя клавиатурой, мышью и кнопочным пультом. Все это время он спокойным голосом описывал свои действия через гарнитуру. Энгельбарт показал первый интерактивный компьютерный экран, продемонстрировал такие возможности, как «вырезать — копировать — вставить», гиперссылки и мультикурсорное редактирование, связь с коллегами, находящимися за сотнями миль, по видеосвязи в формате «картинка в картинке», контроль версий и несколько других концепций, которые вошли в обиход только спустя более десяти лет. Если вы не смотрели запись его выступления (<https://oreil.ly/EycWe>), настоятельно рекомендую.

Тед Нельсон

Современник Энгельбарта, Тед Нельсон, писал о потенциале персональных вычислительных устройств еще в 1965 году (<https://oreil.ly/YlocI>), используя им же придуманные термины «гиперссылки», «гипертекст», «гиперданные» и «гипермедиа». К 1974 году его книга *Computer Lib/Dream Machines* (Tempus Books) сформировала представление о мире, заполненном личными электронными устройствами, связанными друг с другом через Интернет. В это же время Алан Кей (см. раздел «Позднее связывание по Алану Кею» ранее в этой главе) описал устройство *Dynabook* (<https://oreil.ly/wiNEO>), очень похожее на современные компактные ноутбуки и планшеты.

Все эти первые изыскания способов связи и обмена информацией несли в себе одну основную идею, заключавшуюся в том, что глобальное соединение расширит возможности людей и повысит уровень креативности и инноваций. К концу 1980-х Тим Бернерс-Ли собрал успешную систему, которая включала в себя все идеи его предшественников. Реализованная им Всемирная паутина сделала связывание страниц документов безопасным, простым и масштабируемым.

В этом и заключается суть использования API сервисов — определение связей между разными элементами сети для нахождения новых решений.

Джеймс Дж. Гибсон

Примерно в то же время, когда Тед Нельсон знакомил мир с термином «гипертекст», еще один человек, психолог Джеймс Дж. Гибсон (James J. Gibson, <https://oreil.ly/iQuJR>), придумал другое, ставшее актуальным для этой сферы понятие.

В книге *The Senses Considered as Perceptual Systems* (<https://oreil.ly/C5msH>, Houghton-Mifflin), вышедшей в 1966 году, где он писал о том, как люди и другие животные воспринимают окружающий мир и взаимодействуют с ним, Гибсон ввел термин «аффорданс». Вот его определение (<https://oreil.ly/DFTSe>):

«Аффордансы окружающей среды — это то, что она предлагает животному, что она для него обеспечивает и дает, вне зависимости от того, хорошо это или плохо».

Аффордансы поддерживают взаимодействие между животными и средой в том же смысле, в каком гиперссылки Нельсона позволяют людям взаимодействовать с документами в сети. Современник Гибсона Дональд Норман (Donald Norman, <https://oreil.ly/dFmp3>) популяризовал термин «аффорданс» в своей книге *The Design of Everyday Things* (<https://oreil.ly/KqmiE>), вышедшей в 1988 году (Doubleday). Норман, считающийся дедушкой концепции человеко-компьютерного взаимодействия (Human-Computer Interaction, HCI, <https://oreil.ly/jEdZx>), использовал этот термин для определения способов, с помощью которых проектировщики ПО могут понимать такое взаимодействие и способствовать его достижению. Большая часть того, что мы знаем о юзабилити программного обеспечения, зиждется на работах Нормана и других специалистов в этой области.

Гипермедиа опирается на аффордансы. Элементы гипермедиа (ссылки и формы) — это те составляющие веб-среды, которые делают *возможными*¹ дополнительные действия, такие как поиск существующих документов, отправка данных на сервер для хранения и пр. Гибсон и Норман представили психологический и социальный аспекты взаимодействия с компьютером, на которые мы будем опираться в приводимых далее рецептах. По этой причине вы заметите, что многие рецепты включают использование ссылок и форм, которые позволяют подстраивать состояние приложения для разных сервисов.

Ценность сообщений

Как мы видели ранее, Алан Кей представлял объектно-ориентированное программирование как концепцию, основанную на *передаче сообщений* (см. раздел «Позднее связывание по Алану Кею» ранее в этой главе). Тим Бернерс-Ли поддержал эту точку зрения, когда в 1992 году сформировал ориентированный на обмен сообщениями протокол гипертекстовой передачи (Hypertext Transfer Protocol, HTTP, <https://oreil.ly/JEVu6>), а годом позже помог определить формат сообщений в виде языка гипертекстовой разметки (Hypertext Markup Language, HTML, <https://oreil.ly/gQGS4>).

После создания протокола и формата для передачи обобщенных сообщений (вместо передачи локализованных *объектов* или *функций*) будущее Интернета было определено. Этот ориентированный на сообщения подход проще ограничивать, изменять со временем, и он предлагает более надежную платформу для

¹ От англ. afford, то есть выступают аффордансами. — *Примеч. пер.*

будущих доработок, таких как появление совершенно новых форматов (XML, JSON и пр.), и изменения использования самого протокола (документы, сайты, веб-приложения и т. д.).

НЕКОТОРЫЕ НЕ ОСОБО УДАЧНЫЕ ПРИМЕРЫ

Применяемый в HTTP подход с инкапсулированными сообщениями сделал возможными и не особо удачные инновации вроде Java Applets, Flash и XHTML. И хотя протокол HTTP создавался с учетом поддержки технологий наподобие названных провальных альтернатив ориентированному на сообщения HTML, эти форматы прожили недолго и их удаление из экосистемы не нанесло в долгосрочной перспективе вреда самому протоколу. Это лишний раз доказывает отказоустойчивость и гибкость подхода HTTP к взаимодействию на уровне приложений.

Ориентированные на сообщения онлайн-решения имеют параллели и с физическим миром. Колонии насекомых, например термитов и муравьев, известные отсутствием иерархии или руководящих особей, используют для взаимодействия систему сообщений на основе феромонов. Примерно в то же время, когда Нельсон говорил о гипермедиа, а Гибсон — об аффордансах, американский биолог и натуралист Э. О. Уилсон (E. O. Wilson, <https://oreil.ly/0E5AF>) и американский математик Уильям Боссерт (William Bossert) писали работу (<https://oreil.ly/xVHFfn>), посвященную изучению колоний муравьев и использованию ими феромонов как инструмента управления крупными сложными сообществами.

С учетом всего этого вы наверняка не будете удивлены тем, что в основе всех рецептов, приведенных в книге, лежит обмен сообщениями с целью передачи информации между машинами.

Важность словарей

Подход на основе сообщений — это отличное решение в качестве платформы. Но даже универсальные форматы вроде HTML должны передавать информацию понятным способом. В 1998 году, примерно в то же время, когда Рой Филдинг разрабатывал свой подход REST для применения в сети (см. раздел «REST по Филдингу» ранее в этой главе), Питер Морвилл (Peter Morville, <https://oreil.ly/G2Ekz>) и Луи Розенфельд (Louis Rosenfeld, <https://oreil.ly/G2LMX>) выпустили книгу *Information Architecture for the World Wide Web* (O'Reilly). Считается, что именно она послужила началом движения по развитию *информационной архитектуры*. Профессор Дэн Клин (Dan Klyn, <https://oreil.ly/UP6si>) из Мичиганского университета объясняет это понятие, используя три ключевых элемента: *онтология* (конкретное значение), *таксономия* (упорядоченность частей) и *хореография* (правила взаимодействия частей).

Все эти три аспекта относятся к словарю сетевых приложений. Примечательно, что Тим Бернерс-Ли незадолго до успеха Всемирной паутины обратил внимание на проблему со словарями для веб-среды и организовал проект под названием «Схема описания ресурсов» (Resource Description Framework, RDF, <https://www.w3.org/RDF>). RDF и сопутствующие технологии вроде JSON-LD — это пример подхода, согласно которому акцент делается на смысле сообщений, и мы в своих рецептах будем поступать так же.

Соответствуя нашим задачам, хореография Клина основывается на ссылках и формах. Данные, передаваемые между машинами через эти элементы гипермедиа, представляют онтологию. Таксономия же — это связи между сервисами сети, которые совместно формируют интересующие нас распределенные приложения.

Магические строки Ричардсона

Еще одним важным аспектом здесь выступает использование онтологии и ее возможностей при создании в веб-среде сервисов и взаимодействии с ними. И хотя вполне разумно, что каждому приложению нужны собственные согласованные термины, например `givenName`, `familyName`, `voicePhone` и т. д., также важно помнить, что они являются магическими строками, как их назвал Леонард Ричардсон (Leonard Richardson, <https://www.crummy.com/self>) в своей книге *RESTful Web APIs* (<https://oreil.ly/eVpUA>), выпущенной в 2015 году.

ВОСПОЛНЕНИЕ СЕМАНТИЧЕСКОГО ПРОБЕЛА С ПОМОЩЬЮ МАГИЧЕСКИХ СТРОК

Ричардсон объясняет важность применения общих терминов в различных приложениях для восполнения семантического пробела в значении разных компонентов. Он также отмечает, что и при создании межмашинных сервисов без людей не обходится, даже если их роль сводится только к программированию. В книге *RESTful Web APIs* он пишет: «Названия для людей значат очень многое. И хотя потребителями ваших API будут компьютеры, они будут действовать от лица людей, которым важно понимать, что означают эти *магические строки*. Таким образом мы закроем семантический пробел».

Важность идентификаторов, используемых для именования свойств, давно известна. Все движение RDF (см. раздел «Важность словарей» ранее в этой главе) было основано на создании единой для всей сети трактовки грамотно определенных терминов. На уровне спецификаций в книге Эрика Эванса (Eric Evans, <https://oreil.ly/UWTe0>) *Domain-Driven Design* (Addison-Wesley, <https://oreil.ly/iY6Ur>) 2014 года немалое внимание уделяется объяснению принципов «единый язык» (применяемый всеми членами команды для объединения всех процессов в приложении) и «ограниченный контекст» (способ разделения моделей крупных приложений на согласованные части, в которых все термины будут понятны).

Эванс писал свою книгу примерно в то же время, когда Филдинг заканчивал диссертацию. Они оба были сосредоточены на том, как в больших приложениях добиться стабильного единообразного понимания и сохранить его. В то время как Эванс работал над согласованностью *внутри* одной базы кода, Филдинг старался реализовать те же задачи *среди* независимых баз кода.

Именно этот общий контекст отдельно создаваемых и обслуживаемых сервисов является ключевым в рецептах данной книги. При проектировании и реализации сервисов в веб-среде мы стремимся заполнить обозначенный Ричардсоном семантический пробел.

В этом разделе мы просмотрели более чем столетнюю ретроспективу размышлений и инициатив (см. раздел «Эпоха гипермедиа» ранее в этой главе), посвященных использованию машин для лучшей передачи идей по сети связанных сервисов. Мы узнали, как сфера социальной инженерии и психологии определила значимость аффордансов (см. раздел «Джеймс Дж. Гибсон» ранее) в качестве механизма предоставления выбора действий внутри гипермедиакоммуникаций (см. раздел «Ценность сообщений» ранее). Наконец, мы поняли, насколько важны и полезны грамотно определенные и обслуживаемые словари (см. раздел «Важность словарей» ранее) для обеспечения семантической ясности во всей сети.

Эти принципы составляют своеобразный набор руководств для определения полезных рецептов книги. Но прежде чем углубляться в детали каждого паттерна, нужно сделать еще одно отступление, которое даст нам полноценный набор руководящих принципов для понимания всего дальнейшего содержания.

Общие принципы масштабируемых сервисов в веб-среде

Завершая вступительную главу, я хочу познакомить вас с базовыми общими принципами, которые выступали ориентирами при выборе и определении рецептов для этой книги. Вся коллекция базируется на одном общем принципе:

«Используйте глобальную досягаемость, чтобы решать задачи, о которых даже не думали, для людей, с которыми никогда не встречались».

И этот принцип можно разбить на три последовательные части.

Используйте глобальную досягаемость...

В мире огромное число людей, миллионы из которых имеют доступ в Интернет. Когда мы работаем над созданием сервиса, определяем область задачи или реализуем решение, в Сети нам доступен огромный объем информации и креативных идей. Тем не менее слишком часто модели наших сервисов и инструменты реализации

ограничивают зону досягаемости. Бывает очень сложно отыскать то, что нужно, и даже когда мы *находим* готовое креативное решение своей задачи, его встраивание в проект может оказаться слишком дорогостоящим и сложным.

Я старался отобрать и описать рецепты для книги так, чтобы повысить вероятность нахождения вашего решения другими и минимизировать сложность его использования в других проектах. Это означает, что их структура и детали реализации акцентируются на определениях из контекстуальных словарей, примененных к стандартизированным сообщениям и протоколам, которые относительно легко реализовать.

Хорошие рецепты повышают нашу глобальную досягаемость, то есть возможность делиться своими решениями, а также находить и применять решения других.

...для решения задач, о которых вы даже не думали...

Вторая важная часть рассматриваемого принципа состоит в том, что мы стремимся создавать сервисы, которые можно будет использовать для решения задач, о которых даже не думали. Это не значит, что мы стараемся создать некий универсальный сервис, который смогут применять другие, например хранилище данных или механизмы контроля доступа. Да, они тоже нужны, но здесь я имею в виду другое.

Приведу слова Дональда Нормана из видео 1994 года (<https://oreil.ly/Xo4SR>):

«Грамотно спроектированный объект ценен тогда, когда у него есть настолько богатый набор аффордансов, что использующие этот объект люди могут делать с его помощью то, чего его создатель и не предполагал».

Я вижу рецепты из книги как инструменты в мастерской ремесленника. Какую бы работу вы ни делали, она наверняка идет лучше, когда у вас для нее есть подходящий инструмент. Для этой книги я постарался отобрать те рецепты, которые смогут не только расширить ваш инструментарий, но и повысить удовольствие от его использования.

Хорошие рецепты делают грамотно спроектированные сервисы доступными для применения другими таким образом, о котором мы не знали.

...для людей, с которыми никогда не встречались

Наконец, поскольку нашей целью являются сервисы, которые работают в Интернете — глобально доступной среде, нужно понимать, что их будут использовать люди, которых мы наверняка никогда не встретим. По этой причине важно тщательно и явно определять интерфейсы сервисов с помощью связанных и согласованных словарей. Нам нужно применять во всех сервисах единый язык, о котором писал Эрик Эванс. Люди должны легко понимать назначение нашего сервиса без лишних объяснений. Наши реализации должны быть, как говорил Филдинг,

«без состояний» — они должны нести в себе весь контекст, необходимый для понимания и успешного использования сервиса.

Хорошие рецепты позволяют незнакомцам (сервисам и людям) безопасно и успешно взаимодействовать друг с другом для решения задачи.

Временные масштабы

Еще один нюанс, который следует учитывать, — это то, что системы живут своей жизнью и работают в собственных временных масштабах. Интернет существует с начала 1970-х. И хотя его фундаментальные внутренние особенности не изменились, сама сеть развилась совершенно неожиданным образом. И это прекрасно иллюстрирует данное Норманом определение «грамотно спроектированного объекта».

Широкомасштабные системы не только развиваются медленно — даже те возможности, которые редко используются, сохраняются довольно долго. В языке HTML есть устаревшие функции, например `<marquee>`, `<center>`, `<xmp>` и т. д., но вы до сих пор можете найти случаи применения этих элементов языка в веб-среде. Как, оказывается, сложно избавиться от чего-то после того, как это попадет в Интернет. То, что мы делаем сегодня, может привести к каким-то последствиям спустя много лет.

ПРОЕКТИРОВАНИЕ В МАСШТАБАХ ДЕСЯТИЛЕТИЙ

В свои проекты и реализации мы можем закладывать отдаленные временные рамки. К примеру, Филдинг (<https://oreil.ly/CjffEO>) сказал: «REST — это проектирование ПО в масштабах десятилетий: каждая деталь должна способствовать долговечности программного обеспечения и его независимой эволюции».

Естественно, не для всех решений подход с долгосрочной реализацией окажется актуален. Вы можете спешить поскорее создать что-то, не планируя, что оно просуществует долго, — например, короткоживущий сервис для массового обновления каталога вашей продукции. И это вполне нормально. Но по своему опыту скажу: не следует всегда предполагать, что срок существования ваших проектов окажется коротким.

Хорошие рецепты способствуют долговечности и независимой эволюции в масштабах десятилетий.

Все это изменится

В завершение стоит сказать: что бы мы ни создавали, как бы тщательно ни планировали, все это изменится. Интернет в течение десятилетий своего существования развивался непредсказуемо. То же касается роли протокола HTTP и изначально-го формата сообщений HTML. В некоторых случаях программное обеспечение,

которое, по нашим ожиданиям, должно было существовать вечно, уже исчезло, а приложения, которые рассматривались как нечто временное, продолжают жить по сей день.

Что бы мы ни создавали — если мы делаем это на совесть, то наверняка оно будет использоваться неожиданными способами и неизвестными людьми для решения еще неизвестных задач. Тем, кто работает над созданием ПО для сетей, предстоит постоянно удивляться (как приятно, так и неприятно) судьбе своих творений.

Я работал над проектами, которым требовалось более десяти лет, чтобы стать заметными и полезными. И придумывал краткосрочные исправления, которые работают уже более двух десятков лет. Это одна из радостей моей работы. Я постоянно удивляюсь, всегда восхищен и редко разочарован. Даже когда все идет не так, как планировалось, я спокоен, так как знаю, что все это изменится.

Хорошие рецепты учитывают, что нет ничего постоянного и все вокруг нас постоянно меняется.

Теперь, когда мы разобрались с основами, пора глубже познакомиться с технологией и моделью мышления, на основе которых отбирались рецепты для этой книги. Начнем с изучения искусства мышления в контексте гипермедиа.

ГЛАВА 2

Мышление и проектирование в контексте гипермедиа

Не существует отдельных систем. Мир непрерывен, и выделение границ некой системы будет зависеть от контекста и целей дискуссии.

Донелла Х. Мидоуз (Donella H. Meadows)

Прежде чем перейти к рецептам из части II, мы разберем основы веб-среды и принципы ее работы. Преследуя эту задачу, текущая глава подчеркивает ряд важных концепций и технических этапов сферы сетевых вычислений. Эти достижения позволили выработать способ, которым мы взаимодействуем в веб-среде сегодня, и отчасти — способ размышления о вычислениях в целом.

Когда перед нами стоит задача программирования сети, зачастую это подразумевает программирование *машин*. И основными инструментами здесь выступают языки программирования, которые используют память, хранилище данных и их передачу, реализуя соответствующие процессы с помощью функций. Тем не менее, когда дело касается программирования *сети*, возникают новые сложности, требующие новых инструментов и новой модели мышления.

В последующих разделах приводятся исторические материалы и комментарии по их применению в современных попытках выйти за пределы моделей локального программирования с сохранением состояния. В следующем разделе «Закладываем основы с помощью проектирования гипермедиа» описываются идеи, стоящие за рецептами *проектирования*, включая:

- организацию общего взаимодействия между машинами, о котором впервые заговорили в начале 1960-х;
- понятие информационной архитектуры из 1990-х;
- применение гипермедиа в качестве модели программирования независимых машин сети в среде выполнения.

Как видно из рис. 2.1, мышление в гипермедиаструктуре Нельсона подразумевает применение предложенной Роем Филдингом универсальности интерфейсов для обеспечения позднего связывания по Алану Кею. При этом при создании отказоустойчивых решений нужно помнить и о важности масштабирования и независимой развертываемости.

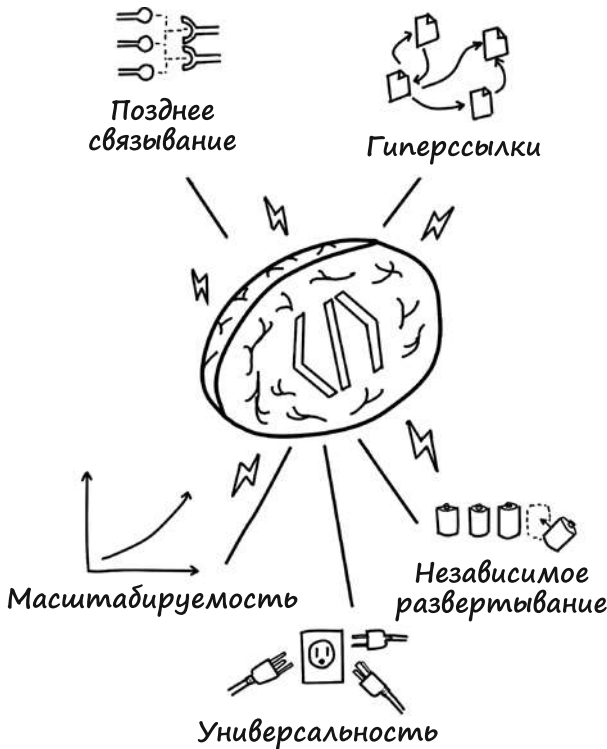


Рис. 2.1. Мышление и проектирование в гипермедиа подразумевают одновременный учет нескольких задач

В разделе «Повышение отказоустойчивости клиентов гипермедиа» далее в этой главе описывается история создания надежных *клиентских приложений*, способных функционировать в сети сервисов. Это означает акцент на важных возможностях потребителей API, повышающих устойчивость и адаптируемость. Таковы, например, упор на протоколы и форматы как на строгую типизацию в клиентских сетях, возможность распознавать указанные в сообщениях подробности взаимодействия и реагировать на них в среде выполнения (ссылки и формы), а также использование семантических словарей для обеспечения общего понимания терминов клиентами и сервисами. Эти три аспекта составляют набор практик, позволяющих создавать стабильных потребителей API, работа которых не прервется при

изменении элементов сервиса, таких как детали протокола, URL ресурсов, схема сообщений и рабочий поток операций.

Основной сложностью проектирования успешных *API сервисов* является необходимость балансировать между стабильностью и возможностью развиваться, то есть способность выполнить данные потребителям API обещания и при этом поддерживать развитие возможностей сервисов, лежащих в основе вашего интерфейса. Принципы, освещенные в разделе «Повышение стабильности и изменяемости сервисов на основе гипермедиа» далее в этой главе, являются ключами для преодоления этой сложности. Сюда входят проблема изменяемости (реализация изменений с течением времени) и потребность в машинном подходе к самообслуживанию для поиска других сервисов в сети и их использования. В ходе чтения книги вы увидите, как можно применять гипермедиа для решения этих проблем.

В разделе «Поддержка распределенных данных» далее в этой главе вводится понятие данных как *свидетельства*: свидетельства некоего действия, а также его последствий. Многие попытки запрограммировать сеть ошибочно начинаются с мыслью о том, что в центре структуры находятся данные. В упомянутом разделе вы увидите, что данные чаще всего стоит рассматривать как *внешнюю* часть структуры — они важны, но находятся не в центре (здесь я буду выступать своеобразным Галилео). Мы также поговорим о роли языков запросов для извлечения информации (information retrieval query languages, IRQLs), сравнив их с языками запросов к базам данных (database query languages, DQLs) и разобрав, почему при программировании сети так важно делать упор на IRQLs.

Наконец, в конце главы, в разделе «Увеличение расширяемости с помощью рабочего потока на основе гипермедиа», мы познакомимся с проблемами проектирования и реализации мультисервисных *рабочих потоков*. В мире программирования, опирающемся на машины, эти процессы моделируются как *оркестрация*, когда за интеграцию сервисов отвечает один супервизор, или как *хореография*, когда сервисы работают напрямую — без посредника. И хотя оба этих подхода являются разумными решениями, включение независимых сервисов в сеть лучше всего реализовывать с помощью общего протокола и словаря. Этот подход опирается на использование ссылок и форм (гипермедиа) для регулирования скоординированного набора независимых действий, выполняющихся в разных местах, с целью решения определенной задачи. Именно в виде такого рабочего потока осуществляется программирование сети.

Кроме того, мы разберем проблемы реализации рабочих потоков веб-среды (они подробно описаны в главе 7), а именно обмен состоянием между сервисами, ограничение области определения одного рабочего потока (то есть задачи), поддержка наблюдаемости рабочего потока и обработка его ошибок в среде выполнения. Материала очень много, так что приступим.

Закладываем основы с помощью проектирования гипермедиа

Первый набор рецептов книги (глава 3) акцентирован на сложностях проектирования. В основе этих рецептов лежат три главные концепции:

- утвержденный формат взаимодействия для обработки соединений между машинами сети;
- модель для интерпретации данных в виде информации;
- техника, позволяющая сообщать машинам в среде выполнения, какие действия допустимы.

Все рецепты в книге посвящены созданию полезных соединений между сервисами приложений, выполняющихся на сетевых машинах. Сегодня самый распространенный способ реализации этого состоит в использовании TCP/IP на уровне пакетов и HTTP на уровне сообщений.

Есть интересная история о том, как Министерство обороны США спроектировало и проспонсировало первые сети — в частности, сеть Агентства по перспективным исследованиям (Advanced Research Projects Agency Network, ARPANET), которая в итоге стала современным Интернетом. И связана эта история с пришельцами. В 1960-е годы, когда США разрабатывали средства компьютерного взаимодействия, предположения о возможной встрече с пришельцами из далекого космоса способствовали принятию ряда проектных решений.

Помимо утверждения протоколов для коммуникации, в ходе проектирования нужно проработать вопросы организации данных и обмена ими между машинами. Для этого мы немного углубимся в тему информационной архитектуры (information architecture, IA), чтобы проникнуться важностью онтологии, таксономии и хореографии. История IA началась примерно в то же время, когда Рой Филдинг разрабатывал архитектурный стиль REST, и на нее сильно повлияло рождение Всемирной паутины, опирающейся на HTTP и HTML. Кроме того, в главе 3 IA используется как организующий фактор, определяющий способ описания возможностей сервисов с помощью общего словаря.

В завершение же мы напрямую перейдем к сути того, как машины, созданные разными людьми, которые никогда не встречались, могут успешно взаимодействовать в реальном времени через открытую сеть, задействуя гипермедиа в качестве двигателя состояния приложения. Предпосылками для этого взаимодействия являются надежные соединения через HTTP и согласованное моделирование с помощью словарей, а возможным его делает техника гипермедиа. Рецепты, приводимые в главе 3, будут определять способы формирования гипермедиа-взаимодействий,

а в последующих главах будет подробно описано, как заставить эти схемы согласованно функционировать.

Итак, посмотрим, как вероятность появления пришельцев, информационная архитектура и гипермедиа сходятся воедино для создания структуры RESTful Web API.

Пришельцы Ликлайдера

В 1963 году Джозеф Карл Робнетт Ликлайдер, он же J. C. R. или «Лик», малоизвестный гражданский сотрудник Министерства обороны США, составил внутриведомственную памятку для своих коллег, работавших в Агентстве перспективных исследований (ARPA). Буквально через несколько лет эта группа создаст ARPANET (<https://oreil.ly/HKWEq>) — предшественника современного Интернета. Однако на той ранней стадии Ликлайдер обращался к аудитории как к «членам и участникам межгалактической сети» (<https://oreil.ly/cv7xO>). Его памятка была посвящена тому, как можно соединить вычислительные машины для успешного взаимодействия друг с другом. Одним из вариантов было сделать так, чтобы все компьютеры на планете использовали одинаковые языки и инструменты программирования, что упростило бы соединение машин, но усложнило их специализацию. Второй вариант подразумевал создание отдельного общего управляющего языка сетевого уровня, который позволял бы машинам задействовать собственные локальные инструменты и языки, а также другого общего языка для взаимодействия по сети. Второй вариант должен был позволить проектировщикам сосредоточиться на оптимизации локальной функциональности, но усложнил бы работу по программированию машин для подключения друг к другу.

В итоге (к счастью для всех нас!) Ликлайдер и его команда выбрали второй подход, предпочтя использовать локальные машинные языки и отдельный общий язык сетевого уровня. Сегодня это может показаться очевидным, но тогда все было не так. Причем этот выбор не просто отражал решение Ликлайдера, а являлся результатом его размышлений с учетом того, что мы можем встретить космических пришельцев. Дело в том, что, пока ARPA трудилось над зарождением эпохи компьютерных вычислений, другое агентство США, NASA, соперничало с Советским Союзом в сфере освоения космоса.

Вот выдержка из памятки Ликлайдера, в которой объединяются космическая гонка 1960-х и компьютерная революция:

«По сути, проблема заключается в том, о чем пишут писатели-фантасты: как наладить коммуникацию между совершенно разными разумными существами?»

Ликлайдер рассуждал о том, как с помощью наших спутников (или наземных передатчиков) подойти к решению проблемы общения с разумными существами из космоса. Он предполагал, что это удастся реализовать путем поэтапного согласования способа общения, то есть с помощью многократного обмена контрольными сообщениями, или метасообщениями (сообщениями о том, как мы отправляем сообщения), пока обе стороны не поймут правила. Спустя десять лет изобретенные

в 1970-х протоколы TCP (<https://oreil.ly/HhKRx>) и IP (<https://oreil.ly/FcB4x>) отразили идеи Ликлайдера и сформировали хребет того Интернета, которым мы наслаждаемся сегодня.

ПРОТОКОЛ ЛИКЛАЙДЕРА

Через 40 лет после того, как Ликлайдер рассуждал о межмашинной коммуникации в космосе, члены Инженерного совета Интернета (Internet Engineering Task Force, IETF) завершили разработку протокола передачи данных для межпланетарных коммуникаций. Он получил название Licklider Transmission Protocol (LTP) и описан в документах IETF под номерами RFC 5325 (<https://oreil.ly/Hk1JD>), RFC 5326 (<https://oreil.ly/KAD3F>) и RFC 5327 (<https://oreil.ly/VZG4L>).

Сегодня мысленный эксперимент Ликлайдера о способе взаимодействия с инопланетянами является сердцем реализации RESTful Web API (RWA). А поскольку мы работаем над проектированием и созданием сервисов, которые взаимодействуют через Интернет, нам тоже нужно применять подход с метасообщениями. И это особенно важно, если учесть, что одной из целей нашей работы является налаживание коммуникации между совершенно разными сервисами. В соответствии с нашим основным принципом (см. раздел «Общие принципы масштабируемых сервисов в веб-среде» главы 1) люди должны иметь возможность уверенно проектировать и создавать сервисы, которые смогут общаться с другими сервисами, созданными другими людьми, будь то вчера, сегодня или в будущем.

Информационная архитектура Морвилла

Период 1990-х стал горячим временем для инициаторов развития Интернета. Придуманная Тимом Бернерсом-Ли Всемирная паутина и ее составляющие HTTP/HTML (см. раздел «Веб по Тиму Бернерсу-Ли» главы 1) вовсю работали, Рой Филдинг был занят определением своей архитектуры REST (см. раздел «REST по Филдингу» главы 1), а Ричард Сол Вурмен (Richard Saul Wurman, <https://oreil.ly/A6Teg>) придумал новый термин «информационный архитектор». В своей книге 1997 года *Information Architects* (<https://oreil.ly/ThMEx,Graphis>) Вурмен предлагает такое его определение:

«Информационный архитектор: 1) человек, который упорядочивает присущие данным закономерности, делая сложное понятным; 2) человек, который создает структуру или карту информации, позволяя другим находить собственные пути получения знаний; 3) новое профессиональное направление XXI века, отвечающее на потребности эпохи в ясности, человеческом понимании и научной организации информации».

Будучи обычным архитектором по образованию, Вурмен в 1984 году учредил проведение конференций по «технологии, развлечениям и дизайну» (Technology,

Entertainment and Design, TED). Являясь также активным писателем, он выпустил около сотни книг на разные темы, включая искусство, путешествия и, что важно для нас, информационный дизайн. Среди людей, которые начали развивать предложенное Вурменом понятие информационной архитектуры, был ученый-библиотековед Питер Морвилл (Peter Morville, <https://oreil.ly/vtchM>). Он считается одним из отцов-основателей движения по развитию информационной архитектуры и написал об этом несколько книг. Самой известной стала *Information Architecture for the World Wide Web* (O'Reilly), которая вышла в 1998 году, а на сегодня уже четырежды переиздавалась.

Эта книга посвящена взаимодействию людей с информацией, а также проектированию и созданию масштабных информационных систем, где обеспечены непрерывное развитие, управление и простота применения. Питер Морвилл отмечает, что система с грамотной информационной архитектурой (IA) помогает пользователям понять, где они находятся, что нашли, что еще есть рядом и чего ожидать. Все эти свойства важны и для систем RWA. Мы будем использовать рецепты для достижения тех же целей в контексте межмашинного взаимодействия.

Одним из способов организации IA в реализациях RWA будет подход посредством моделирования, состоящий из трех частей: онтологии, таксономии и хореографии (см. раздел «Важность словарей» главы 1). При этом несколько рецептов — а именно 3.3, 3.4 и 3.5 — будут посвящены информационной архитектуре.

ОБЪЯСНЕНИЕ ИНФОРМАЦИОННОЙ АРХИТЕКТУРЫ

Дэн Клинтон, основатель The Understanding Group (TUG, <https://oreil.ly/Eu5VC>), записал очень хорошее короткое видео *Explaining Information Architecture* (<https://vimeo.com/8866160>), в котором показал, как онтология, таксономия и хореография связываются вместе, формируя модель информационной архитектуры.

Гипермедиа и априорное проектирование

Одной из причин, по которой я начал составлять свою коллекцию рецептов с темы «проектирование», является то, что в процессе проектирования информационной системы с самого начала устанавливается ряд правил. Аналогично тому, как руководящие принципы (см. раздел «Общие принципы масштабируемых сервисов в веб-среде» главы 1) закладывают основу для принятия решений в информационных системах, рецепты проектирования позволяют воплотить эту основу в жизнь. Именно первый набор рецептов из главы 3 во многом влияет на все остальные рецепты книги.

Таким образом, первоочередное определение этих трех рецептов подобно *априорному проектированию*. Одно из определений слова «априорный» по словарю Merriam-Webster гласит: «Сформированный или предположенный заранее», и именно это мы будем здесь делать. Мы закладываем основные элементы наших

систем заранее. И в использовании этого подхода есть преимущество. Он позволяет определять стабильные элементы системы, поверх которых мы сможем создавать сервисы и реализовывать их взаимодействие.

Создание подхода к проектированию означает, что нам нужна модель, пригодная для более чем одного решения. Например, подход, который работает только для систем управления контентом (content management systems, CMS), но не подходит для систем управления связями с клиентами (customer relationship management systems, CRM), окажется не особо эффективен в плане проектирования. Мы интуитивно понимаем, что два этих очень разных решения имеют много общего на уровне проектирования и технической реализации, но зачастую требуется немалый труд, чтобы вписать эти сходства в согласованную связку — набор принципов проектирования.

И это может стать особенно проблематичным, когда мы хотим создавать решения, которые со временем будут меняться. Решения, которые остаются стабильными при добавлении новых функций, реализации новых технологий и создании дополнительных ресурсов, таких как серверы и клиентские приложения, для взаимодействия с системой. Нам нужен фундаментальный элемент, который обеспечит стабильность даже в случае внесения изменений.

В этом наборе методов проектирования таким фундаментальным элементом является использование гипермедиа, или ссылок и форм (см. раздел «Почему гипермедиа?» главы 1) как средства для организации взаимодействия сервисов. Филдинг называл гипермедиа двигателем состояния приложения (<https://oreil.ly/QnvxO>). Этот двигатель обеспечивает тот самый механизм метасообщений, о котором говорил Ликлайдер (см. раздел «Пришельцы Ликлайдера» ранее в этой главе). Именно применение гипермедиа делает возможным описанное Кеєм позднее связывание (см. раздел «Позднее связывание по Алану Кею» главы 1).

Повышение отказоустойчивости клиентов гипермедиа

Поскольку компьютеры, как говорит Тед Нельсон, «делают то, что вы скажете им делать», мы должны ответственно относиться к тому, что им говорим. В главе 4 мы сосредоточимся на том, что говорим потребителям API (клиентским приложениям). Есть общая тенденция быть чрезвычайно конкретным, отдавая команды компьютеру. Особенно актуально это при создании сервисов на основе API (см. главу 5). Чем точнее наши инструкции, тем вероятнее, что сервис будет действовать ожидаемым образом. Но клиентские приложения работают иначе, и на этом как раз акцентируются клиентские рецепты.

Несмотря на то что сервисы на основе API должны быть стабильными и предсказуемыми, клиентские приложения должны иметь отличную адаптируемость и устойчивость. Они существуют для того, чтобы выполнить задачу. У них есть

цель. Как мы позже будем говорить, важно быть конкретным в том, какова эта цель и насколько детальными мы хотим быть при создании потребителя API.

Очень подробный набор инструкций для клиента API сделает его эффективным при выполнении указанной задачи. Но также сделает его непригодным для почти любой другой работы. И если целевое устройство, для которого он создавался, существенно изменится, это клиентское приложение «сломается». Получается, чем детальнее решение, тем оно менее универсально. И наоборот, если вы хотите иметь возможность повторно использовать потребителей API, то нужно изменить подход к их реализации.



Балансирование простоты применения API (юзабилити) и простоты использования того же API для другой задачи (реюзабилити) — нелегкий процесс. Реюзабилити повышается с помощью абстракции. Протокол HTTP довольно абстрактен (URL, набор методов, коллекция пар «имя — значение» и возможное тело сообщения в одном из многих возможных форматов), и это делает его очень универсальным. Но сам он не особо дееспособен без множества сопутствующих технологий, стандартов и доступных инструментов, например веб-серверов и браузеров.

Рецепты, включенные в главу 4, нацелены на повышение устойчивости клиентских приложений. Это означает акцент на ряде важных возможностей потребителей API, которые повышают отказоустойчивость и адаптируемость. Здесь имеются в виду:

- акцент на протоколах и форматах;
- разрешение деталей взаимодействия в процессе выполнения;
- иной дизайн для межмашинных (machine-to-machine, M2M) взаимодействий;
- использование семантического словаря, общего для клиента и сервера.

Эти четыре пункта составляют набор практик, позволяющих создавать стабильных потребителей API, которые не «сломаются», когда с течением времени будут меняться такие элементы, как детали протокола, URL ресурсов, схема сообщений и рабочий поток операций. Все клиентские рецепты выстроены вокруг этих критериев с целью обеспечения стабильности и устойчивости ваших клиентских приложений.

Разберем каждый из них по очереди.

Привязка к протоколам и форматам

Важным элементом для создания успешных клиентских приложений на основе гипермедиа является *привязывание* клиента к ответам. Несмотря на то что программисты могут об этом не думать, при написании клиента API мы создаем привязку производителей (сервисов) к потребителям (клиентам). Что бы мы ни задействовали в качестве агента привязки, оно будет общим для клиентов и серверов. Самые

эффективные привязки — те, которые редко меняются или не меняются вовсе. Под привязкой здесь мы понимаем фактическое выражение общего понимания для клиентов и сервисов.

Типичными целями привязки выступают URL (например, `/person/123`) или объекты (например, `{id:"123", {person:{...}}}`). Как вариант, может существовать множество фреймворков и генераторов, которые используют этих двух агентов привязки (URL и объекты), чтобы автоматически генерировать статический код для клиентского приложения, которое будет работать с целевым сервисом. И это оказывается отличным способом быстрого развертывания работающих клиентских приложений. Но при этом мы также получаем приложение, которое сложно повторно применять и легко сломать. Например, любые изменения в объектах хранилища API нарушат работу приложения-потребителя. Кроме того, даже если есть идентичный сервис (обладающий таким же интерфейсом), выполняющийся по другому URL, сгенерированный клиент вряд ли сможет успешно с ним взаимодействовать, как раз из-за отличия URL. Использование URL и схем объектов в качестве агентов привязки для долгосрочного применения/повторного применения не будет хорошим решением.

Намного более эффективными целями привязки для веб-API будут протокол (например, HTTP, MQTT и пр.) и формат сообщений (например, HTTP, Collection + JSON и др.). Они намного стабильнее в сравнении с URL и объектами. Фактически они являются более высокой абстракцией каждого. То есть протокол — это более высокая абстракция, чем URL, а форматы сообщений (или медиатипы в веб-среде) — более высокая абстракция схемы объектов. Благодаря своей универсальности и малой вероятности изменения протокол и формат выступают хорошими агентами привязки. Подробнее об этом — в рецептах 4.3 и 4.6.

Например, если клиентское приложение привязано к формату (скажем, Collection + JSON), тогда его можно успешно использовать вместе с любым сервисом (по любому URL), поддерживающим привязки Collection + JSON. Именно это и делают HTML-браузеры уже более 30 лет.

Но протокол и формат — это лишь база прочного основания общего понимания. Другими ключевыми элементами для получения стабильных и надежных потребителей API являются поддержка метаданных в среде выполнения, семантические профили и клиент-ориентированные рабочие потоки.

Обработка в среде выполнения с помощью метаданных

Одной из сложностей при создании гибких клиентов API является обработка всех деталей каждого HTTP-запроса: какой HTTP-метод использовать, какие параметры передавать в URL, какие данные отправлять в телах запросов и как поступать с дополнительными метаданными вроде HTTP-заголовков. Это довольно много информации для отслеживания, и процесс становится особенно хлопотным, когда вам нужно обрабатывать метаданные для каждого HTTP-запроса.

Типичным методом обработки метаданных запроса является их встраивание в интерфейс сервиса. В документации обычно приводятся инструкции о том, как реализовывать отдельное действие для API, например добавлять новую запись.

- Используйте POST с URL `/persons/`.
- Передайте в теле запроса не менее четырех параметров (`givenName`, `familyName`, `telephone` и `email`).
- Используйте для тела запроса формат сериализации `application/x-www-form-urlencoded`.
- В случае успешного завершения ожидайте получения HTTP-кода ответа 201 и наличия в заголовке Location URL новой записи.

Приведенный здесь пример — фактически краткое представление намного более подробной записи, которую я встречаю в большинстве документации. Радует, что большинство программистов уже усвоили эту информацию и сильного возмущения краткий вариант у них не вызовет. А вот не радует то, что написание всего этого в коде ведет нас в ловушку описанного ранее неправильного агента привязки. Любые изменения URL или объекта/параметров приведут к нарушению работы приложения и необходимости его обновления. Причем изменения будут происходить раздражающе часто, особенно на первых этапах жизни API/сервиса.

Избежать этого можно, запрограммировав клиентское приложение на распознавание и учет деталей запроса в метаданных, присылаемых клиенту во время выполнения. Мы будем говорить об этом в рецепте 4.9. Опять же, HTML десятилетиями проделывает подобную работу. Далее приведен пример той же информации в метаданных среды выполнения:

```
<form action="/persons/" method="post",
  enc-type="application/x-www-form-urlencoded">
  <input type="text" name="givenName" value="" required />
  <input type="text" name="familyName" value="" required />
  <input type="tel" name="telephone" value="" required />
  <input type="email" name="email" value="" required />
  <input type="submit" />
</form>
```

Как вы уже догадались, HTML-браузер был запрограммирован на распознавание и учет отправляемых клиенту метаданных. Да, здесь участвует программирование — разовая работа по внедрению поддержки FORMS в сообщениях, но хорошо то, что написать этот код достаточно всего раз.



Здесь в качестве примера метаданных среды выполнения я использовал HTML, но есть много форматов на основе JSON, которые имеют обширную поддержку таких метаданных. Мне регулярно встречаются Collection + JSON (<https://oreil.ly/4Dg58>), SIREN (<https://oreil.ly/8eKMq>) и UBER (<https://oreil.ly/DVX76>).

Поддержка метаданных среды выполнения может значительно упростить написание приложения для человеко-машинного взаимодействия. Существуют библиотеки, которые поддерживают парсинг гипермедиаформатов в понятные для людей пользовательские интерфейсы, во многом аналогично браузерам, которые делают то же самое для HTML. Но вот реализовывать поддержку метаданных среды выполнения для М2М-взаимодействий уже сложнее. Причина в том, что здесь из уравнения исключен человеческий мозг.

Для поддержки стабильных и надежных взаимодействий М2М нам нужно восполнить недостачу человека в этом процессе. На выручку приходят семантические профили.

Сложности межмашинных коммуникаций

Наш мозг — удивительный орган. Настолько удивительный, что мы часто даже не замечаем, сколько он для нас проделывает. Прекрасный пример — акт заполнения простой HTML-формы, такой как та, о которой мы только что говорили. После сканирования страницы глазами, что уже само по себе магия, нашему мозгу требуется обработать немало процессов:

- распознать наличие `FORM`, которую можно заполнить и отправить;
- понять, что нужно предоставить четыре вида входной информации;
- распознать значение `givenName` и прочие значения;
- перебрать память или другой источник в поиске значений для заполнения нужных данных;
- понять, что для отправки данных на сервер нужно нажать кнопку **Отправить**.

Помимо этого, нужна возможность обрабатывать сообщения об ошибках в случае, когда входные данные не предоставлены или предоставлены некорректно, а также любой ответ от сервера, например «невозможно сохранить данные», или какие-то другие строки, которые могут потребовать от человека дополнительных действий.

Когда мы пишем человеко-машинные клиенты API, то можем просто предположить, что вся эта ментальная мощь достается приложению даром — нам ее не нужно программировать. Однако в случае приложений М2М эта бесплатная информационная обработка отсутствует. Вместо этого нам нужно либо встраивать функциональность человеческого мозга в приложение, либо придумывать способ заставить его работать без участия механизмов человеческого сознания.

Если вы хотите позаниматься программированием моделей машинного обучения или искусственного интеллекта, то это один из вариантов. Мы же здесь пойдем другим путем. Клиентские рецепты будут ориентированы на реализацию в приложениях «умеренной степени разумности». Этого можно добиться, используя

медиатипы для обработки таких моментов, как распознавание FORM и ее метаданных. Мы также задействуем семантические профили в качестве средства обработки параметров, которые могут присутствовать в FORM, и связывания этих параметров с локально (на клиентской стороне) доступными данными для заполнения их значений. Кроме того, поговорим о том, как изменять поддержку рабочего потока сервисов, чтобы упростить для M2M-клиентов безопасное взаимодействие с сервисами (подробнее в главе 5).

Использование семантических словарей

До сих пор наиболее успешными M2M-взаимодействиями в веб-среде были те, которые требовали только *чтения* данных без их записи. Хорошими примерами являются веб-сканеры, поисковые боты и аналогичные решения. Отчасти это связано с проблемой идемпотентности и безопасности (решение этих проблем ищите в рецепте 3.6).

Еще одна серьезная сложность M2M-взаимодействий связана со свойствами данных отдельных запросов. Люди имеют в своем распоряжении огромные объемы данных, которых у машин обычно нет. Добавление нового поля в FORM, как правило, не становится особой проблемой для человека, которому это поле нужно заполнить. Но для клиентского M2M-приложения такое изменение может стать критическим. Для решения этой проблемы потребуется предпринять усилия как на стороне клиента, так и на стороне сервера.

Эффективным способом добиться этого будет применение семантических профилей (см. рецепт 3.4). Семантические профили — это документы, подробно описывающие все возможные имена свойств и детали действий (ссылки и формы), используемые для набора задач (например, управления учетной записью, платежных сервисов и т. д.), чтобы определить границы словаря, который клиентское приложение должно понимать. Иными словами, клиент и сервер могут заранее договориться, какие свойства данных потребуются для успешного взаимодействия с приложением. Как это работает, вы увидите в рецепте 4.4.



Используя семантические профили для заблаговременного определения границ сервиса — и общая стабильно сохранять эти границы, — мы получаем еще один важный агент привязки, который особенно хорошо подходит для M2M-взаимодействий. Теперь мы можем применять протокол, формат и профиль как три стабильных, но при этом гибких элемента привязки для клиент-серверного диалога.

Для создания успешных RWA-клиентов требуется еще один важный элемент — возможность для клиентов создавать и использовать *собственные* мультисервисные рабочие потоки вместо того, чтобы быть привязанными к одному сервису статическим, заготовленным интерактивным скриптом.

Поддержка клиент-ориентированных рабочих потоков

Большинство клиентских приложений API статически привязаны к одному сервису. Эти клиенты обычно создаются в единственном экземпляре. Один из фундаментальных способов связывания этих приложений с сервисом выражается в виде *рабочего* потока. Точно так же, как есть приложения, которые в качестве агента привязки используют URL и схемы объектов, есть такие, которые задействуют в его качестве последовательный рабочий поток. Клиенту известен только один набор возможных потоков, и детали этого набора не меняются.

Статически привязанный рабочий поток зачастую заявлен в коде клиентского приложения напрямую. Например, сервис под названием `customerOnboarding` может предлагать следующие URL (со схемой объекта для сопоставления):

- `/onboarding/customer` со схемой `{ customer: {...} }`;
- `/onboarding/contact` со схемой `{ contact: {...} }`;
- `/onboarding/agreement` со схемой `{ agreement: {...} }`;
- `/onboarding/review` со схемой `{ review: {...} }`.

У сервиса есть четыре определенных, выполняемых последовательно шага. Данная последовательность обычно прописана не в коде сервиса, а в его *документации*. Это означает, что преобразование читаемого описания из документации в машинный формат, представленный в виде кода, — обязанность клиентского приложения. Обычно это выглядит так:

```
function onboardCustomer(customer, contact, agreement, review) {  
  http.send("/onboarding/customer", "POST", customer);  
  http.send("/onboarding/contact", "POST", contact);  
  http.send("/onboarding/agreement", "POST", agreement);  
  http.send("/onboarding/review", "POST", review);  
  return "200 OK";  
}
```

Первая сложность в этом примере заключается в том, что из-за статической привязки любое изменение в рабочем потоке сервиса ведет к нарушению работы приложения. Эффективнее будет указать клиенту, какие действия нужно проделать, и предоставить его приложению возможность выбирать и выполнять шаги по мере их возникновения. Для реализации этого можно использовать в ответах средства гипермедиа:

```
function onboardCustomer() {  
  results = http.read("/onboarding/work-in-progress", "GET");  
  while(results.actions) {  
    var action = results.actions.pop();  
    http.send(action.url, action.method, map(action.parameters, local.data));  
  }  
  return "200 OK";  
}
```

В этом упрощенном примере клиент запрашивает у сервиса список действий и затем выполняет их, используя метаданные среды выполнения (см. раздел «Обработка в среде выполнения с помощью метаданных» ранее в данной главе) в сообщении. Клиент продолжает делать это, пока список действий для выполнения не закончится. У этого паттерна есть и другие вариации, о которых мы тоже будем говорить (см. рецепт 4.9). Во втором примере, когда порядок действий или даже их количество меняется, нарушение работы приложения оказывается менее вероятным.

У этого клиент-ориентированного подхода есть еще один вариант, при котором рабочий поток определяет на сервис, а *клиент*. Обычно так происходит, когда клиенту для выполнения задачи нужно привлечь несколько отдельных сервисов. Возьмите предыдущий пример, но на этот раз предположите, что рабочий поток опирается на внешний облачный сервис управления контактами (<http://contacts.example.org/>) и независимый сервис проверки платежеспособности (<http://credit-check.example.gov/>). Эти два внешних сервиса нужно смешать с отдельными сервисами *customer*, *agreement* и *review* вашей компании. В этом мире сервисы друг о друге не знают. Они не контролируют рабочий поток — это делает клиентское приложение. В действительности реальное приложение существует только в виде реализации клиента.

Псевдокод для этого сценария похож на прежний, но включает клиентскую версию серии шагов, описанных в документации конфигурации, которая содержит набор стартовых URL и возможные аргументы для каждого шага:

```
function onboardCustomer(choreography) {  
  while(choreography) {  
    var step = choreography.pop();  
    var action = http.get(step.URL);  
    http.send(action.url, action.method, map(action.parameters, local.data));  
  }  
  return "200 OK";  
}
```

В данном примере *источник* хореографии берется из клиентского приложения, и оно же само решает, когда завершило свою работу. Это называется установкой целей с ориентацией на клиента (см. рецепт 4.16), которую несложно поддерживать, когда у вас уже есть основа в виде стабильных и гибких агентов привязки, опирающихся на протоколы, форматы и профили.



Есть и еще один подход к реализации основанного на клиенте рабочего потока. Он опирается на конкретное значение статуса или набор значений на одном или нескольких серверах. Пример такого решения приводится в рецепте 7.13.

Повышение стабильности и изменяемости сервисов на основе гипермедиа

Основной проблемой при проектировании успешных API сервисов является необходимость балансирования стабильности и способности развиваться, то есть выполнение данных потребителям API обещаний с сохранением возможности развития функциональности сервисов, лежащих в основе вашего интерфейса. Все рецепты главы, посвященной сервисам (глава 5), выбирались с целью облегчить эту задачу.

Корнем этой сложности является неизбежность последующих изменений. Чем дольше сервис эксплуатируется в промышленной среде, тем более вероятно его изменение. В то же время краткосрочные единичные сервисы редко живут достаточно долго для того, чтобы расти или меняться. Они возникают, выполняют свою задачу и исчезают. Именно архитектурный аспект *времени* заставляет нас столкнуться с неизбежными изменениями.

Использование при проектировании интерфейсов сервисов гипермедиаподхода дает нам ряд удобных инструментов для поддержки периодических изменений с сохранением стабильных, предсказуемых интерфейсов. Задействуя гипермедиа в дизайне сообщений, вы можете устанавливать в интерфейсах безопасные зоны для изменений. Средства гипермедиа (ссылки и формы) обеспечивают вектор для реализации изменений. Например, запрограммировав потребителя API на парсинг гипермедиаформ, сервисы могут спокойно изменять детали взаимодействия, такие как URL и HTTP-методы, а также формы сообщений и параметры. Таким образом гипермедиа обеспечивает то, что, по словам Пола Клеменца (Paul Clements, <https://oreil.ly/kyj2i>), профессора Университета Карнеги — Меллона и автора книг, должна обеспечивать хорошая программная архитектура: «*Знает*, что изменяется часто, и упрощает этот процесс».

Но одного только использования в сообщениях гипермедиа недостаточно. Поскольку мы говорим о сети, то сервисам, как правило, нужно общаться с *другими сервисами*. Если между сервисами есть зависимость, мы сталкиваемся с риском создания фатальной цепочки зависимостей, когда изменения в одном сервисе из стека сказываются на всех других, которые на него опираются. В таких случаях могут происходить неожиданные вещи. Порой изменения в одном из *вызываемых* сервисов приводят к изменению в интерфейсе. Иногда эти изменения могут оказаться несовместимыми с задачей *вызывающего* сервиса и стать критическими. Так что нам нужен способ, чтобы учитывать и разрешать проблему зависимости.

Самым простым решением будет полное устранение зависимостей, но обычно это нереально. Мы опираемся на другие сервисы не просто так — они предоставляют нечто нужное, чего у нас нет. Сегодня чаще всего следующим по эффективности

решением этой проблемы является поиск *другого* сервиса с такой же функциональностью. Обычно при этом наблюдается такая цепочка событий.

1. Сервисы аварийно прекращают работу, и об этом отправляются уведомления.
2. Специалисты реагируют на эти уведомления и находят решение — обычно замену сломанному сервису.
3. В промышленную среду отправляется обновленный компонент.

Более продуманным подходом (он реализуется в закрытых системах вроде Kubernetes) будет автоматизация этого процесса. Когда сервисы вводятся в эксплуатацию, их метаданные вносятся в реестры, где другие авторизованные сервисы могут найти их и сразу начать интеграцию в реальном времени, минимизировав время работы вхолостую, а то и вовсе его исключив.

Некоторые из рецептов главы 5 выполняют этот самореализуемый процесс нахождения и интеграции с помощью стандартизации и автоматизации. Подробнее поговорим об этом в главе 7, когда будем разбирать мультисервисные рабочие потоки.

Проблема изменяемости

Честно говоря, создавать интерфейс сервиса — веб-API — не особо сложно. Подтверждается этот факт длинным списком инструментов, которые считывают схему базы данных и/или сканируют существующие кодовые базы, выводя HTTP API, готовые для развертывания в промышленной среде. Создание интерфейса сложностей не представляет. А вот спроектировать продуманный и устойчивый API труднее. Ключевым элементом во всем этом выступает *время*. Сервисы, которые существуют долго, переживают обновления, то есть изменения, которые угрожают простоте и надежности API, соединяющего этот сервис со всем остальным. Именно время лежит в основе проблемы с изменяемостью.

Пока ваш сервис не изменяется, потребности в проектировании и реализации API могут быть минимальными. Это верно, например, в случае API, которые раскрывают сервисы на базе мейнфреймов или мини-компьютеров, выполняющиеся десятилетиями. Они вряд ли будут меняться, поэтому на их входе легко создавать стабильные интерфейсы. В этом контексте хорошо себя проявляют инструменты на основе схемы и объектов.

Но таких долгоживущих сервисов в мире становится все меньше, и все чаще API нужно подключать к недавно созданным сервисам, чей возраст — всего несколько лет. Во многих случаях эти сервисы проектировались под некие сиюминутные нужды, которые со временем расширились. Простым путем здесь будут создание нового интерфейса, присваивание ему идентификатора (например, /v2/) и повторная публикация. Сегодня так делают многие компании.

Причем наиболее предусмотрительные еще долго сохраняют старые версии. Это позволяет клиентским приложениям управлять планом развития в соответствии с собственным графиком. Но многие компании избавляются от старых

версий слишком рано. Это вынуждает потребителей API обновлять свой код более оперативно, чтобы поспевать за изменениями на стороне сервиса. Если учесть, что некоторые клиентские приложения используют более одного API, каждый с собственным графиком обновления, возникает вероятность того, что потребитель API всегда будет сталкиваться с проблемами обновления зависимостей. Это означает, что приложения под управлением API могут регулярно сбоить.

Более грамотным подходом будет применение паттерна, который позволит сервисам развиваться, не вызывая в использующих их клиентских приложениях сбоев. Для этого потребуется такой подход к проектированию API, который будет поддерживать стабильность интерфейса и его способность развиваться. Вот ряд принципов, которые ведут к созданию стабильных, спокойно развивающихся интерфейсов:

- использование «клятвы Гиппократ» в отношении API;
- следование правилу «не изменяй, а добавляй»;
- следование идее создания «вечных» API.

«Клятва Гиппократ» в отношении API

Один из способов решения проблемы с изменяемостью заключается в обещании никогда не «ломать» интерфейс, то есть сохранять его обслуживаемость при изменении. Это и является своеобразной «клятвой Гиппократ». Написанная между V и III веками до нашей эры, эта клятва стала этическим обещанием, которому должны были следовать греческие врачи. Самая известная ее строчка звучит так: «Я буду воздерживаться от причинения всякого вреда и несправедливости» (<https://oreil.ly/M9wUn>). Эту фразу зачастую перефразируют так: «Главное — не навреди».

При создании интерфейсов важно с самого начала придерживаться принципа «не навреди». Это означает, что можно вносить лишь такие изменения, которые не нарушат клятву. Вот три простых правила, которые можно использовать для выполнения клятвы не вносить никаких нарушающих работу изменений.

Ничего не убирать

После документирования и публикации URL конечной точки, поддержки для протокола или медиатипа, сообщения ответа, входного параметра или выходного значения их нельзя удалять при последующих обновлениях интерфейса. В будущих релизах вы можете установить значение на `null` или проигнорировать, но убирать его нельзя.

Ничего не переопределять

Вы не должны изменять смысл или назначение существующего элемента интерфейса. Например, если сообщение ответа было опубликовано как пользовательский объект, вы не можете изменить его на пользовательскую коллекцию. Если выходное свойство `count` было задокументировано как содержащее столько записей, сколько находится в коллекции, то нельзя изменить его значение на количество записей в одной странице ответа.

Делать дополнения опциональными

После первой публикации API любые дополнительные входные или выходные данные должны быть задокументированы как *опциональные* — нельзя добавлять в существующие интерфейсы новые *обязательные* свойства. Это означает, например, что вы не можете добавить новое входное свойство (`backup_email_address`) в действие интерфейса, создающее пользовательскую запись.

Не меняй, а добавляй

Следуя этим трем правилам, вы можете избежать наиболее распространенных критических изменений в API. Но это еще не все. Во-первых, вы всегда можете создавать *новые* конечные точки (URL), устанавливая в них обновленные правила, регламентирующие то, какие входные и выходные данные ожидаются. Возможность просто добавить опции — полезный инструмент поддержания способности сервиса развиваться без изменения существующих элементов.

Например, у вас может иметься возможность создать новый, улучшенный способ вернуть отфильтрованное содержимое. Изначальный вызов API не поддерживал возможность изменения количества возвращаемых в ответе записей. Оно всегда было установлено как максимум 100. Но теперь сервис поддерживает изменение `page_size` ответа. Это означает, что вы можете предложить две опции интерфейса, которые в медиатипе SIREN будут выглядеть, к примеру, так:

```
"actions": [
  {
    "name": "filter-list",
    "title": "Filter User List",
    "method": "GET",
    "href": "http://api.example.org/users/filter",
    "type": "application/x-www-form-urlencoded",
    "fields": [
      { "name": "region", "type": "text", "value": "" },
      { "name": "last-name", "type": "text", "value": "" }
    ]
  },
  {
    "name": "paged-filter-list",
    "title": "Filter User List",
    "method": "GET",
    "href": "http://api.example.org/users/paged-filter",
    "type": "application/x-www-form-urlencoded",
    "fields": [
      { "name": "page-size", "type": "number", "value": "100" },
      { "name": "region", "type": "text", "value": "" },
      { "name": "last-name", "type": "text", "value": "" }
    ]
  }
]
```

Обратите внимание: вы можете подумать, что вместо двух форм будет безопасно предложить одну обновляемую, включающую свойство `page_size` со значе-

нием 100. Так делать не стоит. Клиентское приложение может зависеть в том числе и от возврата в списке более 100 элементов. Изменяя это предустановленное поведение на возвращение только сотни строк, вы можете нарушить работу приложения.

Добавление новых элементов практически всегда безопаснее изменения существующих.

API создаются навечно

Интерфейс, то есть API сервиса — это контракт, который создатели сервиса заключают с потребителями API. Как и в случае с любыми другими контрактами, его условиям нужно следовать, иначе это будет рассмотрено как нарушение обещания. По этой причине с точки зрения проектировщиков API мудро рассматривать API как нерушимые и предполагать, что они будут существовать вечно. Вернер Вогелс (Werner Vogels), технический директор Amazon, преподносит это так (<https://oreil.ly/JdYkh>): «Мы знали, что проектирование API является очень важной задачей, поскольку у нас есть всего один шанс все сделать правильно». И хотя эта задача может показаться утомительной, ее можно существенно упростить, встроив в API возможность безопасного изменения элементов.

Естественно, со временем все меняется. Сервисы развиваются, у них появляются новые возможности, изменяются входные данные и обновляются выходные. Это означает, что заключенный с потребителями API контракт должен отражать возможность будущих изменений. По сути, изменение должно быть записано в соглашении.

Несмотря на то что для проектировщиков интерфейсов кажется разумным наличие возможности точно спрогнозировать, какими будут изменения интерфейса («Через два года мы добавим третье выходное свойство `better-widget`»), с их стороны также разумно создавать интерфейс, который будет учитывать возможность будущих изменений («Потребители API должны игнорировать любые дополнительные свойства, которых не понимают»). Иными словами, проект API должен не только точно описывать существующий в настоящий момент интерфейс сервиса, но и учитывать его возможные будущие изменения таким образом, чтобы потребители в любом случае их пережили.

К счастью, существует хорошо отработанный подход, который позволяет учитывать изменчивость интерфейса со временем. И называется этот подход *гипермедиа*.

Чем поможет гипермедиа

Здесь будет уместным задать вопрос: «Почему важно проектировать интерфейсы, которые будут поддерживать стабильность и способность развиваться?» Это определенно труднее, чем просто создавать статические API, привязанные напрямую к внутренним данным и/или моделям объектов. В таком случае большинству разработчиков не нужно беспокоиться о будущих изменениях. Рой Филдинг как-то сказал: «У разработчиков ПО всегда были проблемы с мышлением во временном

контексте» (<https://oreil.ly/Gb5JA>). Они стремятся просто создать что-либо, решающее поставленную перед ними задачу. Если отвечать на это коротко, то стоит просто сказать, что полезной целью является долгосрочная архитектура в любой своей форме — программной, сетевой, структурной и т. д. Нам всем следует стремиться создавать интерфейсы, которые будут полезными и останутся таковыми даже спустя длительное время.

Более же развернутый ответ в том числе затрагивает влияние грамотно спроектированных интерфейсов на развитие сферы вычислений и ее эгалитарную природу. Тим Бернерс-Ли, которому приписывают разработку структуры HTTP и HTML, сказал: «Веб — среда для всех» (<https://oreil.ly/IJYFB>). Создание продуманных долговечных интерфейсов позволяет людям взаимодействовать с лежащими в их основе сервисами. И чем больше взаимодействий, тем более эффективные и креативные варианты их использования возникают. Причем это может способствовать появлению положительных изменений не только в Интернете, но и в мире в целом. Протокол HTTP и его спутник HTML оказали невероятное влияние на то, как сегодня функционирует мир. Значительная часть успеха HTTP/HTML связана с тем, что спустя более трех десятилетий эти две стабильные и способные развиваться структуры продолжают помогать возникновению инноваций и креативности во всех уголках планеты.

Я не думаю, что моя работа по проектированию и реализации API когда-либо будет такой же влиятельной, но мне нравится верить, что мой вклад в развитие Интернета способствует воплощению того же принципа: «Веб — среда для всех». Поэтому я призываю проектировщиков и разработчиков API сервисов стараться создавать стабильные и способные развиваться интерфейсы.

Как же этого достичь? Стабильность мы можем предоставить потребителям API за счет использования для обмена сообщениями зарегистрированных структурированных медиатипов вроде HTML, HAL, Collection + JSON, SIREN и др. Для поддержки способности развиваться можно применить такие ключевые элементы, которые в сообщениях часто меняются. К ним относятся адреса (URL), действия (ссылки и формы), а также объекты данных (свойства сообщений).

Обеспечение стабильности с помощью форматов сообщений

Клиентские приложения API для эффективной работы должны иметь возможность опираться на стабильные сообщения запросов и ответов. Один из способов реализовать это — придерживаться использования одного из многих зарегистрированных структурированных форматов сообщений. То же самое делает клиент браузера. Он опирается на стабильный формат (HTML) для передачи большинства текстовых сообщений от сервиса клиенту. В веб-среде HTML применяется в качестве предустановленного формата ответа независимо от того, возвращает ли сервис (сайт) учетную информацию, данные социальной сети, игровую отрисовку или что-то иное. Использование хорошо известного и грамотно определенного формата очень важно для создания стабильных интерфейсов.



Подробнее о программировании с помощью структурированных форматов медиатипов читайте в моей книге *RESTful Web Clients* (<https://oreil.ly/C0YYr>, O'Reilly).

При реализации API сервисов вам доступно множество достойных форматов. Лично я для поиска форматов сообщений, отличающихся долгосрочностью и поддержкой, использую реестр медиатипов IANA (Internet Assigned Numbers Authority, Администрации адресного пространства Интернета, <https://oreil.ly/Vgjq2>). Самыми популярными форматами являются HAL (<https://oreil.ly/eeuxf>), SIREN (<https://oreil.ly/zD9q7>) и Collection + JSON (<https://oreil.ly/oxk6e>), но есть и ряд других.

В главе 5 приводятся несколько рецептов, которые помогают с выбором и поддержкой структурированных медиатипов.

Поддержка способности развиваться с помощью средств гипермедиа

Если первый шаг — это обеспечение стабильности посредством медиатипов, то второй шаг (поддержка способности развиваться) можно реализовать путем выбора медиатипов, предлагающих обширные средства гипермедиа. Чем больше гипермедиавозможностей вы включите в сообщения, тем больший потенциал к изменениям сможете поддерживать.

При создании гипермедиаформатов у вас есть возможность включать новые действия (дополнительные формы), а также безопасно обновлять существующие — добавлять свойства, обновлять URL и изменять HTTP-методы. Это самые распространенные элементы, которые будут меняться в интерфейсе сервиса.

Но у этого подхода есть и ограничения. Использование гипермедиаформатов позволяет обеспечить безопасное и последовательное изменение адресов (URL), объектов (свойств данных) и действий (форм). Однако вам может потребоваться также изменить внутренние свойства (имена элементов), например, добавить новое свойство в список возможных входных данных (`middleName`), новое выходное значение (`alternateEmail`) или новый параметр запроса (`/filter/?newRegionFilter=south`). В этом случае нужно будет создать новый документ словаря и позволить клиентам находить и выбирать эти дополнительные словари в среде выполнения (подробнее в главе 3). В главе 5 есть рецепты, нацеленные на предоставление этой возможности.

От самообслуживания до поиска и привязки

Еще одним важным элементом сервисов в веб-среде является возможность для потребителей API самостоятельно обслуживать собственное подключение. Такая возможность просто выбирать нужный сервис и сразу начинать его применять — достойная цель. По сути, именно так люди сегодня взаимодействуют с сайтами. Кто-то находит интересующий его контент и может скопировать ссылку, чтобы либо ею поделиться, либо вставить ее на собственную веб-страницу, где другие

смогут увидеть ее и воспользоваться. В Интернете этот опыт уже стал настолько типичным и неотъемлемым, что мы не представляем его как интеграцию или подключение. Мы просто думаем: «Так работает веб-среда».

Именно так должны работать сервисы. Но чаще всего все оказывается иначе. Вместо этого разработчики вынуждены вручную находить существующие сервисы, выбирать их и интегрировать в собственные решения. Это может быть неприятный процесс со множеством заминок. Бывает трудно найти то, что вам нужно. Когда же ты находишь API, который вроде бы подходит, могут возникнуть сложности с пониманием его документации. Кроме того, бывает трудно успешно интегрировать внешние API в свой сервис. Зачастую разработчики вынуждены повторять этот процесс неоднократно — по разу для каждого API, на который будет опираться сервис. И даже когда все сервисы готовы, протестированы и запущены, весь процесс может начаться заново, когда какой-либо из подключенных API изменит свой интерфейс.

Нам нужен способ автоматизировать основную часть этих действий в процессе, который будет происходить в среде выполнения. Этот процесс я называю «поиск и привязка». Путем стандартизации именования и описания сервисов (метаданные) мы можем автоматизировать механизм их обнаружения (поиска) и интеграции (привязки). Возможность находить удаленные сервисы, которые вы раньше не встречали, и взаимодействовать с ними является основой ныне существующей системы доменных имен (Domain Name System, DNS) (<https://oreil.ly/C69bp>), используемой для подключения машин друг к другу через Интернет.

Поддержка распределенных данных

Глава 6 содержит рецепты, акцентирующие на сохранении данных и их извлечении из подключенных по сети субъектов. На протяжении значительной части истории хранилищ данных в вычислительных средах центральное место в них занимали концепции *систем записи* (systems of record, SOR, <https://oreil.ly/OLPwv>) и *единого источника истины* (single source of truth, SSOT, <https://oreil.ly/RiKUs>). В целом эти концепции нацелены на обеспечение точности информации путем определения единственного места расположения для каждого элемента данных. В этом же контексте появляется принцип *управления мастер-данными* (master data management, MDM, <https://oreil.ly/60rk2>). Контроль за тем, кто может редактировать данные, является здесь ключевым моментом. По сути, в основе систем SSOT/MDM лежит предположение, что у каждого элемента данных есть всего один аутентичный источник. При этом все стороны соглашаются с тем, что это за источник и где он находится.

Тем не менее в открытой веб-среде невозможно проконтролировать, где хранятся данные и сколько копий создается для каждой их коллекции. В действительности правильно предположить, что *всегда* будет существовать больше одной копии любых данных, с которыми вы работаете. Может существовать возможность явно запросить последнее значение точки данных или попытаться отслеживать, какое

расположение нужно рассматривать в качестве точного источника конкретного элемента данных. Но вы должны предполагать, что ваша копия данных не всегда совпадет с той, которая есть у других.

ВСЕ ДАННЫЕ НАХОДЯТСЯ УДАЛЕННО

Понимание того, что данные в Сети имеют множество копий, очень важно. Пренебрегать им нельзя. Автор книг и архитектор ПО Ираклий Надареишвили (Irakli Nadareishvili) (<http://linkedapis.org>) однажды сформулировал аналогичное правило, сказав мне: «Рассматривай все данные так, будто они располагаются удаленно». Воспринимая данные таким образом, вы делаете ряд важных допущений: 1) вы не можете изменить средство хранения или схему; 2) вы не контролируете, кто может считывать информацию из этого источника или записывать в него; 3) в случае недоступности данных (временной или постоянной) вы никак не можете повлиять на это.

В случае сервисов, которым нужно обрабатывать собственные данные, это правило должно повлиять на то, как вы сохраняете данные, которыми вам поручено управлять, получаете доступ к ним и проверяете их. Например, по возможности нужно отслеживать, кто их у вас запрашивал (по журналу). Также нужно отслеживать, кто отправлял для них обновления. Сервисы данных отвечают и за сохранение *целостности локальных данных*. Ни у кого не должно быть возможности сделать недопустимую запись. Везде, где возможно, следует реализовать поддержку *отмены* обновления данных (в рамках ограниченного промежутка времени). Наконец, даже когда получено указание удалить данные, стоит сохранять их копию какое-то время на случай, если их потребуется восстановить (опять же здесь должно действовать установленное временное окно).

Сформулированное Надареишвили правило важно для сервисов, которые зависят также от *удаленных* данных. В ситуациях, когда ваш сервис не управляет данными, с которыми работает, но зависит в этом смысле от других сервисов, желательно всегда спрашивать у удаленных источников последние копии этих данных. Таким образом вы будете учитывать все изменения, которые произошли с момента последнего считывания или получения данных. Кроме того, вы должны быть готовы к отклонению обновлений ваших данных зависимыми сервисами, с которыми взаимодействуете. Сервис, который *сохраняет* данные, отвечает и за их целостность.

Вы должны быть готовы к отмене обновлений данных и при необходимости включать отмену их удаления. Особенно трудным это становится в ходе работы с сервисами, которые взаимодействуют не с одним источником данных (скажем, сервисом, который считывает из сервисов `customerData` и `billingData` или записывает в них). Например, когда обновление `customerData` принимается, а обновление `billingData` отклоняется, вашему сервису нужно знать, важно ли *отменить* изменение `customerData`.

Естественно, бывают случаи, когда сервис управляет собственными данными и зависит от других их источников. Со мной такое случалось нередко. Еще один распространенный сценарий — вам потребуется сохранять локальную кэшированную *копию* данных, полученных из другого источника. Это может повысить производительность и облегчить трафик для широко используемых ресурсов. Но копии — это всего лишь копии. Когда кто-то попросит у вас последнюю копию данных, может потребоваться запросить обновленную версию записи, которая хранится у вас локально.

Данные — это свидетельство действия

При создании ориентированных на данные интерфейсов важно учитывать принцип «данные — это свидетельство действия». Данные являются *следствием* некоторого выполненного действия. Например, при создании или обновлении ресурса создаются или изменяются свойства данных. Данные — это свидетельство, оставшееся после действий по созданию или обновлению. Прodelайте это много раз (например, при множестве изменений от многих клиентских приложений), и вы получите коллекцию действий. Располагая такой коллекцией, можно задавать вопросы о связанных с ней свидетельствах, то есть отправлять запросы. Так что основная ценность хранилищ данных заключается в том, что они дают возможность задавать вопросы.

Если рассматривать данные таким образом, то они становятся своеобразными свидетелями происходящих в веб-среде событий. А самый точный свидетель — тот, который предоставляет лучшее свидетельство. В Интернете лучшее свидетельство — это сами операции обмена HTTP-сообщениями. Лучшим форматом хранения являются сами HTTP-сообщения (и метаданные, и тело). Возможность проинспектировать и воспроизвести процесс HTTP-обмена позволяет создать мощную платформу для хранения данных. Существует даже специальный формат документов (<https://oreil.ly/YvGqi>), предназначенный для регистрации взаимодействий сервера и клиента, — HTTP Archive (HAR).

МЕДИАТИПЫ */HTTP

Существует довольно старый, но редко используемый медиатип `message/http` (<https://oreil.ly/AleUe>), предназначенный для непосредственного перехвата и сохранения HTTP-сообщений. Есть и медиатип `application/http` (<https://oreil.ly/gwtKt>), который служит для перехвата серии HTTP-запросов и ответов в виде группы.

Эти медиатипы вместе с форматом HAR предлагают прекрасные возможности для точного перехвата и сохранения HTTP-операций.

Проблема в том, что прямые HTTP-сообщения (файлы HAR и сообщения *.http) сложно запрашивать эффективно. По этой причине вам может потребоваться сохранять свидетельства HTTP-сообщений в более подходящем для механизма запросов формате. Здесь на выбор доступно множество систем для сохранения и извлечения данных. В одних упор сделан на удобстве записи данных, а в других — на упрощении их запроса. Важно помнить, что все эти системы хранения служат для *оптимизации* управления свидетельствами. Они являются новым представлением действий в веб-среде.

Подытожим. Для веб-сервисов вам нужно обязательно регистрировать и делать пригодными для запроса *все* свидетельства действий. Это означает HTTP-сообщения целиком, включая их метаданные и данные. Средство, которое вы для этого выберете (файлы, документы, строки и т. д.), не так важно, как сохранение качества информации с течением времени.

Внешнее и внутреннее

Какие бы хранилища данных и форматы запросов вы ни использовали в API, важно, чтобы внутренняя модель данных не утекала в интерфейс сервиса. В 2016 году я опубликовал в Twitter (<https://oreil.ly/aPhMk>) аксиому, чтобы помочь людям запомнить важность капсуляции:

«Ваша модель данных — это не ваша объектная модель, и не модель ресурсов, и не репрезентативная модель».

Стоит еще раз освежить в памяти эту аксиому при обсуждении сервисов, которые напрямую отвечают за сохранение данных и управление ими. Используемая вами модель данных *изнутри*, то есть имена и расположение свойств данных, их группировка в таблицы, коллекции и т. д., должна оставаться независимой от модели, которую вы применяете для сообщений в виде документов *снаружи*. Это поможет сохранить слабую связанность между хранилищем и деталями интерфейса, обеспечив более надежный и стабильный опыт как для потребителей, так и для производителей API.

Например, интерфейс сервиса может предоставлять три типа ресурсов: `user`, `job-type` и `job-status`. Не зная ничего о внутреннем хранилище, потребители API легко могут предположить, что в нем есть три коллекции, соответствующие этим типам ресурсов. Но на деле может оказаться, что `job-status` — это просто свойство `job-type` или что и `job-type`, и `job-status` хранятся в коллекции `name-value`, которая содержит и другую аналогичную информацию вроде `user-status`, `shipping-status` и пр. Суть в том, что мы не знаем наверняка, какова в действительности модель хранилища. И это не должно иметь значения для какого бы то ни было потребителя API.

Мы также не знаем, какая *технология* хранения используется для управления только что упомянутым примером данных. Это может быть простая файловая система, сложная объектная база данных или коллекция необработанных HTTP-сообщений. И что самое главное, это не имеет значения также для потребителя API. Важно для него то, какие свойства данных доступны, и способ, которым он может получить к этим данным доступ (и, возможно, их обновить). Детали считывания и записи не должны изменяться часто — это одно из обещаний интерфейса. А вот технология хранения может меняться так часто, как того пожелают создатели сервиса, при условии, что они продолжают выполнять обещания интерфейса.

Все это сводится к еще одному ключевому принципу создания ориентированных на данные сервисов: «Тщательно обдумывайте внешние обещания (интерфейса)». Эти обещания нужно будет выполнять длительное время. И напротив, не слишком беспокойтесь о внутренних технологических деталях. Они могут и наверняка будут меняться часто, исходя из трендов технологий, доступности сервисов, затрат и прочих факторов.

Чтение и запись

Когда речь идет о распределенных данных, правила их чтения и записи кардинально различаются. Например, чаще всего запись можно без проблем отложить — как посредством дизайна, так и с помощью стандартного способа обмена сообщениями в веб-среде. А вот задержку чтения потребители API замечают более наглядно, и это может непосредственно влиять на воспринимаемую скорость и надежность источника данных. Это особенно актуально, когда для выполнения запроса чтения используются несколько источников данных.

Чем большее расстояние (в пространстве или времени) данным предстоит преодолеть, тем выше вероятность того, что потребители API ощутят задержку. В этом случае радует то, что большинство потребителей могут пережить задержки в пределах 1 с без каких-либо серьезных последствий. Правда в том, что *все* запросы требуют времени и не бывает мгновенных ответов. Просто большинство ответов оказываются достаточно короткими для того, чтобы мы не замечали их продолжительность или она нас не волновала.

По факту лучший способ реализации ответов при чтении данных — их предоставление без участия сетевой передачи. Это означает использование локальных кэшированных копий данных. Такой подход снизит зависимость от сети (не придется беспокоиться о невозможности связаться с источником данных) и сократит время ответа, так как нужно будет лишь собрать локальные данные в соответствии с запросом. Подробнее читайте в рецепте 6.9.

Стоит также отметить, что *люди* обычно проще переносят задержки, нежели *машины*. Это означает, что задержки ответов при М2М-взаимодействии могут оказаться большей проблемой, чем в случае взаимодействия человека и машины. По этой

причине желательно реализовывать М2М-взаимодействия с учетом возможных задержек ответов, то есть планировать наперед. Рецепты 6.9 и 6.12 дают возможные способы противодействия эффекту медленных ответов в ориентированных на данные сервисах.

Поскольку задержки чтения более заметны, нужно учитывать их при проектировании интерфейса. Например, если ответы на какие-то запросы наверняка окажутся длительными (как в случае с большим набором данных, составляющим многогигабайтный отчет), следует проектировать такой интерфейс, который сделает эту задержку явной. В случае интерфейсов на основе HTTP это означает сопровождение ответа 202 Accepted последующими сообщениями о состоянии (подробнее — в рецепте 6.5).

Если же мы говорим о записи, то хотя задержка в этом случае и нежелательна, важнее сохранить в ходе процесса целостность записываемых данных. Возможно, одно сообщение записи нужно будет обработать нескольким сервисам (поочередно или параллельно) и каждое взаимодействие несет в себе вероятность ошибки. В некоторых случаях сбой всего в одном из источников хранения означает, что все другие источники тоже должны отклонить или отменить запись. Поэтому стоит ограничить для каждого действия записи количество целей — лучше всего, если она будет одна. Если их будет больше, то возможны проблемы. Подробнее о том, как повысить производительность записи интерфейсов сервисов, читайте в рецептах 6.5, 6.8 и 6.13.

Надежные языки описания данных

За всю историю хранения и запроса данных было создано множество прекрасных платформ. Тем не менее до недавних пор большинство языков и платформ обработки данных не учитывали уникальных требований веб-среды. Немногие языки удобно спроектированы для широко распределенных данных, и некоторые из них очень эффективны в веб-среде, где источников не только много, но они еще и удалены друг от друга.

Чаще всего наиболее эффективным для чтения данных в Интернете оказывается набор языков под названием IRQL (<https://oreil.ly/GIqet>). Эти языки оптимизированы для сопоставления с критерием поиска и возвращения набора документов или указателей на документы. Они также оптимизированы для поиска в больших наборах данных, поскольку большинство IRQL по факту представляют коллекции индексов для внешнего хранилища. В связи с этим для любого сервиса с поддержкой запросов данных по HTTP желательно реализовать какой-нибудь IRQL. Это особенно актуально, если API нужна поддержка только чтения, без записи. Но даже в ситуациях, когда интерфейс поддерживает и чтение, и запись данных, реализация IRQL себя оправдывает, поскольку большинство запросов в любом случае представляют чтение.

ЗАПРОСЫ К БАЗАМ ДАННЫХ И ПОИСКОВЫЕ ЗАПРОСЫ

Есть важное различие между языками запросов данных (<https://oreil.ly/G6KIH>) вроде SQL (Structured Query Language, <https://oreil.ly/5DJJ1>) и языками поисковых запросов (<https://oreil.ly/KwCT5>) вроде Apache Lucene (<https://oreil.ly/SGycR>). Первые возвращают исчерпывающие результаты относительно некоего набора фактов из базы данных. Вторые же (IRQL) возвращают набор документов, соответствующих указанному критерию. В рецептах главы 6 мы будем работать преимущественно с IRQL, то есть с поиском информации. Естественно, можно найти и примеры использования языков запросов данных для поиска документов (обычно строк), соответствующих указанному критерию.

Механизмы извлечения данных

Есть несколько кандидатов на роль IRQL в вашем интерфейсе. Одним из самых популярных является проект Apache Lucene (<https://oreil.ly/XaTMD>). Механизм Lucene достаточно низкоуровневый и в своей оригинальной форме реализуется не очень легко. Но есть несколько других реализаций, созданных на базе Lucene, которые проще развертывать и обслуживать. К примеру, на момент написания книги мне чаще всего встречается механизм Solr (<https://oreil.ly/mAsf7>).

Другие варианты IRQL

Помимо IRQL для чтения данных большинству сервисов потребуется также поддержка их записи. И хотя несколько IRQL-подобных механизмов поддерживают и чтение, и запись, например GraphQL (<https://graphql.org/>), SPARQL (<https://oreil.ly/cSzFt>), OData (<https://www.odata.org/>) и JSON:API (<https://jsonapi.org/>), стоит рассмотреть применение для записи другой технологии.

SQL-подобные механизмы обработки данных

Как правило, для простого хранения данных можно реализовать систему на основе файлов (каждая запись в этой системе будет файлом). Тем не менее, если сервису нужна возможность масштабирования на значительное число пользователей или несколько сотен документов, лучше реализовать хранилище, оптимизированное под запись. Типичные решения опираются на SQL-технологии вроде MongoDB (<https://oreil.ly/q7fM0>), SQLite (<https://oreil.ly/0asA4>), PostgreSQL (<https://oreil.ly/5rcSL>) и пр.

Стриминговые механизмы обработки данных

Если вам нужно обрабатывать большие количества записей в секунду (скажем, тысячи), наверняка потребуется реализовать стриминговый механизм вроде Apache Kafka (<https://kafka.apache.org/>), Apache Pulsar (<https://pulsar.apache.org/>) или их аналогов.

Главное — рассматривать все эти варианты как закулисные детали. Грамотно спроектированный интерфейс для ориентированных на данные сервисов *не будет* раскрывать внутренние технологии. Это позволит обновлять их по необходимости без вреда для внешнего API (см. раздел «Чтение и запись» ранее в этой главе). Например, вы можете начать с системы управления данными на основе файлов, добавить IRQL для поддержки дополнительных запросов, а потом переехать на SQL-решение и в конечном счете на стриминговый механизм обработки. В течение всего этого внутреннего технологического пути внешний интерфейс меняться не должен. Подробнее об этом рассказано в рецептах 6.1, 6.4 и 6.7.

Увеличение расширяемости с помощью рабочего потока на основе гипермедиа

Веб всегда был пространством, где серверы с контентом могут легко соединяться друг с другом (через ссылки), формируя более масштабную систему. Эта возможность легко подключаться к другому сайту, не спрашивая разрешения, писать особый код, создавать макет или вносить изменения в контент была одной из целей создателей Всемирной паутины. Поэтому, предлагая сформировать всемирную информационную систему (<https://oreil.ly/CwzAm>), Тим Бернерс-Ли писал: «Нам следует работать в направлении универсальной связанной информационной системы, в которой обобщаемость и портативность важнее навороченных графических техник и сложных дополнительных средств». В главе 7 приводятся рецепты, которые помогут реализовать именно эту задачу.

И хотя Бернерс-Ли, а до него Тед Нельсон работали над созданием удобных способов связывания именно *документов*, тот же принцип простоты соединения с некоторыми поправками можно применить и к *сервисам*. Связывая несколько сервисов, мы можем решать конкретные задачи. Например, шаги, необходимые для оформления заказа в онлайн-магазине, могут включать действия `computeTotals`, `applyTax`, `arrangePayment`, `scheduleShipping` и `sendConfirmation`.

В ходе работы с локальным исходным кодом эти действия обычно выражаются как операции в базе кода и серия последовательно выполняемых шагов. Однако некоторые языки позволяют также использовать параллельную обработку для одновременного выполнения набора действий. Это может сильно повысить масштабируемость рабочего потока, но требует, чтобы его проектировщики и проектировщики сервиса договорились о четком отделении параллельных задач от последовательных. Даже когда все задачи пригодны для параллельного выполнения, сам рабочий поток может требовать специальной последовательности событий. К примеру, сложно вычислить налог для онлайн-заказа до завершения комплектации корзины.

В случае веб-сервисов каждую функцию нужно рассматривать так, будто она выполняется в отдельном процессе в другом месте. И ввиду распределенной природы

веб-среды нельзя рассчитывать на то, что сервисы будут совместно задействовать общие модели данных, системы хранения или языки программирования. Единственное, что они имеют общего, — это утвержденный *интерфейс*. Общий интерфейс в Интернете определяется спецификацией HTTP. Но это очень низкоуровневое соглашение. Для эффективного выполнения работы сервисам нужно использовать не только общий протокол передачи. Им необходим и своеобразный язык для координации.

Хореография, оркестрация и рабочий поток на основе гипермедиа

Когда мы говорим о координации сервисов, нередко используем понятия *хореографии* и *оркестрации*. Каждое из них представляет структурный взгляд на то, как проектировать и реализовывать координацию сервисов. По своему опыту могу сказать, что веб-среда предлагает третий, более прямой и, думаю, более эффективный взгляд на их координацию — описание рабочих потоков как декларативных документов, которые, подобно HTTP, предоставляют согласованный интерфейс, позволяющий сервисам взаимодействовать друг с другом. Чуть позже я опишу *этот рабочий поток на основе гипермедиа*. Но для начала следует разобраться, что нам предлагают хореография и оркестрация, а также чем они отличаются от подхода на основе гипермедиа.

Централизованная оркестрация

Модель оркестрации сервисов — это централизованный подход к определению рабочих потоков в одном контексте и последующей отправке этого документа в механизм рабочего потока. Таким образом мы создаем интеграционную модель программирования, например BPEL (<https://oreil.ly/3JQaa>), которую проектировщики решения могут использовать для соединения сервисов в логичном потоке выполнения. Этот подход зачастую ассоциируется с управлением оркестром, поскольку в нем есть один «дирижер» (механизм), который ведет группу людей, действующих на основе одинаковой, заранее утвержденной «музыки» (документа рабочего потока).

Преимущество оркестрации заключается в том, что данное решение явно прописывается в скрипте или коде. Это упрощает понимание, проверку и упреждающее тестирование рабочего потока. Использование механизма рабочего потока также облегчает его мониторинг и управление. Существует немало платформ и программных наборов, поддерживающих BPEL и аналогичные языки.

У централизованного подхода есть и свои сложности. Самая серьезная из них — зависимость от центрального механизма выполнения. Если этот механизм не работает или недоступен, вся система рабочего потока может встать. В качестве обходного решения можно запустить механизм внутри кластера (чтобы обрабатывать

вышедшие из строя машины) и в нескольких местах (для обработки проблем сети). Еще одной проблемой является то, что оркестрация зачастую предполагает синхронную обработку и может подталкивать к более связанной реализации.

ИСТОРИЯ ОРКЕСТРАЦИИ

Оркестрация сервисов имеет длинную историю. Пожалуй, самым известным ее истоком является XLang (<https://oreil.ly/Wd02n>) — язык рабочих потоков веб-служб (Web Services Flow Language, WSFL, <https://oreil.ly/3UFgR>) и его стандартизированный потомок, язык выполнения бизнес-процессов (Business Process Execution Language, <https://oreil.ly/haECS>), или WS-BPEL (<https://oreil.ly/rdnKc>). Эти языки созданы для описания серии шагов рабочего потока и решений, а также контроля выполнения этого потока путем отправки его документа во внутренний механизм, такой как Microsoft BizTalk (<https://oreil.ly/BYYLr>), IBM WebSphere (<https://oreil.ly/9QERU>), Apache ODE (<https://oreil.ly/SIFkR>) и пр.

Наконец, модель оркестрации предполагает единую точку контакта. Как правило, сервисы не взаимодействуют друг с другом напрямую, а общаются с механизмом оркестрации. Точнее, этот механизм совершает вызовы к сервисам и составляет собственное итоговое решение.

ОРКЕСТРАЦИЯ ОБЛАКА

Большинство современных облачных сервисов предлагают некий вид оркестрации. Amazon Web Services предоставляет AWS Step (<https://oreil.ly/49G4o>), у Google есть Workflows (<https://oreil.ly/l2LX7>), а Microsoft полагается на Azure Cloud Automation (<https://oreil.ly/l2LX7>).

Проблемы выхода из строя основного механизма и тесно связанного с ним рабочего потока можно решить, создав центральный репозиторий для *документов* рабочего потока и распределив их обработку. Такой распределенный подход очень похож на другой популярный способ проектирования рабочих потоков — *хореографию* сервисов.

Хореография без состояния

В данном контексте хореография описывается как своеобразный танец сервисов. Каждый сервис знает свои движения и действует как независимая сущность, которая взаимодействует с другими. В случае хореографии рабочий поток является продуктом взаимодействия между сервисами.

ПОТАНЦУЕМ?

В описаниях хореографического подхода к реализации рабочего потока часто используется аналогия с танцами. В действительности тема танца в отношении многогранного рабочего потока так сильна, что для обработки интеграции его частей в 2017 году был выпущен целый язык программирования под названием *Ballerina* (<https://oreil.ly/ibzLh>).

У модели, построенной на хореографии, определенно есть свои преимущества. В первую очередь отдельным сервисам не нужно видеть общую картину. Им достаточно знать, как выполнить собственные задачи. Интеграция компонентов рабочего потока приводит к менее связанному решению, а такие решения обычно оказываются более устойчивыми и пригодными к изменениям в будущем. Это может упростить последующую замену деталей решения без нарушения других процессов, обеспечив более гибкую и отказоустойчивую среду рабочего потока.

Естественно, и здесь есть свои недостатки. Одна из проблем в мире хореографии — это сложность мониторинга прогресса отдельного рабочего потока. Поскольку решение формируется из набора отдельных, не имеющих состояния задач, в нем отсутствует единая точка контроля, через которую вы могли бы получать оповещение о проблемах с выполнением задачи.

Также бывает сложно проанализировать общее состояние экосистемы рабочего потока. Есть ли какие-то конкретные задания, например отправка писем, которые проваливаются во всех задачах? Есть ли проблемы во всех задачах, которые отправляют письма по одному нерабочему адресу? Есть ли машины, которые работают плохо независимо от того, какие на них размещены сервисы? Есть ли сервисы, недостижимые из-за проблем сети?

ТАНЦУЯ В ОБЛАКАХ

У всех ведущих облачных провайдеров есть своя версия хореографии сервисов. Google предлагает службы Pub/Sub (<https://oreil.ly/joyxW>) и Evantarc (<https://oreil.ly/2EV9S>). В Microsoft для этого разработали Event Grid (<https://oreil.ly/KITjJ>), а AWS для создания рабочих потоков на основе хореографии предоставляет инструменты Simple Notification Service (SNS, простая служба уведомлений, <https://oreil.ly/JJ9dm>) и Simple Queue Service (SQS, простая служба очередей, <https://oreil.ly/ky0aH>).

Проблема с мониторингом решается путем создания для каждой задачи ресурса прогресса, который будет легко найти и просмотреть. В итоге можно будет дать

сервисам команду отправлять связанным с ними ресурсам отчеты о прогрессе при каждом завершении задания.

И централизованная оркестрация, и рассредоточенная хореография имеют свои преимущества и недостатки. В целом рабочие потоки с небольшим числом шагов и вариантов ветвления хорошо обрабатываются с помощью оркестрации. Определите работу в одном месте (документе) и используйте этот документ в качестве сценария выполнения. Зачастую его можно написать в виде нативного исходного кода (см. рецепт 7.3).

Тем не менее, если рабочий поток довольно обширен, лучше задействовать подход с хореографией. Чем больше количество шагов, тем менее эффективным становится последовательный подход оркестрации. Кроме того, множественные точки ветвления могут затруднить понимание результатов и откат проблемных рабочих потоков. Хореография же упрощает обработку асинхронных рабочих потоков, поддерживает откаты для отдельных составляющих потока и может избавить от ветвления в стиле `if .. then .. else`. Это предполагает, что каждый сервис работает независимо и они разделяют очень мало информации о состоянии. Решения на базе хореографии порой имеют вид исходного кода, но могут быть выражены и на предметно-ориентированном языке (*domain-specific language*, DSL), созданном под нужную задачу. В книге я покажу вам DSL, разработанный как раз для этой цели (см. рецепт 7.4).

Рабочий поток на основе гипермедиа

Есть и другой надежный подход, который опирается на использование средств гипермедиа (форм и ссылок), описывающих детали взаимодействия сервисов в среде выполнения. Я всегда называл этот подход *гипермедийным рабочим потоком*. Он прекрасно работает как для сценариев из небольшого числа шагов, так и для таких, в которых задействовано несколько процессов. В таком случае используется обеспечиваемая хореографией независимая параллельная обработка, а также централизованный подход к оркестрации.

ДЖАЗ КАК АНАЛОГИЯ РАБОЧЕГО ПОТОКА

Помимо аналогий оркестра и танца в контексте координации сервисов я также применил аналогию джаза для описания гипермедийного рабочего потока. При использовании оркестрации есть единый ответственный. В случае хореографии важно, чтобы каждый понимал свою роль и работал в тесном сотрудничестве с другими для получения итогового результата. Что же касается джазовой музыки, то в ней каждый участник команды имеет общее представление о том, что за композицию нужно сыграть, и вносит собственный вклад в ее исполнение.

Самое важное в проектировании и реализации рабочих потоков на основе гипермедиа — обеспечить, чтобы каждый сервис такого потока имел *сборный* интерфейс (см. рецепт 7.1), поддерживающий следующие действия.

Execute

Выполнение действия, например `computeTax`, `createCustomerResource` и т. д.

Repeat

Повторение того же действия, если в первый раз что-то пошло не так.

Revert

Отмена изменений, если произошла ошибка где-то еще.

Cancel

Прекращение обработки задания и отмена всего выполненного ранее.

Когда каждый сервис, называемый *заданием* (task), поддерживает эти действия, их можно собирать вместе в *задачу* (job). Задачи рабочего потока выполняют все задания параллельно и обеспечивают два других важных действия в дополнение к уже рассмотренным.

Continue (продолжить)

Продолжить работу с места ее последней остановки.

Restart (перезапустить)

Начать сначала.

Cancel (отменить)

Прекратить обработку задачи и приказать всем заданиям отменить сделанное ранее.

В главе 7 мы разберем еще кое-какие детали этого языка рабочих потоков, касающиеся общего состояния и тайм-аутов. Причем важно помнить, что описанный здесь интерфейс рабочего потока очень прост и обычно для отдельного сервиса реализуется легко. Но есть тут и сложности. Например, поддержка действия `Revert` имеет последствия, если сервис скрыто вызывает *другие* сервисы. Однако эти детали можно урегулировать с помощью кода и в рамках интерфейса их показывать не нужно.

Еще одно преимущество рабочего потока гипермедиа состоит в том, что все его подробности можно отобразить в одном или нескольких *документах*, а не в виде кода или скрипта. Большинство языков программирования *императивны*, то есть описывают, *как* что-либо выполняется. Документы гипермедиа, напротив, *декларативны*. Декларативные языки описывают, *что* нужно сделать, без явного приведения пошагового процесса выполнения.

Эта возможность описывать действия как задания, а потоки — как задачи позволяет писать сборные сервисы, которые можно включать в рабочий поток, не беспокоясь

о том, как все их части взаимодействуют. Аналогично HTTP гипермедийный рабочий поток стандартизирует внешний интерфейс и позволяет сервисам использовать любые внутренние языки программирования и парадигмы.

Сложности рабочих потоков

Реализация рабочих потоков в веб-среде несет с собой много сложностей, но их все можно разбить на относительно небольшой набор отслеживаемых проблем. Вот те, о которых я буду здесь говорить:

- использование общего состояния, а не моделей данных;
- ограничение рабочих потоков;
- наблюдение рабочих потоков;
- время как элемент рабочего потока;
- обработка ошибок рабочих потоков.

Использование общего состояния, а не моделей данных

Основной камень преткновения при создании надежных масштабируемых рабочих потоков заключается в обработке данных, которые нужно передавать между независимыми сервисами. Большинство известных мне решений подразумевают применение общей модели данных (например, передачу параметров в виде строго типизированной схемы) или даже общее хранилище данных (при этом, например, все сервисы считывают одни и те же таблицы SQL). Ни одно из таких решений не работает как надо.

Более эффективным будет совместное использование свойств с помощью простой модели документа (см. рецепт 7.2). В этих случаях значения данных передаются как строго типизированные *документы* (HTML, Collection + JSON, SIREN, HAL и т. д.) и могут содержать все типы элементов данных. То, какие свойства данных будут представлены в этих документах, определяется заранее установленными семантическими профилями (см. рецепт 3.4) и словарями (см. рецепт 3.3).

Совместную работу с этими документами можно наладить с помощью аффорданса FORM (см. рецепт 3.7) или самого общего адресуемого ресурса. Успешные реализации рабочих потоков поддерживают передачу данных между сервисами без ограничения типов совместно используемых данных.

Ограниченный рабочий поток

Если рассматривать поток как набор ограничений (*execute, repeat, reverse, continue* и *restart*), можно проектировать и реализовывать сервисы согласованно при условии, что они заранее согласятся с правилами. Аналогично имеющемуся у HTML набору правил для гипермедийных аффордансов (HTML.A, HTML.FORM, HTML.IMG и HTML.LINK) с четко определенными семантикой

и результатами рабочий поток можно спроектировать в виде набора аффордансов (*execute* и пр.) с четко определенными семантикой и результатами.

В надежной системе рабочего потока в качестве набора аффордансов определяется общий интерфейс между сервисами.

Наблюдаемый рабочий поток

Еще один важный элемент успешного рабочего потока — это возможность видеть, что в нем происходит в среде выполнения (см. рецепт 7.7). Такая наблюдаемость типична для общих систем потоков вроде пайплайнов DevOps и может оказаться не менее ценной для общего рабочего потока среды выполнения. Вам также потребуется способ вмешаться в поток для исправления проблем и, если это окажется подходящим, продолжить (*continue*) либо перезапустить (*restart*) задачу, а если ничего не работает, то отменить ее (*cancel*).

Продуманная система рабочих потоков предоставляет набор информационных панелей, отражающих процесс выполнения.

Время как элемент рабочего потока

Любой системе, которая пытается скоординировать сервисы, нужно отслеживать *время*, затрачиваемое на выполнение задания или завершение задачи. Естественно, некоторым процессам может потребоваться больше времени на завершение. Вычисление чистой приведенной стоимости или обработка сложных арифметических операций может занять минуты и даже часы. Системы рабочих потоков должны это учитывать (см. рецепт 7.15).

В здоровых системах устанавливается лимит (*maxTTL*) выделенного на работу времени. Если его превысить, придется отменить (*cancel*) процесс.

Обработка ошибок рабочих потоков

Наконец, процессы рабочих потоков должны учитывать ошибки среды выполнения, вызываемые состоянием сети (см. рецепт 7.16) или другими локальными проблемами (см. рецепт 7.17). Существуют способы автоматизации некоторых таких ответов, но иногда машины не могут определиться с решением. Для подобных случаев им нужен способ «позвать на помощь» (см. рецепт 7.18) человека, который примет правильное решение о том, стоит продолжить (*continue*) или отменить (*cancel*) процесс.

Краткое обобщение

Теперь, когда мы изучили предпосылки и внутреннюю подоплеку рецептов дизайнера, клиентов, сервисов, данных и рабочих потоков, можно переходить к более подробному знакомству с ними, чтобы научиться создавать отказоустойчивые и надежные сетевые решения.

Часть II

Рецепты для работы с гипермедиа

Ни один архитектурный шаблон не является изолированным. Каждый из них может существовать в мире только до тех пор, пока поддерживается другими шаблонами.

Кристофер Александер

ГЛАВА 3

Проектирование гипермедиа

По сути, проблема заключается в том, о чем пишут писатели-фантасты: «Как наладить коммуникацию между совершенно разными разумными существами?»

Дж. К. Р. Ликлайдер (1966)

Фундаментальным элементом любого проекта на уровне системы выступает набор общих принципов или представлений о взаимодействии частей этой системы. Именно о решении всеобъемлющей проблемы — как проектировать системы, в которых машины, созданные никогда не встречавшимися людьми, могут успешно взаимодействовать друг с другом, — мы и будем говорить в текущей главе. Рецепты этой главы акцентируются на аспектах проектирования гипермедийных систем. В них мы будем изучать связи между медиатипами, средствами гипермедиа, свойствами данных и семантическими профилями, которые помогают все эти элементы объединить (рис. 3.1).

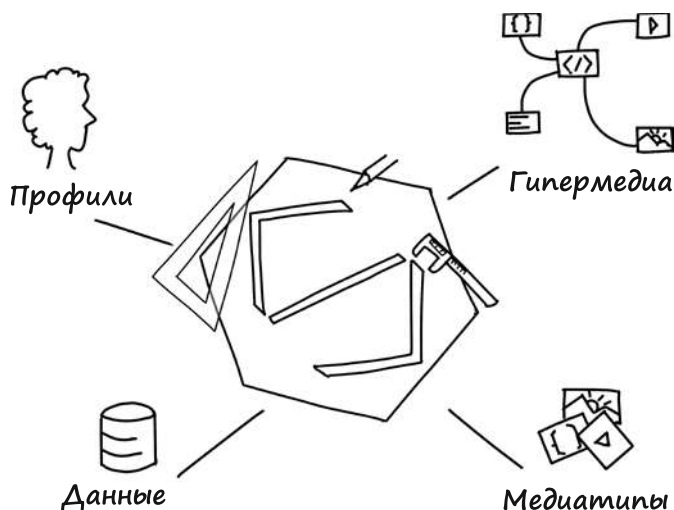


Рис. 3.1. Рецепты проектирования гипермедиа



Если вас интересуют принципы и технологии, стоящие за решениями на основе гипермедиа, читайте раздел «Закладываем основы с помощью проектирования гипермедиа» главы 2.

При взаимодействии в веб-среде основной общего понимания выступает HTTP. Этот протокол устанавливает правила — и *ожидания* — при пересылке данных между сервисами. И несмотря на то, что история HTTP начинается аж в середине 1980-х, эта технология по-прежнему повсеместно применяется и остается актуальной.

Важно помнить, что HTTP — это протокол верхнего уровня, определяющий взаимодействие между машинами сети. Более низкоуровневые протоколы вроде TCP (<https://oreil.ly/qypkt>), IP (<https://oreil.ly/LpNOn>) и UDP (<https://oreil.ly/ES6sA>) обеспечивают основу для перемещения битов информации. Интересно в работе этой низкоуровневой системы коммуникации то, что участники взаимодействия не понимают значения данных, которые передают. Смысл сообщения отделен от него самого.

Принцип обработки информации отдельно от сообщений, используемых для ее передачи, является ключевым фактором, который делает возможным масштабное межмашинное взаимодействие. Работа с информацией на ее собственном уровне тоже очень важна. HTTP — прекрасный пример реализации этого с помощью медиатипов. Я могу отправить вам данные компании о продажах в сообщении HTML. Ту же информацию я могу отправить в CSV-сообщении или в виде простого текста и т. д. *Данные* не зависят от применяемого медиатипа.

Этот принцип разделения вы увидите во всех рецептах текущей главы. Несмотря на то что протокол и медиатип представляют твердо установленные формы разделения, эти рецепты также опираются на дополнительную форму — отделение *словаря* от формата сообщений. Словарь — это набор правил, используемых для передачи значимой информации. Лучше всего коммуникация работает, когда все взаимодействующие стороны задействуют общий словарь.

Представьте, что две ожившие машины общаются друг с другом:

- Машина 1, давай поговорим о здравоохранении, используя словарь HL7 FHIR Release 4 (<https://oreil.ly/vQUQ6>).
- Хорошо, машина 2. Но для отправки сообщений давай применять RDF, а не XML.
- Конечно, машина 1. При условии, что мы будем использовать HTTP, а не MQTT.
- Договорились!



Причем платформа FHIR учитывает, что информацией FHIR можно обмениваться с помощью форматов сообщений XML, JSON и RDF. Это говорит о признании важности отделения предметной информации от используемых для ее передачи форматов.

Еще одним важным принципом работы масштабируемых систем является соглашение о том, как мы это сделаем. В данном случае все реализуется с помощью гипермедиа. Средства гипермедиа, в частности ссылки и формы, применяются для выражения метаданных среды выполнения, необходимых для решения задачи. Сами эти средства являются структурированными объектами, которые сервисы и клиенты утверждают заранее. Тем не менее содержимое этого средства — значения свойств метаданных — предоставляется только в среде выполнения. Сюда входят необходимые для конкретного действия свойства данных, метод протокола, а также целевой URL. Все они определяются во время выполнения. По сути, средства гипермедиа — это еще один уровень разделения. В действительности формат сообщений JSON-LD (<https://json-ld.org/>) для выражения гипермедийной информации внутри сообщения опирается на отдельный словарь (Hydra, <https://oreil.ly/W32Sr>). В этой главе есть несколько рецептов, которые учитывают важность гипермедиа для RWA. В следующих главах также есть ряд специфичных рецептов на основе гипермедиа, отражающих конкретные паттерны реализации.

Наконец, вы встретите рецепты, ориентированные на успешное взаимодействие по HTTP. Они включают такие аспекты, как сетевое обещание (безопасность и идиempотентность) и реалии, связанные с поддержкой M2M-взаимодействий по ненадежным соединениям на больших расстояниях. К счастью, HTTP прекрасно подходит для урегулирования нарушений соединения — он проектировался и создавался, когда большинство HTTP-взаимодействий происходили по относительно медленным телефонным линиям. Тем не менее, для того чтобы HTTP работал действительно хорошо, клиенты и серверы должны договориться о правильном использовании метаданных сообщений, например заголовков HTTP, а также утвердить способ реагирования на сетевые ошибки.

Итак, эта глава содержит рецепты, учитывающие эффективность разделения уровней коммуникации, роль гипермедиа в согласовании того, *как все должно выполняться*, и некоторые общие принципы обеспечения устойчивого взаимодействия в случае ошибок.

3.1. Обеспечение интероперабельности с помощью зарегистрированных медиатипов

Ключом для организации стабильной и надежной системы будет обеспечение долгосрочной интероперабельности между сервисами. Это означает, что сервисы, созданные много лет назад, смогут успешно обмениваться сообщениями с сервисами, которые будут созданы в дальнейшем.

Задача

Как проектировать сервисы, которые сохраняют высокую степень интероперабельности в отдаленном будущем.

Решение

Наиболее эффективный способ обеспечить высокий уровень долгосрочной интероперабельности между сервисами — установить стабильные правила для обмена информацией. Лучшим способом реализовать это в сети будет выбор и документирование поддержки одного или нескольких медиатипов с открытым исходным кодом для обмена данными. Например, медиатип HTTP предлагает строго типизированный формат, к которому браузеры могут привязываться без необходимости понимать *содержимое* HTML-документа. По факту содержимое документа может со временем меняться — могут добавляться абзацы, ссылки, формы, перемещаться текст и т. д., но браузеру обновление не потребуется. Использование HTML обеспечивает совместимость в будущем.

Хороший источник долгосрочных стабильных медиатипов — Администрация адресного пространства Интернета (Internet Assigned Numbers Authority, IANA <https://oreil.ly/rMOWs>). Приемлемыми кандидатами для долгосрочной поддержки RWA являются неструктурированные медиатипы вроде XML и JSON, а также структурированные HTML, Collection + JSON, UBER, HAL и SIREN. Список допустимых медиатипов приводится в приложении В.



Более подробно различие между структурированными и неструктурированными медиатипами описывается в рецепте 3.2.

Обсуждение

При создании сервисов нужно документально фиксировать, какие зарегистрированные медиатипы (registered media types, RMT) ваш сервис поддерживает. Желательно реализовывать поддержку более чем одного RMT, а также давать потребителям возможность определять, какие RMT ваш сервис поддерживает, и указывать предпочтительный для использования при обмене сообщениями (см. рецепт 4.6).



Когда я запускаю сервис, то почти всегда добавляю HTML в качестве одного из установленных форматов сообщений. Он существует уже более 30 лет, позволяет в качестве клиента для сервиса использовать любой популярный браузер (хорошо для тестирования), к тому же есть много парсеров и прочих HTML-инструментов, упрощающих для потребителей процесс обмена данными с сервисом.

Вот еще несколько рекомендаций для выбора медиатипов: они должны поддерживать применение в сообщениях средств гипермедиа (см. рецепт 3.5) и пользовательские расширения, например возможность безопасно добавлять в формат новые функции, не нарушая существующие приложения. Второе окажется кстати, когда вы будете поддерживать непопулярный формат и вам потребуется внести изменения для продления жизни сервиса.

Можно создать собственный медиатип для сервисов. Такой подход сработает, если:

- у вас небольшой круг потребителей API сервиса (например, внутри собственной компании);
- ваш круг потребителей API очень широк (например, Google, Facebook, Amazon и т. д.);
- ваши сервисы являются лидерами в вертикали (например, управление документами, финансовые услуги, здравоохранение и т. п.).

Если ваша ситуация не вписывается в одну из этих категорий, советую *не* создавать собственный медиатип.

РЕШИЛИ СОЗДАТЬ СОБСТВЕННЫЙ МЕДИАТИП?

Если вы решите создать собственный медиатип, то к этому нужно подходить так, будто он станет популярным публичным форматом. Это означает его документирование и подобающую регистрацию (<https://oreil.ly/8u7a8>). Вам также нужно быть готовым к поддержке любого окружения, которое выстроится вокруг него, включая образцы приложений и прочие инструменты. Наконец, нужно учитывать, что все это потребует сделать длительное время. Кроме того, помните, что некоторые люди могут создавать собственные сервисы, используя ваш формат, и нужно будет обеспечить для них надежную долгосрочную поддержку. За многие годы я создал около десятка медиатипов. Значительное внимание привлекли немногие, но я по-прежнему стараюсь ответственно относиться ко всем ним.

Читайте также

- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.
- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.6. Управление форматами представления в среде выполнения.

3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов

Фундаментальным элементом проектирования RWA является реализация поддержки будущей совместимости. То есть нужно сделать так, чтобы сервисы, написанные в прошлом, продолжали взаимодействовать с сервисами в отдаленном будущем. Кроме того, это означает проектирование взаимодействий таким образом,

чтобы будущие изменения уже развернутых сервисов не нарушали работу существующих сервисов или клиентов.

Задача

Как проектировать М2М-взаимодействия, которые будут поддерживать изменения в уже развернутых сервисах, не мешая работать существующим потребителям?

Решение

Чтобы добиться возможности безвредного внесения изменений в существующие сервисы, для передачи информации следует использовать *структурированные медиатипы* (structured media types, SMT). SMT позволяют генерировать стабильное, не вызывающее нарушений сообщение даже в случае изменения *содержимого* этого сообщения (например, свойства записи, списка действий и т. д.). Ключом здесь является выстраивание взаимодействий с помощью сообщения, которое будет совместно использоваться машинами, с сохранением одной и той же структуры, даже когда передаваемые им данные изменяются. Подробности описаны в примере.

Структурированный медиатип предоставляет строго типизированный формат, который не меняется в зависимости от выражаемых данных. Он противоположен неструктурированным форматам сообщений вроде XML и JSON. Подробности описаны в примере.

Желательно использовать во взаимодействиях хорошо известные медиатипы. Удачными кандидатами будут те, которые зарегистрированы посредством IANA (<https://oreil.ly/7FTks>). Хорошим примером SMT является HTML. Два других допустимых, повсеместно применяемых формата SMT — это Collection + JSON и UBER. Более развернутый список медиатипов приведен в разделе «Приемлемые зарегистрированные медиатипы для RESTful Web API» приложения В.

Пример

SMT предоставляет строго типизированный формат, который не изменяется в зависимости от выражаемых данных. Вот пример простого сообщения, заданного в виде HTML:

```
<ul name="Person">
  <li name="givenName">Marti</li>
  <li name="familyName">Contardi</li>
</ul>
...
<ul name="Person">
  <li name="givenName">Marti</li>
  <li name="familyName">Contardi</li>
  <li name="emailAddress">mcontardi@example.org</li>
</ul>
```

Структуру этого сообщения можно обозначить (в очень упрощенном виде) так: один или более элементов *li*, заключенных в один элемент *ul*. Обратите внимание, что внесение дополнительного содержания, например email-адреса, меняет не структуру сообщения (можно использовать тот же валидатор сообщений), а только его *содержимое*.

Сопоставьте предыдущий пример со следующими объектами JSON:

```
{ "Person" : {  
  "givenName": "Marti",  
  "familyName": "Contardi"  
}  
}  
...  
{ "Person" : {  
  "givenName": "Marti",  
  "familyName": "Contardi",  
  "emailAddress": "mcontardi@example.org",  
}  
}
```

Структуру JSON можно выразить так: объект JSON с двумя ключами, *givenName* и *familyName*. Добавление в сообщение адреса email приведет к изменению *структуры* сообщения JSON (например, потребует новой схемы документа JSON), а также его *содержимого*.

Обсуждение

Важным элементом этого рецепта выступает сохранение слабой связи *содержимого* сообщения с его *структурой*. Это позволяет приложениям-потребителям проще и согласованнее проверять входящие сообщения (привязываться к их типу), даже когда их содержимое со временем меняется.

Иначе взглянуть на это решение можно, предположив, что в качестве первого задания потребителям нужно убедиться в правильности структуры получаемых сообщений — в том, что она соответствует базовым правилам построения сообщений. Причем это задание отличается от задания убедиться в *валидности* сообщения, то есть в том, что его *содержимое* соответствует установленным правилам представления информации, например: «Все сообщения *Person* ДОЛЖНЫ включать свойства *givenName*, *familyName* и *emailAddress*».

Для обеспечения будущей совместимости первым делом нужно сделать так, чтобы сообщения, направленные на согласование последующего взаимодействия, сохраняли правильную форму даже при изменении правил относительно того, что должно включать в себя валидное сообщение.

Читайте также

- Рецепт 3.3. Предоставление информации о предметной области через публикуемые словари.
- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.6. Управление форматами представления в среде выполнения.

3.3. Предоставление информации о предметной области через публикуемые словари

Сервисы с грамотно управляемыми словарями более понятны и, следовательно, становятся более популярными. Аналогично необходимости поддерживать зарегистрированные структурированные медиатипы для обеспечения долгосрочной совместимости и надежности вашего сервиса вам нужно поддерживать стабильные и согласованные внутренние детали, чтобы обеспечить продолжительное взаимодействие между производителями и потребителями сервисов.

Задача

Как обеспечить, чтобы имена свойств данных сервиса были понятны другим сервисам — даже тем, которые создавали не вы?

Решение

Для того чтобы свойства данных, используемые вашим сервисом для обмена информацией, были понятны большинству других сервисов, даже тех, которые создавали не вы, вам нужно во внешнем интерфейсе (API) применять хорошо задокументированные и широко известные имена свойств данных. Эти имена должны быть определены в опубликованных словарях. Если это не так, вам следует опубликовать свой словарь данных, чтобы создатели аналогичных сервисов могли найти и применять те же самые термины.

Хорошим источником общеупотребительных словарей для веб-среды является ресурс *Schema.org* (<https://schema.org>). На нем есть сотни четко определенных терминов, применимых к широкому списку случаев. При этом ресурс продолжает

пополняться под чутким контролем. Есть и другие хорошие источники словарей, такие как *Microformats.org* (<https://microformats.org/>) и *Dublin Core* (<https://oreil.ly/aFW7i>).



Небольшое примечание. Некоторые словари, особенно отраслевые, не имеют открытого исходного кода — например, вам необходимо платить за доступ и участие. Кроме того, некоторые инициативы по созданию словарей, даже самые открытые, не только нацелены на создание одного общего простого словаря, но и включают рекомендации и ограничения для архитектуры, платформ и даже SDK.

Для получения долгосрочной поддержки интероперабельности лучше всего обеспечить, чтобы любые используемые вами термины словаря не были связаны с какими-либо другими программными или аппаратными зависимостями.

На момент написания книги существует несколько отраслевых публичных словарей, например PSD2 (<https://oreil.ly/1BXOi>) для платежей, FHIR (<https://oreil.ly/AMxkP>) для здравоохранения и ACORD (<https://oreil.ly/lfSCE>) для страхования.

Запуская сервис, вам следует сопроводить его документом, в котором будут указаны все потребляемые и отправляемые им термины словаря, а также ссылки на их определения. Подробнее о составлении и публикации словарей читайте в рецепте 3.4.

Пример

По возможности следует использовать во внешнем API вашего сервиса хорошо известные термины, даже когда во внутренней системе хранения применяются локальные или специфичные для компании термины.

Например, вот запись *person*, которая отражает термины, применяемые в типичной компании США:

```
{ "collection" : {
  "links": [
    { "rel": "self", "href": "http://api.example.org/persons" }
  ],
  "items" : [
    {
      "href": "http://api.example.org/persons/q1w2e3r4",
      "data" : [
        { "name": "fname", "value": "Dana" },
        { "name": "lname", "value": "Doe" },
        { "name": "ph", "value": "123-456-7890" }
      ]
    }
  ]
}
```

А вот та же запись, которая была изменена включением терминов, доступных в словаре Schema.org:

```
{ "collection" : {
  "links": [
    { "rel": "self", "href": "http://api.example.org/persons" },
    { "rel": "profile", "href": "http://profiles.example.org/persons" }
  ],
  "items" : [
    {
      "href": "http://api.example.org/persons/q1w2e3r4",
      "data" : [
        { "name": "givenName", "value": "Dana" },
        { "name": "familyName", "value": "Doe" },
        { "name": "telephone", "value": "123-456-7890" }
      ]
    }
  ]
}
```

Обратите внимание на использование во втором примере семантической связи `profile`. Подробнее об этом сказано в рецепте 3.4.

Обсуждение

В основе этого рецепта лежит цель — обеспечить, чтобы выражения свойств данных не были тесно связаны с форматом сообщения. По сути, это обратная сторона рецепта 3.2. Использование хорошо известных слабосвязанных словарей также является прекрасным способом защитить ваши сервисы от влияния будущих изменений в любых внутренних моделях данных.



Дополнительные рецепты по обработке данных для RWA описаны в главе 6.

Ограничение внешних интерфейсов использованием только хорошо известных и грамотно задокументированных имен свойств — один из лучших приемов для обеспечения интероперабельности вашего сервиса. Вместе с публикацией семантических профилей (см. рецепт 3.4) этот подход составляет основу управления масштабной информационной системой. Тем не менее проделать все это не так уж легко. Здесь требуются внимание к деталям, тщательное документирование и постоянная поддержка. Команда, которая управляет словарем сообщества и продвигает его, делает важную и полезную работу.

Одной из проблем этого рецепта является то, что *публичный* словарь для ваших сервисов вряд ли будет совпадать с *частным* словарем. Этот частный набор имен обычно привязан к старым внутренним практикам, возможно, принципам проектирования единичных команд или даже решениям, обусловленным готовым

коммерческим (commercial off-the-shelf, COTS) ПО, приобретенным давным-давно. Чтобы решить эту проблему, сервисам нужно реализовывать так называемый слой защиты от разрушения (anti-corruption layer, <https://oreil.ly/pQkt2>). Нет необходимости изменять существующую модель хранилища данных или сервисы.

Важно задокументировать ваш словарь сервиса и поделиться им. Стоит опубликовать список всех «магических» строк (см. раздел «Магические строки Ричардсона» главы 1) в одном месте наряду с кратким описанием и по возможности ссылкой в виде URL-адреса на источник этого описания. Я для этого использую формат Application-Level Profile Semantics (ALPS) (см. рецепт 3.4).

Вот пример документа словаря ALPS:

```
{ "alps" : {
  "descriptor": [
    { "id": "givenName", "def": "https://schema.org/givenName",
      "title": "Given name. In the U.S., the first name of a Person.",
      "tag": "ontology" },
    { "id": "familyName", "def": "https://schema.org/givenName",
      "title": "Family name. In the U.S., the last name of a Person.",
      "tag": "ontology" },
    { "id": "telephone", "def": "https://schema.org/telephone",
      "title": "The telephone number.",
      "tag": "ontology" },
    { "id": "country", "def": "http://microformats.org/wiki/hcard#country-name",
      "title": "Name of the country associated with this person.",
      "tag": "ontology" }
  ]
}
```

При создании словарей RWA желательно указывать один предпочтительный источник определений терминов вместе с одним или несколькими резервными вариантами. Например, ваше руководство по применению словаря может выглядеть так:

«По возможности используйте термины с сайта Schema.org. Если на нем подходящих терминов нет, ищите их на Microformats.org, а затем на Dublin Core. Наконец, если приемлемые термины не удастся найти ни на одном из этих ресурсов, создайте новый термин в репозитории словаря вашей компании по адресу [URL]».

Дополнительные примечания.

Смешивание терминов

Вполне допустимо смешивать термины из разных словарей в одном наборе. В предыдущем примере видно, что я включил три термина со Schema.org и один с Microformats.org.

Ограничение использования синонимов

Как и в реальной жизни, здесь может существовать несколько терминов с одинаковым значением, например `tel` (с `Microformats.org`) и `telephone` (со `Schema.org`). По возможности избегайте синонимов, применяя один термин для всех внешних случаев использования.

Публикация словарей

Процесс публикации и нахождения этих документов в сети описан в рецепте 3.4.

Читайте также

- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.
- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 4.5. Согласование поддержки профилей во время выполнения.

3.4. Описание предметной области с помощью семантических профилей

Помимо отделения словаря свойств данных от структуры сообщений важно приложить усилия к определению передачи этих свойств. Это и является описанием предметной области. Понятие предметной области используется в играх и интерактивном художественном творчестве для установки своего рода направляющих для набора необходимых действий. Кроме того, предметные области в некотором смысле определяют правила игры, а RWA на эти правила опираются.

Задача

Как предоставить подробное описание всех поддерживаемых вашими сервисами свойств данных, объектов и действий так, чтобы его можно было использовать и при проектировании, и при выполнении?

Решение

Чтобы разработчики могли быстро и точно понимать процессы обмена данными и действия, поддерживаемые вашим сервисом, можете опубликовать документ семантического профиля (*semantic profile document, SPD*). SPD содержат полный список всех свойств данных, объектов и действий, которые поддерживает сервис.

Однако семантические профили *не* являются документами определений API вроде OpenAPI, WSDL, AsyncAPI и пр. Более полный список форматов определения API приводится в разделе «Форматы определения API» приложения В.

SPD обычно включают все три элемента информационной архитектуры: онтологию, таксономию и хореографию.



Описание этих трех столпов информационной архитектуры приводится в разделе «Важность словарей» главы 1.

Два распространенных формата SPD — Dublin Core Application Profiles (DCAP) и Application-Level Profile Semantics (ALPS). Поскольку я соавтор ALPS, во всех приводимых здесь примерах используется этот формат. Более подробный список есть в разделе «Форматы документов семантических профилей» приложения В.

Пример

Вот пример документа семантического профиля в формате ALPS. Заметьте, что в нем присутствуют три столпа информационной архитектуры по Морвиллу — онтология, таксономия и хореография. Подробно они описаны в разделе «Важность словарей» главы 1:

```
{ "$schema": "https://alps-io.github.io/schemas/alps.json",
  "alps" : {
    "title": "Person Semantic Profile Document",
    "doc": {"value":
      "Simple SPD example for http://webapicookbook.com[Web API Cookbook]."},
    "descriptor": [
      {"id": "href", "def": "https://schema.org/url",
        "tag": "ontology"},
      {"id": "identifier", "def": "https://schema.org/identifier",
        "tag": "ontology"},
      {"id": "givenName", "def": "https://schema.org/givenName",
        "tag": "ontology"},
      {"id": "familyName", "def": "https://schema.org/familyName",
        "tag": "ontology"},
      {"id": "telephone", "def": "https://schema.org/telephone",
        "tag": "ontology"},

      {"id": "Person", "tag": "taxonomy",
        "descriptor": [
          {"href": "#href"},
```



```

        {"href": "#identifier"},
        {"href": "#givenName"},
        {"href": "#familyName"},
        {"href": "#telephone"}
    ]
},
{"id": "Home", "tag": "taxonomy",
 "descriptor": [
    {"href": "#goList"},
    {"href": "#goHome"}
 ]
},

{"id": "List", "tag": "taxonomy",
 "descriptor": [
    {"href": "#Person"},
    {"href": "#goFilter"},
    {"href": "#goItem"},
    {"href": "#doCreate"},
    {"href": "#goList"},
    {"href": "#goHome"}
 ]
},
{"id": "Item", "tag": "taxonomy",
 "descriptor": [
    {"href": "#Person"},
    {"href": "#goFilter"},
    {"href": "#goItem"},
    {"href": "#doUpdate"},
    {"href": "#doRemove"},
    {"href": "#goList"},
    {"href": "#goHome"}
 ]
},
{"id": "goHome", "type": "safe", "tag": "choreography", "rt": "#Home"},
{"id": "goList", "type": "safe", "tag": "choreography", "rt": "#List"},
{"id": "goFilter", "type": "safe", "tag": "choreography", "rt": "#List"},
{"id": "goItem", "type": "safe", "tag": "choreography", "rt": "#Item"},
{"id": "doCreate", "type": "unsafe", "tag": "choreography", "rt": "#Item"},
{"id": "doUpdate", "type": "idempotent", "tag": "choreography", "rt": "#Item"},
{"id": "doRemove", "type": "idempotent", "tag": "choreography", "rt": "#Item"}
]
}
}

```

На рис. 3.2 показана схема рабочего потока (*хореография*) для файла `person-alps.json`. Полную отрисовку ALPS (файл ALPS, схему и документацию) этого семантического профиля можно найти на сайте книги (<http://webapicookbook.com/>).

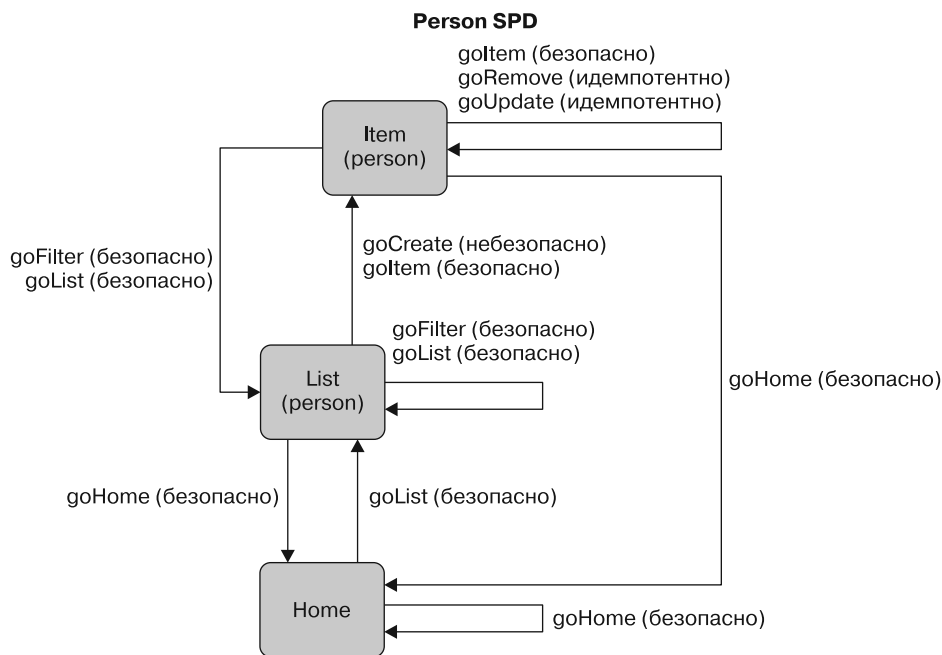


Рис. 3.2. Рабочий поток Person в формате ALPS

Обсуждение

Использование SPD — важная часть создания удобных RWA. Этот подход довольно нов и пока не особо распространен. Несмотря на то что ALPS и DCAP существуют уже около десяти лет, для семантических профилей создано не так уж много инструментов и письменных руководств. Хотя мой опыт работы с ALPS говорит о том, что вскоре мы увидим больше сопутствующих семантическим профилям материалов, даже если это будет что-то новое, а не ALPS и DCAP.

SPD — это своеобразная машиночитаемая документация интерфейса. Эти документы создаются для описания основных элементов интерфейса — свойств базового уровня, составных объектов и действий. SPD не содержат деталей реализации, таких как темы MQTT, ресурсы HTTP, методы протоколов, коды возврата и т. п. Эти детали остаются для тех, чьей задачей является написание реального кода, который выполняется за интерфейсом, описываемым семантическими профилями.

Вот еще несколько моментов, которые стоит учитывать при использовании семантических профилей.

Предназначены для обширного использования, в том числе предварительного

Ценность одного семантического профиля тем выше, чем больше сервисов его применяют. Это означает, что вам следует создавать профили, удобные

для использования другими (см. приложение А). Стоит делать семантические профили более обобщенными. Чем более специализированными вы их делаете, тем меньше будет их аудитория.

Не используйте семантические профили в качестве определений API

Семантические профили — это *описания*, а не *определения*. Многие детали, которые нужно указывать в определении интерфейса, не относятся к семантическим профилям. Скажем, URL-адреса и паттерны URL нужно включать в определения API (например, OpenAPI и т. д.), но не в семантические профили.

Используйте семантические профили для описания информационной архитектуры

Желательно сопровождать элементы семантических профилей *тегами*, чтобы показать, какие из них описывают онтологию, какие — таксономию, а какие — хореографию (см. раздел «Пример» ранее в этом рецепте).

Включайте семантические профили во все ответы

Хорошей практикой будет возвращать URI вашего семантического профиля с каждым ответом протокола. Подробнее об этом рассказывается в главах 4 и 5.

Избегайте изменений в опубликованных семантических профилях

Поскольку клиенты и/или сервисы могут создавать логическую зависимость от опубликованного вами семантического профиля, желательно *не вносить* критических изменений в SPD после его релиза. Если это все же потребуется, лучше создать новый документ профиля по новому URI (например, `api.example.org/profiles/personV2`), сохранив онлайн существующий профиль без изменений.

Делайте семантические профили доступными из единой точки

Чтобы упростить для людей и машин поиск ваших семантических профилей, создайте в сети единую точку, где их легко найти. Это может быть набор самих документов профилей или страница с указателями (URL) на другие места, где они хранятся. Второй вариант хорошо подходит в случаях, когда вы не автор профиля, но хотите контролировать то, какие профили обычно используются.

Читайте также

- Рецепт 3.1. Обеспечение интероперабельности с помощью зарегистрированных медиатипов.
- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 4.6. Управление форматами представления в среде выполнения.
- Рецепт 4.9. Использование средств гипермедиа в ответе.

3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа

Использование в ответах встроенных средств гипермедиа для информирования потребителей API о доступных на данный момент действиях — это фундаментальный элемент RWA. Как говорилось в разделе «Позднее связывание по Алану Кею» главы 1, средства гипермедиа позволяют реализовывать *очень слабое связывание*, которое, в свою очередь, упрощает изменение сервисов в будущем.

Задача

Как продлить жизнь сервисов, создавая системы с поддержкой безопасных, не нарушающих рабочий поток изменений, а также возможности внесения в сервисы краткосрочных корректировок путем настройки пользователем обмена данными в среде выполнения?

Решение

Лучший способ реализации поддержки краткосрочных и долгосрочных изменений сервисов продуктов — использование встроенных аффордансов гипермедиа для обозначения деталей зависящих от контекста процессов обмена данными в среде выполнения. Это означает, что нужно задействовать такие форматы обмена сообщениями, которые поддерживают встроенные средства гипермедиа (см. рецепты 3.1 и 3.2).

Пример

HTML предлагает отличный вариант встроенных средств гипермедиа для показа процессов обмена данными в среде выполнения. Вот пример:

```
<html>
  <head>
    <title>Create Person</title>
    <link rel="profile" href="http://api.example.org/profiles/person" />
    <style>
      input {display:block;}
    </style>
  </head>
  <body>
    <h1>Create Person</h1>
    <form name="doCreate" action="http://api.example.org/person/"
      method="post" enctype="application/x-www-form-urlencoded">
      <fieldset>
        <hidden name="identifier" value="q1w2e3r4" />
        <input name="givenName" placeholder="givenName" required/>
        <input name="familyName" placeholder="familyName" required/>
```

```

        <input name="telephone" placeholder="telephone" pattern="[0-9]{10}"/>
        <input type="submit" />
        <input type="reset" />
        <input type="button" value="Cancel" />
    </fieldset>
</form>
</body>
</html>

```

Эта форма показывает одно предустановленное входное значение (*identifier*) и три дополнительных, два из которых обязательные (*givenName*, *familyName*), а одно *должно* передавать валидатор *pattern* (*telephone*). Кроме того, в этой форме указаны три возможных действия (*submit*, *reset* и *cancel*), а также детали, описывающие URL, HTTP-метод и метаданные для действия *submit*. Еще лучше то, что HTML-совместимый клиент, например браузер, может поддерживать все эти действия, не нуждаясь в индивидуально настроенном программном коде.

Вот простой пример в формате Collection + JSON:

```

{ "collection" :
  {
    "version" : "1.0",
    "href" : "http://api.example.org/person/",

    "links": [
      { "rel": "self", "href": "http://api.xample.org/person/doCreate"},
      { "rel": "reset", "href": "http://api.example.org/person/doCreate?reset"},
      { "rel": "cancel", "href": "http://api.example.org./person"}
    ],
    "template" : {
      "data" : [
        { "name" : "identifer", "value": "q1w2e3r4"},
        { "name" : "givenName", "value" : "", "required": true},
        { "name" : "familyName", "value" : "", "required": true},
        { "name" : "telephone", "value" : "", "regex": "[0-9]{10}" }
      ]
    }
  }
}

```

Опять же Collection + JSON-совместимое клиентское приложение сможет обеспечить применение всех входных правил, описанных в ответе гипермедиа, не требуя дополнительного программирования.

В обоих случаях *значение* входных элементов и свойств метаданных потребителю API понимать не нужно — необходимо лишь обеспечить их заполнение. Так что количество входных данных, целевой URL HTTP-метода POST и даже некоторые правила могут со временем меняться, не мешая клиентскому приложению продолжать устанавливать ограничения. Как вариант, может измениться правило валидации значения *telephone* — например, оно может зависеть от контекста среды выполнения приложения.

Обсуждение

Применение сообщений со встроенными средствами гипермедиа — пожалуй, самый важный шаг при создании RESTful Web API. Существует несколько структурированных медиатипов (см. раздел «Приемлемые зарегистрированные медиатипы для RESTful Web API» приложения В), и их поддержка с помощью библиотеки парсинга является однотипным видом расходов, которые окупаются при каждом использовании этой библиотеки.



Углубленный разбор гипермедийных клиентских приложений вы найдете в другой моей книге, *RESTful Web Clients*.

Несмотря на ценность встроенных средств гипермедиа, их применение несет определенные издержки. Во-первых, это ограничение в *процессе проектирования*. И потребители API, и их производители должны согласиться на использование гипермедиа. Это своеобразная стоимость входа при создании RWA. И хотя это ничем не отличается от «вы должны задействовать HTML, CSS и JavaScript в ответах браузерам», я все равно встречаю многих архитекторов и разработчиков, которые не жалуют идею применения ответов с гипермедиа. Когда вы решите создать RWA, вам порой потребуется помочь другим преодолеть эту преграду. Подспорьем в этом для вас выступит материал части I.

Выбор «правильного» структурированного медиатипа может легко превратиться в проблему. Существует много их вариантов, и некоторые специалисты оказываются в затруднении, так как выбрать можно только один. По правде говоря, вы можете использовать возможности HTTP (см. рецепт 5.6) для поддержки нескольких форматов ответов и выбора лучшего варианта в среде выполнения. Этой технологии уже 30 лет, так что есть много примеров и сопутствующих фрагментов кода, которые помогут вам с этим разобраться. Вы даже можете изначально выбрать один формат (обычно HTML), а потом постепенно добавлять другие, не нарушая работу приложений.

Выражение состояния предметной области с помощью гипермедиа типов также порой создает сложности. Разработчикам нужна возможность преобразовывать внутренние правила объектов и моделей в допустимые средства гипермедиа — *forms*, *inputs*, *links* и т. д. Этот навык трансформации нужно будет выработать.

Работа по отправке и получению форматов гипермедиа в среде выполнения — тоже типичная задача. Подробности о том, как упростить этот процесс с помощью HTTP, читайте в рецепте 5.6. Кроме того, для написания потребителей API, способных применять для навигации ответы с гипермедиа элементами, также потребуется навык. Этой теме посвящены многие рецепты из главы 4.

Что еще стоит учитывать при использовании встроенных форматов гипермедиа?

Поддержка изменения контекста

Вы можете изменить детали гипермедиа в ответе, исходя из пользовательского контекста и/или состояния сервера. Например, для авторизованного администратора в форме могут присутствовать пять полей, а для анонимных пользователей — всего три.

Возможность изменения расположения сервиса

При использовании средств гипермедиа для описания возможных действий вы можете столкнуться с концепцией позднего связывания по Алану Кею. Клиенты гипермедиа спроектированы так, чтобы искать в сообщении идентификаторы («Где находится элемент управления `update-customer?`»), понимать эти элементы и действовать в соответствии с обнаруженными в них метаданными. Все это может происходить в среде выполнения, а не при проектировании или сборке. Это означает, что метаданные действия могут измениться, когда сервис находится в промышленной среде. Например, вы можете изменить целевой URL действия с локальной конечной точки внутри текущего сервиса на внешнюю конечную точку на другой машине, не нарушая работы клиентского приложения.

Адаптация к изменению рабочего потока

Использование гипермедиа дает возможность вносить изменения в многоэтапные процессы рабочих потоков. Ответы сервисов могут возвращать один или несколько шагов для выполнения и ожидать, что клиентское приложение выполнит действия в соответствии с каждым. В изначальном релизе могут присутствовать три шага: создание учетной записи, добавление связанной компании и добавление нового контакта. Позднее эти шаги могут быть сведены к двум: создание учетной записи и добавление связанной компании и контакта. До тех пор, пока клиентское приложение следует переданным посредством гипермедиа инструкциям (без жесткого встраивания этих шагов в код), подобные изменения не нарушат его работу. Рецепты реализации рабочих потоков на основе гипермедиа описаны в главе 7.

Читайте также

- Рецепт 3.1. Обеспечение интероперабельности с помощью зарегистрированных медиатипов.
- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 4.11. Проверка свойств данных в среде выполнения.
- Рецепт 5.6. Поддержка согласования HTTP-содержимого.

3.6. Проектирование последовательных операций записи с помощью идемпотентных действий

Особенно сложной проблемой в ходе работы с внешними сервисами в контексте межмашинного взаимодействия по HTTP является провал POST. В качестве примера можно привести случай, когда клиентское приложение отправляет POST-запрос сервису банковских счетов для списания 50 кредитов с конкретного счета, но *ответа на этот запрос так и не получает*. Может, запрос не прибыл на сервер? Или он прибыл, был обработан, но клиент не получил ответ 200 OK? Вопрос в том, должен ли клиент повторить свой запрос.

У этой проблемы есть простое решение, требующее использовать вместо HTTP-метода POST метод PUT.

Задача

Как спроектировать выполняемое по HTTP действие записи, которое исключит вероятность двойного размещения данных? Как понять, будет ли безопасной повторная отправка HTTP-запроса, если клиентское приложение не получит ответа на первый?

Решение

Все действия записи данных должны отправляться с помощью HTTP-метода PUT, а не POST. Действия PUT можно легко выстроить так, чтобы предотвратить двойное размещение и обеспечить безопасный повтор этого действия, если клиентское приложение не получает ответа от сервера.

Пример

Под влиянием паттерна базы данных CRUD (create, read, update, delete — создание, чтение, обновление, удаление) в течение многих лет действие **create** отображалось в HTTP-действие POST, а действие **update** — в HTTP-метод PUT. Тем не менее запись данных на сервер с помощью POST создает проблему, поскольку это действие нельзя постоянно повторять. Иными словами, использование POST не является *идемпотентным*. То есть POST не возвращает один и тот же результат при повторении для одного и того же ресурса. При этом по своей сути операция PUT является идемпотентной — она обеспечивает одинаковые результаты даже при многократном повторении одного действия.

Чтобы избежать случайного создания двух ресурсов, когда нужен только один, стоит всегда использовать для записи данных на другую машину в сети метод PUT:

```
**** REQUEST
PUT /person/q1w2e3 HTTP/2.0
Host: api.example.org
Content-Type: application/x-www-form-urlencoded
If-None-Match: *
```



```
givenName=Mace&familyName=Morris
```

```
**** RESPONSE
HTTP/2.0 201 CREATED
Content-Type: application/vnd.collection+json
ETag: "p0o9i8u7y6t5r4e3w2q1"
...
```

Имейте в виду, что, хотя это и не распространено, правильным будет, если ответ на запрос PUT будет иметь вид **201 CREATED**, если по этому адресу нет ресурса. Но как сообщить сервису, что вы планируете создать новый ресурс, а не обновить существующий? Для этого нужно применить **If-None-Match**. В предыдущем примере заголовок **If-None-Match: *** гласит: «Создать новый ресурс по указанному URL, если там еще нет ресурса».

Если бы я хотел задействовать PUT для замены существующего ресурса, то использовал бы следующее взаимодействие по HTTP:

```
**** REQUEST
PUT /person/q1w2e3 HTTP/2.0
Host: api.example.org
Content-Type: application/x-www-form-urlencoded
If-Match: "p0o9i8u7y6t5r4e3w2q1"
```

```
givenName=Mace&familyName=Morris
```

```
**** RESPONSE
HTTP/2.0 200 OK
Content-Type: application/vnd.collection+json
ETag: "o9i8u7y6t5r4e3w2q1p0"
...
```

Здесь HTTP-запрос помечен как **If-Match**: — заголовок, который содержит тег сущности (или ETag), указывающий точную версию ресурса, которую вы хотите обновить по адресу `/person/q1w2e3`. Если ETag по этому URL не соответствует указанному в запросе (например, ресурса не существует или его недавно уже обновили), тогда метод PUT будет отклонен с ответом **HTTP 412 Precondition Failed**.

Обсуждение

Использование паттерна PUT-Create помогает понять, когда можно повторить операцию записи по HTTP. Такой подход работает, потому что в соответствии с дизайном PUT можно повторять и это не вызовет нежелательных побочных эффектов. POST же не обеспечивает такую возможность. Было немало попыток сделать POST повторяемым. Двумя примерами являются протоколы HTTPLR (<https://oreil.ly/KJh9v>) Билла де Оры и POE (<https://oreil.ly/1wFmz>) Марка Ноттингема. Ни один из них так и не стал популярным.

Применение метода PUT для создания новых ресурсов требует чуть больше предварительной работы как для клиентов, так и для серверов в основном потому, что

все зависит от правильного использования HTTP-заголовков `If-None-Match`, `If-Match` и `Etag`. Мой опыт говорит, что лучше встраивать этот рецепт в код серверного и клиентского приложений. Обычно у меня в локальном коде есть специальные методы `createResource(...)` и `updateResource(...)` или аналогичные, которые знают, как создать правильный запрос PUT (включая правильные заголовки) и как отвечать, когда что-то идет не так. Удобство этих методов в том, что они снижают излишние затраты на применение подхода PUT-Create и обеспечивают его согласованную реализацию для клиента и сервера.



Связанные с этой темой рецепты создания и обновления ресурсов описаны в главах 4 и 5.

В текущий рецепт включен еще один, а именно возможность для клиентских приложений передавать идентификаторы ресурсов. При использовании POST обычно ожидается, что сервис создаст новый идентификатор ресурса — как правило, единообразный ID вроде `/person/1`, `/person/2` и т. д. Если же мы задействуем PUT, ожидается, что клиенты предоставят идентификатор ресурса, то есть: «Загрузите файл `my-todo-list.doc` в мое хранилище документов». Подробнее об этом читайте в главах 4 и 5.

Встраивание поддержки идемпотентных записей с помощью паттерна PUT-Create означает, что клиентское и серверное приложения будут более надежными и согласованными, даже если этого нельзя сказать об используемых вами сетевых соединениях.

Читайте также

- Рецепт 3.8. Проектирование с учетом возможности повторения действий.
- Рецепт 4.2. Программирование клиентов, понимающих HTTP.
- Рецепт 5.15. Повышение надежности с помощью идемпотентной операции создания.
- Рецепт 6.2. Делаем все изменения идемпотентными.

3.7. Обеспечение интероперабельности с помощью передачи состояния между сервисами

Важной частью создания коллекции интероперабельных сервисов в веб-среде является обеспечение возможности для каждого сервиса действовать независимо и участвовать в достижении общей цели или нахождении решения. Нам нужно сделать так, чтобы API наших сервисов не предполагали, что единственный допустимый вариант использования — это когда клиенты являются пленниками сервиса. Сервисы нужно рассматривать как часть целого, а не просто целое.

Задача

Как сделать так, чтобы интерфейсы наших сервисов работали, даже когда сервис является частью более крупного решения, которое было создано кем-то (клиентское приложение API) и интегрируется с другими API? Что нужно сделать, чтобы наши сервисы можно было легко интегрировать с другими сервисами, которых мы раньше не видели и которые не контролируем, с сохранением безопасности системы и целостности данных?

Решение

Бывает непросто сделать так, чтобы ваш сервис имел возможность взаимодействовать с другими, неизвестными сервисами. Проще и безопаснее всего добиться этого путем реализации API в виде набора отдельных операций без состояния. Они смогут получать входные данные, выполнять действия, возвращать результаты и забывать о произошедшем.

Вычисление значений — хороший пример простой передачи данных между сервисами без сохранения состояния. Представьте, например, валидатор почтового кода, который поддерживает передачу тела с полным адресом и, возможно, некоторыми идентификационными метаданными, возвращая соответствующий почтовый код. Все необходимые данные о состоянии передаются в паре «запрос/ответ».

Тем не менее будут случаи, когда вам нужно передать блок данных сервису, дать этому сервису какое-то время на их обработку, а затем экспортировать результаты. В таких случаях хорошим решением могут стать более явные операции импорта/экспорта.

Использование форм

Проще всего передавать состояние между сервисами путем отправки свойств данных через форму, предоставляемую сервисом, *получающим* эту информацию. Именно так в основном работают HTML и другие гипермедиаформаты. Но некоторые сервисы для обработки входящих данных могут требовать передачи дополнительных метаданных или не поддерживать нативные средства гипермедиа. В таких случаях нужно спроектировать в интерфейсе дополнительную поддержку для явной передачи данных о состоянии.

Использование операций импорта/экспорта

Следующим простым способом создания удобного для стороннего применения сервиса является поддержка действия, позволяющего передавать состояние от одного сервиса другому. Как правило, это означает предоставление операции, которая импортирует (`importState`) состояние откуда-то, и операции, которая экспортирует (`exportState`) состояние вашего сервиса куда-то.

Вместо того чтобы добавлять действия импорта/экспорта в интерфейс, можно встроить эту функциональность в выбранные существующие действия API.

Например, можно организовать получение входных данных через элемент FORM при создании новой учетной записи пользователя.

Предыдущие случаи — это примеры передачи состояния «по значению» — отправки точных значений состояния от одного сервиса другому в среде выполнения с помощью описаний FORM. Это самый простой способ взаимодействия с сервисами, которые вы не контролируете.

Предоставление ссылки

Также можно организовать передачу состояния «по ссылке», предоставив URL-адрес, указывающий на коллекцию данных, которые вы хотите передать. Для этого потребуется чуть бóльшая координация между сервисами, поскольку оба они должны заранее знать детали обмена данными через URL, а также формат, в котором эти общие данные хранятся. Подробнее читайте в рецепте 7.2.

Пример

Есть три способа обеспечения передачи состояния между сервисами:

- передача по значению с помощью встроенных существующих FORM;
- передача по значению с помощью специальных операций, например `importState` и `exportState`;
- передача по ссылке (через общий URL) с помощью специальных операций либо FORM.

Передача по значению с помощью форм

Передача состояния между сервисами посредством форм — это простейший и самый прямой способ решить задачу:

```
<form action="http://api.example.org/shopping/cart"
  method="post" name="cartCreate">
  <input name="cartId" value="q1w2e3r4t5y6u7i8" />
  <input name="cartName" value="Mike's Cart" />
</form>
```

Такое решение прекрасно работает, когда коллекция данных о состоянии невелика, а получающий сервис не требует, чтобы сеанс с состоянием существовал заранее.

Передача по значению с помощью специальных операций

Поддержку передачи состояний можно реализовать путем добавления в интерфейс вашего сервиса операций импорта и экспорта. Например, сервис может предлагать возможность управления корзиной для товаров. Наряду с типичными шагами (`create`, `addItem`, `removeItem` и `checkout`) можно добавить шаг, импортирующий существующую коллекцию из корзины, и шаг, экспортирующий ее.

Вот фрагмент ALPS, в котором показаны действия простого API для покупок с поддержкой операций импорта и экспорта:

```
{
  $schema: "https://alps-io.github.io/schemas/alps.json",
  alps: {
    version: "1.0",
    title: "Simple Shopping Cart",
    doc: {value: "A simple shopping cart service"},

    "descriptor" : [
      {
        "id" : "doCartImport",
        "type" : "idempotent",
        "rt" : "#cartCollection",
        "tag" : "choreography",
        "descriptor" : [
          "href" : "#cartCollection"
        ]
      },
      {
        "id" : "doCartExport",
        "type" : "idempotent",
        "rt" : "cartCollection",
        "tag" : "choreography"
      }
    ]
  }
}
```

Такое решение прекрасно работает при довольно большом и/или сложном наборе данных состояний для передачи, например, если коллекция содержит 15 элементов и метаданных. Плюс в том, что вы реализуете передачу в один шаг. Но есть и минус — нужно координировать эту передачу заранее, поскольку обе стороны должны утвердить имена свойств данных, содержимое и общую форму передаваемой коллекции.

Передача по ссылке

Передачу состояния между сервисами можно организовать также путем предоставления указателя на коллекции состояний, хранящихся в другом месте, доступ к которому есть у обеих сторон. Проще всего реализовать это предоставлением URL, указывающего на ресурс, который будет соответствовать ожидаемым или явно запрошенным получающей стороной медиатипу и формату словаря.

Это взаимодействие будет выглядеть так (в HTML):

```
<form action="http://api.example.org/users/q1w2e3r4"
  method="post" enctype="multipart/form-data">
  <input type="file" name="userData" accept="application/vnd.collection+json"/>
</form>
```

Данный пример описывает операцию загрузки, которую может реализовать целевой сервис. Ключом к успеху здесь выступает то, что обе стороны уже договорились об ожидаемом формате данных и содержимом словаря. Такие операции передачи можно документировать при публикации API, чтобы другие понимали, как интерфейс вашего сервиса предполагает отправлять информацию другим сервисам или получать ее от них.

Обсуждение

По возможности передавайте формы гипермедиа в ответах API, которые обрабатывают передачу состояния за один шаг. Это позволит другим сервисам действовать как клиентам API и выполнять передачу напрямую. Для этого не потребуется дополнительная координация между клиентами и серверами.

Кроме того, нужно свести оркестрацию между сервисами к минимуму. Например, лучше обрабатывать передачу состояний за один шаг, а не требовать от потребителей API перед выполнением необходимых действий сперва запустить сеанс или авторизоваться. Если необходима идентификация, пусть это взаимодействие будет результатом перенаправления от целевой передачи состояния:

```
**** REQUEST ****
POST /shopping/cartCreate HTTP/1.1
Host: api.example.org
Content-Type: application/x-www-form-urlencoded

cartId=q1w2e3r4&cartName=Mike '%s%20Cart

**** RESPONSE ****
HTTP/1.1 Unauthorized
WWW-Authenticate: Basic

**** REQUEST ****
POST /shopping/cartCreate HTTP/1.1
Host: api.example.org
Content-Type: application/x-www-form-urlencoded
Authorization: Basic q1i8w2o9e3p0r4u7t5...

cartId=q1w2e3r4&cartName=Mike '%s%20Cart

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Authorization: Basic q1i8w2o9e3p0r4u7t5...
Link: <http://docs.alps.io/shopping.json>; rel="profile"

{
  "alps" : {
    ...
  }
}
```

Если вы решите реализовать поддержку передачи состояния «по ссылке», то лучше использовать структурированные медиатипы вроде HAL, SIREN, Collection + JSON и т. д., как и формат сообщений, сопровождая их ссылками на словарь, например ALPS URI. Это повысит вероятность того, что загружаемые данные будут приняты, только когда получающий сервис подтвердит, что понял формат и словари.

РАБОТА С ОГРАНИЧЕНИЯМИ ПРИ СОВМЕСТНОМ ИСПОЛЬЗОВАНИИ РЕСУРСОВ РАЗНЫМИ ИСТОЧНИКАМИ

Имейте в виду, что HTML-браузеры не будут поддерживать отправку сообщений методом POST или загрузку документов из внешних доменов. Это связано с ограничениями совместного применения ресурсов разными источниками (Cross-Origin Resource Sharing, CORS) (<https://oreil.ly/AKyDh>), реализуемыми HTML-браузерами. Если вы создаете API, который будет поддерживать передачу состояний «по ссылке», допустим, загрузку файлов, то вам может потребоваться также отправлять заголовки (например, Access-Control-Allow-Origin и т. д.), чтобы потребители API могли успешно загружать свои документы.

Читайте также

- Рецепт 4.15. Сохранение состояния.
- Рецепт 6.8. Игнорирование неизвестных полей данных.
- Рецепт 7.2. Поддержка общего состояния в рабочих потоках.

3.8. Проектирование с учетом возможности повторения действий

В веб-среде велика вероятность того, что какой-то запрос не достигнет цели, окажется невалидным или просто исчезнет без ответа. Эта свойственная сети ненадежность означает, что нам нужно вести разработку с акцентом на выживаемость и по возможности преодолевать эти проблемы, чтобы создаваемые интерфейсы имели долгосрочную устойчивость. Данный рецепт опирается на принцип *идемпотентности*, и HTTP-запросы сделаны повторяемыми.

Задача

Иногда HTTP-запрос не выполняется из-за сетевых или иных временных проблем. Как сделать так, чтобы такие запросы можно было надежно повторить, не вызвав других проблем? Что такое идемпотентность, как она работает и как встроить поддержку повторяемости в интерфейсы сервисов?

Решение

HTTP изначально создавался с учетом вероятности того, что некоторые запросы могут не выполниться. Как правило, эти сбои являются временными проблемами сети и повтор запроса решает проблему. Повтор *безопасных* HTTP-запросов — тех, которые доступны только для чтения, — простая задача. А вот повтор таких, которые изменяют ресурсы, реализовать намного сложнее. Несколько популярных решений этой проблемы описаны в рецепте 7.16.

Сетевая идемпотентность

Есть два приема, которые вы можете применить для повышения вероятности успешного повтора HTTP-запросов. Так, можно использовать *идемпотентные* HTTP-методы (GET, PUT и DELETE), чтобы считывать, записывать и удалять ресурсы. Это не подразумевает применение метода POST, поскольку в него не заложена поддержка идемпотентных обновлений. Я для создания ресурсов по возможности задействую PUT (см. рецепт 5.15). Требуется чуть больше поработать, чтобы настроить PUT, зато это значительно повышает надежность сервиса.

Идемпотентность операций

А сами операции записи нужно построить таким образом, чтобы они поддерживали идемпотентность. Зачастую это означает, что *не стоит* использовать паттерны вроде инкрементирования (добавления единицы к каждому общему) или вычисления процентов (увеличения цены реализации на 5 %), а нужно вместо них применить подменные значения и валидаторы (если текущая цена 100, изменить ее на 105).

Опираясь на метод PUT для обновления ресурса и используя в телах обновлений замену вместо инкрементирования, вы можете повысить успешность повторяемости операций записи интерфейса вашего сервиса.

Пример

Важна повторяемость на уровне не только сети, но и сообщений. В этом разделе приводятся примеры для обоих случаев.

Повторяемость на уровне сети

Для поддержки повторяемости на сетевом уровне при проектировании операции создания и обновления ресурсов с помощью PUT мы опираемся на идемпотентность этого метода:

```
**** REQUEST ****
PUT /onboarding/q1w2e3r4
Host: api.example.org
```



```
Content-Type: application/x-www-form-urlencoded
Content-Length: NN
...

id=q1w2e3r4&givenName=Mork&familyName=Morkelson&...

**** RESPONSE ****
503 Service Unavailable
Content-Type: application/problem+json
Content-Length: XX
...
{
  "type": "https://example.com/probs/unavailable",
  "title": "Server Unavailable",
  "detail": "Can't reach target server",
  "instance": "/onboarding/q1w2e3r4",
  "status": 503
}
```

В этом примере PUT не выполнялся из-за проблемы с другим сервером на его пути. Поскольку этот метод идемпотентен, запрос можно безопасно повторить. Фактически этот запрос можно повторить, даже если вы не получите HTTP-ответа (редкий, но возможный исход).

Повторяемость сообщений

Даже когда взаимодействие на уровне сети идемпотентно, например, вы используете PUT вместо POST, можно обеспечить повторяемость HTTP-запроса, реализовав поддержку в сообщении функциональной идемпотентности.

Например, при обновлении всех цен в каталоге продуктов можете сделать это неидемпотентно:

```
**** REQUEST ****
PUT /catalog/priceUpdate
Host: api.example.org
Content-Type: application/x-www-form-urlencoded
Accept: application/vnd.siren+json
....

updatePercent=.05
```

В этом примере, когда запрос PUT получен, значение `updatePercent` используется для обновления 55 товаров в каталоге компании.

Но что произойдет, если обновление остановится на половине списка? Некоторые товары окажутся обновлены, а остальные — нет. А если мы не получим ответа на запрос? Был ли он успешно применен? Ко всем ли товарам? Безопасно ли повторить это действие?

Мы можем значительно повысить возможность повторения в этом примере, сделав его идемпотентным на уровне сообщений:

```
**** REQUEST ****
PUT /catalog/priceUpdate
Host: api.example.org
Content-Type:text/csv
Accept: application/vnd.siren+json
....

productId, currentPrice,newPrice
q1w2e3, 100,105
t5y6u7, 200,210
i8o9p0, 250,265
i8y6r4, 50,55
...
```

Обратите внимание на то, что это сообщение обновления включает свойства `productId`, `currentPrice` и `newPrice`. Теперь это действие можно безопасно повторить. Сервис может проверить, установлена ли цена конкретного `productId` на `currentPrice`, прежде чем применять значение `newPrice`. В противном случае операция пропустит обновление цены для этой записи и перейдет к следующей.



Еще одним возможным решением для этого примера будет простое включение значений `productId` и `newPrice`. Многократное выполнение процесса каждый раз по-прежнему будет приводить к получению корректных цен.

Для обеспечения возможности повтора сообщений важно опираться не только на HTTP-метод.

Обсуждение

Для проектирования повторяемых обновлений зачастую требуется заранее проделать дополнительную работу, чтобы лучше объяснить изменения, которые вы хотите применить к ресурсу или коллекции ресурсов. Эти усилия оправдают себя, едва лишь в ходе крупного обновления возникнет неожиданный сбой.

PUT ИЛИ POST?

Поскольку повторяемость так важна, в большинстве примеров книги для создания и обновления ресурсов вместо метода `POST` используется `PUT`. `POST` можно сделать идемпотентным, добавив соответствующий ключ, подобный тому, что предлагается для IETF (<https://oreil.ly/7dnC3>). Но я все равно обычно применяю `PUT`. Более подробно эта тема раскрывается в рецепте 3.6.

Читайте также

- Рецепт 3.6. Проектирование последовательных операций записи с помощью идемпотентных действий.
- Рецепт 5.15. Повышение надежности с помощью идемпотентной операции создания.
- Рецепт 6.2. Делаем все изменения идемпотентными.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.

3.9. Проектирование с учетом возможной отмены действий

Бывают случаи, когда после применения изменения к одному или нескольким ресурсам вам потребуется это изменение откатить. Текущий рецепт показывает, как можно реализовать поддержку отмены изменения в интерфейсах сервисов.

Задача

Что происходит, когда вам нужно откатить изменение в одном или нескольких ресурсах? Как спроектировать обновления так, чтобы они поддерживали возможность безопасной и легкой отмены изменений?

Решение

Есть два основных способа откатить обновление:

- поддержка локальной отмены (см. рецепт 7.17);
- отправка еще одного HTTP-запроса, отменяющего предыдущий.

Первый пример (поддержка локальной отмены) описан в рецепте 7.17, поэтому здесь я в его детали углубляться не стану. Достаточно сказать, что локальная отмена работает для случаев, когда нет других зависимых действий (например, вследствие обновления, которое вы хотите отменить, не были изменены никакие другие действия). Если же ваше обновление повлияло на другие записи в системе или является лишь одним из нескольких, которые могут быть применены к одному ресурсу, то реализовать поддержку отмены будет сложнее.

Отправка второго запроса

В случае, когда была изменена всего одна запись (например, ориентированный на данные ресурс обновил систему записи), обычно можно отправить другой HTTP-запрос, который отменит действие первого. Так, если вы отправили метод PUT для замены в записи свойства `givenName`, то обычно можете отправить

другой PUT для замены того же свойства прежним значением. Это предполагает, что вы *знаете* прежнее значение и вносите изменение безопасно (например, никто другой за это время не изменял эту запись).

Отправка специального запроса

Бывают случаи, когда простой повтор PUT не работает. Возможно, вы отправили метод DELETE для ресурса, а теперь хотите этот ресурс вернуть. Метода UNDELETE не существует. Вместо этого нужно восстановить существующую запись с помощью специальной команды. Безопаснее всего будет создать другую специальную операцию (`undoDelete`), использующую метод PUT и необходимый аргумент — обычно прежний URL-адрес вместе со значением для валидации вроде заголовка ETag. Это означает, что в интерфейс сервиса должна быть заложена поддержка такой возможности, включая сохранение удаленных (DELETE) ресурсов на случай, если их потребуется восстановить.

Пример

Для добавления в интерфейс поддержки отмены операций можно использовать подход с двумя PUT или специальной командой.

Откат с помощью двух PUT

Рассмотрим пример, в котором первое действие PUT отменяется с помощью второго:

```
**** REQUEST ****
GET /users/q1w2e3r4
Host: api.example.org
Accept: application/html

**** RESPONSE ****
200 OK
Content-Type: application/html
ETag: "w/p0o9i8u7y6t5r4"
...

<html>
...
<div class="user">
  <span class="givenName">Millie</span>
  <span class="familyName">Murcheson</span>
</div>
...
</html>
```

Вот первое обновление:

```
**** REQUEST ****
PUT /users/q1w2e3r4
```

```
Host: api.example.org
Content-Type: application/x-www-form-urlencoded
Accept: application/html
If-Match: "w/p0o9i8u7y6t5r4"
```

```
givenName=Molly&familyName=Murcheson
```

```
**** RESPONSE ****
```

```
200 OK
```

```
Content-Type: application/html
```

```
If-Match: "w/q2w3e4r5t6y7u8"
```

```
...
```

```
<html>
```

```
...
```

```
<div class="user">
```

```
  <span class="givenName">Molly</span>
```

```
  <span class="familyName">Murcheson</span>
```

```
</div>
```

```
...
```

```
</html>
```

И наконец, можно откатить изменение с помощью еще одного PUT:

```
**** REQUEST ****
```

```
PUT /users/q1w2e3r4
```

```
Host: api.example.org
```

```
Content-Type: application/x-www-form-urlencoded
```

```
Accept: application/html
```

```
If-Match: "w/q2w3e4r5t6y7u8"
```

```
givenName=Millie&familyName=Murcheson
```

```
**** RESPONSE ****
```

```
200 OK
```

```
Content-Type: application/html
```

```
ETag: "w/i9u8y7t6r5e4"
```

```
...
```

```
<html>
```

```
...
```

```
<div class="user">
```

```
  <span class="givenName">Millie</span>
```

```
  <span class="familyName">Murcheson</span>
```

```
</div>
```

```
...
```

```
</html>
```

Обратите внимание на применение ETag для версионирования ответа и заголовков If-Match, чтобы сделать PUT условным.

Использование специальной команды

В более сложных случаях для отката потребуется использовать особую команду.

Вот действие, которое удаляет ресурс из коллекции:

```
**** REQUEST ****
DELETE /users/q1w2e3r4
Host: api.example.org
Accept: application/html
If-Match "w/y6t5r4e3w2q1"
```

```
**** RESPONSE ****
204 No Content
...
```

А вот специальная команда для интерфейса, которая поддерживает откат предыдущего действия DELETE:

```
**** REQUEST ****
PUT /users/rollback?id=q1w2e3r4
Host: api.example.org
Accept: application/html
If-Match "w/y6t5r4e3w2q1"
```

```
**** RESPONSE ****
201 Created
Location: /users/q1w2e3r4
...
```

Обратите внимание на использование кода 201 Created для подтверждения восстановления ресурса.

Обсуждение

Преимущество отправки специального запроса в том, что клиентскому приложению для отмены изменения не нужно вспоминать детали изначального ресурса. Недостаток же в том, что интерфейсам сервисов нужно брать на себя ответственность за сохранение истории изменения ресурсов.

Когда ничто не помогает, вам может потребоваться решить проблему с отменой напрямую — обратиться к поддержке сервиса, чтобы они воссоздали предыдущие состояния платформы. Подробнее этот подход описан в рецепте 7.18.

URL для использования в специальных командах отката ничем особым не отличается от обычного. Я видел, как команды в качестве исходного целевого ресурса задействуют то же пространство URL (`/users/q1w2e3r4`) или специальный ресурс для отката (`/rollbacks/users/q1w2e3r4`). Я также видел совмещение этих двух подходов (см. предыдущий пример).

Для ситуаций, в которых одно действие изменяет несколько ресурсов (например, `removeCustomerData` влияет на хранилища данных о клиентах, заказах и продажах), может потребоваться спроектировать специальную возможность отката ресурса и по возможности реализовать собственную версию локальной отмены (см. рецепт 7.17) для любого другого сервиса. Все это может оказаться непростой задачей и потребовать сотрудничества с владельцами других сервисов.

Читайте также

- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.
- Рецепт 7.17. Поддержка локальной отмены или отката.
- Рецепт 7.18. Призыв на помощь.

3.10. Проектирование с учетом возможности расширения сообщений

Как правило, когда сервис становится популярным, начинают возникать идеи для улучшения его интерфейса. Один из способов — расширение тела сообщения HTTP-ответа. В рецепте показывается, как легко спроектировать интерфейс с учетом дальнейшего расширения тела сообщений без вреда для потребителей API.

Задача

Вы получаете запросы на изменение тела ответа вызова API. Некоторые просят добавить новые свойства, другим нужно, чтобы скалярное свойство (например, `phoneNumber`) стало массивом (`phoneNumbers`). Как сделать возможным подобное расширение без вреда для потребителей API, уже использующих текущий релиз? Каковы ограничения при расширении сообщений?

Решение

Эффективный способ изменения тел ответа, уже находящихся в производственной среде, — следовать правилу «не меняй, а добавляй» (см. раздел «Не меняй, а добавляй» главы 2).

Это означает, что нужно оставить все существующие элементы нетронутыми и просто включить в тело ответа новые. Также можете заложить в свои форматы ответов поддержку простой коллекции пар «имя — значение», в которую можно будет вносить дополнительные свойства любого типа, включая массивы. Подробнее — в примерах далее.

Пример

Рассмотрим простое тело сообщения HTTP-ответа:

```
{
  "name": "Merk Muffly",
  "region", "southwest",
  "age": 21
}
```

Есть несколько вариантов расширения этого объекта так, чтобы в дальнейшем он поддерживал возможность внесения изменений.

Добавить коллекцию свойств

Вы можете добавлять новые свойства, включив в изначальный проект коллекцию пар «имя — значение» (name-value pair, NVP):

```
{
  "name": "Merk Muffly",
  "region", "southwest",
  "age": 21,
  "nvp" : [...]
}
```

Теперь можно добавлять любые другие свойства (скалярное, массив, объект), не нарушая существующей структуры:

```
{
  "name": "Merk Muffly",
  "region", "southwest",
  "age": 21,
  "nvp" : [
    {"hatsize" : "3"},
    {"phoneNumbers": ["123-456-7890", "980-657-3421"]},
    {"address": {"street": "...", "city": "...", "state": "...", "zip": "..."}}
  ]
}
```

Добавление параллельных свойств

Бывают случаи, когда нужно изменить существующее свойство или расширить его на несколько связанных свойств. Например, вам может потребоваться изменить свойство `name`, расширив его на использование `givenName` и `familyName`. Вместо замены свойства `name` можете просто *добавить* два новых:

```
{
  "givenName": "Merk",
  "familyName": "Muffly",
  "name": "Merk Muffly",
}
```



```

"region", "southwest",
"age": 21,
"nvp" : [
  {"hatsize" : "3"},
  {"phoneNumbers": ["123-456-7890", "980-657-3421"]},
  {"address": {"street": "...", "city": "...", "state": "...", "zip": "..."}}
]
}

```

Теперь при возврате сообщения в HTTP-ответе будут поддерживаться оба формата. Потребители API, которые понимают свойство `name`, по-прежнему смогут его находить, а те, которые понимают новые свойства `givenName` и `familyName`, будут находить их.

Важно отметить, что правила тел сообщений, *отправляемых* в сервис посредством PUT или POST, тоже нужно будет скорректировать. В этом случае сервис можно изменить так, чтобы он мог получать все три значения: `name`, `givenName` и `familyName`. Когда клиент API отправляет только `name`, сервис сможет выдать переданное значение и сохранить результаты в `givenName` и `familyName`. И напротив, когда клиент API отправляет `givenName` и `familyName`, сервис может совместить эти значения и сохранить результаты в `name`.

Поддержка нескольких форматов вывода

В некоторых случаях изменения в сообщении ответа оказываются настолько обширными, что простой корректировки нескольких свойств оказывается недостаточно. Вместо этого можно возвращать несколько версий одной и той же информации в одном ответе. В таком случае потребуется заложить в изначальное сообщение возможность размещения нескольких форматов вывода. Это можно сделать, включив дополнительный корневой элемент во все структуры ответов.

Вот как выглядит образец объекта с дополнительным корнем для размещения форматов вывода:

```

{"message" : {
  "person" : {
    "givenName": "Merk",
    "familyName": "Muffly",
    "name": "Merk Muffly",
    "region", "southwest",
    "age": 21,
    "nvp" : [
      {"hatsize" : "3"},
      {"phoneNumbers": ["123-456-7890", "980-657-3421"]},
      {"address": {"street": "...", "city": "...", "state": "...", "zip": "..."}}
    ]
  }
}}

```

Теперь, когда важные свойства заключены в элемент `person`, можно легко возвращать новые варианты `person` или даже других новых элементов сообщений:

```
{ "message" : {
  "personv2": {...},
  "metadata": [...],
  "links": [...],
  "person" : {
    "givenName": "Merk",
    "familyName": "Muffly",
    "name": "Merk Muffly",
    "region", "southwest",
    "age": 21,
    "nvp" : [
      { "hatsize" : "3"},
      { "phoneNumbers": [ "123-456-7890", "980-657-3421" ] },
      { "address": { "street": "...", "city": "...", "state": "...", "zip": "..."} }
    ]
  }
}
```

До тех пор, пока изначальная структура включает первичный корневой элемент, добавление новых версий тел сообщений или совершенно новых корневых элементов не навредит существующим потребителям API.

Обсуждение

Эти правила проектирования особенно полезны, если вы решите *не использовать* существующий гипермедиаформат, а создать свой.



В рассмотренных рецептах большинство предопределенных форматов сообщений (HTML, HAL, SIREN и Collection + JSON) следуют описанным правилам расширяемости.

Основная сложность успешного применения этих паттернов расширения в том, что потребители API могут применять к входящим сообщениям строгие валидаторы схем (JSON-Schema, XML-Schema и т. д.). Можно попросить их не совершать эту ошибку, но гарантированно предотвратить ее не получится.

Как правило, вы можете применять эти паттерны к существующим форматам сообщений, например HTML, HAL и пр., не нарушая работы существующих клиентских приложений.

Читайте также

- Рецепт 3.10. Проектирование с учетом возможности расширения сообщений.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.

- Рецепт 4.12. Использование документов схем для валидации исходящих сообщений.
- Рецепт 6.10. Изменение моделей данных в промышленной среде.
- Рецепт 6.11. Расширение удаленных хранилищ данных.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.

3.11. Проектирование с учетом возможности изменения интерфейсов

Аналогично тому, как активно используемые API будут генерировать запросы на изменение форматов сообщений, они сами будут получать запросы на свое изменение, включая URL, HTTP-методы и входные аргументы. Текущий рецепт предлагает три правила, которые позволят обновлять интерфейсы сервисов, не мешая работать их потребителям.

Задача

Как обновлять интерфейс сервиса, в том числе изменять URL, HTTP-методы и входные аргументы, не нарушая работу существующих клиентов? Каковы три правила внесения безвредных изменений? Какие еще принципы стоит учитывать при обновлении интерфейса сервиса?

Решение

Практически любой API рано или поздно меняется. Одним из преимуществ работы в виртуальном мире является то, что изменения в нем менее затратны, чем в физическом мире. Но есть здесь и свои сложности. Тем не менее вы можете изначально изменить часть своего проекта, следуя общим принципам и трем устойчивым правилам.

Главное — не навреди

Фраза «Главное — не навреди» происходит из клятвы Гиппократы. Эту клятву, придуманную в IV веке до нашей эры, давали многие врачи в качестве обещания оказывать пациентам надлежащую помощь. Сама клятва намного обширнее, эта фраза — лишь наиболее известная ее часть. В этом смысле API тоже заслуживают такой клятвы.

Как только ваш API оказывается в промышленной среде, он начинает жить своей жизнью. Применяя этот интерфейс, другие приложения начинают от него зависеть. Вы же, как автор данного API, несете ответственность перед всеми его пользователями. И одной из первых ваших обязанностей является не навредить. Если вы вносите в работающий API изменение, которое вызовет сбой в работе одного из ваших потребителей, то нарушите «клятву Гиппократы» в отношении API.

Разветвление

Иногда вы будете оказываться в ситуации, когда нужно внести в работающий API критическое изменение. Например, вы можете выпустить API, который позволяет клиентам обновлять личную информацию на ваших серверах, а через год компания решит, что больше не хочет позволять клиентам редактировать свои данные напрямую. Когда такое происходит, нужно выпустить новый API с желаемой функциональностью и попросить пользователей существующего API обновить свои приложения до последней версии. Пока это будет происходить, вам потребуется какое-то время поддерживать работу двух API.

Этот процесс называется разветвлением интерфейса. Вы создаете новый API на основе существующего и поддерживаете работу обоих, пока все потребители не обновят свои клиентские приложения. В течение этого процесса можно выполнить кое-какие действия, чтобы упростить переход со старого релиза на новый.

Три правила изменения интерфейса сервиса

Когда решите обновить существующий интерфейс, вам нужно будет следовать трем правилам:

- ничего не убирать;
- ничего не переопределять;
- сделать дополнения опциональными.

Как только вы опубликуете свой API, все URL-адреса, сопутствующие свойства (входные и выходные) и даже HTTP-методы в документации станут *обещаниями*, которые вам нельзя нарушать. Использование форматов гипермедиа, которые позволяют включать входные метаданные (FORM) в среде выполнения, снизит зависимость от деталей протокола вроде HTTP-методов и входных аргументов, но изменение существующих обещаний станет ошибочным решением. Вместо этого вы можете *добавить* новые возможности, которые предоставят необходимую функциональность.

Вы также не можете переопределять существующие элементы интерфейса. Если определите параметр запроса `?size` как размер страницы данных, то не сможете потом еще раз переопределить его значение как размер шляпы или ботинка. Переопределение элементов равноценно их замене и является критическим изменением.

Наконец, любые нововведения можно *добавить* в интерфейс как опциональные элементы при условии, что они ничего из него не уберут и ничего не переопределят. Если вы придумаете дополнительные входные аргументы, то можете добавить их в FORM, но не должны делать *обязательными*. Если хотите создать новое действие, содержащее новые обязательные входные аргументы, то можете определить

новую конечную точку, метод или коллекцию аргументов (новую FORM). Но вы не можете сделать вызов этого действия обязательным.



Подробнее об изменении тела ответа интерфейса сервиса читайте в рецепте 3.10.

Пример

Если вы добавляете аргумент во входную форму, нужно сделать этот аргумент опциональным и предоставить его базовое значение:

```
<!-- текущая форма поиска -->
<form action="..." method="GET" name="findUsers">
  <input name="givenName" value="" required="true" />
  <input name="familyName" value="" required="true" />
  <input type="submit" />
</form>

<!-- обновленная форма поиска -->
<form action="..." method="GET" name="findUsers">
  <input name="givenName" value="" required="true" />
  <input name="familyName" value="" required="true" />
  <input name="regions" value="all" required="false" />
  <input type="submit" />
</form>
```

Обратите внимание на то, что эти безвредные изменения работают в обоих направлениях. У потребителей API должна быть возможность отправить на сервер любую форму для получения результатов. Если будет отсутствовать аргумент `region`, сервис должен *предположить*, что было отправлено предустановленное значение ("all").

Если вы хотите изменить действие, которое содержит новые обязательные входные аргументы, то можете просто *добавить* это действие в интерфейс и позволить клиентским приложениям игнорировать эту форму, пока они не выполнят обновление для ее поддержки.

Вот форма, которая получает заказ на обработку:

```
"actions": [
  {
    "name": "processOrder",
    "title": "Process Order",
    "method": "PUT",
    "href": "/orders/processing/q1w2e3r4",
    "type": "application/x-www-form-urlencoded",
```

```

    "fields": [
      { "name": "orderNumber", "type": "hidden", "value": "42" },
      { "name": "productCode", "type": "text", "value": "..." },
      { "name": "quantity", "type": "number", "value": "..." }
    ]
  },
  {
    "name": "ProcessSalesRepOrder",
    "title": "Process Sales Rep Order",
    "method": "PUT",
    "href": "/orders/salesrep/p0o9i8u7",
    "type": "application/x-www-form-urlencoded", "fields": [
      { "name": "orderNumber", "type": "hidden", "value": "42" },
      { "name": "productCode", "type": "text", "value": "..." },
      { "name": "quantity", "type": "number", "value": "..." },
      { "name": "salesRep", "type": "text", "value": "..." }
    ]
  }
]

```

Здесь была добавлена возможность, позволяющая сотрудникам отдела продаж получать процент с реализации товаров. Но они могут получить его, только если с информацией о заказе будет отправлено их имя. Это обязательный элемент нового действия `processSalesRepOrder`. Хорошим решением будет включить оба действия и позволить клиентским приложениям использовать то, которое они понимают.

Вы можете изменять существующий интерфейс спустя любой промежуток времени с момента его вывода в промышленную среду при условии, что не навредите ему.

Обсуждение

Несмотря на всю важность документирования вашего API и всех его изменений, этого недостаточно, особенно когда вы не знаете всех тех, кто будет использовать API (см. приложение А). Дело в том, что вы не будете уверены, прочтут ли пользователи вашу документацию, а если и прочтут, то нет гарантии, что у них найдутся время и ресурсы для обновления своих клиентских приложений.



Изменения в словаре интерфейса сервиса (см. рецепт 3.3) требуют внимания к деталям. Также следите за тем, чтобы не навредить при обновлении своих семантических профилей.

Если вы вдруг подумаете, что можно безопасно изменить несколько деталей интерфейса своего API, не создав проблем для потребителей, то имейте в виду, что зачастую это не так. Об этом писал сотрудник Google Хайрам Райт (Hyrum Wright), когда сформулировал принцип, который позже стал известен как закон Хайрама. Он сказал: «При достаточном числе пользователей API неважно, что вы обещаете в контракте, — от любого наблюдаемого поведения вашей системы кто-нибудь да будет зависеть».

Один из вариантов убедиться в том, что изменения вашего интерфейса не окажут критического влияния, — применение существующего набора тестов к *новому* интерфейсу. Выполнив все старые тесты в отношении нового API, вы можете примерно понять, как существующие клиентские приложения будут реагировать на его изменения.

Читайте также

- Рецепт 3.10. Проектирование с учетом возможности расширения сообщений.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 4.12. Использование документов схем для валидации исходящих сообщений.
- Рецепт 6.10. Изменение моделей данных в промышленной среде.
- Рецепт 6.11. Расширение удаленных хранилищ данных.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.

ГЛАВА 4

Клиенты гипермедиа

Компьютеры хороши тем, что действуют согласно вашим инструкциям, хотя это же является и их недостатком.

Тед Нельсон

Написание приложений, которые работают с внешними сервисами в веб-среде через API, требует совмещения конкретики (*что именно* делать) и необходимости обобщения (*как* это делать), тем самым усложняя задачу. Рецепты этой главы посвящены тому, *что* мы приказываем клиентским приложениям делать, применяя локальный код, и *как* мы это им говорим, используя протокол и обмен сообщениями. Такая комбинация из «что» и «как» составляет основу стабильных, но при этом гибких потребителей API.



Дополнительная информация по теме создания потребителей API приводится в разделе «Повышение отказоустойчивости клиентов гипермедиа» главы 2.

Как правило, излишняя конкретика в отношении того, что клиенты могут, а что не могут делать, приводит к созданию приложений, которые легко испортить и сложно применить повторно. Более эффективным подходом будет создание потребителей API, которые обеспечивают минимум инструкций, регламентирующих, *как* взаимодействовать (например, с помощью протокола, модели сообщений и словаря) с серверами, позволяя последним определять все остальные детали (*что*) в среде выполнения. Как показано на рис. 4.1, рецепты текущей главы акцентируются на использовании форматов гипермедиа через протокол HTTP для поддержки валидации, управления состоянием и создания ориентированных на цель клиентских приложений. Именно этим занимаются HTML-браузеры уже более 30 лет, и мы в отношении потребителей API будем применять аналогичный подход.

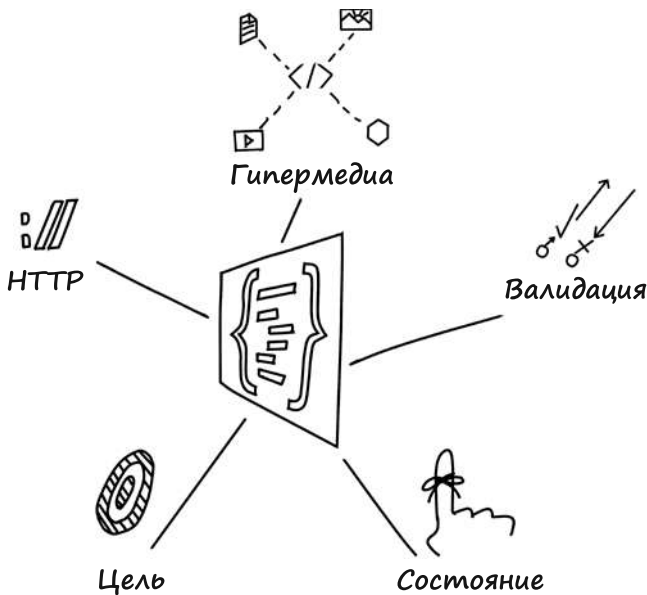


Рис. 4.1. Рецепты для клиентов гипермедиа

Это не значит, что нам нужно воссоздавать браузер. Данное клиентское приложение ориентировано на значительное удобство применения, когда человек выступает мозгом, стоящим за интерфейсом (хотя в реальности перед интерфейсом). В HTML-формат заложена возможность управления визуальной отрисовкой сообщений, а также анимацией содержимого с помощью встроенных скриптов. Этот акцент на UI и людях для большинства веб-сервисов и потребителей API не требуется.

Вместо этого в отношении веб-сервисов нам нужно сосредоточиться на M2M-взаимодействиях и на том, как компенсировать отсутствие важного элемента — человека. Рецепты текущей главы отбирались по тому принципу, что они позволяют добиться многократного повторного применения без участия людей на каждом шаге клиент-серверного взаимодействия. В связи с этим акцент в них делается на четырех элементах устойчивых клиентов:

- использование протоколов и форматов;
- применение метаданных среды выполнения;
- решение проблемы M2M-интерфейсов;
- использование семантических профилей в качестве основы взаимопонимания между клиентом и сервером.

Опираясь на этот перечень, разберем набор клиентских рецептов для RESTful Web API.

4.1. Ограничение использования жестко прописанных URL

Изменение URL на сервере может создать для клиентских приложений раздражающие проблемы. Иногда такие изменения происходят в связи с добавлением или удалением функциональности — от подобных проблем клиентское приложение защитить сложно. Порой URL сервиса меняется из-за его «переезда» с одной платформы или группы реализации на другую. Как ограничить влияние происходящих на сервере изменений URL на клиентское приложение?

Задача

Как уменьшить вероятность сбоя клиентского приложения при изменении URL сервиса?

Решение

Лучший способ ограничить влияние изменений URL сервиса на клиентское приложение — это абстрагироваться от фактических значений URL.

Используйте именованные переменные URL

Во-первых, клиента нужно запрограммировать так, чтобы любая ссылка на URL внутри этого приложения была указанной переменной, а не фактическим адресом URL. Этот подход аналогичен подходу с локализацией клиентского приложения для одного языка (французский, арабский, корейский и пр.).

Сохраняйте значения URL в файлах конфигурации

Во-вторых, после установки имен переменных для URL нужно вынести их фактические строки из исходного кода в файл конфигурации. Это позволит обновлять URL без необходимости изменять исходный код или повторно компилировать приложение. В зависимости от платформы после изменения файла конфигурации вам все равно может потребоваться повторно развернуть клиентское приложение.

Сократите количество URL, которые нужно запомнить, до одного

Если возможно, ограничьте общее число URL-адресов, которые ваше приложение должно знать. Если оно обращается к сервису с поддержкой форматов гипермедиа (Collection + JSON, SIREN, HAL и т. д.), то ему наверняка потребуются запомнить только один URL — стартовый или домашний (см. рецепт 5.1).

Убедите команды сервиса использовать гипермедиа или файлы конфигурации

Наконец, если вы можете повлиять на людей, которые пишут используемые вами сервисы (например, на коллег), попробуйте убедить их задействовать форматы гипермедиа, передающие URL-адреса в среде выполнения, или хотя бы предоставить скачиваемый файл конфигурации, в котором будут сохранены все именованные переменные URL, необходимые приложению при выполнении.

Пример

Итого есть четыре способа снизить влияние изменения URL сервиса.

- Использовать в коде именованные переменные URL-адресов.
- Переместить URL-адрес в файлы конфигурации клиента.
- Ограничить число URL-адресов, которые клиент должен знать заранее.
- Сделать так, чтобы сервисы предоставляли вам URL-адреса.

Использование именованных переменных URL

Далее показан пример кода, отражающий список именованных переменных URL, хранящихся в коде клиента:

```
// Инициализация именованных переменных для URL-адресов сервиса
var serviceURLs = {};
serviceURLs.home = "http://service.example.org/";
serviceURLs.list = "http://service.example.org/list/";
serviceURLs.filter = "http://service.example.org/filter/";
serviceURLs.read = "http://service.example.org/list/{id}/";
serviceURLs.update = "http://service.example.org/list/{id}/";
serviceURLs.remove = "http://service.example.org/list/{id}/";
serviceURLs.approve = "http://service.example.org/{id}/status";
serviceURLs.close = "http://service.example.org/{id}/status";
serviceURLs.pending = "http://service.api.example.org/{id}/status";

/* позже в коде клиента... */

// Использование именованных переменных URL
const http = new XMLHttpRequest();

// Отправка запроса
http.open("GET", serviceURLs.list);
http.send();

// Обработка ответов
http.onload = function() {
  switch (http.responseURL) {
    case serviceURLs.home:
      ...
      break;
    ...
  }
}
```

Заметьте, что некоторые значения URL используются в разных именах переменных. Так часто бывает, когда важная информация передается через тело HTTP и/или HTTP-метод (например, HTTP PUT или HTTP DELETE). Вы также заметите, что некоторые URL содержат шаблонное имя (например, `http://api.example.org/list/{id}/`). Это означает, что клиент и сервер *должны* задействовать общий стандарт для кодирования URL, к которым применен шаблон. Я советую использовать библиотеки, которые соответствуют IETF RFC 6570 (<https://oreil.ly/ApkCn>).

Загрузка именованных переменных URL из конфигурации

После настройки кода на применение только именованных переменных URL нужно перенести все эти URL из кодовой базы в файл конфигурации:

```
<html>
<head>
  <script type="text/javascript"
    src="client.example.org/js/service-urls-config.js"></script>
  <script type="text/javascript">
    ...
    // Использование именованных переменных URL
    const http = new XMLHttpRequest();

    // Отправка запроса
    http.open("GET", serviceURLs.list);
    http.send();

    // Обработка ответов
    http.onload = function() {
      switch (http.responseURL) {
        case serviceURLs.home:
          ...
          break;
          ...
      }
    }
    ...
  </script>
</head>
<body>
  ...
</body>
</html>
```

Файл конфигурации, который выглядит аналогично коду из примера, сохраняется там же, где и код клиента (<http://client.example.org/js/>). Но сохранить его можно в любом месте, к которому у приложения есть доступ, включая файловую систему, поддерживаемую платформой клиента.

Ограничение числа запоминаемых URL до одного с помощью гипермедиа

Если ваш сервис использует форматы гипермедиа, которые автоматически предоставляют URL-адреса в среде выполнения, можете ограничить число URL, которые вашему приложению нужно знать заранее, до, возможно, *одного* — стартового или домашнего URL. Тогда клиент сможет находить любой другой URL на основе информации, встроенной в `id`, `name`, `rel` и/или `tags` (см. рецепт 5.1):

```
/* find-rel.js */
var startingURL = "http://service.example.org/";
var thisURL = "";
var link = {};
```

```
// Использование именованных переменных URL
const http = new XMLHttpRequest();

// Отправка запроса
http.open("GET", serviceURLs.list);
http.send();

// Обработка ответов
http.onload = function() {
  switch (http.responseURL) {
    case startingURL:
      link = findREL("list");
      if(link.href) {
        thisURL = link.href;
        ...
      }
      ...
      break;
      ...
  }
}
```

Наконец, если вы можете повлиять на команды разработки сервиса, попробуйте убедить их использовать гипермедиаформаты, чтобы клиентским приложениям нужно было запомнить только стартовый URL. Либо попросите разработчиков сервера предоставить удаленный файл конфигурации со всеми именованными переменными, определяющими нужные клиенту статические URL.

Обсуждение

Лучший способ ограничить влияние изменений URL сервиса на клиента — это вынести их за пределы кода клиента. Эффективнее всего сделать так, чтобы сервис использовал гипермедиаформаты, которые сообщают URL в среде выполнения, или предоставил конфигурационную информацию, содержащую список URL в виде именованных переменных. Если добиться этого от сервиса не удастся, тогда решение вопроса ляжет на плечи разработчиков клиентского приложения. Это будет означать применение файла конфигурации клиента или, если загрузка конфигурации невозможна (например, по причинам безопасности), создание в коде собственной клиентской коллекции именованных переменных.



Важно понимать, что использование клиентских файлов конфигурации — не лучшее решение. Об изменении URL сервиса следует сообщать разработчику клиента, чтобы можно было обновлять информацию на клиентской стороне — конфигурацию или код. Это может вызвать задержку между изменением сервиса и клиента. Вы можете настроить мониторинг изменений на стороне клиента, но это только добавит работу и способно сократить время реагирования клиентских приложений на необъявленные изменения.

Ранее отмечалось, что некоторые переменные URL могут использовать одни и те же данные URL, но служить для совершенно разных целей. Например, URL сервиса для чтения, обновления и удаления может быть одним (`http://service.example.org/{id}`). Это ведет к следующему шагу, когда клиентские приложения сохраняют всю коллекцию метаданных запросов и в коде тоже (URL, метод, заголовки, строку запроса и/или информацию тела). Подробнее об этой возможности и случаях ее применения читайте в рецепте 4.10.

Читайте также

- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 4.10. Поддержка ссылок и форм для сервисов без поддержки гипермедиа.
- Рецепт 5.1. Публикация не менее одного стабильного URL.
- Рецепт 7.8. Возвращение всех сопутствующих действий.

4.2. Программирование клиентов, понимающих HTTP

Иногда вспомогательные библиотеки, обертки и/или наборы разработки API (software development kits, SDK) *усложняют* для клиентских приложений решение поставленных задач, вместо того чтобы их упрощать. По этой причине важно обеспечить для клиентских приложений возможность общаться с сервисами через HTTP или любой другой целевой протокол.

Задача

Когда сервисы предлагают SDK или вспомогательные инструменты, бывает, что клиент не может найти конкретную локальную функцию, необходимую для решения задачи. Как сделать так, чтобы клиентское приложение могло обходить любые ограничения любых вспомогательных библиотек или SDK, предоставляемых сервисом?

Решение

Лучший способ создания эффективных и гибких клиентских приложений — обеспечить понимание ими используемого протокола. Это означает, что они могут при необходимости общаться напрямую с сервисом посредством HTTP-протокола (или любого другого, применяемого сервисом). Для этого даже в случае применения клиентским приложением SDK или вспомогательных библиотек нужно сделать так, чтобы это приложение умело общаться непосредственно с помощью HTTP. Это позволит вашему приложению задействовать сервис таким образом, какого авторы сервиса изначально не предполагали.

СЕРВИСЫ МОГУТ УЧИТЬСЯ У КЛИЕНТОВ

Важно отметить, что провайдеры сервисов могут многое узнать, отслеживая способы, которыми приложения обращаются к их API (включая порядок этого обращения). В некоторых случаях клиентские приложения будут знакомить сервисы с функциональными возможностями и рабочими потоками, которые их интересуют. На основе этой информации создатели сервисов смогут выпускать их новые версии, делая эти задачи безопаснее и проще.

При необходимости можно без проблем использовать доступные SDK сервиса вместе с прямыми HTTP-вызовами. Удачной идеей будет создать собственную высокоуровневую HTTP-библиотеку, которую можно легко вызывать при необходимости.

Пример

Несмотря на то что большинство современных языков предоставляют HTTP-библиотеки и функции, грамотным решением будет создать собственную библиотеку. Это подразумевает обработку деталей как HTTP-запросов, так и HTTP-ответов. Вот простой пример вспомогательной JavaScript-библиотеки для HTTP-вызовов:

```
var ajax = {

  // Здесь код инициализации и настройки...

  /** ПУБЛИЧНЫЕ МЕТОДЫ **/
  args = {url:string, headers:{}, queryString:{}, body:{},
    callback:function, context:string}
  /***/
  httpGet: function(args) {...},
  httpGetXML: function(args) {...},
  httpGetJSON: function(args) {...},
  httpGetPlainText: function(args) {...},
  httpPost: function(args) {...},
  httpPostVars: function(args) {...},
  httpPostForXML: function(args) {...},
  httpPostForJSON: function(args) {...},
  httpPostForPlainText: function(args) {...},
  httpPut: function(args) {...},
  httpHead: function(args) {...},
  httpDelete: function(args) {...},
  httpOptions: function(args) {...},
  httpTrace: function(args) {...},

  // Детали реализации далее...
}
```

Впоследствии эту библиотеку можно вызывать с помощью такого кода:

```
function getDocument(url) {
    var args = {};
    args.url = url;
    args.callbackFunction = ajaxComplete;
    args.context = "processLinks";
    args.headers = {'accept':'application/vnd.collection+json'}

    ajax.httpGet(args)

    // Позже...

function ajaxComplete(response,headers,context,status,msg)
{
    switch(status) {...} // обработка согласно статусу
    switch(context) {...} // обработка согласно контексту
}
```

Этот пример является псевдокодом из клиентской реализации на JavaScript для браузерных приложений. Конкретные детали вашей библиотеки значения не имеют. Важно то, что клиентское приложение может быстро и безопасно создать HTTP-запрос и обработать HTTP-ответ. Обратите внимание: эта библиотека создается для упрощения HTTP-вызовов и не ограничивает доступ к протоколу.

Обсуждение

Важно не только предоставлять клиентским приложениям прямой доступ к HTTP-протоколу, но и программировать их так, чтобы разработчик всегда знал об использовании HTTP-вызовов. Может показаться удачной идея скрыть HTTP под семантическими методами таким образом:

```
var serviceClient = {
    findUser : function(userId) {...},
    assignUserWorkTickets(userId,ticketArray) {...}
}
```

Но второй вызов в этом примере (`assignUserWorkTickets`) может оказаться минным полем. Сколько HTTP-вызовов здесь задействовано? Всего один? Возможно, по одному на каждый элемент в `ticketArray`? Вызовы тикетов выполняются последовательно или параллельно? Разработчики должны видеть, с чем они имеют дело, когда используют SDK или библиотеку. Даже такие простые вещи, как изменение вызовов вспомогательной библиотеки с `assignUserWorkTickets` на `sendHttpRequests({ requestList:WorkTickets, parallel:true })`, поможет разработчикам понять последствия своих действий.

Кроме того, понимающий HTTP клиент необязательно будет сложным. Клиентские приложения могут обойтись объектом `XMLHttpRequest`, не углубляясь в дебри HTTP. Пример описан в рецепте 4.1.

Недостаток этого подхода в том, что некоторые сервисы могут захотеть защититься от неопытных или недобросовестных разработчиков клиентских приложений. Они будут использовать SDK, чтобы контролировать способ, которым приложение взаимодействует с сервисом. Такой подход не остановит злоумышленников и зачастую лишь будет раздражать порядочных разработчиков, пытающихся решить собственные задачи способами, о которых провайдер сервиса не предполагал. Я видел много случаев, когда команды клиентской стороны отказывались от API, потому что находили требуемый SDK неподходящим.

Читайте также

- Рецепт 4.1. Ограничение использования жестко прописанных URL.
- Рецепт 4.6. Управление форматами представления в среде выполнения.
- Рецепт 5.1. Публикация не менее одного стабильного URL.
- Рецепт 5.6. Поддержка согласования HTTP-содержимого.
- Рецепт 6.6. Возврат на запросы данных HTTP-кода состояния 200 или 400.
- Рецепт 6.9. Повышение производительности с помощью директив кэширования.

4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений

Чтобы продлить жизнь клиентского приложения и повысить его стабильность, стоит научить приложение говорить на языке хорошо известных медиатипов.

Задача

Как сделать так, чтобы клиентское приложение было более устойчивым к небольшим изменениям и не слишком привязанным к области задач или работающему сервису?

Решение

Отличным способом создания устойчивых клиентских приложений является их тесная привязка к возвращаемым сервисом сообщениям, а не к конкретному сервису или предметной области. Тогда в случае небольших изменений сервиса при сохранении того же самого медиатипа ответа клиентское приложение, скорее всего, продолжит функционировать корректно.

Пример

Существует много вариантов программирования клиента API. Можно привязать его непосредственно к сервису и его объектам документов или к структурированным медиатипам, которые этот сервис отправляет. Почти во всех случаях лучше привязывать приложение именно к медиатипам, а не к типам интерфейса сервиса.

Далее показан пример приложения ToDo¹. Этот клиент реализован с привязкой ко всем действиям, описанным в документации API:

```
/* to-do-functions.js */
var thisPage = function() {
  function init() {}
  function makeRequest(href, context, body) {}
  function processResponse(ajax, context) {}
  function refreshList() {}
  function searchList() {}
  function addToList() {}
  function completeItem() {}
  function showList() {}
  function attachEvents() {}
};
```

Детали происходящего во всех функциях здесь неважны. Важно то, что каждое действие в области (*refresh*, *search*, *add* и т. д.) описано в *исходном коде* клиента. Когда сервис получит новую функциональность, например возможность устанавливать для задачи целевую дату (*setDueDate*), его код окажется устаревшим и не сможет адаптироваться к этой функциональности. Дополнительная сложность возникнет, если изменятся детали, такие как аргументы или прочие метаданные действий (например, *POST* станет *PUT*, поменяется URL и т. п.).

Взгляните на клиента, который использует модель на основе сообщений. В этом случае код фокусируется не на конкретных действиях ToDo, а на сообщениях (например, HTML, HAL, SIREN и т. д.), которые клиент отправляет и получает. В действительности этот подход на основе сообщений будет одинаковым вне зависимости от назначения сервиса или темы:

```
/* to-do-messages.js */
var thisPage = function() {
  function init() {}
  function makeRequest(href, context, body) {}
  function processResponse(ajax, context) {}
  function displayResponse() {}
  function renderControls() {}
  function handleClicks() {}
};
```

¹ Список задач. — *Примеч. пер.*

Детали отсутствуют и здесь, но суть в том, что функциональность клиента строится на выполнении запросов, обработке ответов, отображении результатов, средствах управления вводом и перехвате событий щелчков кнопкой мыши. Конкретные действия (`refresh`, `search`, `add` и т. д.) будут описаны в *сообщении ответа* посредством средств гипермедиа, а не кода. Это означает, что при добавлении сервисом новой возможности, например операции `setDueDate`, клиент увидит новые действия, описанные в сообщении, и автоматически отобразит их. То есть для поддержки новых операций писать дополнительный код не потребуется.

Привязка к сообщениям означает, что добавление новых действий и большинство изменений существующих являются «бесплатными», то есть не требуют написания дополнительного кода.

Обсуждение

Перестройка кода клиента таким образом, чтобы он опирался не на внутренние операции, а на сообщения, — очень эффективный способ создания устойчивых приложений для веб-среды. Именно такой подход использует HTML-браузер, который уже более 30 лет функционирует без значительных изменений интерфейса. И это несмотря на то, что внутренний код HTML-браузеров неоднократно полностью изменялся, существует несколько конкурирующих версий таких браузеров, и функциональность HTML тоже со временем менялась.

Как и в рецепте 4.2, который описывает создание клиентов API, понимающих HTTP, ориентирование этих клиентов на сообщения придает вашему проекту дополнительный слой устойчивости. Как мы увидим позже в текущей главе (см. рецепт 4.5), в этом «пироге» клиентской устойчивости есть и другие слои. Каждый из них ведет к созданию более надежной и стабильной основы для построения конкретного решения.

Одним из недостатков описанного подхода является то, что он зависит от подпадающей поддержки сервисами строго типизированных моделей сообщений для ответов ресурсов (см. рецепт 4.3). Если нужный вам сервис отвечает только неструктурированными медиатипами вроде XML и JSON, будет сложно использовать для клиентских приложений модель программирования на основе сообщений. Если вы можете повлиять на команды своего сервиса (например, вы трудитесь в IT-отделе корпорации), то стоит убедить коллег задействовать высокоструктурированные медиатипы вроде HTML, HAL, SIREN, Collection + JSON и пр. В главе 3 приведено несколько рецептов с акцентом на значимость использования медиатипов в качестве опорного принципа построения веб-сервисов.

Читайте также

- Рецепт 3.1. Обеспечение интероперабельности с помощью зарегистрированных медиатипов.
- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.

- Рецепт 4.2. Программирование клиентов, понимающих HTTP.
- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.
- Рецепт 4.6. Управление форматами представления в среде выполнения.

4.4. Программирование эффективных клиентов, понимающих профили словарей

Важно закладывать в клиентские приложения не только умение «говорить» на HTTP и привязываться к форматам сообщений, но и возможность использовать в диалоге словарь сервиса и/или предметной области. Для этого вам нужно запрограммировать клиентские приложения распознавать и применять предметные словари, выраженные в виде внешних документов профилей.

Задача

Как сделать так, чтобы клиентское приложение понимало словарь (имена свойств вызовов и действий) сервисов, с которыми взаимодействует, даже в случае периодического изменения этих сервисов?

Решение

Чтобы клиенты и серверы могли общаться в среде выполнения на одном языке, нужно обеспечить, чтобы в них было заложено понимание общих терминов, например имен данных и действий. Реализовать это можно с помощью одного из известных форматов словарей, включая:

- RDF (Resource Description Framework Schema) — схему описания ресурсов (<https://oreil.ly/GX41I>);
- OWL (Web Ontology Language) — язык описания онтологий для семантической паутины (<https://oreil.ly/TSviq>);
- DCAP (Dublin Core Application Profiles) — прикладные профили Дублинского ядра (<https://oreil.ly/eR9Px>);
- ALPS (Application-Level Profile Semantics) — семантики профилей прикладного уровня (<https://oreil.ly/eTVa2>).

Поскольку я соавтор спецификации ALPS, во многих примерах в качестве словаря будет использоваться именно этот стандарт. Также вполне можно задействовать формат, который здесь не указан. Главное, чтобы ваше приложение и сервисы, с которыми оно взаимодействует, утвердили для описания области задачи единый формат словаря.

Пример

Выражение предметной области сервиса в виде словаря позволяет клиенту и сервису *утвердить общее понимание*. Об этом говорилось в рецепте 3.4. Вот набор терминов словаря для простой предметной области:

Простое приложение ToDo

Цель

Отслеживать записи 'ToDo', чтобы повысить своевременность и точность последующих действий клиентов.

Данные

В первом проходе приложения нужно отслеживать следующие свойства данных:

- * ****id**** : глобально уникальное значение для каждой записи ToDo.
- * ****body**** : текстовое содержимое записи ToDo.

Действия

Эта версия приложения должна поддерживать следующие операции:

- * ****Home**** : стартовое расположение сервиса.
- * ****List**** : возврат списка всех активных записей ToDo в системе.
- * ****Add**** : добавление в систему новой записи ToDo.
- * ****Remove**** : удаление из системы выполненной записи ToDo.

После преобразования в ALPS этот документ с историей будет выглядеть так:

```
{
  "$schema": "https://alps-io.github.io/schemas/alps.json",
  "alps" : {
    "version": "1.0",
    "title": "ToDo List",
    "doc" : {"value": "ALPS ToDo Profile (see [ToDo Story](to-do-story.md))"},

    "descriptor": [
      {"id": "id", "type": "semantic", "tag": "ontology"},
      {"id": "title", "type": "semantic", "tag": "ontology"},

      {"id": "home", "type": "semantic", "tag": "taxonomy",
        "descriptor": [{"href": "#goList"}]
      },

      {"id": "list", "type": "semantic", "tag": "taxonomy", "descriptor": [
        {"href": "#id"},
        {"href": "#title"},
        {"href": "#goHome"},
        {"href": "#goList"},
        {"href": "#doAdd"},
        {"href": "#doRemove"}
      ]
    }
  },
}
```

```

    {"id": "goHome", "type": "safe", "rt": "#home", "tag": "choreography"},
    {"id": "goList", "type": "safe", "rt": "#list", "tag": "choreography"},
    {"id": "doAdd", "type": "unsafe", "rt": "#list", "tag": "choreography",
      "descriptor": [{"href": "#id"}, {"href": "#title"}]},
    {"id": "doRemove", "type": "idempotent", "rt": "#list", "tag": "choreography",
      "descriptor": [{"href": "#id"}]}
  ]
}
}

```

Этот документ ALPS содержит все возможные элементы данных и действий вместе с дополнительными метаданными, описывающими создание запросов к сервису и ожидаемые ответы. Он может быть источником материала для создания клиентских приложений, способных успешно взаимодействовать с любым сервисом, поддерживающим этот профиль, особенно с сервисом с поддержкой гипермедиа.

Например, если ToDo-совместимый сервис работает по адресу localhost:8484, ToDo-совместимый клиент может действовать так:

```

# данные для обработки
STACK PUSH {"id": "zaxscdvf", "body": "testing"}

# поддерживаемые словари и формат
CONFIG SET {"profile": "http://api.examples.org/profiles/todo-alps.json"}
CONFIG SET {"format": "application/vnd.mash+json"}

# запись в сервис
REQUEST WITH-URL http://api.example.org/todo/list WITH-PROFILE WITH-FORMAT
REQUEST WITH-FORM doAdd WITH-STACK
REQUEST WITH-LINK goList
REQUEST WITH-FORM doRemove WITH-STACK
EXIT

```



В приложении Г приводится краткий обзор программирования на HyperLANG с помощью HyperCLI.

Заметьте, что клиенту нужно знать только стартовый URL и имена действий (`doAdd`, `goList` и `doRemove`) и свойств данных (`id` и `body`). Клиент также должен сообщить сервису, какой словарь (`profile`) и медиатип (`format`) будет использоваться при взаимодействии. Все эти решения принимаются в процессе *программирования* приложения. Оставшаяся часть скрипта будет посвящена вызову действий и отправке данных, о валидности которых клиент знает заранее, на основе опубликованного словаря. Поскольку в этом случае известно, что сервис поддерживает ответы с гипермедиа, клиент может смело предположить, что ссылки и формы словаря будут обнаружены в ответе сервиса и правильно закодированы.

Обсуждение

Документы словаря выступают связующим звеном между создателями API и их потребителями, не образуя при этом тесной связи. Как мы видели в последнем примере, использование очищенных от ошибок словарей и хорошо известных форматов медиатипов значительно уменьшает связанность между клиентом и сервисом без утраты деталей.

Сервисы зачастую публикуют документы определений API, например, OpenAPI (<https://oreil.ly/uyE9c>), AsyncAPI (<https://oreil.ly/gcOVb>), RAML (<https://oreil.ly/9AKUs>) и т. д. Это оказывается на руку специалистам, желающим реализовать единичный экземпляр *сервиса* API, но для потребителей API особой пользы не представляет. Когда в клиентском приложении для создания потребителя API используется спецификация реализации сервиса, такое приложение получается тесно привязанным к этой конкретной реализации. Как результат, любые ее изменения приведут к нарушению его работы. Более эффективным подходом будет привязка потребителя API к опубликованному документу *профиля*.

Если сервис, с которым вы работаете, не публикует стабильный документ словаря, желательно создать свой. Используйте документацию определений API (OpenAPI и т. д.) в качестве руководства для создания собственного документа в формате ALPS (или в другом на свое усмотрение), после чего опубликуйте его для своей команды (и других) при создании клиента API для этого сервиса. Также стоит включить этот документ ALPS в репозиторий исходного кода клиентского приложения, чтобы обращаться к нему в дальнейшем.

Как показано в предыдущем примере, максимальную пользу от профилей словарей можно получить, когда сервис, с которым вы работаете, возвращает ответы с поддержкой гипермедиа. Если это не так и вы не можете повлиять на команды сервиса, то все равно можно запрограммировать клиента таким образом, будто ответ поддерживает средства гипермедиа (см. рецепт 4.10).

Читайте также

- Рецепт 3.3. Предоставление информации о предметной области через публикуемые словари.
- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.5. Согласование поддержки профилей во время выполнения.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.9. Публикация документов определений сервисов.

4.5. Согласование поддержки профилей во время выполнения

Клиентские приложения, которые зависят от сервисов, предоставляющих ответы с помощью предопределенных семантических профилей, должны иметь возможность убедиться, что эти сервисы будут «говорить» с помощью нужного словаря.

Задача

Вы запрограммировали клиента на работу с любым сервисом, совместимым, например, с семантическим профилем `ToDo`. Как убедиться в среде выполнения, что сервис, который вы собираетесь использовать, поддерживает словарь `to-do`?

Решение

Клиентское приложение может уверенно проверять наличие поддержки семантического профиля в сервисе с помощью паттерна «согласование профиля». Клиенты могут применять заголовок запроса `accept-profile` для указания ожидаемого семантического профиля, а сервисы — заголовок `content-profile`, чтобы указать, какой семантический профиль поддерживается.

Как и согласование содержания (см. рецепт 5.6), согласование профиля позволяет клиентам и серверам обмениваться деталями метаданных и представлением ресурсов, а конкретно клиентам — принимать решения относительно приемлемости получаемых от серверов ответов. Когда сервисы возвращают профили ресурсов, не соответствующие запросу клиента, он может отклонить такой ответ с ошибкой, запросить у сервера дополнительную информацию либо продолжить обработку.

Пример

Клиенты могут определять поддерживаемые ресурсами сервиса семантические профили несколькими способами. Согласование профиля является для клиента наиболее прямым способом указать, какой профиль его интересует, и проверить, какой профиль сервис поддерживает.

Вот простой пример подобного согласования:

```
*** REQUEST
GET /todo/list HTTP/1.1
Host: api.example.org
Accept-Profile: <http://profiles.example.org/to-do>
```

```
*** RESPONSE
HTTP/2.0 200 OK
Content-Profile: http://profiles/example.org/to-do
```

...

Здесь клиент использует заголовок `accept-profile` для указания нужного словаря, а сервис — заголовок `content-profile`, чтобы сообщить клиенту, какой профиль задействуется для составления возвращаемого представления ресурсов.

Серверы также могут возвращать код состояния HTTP 406 (Not Acceptable), когда клиент запрашивает семантический профиль, который сервис не поддерживает. В таком случае сервис должен возвращать метаданные, указывающие, что конкретно он поддерживает:

```
*** REQUEST
GET /todo/list HTTP/1.1
Host: api.example.org
Accept-Profile: <http://profiles.example.org/to-do/v3>

*** RESPONSE
HTTP/2.0 406 Not Acceptable
Content-Type: application/vnd.collection+json

{ "collection":
  {
    "links" : [
      { "rel": "profile", "href": "http://profiles.example.org/todo/v1" },
      { "rel": "profile", "href": "http://profiles.example.org/todo/v2" },
    ]
  },
  "error" : {
    "title" : "Unsupported Profile",
    "message" : "See links for supported profiles for this resource."
  }
}
```

Сервисы также могут предоставлять клиентским приложениям способ заблаговременно запрашивать подробности относительно поддерживаемых семантических профилей. Для этого сервисы предлагают одну или несколько ссылок со значением `profile`, каждая из которых будет указывать на поддерживаемый документ семантического профиля. Вот пример:

```
{ "collection":
  {
    "title" : "Supported Semantic Profiles",
    "links" : [
      { "rel": "profile", "href": "http://profiles.example.org/todo/v1" },
      { "rel": "profile", "href": "http://profiles.example.org/todo/v2" },
      { "rel": "profile", "href": "http://profiles.example.org/todo/v3" }
    ]
  }
}
```

Эти ссылки могут присутствовать в любом представлении ресурсов, поддерживающем данный профиль. Даже когда имеется поддержка всего одного семантического профиля, правильно будет указывать в ответах ссылки на профили, чтобы клиенты

(и их разработчики) могли понять, какие словари поддерживаются. В таком случае клиенты смогут просмотреть представленную в ответе коллекцию ссылок и определить, есть ли среди них идентификатор того профиля, который их интересует.

Список поддерживаемых профилей можно представить и в виде HTTP-заголовков:

```
*** REQUEST
GET /todo/list HTTP/1.1
Host: api.example.org
Accept-Profile: <http://profiles.example.org/to-do/v3>

*** RESPONSE
HTTP/2.0 406 Not Acceptable
Content-Type: application/vnd.collection+json
Links <http://profiles.example.org/todo/v3>; rel="profile",
<http://profiles.example.org/todo/v2>; rel="profile",
<http://profiles.example.org/todo/v1>; rel="profile"
```

Обсуждение

Для клиентских приложений, которые закодированы в профиль, будет правильным проверять все входящие ответы, чтобы определить, смогут ли они должным образом обработать этот ресурс. Поскольку серверы могут указывать в ответе более одной ссылки, клиентские приложения должны предполагать, что все метаданные профилей сообщаются в виде коллекции, и искать в ней идентификатор нужного.

Согласование семантических профилей с помощью заголовков `accept-profile` и `content-profile` не особо популярно. Спецификация, описывающая этот процесс, по-прежнему находится в разработке (<https://oreil.ly/iBy3h>). На момент написания книги последняя работа W3 в отношении профилей (<https://oreil.ly/geeWI>) все еще не завершена. Тем не менее внутри закрытой системы, например в корпоративной IT-среде, реализация согласования профилей может стать удачным способом продвижения использования семантических профилей в целом.

Я не советую слишком углубляться в детали определения версий семантических профилей (v1, v1.1, v1.1.1 и т. д.), поскольку это только добавляет работы в среде выполнения как для клиентов, так и для удаленных сервисов. Когда требуется внести изменения в профили, стоит как можно дольше сохранять их обратно совместимыми. Если же обновление приводит к критическим изменениям, тогда нужно скорректировать профиль, указав в нем новую версию (v1 → v2).

Читайте также

- Рецепт 3.3. Предоставление информации о предметной области через публикуемые словари.
- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.

- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.8. Поддержка общих словарей в стандартных форматах.

4.6. Управление форматами представления в среде выполнения

У клиентских приложений, которым нужно взаимодействовать более чем с одним сервисом, может возникнуть потребность общаться с помощью нескольких форматов сообщений (HTML, HAL, SIREN, Collection + JSON и т. д.). То есть им нужно будет распознавать разные форматы и уметь работать с ними в среде выполнения.

Задача

Как реализовать для клиентских приложений возможность обрабатывать более одного типа сообщений в запросах, а также проверять типы получаемых от сервисов сообщений и согласованно обрабатывать информацию в разных форматах?

Решение

Работающие с сервисами приложения должны уметь управлять форматами представления ресурсов (HTML, HAL, SIREN, Collection + JSON и т. д.), используя HTTP-заголовки `Accept` и `Content-Type`. Этот акцент на сообщениях (см. рецепт 4.3) придаст клиентскому коду стабильность, не привязывая его слишком тесно к конкретному контексту применяемых сервисов.

РАЗРАБОТКА НА ОСНОВЕ МЕДИАТИПОВ

Первый шаг в управлении форматами сообщений в веб-среде — это использование заголовков `Accept` и `Content-Type`. Они позволяют указать, какой формат сообщений ожидается от сервиса, и проверить возвращаемое этим сервисом представление.

Когда клиент делает запрос к API, в него нужно включить заголовок `Accept`, указывающий структурированный формат (-ы), который понимает приложение. Затем, когда сервер ответит, нужно проверить представленный им заголовок `Content-Type`, указывающий поддерживаемые форматы. Если клиент не понимает возвращенного формата, следует остановить обработку и сообщить о проблеме. В приложениях, управляемых людьми, это обычно делается с помощью всплывающего сообщения или встроенного содержания. В случае M2M-приложений данный процесс можно

реализовать в виде отправки вызывающей машине сообщения об ошибке и/или контролирующему субъекту сообщения журнала:

```
*** REQUEST
GET /todo/list HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json

*** RESPONSE
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
...
```

Кроме того, клиентские приложения нужно программировать, разделяя зону ответственности (separation of concern, SoC) между форматами сообщений, которыми обмениваются сервисы, и внутренним представлением этой информации. Когда сервер возвращает ответ, клиентское приложение может перенаправить тело этого сообщения нужному обработчику. Именно здесь сообщение переводится во внутреннюю модель, которую клиент сможет использовать для инспектирования запроса и управления им.

Это означает внутреннюю реализацию паттерна Message Translator (транслятор сообщений), чтобы клиенты могли согласованно выполнять преобразование между внутренними объектными моделями и внешними сообщениями.

Пример

Вот пример кода, проверяющего медиатип ответа сервиса и соответствующим образом обрабатывающего его:

```
function handleResponse(ajax,url) {
  var ctype
  if(ajax.readyState===4) {
    try {
      ctype = ajax.getResponseHeader("content-type").toLowerCase();
      switch(ctype) {
        case "application/vnd.collection+json":
          cj.parse(JSON.parse(ajax.responseText));
          break;
        case "application/vnd.siren+json":
          siren.parse(JSON.parse(ajax.responseText));
          break;
        case "application/vnd.hal+json":
          hal.parse(JSON.parse(ajax.responseText));
          break;
        default:
          dump(ajax.responseText);
          break;
      }
    }
  }
}
```

```
        catch(ex) {  
            alert(ex);  
        }  
    }  
}
```

Заметьте, что данный клиент может общаться, используя три формата: Collection + JSON, SIREN и HAL. Если сервис вернет какой-то другой, приложение отправит его ответ в процедуру `dump()` для дополнительной обработки.

Обсуждение

Организация клиентского кода на основе форматов сообщений лучше всего работает, когда сервисы поддерживают хорошо структурированные форматы вроде HTML, HAL, SIREN, Collection + JSON и пр. Если сервисы применяют неструктурированные форматы ответов вроде XML и JSON, то для поддержки управляемого сообщениями клиентского кода вам потребуются дополнительные метаданные. В этом случае пригодятся документы схем наподобие XML Schema (<https://oreil.ly/aVNKK>) и JSON Schema (<https://oreil.ly/Qwi4k>). Подробнее об использовании схем на клиентской стороне читайте в рецепте 4.7.

Определение формата входящего сообщения — это лишь первый шаг. Нужно также иметь возможность спарсить и зачастую перевести эти сообщения для внутреннего применения. Для браузеров можно создать одну JavaScript-библиотеку, преобразующую входящие форматы JSON (HAL, SIREN, Collection + JSON) в HTML для рендеринга. Далее приведены высокоуровневые методы для преобразования сообщений API на основе JSON в HTML:

```
// collection+JSON-->HTML  
function parse(collection) {  
    var elm;  
  
    g.cj = collection;  
  
    title();  
    content();  
    links();  
    items();  
    queries();  
    template();  
    error();  
  
    elm = d.find("cj");  
    if(elm) {  
        elm.style.display="block";  
    }  
}
```

```
// HAL --> HTML
function parse(hal) {
    g.hal = hal;

    title();
    setContext();
    if(g.context!="") {
        selectLinks("app", "toplinks");
        selectLinks("list", "h-links");
        content();
        embedded();
        properties();
    }
    else {
        alert("Unknown Context, can't continue");
    }

    elm = d.find("hal");
    if(elm) {
        elm.style.display="block";
    }
}

// SIREN --> HTML
function parse(msg) {
    var elm;

    g.siren = msg;

    title();
    getContent();
    links();
    entities();
    properties();
    actions();

    elm = d.find("siren");
    if(elm) {
        elm.style.display="block";
    }
}
```

Каждый парсер сообщений создан с учетом преобразования выбранного формата в HTML для рендеринга в браузере. Это пример реализации паттерна Message Translator (<https://oreil.ly/Q3Pan>). Писать такие односторонние трансляторы для высокоструктурированных форматов несложно. Поскольку цель, то есть HTML, не относится к конкретной предметной области, один транслятор может оказаться эффективным для любого тематического приложения или контекста. Я всегда держу под рукой браузерный парсер медиатипов, чтобы иметь возможность быстро создавать приложения рендеринга для сервисов, использующих эти форматы.

Эта задача усложняется, когда нужно перевести входящее сообщение во внутреннюю объектную модель или граф для конкретной предметной области. В таких

случаях хорошо сработает применение других метаданных наподобие семантических профилей (см. рецепт 4.4) или схем (см. рецепт 4.7) в качестве руководства для перевода входящих сообщений.

Еще больше задача усложняется, когда все, с чем нужно работать вашему клиенту, является неструктурированными документами вроде XML или JSON и отсутствует надежная схема или профиль. В таких случаях вам потребуется больше времени уделить созданию собственных трансляторов на основе понятной человеку документации. При этом вероятно, что небольшие изменения в документации могут создать серьезные проблемы для транслятора.

Читайте также

- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.
- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 5.6. Поддержка согласования HTTP-содержимого.

4.7. Использование документов схем в качестве источника метаданных сообщений

Когда сервисы не поддерживают семантические профили или структурированные медиатипы, у клиентских приложений все равно есть возможность запрограммировать семантическую спецификацию, если этот сервис возвращает ссылки на схемы, описывающие содержимое сообщения:

- задействовать схему, в которой профилей и медиатипов недостаточно либо они отсутствуют;
- *не использовать* схемы для проверки *входящих* сообщений;
- *использовать* схему для проверки *исходящих* сообщений.

Задача

Как создавать устойчивые клиентские приложения на основе схемы сообщений для неструктурированных медиатипов (XML или JSON) вместо семантических профилей?

Решение

В случаях, когда сервисы последовательно возвращают документы схем с неструктурированными форматами сообщений (XML или JSON) в виде ответов в среде выполнения, вы можете написать устойчивое приложение без поддержки семантических профилей. Для этого потребуется организовать код клиента, управляемый исключительно информацией схемы.

Наиболее распространенными примерами таких форматов схем являются XML Schema (<https://oreil.ly/aVNKK>) и JSON Schema (<https://oreil.ly/Qwi4k>).

Пример

Когда сервисы предоставляют документы схем, можно использовать их также в качестве дополнительных метаданных для входящих сообщений. Лучше всего это работает, когда сам ответ содержит указатель на схему. В среде выполнения нежелательно проверять входящие сообщения, применяя схему, если только вы не планируете отклонять все недопустимые входные данные. Как правило, ответы в среде выполнения будут иметь небольшие различия (например, отсутствие дополнительных полей, иной порядок элементов в ответе и т. д.), которые не являются вескими причинами для отклонения входящего сообщения. Тем не менее удобно использовать предоставленные URI/URL схем в качестве *идентификатора* для подтверждения того, что сервис вернул ответ с ожидаемыми семантическими метаданными.



Использование документов схем для проверки входящих сообщений идет вразрез с законом Постеля (<https://oreil.ly/VWvC6>), который гласит: «Будь консервативен в том, что делаешь, и либерален в том, что принимаешь от других». Строгая типизация входящих сообщений (вместо того чтобы «быть либеральным в том, что принимаешь») наверняка приведет к отклонению ответов, которые в ином случае отлично сработали бы для клиента и сервера. Спецификация RFC 1122 (<https://oreil.ly/kSM3w>), составленная Робертом Брэнденом, подробно разбирает проектирование и реализацию возможностей, которые могут безопасно допускать небольшие вариации деталей сообщений и протокола. Более развернуто эта тема рассматривается в рецептах 4.12 и 4.13.

Согласование схем

Клиентские приложения могут согласовывать с сервисом использование схем. Этот прием полезен, когда и клиент, и сервер заранее знают, что есть несколько версий одной общей схемы (user-v1, user-v2 и т. д.), и вы хотите обеспечить, чтобы обе стороны работали с одной и той же версией. В устаревшей заявке (<https://oreil.ly/f2jS2>) от W3C предлагалось применять HTTP-заголовки `accept-schema` и `schema`:

```
*** REQUEST
GET /todo/list HTTP/1.1
Host: api.example.org
Accept-Schema: <urn:example:schema:e-commerce-payment>
```



```
*** RESPONSE
HTTP/1.1 200 OK
Schema: <urn:example:schema:e-commerce-payment>
...
```

В рабочей среде я такого подхода к согласованию не встречал. Если он предлагается сервисом, разработчику клиентского приложения стоит изучить его — любые семантические данные лучше, чем ничего.

Идентификатор схемы в заголовке Content-Type

Идентификатор схемы также можно передавать от сервера клиенту в заголовке Content-Type:

```
*** REQUEST
GET /todo/list
HTTP/1.1
Host: api.example.org
Accept: application/vnd.hal+json

*** RESPONSE HTTP/1.1 200 OK
Content-Type: application/vnd.hal+json;schema=urn:example:schema:e-commerce-payment
...
```

Важно отметить, что добавление дополнительных граней (после ;) в строку медиатипа может усложнить проверку формата сообщений. Кроме того, некоторые определения медиатипов явно запрещают включение дополнительных метаданных в строку их идентификаторов (например, это правило действует для JSON).

Идентификатор схемы в документе

Чаще всего идентификатор схемы указывается в теле сообщения:

```
{
  "$schema": "https://alps-io.github.io/schemas/alps.json",
  "alps" : {
    "version": "1.0",
    "title": "ToDo List",
    "doc" : { "value": "A suggested ALPS profile for a ToDo service" },
    "descriptor": [
      { "id": "id", "type": "semantic", "def": "http://schema.org/identifier" },
      { "id": "title", "type": "semantic", "def": "http://schema.org/title" },
      { "id": "completed", "type": "semantic",
        "def": "http://mamund.site44.com/alps/def/completed.txt" }
    ]
  }
}
```

Данный идентификатор необязательно должен быть разыменовываемым URL-адресом (<https://alpsio.github.io/schemas/alps.json>). Это может быть просто URI- или URN-строка идентификаторов (`urn:example:schema:e-commerce-payment`). Полезность применения разыменовываемого URL заключается в том, что человек может перейти по этой ссылке для получения дополнительной информации.

Программирование клиентов с поддержкой схем

Разобравшись с тем, как идентифицировать информацию схемы, можно использовать ее для управления кодом приложения:

```
function handleSchema(ajax, schemaIdentifier) {
    var schemaType
    try {
        schemaType = schemaIdentifier.toLowerCase()
        switch(ctype) {
            case "https://api.example.com/schemas/task":
                task.parse(JSON.parse(ajax.responseText));
                break;
            case "https://api.example.com/schemas/task-v2":
                task.parse(JSON.parse(ajax.responseText));
                break;
            case "https://api.example.com/schemas/user":
            case "https://api.example.com/schemas/user-v2":
                user.parse(JSON.parse(ajax.responseText));
                break;
            case "https://api.example.com/schemas/note":
                note.parse(JSON.parse(ajax.responseText));
                break;
            default:
                dump(ajax.responseText);
                break;
        }
    }
    catch(ex) {
        alert(ex);
    }
}
```

Заметьте, что организация кода на основе определений схемы зачастую требует намного большей работы, чем при использовании форматов семантических профилей вроде ALPS. Схемы обычно связаны с конкретной предметной областью и более склонны к изменениям, чем семантические профили. В этом примере можно видеть две очевидно несовместимые ссылки на объектные схемы `task`. Кроме того, в нем показано, что, несмотря на присутствие двух разных схем `user`, их версии (по крайней мере, насколько это известно клиенту) совместимы.

Обсуждение

Основная ценность схем проявляется при создании и отправке сообщений (см. рецепт 4.12). Тем не менее клиентские приложения могут использовать сами идентификаторы схем (URL или URI) в качестве рекомендуемых значений для обработки входящих сообщений. Эти идентификаторы, по сути, становятся значениями, подтверждающими, что полученный клиентом ответ будет содержать информацию, необходимую ему для выполнения своей работы.



В этой редакции книги я не буду говорить о ведении реестров схем. Но я рассказал о пользе зарегистрированных словарей (намного менее ограниченных в сравнении со схемами) в рецептах 3.4 и 4.4.

Полезность согласования идентификаторов схем тоже ограничена. Согласование может стать хорошим способом для клиента сообщить сервисам о том, какую схему он поддерживает. Это позволяет сервисам при необходимости *отклонить* запросы с помощью HTTP-кода состояния **406 Not Acceptable**. Но этот подход также противоречит закону Постеля.

Клиентские приложения, запрограммированные на использование схем, создадут больше проблем, чем те, которые созданы для применения медиатипов (см. рецепт 4.3) или профилей (см. рецепт 4.4). Причина в том, что документы схем обычно представляют намного большую детализацию (объекты), а схема объектов, как правило, изменяется чаще, чем общий словарь или формат сообщений.

JSON Schema позволяет успешно выполнять валидацию образца документа, даже если у него есть дополнительные свойства, отсутствующие в схеме (`additionalProperties: true|false`). По умолчанию этот параметр установлен как `true`, что позволяет дополнительным свойствам появляться, не вызывая ошибку при проверке схемы. Кроме того, изменение порядка свойств в объекте тоже не вызовет ошибку при проверке.



Включение во входящее сообщение неизвестных свойств само по себе угрозы для безопасности не представляет. Например, это нередко происходит в формате HTML. А вот попытка получить и обработать неизвестные свойства *может* создать уязвимое место. Как вы увидите в рецепте 4.13, клиентским приложениям лучше игнорировать неизвестные свойства и обрабатывать только те значения, обработка которых предусмотрена их проектом.

В XML-документах валидаторы схем довольно требовательны. Для прохождения проверки элементы XML-сообщений должны располагаться в том же порядке, который указан в документе схемы, и в большинстве случаев появление нового элемента или свойства будет вызывать ошибку валидации.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 4.12. Использование документов схем для валидации исходящих сообщений.
- Рецепт 4.13. Использование запросов документов для проверки входящих сообщений.

4.8. Каждому важному элементу ответа нужен идентификатор

По аналогии с тем, что у каждого важного ресурса в RESTful API есть идентификатор (URL), его заслуживают и все важные действия или элементы данных ответа. У клиента должна быть возможность не только обнаружить правильный ресурс, но и найти и применить правильное действие или коллекцию данных в нем.

Задача

Как сделать так, чтобы клиентские приложения могли находить в ответе нужные им действия и/или данные? Кроме того, как обеспечить сохранение этой возможности в процессе развития сервиса?

Решение

Для обеспечения наличия в ответах API стабильных и понятных идентификаторов важно выбирать медиатипы, которые структурно поддерживают разные типы идентификаторов. И здесь отлично поможет HTML. В качестве примера медиатипа, поддерживающего несколько структурных идентификаторов, я создал формат Machine Accessible Semantic Hypermedia (MASH).

Вот эти четыре типа.

ID

Элемент уровня документа, имеющий одно значение, например `id=1q2w3e4r`.

NAME

Элемент уровня приложения, имеющий одно значение, например `name=createUserForm`.

REL

Системный элемент со множеством значений, например `rel=create-form self`.

TAG

Неуникальный элемент со множеством значений, созданный под конкретное решение, например `tag=users page-level`.

Еще одним примером последнего вида идентификаторов является HTML-элемент CLASS.

Клиентские приложения нужно программировать так, чтобы они могли найти нужный блок FORM, LINK или DATA, используя не менее одного вида этих идентификаторов.

Пример

В качестве примера важности применения медиатипов с поддержкой структурированных идентификаторов рассмотрим следующую задачу.

1. Получить доступ к начальному URL сервиса `users`.
2. Внутри ответа на этот запрос найти форму для поиска существующих пользователей.
3. Заполнить эту форму, чтобы найти пользователя с ником `mingles` и выполнить запрос.
4. Сохранить итоговые свойства этого ответа в памяти клиента.
5. Внутри ответа на запрос найти форму для обновления ресурса.
6. Используя данные из ответа, заполнить форму для обновления, изменить адрес email на `mingles@example.org` и выполнить запрос.

Человек может проделать все это без особых проблем (например, в HTML-браузере), а вот запрограммировать на аналогичные действия машину будет куда сложнее. Представьте, что вам нужно обновить адреса электронной почты для десяти или более аккаунтов. А теперь прибавьте к этой задаче необходимость обработать, например, несколько версий пользовательского сервиса либо сервис с поддержкой нескольких форматов ответов или изменившийся со временем. В этих случаях клиент может применить такой скрипт:

```
# Пример hyper-cli

ECHO Updating email address for mingles

ACTIVATE http://api.example.org/users

ACTIVATE WITH-FORM nickSearch
  WITH-QUERY {"nick":"mingles"}

ACTIVATE WITH-FORM userUpdate
  WITH-BODY {"name":"Miguelito","nick":"mingles","email":"mingles@example.org"}

ACTIVATE WITH-FORM emailSearch
  WITH-QUERY {"email":"mingles@example.org"}

ECHO Update Confirmed

EXIT

# eof
```



Подробнее использование HyperCLI и HyperLANG в качестве клиентского приложения с поддержкой гипермедиа описано в приложении Г.

Обратите внимание на то, что здесь клиентское приложение применяет идентификаторы для трех действий: `nickSearch`, `userUpdate` и `emailSearch`, а также для коллекции свойств `name`, `nick` и `email`. Реальный формат представления (HTML, MASH, SIREN и т. д.) в этом скрипте неважен — он обрабатывается клиентом. Также заметьте, что тут передается единственный изначальный URL для связи с сервисом. Остальные URL передаются в ответах (внутри форм).

Обсуждение

Большинство практик проектирования HTTP отчетливо показывают, что каждый ресурс заслуживает собственного идентификатора в виде URL. Это легко увидеть по типичным HTTP-запросам к API:

```
http://api.example.org/customers/123
http://api.example.org/users?customer=123
```

Это верно и когда дело доходит до описания взаимодействий внутри одного ответа. Рассмотрим в качестве примера следующий ответ от `/users?customer=123`:

```
{
  "id": "aqswdefrgt",
  "href": "http://api.example.org/customers/123",
  "name": "customer-record",
  "rel": "item http://rels.example.org/customer",
  "tags": "item customer read-only",
  "data": [
    "identifier": "123",
    "companyName": "Ajax Brewing",
    "address": "123 Main St, Byteville, MD",
    "users": "http://api.example.org/users?customer=123"
  ]
}
```

Заметьте, что в этой записи присутствуют следующие метаданные.

id

ID на уровне документа, который клиенты могут использовать для обнаружения этой записи в любом списке.

href

Системный URL, который клиенты могут применять для извлечения этой записи из сервиса.

name

Семантическое имя на уровне приложения, по которому клиенты могут получать записи этого типа.

rel

Коллекция имен на уровне приложения, которые клиенты могут использовать для получения записей этого типа или экземпляра.

tags

Коллекция имен на уровне приложения, по которым клиенты могут отфильтровывать записи различным образом.

Добавленный здесь пример `href` показывает, что системный идентификатор (`http://api.example.org/customers/123`) необязательно должен совпадать с внутренним `id` (`aqswdefrgt`). В действительности правильно будет *предположить* их разделенность, чтобы избежать проблем при изменении URL вследствие переезда сервиса, установки новых прокси-серверов и пр.

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 5.14. Повышение скорости обработки с помощью клиентских идентификаторов.
- Рецепт 7.13. Использование наблюдения состояния для реализации рабочего потока под управлением клиента.

4.9. Использование средств гипермедиа в ответе

Одним из принципов REST-сервисов является применение гипермедиа в качестве двигателя состояния приложения, или, как это назвал Рой Филдинг, обусловленное гипермедиа ограничение. Сервисы, которые отправляют ответы в форматах гипермедиа, значительно упрощают для клиентских приложений определение в *среде выполнения* возможных действий и способа их реализации. При этом информация гипермедиа, которая приводится в ответе, может быть чувствительной к контексту, например к текущему состоянию сервиса (запись уже существует?), пользовательскому контексту (есть ли у пользователя права на редактирование записи?) и контексту самого запроса (не редактировал ли кто-либо уже эти данные?).

Задача

Когда потребитель API получает от провайдеров сервиса ответы на основе гипермедиа, как ему задействовать включенную в них информацию таким образом, чтобы упростить ее обработку для людей и машин?

Решение

При создании потребителей API в них нужно закладывать понимание форматов ответов (см. рецепт 4.3). В случае типов гипермедиа у каждого формата есть его собственная подпись гипермедиа — список элементов гипермедиа, называемых факторами гипермедиа или Н-факторами (<https://oreil.ly/WQfg1>), которые этот медиатип может выразить. Создание клиентских приложений, знающих этот медиатип и понимающих девять Н-факторов, значительно повысит гибкость и устойчивость клиентского приложения. Эти девять Н-факторов (см. табл. 4.1 и 4.2) могут появляться в ответе RESTful-сервиса.

Таблица 4.1. Пять связывающих факторов

Название	Описание	Пример
LE, встраиваемая ссылка	Добавляет содержание в текущее представление	HTML <code></code> tag
LO, исходящая ссылка	Переносит клиента в новое представление	HTML <code><a /></code> tag
LI, шаблон ссылок	Поддерживает добавление параметров перед выполнением	Шаблоны URI <code>http://example.org/users/{id}</code>
LN, неидемпотентная ссылка	Описывает неидемпотентное действие	<code><FORM method=POST ...></code>
LI, идемпотентная ссылка	Описывает идемпотентное действие	SIREN <code>"actions": [{"name": "update-item", "method": "put" ...}]</code>

Таблица 4.2. Четыре управляющих фактора

Название	Описание	Пример
CR, элемент управления для запросов на чтение	Описывает подробности чтения	<code>Accept:application/vnd.hal+json</code>
CU, элемент управления для запросов на запись	Описывает подробности записи	<code>enctype="application/x-www-form-urlencoded"</code>
CM, элемент управления для HTTP-методов	Описывает, какой HTTP-метод использовать	<code><FORM method="GET" ... ></code>
CL, элемент управления для ссылочных связей	Описывает ссылочную связь действия	<code><link rel="create-form" ... /></code>

У каждого формата гипермедиа есть собственная подпись.

На рис. 4.2 представлены три медиатипа: SVG, Atom и HTML. Заметьте, что формат SVG поддерживает только факторы LE и LO, например встраиваемые ссылки вроде `<IMAGE href="...">` и исходящие ссылки наподобие `<A href="...">Go Home</a>`.

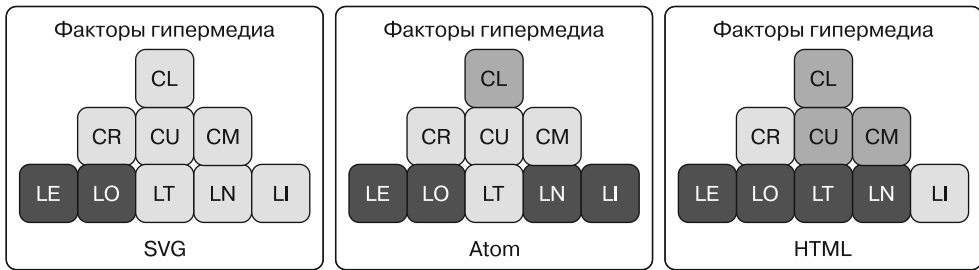


Рис. 4.2. Наглядное представление факторов гипермедиа различных медиатипов

Пример

Понимание вашим клиентом того, с какими форматами он, скорее всего, столкнется, поможет ему распознать и при необходимости предложить человеку или машине варианты, которые позволят им выполнить свою задачу.

Например, клиент HyperCLI (см. приложение Г) поддерживает ряд форматов гипермедиа. При использовании формата наподобие SIREN вы можете запрограммировать этого клиента на добавление, поиск, изменение или удаление содержания на основе указанных в ответах средств гипермедиа. Далее приводится короткий скрипт SIREN, написанный на HyperLANG.

```
#
# Сеанс редактирования SIREN
# прочитать запись, сохранить ее, изменить и записать обратно на сервер
#

# ** выполнение первого запроса
REQUEST WITH-URL http://rwcbook10.herokuapp.com

# ** извлечение первой записи в списке
REQUEST WITH-PATH $.entities[0].href

# ** передача свойства элемента в стек
STACK PUSH WITH-PATH $.properties

# ** изменение значения свойства tags в стеке
STACK SET {"tags":"fishing,\\.skiing,\\.hiking"}

# ** использование переданной формы редактирования и обновленного стека
# для отправки изменений
REQUEST WITH-FORM taskFormEdit WITH-STACK

# ** подтверждение изменения
SIREN PATH $.entities[0]

# ** завершение сеанса
EXIT
```



В приложении Г приводится краткое введение в программирование на HyperLANG с помощью Hyper CLI.

Заметьте, что в этом примере был передан только стартовый URL. Остальные действия выполнялись на основе метаданных гипермедиа, полученных в каждом ответе сервиса.

Обсуждение

Этот рецепт отражает принцип *использования* данных гипермедиа для решения проблем напрямую. Он опирается на передаваемые сервисами подробности, поэтому пригодится, только когда сервисы поддерживают форматы гипермедиа в ответах.

Преимущество использования передаваемых в скриптах гипермедиаданных заключается в том, что это снижает объем жестко прописываемых инструкций в клиентском приложении. Вы можете упростить для сервисов изменение деталей гипермедиа (например, URL, HTTP-методов, поддерживаемых форматов и т. д.), не нарушая работы клиентского приложения. В предыдущем примере скрипт не прописывает жестко HTTP URL, метод или тип кодировки для действия `taskFormEdit`. Он просто задействует передаваемые сервисом детали.

Этот рецепт лучше всего работает, когда клиентское приложение понимает не только формат ответа (например, SIREN), но и используемый сервисом семантический профиль (тематическую область). Так, в приведенном ранее коде клиент заранее знает, что есть средство гипермедиа `taskFormEdit`, которое он может применять для изменения существующей записи. Подробнее о семантических профилях сказано в рецепте 4.4.

Подпись Н-фактора в форматах гипермедиа может широко изменяться и будет влиять на то, требуется или нет жестко прописывать инструкции в клиенте API. Например, формат SIREN поддерживает весь диапазон Н-факторов, а HAL — лишь некоторые из них. В противоположность этому не-гипермедиаформат вроде `application/json` не поддерживает встроенные Н-факторы, усложняя тем самым написание устойчивых клиентов API для сервисов, которые поддерживают только простые ответы JSON.

Как правило, чем больше Н-факторов поддерживает медиатип, тем с большей вероятностью у вас получится клиент, способный спокойно переживать изменения деталей гипермедиа в будущем.

Читайте также

- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.
- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.

- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.7. Использование медиатипов для запросов данных.

4.10. Поддержка ссылок и форм для сервисов без поддержки гипермедиа

Бывают случаи, когда вам нужно взаимодействовать с сервисами, которые не предлагают ссылок и форм, то есть ответов в стиле гипермедиа. В таких сценариях можно применять на стороне клиента удобный подход, который позволит компенсировать отсутствие поддержки гипермедиа.

Задача

Как получить преимущества отделения API и четко описанных взаимодействий, доступных при использовании ответов в стиле гипермедиа, когда сервис, с которым вы работаете, такие форматы ответов не поддерживает?

Решение

Когда вы работаете с сервисами, которые не предлагают ответы в стиле гипермедиа, но все же хотите иметь возможность использовать на стороне клиента четко описанные ссылки и формы, можете запрограммировать свое клиентское приложение на передачу в среде выполнения собственных ссылок и форм, исходя из исходной документации API.

Пример

Клиент API на основе гипермедиа получает ответ, определяет в нем элементы действий (ссылки и формы) и отрисовывает их (в случае UI для человека) вместе с полученными данными. Высокоуровневое представление кода такого клиента будет следующим:

```
// Низкоуровневые детали HTTP
function req(url, method, body) {
  var ajax = new XMLHttpRequest();
  ajax.onreadystatechange = function(){rsp(ajax)};
  ajax.open(method, url);
  ajax.setRequestHeader("accept",g.atype);
  if(body && body!==null) {
```

```

        ajax.setRequestHeader("content-type", g.ctype);
    }
    ajax.send(body);
}

function rsp ajax {
    if(ajax.readyState===4) {
        g.msg = JSON.parse(ajax.responseText);
        showTitle();
        showToplinks();
        showItems();
        showActions();
    }
}

```

Когда ответ получен, клиент просматривает тело его сообщения и обрабатывает все ключевые задачи. К примеру, так может выглядеть метод `showTopLinks()`:

```

// Отправка ссылок для всех представлений
function showToplinks() {
    var act, actions;
    var elm, coll;
    var menu, a;

    elm = d.find("toplinks");
    d.clear(elm);
    menu = d.node("div");

    actions = g.actions[g.object]; // Получение метаданных ссылок
    for(var act in actions) {
        link = actions[act]
        if(link.target=="app") {
            a = d.anchor({
                href:link.href,
                rel:(link.rel||"collection"),
                className:"action item",
                text:link.prompt

            });
            a.onclick = link.func;
            d.push(a, menu);
        }
    }
    d.push(menu,elm);
}

```

Этот код отрисует верхние ссылки в пользовательском интерфейсе. Источником для отрисовки послужат метаданные ссылок. Этот клиент получает их в одной строке кода. В ответах с гипермедиа эти метаданные находятся в теле сообщений. Но даже если вы получаете только простые ответы с данными в формате JSON, можете применять тот же код. Нужно лишь изменить источник поступления метаданных ссылок.

В приведенном далее примере эти метаданные поступают из внутренней структуры, которая содержит все ссылки и формы для конкретного контекста (users, tasks и т. д.). Все эти метаданные были получены из предназначенной для человека документации API. Правила возможных действий и аргументов были перенесены из документации в структуру клиентского кода:

```
// Действия с пользовательским объектом
g.actions.user = {
  home:      {target:"app", func:httpGet, href:"/home/", prompt:"Home"},
  tasks:     {target:"app", func:httpGet, href:"/task/", prompt:"Tasks"},
  users:     {target:"app", func:httpGet, href:"/user/", prompt:"Users"},
  byNick:    {target:"list", func:jsonForm, href:"/user",
              prompt:"By Nickname", method:"GET",
              args:{
                nick: {value:"", prompt:"Nickname", required:true}
              }
            },
  byName:    {target:"list", func:jsonForm, href:"/user",
              prompt:"By Name", method:"GET",
              args:{
                name: {value:"", prompt:"Name", required:true}
              }
            },
  add:       {target:"list", func:jsonForm, href:"/user/",
              prompt:"Add User", method:"POST",
              args:{
                nick: {value:"", prompt:"Nickname", required:true,
                      pattern:"[a-zA-Z0-9]+"},
                password: {value:"", prompt:"Password", required:true,
                           pattern:"[a-zA-Z0-9!@#$$%^&*~]+"},
                name: {value:"", prompt:"Full Name", required:true},
              }
            },
  item:      {target:"item", func:httpGet, href:"/user/{id}", prompt:"Item"},
  edit:      {target:"single", func:jsonForm, href:"/user/{id}",
              prompt:"Edit", method:"PUT",
              args:{
                nick: {value:"{nick}", prompt:"Nickname", readOnly:true},
                name: {value:"{name}", prompt:"Full Name", required:true}
              }
            },
  changepw:  {target:"single", func:jsonForm, href:"/task/pass/{id}",
              prompt:"Change Password", method:"POST",
              args:{
                nick: {value:"{nick}", prompt:"NickName", readOnly:true},
                oldPass: {value:"", prompt:"Old Password", required:true,
                          pattern:"[a-zA-Z0-9!@#$$%^&*~]+"},
                newPass: {value:"", prompt:"New Password", required:true,
                          pattern:"[a-zA-Z0-9!@#$$%^&*~]+"},
                checkPass: {value:"", prompt:"Confirm New", required:true,
                            pattern:"[a-zA-Z0-9!@#$$%^&*~]+"},
              }
            },
}
```

```
assigned: {target:"single", func:httpGet, href:"/task/?assignedUser={id}",
           prompt:"Assigned Tasks"}
};
```

Теперь вы можете использовать ссылки и формы для сервисов, которые не предоставляют их в среде выполнения.

Обсуждение

Этот рецепт опирается на необходимость проделать работу по переводу правил из документации API в машиночитаемую форму. Именно это делают гипермедиа-совместимые сервисы. Если сервис не выполняет эту работу автоматически, разработчики клиента могут сэкономить массу времени, сделав ее один раз и поделившись результатами. Рассмотрите вариант размещения всех метаданных ссылок и форм для сервиса в отдельном модуле или внешнем файле конфигурации. А лучше по возможности убедите команду сервера сделать это за вас.

Некоторые форматы спроектированы специально для захвата и передачи метаданных ссылок и форм. Одним из примеров является язык перевода веб-служб (Web Service Transition Language, WSTL). Вам также может быть доступно использование JSON HyperSchema. Подробнее об этом — в разделе «Сопутствующие форматы гипермедиа» приложения В.

Но есть здесь и ряд сложностей, которые в описанном примере не раскрыты. Самая значительная — это управление пользовательским контекстом. Одним из примеров является случай, когда администраторам должны быть доступны действия, которые недоступны гостевым пользователям. В этом случае надежным подходом будет создание отдельного набора метаданных действий для каждой роли безопасности, которые ваше приложение поддерживает.

Еще одной проблемой, с которой придется столкнуться, является случай, когда сервис изменяется, но локальные сведения о метаданных за ним не успевают. Это еще один пример проблем с версионированием, возникающих, когда клиенты работают со статическими метаданными в среде выполнения. Хорошо, что при изменении правил действий на сервере клиентскому приложению нужно обновить только метаданные конфигурации, а не всю базу кода приложения.

Читайте также

- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.
- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.

- Рецепт 4.6. Управление форматами представления в среде выполнения.
- Рецепт 5.3. Преобразование внутренних моделей во внешние сообщения.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.

4.11. Проверка свойств данных в среде выполнения

При поддержке набора входных данных для клиентских приложений бывает сложно правильно описать их валидные значения в среде выполнения. При этом использование предопределенных правил, задокументированных только в понятном для человека виде, может со временем ограничить адаптивность клиентского приложения и уменьшить его полезность.

Задача

Как клиентам API последовательно обеспечивать использование правильных свойств метаданных во входных значениях в среде выполнения? Кроме того, как обеспечить, чтобы изменяемые со временем правила ввода продолжали учитываться клиентами API и эти изменения не нарушили работу уже развернутых клиентов?

Решение

Эффективным подходом к реализации клиентами API проверки свойства данных в среде выполнения будут распознавание и учет расширенных описаний входных данных (rich input descriptions, RID) внутри HTTP-ответов. Отличный пример RID можно встретить в формате HTML. Этот формат для описания ожидаемых входных данных (`INPUT.type`, <https://oreil.ly/kr2Mp>) и определения допустимых значений (<https://oreil.ly/clkaO>) `INPUT.pattern`, `INPUT.required`, `INPUT.size` и т. д. опирается на элемент `INPUT` и коллекцию атрибутов.

Клиентские приложения должны поддерживать любые проверки качества данных, предоставляемые входящим форматом сообщений (HAL, SIREN, Collection + JSON и т. д.).

Пример

Спецификация HAL-FORMS (<https://oreil.ly/ID2t7>) — хороший пример формата API с поддержкой RID. HAL-FORMS — это расширение медиатипа HAL, разработанного специально для добавления поддержки RID в формат HAL. Эта спецификация разделяет RID на три группы: основные, дополнительные и специальная категория — опции.

В основном разделе приводятся следующие элементы, которые клиенты *должны* поддерживать: `readOnly`, `regex`, `required` и `templated`. В дополнительном указаны элементы, которые клиенты *могут* поддерживать: `cols`, `max`, `maxLength`, `min`, `minLength`, `placeholder`, `rows`, `step` и `type`. Опции же — это отдельный класс для проверки входных данных, который реализует описательные перечисления, например `small`, `medium`, `large` и т. п.

Вот пример описания опций из HAL-FORMS:

```
{
  "_templates" : {
    "default" : {
      ...
      "properties" : [
        {
          "name" : "shipping",
          "type" : "radio",
          "prompt" : "Select Shipping Method",
          "options" : {
            "selectedValues" : ["FedEx"],
            "inline" : [
              {"prompt" : "Federal Express", "value" : "FedEx"},
              {"prompt" : "United Parcel Service", "value" : "UPS"},
              {"prompt" : "DHL Express", "value" : "DHL"}
            ]
          }
        }
      ]
    }
  }
}
```

Обсуждение

Формат SIREN поддерживает широкий спектр RID (<https://oreil.ly/GUNOr>) путем простого обращения к HTML-спецификации в отношении типов входных данных (<https://oreil.ly/TOWGA>).

RID зачастую можно использовать для определения *рендеринга* входного элемента управления, например, выпадающего списка, выбора даты и т. д. Этот подход нередко применяется, когда сообщение ответа получает клиент API, предназначенный для использования человеком, но может игнорироваться, когда клиент создан для M2M-взаимодействий.

Поддержка свойства регулярных выражений для описания валидных входных данных может сработать во многих случаях и как минимум уменьшить нагрузку на клиентское приложение. Это особенно актуально для M2M-взаимодействий, когда нет потребности в подсказках относительно отрисовки пользовательского интерфейса.

Если ответ сервиса не включает встроенные RID среды выполнения, разработчикам клиента нужно обратиться к документации и заложить в потребителя API поддержку описанных там видов RID. Как правило, это означает либо создание встроенной в код реализации RID, либо применение конфигурации. Вне зависимости от деталей реализации желательно преобразовать человекочитаемые правила проверки входных данных в машиночитаемые алгоритмы. Подробнее об этом — в рецепте 4.10.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 4.8. Каждому важному элементу ответа нужен идентификатор.
- Рецепт 4.10. Поддержка ссылок и форм для сервисов без поддержки гипермедиа.
- Рецепт 4.12. Использование документов схем для валидации исходящих сообщений.
- Рецепт 4.13. Использование запросов документов для проверки входящих сообщений.
- Рецепт 4.14. Валидация входящих данных.
- Рецепт 6.8. Игнорирование неизвестных полей данных.

4.12. Использование документов схем для валидации исходящих сообщений

Для клиентского приложения важно стараться *отправлять* на серверы валидные сообщения (особенно *тела* сообщений). Одним из способов обеспечить отправку валидных тел сообщений является их проверка с помощью документа схемы. Но всегда ли будет удачной идея задействовать проверку схемы при отправке тел запроса?

Задача

Как клиентскому приложению обеспечить отправку правильно сформированных и валидных тел запросов при совершении запросов к сервисам?

Решение

Отправка хорошо сформированных и валидных тел запросов с помощью HTTP-методов вроде POST, PUT и PATCH очень важна. Чтобы уменьшить вероятность отклонения запроса, следует проверять структуру и содержимое его тела до отправки в сервис на обработку. По сути, ответственность за правильное формирование

валидного тела запроса лежит на *клиентском* приложении, а за подбирающую обработку всех валидных запросов — на *сервисе*. Это пример выполнения закона Постеля (он же принцип надежности, <https://oreil.ly/ctpPI>).

Очень эффективным способом выполнить это требование будет подтверждение валидности тела сообщения с помощью документов схем. И у JSON, и у XML есть строгие спецификации схем. Причем существует также множество инструментов для реализации их поддержки.

Клиентам нужно обрабатывать два уровня подтверждения, а именно: проверять правильность составления документа и его валидность. Правильно составленные документы имеют соответствующую *структуру*, которая обеспечит возможность их успешного парсинга. Валидность же документа определяется не только правильной формой, но и тем, имеют ли его *структурные элементы* (например, `street_name`, `purchase_price`) и *значения* этих элементов правильный тип данных и находятся ли сами значения в допустимом диапазоне (например, `purchase_price` — это численное значение больше 1 и меньше 1000).

В любых случаях, когда клиентские приложения должны отправлять тела сообщений запросов, им нужно выполнять предварительную проверку. Если форматом тела является JSON или XML, следует использовать документы схем и проверочные библиотеки. В случае других форматов (`application/x-www-form-urlencoded` и т. д.) клиентское приложение должно опираться на собственные процедуры валидации.

Пример

Вот пример простого тела сообщения, описанного в формате FORMS + JSON:

```
{
  id: "filter", name: "filter",
  href: "http://company-atk.herokuapp.com/filter/",
  rel: "collection company filter",
  title: "Search",
  method: "GET",
  properties: [
    {name: "status", value: ""},
    {name: "companyName", value: ""},
    {name: "stateProvince", value: ""},
    {name: "country", value: ""}
  ]
}
```

А вот простой документ JSON Schema, который можно использовать для проверки этого тела сообщения:

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
```

```
"properties": {
  "status": {
    "type": "string",
    "enum": ["pending", "active", "closed"]
  },
  "companyName": {
    "type": "string"
  },
  "stateProvince": {
    "type": "string"
  },
  "country": {
    "type": "string"
  }
}
```



Тема JSON Schema слишком обширна для освещения в этой книге. Для более подробного ее изучения рекомендую ресурс <https://json-schema.org>.

Вот пример тела, которое может отправить клиент:

```
{
  "status": "pending",
  "country": "CA"
}
```

Наконец, вот пример использования популярного валидатора JSON (ajv, <https://oreil.ly/FINg1>), который я часто применяю:

```
/*
 * Загрузка файла схемы из внешнего источника.
 * Передача сообщения JSON для отправки.
 * Обработка и возвращение результата/ошибок.
 */

function jsonMessageCheck(schema, message) {
  var schemaCheck = ajv.compile(schema);
  var status = schemaCheck(message);
  return {status: status, errors: schemaCheck.errors};
}
```

Проверка XML-сообщений с помощью XML-схемы

XML-сообщения проверяются аналогично с помощью документов XML-схемы и инструментов. Для этого существует несколько подходов, выбор которых зависит от вашей платформы.

Возьмем тот же пример. Вот XML-схема для фильтрации записей компании:

```
<xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified"
  xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="message">
    <xs:complexType>
      <xs:all>
        <xs:element name="status" type="xs:string"
          minOccurs="0" maxOccurs="1"/>
        <xs:element name="companyName" type="xs:string"
          minOccurs="0" maxOccurs="1"/>
        <xs:element name="stateProvince" type="xs:string"
          minOccurs="0" maxOccurs="1"/>
        <xs:element name="country" type="xs:string"
          minOccurs="0" maxOccurs="1"/>
      </xs:all>
    </xs:complexType>
  </xs:element>
</xs:schema>
```



Подробнее по теме XML и XSD читайте документацию XML Schema (<https://www.w3.org/TR/xmlschema>) и XML Schema 1.1 (<https://www.w3.org/TR/xmlschema11-1>).

Вот образец XML-сообщения:

```
<message>
  <status>pending</status>
  <country>CA</country>
</message>
```

А это простой пример кода клиента для проверки сообщения в среде выполнения:

```
var xsd = require('libxmljs2-xsd');

function xmlMessageCheck(schema, message) {
  var schemaCheck;
  var errors;
  var status = false;

  try {
    schemaCheck = xsd.parseFile(schema);
    status = true;
  } catch {
    status = false;
    errors = schemaCheck.validate(message);
  }
  return {status:status,errors:errors}
}
```

Этот пример опирается на модуль узла, основанный на библиотеке Linux `xml-lib` (<https://oreil.ly/MqnEO>).

Проверка FORMS с помощью JSON Schema

Проверка тел сообщений `application/x-www-form-urlencoded` требует чуть больших усилий, но если они будут простыми, можно преобразовать сообщения `application/x-www-form-urlencoded` в формат JSON и затем применить к ним документ JSON Schema:

```
const qs = require('querystring');

/*
 * Считывание документа JSON schema из внешнего источника.
 * Отправка строки form-urlencoded.
 * Преобразование данных в JSON и их передача в модуль проверки json.
 * Возвращение результатов из модуля проверки json.
 */
function formJSONValidator(schema, formData) {
  var jsonData = qs.parse(formData);
  var results = jsonMessageCheck(schema, jsonData);
  return results // {status:status,schema.errors}
}
```

Заметьте, что эта процедура зависит от метода `jsonMessageCheck`, который мы видели ранее в этом разделе.

Обсуждение

Сервисы должны делать все необходимые схемы доступными онлайн и ссылаться на них при каждом взаимодействии, опирающемся на проверяемые сообщения. Это означает, что клиентские приложения должны искать в ответах сервисов ссылки на схемы. Далее показаны типичные места их нахождения.

В HTTP-заголовке Link:

```
**** REQUEST
GET /api.example.org/users/search HTTP/1.1
Accept: application/forms+json;

*** RESPONSE
200 OK HTTP/1.1
Content-Type: application/forms+json;
Link: <schemas.example.org/service1/user-schema.json>; profile=schema

{...}
```

Внутри тела ответа:

```
{
  "id" : "za1xs2cd3",
  "type": "search",
  "schema" : "api.example.org/schemas/user-search.json",
  "links" : [
```

```

{
  "id" : "q1w2e3r4"
  "name" : "user",
  "href" : "http://api.example.org/q1w2e3r4",
  "title" : "User Search",
  "method" : "GET",
  "properties": [
    {"name":"familyName", "value":""},
    {"name":"givenName", "value":""},
    {"name":"sms", "value":""},
  ]
}
],
...
}

```

Проверять тела сообщений с помощью JSON/XML Schema бывает непросто. Например, JSON Schema намного снисходительнее XML Schema. И хотя есть другие инструменты с поддержкой преобразования XML Schema в JSON Schema в среде выполнения, это редко дает должный результат. Лучше всего продолжать записывать схемы в нативном формате.

Трюк с преобразованием `application/x-www-form-urlencoded` в JSON хорошо сработает при условии, что итоговый JSON будет рассматриваться как простые пары «имя/значение». Большинство сообщений в формате FORMS следуют этому паттерну, но бывают случаи, когда сервисы креативят с именами полей FORM, создавая вложенную иерархию. Вот пример:

```
user.familyName=mamund&user.givenName=ramund&user.sms=+12345678901
```

Он подразумевает создание на стороне сервера следующего сообщения:

```

{
  "user" : {
    "familyName" : "mamund",
    "givenName" : "ramund",
    "sms" : "+12345678901"
  }
}

```

По мере того как сервис начинает использовать все более хитроумные паттерны именования, предлагаемый трюк с преобразованием строк FORM в документы JSON постепенно утрачивает свою надежность. Чтобы этого избежать, желательно вместо него написать валидатор.

В лучшем случае сервисы будут предоставлять разработчикам клиентских приложений правильно написанные и актуальные документы схем для применения в среде выполнения. Если же целевой сервис этого делать не будет, разработчикам нужно обратиться к документации его API и создать собственные документы. Эти документы необходимо разместить онлайн (возможно, за защищенным

URL), где они будут доступны для клиентских приложений. Нужно также постараться поддерживать актуальность этих документов в соответствии с изменениями сервиса.



Если сомневаетесь, напишите собственный код для проверки сообщений, чтобы избежать проблем с плохо составленными или неадекватно обслуживаемыми документами схем.

Грамотное построение сервисов гарантирует, что любые изменения схем в них окажутся обратно совместимыми с прежними версиями. Тем не менее клиентские приложения должны быть готовы к работе с сервисами, которые не придерживаются этого принципа. Если целевой сервис не соблюдает обратную совместимость, клиенты могут быть вынуждены исключить проверку в среде выполнения на основе документов схемы и вместо этого написать собственные валидаторы для обеспечения совместимости при движении вперед.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 4.13. Использование запросов документов для проверки входящих сообщений.
- Рецепт 6.7. Использование медиатипов для запросов данных.
- Рецепт 7.13. Использование наблюдения состояния для реализации рабочего потока под управлением клиента.

4.13. Использование запросов документов для проверки входящих сообщений

Когда клиентские приложения API получают от сервиса ответ, они должны проверять входящие данные на предмет их соответствия ожидаемому формату. Особенно актуально это для M2M-взаимодействий, в которых человек редко имеет возможность просмотреть и проверить входящие сообщения.

Задача

Как клиентское приложение API может безопасно и последовательно убеждаться, что входящий ответ сервиса содержит ожидаемые данные в запрошенном формате?

Решение

Клиентским приложениям следует проверять входящие ответы сервисов на трех уровнях.

Протокол

Включает ли ответ ожидаемые детали уровня HTTP-протокола (код состояния, тип содержимого и любые другие важные заголовки)?

Структура

Состоит ли тело ответа, если таковое имеется, из ожидаемых структурных элементов? Например, есть ли в нем ожидаемое свойство `title`? Содержит ли тело один или несколько элементов `link` с ожидаемыми `rel`, `name` или иными свойствами и т. д.?

Значение

Если ожидаемые структурные элементы, например ссылки, формы, свойства данных, существуют, содержат ли они ожидаемые *значения*, например `rel="self"`? В случае свойств данных — имеют ли их элементы ожидаемые типы (строка, целое число и т. д.) и находятся ли значения этих свойств в рамках допустимых диапазонов? Например, содержат ли свойства типа `Date` допустимые календарные даты? Находятся ли их численные значения в рамках нижних и верхних границ?



Подробнее о том, как использовать документы схемы для проверки входящих сообщений, читайте в рецепте 4.12.

Пример

Вот упрощенная версия проверки входящего сообщения на уровне протокола, структуры и значений:

```
var response = httpRequest(url, options);
var checks = {};
var message = {};

// Проверки уровня протокола
checks.statusOK = (response.statusCode===200?true:false)
checks.contentTypeCollectionJSON = (
  response.headers["content-type"].toLowerCase().indexOf("vnd.collection+json")
  ?true:false
);
checks.semanticTypeTaxes = (
  response.headers["profile-type"].toLowerCase().indexOf("taxes.v1.alps")
  ?true:false
);
```



```
// Структурные проверки
checks.body = JSON.parse(response.body);
checks.submitLink = (body.filter("submit").length>0?true:false);
checks.rejectLink = (body.filter("reject").length>0?true:false);
checks.countryCode = (body.filter("countryCode").length>0?true:false);
checks.stateProvince = (body.filter("stateProvince").length>0?true:false);
checks.salesTotal = (body.filter("salesTotal").length>0?true:false);

// Проверки значений
checks.submit = checkProperties(submitLink, ["href","method","encoding"]);
checks.reject = checkProperties(rejectLink, ["href","method","encoding"]);
checks.country = checkProperties(countryCode, ["value"],filters.taxRules);
checks.stateProv = checkProperties(stateProvince, ["value"],filters.taxRules);
checks.salesTotal = checkProperties(salesTotal, ["value"],filters.taxRules);

// Вычисления
if(validMessage(checks) {
    var taxes = computeTaxes(checks);
    checks.taxes = checkProperties(taxes,filters.taxRules);
}

// Отправка результатов
if(validMessage(checks) {
    message = formBody(checks.submit, checks.taxes);
}
else {
    message = formBody(checks.reject,checks.taxes);
}
response = httpRequest(taxMessage);
```

Здесь также видно, что после всех проверок (при условии их прохождения) код выполняет ожидаемую задачу (вычисление налогов) и отправляет результаты. Этот пример наглядно демонстрирует, что основные процессы кода заключаются в безопасной валидации входящего сообщения перед выполнением небольшого объема работы. Это *особенность* грамотно построенных клиентских приложений, особенно в случае М2М-взаимодействий.

Обсуждение

Несмотря на громоздкость примера, следует отметить, что основную часть его кода можно сгенерировать, а не писать вручную. Это значительно сокращает усилия, необходимые для составления продуманного клиента API, и повышает безопасность и согласованность приложения в среде выполнения.

Не опирайтесь на схемы

Вас может удивить то, что я не предлагаю использовать для проверки входящих сообщений схемы JSON Schema, XML Schema и т. п. Мой опыт свидетельствует: применение схем, особенно строгих, к входящим сообщениям обуславливает

слишком большое количество ложных отрицательных результатов, то есть отклонений сообщений, которые не соответствуют схеме, но все равно могут быть успешно и безопасно обработаны клиентом. Это особенно актуально для XML Schema — она строгая по умолчанию. Даже валидные элементы, находящиеся в документе в разном порядке, могут привести к провалу анализа схемы. Кроме того, схемы не охватывают необходимые проверки на уровне протокола.

Подумайте об использовании запросов путей JSON/XML

Приведенный ранее пример опирается на некоторые возможности JavaScript для выполнения фильтрующих проверок. Эти строки кода можно заменить более обобщенным решением с помощью запросов JSON Path. Я для этого применяю библиотеку JavaScript/NodeJS под названием JSON Path-Plus (<https://oreil.ly/y3dli>). Тем не менее здесь нужно быть внимательным, поскольку на момент написания книги спецификация JSONPath в IETF до сих пор не завершена и наверняка сильно изменится.

А вот спецификация XML Path (<https://oreil.ly/UbIwj>) уже достаточно зрелая и является очень надежным источником для проверки сообщений в формате XML. В W3C даже ведется работа по расширению области XML Path версии 3.1 (<https://oreil.ly/uKBDe>) дополнительными запросами JSON.

Подробнее о фильтрации сообщений читайте в рецепте 4.1.4.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 4.12. Использование документов схем для валидации исходящих сообщений.
- Рецепт 6.7. Использование медиатипов для запросов данных.
- Рецепт 7.13. Использование наблюдения состояния для реализации рабочего потока под управлением клиента.

4.14. Валидация входящих данных

При создании клиентов API важно знать, как обрабатывать входящие сообщения. По сути, каждый ответ сервиса должен рассматриваться как опасный, пока не доказано обратное. Сообщение может содержать вредоносные данные, скрытые скрипты для выполнения и/или просто плохие данные, способные нарушить поведение клиентского приложения.

Наличие стратегии безопасной обработки входящих полезных нагрузок очень важно для создания успешной экосистемы API. И это особенно верно в системах

с М2М-взаимодействиями, поскольку в них у людей меньше возможностей для анализа и исправления либо отклонения сомнительного содержимого.

Задача

Как клиентскому приложению обеспечить, чтобы входящее сообщение не содержало опасного или вредоносного контента?

Решение

Клиентские приложения API должны тщательно инспектировать входящие сообщения (ответы сервисов) и удалять либо игнорировать любое вредоносное содержимое. Это можно обеспечить за счет использования при написании клиента нескольких базовых принципов:

- всегда фильтровать входящие данные;
- при подтверждении содержимого опираться только на модель белого списка — не черного;
- использовать диапазон «минимум/максимум» для всех значений данных и отклонять те, которые за него выходят.

Эти общие правила подкрепляются еще одной линией защиты клиентских приложений — *синтаксической* валидацией (на основе ожидаемого типа данных) и *семантической* валидацией (на основе ожидаемого значения этого типа). Примерами синтаксической валидации являются сложные типы вроде почтовых кодов, телефонных номеров, личных идентификационных номеров (страховые номера США и т. п.) и другие предопределенные типы. Клиентское приложение должно уметь определять, когда ожидаются эти типы данных, и проверять их *структуру*. Например, почтовые коды США состоят из двух наборов цифр: первыми идут пять обязательных, а за ними при необходимости через дефис четыре дополнительные цифры. Грамотно построенные форматы сообщений предоставляют информацию о типах данных в ответе в среде выполнения.



Основной смысл в том, что клиенты API должны следовать стратегии «защитись любой ценой, включая позицию полного отказа обрабатывать запрос». Это несколько противоречит закону Постеля (он же принцип надежности), но по веской причине. Клиенты API должны быть внимательны только к тем значениям содержимого, которые понимают, а также следить за адекватностью этих значений и выполнять в отношении их полей только предопределенные операции.

Клиентские приложения также должны стараться проверять *семантическое* значение содержимого сообщения. Например, если предоставлен диапазон дат — скажем, `startDate` и `stopDate`, важно убедиться, что значение `startDate` идет до значения `stopDate` на временной шкале.

Пример

Примером описанной поддержки синтаксической информации о типах является формат Collection + JSON:

```
{ "collection" :
  {
    ...
    "items" : [
      {
        "rel" : "item person"
        "href" : "http://example.org/friends/jdoe",
        "data" : [
          { "name" : "full-name", "value" : "J. Doe",
            "prompt" : "Full Name", "type" : "string"},
          { "name" : "email", "value" : "jdoe@example.org",
            "prompt" : "Email", "type" : "email"},
          { "name" : "phone", "value" : "123-456-7890",
            "prompt" : "Telephone", "type" : "telephone"},
          { "name" : "birthdate", "value" : "1990-09-30",
            "prompt" : "Birthdate", "type" : "date"}
        ]
      }
    ]
    ...
  }
}
```

Обсуждение

Немногие форматы представления автоматически сообщают для значений содержимого метаданные, описывающие типы данных и их диапазон. По возможности старайтесь убедить сервисы, которые использует ваше клиентское приложение, добавлять эту информацию в виде метаданных в среде выполнения. Удачными форматами для этого будут JSON-LD, UBER, Collection + JSON и любые другие форматы RDF (Resource Description Framework — инфраструктура описания ресурсов).

Для случаев, когда метаданные типов/диапазона в среде выполнения недоступны, разработчики клиента должны обратиться к человекочитаемой документации и создать собственные метаданные для применения к входящим сообщениям. И хотя этот внешний источник может со временем утратить актуальность (в случае обновления документации), наличие машиночитаемого набора правил фильтрации содержимого окажется довольно полезным.

Вот пример возможных метаданных типов и диапазона для клиента, вычисляющего продажи и налог на добавленную стоимость (НДС):

```
var filters = {}
filters.taxRules = {
  country: { "type": "enum", "value": [ "CA", "GL", "US" ] },
```

```
stateProvince:{"type":"enum","value":[...]},
salesTotal:{"type":"range","value":{"min":"0","max":"1000000"}}
}
```

При получении сообщения ответа желательно создавать для него внутреннее отфильтрованное представление и позволять коду выполняться только для него, но не для оригинала. Это снизит вероятность того, что ваше приложение по ошибке получит неправильно сформированные данные, а также ограничит возможные риски для него:

```
// Выполнение запроса, извлечение тела и очистка
var reponse = httpRequest(url, options);
var message = filterResponse(response.body, filters.taxRules);
```

В этом примере переменная `message` содержит информацию, признанную безопасной для приложения. Заметьте, что процедура `filterResponse()` может выполнять ряд операций. В частности, она должна искать только то содержимое, которое интересует клиентское приложение. Например, если этот клиент вычисляет продажи или НДС, то его могут интересовать только поля `country`, `stateProvince` и `salesTotal`. Данный фильтр может возвращать эти поля, лишь убедившись в безопасности их значений, например, если поле `country` содержит один из валидных кодов ISO страны и т. д. Результаты можно применять для форматирования другого сообщения с целью использования в будущих запросах.

Заметьте, что в этом примере необходимо выполнить дополнительную *семантическую* валидацию, чтобы убедиться в совместимости значений `country` и `stateProvince`. Например, `country="CA"` и `stateProvince="KY"` окажутся семантически некорректными.

Наконец, бывают случаи, когда клиент API выполняет задачу получения рабочего документа, его обработки, обновления и отправки в другое приложение или сервис. В таких ситуациях все равно желательно отфильтровывать содержимое документа, оставляя только нужные клиенту поля, проверять эти значения, выполнять операцию и обновлять исходный документ перед его отправкой. Ваш клиент API не отвечает за очистку всего документа, который нужно передать далее. Единственная его обязанность состоит в извлечении нужных полей, их очистке, выполнении заданной работы и обновлении исходного документа. Здесь нужно предполагать, что любые другие приложения, которые работают с этим документом, реализуют аналогичный уровень самозащиты.

Вот упрощенный пример:

```
function processTaxes(args) {
  var results = {};
  var response = httpRequest(args.readUrl, args.readOptions);
  var message = filterResponse(response.body, args.rules);

  if(message.status!=="error") {
    results = computeTaxes(message);
  } else {
```

```
        results = invalidMessage(message)
    }

    options.requestBody = updateBody(response.body, message); response =
    httpRequest(args.writeUrl, args.writeOptions);
}
```

Подробнее об очистке входящих данных читайте в документации OWASP Input Validation Cheat Sheet (<https://oreil.ly/haoIb>).

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 4.12. Использование документов схем для валидации исходящих сообщений.
- Рецепт 4.13. Использование запросов документов для проверки входящих сообщений.
- Рецепт 6.7. Использование медиатипов для запросов данных.

4.15. Сохранение состояния

В системе RESTful бывает сложно обнаружить и получить информацию о временном состоянии запросов и ответов. Она может находиться в трех возможных местах: на сервере, на клиенте или в передаваемых между ними сообщениях. Какой вариант будет лучше и при каких обстоятельствах?

Задача

Как клиент может успешно отслеживать временное состояние взаимодействий сервера и клиента? Где располагается информация об этом состоянии, когда одно клиентское приложение взаимодействует с несколькими компонентами на сервере?

Решение

Самым безопасным местом для хранения временного состояния — состояния приложения, определяемого запросами и ответами в среде выполнения, — является клиент. Он отвечает за выбор серверов для взаимодействия и инициирует HTTP-запросы. Именно клиент обладает наиболее точной картиной состояния приложения, особенно когда для выполнения задачи он привлек несколько несвязанных сервисов.

Реализовать это клиентскому приложению проще всего с помощью ведения подробной истории каждого взаимодействия «запрос/ответ» и выбора из нее (или вычисления) важных данных о состоянии приложения. Большинство инструментов программирования поддерживают сериализацию HTTP-потокков.

Как правило, клиенту нужно регистрировать и сохранять следующие элементы.

Элементы запроса:

- URL;
- метод;
- заголовки;
- строка запроса;
- тело запроса.

Элементы ответа:

- URL;
- статус;
- тип содержимого;
- заголовки;
- тело.

Заметьте, что значение URL из запроса может не совпадать с URL ответа ввиду своей неполноты (например, отсутствия элемента протокола — `api.example.org/users`) или в результате перенаправления на стороне сервера.

Пример

Далее приведен фрагмент кода из клиентского приложения HyperCLI, которое управляет элементами запросов и ответов в HTTP-взаимодействиях:

```
...
// Сбор информации о запросе для дальнейшего использования
requestInfo = {};
requestInfo.url = url;
requestInfo.method = method;
requestInfo.query = query;
requestInfo.headers = headers;
requestInfo.body = body;

// Выполнение фактического вызова
if(body && method.toUpperCase()!="GET") {
  if(method.toUpperCase()=="DELETE") {
    response = request(method, url, {headers:headers});
  }
}
```

```

    else {
        response = request(method, url, {headers:headers, body:body});
    }
} else {
    if(body) {
        url = url + querystring.stringify(body);
    }
    response = request(method, url, {headers:headers});
}
response.requestInfo = requestInfo;
responses.push(response);
...

```

Этот код создает стек магазинного типа (push-down stack), содержащий информацию о каждом HTTP-взаимодействии. Теперь то же клиентское приложение может вспомнить историю взаимодействий и использовать детали запросов/ответов для принятия решений о дальнейших действиях.

Вот вывод `SHOW HELP` команды `DISPLAY`, возвращающей детали доступной истории HTTP-взаимодействий:

```

DISPLAY|SHOW (synonyms)
  REQUEST (returns request details – URL, Headers, querystring, method, body)
  URL (returns the URL of the current response)
  STATUS|STATUS-CODE (returns the HTTP status code of the current response)
  CONTENT-TYPE (returns the content-type of the current response)
  HEADERS (returns the HTTP headers of the current response)
  PEEK (displays the most recent response on the top of the stack)
  POP (pops off [removes] the top item on the response stack)
  LENGTH|LEN (returns the count of the responses on the response stack)
  CLEAR|FLUSH (clears the response stack)
  PATH <jsonpath-string|$$> (applies the JSON Path query to current response)

```



Более подробно HyperLANG и HyperCLI описаны в приложении Г.

Обсуждение

Налаживание отслеживания HTTP-запросов и ответов требует некоторой предварительной работы, но в итоге оказывается очень полезным для клиентских приложений. Для снижения затрат на поддержку этой функциональности стоит создать повторно используемую библиотеку, занятую исключительно обработкой стека запросов/ответов, и передавать ее во всех клиентских HTTP-приложениях.

Но отслеживания одних только взаимодействий в стеке обычно для клиентов недостаточно. Вам также потребуется мониторить выбранные значения, отслеживая их как переменные состояния. В предыдущем примере видно, что предполагаемая функциональность включает использование JSON Path в качестве способа инспектирования и выбора свойств из одной пары «запрос/ответ». Что-то подобное

можно применять и для отслеживания важных значений состояния. Подробнее об отслеживании важных переменных на стороне клиента читайте в рецепте 4.16.

Если вы работаете с клиентским приложением, которое не управляет собственным пространством файлов (например, веб-браузером), то можете настроить внешний сервис, который будет отслеживать HTTP-взаимодействия и предоставлять их удаленно (через HTTP!). Такой подход обеспечит ту же функциональность, но будет опираться на живую HTTP-связь между клиентом и сервисом обработки взаимодействий. Это также означает, что сервис может работать чуть медленнее, чем нативная система на основе файлов, и оказываться недоступным из-за сетевых проблем.

Читайте также

- Рецепт 4.14. Валидация входящих данных.
- Рецепт 4.16. Наличие цели.
- Рецепт 6.8. Игнорирование неизвестных полей данных.
- Рецепт 7.13. Использование наблюдения состояния для реализации рабочего потока под управлением клиента.

4.16. Наличие цели

Бывают случаи, когда клиентскому приложению нужно продолжить работу (совершая запросы, выполняя необходимые действия, отправляя изменения и т. д.), пока не будет достигнута заданная цель (пустая очередь, нужное общее количество и т. п.). Особенно актуально это для приложений, имеющих определенный уровень автономии или автоматической обработки. Реализация данного механизма создает определенные сложности, включая случаи, когда *клиент* знает цель, а используемые им *сервисы* — нет.

Задача

Как запрограммировать клиентское приложение продолжать обработку до достижения заданной цели? Кроме того, что нужно для создания клиентских приложений, имеющих приватную цель, которая не сообщается сервисам или непонятна им?

Решение

Для создания клиентских приложений, способных продолжать работать до достижения цели, нужно заложить в них определенный уровень автономии. Приложение должно понимать, какое свойство (или свойства) нужно отслеживать, знать целевые значения для них, уметь находить и отслеживать эти значения во времени, а также понимать, когда будет достигнут конец и как прекратить обработку.

Программирование этого решения, по сути, представляет работу большинства игровых движков или автономных роботизированных систем. Клиентские приложения могут следовать классической модели искусственного интеллекта PAGE (<https://oreil.ly/5rwjg>).

Объект восприятия

Свойства, которые нужно отслеживать.

Действия

Задачи, которые клиент может выполнять, чтобы влиять на значения свойств (чтение, запись, вычисление).

Цели

Целевые значения.

Окружение

Игровое поле или пространство, в котором работает клиент. В нашем случае это будут сервисы, с которыми он взаимодействует, чтобы видеть воспринимаемые объекты и влиять на них в процессе достижения целей.

Пример

Есть две модели, по которым вам, скорее всего, потребуется реализовывать цели: заданная конечная цель (defined exit goal, DEG) и заданное целевое состояние (defined state goal, DSG). Для каждой требуется своя программная конструкция.

Заданная конечная цель

В случае DEG создается программа, которая приостанавливает или завершает некий процесс при достижении заданной цели. Пример, приведенный далее, был взят из автономного клиента гипермедиа, который ищет выход из неопределенного лабиринта:

```
function processLinks(response,headers)
{
    var xml,linkItem,i,rel,url,href,flg,links,rules;

    flg = false;
    links = [];
    rules = [];

    // Получение всех ссылок в этом документе
    g.linkCollection = [];
    xml = response.selectNodes('//link');
    for(i=0;i<xml.length;i++)
    {
        rel = xml[i].getAttribute('rel');
        url = xml[i].getAttribute('href');
        linkItem = {'rel':rel,'href':url};
```

```

    g.linkCollection[g.linkCollection.length] = linkItem;
}

// Есть ли выход?
href = getLinkElement('exit');
if(href!='')
{
    printLine('*** Done! '+href);
    g.done = true;
    return;
}

// Есть ли вход?
if(flag==false && g.start==false)
{
    href = getLinkElement('start');
    if(href!='')
    {
        flag=true;
        g.start=true;
        g.href = href;
        g.facing = 'north';
        printLine(href);
    }
}

// Хорошо, пойдем через дверь
rules = g.rules[g.facing];
for(i=0;i<rules.length;i++)
{
    if(flag==false)
    {
        href=getLinkElement(rules[i]);
        if(href!='')
        {
            flag=true;
            g.facing=rules[i];
            printLine(href);
            continue;
        }
    }
}

// Обновление указателя, обработка следующего хода
if(href!='')
{
    g.href = href;
    nextMove();
}
}

```

Заметьте, что это приложение имеет цель найти выход и продолжает перемещаться из комнаты в комнату, пока ее не достигнет.

Заданное целевое состояние

Иногда нужно запрограммировать клиентское приложение на сохранение нужного состояния — DSG. Например, требуется, чтобы приложение мониторило температуру воздуха в комнате и активировало нагревающие/охлаждающие механизмы для поддержания этого состояния. Вот фрагмент кода, делающего именно это:

```
// Установка контрольных значений
var roomURL = "http://api.example.org/rooms/13";
var min = 18;
var max = 22;
var wait = (15*60*1000);

// Настройка периодических проверок
setInterval(checkTemp(roomURL,min,max),wait));

// Выполнение проверки
function checkTemp(roomURL, minTemp, maxTemp) {
    var rtn, temp;

    rtn = "";
    response = httpRequest(roomURL);
    printLine(req)

    if(response.temp<minTemp) {
        rtn = response.form("heat");
    }
    if(response.temp>maxTemp) {
        rtn = response.form("cool");
    }

    if(rtn!="") {
        response = httpRequest(rtn);
        printLine();
    }
}
```

В этой простой модели DSG приложению поставлена задача поддерживать температуру в диапазоне 18–22 °С. Ее показатель проверяется каждые 15 минут и при необходимости нужным образом корректируется. Естественно, используемый этим клиентом сервис не знает, что у клиента на уме, он просто отвечает на запросы информацией о состоянии комнаты. В этом примере видно, что ответ от сервиса включает данные о температуре `req.temp`, а также средства гипермедиа для ее изменения `req.form("heat")` и `req.form("cool")`. Важно отметить, что данный сервис мониторинга (<http://api.example.org/rooms>) может ничего не знать о том, как корректировать температуру, за что могут отвечать другие сервисы.

Обсуждение

Программирование ориентированных на цель клиентов (DEG и DSG) предполагает несколько ключевых моментов. Во-первых, клиент должен знать конечную цель заранее. Во-вторых, он должен знать, как ее достичь (поиск выхода, корректировка температуры и т. д.). В-третьих, клиент должен уметь найти точное представление текущего состояния; в-четвертых — правильно оценивать его относительно цели. Для успешной реализации ориентированного на цель клиента нужны все эти элементы.



Для текущего рецепта в примерах клиентов DEG и DSG предполагается, что все планирование и критерии для оценки определяются заранее и/или передаются при запуске клиента (через файл конфигурации). Здесь в клиентское приложение не закладываются никакие механизмы машинного обучения или планирования.

В примере с температурой клиент знает, что его цель — температура в помещении в диапазоне 18–22 °С. Он также знает, что для ее достижения можно использовать в HTTP-ответе формы "heat" и "cool". Кроме того, ему известно, что можно задействовать URL комнаты для проверки текущей температуры в ней, и он способен сравнить возвращенное значение температуры с внутренними значениями целевого максимума и минимума.

Иногда этап сравнения оказывается более сложным. Например, в случае регулирования температуры настройки могут различаться в зависимости от времени суток, присутствия в помещении людей, на основе текущей стоимости 1 кВт·ч и т. д. Тогда решение о выполнении конкретного действия будет опираться на более сложный механизм, отслеживающий и использующий большее количество значений.

Иногда сравнение может выполнять другой сервис — не клиент. Например, решение о покупке/продаже акций может поступать от сервиса, получающего широкий спектр входных данных, предоставляемых, например, клиентским приложением, и возвращающего этому приложению рекомендацию к действию.

По возможности стоит реализовать отслеживаемые свойства и локальные значения для сравнения в виде настраиваемого набора точек данных. Это избавит от жесткого прописывания решений в самом клиентском приложении. Можно, например, предоставлять клиенту управляющие данные через файл конфигурации или даже удаленный HTTP-вызов другого сервиса, который их содержит.

Наконец, в ориентированные на цель клиентские приложения важно добавлять возможность выхода. Например, если клиент с DEG не может найти выход из лабиринта, то после определенного момента ему, возможно, потребуется сдаться, иначе он просто застрянет в бесконечном цикле. Значением выхода тоже должен

управлять клиент. Использование для принятия решения о выходе другого сервиса может не сработать, если этот удаленный сервис окажется недоступен или отправит клиенту некорректные данные.

То же касается клиентских приложений с моделью DSG. Представьте, что в примере с контролем температуры сервис, возвращающий данные датчиков, выходит из строя или оказывается недоступным. Что делать клиенту? Просто продолжать холостые проверки? Более грамотным решением будет добавление кода, активирующего оповещение, когда датчики не отвечают или сообщают данные, далеко выходящие за предполагаемые границы.

Читайте также

- Рецепт 4.14. Валидация входящих данных.
- Рецепт 5.3. Преобразование внутренних моделей во внешние сообщения.
- Рецепт 6.8. Игнорирование неизвестных полей данных.
- Рецепт 7.13. Использование наблюдения состояния для реализации рабочего потока под управлением клиента.

Сервисы на основе гипермедиа

Наилучшая программная архитектура знает, какие изменения происходят часто, и упрощает их.

Пол Клементи (Paul Clements), автор Software Architecture in Practice (Pearson)

Первостепенной проблемой при проектировании и реализации API для сервисов является балансирование между юзабилити и способностью развиваться. Для API сервисов важно быть четкими и понятными. В то же время очень важно, чтобы они могли изменяться в будущем. Наконец, ценность API сервисов зачастую связана с *сохранением ими надежности с течением времени*. Вполне можно определить API, решающий конкретную текущую задачу. Но со временем ее параметры могут меняться, как и ее цели, а API в идеале должен сохранять свою полезность. И это непростая задача для проектировщика интерфейса сервиса.



Более развернуто основы создания интерфейсов сервисов описаны в разделе «Повышение стабильности и изменяемости сервисов на основе гипермедиа» главы 2.

Как сказано во вступлении к этой главе, очень важно понимать, что в дизайне ПО будет меняться часто, и максимально упрощать этот процесс. Особенно актуально это для API сервисов. API — это контракт, то есть обещание, которое необходимо сдерживать.

Отличный пример этого — то, как построен протокол HTTP. Практически все его ключевые аспекты изменяемые. HTTP-методы, коды состояния, URL, список возможных заголовков и список возможных форматов тела — все это абстрактные коллекции, которые со временем могут дорабатываться. *Структура* HTTP-сообщений постоянна — это обещание. А вот *содержимое* этих сообщений может меняться — в некоторых случаях даже внутренняя информация *управляющего действия*, например HTTP-методы, URL, доступные форматы сообщений и т. д.

Все это является ключом к созданию стабильных, полезных API для сервисов, и именно об этом пойдет речь в данной главе.



Поскольку книга посвящена интерфейсам, мы не будем углубляться во внутренние детали стоящих за ними сервисов. Тем не менее важно хорошо понимать внутреннее устройство масштабируемых и надежных сервисов в веб-среде, поэтому в приложении Б приводятся рекомендуемые материалы по проектированию и созданию таких сервисов.

Все рецепты этой главы посвящены упрощению проектирования и реализации API, нацеленных на правильный баланс между степенью специализации и способностью развиваться. Основной особенностью такого API является его возможность включать операционные метаданные в среде выполнения, а не опираться исключительно на проект и описания среды разработки. В нашем случае возможность изменять сообщения в среде выполнения будет опираться на использование гипермедиаформатов, соответствующих общей схеме проекта самого HTTP-протокола: отчетливая структура сообщений для поддержания изменчивого содержимого.

При создании API сервисов вам нужно балансировать между влиянием со стороны метаданных среды проектирования и среды выполнения (рис. 5.1). Еще одним ключевым элементом успешной реализации интерфейса сервиса является возможность поддерживать несколько форматов представления с помощью согласования содержимого.

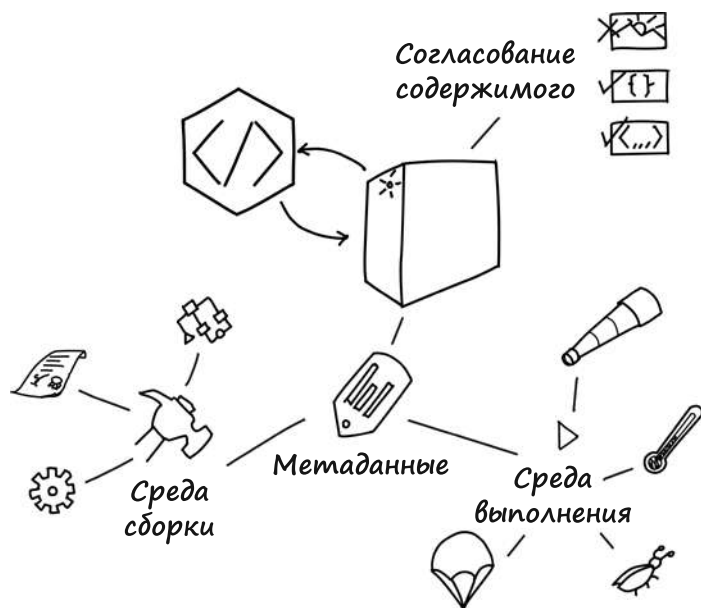


Рис. 5.1. Рецепты сервисов в гипермедиа

5.1. Публикация не менее одного стабильного URL

Когда сервисы размещают свой интерфейс в сети, им нужно обеспечить для потребителей API легкий и согласованный доступ через него. Это означает размещение API в стабильном месте с использованием URL. Но значит ли это, что само расположение никогда не должно меняться? А если вы его все же измените, как сообщить об этом потребителям API?

Задача

Как установить стабильное сетевое расположение (URL) для API сервиса и какие шаги потребуются, чтобы обеспечить удобство его нахождения даже в случае переезда сервиса на новое место?

Решение

Интерфейсы сервисов должны обещать не менее одного стабильного URL, который потребители API смогут использовать для обнаружения находящегося за этим API сервиса и взаимодействия с ним. Конкретный URL значения не имеет. Это должна быть стартовая точка API. В качестве нее может выступать ресурс, возвращающий детали об этом API, включая точку входа. Ею может быть также изначальный листинг или состояние сервиса, например предустановленный список пользователей и пр. Или это может быть некий другой ответ, который потребители API смогут применять для инициирования взаимодействия с сервисом, например логин, способ установить сеанс с сохранением состояния и пр.

Значение URL может быть простым вроде `http://api.example.com/home` или довольно сложным вроде `http://v1.home.api.example.org/q1w2e3r4t5`. Здесь важно, чтобы имелось не менее одного адреса, который потребители смогут запомнить (то есть записать в исходный код или файлы конфигурации), обеспечивающего возможность подключения к сервису через этот API. Наконец, не требуется, чтобы API сервиса имел *только один* стабильный опубликованный URL. Тем не менее он должен иметь *как минимум один* URL, на который потребители смогут уверенно опираться в долгосрочной перспективе.

Стабильные URL должны публиковаться в человекочитаемой документации к API. Они также могут присутствовать в HTTP-заголовке `Link` (`https://oreil.ly/tMGnn`) или в виде элемента `link` в теле ответа API, который поддерживает такие элементы.



В некоторых случаях стоит зарегистрировать для API сервиса хорошо известный URL. Однако это не так уж просто. Подробнее — в спецификациях RFC 8615 (<https://oreil.ly/Rni9a>) и RFC 7595 (<https://oreil.ly/8mhMG>).

Публикация стабильного URL не означает, что находящийся за ним сервис не может перемещаться. Если он переедет, то запросы к его стабильному URL можно перенаправить на новый адрес, используя код состояния **301 Moved Permanently**.

Пример

При указании стабильного URL вашего API в документации обязательно обозначьте это как *обещание* того, что он не изменится в будущем и при необходимости станет перенаправлять клиентские приложения на новый адрес.

Для отправки стабильного URL вашего API можно использовать заголовок **Link** в HTTP-ответе:

```
**** REQUEST
GET / HTTP/1.1
Host: api.example.org

**** RESPONSE
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
ETag: "p0o9i8u7y6t5r4e3w2q1"
Link: <http://api.example.org/home>; rel="home"
...
```

Стабильный URL может быть указан и в теле любого ответа, поддерживающего встроенные ссылки (или всех ответов). Вот пример в виде ответа **Collection + JSON**:

```
{ "collection" :
{
  "version" : "1.0",
  "href" : "http://api.example.org/friends/",

  "links" : [
    { "rel" : "home", "href" : "http://api.example.org/home" }
  ],

  "items" : [...],
  "queries" : [...],
  "template" : [...]
}
}
```

Когда сервис меняет свое местоположение, API должен продолжать поддерживать изначально опубликованный стабильный URL и автоматически перенаправлять потребителей на новый адрес:

```
**** REQUEST
GET / HTTP/1.1
Host: api.example.org
```

```
**** RESPONSE
HTTP/1.1 301 Moved Permanently
Location: http://new.example.org/home

**** REQUEST
GET /home HTTP/1.1
Host: new.example.org

**** RESPONSE
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
ETag: "p0o9i8u7y6t5r4e3w2q1"
Link: <http://api.example.org/home>; rel="home"
...
```

Обсуждение

Интерфейсы сервисов должны поддерживать *не менее одного* стабильного URL. Любой такой URL является обещанием потребителям API, которое не должно быть нарушено с течением времени. Вполне можно пообещать и более одного стабильного URL, но это лишь повысит объем работы, которую проектировщикам API придется проделать для выполнения долгосрочных обещаний по *всем* стабильным URL.

Нужно помнить, что клиентские приложения для обеспечения продуктивного опыта использования API сервиса могут запомнить столько URL, сколько захотят. И если сервис обещает лишь один URL, это не значит, что клиент не может запомнить другие URL в его API.

Сервисы не должны предполагать (или требовать), чтобы все клиенты начинали взаимодействие с посещения стабильного URL. Независимо от количества стабильных URL, которые пообещал API, клиенты могут спокойно взаимодействовать с интерфейсом так, как им нравится, при условии выполнения ими допустимых запросов.

Я обычно указываю на стабильный URL интерфейса сервиса с помощью значения ссылочной связи "home". Вы можете использовать любое другое значение — главное, хорошо его задокументируйте и не меняйте идентификатор.

Будет хорошей идеей отправлять стабильный URL в виде заголовка Link во всех ответах API. Также может иметь смысл отправлять его в теле ответа, если такое возможно (это не получится в случае тел PNG или MP3).

Читайте также

- Рецепт 4.1. Ограничение использования жестко прописанных URL.
- Рецепт 4.8. Каждому важному элементу ответа нужен идентификатор.
- Рецепт 5.13. Облегчение поиска сервисов с помощью реестра сервисов среды выполнения.

5.2. Предотвращение утечек внутренних моделей

Нередко причиной «поломки» интерфейсов сервисов являются изменения внутренних данных, объектных моделей или моделей процессов в коде сервиса. Важно предотвращать вероятность утечки этих элементов во внешний API.

Задача

Как уменьшить вероятность прямого попадания внутренних данных сервиса, а также его объектной модели и/или модели процессов в API этого сервиса?

Решение

Лучшим способом ограничить возможность раскрытия внутренних деталей сервиса для внешнего API будет проектирование внешнего интерфейса как работающего отдельно от внутренних моделей сервиса. Это означает обособленное определение функциональности API сервиса, а не просто раскрытие кода сервиса для внешнего мира.

По сути, реализации API (интерфейсы) должны сохранять внутренние модели сервиса закрытыми. Для этого нужно сделать так, чтобы свойства данных, коллекции свойств и входные/выходные параметры API сами выражались как согласованные элементы, то есть были отделены от любых внутренних моделей сервиса. Интерфейс следует рассматривать как имеющий собственную независимую структуру и процесс реализации, а не просто как оболочку для существующего компонента сервиса бэкенда.

Пример

Самый быстрый способ избежать утечки данных/моделей объектов — их игнорирование при проектировании API. Предположим, что есть простой сервис ToDo с поддержкой двух локальных коллекций объектов данных, `item` и `user`. Они могут выглядеть так:

```
{ "items" : [
  { "id": "q1w2e3r4", "text": "This is an item", "status": "active", "nick": "mork" },
  ...
]}

{ "users": [
  { "nickname": "mork", "name": "Mark Morkelsen" },
  ...
]}
```

Предположим также, что возможными значениями поля `status` могут быть только `active` или `closed`.

Вы можете решить, что интерфейс тоже должен моделировать два объекта — `item` и `user`. Но вполне допустимо смоделировать и три:

```
{ "items" : [
  { "id": "q1w2e3r4", "text": "This is an item", "status": "active", "nick": "mork" },
  ...
]}

{ "users" : [
  { "nickname": "mork", "name": "Mark Morkelsen" },
  ...
]}

{ "status" : [
  { "name": "active" },
  { "name": "closed" },
  ...
]}
```

Или даже один:

```
{ "todo" : [
  { "id": "q1w2e3r4", "text": "This is an item",
    "status": "active",
    "nickname": "mork",
    "name": "Morkelsen"
  },
  ...
]}
```

Суть здесь в том, чтобы *не* принимать вслепую модель сервиса в качестве модели интерфейса. Да, при изменении внутренней модели сервиса может потребоваться дополнительная работа для сохранения совместимости между опубликованным интерфейсом и локальным сервисом. Но это должно реализовываться в рамках проекта API.

Обсуждение

По возможности я моделирую свойства API как одну плоскую коллекцию, как в предыдущем примере. Это упрощает клиентскую часть процесса и освобождает сопутствующие сервисы для использования любой модели локального хранилища данных, какая их интересует. Сервисы даже могут изменять модель хранилища без вреда для API. Денормализация любой внутренней модели упрощает сохранение стабильной внешней модели даже при последующих изменениях внутренней.



Много лет назад в ходе обсуждения этой темы я вывел аксиому (<https://oreil.ly/xgm2g>): «При проектировании API важно помнить, что ваша модель данных не является объектной моделью, моделью ресурсов или моделью сообщений». Я также записал скринкаст под названием *Web API Design Maturity Model* (<https://oreil.ly/aUhde>), в котором раскрыл эту идею более подробно.

Важно помнить, что API — это независимый интерфейс. Вполне нормально, если интерфейс ожидает, что человек создаст новую запись для отправки денормализованной коллекции свойств, которые будут перегруппированы и написаны для нескольких мест хранения. До тех пор, пока сервис может безопасно обрабатывать эти данные (например, следить, чтобы не утрачивались обновления и не происходили конфликты данных), сойдет любой вариант.



Подробнее о работе с хранилищем данных речь пойдет в главе 6.

Проектировщики API могут создавать любое нужное им число HTTP-ресурсов для поддержки ожидаемых взаимодействий. Эти взаимодействия не обязаны соответствовать идее CRUD (create, read, update, delete — создание, чтение, обновление, удаление), типичной для HTTP API. Например, табл. 5.1 — это ресурс, созданный для поддержки сервиса ToDo, который упоминался ранее.

Таблица 5.1. Возможная структура ресурсов для интерфейса сервиса ToDo

Действие	URL	Метод	Тело запроса	Тело ответа
Чтение списка	/todo/	GET	none	[{id,text,status,nick,name}]
Фильтрация списка	/todo/?text={text}	GET	none	[{id,text,status,nick,name}]
Создание элемента	/todo/	PATCH	{id,text,status,nick,name}	[{id,text,status,nick,name}]
Обновление текста элемента	/todo/	PATCH	{id,text}	[{id,text,status,nick,name}]
Обновление псевдонима элемента	/todo/	PATCH	{id,nick,name}	[{id,text,status,nick,name}]
Обновление статуса элемента	/todo/	PATCH	{id,status}	[{id,text,status,nick,name}]

Обратите внимание: табл. 5.1 указывает, что список ToDo является единым ресурсом, который можно изменять, добавляя и редактируя его элементы. То, как сервис организует эту информацию и/или *сохраняет* ее, не интересует потребителя

интерфейса до тех пор, пока клиентское приложение может проделывать всю необходимую работу.



Таблица 5.1 не включает дополнительные возможности HTTP, такие как ETag, If-Match, Last-Modified и прочие заголовки. Подробности описаны в рецепте 6.2.

Читайте также

- Рецепт 3.1. Обеспечение интероперабельности с помощью зарегистрированных медиатипов.
- Рецепт 3.3. Предоставление информации о предметной области через публикуемые словари.
- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 5.3. Преобразование внутренних моделей во внешние сообщения.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.1. Скрытие внутренних деталей хранилища данных.
- Рецепт 6.3. Скрытие связей данных для внешних действий.

5.3. Преобразование внутренних моделей во внешние сообщения

Для обеспечения слабой связанности между развивающимися сервисами в их взаимодействие необходимо заложить стабильную основу. Проверенной временем стратегией в этом случае будет использование в ответах API известных высокоструктурированных зарегистрированных медиатипов.

Это означает, что интерфейс сервиса при изменении формата сообщения должен отвечать одной и той же *семантической* информацией.

Задача

Как без лишних затрат и хлопот с реализацией организовать интерфейс сервиса так, чтобы он поддерживал согласование содержимого с потребителями? Какие паттерны нужно поддерживать в коде интерфейса, чтобы согласованно

представлять данные сервиса с помощью стандартизированного и хорошо известного медиатипа? Наконец, какие преимущества и недостатки сопутствуют представлению одних и тех же внутренних данных в виде разных форматов ответов сообщений?

Решение

Эффективным способом обеспечить слабую связанность структуры *интерфейса* сервиса с внутренней моделью данных или объектов этого сервиса является отдельная реализация представления. Иными словами, нужно активно участвовать в проектировании представлений, а не просто использовать прямую сериализацию внутренних моделей во внешние сообщения.

Для представления информации в HTTP-взаимодействиях было разработано несколько форматов. Они содержат довольно много структур (например, `<html>`, `<head>`, `<body>` и т. д.) для поддержки клиентских приложений, способных привязываться к формату сообщения, а не к его содержимому (например, `givenName`, `id`, `telephone` и т. д.).



Список рекомендуемых медиатипов ответа приводится в приложении В.

Процесс отображения частей внутренней модели данных/объектов сервиса во внешнюю объектную модель интерфейса обычно требует совмещения алгоритмического и креативного подходов к проектированию. Для некоторых форматов, таких как неструктурированные XML и JSON, такое отображение может быть довольно буквальным — простой сериализацией данных в сообщение. Но это неудачная идея. Например, прямая сериализация модели данных в модель сообщений ведет к раскрытию внутренней модели для внешнего мира и получению хрупкой реализации, которая в случае изменения внутренней модели (а она будет меняться) наверняка сломается. Подробнее читайте в рецепте 6.10.

Поэтому в качестве основы для обмена представлениями лучше выбрать структурированный медиатип (SMT) вроде SIREN, Collection + JSON или даже HTML. Отображение внутренних данных в один из этих форматов поначалу потребует больших усилий — вы можете сделать это вручную или задействовать готовую библиотеку для конкретного медиатипа, зато со временем окупится, поскольку любые будущие изменения или даже дополнительные интерфейсы смогут отрисовывать валидные внешние сообщения для потребителей API, используя одну библиотеку.

Подробнее о проектировании интерфейсов сервисов, не зависящих от внутреннего сервиса, говорится в рецепте 5.17.

Пример

Код интерфейса сервиса, который поддерживает ответы с выбираемыми медиатипами, должен реализовывать два основных элемента этой поддержки для среды выполнения. Во-первых, API сервиса должен уметь определять, какие форматы клиент предпочитает, а во-вторых, должен использовать эту информацию для отрисовки внутренних данных в предпочтительном формате.

HTTP определяет заголовок **Accept**, позволяющий клиентам описывать в отправляемых запросах свои предпочтения относительно форматов сообщений. Когда API сервиса видит заголовок **Accept**, ему нужно спарсить его значение и сделать соответствующий выбор. Более подробно об интерпретации содержимого заголовка **Accept** (<https://oreil.ly/GLlue>) читайте в спецификации HTTP.

Простой обмен сообщениями может выглядеть так:

```
*** REQUEST
GET /todo/list HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json, application/vnd.uber+xml

*** RESPONSE
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
```

Заметьте, что здесь интерфейс сервиса выбрал формат `application/vnd.collection+json`. Когда предлагается более одного варианта, сервисы могут выбирать, какой формат вернуть, включая возвращение предустановленного, который даже не был указан клиентом в списке. Подробнее об этом читайте в рецепте 5.5.

После выбора формата для выражения ответа API нужно отобразить эти внутренние данные в итоговое представление согласно спецификациям формата. Зачастую это несложный процесс, но иногда внутренние данные отобразить сложно и они могут даже не включаться в ответ.

Вот пример внутренних данных сервиса, которые нужно отобразить в ответе:

```
{
  "user" : {
    "id": "q1w2e3r4t5",
    "givenName": "Mark",
    "familyName": "Morkelsen",
    "nickName": "mork",
    "telephones": [
      { "type": "home", "value": "1-234-567-8901" },
      { "type": "work", "value": "1-987-654-3210" }
    ]
  }
}
```

Теперь предположим, что клиент отправил следующий запрос:

```
GET /users/q1w2e3r4t5 HTTP/1.1
Host: api.example.org
Accept: application/json
```

Простым, но нерекомендуемым способом представления этих данных будет их простая прямая сериализация:

```
*** RESPONSE
HTTP/1.1 200 OK
Content-Type: application/json
...

{
  "user" : {
    "id" : "q1w2e3r4t5",
    "givenName" : "Mark",
    "familyName" : "Morkelsen",
    "nickName" : "mork",
    "telephone" : [
      { "type" : "home", "value" : "1-234-567-8901"},
      { "type" : "work", "value" : "1-987-654-3210"}
    ]
  }
}
```

Теперь взглянем на запрос для представления с использованием формата `text/csv`:

```
GET /users/q1w2e3r4t5 HTTP/1.1
Host: api.example.org
Accept: text/csv
```

```
*** RESPONSE
HTTP/1.1 200 OK
Content-Type: application/json
...
"id","givenName","familyName","nickName","telephone_home","telephone_work"
"q1w2e3r4t5","Mark","Morkelsen","mork","1-234-567-8901","1-987-654-3210"
```

Заметьте, что внутренний объект `telephone` был отображен в CSV-ответе как набор плоских свойств. Клиенты могут даже не знать внутренней формы этих данных.

Наконец, вот пример представления, когда клиент включил в запрос медиатип `application/vnd.collection+json`:

```
{ "collection" :
{
  "version" : "1.0",
  "href" : "http://example.org/users/q1w2e3r4t5",

  "links" : [
    { "rel" : "users", "href" : "http://example.org/users"},
    { "rel" : "products", "href" : "http://example.org/products"},
  ]
}
```

```
{
  "rel" : "services", "href" : "http://example.org/services"
},
"items" : [
  {
    "href" : "http://example.org/users/q1w2e3r4t5",
    "data" : [
      {
        "name" : "id", "value" : "q1w2e3r4",
        "prompt" : "Identifier"},
      {
        "name" : "givenName", "value" : "Mark",
        "prompt" : "First Name"},
      {
        "name" : "familyName", "value" : "Morkelsen",
        "prompt" : "Last Name"},
      {
        "name" : "nickName", "value" : "Mork",
        "prompt" : "Nick"}
    ],
    "links" : [
      {
        "rel" : "telephones", "prompt" : "Telephones",
        "href" : "http://examples.org/users/q1w2e3r4t5/telephones"
      }
    ]
  }
]
}
```

В этом случае API сервиса передал в представление немало дополнительной информации, в том числе отобразил значения `prompt` в каждую точку данных, включая элементы `link`, указывающие на связанные данные, и даже переместил массив данных `telephone` в другой ресурс.

Основная идея здесь в том, что при проектировании механизма представления данных сервиса не нужно слепо ограничиваться сериализацией внутренних моделей.

Обсуждение

На этапе реализации API сервиса желательно документировать правила, которые вы используете для преобразования внутренних моделей во внешние сообщения. Это поможет делиться алгоритмами с другими разработчиками и упростит отладку любых ошибок, которые вы попутно обнаружите. Однако нежелательно публиковать это описание представления для потребителей API. Они не должны знать внутренние данные или объектные модели, скрытые за вашими интерфейсами.

Со временем вы наверняка столкнетесь с изменением внутренних моделей вашего сервиса. Во многих случаях эти изменения *не должны* приводить к изменениям внешних сообщений. Например, если внутреннее свойство модели `user.id` изменится на `user.identifier`, нет необходимости изменять внешнее свойство.

Тем не менее некоторые изменения внутренних моделей подразумевают внесение новых свойств (`user.hatsize`), которые могут отражаться во внешних сообщениях, когда эти изменения не приводят к нарушению работы существующих потребителей API. Добавление поля в конец строки `text/csv` может быть вполне приемлемо,

а вот изменение порядка строк, например вставка нового поля в качестве первого в строке, вряд ли будет хорошей идеей.



Если вы ожидаете, что внутренние модели будут изменяться часто, выберите формат представления, предназначенный для уменьшения влияния таких изменений. Например, Collection + JSON хорошо подходит для изменчивых моделей (все свойства в нем представлены как `data.name` и `data.value`), но SIREN и HAL — нет (поскольку они привязываются к свойствам объекта напрямую).

И хотя можно жестко запрограммировать интерфейс сервиса *всегда* отправлять один и тот же формат (HAL, SIREN и т. д.), обычно желательно использовать процесс согласования HTTP-содержимого (см. рецепт 5.6), чтобы потребители API могли указывать свои предпочтения в среде выполнения с помощью заголовка Ассерта. Дополнительные подробности о том, как обрабатывать предпочтения клиентов в отношении медиатипов, читайте в рецепте 5.5.

Я обычно реализую поддержку не менее двух, а лучше трех или более форматов в зависимости от ситуации. Во-первых, стоит выбрать один медиатип с высоким показателем фактора гипермедиа (Hypermedia Factor), например Collection + JSON или SIREN (подробнее об Н-факторах читайте в рецепте 4.9). Во-вторых, желательно иметь поддержку простого формата сериализации вроде JSON, XML и/или CSV, поскольку многие потребители API смогут использовать их быстро. Для поддержки дополнительных форматов можно добавить любые другие, которые вы и/или потребители вашего API предпочитаете.

Если это уместно, будет нелишним реализовать и поддержку HTML-представления API вашего сервиса. Это поможет разработчикам сервиса при тестировании дизайна интерфейса, а также потребителям API, которые хотят поэкспериментировать с API, не затрачивая много времени на программирование его клиента. Наконец, формат HTML может стать надежным выбором для коллекции сервиса в качестве одного из медиатипов с высоким Н-фактором.

Читайте также

- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.
- Рецепт 4.5. Согласование поддержки профилей во время выполнения.
- Рецепт 4.6. Управление форматами представления в среде выполнения.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 5.5. Оповещение клиента о поддерживаемых настройках.
- Рецепт 5.6. Поддержка согласования HTTP-содержимого.
- Рецепт 6.10. Изменение моделей данных в промышленной среде.

5.4. Выражение внутренних функций в виде внешних действий

После внедрения практики стандартизации ответов с помощью структурированных медиатипов вам потребуется разобраться с правильным выражением внутренней функциональности находящегося за вашим интерфейсом сервиса или сервисов. Для этого нужно следовать практике выражения не только моделей данных сервиса, но и моделей его действий — функций, к которым должен быть доступ через внешний API.

Задача

Как решить, какие внутренние функции сервиса нужно раскрыть через API, и как последовательно выражать эти методы, чтобы потребители API могли их находить и использовать ожидаемым способом? Какие форматы и/или паттерны доступны для преобразования внутренних функций сервисов в действия, осуществляемые через внешний интерфейс?

Решение

Наиболее прямым способом выразить внутренние функции в виде внешних действий будет их представление с помощью средств гипермедиа (см. рецепт 4.9). Например, в HTML для представления внутренней функции сервиса можно использовать имя функции (`<form name="approveUser" ...>`), входные аргументы (элементы управления `<input>`) и выходные возвращаемые значения. Тем не менее не всегда возможно (и не всегда желательно) привязывать внешние действия напрямую к внутренним операциям сервиса.

Обычно при их выражении лучше использовать утвержденные внешние словари. Например, внутренний параметр данных может иметь имя `firstName`. Но во внешнем интерфейсе тот же параметр будет выражен как `givenName` (см. рецепт 3.3). Код API может обрабатывать переход между внутренними и внешними именами. Это дает преимущество в виде защиты внешнего интерфейса от внутренних изменений сервиса. Продолжим прежний пример — обновления сервиса могут приводить к трансформации `firstName` в `fNameValue`. В таком случае изменить потребуется только код промежуточного ПО, а не сам внешний интерфейс.

Наконец, некоторые внутренние действия, особенно те, которые включают сложные вычисления или процессы, лучше выражать в упрощенной форме. Сервис (`onboardingSvc`) может требовать связи с несколькими зависимыми сервисами (`userSvc`, `customerRelationsSvc` и `accountSalesSvc`), обмениваясь с ними данными для завершения задачи. Зачастую лучше выразить это в виде одного действия (`sendOnboardingData`), а не нескольких зависимых (подробности — в примере далее).

По сути, проектирование взаимодействий с API нужно рассматривать как независимый процесс. Здесь вы можете свободно проявлять изобретательность. Проектируйте хороший интерфейс — не нужно просто отражать внутренние действия сервиса. Предположите, что между обменом данными с API и развитием внутреннего сервиса может присутствовать этап перевода. Главное — стремитесь сохранять слабую связанность между сервисом и публикуемым интерфейсом.

Пример

При прямом переводе внутренней операции во внешнее действие можно начать с примера функции сервиса:

```
function approveUser(userId, nickname, approver, level) {
    var approval = {};

    approval.userId = userId;
    approval.nickname = nickname;
    approval.approver = approver;
    approval.level = level;

    return data.create(approval);
}
```

А затем перевести эти внутренние элементы в HTML-объект FORM в сообщении ответа:

```
<form name="approveUser" enctype="application/x-www-form-urlencoded"
    method="post" action="http://api.example.org/users/approvals"
    target="approvalDisplay">
    <input name="userNick" type="string" value="mork" />
    <input name="approverName" type="string" value="Mr. Roboto" />
    <input name="approveLevel" type="string" value="nominal" />
    <input type="submit" value="Approve User" />
</form>
<iframe id="approvalDisplay" />
```

Однако, как уже говорилось, прямой перевод редко окажется хорошей идеей. Вот пример, который для выражения внешнего действия опирается на хорошо известный словарь (в данном случае значения из библиотеки Schema.org, <https://schema.org>), несмотря на то что сервисом он не поддерживается.

Рассмотрим простой шаблон внутреннего метода для обновления пользовательского объекта:

```
function updateUser(id, fname, lname, email) {
    ...

    return userObject
}
```

Теперь взглянем на действие интерфейса (выражено как шаблон Collection + JSON), используя имена свойств, взятые из библиотеки Schema.org:

```
{
  "collection" {
    "template" : {
      "data" : [
        {"name" : "identifier", "value" : "q1w2e3r4", "prompt" : "User ID"},
        {"name" : "givenName", "value" : "Mark", "prompt" : "First Name"},
        {"name" : "familyName", "value" : "Morkelsen", "prompt" : "Last Name"},
        {"name" : "email", "value" : "mork@example.org", "prompt" : "Email"}
      ]
    }
  }
}
```

Наконец, код, который принимает действие обновления, может обработать перевод внутренних имен во внешние. Для этого примера метод, который выполняет отображение во внешнюю форму, задействует имена свойств из Schema.org (`identifier`, `givenName`, `familyName` и `email`), сохраняя их значения в локальных свойствах пользовательского объекта (`id`, `fname`, `lname` и `email`):

```
function updateAction(identifier, givenName, familyName, email) {
  var user = data.read(identifier);
  if(user) {
    user.id = identifier;
    user.fname = givenName;
    user.lname = familyName;
    user.email = email;
    user = data.write(user);
  }
  return user;
}
```

Бывают случаи, когда в интерфейсе есть внешние действия, которые не отображаются напрямую во внутренние функции. Рассмотрим одно из таких действий:

```
<form name="declineContract" method="post" action = "...">
  <input name="customerId" value="q1w2e3r4t5" />
  <input name="contractId" value="o9i8u7y6.t5r4" />
  <input name="salesRepName" value="Mandy Miningham" />
  <input name="reviewerName" value="Mark Morkelsen" />
  <input name="reasonCode" value="Q201.B" />
  <textarea name="comments">Unable to locate collateral</textarea>
  <input type="submit" value="Decline Contract" />
</form>
```

В коде внутреннего сервиса нет метода `declineContract`. Тем не менее есть несколько действий, которые необходимо завершить при отклонении контракта.

1. Создать в журнале запись `declined`.
2. Обновить запись `customer`.
3. Обновить запись `contract`.

4. Обновить запись `salesRep`.
5. Обновить запись `reviewer`.
6. Обновить агентство `nationalCredit`.

В этом случае API сервиса наверняка содержит функции, которые ему нужно выполнить в отношении нескольких зависимых сервисов. Некоторые из этих функций могут быть локальными (например, внутренние объекты в коде сервиса), а некоторые — удаленными (например, другие внешние сервисы вроде агентства `nationalCredit`). Все эти детали скрыты от потребителя API.



В последнем примере не показано никакой обработки возможных ошибок, сбоев обновлений данных и т. п. Подробнее о том, как обрабатывать эти случаи, читайте в главе 7.

Обсуждение

Внедрение практики проектирования внешних действий — отличный способ улучшить характеристику слабой связанности вашего API. Это особенно актуально, когда вы создаете внешние действия, не имеющие прямого аналога в виде сервиса, поскольку изменения в базовом сервисе необязательно должны напрямую отражаться в этих независимых действиях. По сути, чем большую работу вы продельваете для проектирования действий интерфейса, тем меньше вероятность того, что будущие изменения сервиса, даже кажущиеся критическими, негативно повлияют на этот API.

Иногда сервис будет иметь последовательность внутренних операций (`intializeCustomer`, `collectUserData`, `collectBankData` и `finalizeCustomerData`). По возможности желательно выражать их в виде одного внешнего действия, собирая все важные данные и затем позволяя коду API разобраться со всей последовательностью. Это можно реализовать в виде одной формы со всеми входными данными или при необходимости использовать рабочий поток в стиле «работа выполняется» (см. рецепт 7.10). Так вы избежите необходимости обрабатывать будущие изменения в порядке последовательности действий, а также этапа добавления или удаления.

Бывают случаи, когда внутренние сервисы в операциях управления опираются на логические флаги, например:

```
function approveUser(bool) {  
  
    if(bool===true) {  
        ...  
    }  
    else {  
        ...  
    }  
}
```



```

    return results;
}

```

Логические операции лучше не раскрывать для внешнего действия. Это ограничит возможность изменения или расширения их списка со временем. Вместо этого желательно раскрывать перечисления (здесь выражены в HAL-FORMS), позволяя клиентам и сервисам решать, как отрисовывать эти значения:

```

{
  "_templates" : {
    "default" : {
      ...
      "properties" : [
        {
          "name" : "approveUser",
          "prompt" : "User Approval",
          "options" : {
            "selectedValues" : ["No"],
            "inline" : ["No", "Yes"]
          }
        }
      ]
    }
  }
}

```

Теперь код внутри API перед вызовом внешнего сервиса может переводить значения "Yes"/"No" в `true` или `false`. Кроме того, если дальнейшие изменения сервиса будут приводить к возможному статусу `pending`, элемент `options` формы можно будет обновить без изменения API.

В большинстве своих реализаций я пишу небольшую функцию отображения, которая автоматизирует задачу перевода внутренних и внешних значений на основе набора объявлений. В итоге мне приходится обновлять правила для всего API только в одном месте.

Читайте также

- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 6.3. Соккрытие связей данных для внешних действий.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.
- Рецепт 7.17. Поддержка локальной отмены или отката.

5.5. Оповещение клиента о поддерживаемых настройках

Протокол HTTP предоставляет множество связанных с содержимым параметров, которые клиенты могут использовать для обозначения своих предпочтений. HTTP поддерживает набор параметров заголовков, метод `OPTIONS` и значения ссылочных связей. Тем не менее иногда нам нужен более широкий способ для определения и документирования клиентских пожеланий относительно взаимодействий. Решением будет совмещение опций протокола HTTP и нескольких дополнительных семантических ссылок посредством отдельного адресуемого ресурса параметров.

Задача

Как сервис может последовательным и масштабируемым способом сообщить потребителям API обо всех доступных им настройках? Какие HTTP-заголовки доступны? Когда есть смысл использовать HTTP-метод `OPTIONS`? Когда лучше опираться на ссылочную связь и другие значения для указания возможностей API сервиса?

Решение

Одним из преимуществ использования протокола HTTP для доступа к сетевым сервисам является то, что у него есть множество настраиваемых значений, которые клиенты и серверы могут задействовать для согласования деталей параметров запросов и ответов. По сути, сервисы могут анонсировать один или несколько выбираемых аспектов обмена сообщениями, а клиенты — использовать эту информацию для оповещения сервисов о своих предпочтениях, и все это в среде выполнения.



Вы заметите, что этот список содержит элементы из множества источников, таких как протокол HTTP, HTML-свойство `FORMS` и даже значение указанной ссылочной связи. Работа по анонсированию доступных для выбора опций ответа имеет сложную историю, поэтому нам приходится обходиться имеющимися возможностями.

Далее приведен краткий обзор всех настраиваемых значений в HTTP с описанием того, как клиенты и сервисы могут применять их для адаптации обмена сообщениями к своим потребностям.

Асепт

HTTP-заголовок `Accept` (<https://oreil.ly/vxcG8>) используется для возвращения списка поддерживаемых медиатипов. Несмотря на то что это необязательно, значения в этом списке должны быть взяты из документа IANA Media Types (<https://oreil.ly/5ePjT>) или какого-либо другого источника.

Allow

HTTP-заголовок **Allow** (<https://oreil.ly/Ku60c>) применяется для возвращения списка поддерживаемых сервисом HTTP-методов. Значения, входящие в этот список, должны быть взяты из IANA Hypertext Protocol Method Registry (<https://oreil.ly/dXZ1d>) — реестра методов протокола передачи гипертекста IANA.

enctype

Свойство **enctype** HTML-форм (<https://oreil.ly/dvo4t>) используется для возвращения списка поддерживаемых медиатипов запроса. И хотя это необязательно, значения для этого списка должны быть взяты из документа IANA Media Types (<https://oreil.ly/hD5CN>) или другого источника.

charset

HTTP-параметр **charset** (<https://oreil.ly/idWys>) применяется для указания списка поддерживаемых в телах ответов наборов символов. Значения, содержащиеся в этом списке, должны быть взяты из документа IANA Character Sets (наборы символов IANA) (<https://oreil.ly/yMJjy>).

encoding

HTTP-параметр **encoding** (<https://oreil.ly/GVk6K>) используется для возвращения списка поддерживаемых в телах ответов кодировок. Значения для этого списка следует брать из IANA Content Encoding Registry (<https://oreil.ly/MNZRU>) — реестра кодировок содержимого IANA.

language

HTTP-тег **language** (<https://oreil.ly/0muZk>) применяется для возвращения списка поддерживаемых в телах ответа естественных языков. Допустимые для этого списка значения нужно брать из IANA Language Subtag Registry (<https://oreil.ly/7YGxV>) — реестра субтегов языков IANA.

profile

Значение ссылочной связи **profile** (<https://oreil.ly/7daeB>) используется для возвращения списка поддерживаемых в ответах профилей. Содержимое этого элемента может быть представлено списком валидных URI, отражающих поддерживаемые словари или другие документы с описаниями (подробнее — в рецепте 5.7).



HTTP поддерживает еще одно значение для настройки — **accept-ranges**. Оно позволяет клиентам сообщать сервисам о том, что они с помощью заголовка **ranges** могут поддерживать в ответах значения диапазона. Подробности описаны в спецификации HTTP (<https://oreil.ly/auRyS>).

Информация обо всех этих параметрах должна быть доступна клиентскому приложению посредством HTTP-метода **OPTIONS** или представления ресурсов, возвращаемого при переходе по значению ссылочной связи **"meta"** (**rel="meta"**) (<https://oreil.ly/TztQq>). Подробности показаны в примере.

Клиенты могут запрашивать информацию о поддерживаемых настройках и использовать значения из полученного ответа в очередном запросе, сообщая сервису, какие форматы, языки, кодировки и пр. ему следует применять.

Даже когда сервис поддерживает всего один возможный ответ, например `"language": "en"`, он все равно должен возвращать полный ресурс `meta`, чтобы помочь клиентам найти значения для этих важных параметров.

Стоит поддерживать ссылку на ресурс `"Home"` интерфейса сервиса, указывающий на ресурс `meta` с доступными для клиента настройками. В нем клиенты найдут список всех доступных им опций, которые поддерживает сервис, и, возможно, их значения.

На рис. 5.2 показана схема взаимодействия, включающего поддержку предоставляемого ресурсом `meta` списка доступных для клиента опций.

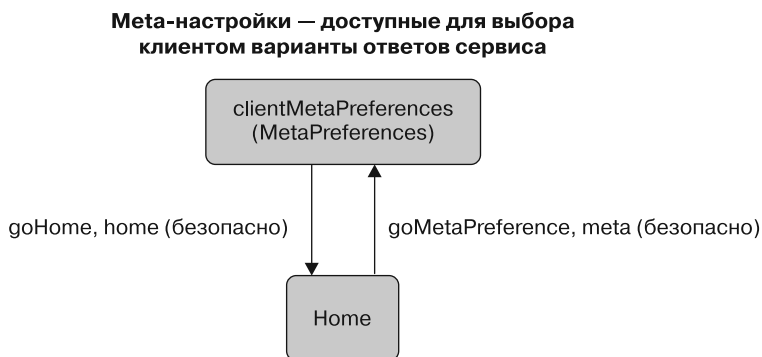


Рис. 5.2. Рецепт настроек `meta`



Подробности о ресурсе `meta` можно узнать из онлайн-коллекции документов ALPS (<https://oreil.ly/RKhJ8>), связанных с этой книгой.

В следующем разделе показаны примеры того, что может возвращать API, когда активна ссылка `meta`.

Пример

В случае запроса представление ресурса `meta` должно перечислять все доступные для клиента настройки, которые поддерживает API, вместе с их значениями. Естественно, может быть, что API сервиса не поддерживает никаких настроек

(или поддерживает какие-то). В таком случае содержимое ответа будет соответствующим.

Вот пример (в формате HAL) ответа на ссылку `meta` или выполнение HTTP-метода `OPTIONS`:

```
{
  "_links" : {
    "self" : {"href" : "http://api.example.com/user-service/meta-preferences"},
    "home" : {"href" : "http://api.example.com/user-service/"},
    "profile" : {"href" :
      "https://webapicookbook.github.io/alps-documents/meta/meta-preferences.json"
    }
  },
  "allow" : "GET PUT PATCH DELETE HEAD OPTIONS",
  "accept" : "application/vnd.hal+json application/vnd.collection+json",
  "enctype" : "application/x-www-form-urlencoded application/json",
  "charset" : "utf-8, iso-8859-1;q=0.7",
  "encoding" : "deflate gzip compress",
  "language" : "en es fr",
  "profile" : "https://alps.example.org/fhir-4.0.1"
}
```

И хотя спецификация HTTP не требует включать в ответ `OPTIONS` тело, будет хорошей практикой в любом случае возвращать это представление `meta-preferences`. Когда `OPTIONS` специфичны для конкретного ресурса, сервисы могут соответствующим образом корректировать ответ `meta-preferences`. Например, ссылка для скачивания может поддерживать более одного формата. Вот пример ответа `OPTIONS` в формате `SIREN`:

```
**** REQUEST ****
OPTIONS /file-system/download HTTP/1.1
Host: api.example.org
...

*** RESPONSE ***
HTTP/1.1 200 OK
Content-Type: application/vnd.siren+json
Content-Length: XX
Cache-Control: max-age=604800
Allow: GET PUT DELETE HEAD OPTIONS
Accept: application/zip application/gzip
Accept-Charset: utf-8
Accept-Encoding: compress
Accept-Language: en
Link: <https://webapicookbook.github.io/alps-documents/about/about.json>; \
  rel="profile"
```

```
{
  "class" : [ "meta preferences" ],
  "links" : [
    { "rel" : [ "self" ], "href" : "http://api.example.org/file-system/download" },
    { "rel" : [ "home" ], "href" : "http://api.example.org/file-system/" },
    { "rel" : [ "profile" ],
      "href" : "https://api.example.org/profiles/meta-preferences.json"
    }
  ],
  "properties" : {
    "allow" : "GET PUT DELETE HEAD OPTIONS",
    "accept" : "application/zip application/gzip",
    "enctype" : "application/json",
    "charset" : "utf-8",
    "encoding" : "compress",
    "language" : "en",
    "profile" : "https://alps.example.org/downloads/"
  }
}
```

Здесь мы видим, что и пространство заголовка, и тело ответа содержат список доступных для клиента параметров настройки. Стоит включить оба этих набора информации, поскольку некоторые клиентские приложения могут парсить только данные заголовка, пропуская содержимое тела ответа.

Обсуждение

Редко случается, что один интерфейс сервиса поддерживает весь диапазон представленных здесь настроек. Чаще всего клиентов интересуют `allow`, `accept`, `enctype` и иногда `language`. Опции `charset` и `encoding` указываются редко.

Даже если интерфейс сервиса поддерживает всего одно значение элемента настроек, этот элемент должен быть указан в ответе `meta-preferences`. Таким образом, ответ `meta-preferences` становится документацией опций сервиса в среде выполнения и потребители API могут опираться на него для получения актуального списка доступных параметров.



Не забудьте включить в свою документацию API элемент, сообщающий разработчикам, что ваш интерфейс поддерживает `OPTIONS` и/или значение ссылочной связи `meta`.

Есть недостаток в использовании для передачи клиенту информации о поддерживаемых настройках только заголовка `OPTIONS`. В спецификации HTTP ясно сказано, что `OPTIONS` некешируемый. Если у вас будет множество потребителей API, отправляющих запросы `OPTIONS`, это может вызвать проблемы с масштабированием интерфейса. При этом предоставление отдельного ресурса `meta-preferences` (например, `<link rel="meta" href="http://api.example.org/meta-preferences/" />`) означает, что вы можете наделить его долго хранящимся кэшем и снизить затраты на составление и возвращение ответов с доступными для клиентов настройками.

Читайте также

- Рецепт 3.3. Предоставление информации о предметной области через публикуемые словари.
- Рецепт 4.5. Согласование поддержки профилей во время выполнения.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.8. Поддержка общих словарей в стандартных форматах.

5.6. Поддержка согласования HTTP-содержимого

Одним из уникальных аспектов протокола HTTP является возможность выбирать предпочтительные форматы для тела ответа. Позаимствованная из спецификаций электронной почты реализация заголовка `content-type` для HTTP привела к созданию почти 1500 медиатипов, зарегистрированных в официальном реестре IANA Media Types. Причем каждый год их список пополняется. Сложность же состоит в том, чтобы понять, какой медиатип использовать, когда использовать и как сделать возможным для клиентов и сервисов согласование содержимого сообщений.

Задача

HTTP позволяет клиентам и сервисам начинать процесс согласования содержимого, в ходе которого они выбирают, какой медиатип применять при обмене сообщениями. Как это работает? В чем различие между упреждающим и реактивным согласованием содержимого? И что интерфейсам сервисов нужно делать для поддержки согласования содержимого, когда это уместно?

Решение

Важной особенностью HTTP является возможность поддержки нескольких форматов ответа. Это позволяет одному и тому же ресурсу возвращать HTML для одних потребителей, CSV для других и т. д. Еще больше возможностей обеспечивает способность HTTP согласовывать формат ответа в среде выполнения. Иными словами, потребители и производители API могут при каждом запросе решать, какой формат сообщений использовать в ответе.

В спецификации HTTP описаны два основных вида согласования содержимого: упреждающий и реактивный (<https://oreil.ly/3Q9FP>). Каждый имеет собственные преимущества и недостатки.

Упреждающее согласование содержимого

Упреждающее согласование содержимого (proactive content negotiation, PCN, <https://oreil.ly/CCqJm>) упрощает задачу в первую очередь для HTTP-сервисов. В случае его применения клиенты API посылают список из одного или нескольких

медиа типов, предпочтительных для использования в ответах, а сервер с помощью полученного списка решает, какой из них задействовать. Иногда это называется согласованием под руководством сервера.

Клиенты указывают свои предпочтения, применяя в запросах заголовок `Accept`, а серверы обозначают свой итоговый выбор, используя в ответах заголовок `Content-Type` (примеры будут приведены далее).

В случае PCN клиенты высказывают серверам свои предпочтения, а те уже принимают итоговое решение относительно формата ответа. При этом сервис может ответить форматом, *не* включенным в предоставленный клиентом заголовок `Accept`. На показанном здесь примере с запрошенным форматом `JSON` мы увидим, что, если сервис такие форматы не поддерживает, он может ответить, используя `text/plain` или `application/HTML`.

Реактивное согласование содержимого

Альтернативой упреждающему согласованию со стороны сервиса выступает его реактивный вариант (reactive content negotiation, RCN, <https://oreil.ly/RJT0v>). В этом случае сервис отправляет клиенту список возможных форматов представления, и тот должен выбрать предпочтительный, указав его во втором запросе. Такой механизм передает инициативу выбора в руки клиентского приложения, но также подразумевает, что каждый раз будет выполняться дополнительный круг обмена сообщениями.

RCN отлично работает, когда имеется множество переменных (язык, формат сообщений, схема кодировки и т. д.), но детали взаимодействия клиента и сервера для выбора предпочтительного формата в спецификации HTTP не прописаны. Типичным подходом является возвращение сервером HTTP-кода состояния `300 Multiple Choices` с телом ответа, содержащим список опций для просмотра клиентами. Ответ `300 Multiple Choices` (<https://oreil.ly/0AP9I>) зачастую включает в себя ряд заголовков `Link` и, возможно, заголовок `Location` на случай, если клиент захочет просто выполнить автоматическое перенаправление.

Пример

HTTP поддерживает как упреждающую, так и реактивную схему согласования содержимого ответов. Далее приводится несколько примеров.

Упреждающее согласование

При использовании PCN клиент отправляет список предпочтительных форматов ответа в заголовке `Accept`, а сервер делает итоговый выбор:

```
**** REQUEST ****
GET /list HTTP/1.1
Accept: application/vnd.siren+json, application/vnd.hal+json, application/json
...
```



```
**** RESPONSE ****
```

```
200 OK HTTP/1.1
```

```
Content-Type: application/json
```

```
....
```

В этом случае сервер выбрал в качестве медиатипа `application/json`. Клиенты могут предоставлять дополнительную информацию о своих предпочтениях в виде параметра *quality* (качество), выражаемого значением *q*, например:

```
**** REQUEST ****
```

```
GET /list HTTP/1.1
```

```
Accept: application/vnd.hal+json;q=0.8, application/json;q=0.4
```

```
...
```

```
**** RESPONSE ****
```

```
200 OK HTTP/1.1
```

```
Content-Type: application/json
```

```
....
```

В этом примере клиент указал на повышенный интерес к применению медиатипа `application/vnd.hal+json`. Тем не менее сервер все равно выбрал для ответа формат `application/json`.



Подробнее о значениях качества (*q*) читайте в спецификации HTTP (<https://oreil.ly/RDTl5>).

Реактивное согласование содержимого

При использовании схемы RCN сервер возвращает список поддерживаемых вариантов представления, а клиент принимает итоговое решение, повторно отправляя запрос:

```
**** REQUEST ****
```

```
GET /search HTTP/1.1
```

```
**** RESPONSE ****
```

```
HTTP/1.1 300 Multiple Choices
```

```
Link: <http://api.example.org/html/search>;rel="alternate html"
```

```
Link: <http://api.example.org/api/search>;rel="alternate api"
```

```
Location: http://api.example.org/html/search
```

В очередном ответе по схеме RCN возвращаются значения ссылок в виде содержащихся в теле HTML-тегов `anchor`:

```
**** REQUEST ****
```

```
GET /search HTTP/1.1
```

```
**** RESPONSE ****
```

```
HTTP/1.1 300 Multiple Choices
```

```
Content-Type: text/html
```

```
<html>
  <title>Multiple Choices</title>
  <body>
    <h1>Multiple Choices</h1>
    <ul>
      <li><a href="http://api.example.org/html/search">HTML</a></li>
      <li><a href="http://api.example.org/api/search">API</a></li>
    </ul>
  </body>
</html>
```

Обсуждение

Когда сервисы поддерживают несколько форматов представления, чаще всего используется упреждающая схема согласования содержимого. Это простейший вариант взаимодействия, несмотря на то что клиент может не получить предпочтительный формат.

У RCN есть существенные недостатки. В этом случае серверы, по сути, угадывают, какой формат нужно вернуть. Такая схема хорошо подходит, когда клиент и сервис поддерживают одни и те же форматы, но совсем не к месту, когда прямых совпадений нет. В таком случае сервисы, как правило, просто возвращают то, что предпочтительно *для них*. Единственной реальной альтернативой для сервисов будет возврат кода состояния **406 Not Acceptable**, когда они не могут предоставить формат, соответствующий предпочтениям клиента, которые он указал в заголовке **Accept**.

Недостатком использования PCN является то, что клиенты зачастую не знают, что конкретно сервер поддерживает. Сервисы могут решить эту проблему реализацией поддержки паттерна «meta», позволяющего им сообщить клиенту о доступных опциях (см. рецепт 5.5).

Я редко встречаю реактивные версии согласования содержимого в реальной работе. Поскольку детали ответа при такой схеме согласования отчетливо не прописаны, не советую использовать ее для M2M-взаимодействий, если только обе стороны не договорятся об этих деталях заранее.

Одним из способов избежать «танцев» согласования для клиента и сервиса является выделение пространств URL для каждого формата представления, например:

- <http://api.example.org/siren/search>;
- <http://api.example.org/hal/search>;
- <http://api.example.org/collection-json/search>;
- <http://api.example.org/html/search>.

Естественно, недостатком в таком случае будет то, что интерфейсу сервиса потребуется поддерживать намного больше путей URL.

В моей работе самым эффективным зарекомендовал себя подход с указанием поддерживаемых форматов в документации API на этапе проектирования с последующим оповещением сервиса о предпочтениях клиента (см. рецепт 5.5) и использованием в среде выполнения упреждающей схемы согласования.

Читайте также

- Рецепт 3.1. Обеспечение интероперабельности с помощью зарегистрированных медиатипов.
- Рецепт 4.5. Согласование поддержки профилей во время выполнения.
- Рецепт 5.5. Оповещение клиентов о поддерживаемых настройках.
- Рецепт 6.7. Использование медиатипов для запросов данных.

5.7. Публикация полноценных словарей для клиентов

При создании сервисов, с которыми будут взаимодействовать другие сервисы (например, M2M-интерфейсы), важно обеспечить, чтобы обе стороны хорошо понимали друг друга. То есть каждое сообщение должно быть полноценным, и обе стороны должны понимать, что в нем содержится. В этом заключается вся суть RESTful-реализаций. Рой Филдинг объясняет это так: «RESTful API (правильно реализованное) — это просто сайт для клиентов, имеющий ограниченный словарь» (<https://oreil.ly/dxgtv>).

Задача

Как сделать так, чтобы потребитель API правильно понимал ответы сервиса? Что подразумевается под отправкой самоописательных сообщений? Что нужно включать в эти описания?

Решение

Лучшим способом обеспечить понимание потребителем API сообщений этого API будет создание *полноценного* словаря, содержащего все важные свойства данных и действий, которые могут встречаться в сообщении. Леонард Ричардсон называет эти значения магическими строками (<https://oreil.ly/ABEku>), которые несут некий смысл. Используя эти строки, потребители и производители API могут сообщать друг другу важную информацию. В этой книге я называю такие коллекции магических строк *словарями* и иногда *профилями*.

В RFC 6906 (<https://oreil.ly/7daeB>) ссылочная связь `profile` определяется как «позволяющая клиентам узнать дополнительную семантику (ограничения, соглашения, расширения), связанную с представлением ресурса». Применение профиля к ресурсу не *меняет* содержания сообщения. Напротив, профили предоставляют *дополнительную* информацию о сообщениях.

ПРОФИЛИ ПОД ДРУГИМИ НАЗВАНИЯМИ

Я встречал этот список свойств данных и действий под разными именами, такими как словарь, глоссарий и даже схема. Но я бы не стал использовать термин «словарь данных» или «модель данных», потому что словари сервисов определяют не только свойства данных или именованных объектов, но и все детали действий, включая ссылки, формы и строки запросов.

Профиль сервиса содержит *все* идентификаторы, которые он применяет в сообщениях. Рассмотрите, к примеру, следующую HTML FORM:

```
<form name="create-template" action="http://api.example.org/users"
  method="post" enctype="application/x-www-form-urlencoded">
  <input name="id" value="q1w2e3r4" />
  <input name="familyName" value="Mark" />
  <input name="givenName" value="Morkelson" />
  <input name="telephone" value="+1-555-123-4567" />
  <input name="email" value="mork@example.org" />
  <input name="status" value="active" />
  <input type="submit" />
</form>
```

В этой форме словарь передает следующую информацию: `create-template`, `id`, `familyName`, `givenName`, `telephone`, `email` и `status`. И сервис, и потребитель API должны распознавать и понимать, что эти элементы представляют. Заметьте, что я не привел здесь *значения* этих элементов, например `id=q1w2e3r4` — только их имена. Я также не включил в этот список структурные элементы медиатипов, например `name`, `action`, `method`, `enctype` и т. д.

Недостаточно просто определить элементы словаря. Нужно еще пояснить, что они представляют и каким образом могут присутствовать в сообщении. Например, семантическое значение `create-template` в HTTP-сообщениях может быть представлено как значение `name` элемента `form`. Это определяет форму, которую клиенты могут использовать для добавления в систему новых записей.

В документы семантических профилей можно включать также дополнительную информацию, такую как источник значения строки. Например, определение `familyName` находится в онлайн-словаре Schema.org (<https://oreil.ly/bKUaB>).

Пример

Далее представлены примеры словарей (семантических профилей) для API person, находящегося в репозитории книги (<https://oreil.ly/SamlR>). Вот образец вывода этого сервиса:

```
{
  "collection": {
    "version": "1.0",
    "href": "http://localhost:8181",
    "title": "BigCo Activity Records",
    "links": [
      {
        "name": "home", "href": "http://localhost:8181/",
        "rel": "home", "prompt": "Home"
      },
      {
        "name": "list", "href": "http://localhost:8181/list/",
        "rel": "list", "prompt": "List"
      }
    ],
    "items": [
      {
        "id": "22s3k36pkn4",
        "rel": "person",
        "href": "http://localhost:8181/22s3k36pkn4",
        "data": [
          {"name": "id", "value": "22s3k36pkn4", "prompt": "id"},
          {"name": "givenName", "value": "Mork", "prompt": "givenName"},
          {"name": "familyName", "value": "Mockery", "prompt": "familyName"}
        ],
        "links": [
          {
            "name": "read", "href": "http://localhost:8181/22s3k36pkn4",
            "rel": "read", "prompt": "Read"
          }
        ]
      }
    ],
    "queries": [
      {
        "name": "filter",
        "href": "http://localhost:8181/filter/",
        "rel": "filter",
        "prompt": "Search",
        "data": [
          {"name": "givenName", "value": ""},
          {"name": "familyName", "value": ""}
        ]
      }
    ]
  },
}
```

```
"template": [
  { "name": "id", "value": "", "prompt": "id" },
  { "name": "givenName", "value": "", "prompt": "givenName" },
  { "name": "familyName", "value": "", "prompt": "familyName" }
]
}
```

Как минимум вам следует перечислить все семантические идентификаторы и включить описание (табл. 5.2).

Таблица 5.2. Семантический профиль API Person

Идентификатор	Описание
id	Идентификатор записи данных человека
givenName	Собственное имя человека
familyName	Фамилия человека
person	Указатель на запись данных человека
home	Средство гипермедиа для перехода на представление Home
list	Средство гипермедиа для перехода на представление List
read	Средство гипермедиа для перехода в одну запись данных человека
filter	Средство гипермедиа для фильтрации представления List

В качестве дополнительной информации можно указать места возможного появления в сообщении каждого элемента (табл. 5.3).

Таблица 5.3. Семантический профиль Person API с указанием расположения элементов

Идентификатор	Описание	Элемент
id	Идентификатор записи данных человека	name
givenName	Собственное имя человека	name
familyName	Фамилия человека	name
person	Указатель на запись данных человека	rel
collection	Указатель на коллекцию записей	rel
item	Указатель на одну запись	rel
home	Средство гипермедиа для перехода на представление Home	rel, name
list	Средство гипермедиа для перехода на представление List	rel, name
read	Средство гипермедиа для перехода в одну запись данных человека	rel, name
filter	Средство гипермедиа для фильтрации представления List	rel, name

Важно помнить, что третий столбец в табл. 5.3 (Элемент) наверняка будет содержать разные элементы в зависимости от возвращенного медиатипа. Например, если сервис вернет представления `person`, используя медиатип `application/forms+json`, получится табл. 5.4.

Таблица 5.4. Семантический профиль API Person с указанием расположения медиатипов

Идентификатор	Описание	Элемент (Cj)	Элемент (Fj)
id	Идентификатор записи данных человека	name	KEY
givenName	Собственное имя человека	name	KEY
familyName	Фамилия человека	name	KEY
person	Указатель на запись данных человека	rel	KEY
collection	Указатель на коллекцию записей	rel	rel
item	Указатель на одну запись	rel	rel
home	Средство гипермедиа для перехода на представление Home	rel, name	rel, name, id
list	Средство гипермедиа для перехода на представление List	rel, name	rel, name, id
read	Средство гипермедиа для перехода в одну запись данных человека	rel, name	rel, name, id
filter	Средство гипермедиа для фильтрации представления List	rel, name	rel, name, id

Заметьте, что для формата FormsJSON некоторые семантические значения указываются как структурные элементы в виде JSON (`id`, `givenName`, `familyName` и `person`), а другие — в виде элементов значений (`collection`, `item`, `list`, `home`, `read` и `filter`). Именно такая смесь структуры и значений делает каждый медиатип уникальным. Если ваш сервис поддерживает более одного медиатипа, потребуется передавать информацию о семантических профилях, в которой семантические элементы будут правильно сопоставлены с элементами сообщения:

```
{
  "person": {
    "links": [
      {
        "id": "home",
        "name": "home",
        "href": "http://localhost:8181/",
        "rel": "home",
        "title": "Home",
        "method": "GET",
        "properties": []
      },
      {
        "id": "list",
```

```
    "name": "list",
    "href": "http://localhost:8181/list/",
    "rel": "list collection",
    "title": "List",
    "method": "GET",
    "properties": []
  },
  {
    "id": "filter",
    "name": "filter",
    "href": "http://localhost:8181/filter/",
    "rel": "filter collection",
    "title": "Search",
    "method": "GET",
    "properties": [
      {"name": "givenName", "value": ""},
      {"name": "familyName", "value": ""}
    ]
  }
],
"items": [
  {
    "links": [
      {
        "id": "read_22s3k36pkn4",
        "name": "read",
        "href": "http://localhost:8181/22s3k36pkn4",
        "rel": "read item",
        "title": "Read",
        "method": "GET",
        "properties": []
      }
    ],
    "id": "22s3k36pkn4",
    "givenName": "Mork",
    "familyName": "Mockery"
  }
]
}
```

Обсуждение

По сути, семантические профили несут в себе элементы предметной области (person, list, filter и т. д.) сервиса, так что их можно рассматривать как способ документирования этой области.

Стоит отметить, что семантические профили не поясняют, *как* работают средства гипермедиа filter. Они лишь сообщают, что эти средства могут присутствовать

в ответе. Семантические профили предполагают, что потребители API осознают независимость медиатипа от профиля. Это разделение между пониманием форматов сообщений и пониманием их *внутренней* семантики очень важно для грамотного построения гипермедиа-сервисов.

Когда ваш сервис вместо зарегистрированного медиатипа использует формат сериализации вроде простого JSON или XML, обычно он должен встраивать семантику своей предметной области в структуру сообщения. Например, вот возможный ответ от сервиса `person` в формате XML:

```
<list>
  <home href="..." rel="collection"/>
  <list href="..." rel="collection"/>
  <filter href="..." method="get" rel="collection">
    <givenName></givenName>
    <familyName></familyName>
  </filter>
  <person id="..." rel="item read" href="...">
    <familyName>...</familyName>
    <givenName>...</givenName>
  </person>
</list>
```

Его отображение в семантический профиль представлено в табл. 5.5.

Таблица 5.5. Семантический профиль API Person с указанием расположения JSON и XML

Идентификатор	Описание	Элемент (Cj)	Элемент (Fj)	Элемент (XML)
id	Идентификатор записи данных человека	name	KEY	ATTRIBUTE
givenName	Собственное имя человека	name	KEY	ELEMENT
familyName	Фамилия человека	name	KEY	ELEMENT
person	Указатель на запись данных человека	rel	KEY	ELEMENT
collection	Указатель на коллекцию записей	rel	rel	rel
item	Указатель на одну запись	rel	rel	rel
home	Средство гипермедиа для перехода на представление Home	rel, name	rel, name, id	ELEMENT
list	Средство гипермедиа для перехода на представление List	rel, name	rel, name, id	ELEMENT
read	Средство гипермедиа для перехода в одну запись данных человека	rel, name	rel, name, id	rel
filter	Средство гипермедиа для фильтрации представления List	rel, name	rel, name, id	ELEMENT

В этом случае семантика предметной области настолько переплетена с форматом сообщения, что изменения внутренних элементов этой области могут вызвать структурные изменения во внешнем интерфейсе, например переименование элементов, добавление/удаление атрибутов и т. п. По возможности нужно сохранять независимость деталей сообщений (внешнего интерфейса) от деталей предметной области (внутренней функциональности). Когда я вижу, что семантика предметной области выражена в виде структурных элементов в ответе, это сигнал о том, что мне нужно пересмотреть дизайн сообщений, чтобы исключить утечку внутренних деталей во внешний интерфейс. Подробнее эта тема раскрыта в рецепте 5.4.

Читайте также

- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 5.3. Преобразование внутренних моделей во внешние сообщения.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 5.8. Поддержка общих словарей в стандартных форматах.

5.8. Поддержка общих словарей в стандартных форматах

Когда интерфейсы сервисов внедряют практику использования для обмена сообщениями стандартных структурированных медиатипов, им также нужно стандартизировать способ, которым специфичные для предметной области свойства и действия (или *семантика*) сервиса выражаются в среде выполнения. Это аналогично сложности с выбором формата сообщений. Решением будет применение для выражения смысла стандартизированных форматов. В реальности нужно выразить внутренние данные или объектные модели в виде согласованных внешних семантических моделей.

Задача

Как стандартизировать способ передачи сути содержимого между машинами? Какие форматы доступны для этой работы и какой паттерн использовать для выражения внутренних данных и моделей процессов в виде стандартизированных семантических моделей?

Решение

Аналогично тому как потребители API применяют стандартизированные протоколы (например, HTTP) и форматы сообщений (например, SIREN, Collection + JSON и пр.), они могут применять и стандартизированные словари. Это означает перечисление всех свойств данных и имен действий в стандартном, понятном для клиента формате.

Чтобы это реализовать, потребуется один ресурс для всех возможных имен элементов, которые могут появляться в API. Например, любое значение, которое может присутствовать в атрибутах вроде `id`, `name`, `class`, `rel` или аналогичных структурных элементах. Коллекция этих значений может описывать тему или предметную область, например «управление пользователями» или «управление учетными записями».

Есть два варианта передачи данных словаря в стандартизированной форме. Одно из старейших известных решений — применение форматов Resource Description Framework (RDF, схема описания ресурсов, <https://www.w3.org/RDF>) наподобие RDF/XML (<https://oreil.ly/jmQS3>), Turtle (<https://oreil.ly/UV1eM>) и JSON-LD (<https://oreil.ly/sRV7J>). Существует несколько ориентированных на схемы RDF языков, включая OWL (<https://oreil.ly/YTvwI>) и RDF Schema (<https://oreil.ly/8XOy6>). Чаще всего в подобных случаях мне встречается использование в API формата JSON-LD.

Можно также задействовать документы схем: XSD (<https://oreil.ly/OK7jO>) для XML-сообщений и JSON Schema (<https://json-schema.org/>) для сообщений на основе JSON. И хотя это вполне устоявшиеся форматы, для которых есть множество инструментов, следует знать, что традиционное применение XSD и JSON Schema ограничено определением имен свойств данных словаря и обычно не включает имена действий, например имена форм, идентификаторы ссылочных связей и пр.

Я для документирования словарей использую формат Application-Level Profile Semantics (ALPS, семантика профилей прикладного уровня, <https://oreil.ly/6DLZn>). Существуют сериализации ALPS для XML, JSON и даже YAML.

Пример

Далее показана RDF-версия сервиса `person`, выраженная в словаре FOAF (friend of a friend) (<https://oreil.ly/jVQRj>):

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
```

```
<#JW>
  a foaf:Person ;
```

```
foaf:name "James Wales" ;
foaf:mbox <mailto:jwales@bomis.com> ;
foaf:homepage <http://www.jameswales.com> ;
foaf:nick "Jimbo" ;
foaf:depiction <http://www.jameswales.com/aus_img_small.jpg> ;
foaf:interest <http://www.wikimedia.org> ;
foaf:knows [
  a foaf:Person ;
  foaf:name "Angela Beesley"
] .

<http://www.wikimedia.org>
  rdfs:label "Wikimedia"
```

На рис. 5.3 представлена схема того же RDF-документа с использованием сервиса визуализации `isSemantic` (<https://oreil.ly/UAPe8>).

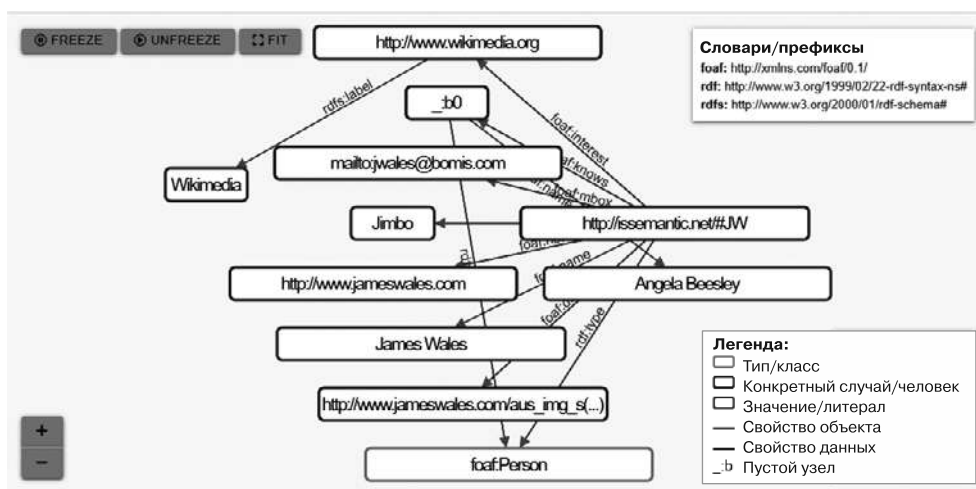


Рис. 5.3. FOAF-представление RDF-версии person в виде FOAF

В качестве другого подхода далее показана часть документа ALPS, описывающего API сервиса `person`, который обсуждается в этом рецепте. Подробнее формат ALPS рассмотрен в рецепте 3.4:

```
{
  "$schema": "https://alps-io.github.io/schemas/alps.json",
  "alps": {
    {
      "version": "1.0",
      "title": "Person Service API",
```

```

"descriptor" : [
...
{"id": "home", "type": "semantic",
  "title": "Home (starting point) of the person service",
  "tag": "taxonomy",
  "descriptor": [
    {"href": "#goHome"},
    {"href": "#goList"}
  ],
  "doc" : {"value" : "Person API starting point"}
},
{"id": "collection", "type": "semantic",
  "title": "List of person resources",
  "tag": "taxonomy",
  "descriptor": [
    {"href": "#person"},
    {"href": "#goHome"},
    {"href": "#goList"},
    {"href": "#goFilter"},
    {"href": "#goItem"},
    {"href": "#doCreate"}
  ],
  "doc" : {"value" : "List of person resources"}
},
{"id": "item", "type": "semantic",
  "title": "Single person resource",
  "tag": "taxonomy",
  "descriptor": [
    {"href": "#person"},
    {"href": "#goHome"},
    {"href": "#goList"},
    {"href": "#goFilter"},
    {"href": "#goItem"},
    {"href": "#doUpdate"},
    {"href": "#doStatus"},
    {"href": "#doRemove"}
  ],
  "doc" : {"value" : "A single person resource"}
},
...
]
}
}

```

На рис. 5.4 мы видим графическое представление всего профиля `person` с использованием инструмента `app-state-diagram (ASD)` (<https://oreil.ly/G7Rnn>).

Ключевое различие между RDF и ALPS в том, что RDF акцентируется на связи между элементами *данных*, а ALPS — на связи между элементами *действия*. Этот пробел хорошо восполняет формат словаря Hydra (<https://oreil.ly/yEGsL>).

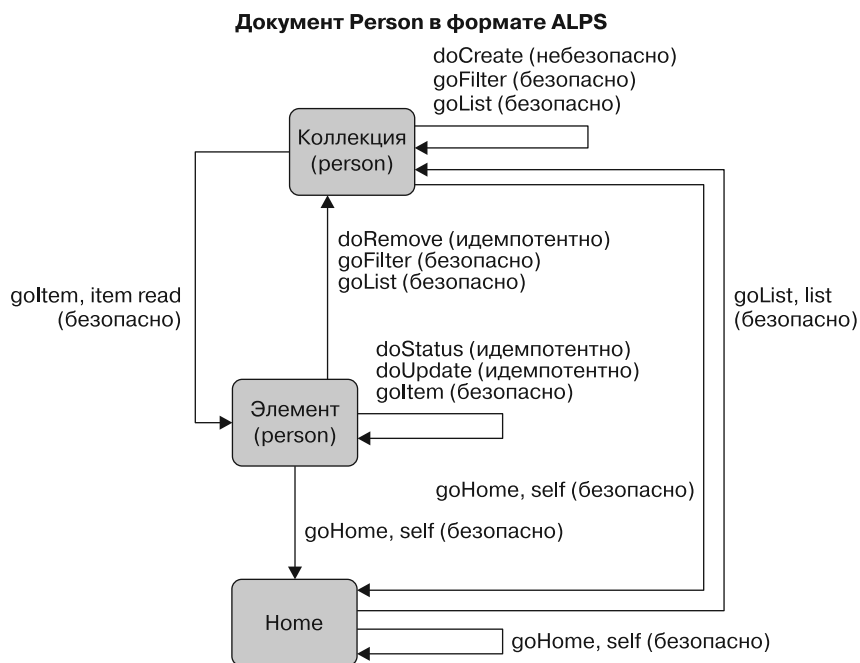


Рис. 5.4. Представление person с помощью документа ALPS

Обсуждение

API сервисов могут использовать URL семантических профилей в качестве *идентификаторов*, при этом от них не требуется точно понимать, что содержится в документе семантических профилей. Например, включая URL профиля в коллекцию заголовков для ответов, API сервиса сообщает потребителям API, с помощью какого словаря он говорит:

```

**** REQUEST ****
GET /shopping/ HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Link: <http://docs.alps.io/shopping-v2.json>; rel="profile"

```

В этом примере сервис объявляет, что поддерживает версию 2 профиля shopping. Подробнее о том, как провайдеры и потребители API могут согласовывать поддержку профилей, читайте в рецепте 4.5.

Профили, которые подробно описывают семантику интерфейса, не выполняют точно такую же роль, что и документы схем. Схемы используются для описания объектов или сообщений документов, а семантические профили — для описания способа, которым эти объекты или документы взаимодействуют друг с другом. Публикация профиля помогает разработчикам (и машинам) понять область задачи, то есть доступные возможности API. В зависимости от применяемых форматов сообщений вам может потребоваться публиковать документы схем, чтобы люди и машины могли понимать типы объектов и документов, передаваемых в рамках области задачи. Подробнее об использовании схем для API читайте в рецепте 4.7.

Важно отметить, что семантические профили вроде ALPS — это не то же самое, что определения API, такие как OpenAPI, AsyncAPI, Protobuf, SOAP и т. д. Они предоставляют полноценный словарь свойств и действий для коллекции API, которые поддерживают этот профиль. При этом документы определений API описывают детали реализации одного экземпляра API сервиса — того, который находится, например, по адресу <http://api.example.org/shopping>.

Важная часть правильной поддержки семантических профилей — отображение их элементов в элементы медиатипов в передаваемых сообщениях. Подробнее о документировании отображения документов профиля в медиатипы читайте в рецепте 5.7.

Желательно включать семантические профили во всю документацию API сервиса. Это поможет потребителям API, предоставив им подробный список элементов словаря и важные подсказки относительно того, как можно перемещаться по области задачи вашего сервиса. Более полный список активов, которые API сервисов должны предоставлять потребителям, приведен в рецепте 5.10.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.5. Согласование поддержки профилей во время выполнения.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 5.6. Поддержка согласования HTTP-содержимого.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.10. Публикация метаданных API.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.

5.9. Публикация документов определений сервисов

Большинству API сервисов нужно публиковать описательные документы определений (Service Definition Document, SDD), которые для успешного взаимодействия должны быть понятными как поставщикам, так и потребителям этих API. Сегодня наиболее распространенный способ добиться этого — публикация определений API в одном из нескольких стандартных форматов.

Задача

Какие существуют стандартные форматы SDD? Как можно легко предоставить SDD потребителям API? Какие при этом нужно использовать URL и/или значения ссылочных связей? Нужно ли в других местах вместе с прямыми ссылками на API указывать также ссылки на SDD? А в других документах?

Решение

Для большинства API есть фиксированный набор URL и тел сообщений, которые предоставляются в среде выполнения. Эти фиксированные конечные точки и объекты можно задокументировать в разных форматах, исходя из стиля реализации интерфейса сервиса.

Вот наиболее распространенные стили API и связанные с ними SDD.

HTTP CRUD

OpenAPI, язык описания веб-приложений (Web Application Description Language, WADL).

Событийно-ориентированные

AsyncAPI, CloudEvents.

Удаленные процедуры

Protocol buffers, XML-RPC, JSON-RPC.

Удаленный запрос данных

Язык описания схем (Schema Definition Language) (для GraphQL), OData.

Система обмена сообщениями

Язык определения веб-сервисов (Web Service Definition Language, WSDL).

Гипермедиа

Семантика профилей уровня приложений (Application-Level Profile Semantics, ALPS).

Как правило, SDD создаются для упрощения работы по генерации человекочитаемой документации и/или написанию потребительского кода API. Иногда SDD генерируются из кода. Например, OpenAPI и WSDL можно сгенерировать на основе существующего исходного кода. На момент написания книги многие из этих форматов пишутся от руки в рамках проектирования интерфейсов.

Но как бы ни создавался документ, желательно публиковать его в легкодоступном месте. Это поможет разработчикам клиентского приложения быстро набрать обороты в успешном использовании API. Лучше всего делать это путем возвращения ссылки на SDD в HTTP-ответах, либо в виде заголовка `link` тела ответа, либо обоими способами. В RFC 8631 прописано, что значение ссылочной связи `service-desc` (<https://oreil.ly/MNR2H>) нужно задействовать при идентификации ссылок, указывающих на документы SDD. Стоит также возвращать указатель на SDD в заголовке `Link` ответа на HTTP-запрос `OPTIONS`. Подробнее — в примерах далее.



Желательно включать ссылку на документ SDD в документ метаданных API (см. рецепт 5.10).

Пример

Есть три удачных места, в которых можно указать расположение SDD вашего API: в заголовке HTTP-ответа `Link`, метаданных сервиса и HTTP-ответе `OPTIONS`.

SDD в HTTP-ответах

Вы можете вернуть указатель на SDD API в заголовке `Link` любого ответа API:

```
HTTP/1.1 200 OK
Content-Type: application/vnd.hal+json
Link: <http://api.example.org/service-desc>; rel=service-desc
...
```

Можно также вернуть эту информацию в теле ответа:

```
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
...
{ "collection" : {
  "links" : [
    { "rel": "service-desc", "href": "http://api.example.org/service-desc" }
  ]
  ...
}}
```

Если формат это поддерживает, вернуть расположение SDD можно и в заголовке, и в теле ответа.

SDD в ответах с метаданными сервиса

Помимо этого, информацию о расположении SDD можно приводить в документе метаданных API (см. рецепт 5.10). Лично я рекомендую использовать для метаданных сервиса формат `APIs.json`, в спецификации которого есть зарезервированные ключевые слова для большинства распространенных форматов SDD.

Вот фрагмент документа APIs.json, указывающего на поддержку OpenAPI SDD:

```
{
  "name": "Example API",
  "type": "Index",
  "created": "2014-04-07",
  "modified": "2020-09-03",
  "url": "http://example.com/apis.json",
  "specificationVersion": "0.14",
  "apis": [
    {
      "name": "Example API",
      "humanURL": "http://example.com",
      "baseURL": "http://api.example.com",
      "properties": [
        {"type": "OpenAPI", "url": "http://example.com/openapi.json"}
      ]
    }
  ]
  ...
}
```



Если вы используете формат SDD, который в спецификации APIs.json на данный момент не описан, то можете добавить собственный формат в качестве расширения.

SDD в HTTP-ответах OPTIONS

Можно включить SDD в заголовок `Link` и/или в тело ответа `OPTIONS`:

```
**** REQUEST ****
```

```
OPTIONS / HTTP/1.1
```

```
Host: api.example.org
```

```
...
```

```
**** RESPONSE ****
```

```
HTTP/1.1 200 OK
```

```
Content-Type: application/vnd.siren+json
```

```
Content-Length: XX
```

```
Cache-Control: max-age=604800
```

```
Allow: GET PUT DELETE HEAD OPTIONS
```

```
Accept: application/zip application/gzip
```

```
Accept-Charset: utf-8
```

```
Accept-Encoding: compress
```

```
Accept-Language: en
```

```
Link: <https://api.example.org/service-desc>; rel="service-desc"
```

Подробнее об использовании заголовка `OPTIONS` читайте в рецепте 5.5.

Обсуждение

Желательно по возможности сохранять документ SDD в корневом каталоге интерфейса сервиса. Будет нелишним назвать этот документ `service-desc`. Если вы будете последовательно выполнять эти два условия, то потребители API смогут догадаться о расположении SDD, что упростит задачу для разработчиков API.

Описанная коллекция форматов — это не исчерпывающий их список. Читатель может использовать и другие форматы SDD, а некоторые крупные организации даже создают собственные их варианты. Я рассмотрел те SDD, которые в своей практике встречал чаще всего.

Бывает, что сервис поддерживает несколько стилей интерфейсов, например HTTP CRUD и GraphQL. В таком случае можно возвращать в заголовке или теле ответов несколько документов SDD:

```
HTTP/1.1 200 OK
Content-Type: application/vnd.hal+json
Link: <http://api.example.org/openapi/service-desc>; rel="service-desc"; \
type="application/openapi+json"
Link: <http://api.example.org/graphql/service-desc>; rel=service-desc"; \
type="application/sdl+json"
...
```

Для указания ожидаемого формата ответа можно использовать свойство `type` заголовка `Link` (см. предыдущий пример). Проблема здесь в том, что у большинства форматов SDD в IANA (<https://oreil.ly/EEAst>) нет собственной строки идентификаторов медиатипа. Лично я собрал для работы конкретный набор идентификаторов медиатипов и включаю их в документацию API. Это не идеальное решение, но оно дает ожидаемый результат. Надеюсь, к моменту прочтения вами этой книги уже большее число форматов SDD будут иметь собственные зарегистрированные идентификаторы медиатипов.

Заметьте, что нет утвержденного стандартного формата SDD для API на основе гипермедиа. Пока я рекомендую включать документ ALPS в `profile API` (подробнее — в рецепте 5.7).

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 4.6. Управление форматами представления в среде выполнения.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.10. Публикация метаданных API.

5.10. Публикация метаданных API

По мере роста числа прикладных интерфейсов в открытой веб-среде возникает потребность в общем формате и словаре для описания метаданных API. Это особенно важно, когда поставщики сервисов хотят передать значимую информацию об API, не прибегая к личным встречам, взаимодействиям или частному описанию и документированию. К счастью, на передачу этой информации есть уверенный претендент — спецификация APIs.json (<https://oreil.ly/Imjf1>).

Задача

Как представить важные метаданные об интерфейсе сервиса (поддерживаемые форматы, словари, документацию, детали безопасности и т. д.) стандартизированным образом, который позволит потребителям API легко найти и понять метаязыковые детали этого интерфейса?

Решение

Очень надежный способ каталогизации и публикации метаданных API — использование открытой спецификации APIs.json (<https://oreil.ly/gQtXo>). Она содержит словарь и структурную схему, которая охватывает большинство важных метаданных для API — базовые определения, свойства API, значимые URL, ключевые контакты, мейнтейнеров и пр. При этом она является расширяемой, позволяя поставщикам API дополнять документ APIs.json (с учетом обратной совместимости).

APIS.JSON, APIS.YAML И APIS.TXT

Документы APIs.json могут быть представлены в виде файлов JSON, YAML или TXT. Узнать больше об этой спецификации можно на ее сайте (<http://apisjson.org/>). Кроме того, на <http://apis.io> есть публично доступная версия поискового движка для документов APIs.json.

Спецификация APIs.json определяет несколько ключевых разделов документа.

Основа (Root)

Имя, описание, URL и прочая базовая информация об API.

API

Список указанных в документе API.

Инструменты (Tools)

Список доступных в ходе работы с API инструментов с открытым исходным кодом.

Спецификации (Specifications)

Список открытых спецификаций, применяемых в работе с API.

Ресурсы (Resources)

Список ресурсов, доступных для операций API.

Стандартные свойства (Common)

Список стандартных свойств для использования во всех API и инструментах.

Дополнительные документы (Include)

Другие документы APIs.json, включаемые в текущий.

Мейнтейнеры (Maintainers)

Группа людей и/или организаций, обслуживающих этот API.

Поставщики могут документировать метаданные API в одном документе с названием `apis.json`, доступном в корневом каталоге сервиса (например, <http://api.example.org/apis.json>). API также способны предоставлять значение ссылочной связи, которое можно включить в HTTP-заголовок и/или в тело HTTP-ответа. При указании на документ APIs.json для API рекомендуется использовать значение связи `service-meta`, определенное в RFC 8631 (<https://oreil.ly/ymnrm>).

Будет хорошей идеей реализовать поддержку ссылки на ресурс Home интерфейса вашего сервиса, указывающего на ресурс `service-meta` (рис. 5.5).

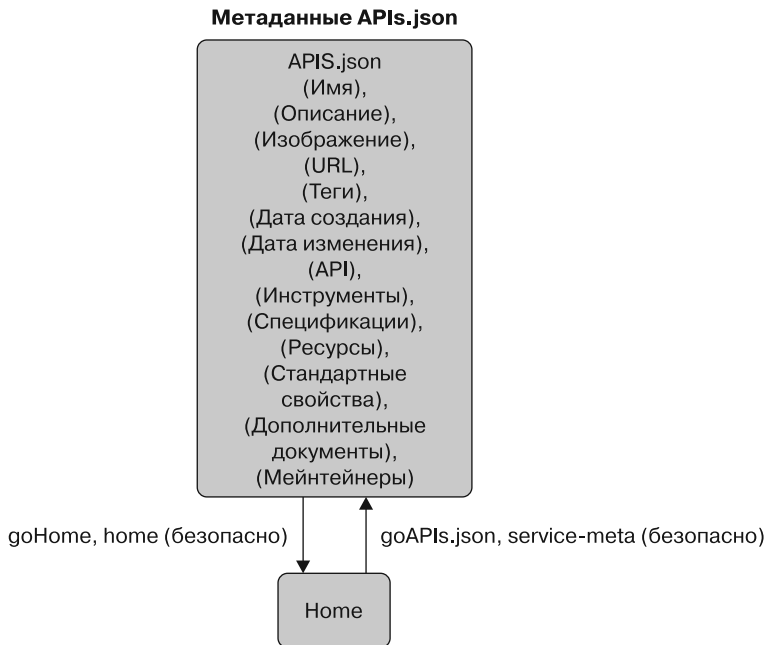


Рис. 5.5. Метаданные API



Подробнее о ресурсе `service-meta` для `APIs.json` можно почитать в онлайн-коллекции связанных с этой книгой документов ALPS (<https://oreil.ly/RKhJ8>).

Пример

Вот небольшой пример документа `APIs.json`:

HTTP/1.1 200 OK

Content-Type: application/apis+json

...

```
{
  "name": "Example API",
  "type": "Index",
  "description": "This is an example APIs.json file.",
  "image": "https://api.example.org/logo.jpg",
  "tags": ["Application Programming Interface", "API"],
  "created": "2014-04-07",
  "modified": "2020-09-03",
  "url": "http://example.com/apis.json",
  "specificationVersion": "0.14",
  "apis": [
    {
      "name": "Example API",
      "description": "This provides details about a specific API.",
      "humanURL": "http://example.com",
      "baseURL": "http://api.example.com",
      "tags": ["API", "Application Programming Interface"],
      "properties": [
        { "type": "Documentation", "url": "https://example.com/documentation" },
        { "type": "OpenAPI", "url": "http://example.com/openapi.json" }
      ],
      "contact": [{"FN": "APIs.json", "email": "info@apisjson.org"}]
    }
  ],
  "specifications": [
    { "name": "OpenAPI", "url": "https://openapis.org" },
    { "name": "JSON Schema", "url": "https://json-schema.org/" }
  ],
  "common": [
    { "type": "Signup", "url": "https://example.com/signup" },
    { "type": "Authentication", "url": "http://example.com/authentication" },
    { "type": "Login", "url": "https://example.com/login" }
  ],
  "maintainers": [{"FN": "Mark Morkelson", "email": "mork@example.org"}]
}
```

Сервисы могут объявлять о доступности документа `APIs.json` в HTTP-ответах:

HTTP/1.1 200 OK

Content-Type: HTML

Link: <<http://api.example.org/apis.json>>; rel=service-meta

...

```
<html>
  <head>
    <link rel="service-meta" href="http://api.example.org/apis.json" />
  </head>
  <body>
    ...
  </body>
</html>
```

Клиентские приложения API также могут автоматически искать документы APIs.json, обращаясь к предустановленному расположению файла, указанному в спецификации:

```
GET /apis.json HTTP/1.1
Accept: application/apis+json, application/yaml, text/plain
Host: http://api.example.org
```

Обсуждение

Важно отметить, что спецификации APIs.json должны передавать информацию об интерфейсе сервиса, но не *описывать* или *определять* этот интерфейс подробно. Для этого были созданы другие форматы, такие как OpenAPI, AsyncAPI, Protobuf. Подробнее о форматах определения сервисов и способах сообщения о них потребителям API читайте в рецепте 5.9.



На момент написания книги спецификация APIs.json еще не завершена. Тем не менее я нахожу этот формат (и связанный с ним поисковый движок) стабильным и полезным, поэтому рекомендую читателям иметь эту спецификацию в виду.

Одной из причин применения формата APIs.json для метаданных сервиса является наличие связанного с ним проекта с открытым исходным кодом, предоставляющего поисковый движок, использующий этот формат в качестве входного. Текущую редакцию поискового движка APIs.json можно найти онлайн (<https://oreil.ly/SYgMp>). На момент написания книги есть возможность сделать форк или скачать этот проект и начать создание собственного механизма поиска API для размещения документов APIs.json.

Дополнительной особенностью поискового движка APIS.json является поддержка собственного API, предназначенного для автоматизации загрузки и проверки документов APIs.json в рамках этого механизма. Это позволяет создавать автоматизированный процесс для регистрации интерфейсов с помощью целевых движков поиска API.

Спецификация APIs.json содержит более 50 зарезервированных слов. Большинство из них известны как элементы свойств и используются для значений `type` в записях раздела `common` документа APIs.json. Вот лишь несколько примеров:

- Signup;
- Login;

- TermsOfService;
- InterfaceLicense;
- PrivacyPolicy;
- Security;
- StatusPage;
- Pricing;
- Rate Limits.

Вы можете дополнять спецификацию, добавляя в этот список собственные имена.

Еще одной спецификацией, которую можно использовать для передачи метаданных API, является JSON Home (<https://oreil.ly/fzSoX>). Однако инициатива по ее созданию уже несколько лет стоит на месте, и я не видел, чтобы она применялась так же часто, как рассмотренная здесь спецификация APIS.json.

Читайте также

- Рецепт 3.3. Предоставление информации о предметной области через публикуемые словари.
- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.9. Публикация документов определений сервисов.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.

5.11. Поддержка мониторинга состояния сервисов

Одно из преимуществ использования микросервисов в веб-среде заключается в том, что вам не нужно программировать всю функциональность самим. В этом вопросе можно положиться на результаты творческих усилий других. Тем не менее добавление зависимых сервисов повышает вероятность сбоя в распределенной сети. Хорошим способом отслеживания статуса зависимых сервисов будет регулярная проверка состояния для подтверждения их правильного функционирования.

Задача

Как согласованно и регулярно мониторить состояние сервиса в веб-среде для подтверждения его должного функционирования? Как правильно мониторить значения для анализа состояния сервиса? Какой имеется стандартный формат

для передачи этих значений? Как заявить о поддержке мониторинга состояния и делать доступными собранные о его состоянии данные извне?

Решение

Мониторинг состояния сервисов в веб-среде традиционно сопряжен со сложностями. Радует то, что за последние годы была проделана значительная работа по стандартизации этого процесса. Существует устаревший документ *Health Check Response Format for HTTP APIs* (<https://oreil.ly/QPgD7>). Эта спецификация была опубликована в 2018 году, и я довольно долго использовал ее в качестве руководства для реализации проверок состояния.

В ней определено несколько свойств, включая следующие основные.

`status`

Указывает состояния сервиса (`pass`, `fail`, `warn`).

`version`

Публичная версия сервиса.

`releaseId`

Номер версии/релиза сервиса.

`notes`

Массив заметок, относящихся к текущему состоянию.

`output`

Необработанный вывод ошибок для состояний `fail` или `warn`.

`checks`

Объект, предоставляющий подробную картину состояния дополнительных нижерасположенных систем.

`links`

Объект, содержащий ссылочные связи и URI для внешних ссылок, которые *могут* нести дополнительную информацию о состоянии сервиса.

`serviceId`

Уникальный идентификатор сервиса.

`description`

Понятное человеку описание сервиса.



Объект `checks` представляет коллекцию связанных нижерасположенных сервисов и в некотором смысле обеспечивает превентивный анализ их состояния. Этот объект имеет еще несколько свойств. Подробнее о нем можно узнать в спецификации.

Вот ряд дополнительных понятий, которые нужно иметь в виду при реализации спецификации Health Check.

Ресурс состояния (Health resource)

Сервисы должны предоставлять конечную точку, которую можно использовать для запроса документа о состоянии. Спецификация предлагает (но не требует) сервисам применять для этого URL `/health`.

Значение ссылочной связи состояния (Health link relation value)

И хотя в спецификации это не указано, для определения URL, который будет возвращать информацию о состоянии, я также использую значение ссылочной связи `health-check`.

Медиатип для выражения состояния (Health media type)

Возвращая информацию о состоянии, сервисам следует задействовать идентификатор медиатипа `application/health+json` и возвращать валидный документ о состоянии (на основе спецификации).

Кэширование (Caching)

Сервисы должны включать директиву HTTP-кэширования с ответом (`max-age`, `ETag` и т. д.) для снижения влияния частых запросов и исключения вектора атаки «отказ в обслуживании». Естественно, клиенты тоже должны учитывать директивы кэширования.

Подробнее — в примерах.

Будет хорошей идеей реализовать поддержку ссылки на ресурс Номе интерфейса сервиса, который будет указывать на ресурс `health` (рис. 5.6).



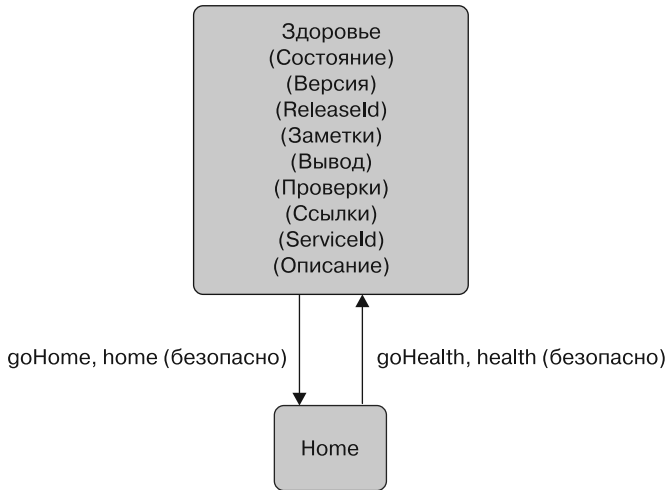
Подробнее о ресурсе `health` читайте в онлайн-коллекции связанных с книгой документов ALPS (<https://oreil.ly/RKhJ8>).

Пример

Типичный ответ `application/health+json` выглядит так:

```
HTTP/1.1 200 OK
Content-Type: application/health+json
Cache-Control: max-age=3600
ETag: "w\i8u7y6t5r4e3w2"
...
```

```
{
  "status": "pass",
  "version": "1",
  "releaseId": "1.2.2",
```

Формат ответа для проверки состояния HTTP API**Рис. 5.6.** Рецепт для мониторинга состояния сервиса

```

"notes": [""],
"output": "",
"serviceId": "f03e522f-1f44-4062-9b55-9587f91c9c41",
"description": "health of authz service",
"checks": {
  "cassandra:responseTime": [
    {
      "componentId": "dfd6cf2b-1b6e-4412-a0b8-f6f7797a60d2",
      "componentType": "datastore",
      "observedValue": 250,
      "observedUnit": "ms",
      "status": "pass",
      "affectedEndpoints" : [
        "/users/{userId}",
        "/customers/{customerId}/status",
        "/shopping/{anything}"
      ],
      "time": "2018-01-17T03:36:48Z",
      "output": ""
    }
  ],
  ...
}

```

Обратите внимание на использование заголовков `cache-control` и `Etag`. С их помощью мы инструктируем клиентские приложения сохранять локальные копии документов для уменьшения трафика сервиса.

Информация о проверке состояния в ответах OPTIONS

Вы можете заявить о поддержке проверок состояния, включая в ответы ссылочное значение `link`. Я делаю так для всех ответов `OPTIONS`:

```
**** REQUEST ****
OPTIONS / HTTP/1.1
Accept: application/vnd.collection+json, application/html, application/json
...
```

```
**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Link: <http://api.example.org/health>; rel="health-check"
...
```

Информация о проверке состояния в ответах service-meta

Я также добавляю поддержку проверок состояния в документ метаданных сервиса (см. рецепт 5.10):

```
HTTP/1.1 200 OK
Content-Type: application/apis+json
...
```

```
{
  "name": "Example API",
  "type": "Index",
  "description": "This is an example APIs.json file.",
  "image": "https://api.example.org/logo.jpg",
  "tags": ["Application Programming Interface", "API"],
  "created": "2014-04-07",
  "modified": "2020-09-03",
  "url": "http://example.com/apis.json",
  "specificationVersion": "0.14",
  "apis": [
    {
      "name": "Example API",
      "description": "This provides details about a specific API.",
      "humanURL": "http://example.com",
      "baseURL": "http://api.example.com",
      "tags": ["API", "Application Programming Interface"],
      "properties": [
        {"type": "Documentation", "url": "https://example.com/documentation"},
        {"type": "OpenAPI", "url": "http://example.com/openapi.json"},
        {"type": "Health", "url": "http://example.com/health.json"}
      ],
      "contact": [{"FN": "APIs.json", "email": "info@apisjson.org"}]
    }
  ],
  ...
}
```

Заметьте, что тип "Health" в APIs.json — это дополнительное значение, не включенное в спецификацию APIs.json. Подробнее читайте в рецепте 5.10.

Обсуждение

В большинстве случаев проверки состояния помогают, если освещают основные параметры. Для потребителей сервиса обычно вполне достаточно предоставления `pass`, `fail` и `warn`. Но использовать эти проверки для диагностики или отладки нежелательно, поскольку они отражают состояние *интерфейса*, а не скрытого за ним сервиса.



Данный рецепт основан на версии draft-06 документа *Health Check Response Format for HTTP APIs*. К моменту прочтения вами этой книги могут быть опубликованы новые фрагменты итогового утвержденного RFC. Рекомендуем следить за этой спецификацией, чтобы быть в курсе ее изменений.

Некоторые читатели могут пожелать настроить конечную точку проверки состояния под именем `callback`, которая позволит другим подписываться на регулярные ответы сервиса. Так делать не стоит. По мере роста числа пользователей вашего сервиса может возникнуть необходимость обработки сотен или даже тысяч запросов обратных вызовов. Это запросто перегрузит сервис, так что лучше задействовать конечную HTTP-точку `GET` со значением `cache-control`, которая будет уменьшать влияние большого числа запросов состояния. Получится весьма иронично, если из-за проверок состояния сервис не сможет выполнять соглашения об уровне оказания услуг.

В спецификации указано, что ответы на проверки состояния являются динамическими, то есть их содержимое может настраиваться в соответствии с контекстом вызова. Например, анонимные запросы могут возвращать только значение `status` (`pass`, `fail`, `warn`), а авторизованные (например, от администраторов) — получать подробную информацию о состоянии сервиса.

В спецификации не приводится особых деталей о расширении ответов `application/health+json`. Если вы хотите включить дополнительную информацию, не отражаемую штатными свойствами, делайте это с учетом обратной совместимости и предоставляйте ссылки на понятную человеку документацию вашего расширения.

Читайте также

- Рецепт 3.8. Проектирование с учетом возможности повторения действий.
- Рецепт 3.9. Проектирование с учетом возможной отмены действий.
- Рецепт 5.12. Стандартизация отчетов об ошибках.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.
- Рецепт 7.17. Поддержка локальной отмены или отката.

5.12. Стандартизация отчетов об ошибках

Все интерфейсы сервисов сталкиваются с ошибками, и загвоздка в том, чтобы их правильно обработать. На первом этапе нужно наладить согласованный и удобный механизм предоставления приложениям-потребителям API *отчета* об изначальной ошибке. Этой задаче посвящено рабочее предложение RFC 7807.

Задача

Как лучше всего веб-сервисам сообщать об ошибках среды выполнения? Какую информацию следует возвращать вызывающему в случае ошибки? Каков лучший формат для этой информации? Что могут сделать сервисы, чтобы отчет не привел к неожиданному зависанию или падению столкнувшегося с ошибкой клиентского приложения?

Решение

Информирование об ошибках — важнейшая часть реализации стабильных и удобных в использовании интерфейсов сервисов. Этот процесс нужно по возможности обрабатывать так, чтобы и потребитель, и поставщик API могли исправлять любые ошибки, продолжая функционировать должным образом. Ключом здесь является передача информации об ошибках стандартизированным способом, то есть так, чтобы потребители могли понять, что речь идет об ошибке, и предпринять необходимые для ее исправления действия.

Очень удобно сообщать потребителям API об ошибках с помощью медиатипа Problem Details, определенного в RFC 7807 (<https://oreil.ly/4isUg>). Его применение повышает вероятность того, что клиенты распознают ошибку и окажутся лучше подготовленными к разрешению описанной в сообщении проблемы (подробнее — в примере далее).

ВСТРОЕННЫЙ МЕХАНИЗМ ИНФОРМИРОВАНИЯ

В некоторых случаях в форматы сообщений заложен механизм информирования об ошибках. Например, формат CollectionJSON содержит объект `error`. Когда этот формат позволяет информировать об ошибке прямо в сообщении, лучше всего так и делать, не прибегая к специальному сообщению RFC 7807.

Ключом к успеху здесь будет рассмотрение ошибок как альтернативного ответа, а не провалившегося. Иными словами, всегда включайте отчет об ошибках в качестве *функциональности* интерфейса, а не в качестве состояния сбоя.

Пример

Спецификация сообщения Problem Details (RFC 7807) определяет небольшой набор элементов.

type

Ссылочный URI (RFC 3986), описывающий тип проблемы.

title

Короткое описание типа проблемы.

status

Сгенерированный сервисом HTTP-код состояния (выражается числом, а не строкой).

detail

Подробное объяснение конкретной проблемы.

instance

Ссылочный URI, указывающий на конкретную проблему (не обязателен).

В спецификации определено предустановленное значение `type`: `"about:blank"`. Если в свойстве `type` есть URI, спецификация требует, чтобы он указывал на понятное для человека сообщение с описанием типа проблемы.

Вот стандартное сообщение о деталях проблемы:

HTTP/1.1 403 Forbidden

Content-Type: application/problem+json

Content-Language: en

```
{
  "type": "https://example.com/probs/out-of-credit",
  "title": "You do not have enough credit.",
  "detail": "Your current balance is 30, but that costs 50.",
  "instance": "/account/12345/messages/abc",
  "status": 403
}
```

Можно расширить это описание собственными свойствами:

HTTP/1.1 403 Forbidden

Content-Type: application/problem+json

Content-Language: en

```
{
  "type": "https://example.com/probs/out-of-credit",
  "title": "You do not have enough credit.",
  "detail": "Your current balance is 30, but that costs 50.",
```

```

"instance": "/account/12345/msgs/abc",
"status": 403,
"balance": 30,
"accounts": ["/account/12345", "/account/67890"]
}

```

Заметьте, что URL в элементе **type** должен указывать на документ, определяющий суть проблемы **out-of-credit**. Эта документация должна включать также определения элементов **balance** и **accounts**. Таким образом свойство **type** указывает на семантический профиль представления этой проблемы. Подробнее о семантических профилях читайте в рецепте 5.7.

Свойства **type**, **title** и **status** сообщения о деталях проблемы являются фиксированными — всегда остаются одинаковыми вне зависимости от того, какой API или сервис возвращает это сообщение. А вот свойства **detail** и **instance** специфичны для текущего случая. Вот пример:

```

HTTP/1.1 403 Forbidden
Content-Type: application/problem+json
Content-Language: en

{
  "type": "https://example.com/probs/out-of-credit",
  "title": "You do not have enough credit.",
  "detail": "Your current balance is 20, but that costs 50.",
  "instance": "/account/12345/msgs/q1w2e3",
  "status": 403,
  "balance": 20,
  "accounts": ["/account/r4er3w2", "/account/y6t5r4"]
}

```

Теперь сравните этот описывающий проблему ответ с предыдущим. В обоих случаях значения **type**, **title** и **status** одинаковы, а **details**, **instance**, **balance** и **accounts** различаются.

В спецификации RFC 7807 зарегистрированы оба идентификатора медиатипов: **application/problem+json** и **application/problem+xml**. Но в ней не приводятся примеры XML-версии сообщения о проблеме. Ответ вот:

```

HTTP/1.1 403 Forbidden
Content-Type: application/problem+xml
Content-Language: en

<problem-details>
  <type>https://example.com/probs/out-of-credit</type>
  <title>You do not have enough credit.</title>
  <detail>Your current balance is 30, but that costs 50.</detail>
  <instance>/account/12345/msgs/abc</instance>
  <status>403</status>
</problem-details>

```




На момент написания книги Инженерный совет Интернета (IETF) работает над обновлением документа RFC 7807. И хотя доработка не должна стать критической, читателям следует внимательно следить за документом и быть готовыми к любым его изменениям.

Обсуждение

В спецификации говорится о том, что сообщения с описанием проблемы не нужны, когда достаточно простого HTTP-отчета о состоянии вроде 4xx или 5xx. Например, если пользователь попытается обновить существующую запись, не имея на то соответствующих прав, будет достаточно вернуть на запрос PUT ответ 403. А вот если он отправит в сообщении PUT недопустимое тело, стоит вернуть ответ с описанием проблемы и предложением способа ее исправления.

Сервисы не должны применять этот формат для возврата клиентскому приложению отладочной информации. Он предназначен для сообщения деталей об *интерфейсе*, а не о скрытом за API сервисе.

При создании нового `type`, описывающего детали проблемы, следует документировать три элемента:

- URI этого `type` (например, `http://api.example.org/problems/insufficient-funds`);
- `title` (например, «На вашем счете недостаточно средств для совершения этой транзакции»);
- соответствующий проблеме код `status` (например, 403).

Желательно ограничить количество новых типов описания проблемы и создавать такие, которые будут достаточно обобщенными для повторного применения. Также можно настроить представления проблемы под свои потребности, используя элементы сообщения `detail` (текст) и `instance` (URL).

Кроме того, можно включать в ответ заголовок `Retry-After`, указывая потребителям API на возможность получения подробностей относительно повтора запроса.

Читайте также

- Рецепт 3.8. Проектирование с учетом возможности повторения действий.
- Рецепт 3.9. Проектирование с учетом возможной отмены действий.
- Рецепт 5.11. Поддержка мониторинга состояния сервисов.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.
- Рецепт 7.17. Поддержка локальной отмены или отката.

5.13. Облегчение поиска сервисов с помощью реестра сервисов среды выполнения

Ключевая цель всех интерфейсов API — возможность легко обнаруживать нужные вам сервисы и последовательно их использовать, в том числе повторно, в широком спектре продуктов и приложений. Данный рецепт посвящен облегчению *нахождения и повторного применения* вашего сервиса в среде выполнения (не во время проектирования и разработки) за счет его регистрации в общем реестре сервисов среды выполнения (Runtime Service Registry, RSR).

Задача

Что должны делать поставщики сервисов, чтобы их продукты можно было легко и последовательно регистрировать в целевом RSR? Какие внутренние функции должен поддерживать каждый интерфейс, чтобы сервис было легко найти? Как сервисы могут сообщать реестрам показатели своего состояния в среде выполнения? Какие метаданные должен предоставлять сервис, чтобы машинам и людям было проще найти его?

Решение

Применение RSR способно облегчить обнаружение и повторное использование работающих сервисов веб-среды. Когда сервис развертывается в сети и запускается, он должен публиковать себя в одном из RSR. По сути, таким образом он сообщает реестру о своем расположении, предоставляемой функциональности и прочую сопутствующую информацию, такую как медиатипы (см. рецепт 3.1), общие словари (см. рецепт 3.3), определения для взаимодействия (см. рецепт 5.9) и сведения о состоянии (см. рецепт 5.11). RSR можно рассматривать как своеобразную систему доменных имен (Domain Name System, DNS) для сервисов — надежное место, где можно находить сервисы и подключаться к ним.



Текущий рецепт посвящен поддержке машинного обнаружения и использования API. Есть также варианты реализации поддержки управляемого человеком обнаружения API на этапах проектирования или создания, такие как порталы API, их каталоги и пр. Данный рецепт (совместно с рецептом 5.10) можно применить для реализации поддержки в среде проектирования и создания API.

Для того чтобы сервис можно было легко найти и задействовать в веб-среде, в том числе повторно, он должен поддерживать следующее.

Регистрация при первом запуске

Сервису необходимо при развертывании автоматически регистрироваться в одном или нескольких RSR. Это обеспечит возможность его обнаружения другими сервисами.

Периодические отчеты о состоянии

Сервис должен регулярно (иногда интервал задается RSR) отмечаться в реестре с помощью предоставленного при регистрации URL, подтверждая свою работоспособность. При желании он может также передавать статистику своей работы, например, сколько раз он был вызван за последний отчетный период, количество обнаруженных ошибок (400/500) и время отклика.

Отмена регистрации при отключении

При отключении — преднамеренном или вследствие критической ошибки — сервис должен автоматически отменять регистрацию во всех RSR. Таким образом исключается сценарий, когда другие сервисы могут попытаться к нему подключиться в момент недоступности.



Заметьте, что все три описанных действия внутренние для сервиса, поэтому нет необходимости выражать эту функциональность в рамках интерфейса API.

Пример

Создание сервиса с поддержкой RSR означает закладывание в него функции автоматической регистрации при запуске, отправку им сигналов о своем состоянии и отмену регистрации при отключении. Вот простой пример регистрации при запуске в предварительно указанном RSR с помощью пакета `discovery`:

```
var srsResponse = null;
var srsRegister({url:"...", "name":"...", ....});

// Регистрация сервиса с предустановленными параметрами
discovery.register(srsRegister, function(data, response) {
  srsResponse = JSON.parse(data);
  initiateKeepAlive(srsResponse.href, srsResponse.milliseconds);
  http.createServer(uuidGenerator).listen(port);
  console.info('uuid-generator running on port '+port+'.');
});
```

При регистрации в реестр необходимо передать следующие метаданные сервиса.

serviceURL

URL регистрируемого сервиса.

serviceName

Текстовое имя регистрируемого сервиса.

semanticProfile

Разделенный пробелами список URI профилей (см. рецепт 3.4).

mediaType

Разделенный пробелами список идентификаторов `mediaType` (см. рецепт 3.1).

apiDefinitions

Коллекция аннотированных ссылок, указывающих на документы с определениями API (см. рецепт 5.9).

tags

Разделенный пробелами список доступных для поиска ключевых слов (например, "gdpr banking loans").

Находясь в рабочем состоянии, сервис должен периодически отправлять в реестр сигналы, подтверждая свою работоспособность. Приведенный далее пример показывает, как это реализовать с помощью метода `initiate KeepAlive(...)`. Данный метод также может отправлять дополнительную информацию (или `href`, указывающую на эти данные, подробнее — в рецепте 5.11).

Когда сервис отключается или по плану, или из-за критической ошибки, его необходимо исключить из реестра, чтобы другие сервисы не пытались использовать отсутствующий экземпляр (или кластер). Вот пример на NodeJS, демонстрирующий отмену регистрации сервиса при отключении:

```
// Настройка отключения возможности обнаружения
process.on('SIGTERM', function () {
  discovery.unregister(null, function(response) {
    try {
      uuidGenerator.close(function() {
        console.log('gracefully shutting down');
        process.exit(0);
      });
    } catch(e){}
  });
  setTimeout(function() {
    console.error('forcefully shutting down');
    process.exit(1);
  }, 10000);
});
```

Обратите внимание на дополнительный метод `setTimeout`, позволяющий при необходимости отключить сервис принудительно.

Поскольку критический сбой может предшествовать сообщению `discovery.unregister`, в RSR обычно установлена максимальная длительность ожидания сигнала с подтверждением работоспособности. Если ожидание превысит это время, RSR наверняка удалит запись о регистрации сервиса или установит для него состояние «отключен» либо иное значение.

Обсуждение

Каждый работающий экземпляр сервиса или кластер должен быть представлен в RSR. Например, если один экземпляр сервиса `netPresentValueCalculator` работает на машине в Северной Америке, а другой — на машине в Северной Африке,

они будут представлены в реестре в виде двух записей. И наоборот, один и тот же экземпляр сервиса может быть представлен в нескольких RSR. Например, он может быть зарегистрирован в реестре, находящемся в Северной Америке, и в другом реестре — в Австралии.

Помните, что один сервис (например, `computeInterest`) может иметь несколько *версий*. Если они выполняются как отдельные экземпляры, в RSR должна быть представлена каждая. Это позволит потребителям API найти нужную им конкретную версию.



Для регистрации вашего сервиса в RSR наверняка потребуются наличие аутентифицированного пользователя. Это особенно актуально для RSR, доступных в веб-среде.

Кроме того, сервисы могут задействовать RSR для нахождения необходимых внешних сервисов и подключения к ним. Например, если ваш сервис `computeInterest` опирается также на внешние сервисы `taxTable` и `federalRates`, он может использовать RSR, в частности, для поиска и привязки предпочтительной версии каждого из них.

При регистрации сервиса желательно предоставлять как можно больше метаданных, чтобы максимально облегчить его обнаружение. Обязательно добавьте ключевые слова в поле `tags`, а также укажите `serviceProfile`, `apiDefinitions` и поддерживаемые `mediaTypes`.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.9. Публикация документов определений сервисов.
- Рецепт 7.6. Поддержка RESTful-языка управления задачами.

5.14. Повышение скорости обработки с помощью клиентских идентификаторов

Вполне типично ожидать от серверов генерации и возврата идентификаторов для всех ресурсов, создаваемых клиентскими приложениями. Это упрощает процесс получения уникальных URL ресурса, но имеет свои недостатки, особенно когда задействованы несколько процессов (создать A, получить уникальный ID от A, использовать его для создания B и т. д.).

Задача

Когда стоит позволить клиентам предоставлять уникальные идентификаторы и как сделать это безопасно и согласованно? Как проще и надежнее всего реализовать поддержку ID ресурсов, предоставляемых потребителями?

Решение

Простым и надежным способом генерации уникальных идентификаторов будет использование метки с информацией о дате и времени или вывода генератора случайных чисел. Вот пример с применением генератора случайных чисел в JavaScript:

```
var id = parseInt(String(Math.random()).substring(2)).toString(36);  
// id = "906Lem09xu8"
```

Дату и время и/или случайные числа можно использовать также для генерации всемирно уникального идентификатора (Universally Unique Identifier, UUID) (<https://oreil.ly/oHXrO>). Подробнее — в примерах. Существует спецификация для создания UUID (RFC 4122), которая может стать руководством для написания простой процедуры. Более того, есть немало открытых, совместимых с этой спецификацией генераторов UUID практически для всех языков программирования. Некоторые платформы программирования даже предлагают прямой вызов для создания UUID (например, NodeJS v17 поддерживает функцию UUID в модуле `cryptoAPI`).



Предоставляемые клиентом идентификаторы позволяют сократить последовательную обработку многоэтапных рабочих потоков и заменить их реализациями с параллельной обработкой (см. пример).

В клиентские приложения должны быть заложены генерация собственного уникального идентификатора и либо включение его в URL, либо передача в теле HTTP-запроса POST, PUT или PATCH, отправляемого к API сервиса. Перед обработкой запроса API может просмотреть это значение, чтобы подтвердить его уникальность. Если оно окажется неуникальным, сервис может вернуть код состояния 409 Conflict с дополнительной информацией, поясняющей клиенту, как эту проблему исправить.

Пример

Очень простым способом генерации уникального идентификатора будет использование встроенного в язык генератора случайных чисел. Вот пример такой реализации на JavaScript:

```
// Генерация уникальных идентификаторов  
function makeId() {  
  var rtn;
```

```

    rtn = String(Math.random());
    rtn = rtn.substring(2);
    rtn = parseInt(rtn).toString(36);

    return rtn;
}

```

Вызов функции `makeId()` сгенерирует компактные строки идентификаторов, похожие на "1oyte4x0zep".

Некоторые генераторы создают не настолько случайные числа, насколько большинству из нас хотелось бы. Поэтому можно комбинировать значение метки даты и времени с полученным случайным числом и генерировать результат, соответствующий RFC 4122. Вот пример функции, которая это реализует:

```

// Генерация UUID (открытый доступ/MIT)
function generateUUID() {
    var d = new Date().getTime();
    var d2 = ((typeof performance !== 'undefined') &&
        performance.now() && (performance.now()*1000)) || 0;
    return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, function(c) {
        var r = Math.random() * 16;
        if(d > 0){
            r = (d + r)%16 | 0;
            d = Math.floor(d/16);
        } else {
            r = (d2 + r)%16 | 0;
            d2 = Math.floor(d2/16);
        }
        return (c === 'x' ? r : (r & 0x3 | 0x8)).toString(16);
    });
}

```

Вывод функций генерации UUID с гораздо меньшей вероятностью приведет к коллизии, но полученный результат, например "e6db8698-7128-478d-8658-12c2cb9dd126", окажется длиннее того, который был получен в простом примере ранее.

Эти уникальные значения можно получать также от клиентского приложения в теле сообщения HTTP-операций POST:

```

<p>Create a New Person</p>
<form name="create" action="/persons/" method="post">
    <input type="hidden" name="unique-id" value="1oyte4x0zep" />
    <input type="text" name="name" value="Mark Morkleson" />
    <input type="submit" />
</submit>

```

Или внутри URL при выполнении HTTP-операций PUT:

```

<p>Create a New Person</p>
<form name="create" action="/persons/1oyte4x0zep" method="put">
    <input type="text" name="name" value="Mork Markleson" />
    <input type="submit" />
</submit>

```



Подробнее об использовании метода PUT для создания ресурсов читайте в рецепте 5.15.

Подтверждение приемлемости идентификатора — обязанность сервера. Например, он проверяет, есть ли уже такой идентификатор (и связанный с ним ресурс), правильно ли он отформатирован, например имеет ли нужную длину строки с необходимыми символами, и пр. Если по какой-то причине переданный клиентом идентификатор оказывается неприемлемым, интерфейс сервиса должен отклонить операцию записи, ответив HTTP-кодом 409 Conflict.

Обсуждение

Некоторые разработчики клиентских приложений противятся идее предоставления собственных уникальных значений. Вы можете успокоить их, предоставив библиотеку кодов, генерирующую уникальные идентификаторы, и посоветовав использовать ее так же, как они использовали бы SDK (software development kit — пакет программ для разработки приложений).

В качестве подстраховки можно реализовать в интерфейсе сервиса резервный механизм генерации идентификатора на случай, если он не будет предоставлен клиентом. Для этого можно выполнить простую проверку с помощью if:

```
function addItem(id, body) {
    var item;

    if (id) {
        item.id = id;
    } else {
        item.id = makeId();
    }
    ...
}
```

Это будет особенно полезно, если API — лишь тонкий посредник для сервиса бэкенда.

Одним из преимуществ использования предоставляемых клиентом идентификаторов является то, что это помогает сократить потребность в последовательной обработке рабочего потока. Рассмотрим в качестве примера случай, когда нужно обновить три ресурса: `customer`, `account` и `salesRecord`. Предположим также, что запись `account` должна иметь в качестве свойства идентификатор `customer`, а `salesRecord` — идентификаторы `customer` и `account`. В этом сценарии, если идентификаторы предоставляют сервисы, ваш рабочий поток может выглядеть так:

```
writeCustomer()
    .then(function(customerResponse) {
        return writeAccount(accountResponse);
    }).then(function(nextResponse) {
        return writeSalesRecord(salesRecordResponse);
    });
```



```
}).then(function(finalResponse) {  
    console.log('Final response: ' + finalResponse);  
}).catch(failureCallback);
```

По сути, здесь каждый шаг для начала выполнения должен ожидать завершения предыдущего. Если же вы используете клиентские идентификаторы, то можете проделать следующее:

```
var cId = makeId();  
var aId = makeId();  
var sId = makeId();  
Promise.all([  
    writeCustomer(cId),  
    writeAccount(cId,aId),  
    writeSalesRecord(cId,aId,sId)  
])  
.then(() => console.log('All done!'))  
.catch(function(err) {  
    rollbackAll(cId,aId,sId);  
    console.log('Write failed!');  
});
```

Побочным эффектом использования клиентских идентификаторов является то, что идентификаторы ресурсов оказываются непоследовательными и человек редко может легко их прочесть или запомнить. Это отсутствие последовательности может усложнить для хакеров подбор идентификаторов ваших ресурсов, а для законопослушных разработчиков — навигацию по коллекции записей. Исправить вторую проблему можно, позволив клиентам передавать альтернативные ID, которые можно выводить в списках и иных формах, в то время как сгенерированный идентификатор будет скрыт в пользовательском интерфейсе:

```
<ul>  
  <li id="q1w2e3r4">Sally-R</li>  
  <li id="p0o9i8u7">Jane-Q</li>  
  <li id="6y5t4r3e">Barb-Z</li>  
</ul>
```

Читайте также

- Рецепт 3.6. Проектирование последовательных операций записи с помощью идемпотентных действий.
- Рецепт 3.8. Проектирование с учетом возможности повторения действий.
- Рецепт 3.9. Проектирование с учетом возможной отмены действий.
- Рецепт 5.15. Повышение надежности с помощью идемпотентной операции создания.
- Рецепт 6.2. Делаем все изменения идемпотентными.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.

5.15. Повышение надежности с помощью идемпотентной операции создания

Несмотря на то что стандартной практикой создания ресурсов в веб-API является использование HTTP-метода `POST`, это не самый надежный способ. В действительности, когда дело касается М2М-взаимодействий (например, полностью без участия человека), `POST` может создать проблемы, связанные с возможной *потерей ответов*. Здесь нам нужен такой подход к созданию ресурсов, который независимо от возможных сбоев в сети будет надежным и повторяемым.

Задача

При создании ресурсов с помощью HTTP-метода `POST` есть вероятность столкнуться с проблемой утеранных ответов. Например, клиент API отправляет `POST`-запрос для перевода 500 долларов со счета А на счет В, но ответа так и не получает. Ни `200 OK`, ни `400 Bad Request`, ни `500 Server Error` — ничего.

Что клиенту в этой ситуации делать? Достиг ли запрос сервера? Возникла ли на стороне сервера ошибка, которая запрос отклонила? Что если запрос достиг сервера и был выполнен, но ответ `200 OK` просто потерялся в сети? В последнем случае повтор запроса может привести к дублированию перевода средств.

Как избежать негативных последствий утраты ответов при записи данных в сервис?

Решение

Утеранный ответ на HTTP-запросы `POST` ведет к проблемам, потому что состояние ресурса сервера оказывается неясным. Был ли сервер обновлен? Дополнительно ситуацию усугубляет то, что в HTTP-спецификации (<https://oreil.ly/M2ZQg>) метод `POST` не определяется как *идемпотентный*. Это означает, что, в отличие от `PUT` (<https://oreil.ly/35Mmc>), `POST`-запрос «в случае сбоя связи до момента прочтения клиентом ответа сервера автоматически повторить нельзя».



Важным элементом этого рецепта является использование предоставляемых клиентом уникальных URL. Подробнее читайте в рецепте 5.14.

Решением этой проблемы будет применение `PUT` в любых случаях, когда клиенты API хотят изменить ресурс сервиса или созданием новой записи, или обновлением существующей. Поскольку метод `PUT` идемпотентный, клиентские приложения могут уверенно повторять запросы, предназначенные для изменения данных сервиса, не беспокоясь об ошибочном двойном обновлении ресурсов.



Есть и другие варианты поддержки идемпотентных обновлений при использовании метода `POST`. Одним из последних примеров является поле HTTP-заголовка `Idempotency-Key` (<https://oreil.ly/TTtQM>). Я же продолжаю призывать к реализации решений в соответствии с инструкциями этого рецепта, поскольку они прекрасно работают для существующих версий HTTP, не требуя новых заголовков или других расширений.

Клиентские приложения могут использовать также заголовок `If-None-Match` для выражения желания создать ресурс или заголовок `If-Match` — для обновления существующего.

Таким образом клиентские приложения избегают возможных проблем с получением ответов.

Пример

Ключом к решению проблемы с потерей ответов будет использование для записи данных на сервер идемпотентного HTTP-метода (`PUT`). Когда вы хотите, чтобы клиентские приложения создавали новые ресурсы, запрограммируйте их на применение метода `PUT`, передачу полного URL и включение заголовка `If-None-Match` со значением `"*"`.

Вот пример использования `PUT` для создания HTTP-ресурсов:

```
PUT /persons/q1w2e3r4
Host: api.example.org
Content-Type: application/json
Content-Length: XXX
If-None-Match : "*"
{"name":"Mark Morkelson"}
```

При получении этого запроса сервис может проверить, существует ли уже ресурс по предоставленному URL, и если с подстановочным ETag `"*"` совпадений нет (if none match), сервер создаст ресурс и вернет следующий ответ:

```
201 Created
Location: http://api.example.org/persons/q1w2e3r4
```

Если же ресурс по этому URL уже существует, ответ сервера может быть таким:

```
209 Conflict
Content-Type: text/plain
```

```
Unable to create. Resource already exists.
```

Когда клиентские приложения хотят обновить существующие ресурсы, им нужно вместе с запросом PUT передать уникальный ETag целевого ресурса (предоставляется сервером). Вот пример обмена сообщениями:

```
**** REQUEST ****
GET /persons/q1w2e3r4
Accept: text/plain

**** RESPONSE ****
200 OK
Content-Type: application/vnd.collection+json
ETag: "w/p0o9i8u7y6yt5r4"

{"collection": {
  "items": [
    {"href" : "/persons/q1w2e3r4", "data" : [ {"name" : "Mark Morkleson"} ]}
  ],
  "template" : { "data" : [ {"name" : "Mork Markleson"} ] }
}}

**** REQUEST ****
PUT /persons/q1w2e3r4
If-Match: "w/p0o9i8u7y6yt5r4"
Content-Type: application/x-www-form-urlencoded
Accept: application/vnd.collection+json

name=Mork%20Markleson

**** RESPONSE ****
200 OK
Content-Type: application/vnd.collection+json
ETag: "w/i8u7y6t5r4e3"

{"collection": {
  "items": [
    {"href" : "/persons/q1w2e3r4", "data" : [ {"name" : "Mork Markleson"} ]}
  ]
}}
```

Заметьте, что в случае обновления заголовков **If-Match** отправляется вместе со значением заголовка **ETag** существующего ресурса. Получив такой запрос PUT, сервер может использовать значение заголовка **If-Match**, чтобы подтвердить существование ресурса и то, что он не был изменен каким-либо другим процессом обновления (это привело бы к новому значению ETag). Если обновления не происходит (например, ресурса нет или его тег не соответствует), сервер может вернуть ответ **409 Conflict**.

Обсуждение

Передача при создании записи полного URL некоторым проектировщикам API может показаться странной. Но это стандартный способ, которым большинство клиентских приложений в HTTP-среде реализуют загрузку файлов — изображений, PDF-документов и т. п.

Данный рецепт опирается на взаимодействие клиентских приложений с HTTP-заголовками (ETag, If-None-Match и If+Match). Это может стать проблемой для некоторых потребителей API, например для браузеров, ограниченных HTML FORMS. Но вы все равно можете заставить этот механизм работать, передав связанную с ETag информацию в виде скрытых полей в формах, предоставляемых сервером. Технически можно даже использовать для этого рецепта метод POST. Тогда реализация HTML-браузера будет выглядеть так:

```
<p>Create a New Person</p>
<form name="create" action="/persons/q1w2e3r4" method="post">
  <input type="hidden" name="if-none-match" value="*" />
  <input type="text" name="name" value="Mark Morkleson" />
  <input type="submit" />
</submit>

<p>Update an Existing Person</p>
<form name="update" action="/persons/q1w2e3r4" method="post">
  <input type="hidden" name="if-match" value="w/p0o9i8u7y6yt5r4" />
  <input type="text" name="name" value="Mork Markleson" />
  <input type="submit" />
</submit>
```

И хотя это сработает для POST, я все равно рекомендую программировать серверы на использование именно PUT, поскольку он определен как идиempотентный (позволяющий выполнять повторы), а POST только позиционирует взаимодействие как повторяемое.

Придерживаясь применения идиempотентных методов для всех операций записи, сервисы также могут предоставить поддержку автоматических повторов операций обновления. Большинство HTTP-библиотек поддерживают автоматические повторы при использовании метода GET, но я еще не встречал такой, которая поддерживает их и для PUT или DELETE. Подробнее об этом варианте читайте в рецепте 5.16.

Читайте также

- Рецепт 3.6. Проектирование последовательных операций записи с помощью идиempотентных действий.
- Рецепт 5.14. Повышение скорости обработки с помощью клиентских идентификаторов.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 6.2. Делаем все изменения идиempотентными.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.

5.16. Предоставление зависимым сервисам запасного варианта выполнения

Если сервис вызывает другой сервис, то он от него *зависит*. Поскольку вызовы сервисов опираются на сетевое взаимодействие, здесь может возникнуть много разных проблем. И хотя вероятность неуспеха вызова от одного зависимого сервиса весьма невелика, в случае зависимости вашего сервиса от нескольких внешних она возрастает экспоненциально. При реализации API сервисов необходимо эту вероятность учитывать и быть готовым к решению возможных проблем.

Задача

Как в случаях, когда сервис зависит от другого, внешнего сервиса, можно уменьшить риск сбоя? Какие паттерны можно использовать, чтобы снизить вероятность появления таких фатальных зависимостей, и как обходить их в среде выполнения, когда они возникают? Как обеспечить надежную работу сервиса в случае *чтения* данных с зависимых сервисов и *записи* данных в них?

Решение

Бывают случаи, когда API фактически представляет интерфейс, агрегирующий другие API. Например, API для покупок может быть реализован в виде комбинации трех других API: API корзины покупателя, платежного API и API доставки. Опасность в том, что один или несколько этих зависимых API могут оказаться недостижимыми в среде выполнения из-за сетевых или локальных проблем сервиса. На этот случай каждому зависимому API необходим надежный запасной план.



Важно помнить, что ваши решения должны учитывать и сценарии чтения, и сценарии записи.

Защиту интерфейса сервиса от сбоев сети или *других* сервисов, от которых он зависит, можно реализовать разными способами.

Автоматические повторы

В качестве одного из решений можно предположить, что сбой временный и повтор после небольшой паузы решит проблему. В большинство HTTP-библиотек функциональность автоматического повтора встроена для запросов GET, но немногие HTTP-модули поддерживают ее для небезопасных действий, например PUT и DELETE. Однако вполне можно подстроить код своей программы для обработки этих случаев.

Статический резервный вызов

По аналогии с тем, как можно настроить (или жестко прописать) изначальные зависимости API (например, `var shoppingAPI = "http://shopping.example.com/home"`), вы можете дополнить конфигурацию вторым, резервным расположением того же сервиса (`var shoppingAPIFallback = "http://other-shopping.example.com/home"`). В этом случае, когда сервис по своему базовому расположению не ответит, код среды выполнения будет перенаправлять вызов на альтернативный сервис. Такое решение имеет некоторые последствия для управления состоянием — подробнее читайте в рецепте 3.7.

Динамический резервный вызов

Можете заложить в свой агрегатор API возможность включать реестр сервисов для поиска доступной альтернативы, обеспечивающей нужную функциональность. По сути, это будет поиск в среде выполнения, аналогичный системе DNS, позволяющей находить различные машины. Подробнее читайте в рецепте 5.13.

Постановка запросов в очередь для последующего повтора

Если у вас не получается подключиться к базовому или ожидаемому альтернативному сервису, можете просто разместить запрос в очереди и повторить его позднее, когда не работающий в данный момент сервис окажется доступным. Для этого потребуется настроить и поддерживать локальную очередь и код для обработки ее содержимого. Как правило, изначальным ответом в этом случае будет HTTP-код **202 Accepted** с телом, описывающим, как потребитель API может отслеживать обработку отложенного запроса (см. рецепт 7.15). Естественно, вам потребуется включить эту возможность в задокументированный дизайн интерфейса, чтобы потребители API были готовы к подобному ответу.

Прекращение обработки

Последним решением будут прекращение обработки запросов, извещение потребителя API о том, что вы сейчас не можете обработать его запрос, и возврат HTTP-кода ошибки **500**. Это самый безопасный и наименее функциональный вариант. В случае него по возможности следует вернуть тело ответа с указанием примерного времени, когда потребитель API сможет повторить свой запрос. Помните, что отправленный вами код **500 Internal Service Error** может привести к тому, что потребитель API запустит собственный резервный код, способный повлиять на потребителей его приложения, и так далее по цепочке.



Если зависимый сервис применяется в основном как простой источник данных (например, в качестве списка почтовых кодов, стран, штатов/округов, номеров продуктов и т. д.), то вам может быть доступно использование некоторых рецептов кэширования, описанных в главе 6.

По сути, вам нужно реализовать API так, чтобы он учитывал возможный сбой других элементов системы, от которой он зависит, и встроить в него не менее одного из

описанных резервных решений для каждой возможной точки сбоя. В любом случае *предотвратить* сбой не получится, но вы можете подготовиться и избежать их влияния. Как сказал Джон Галл (John Gall) (<https://oreil.ly/fBrWj>): «Любая крупная система основную часть времени будет работать в режиме сбоя».

А КАК НАСЧЕТ ШАБЛОНА CIRCUIT-BREAKER?

Вы могли ожидать применения в этом рецепте и других решений, таких как Circuit Breaker (автоматический выключатель, <https://oreil.ly/VjvV8>). Поскольку настоящая книга ориентирована преимущественно на М2М-взаимодействия, я не стал включать в нее многие устоявшиеся решения на основе кода вроде Circuit Breaker и пр. В предисловии упоминаются другие книги, посвященные программированию микросервисов, включая Circuit Breaker и другие шаблоны.

Пример

В этом рецепте описано несколько возможных решений, далее приведены примеры каждого.

Автоматические повторы

Вы можете настроить код вызова API на автоматический повтор запросов. Ключевыми элементами в этом процессе будут:

- тип сбоя запроса (например, вернулся HTTP-код 502);
- тип совершаемого запроса (например, GET, PUT, DELETE);
- время ожидания между повторами (например, 250 мс);
- количество повторов до их прекращения (например, повторить запрос три раза).

Внутренний код API может выглядеть так:

```
var reqParams = {} // Параметры запроса
reqParam.host = "https://api.example.com"
reqParams.url = "/users/q1w2e3";
reqParams.body = "mork=mamund&name=Mike Morkelsen";
reqParams.method = "PUT";
reqParams.waitMS = 300;
reqParams.retryAttempts = 3;
reqParams.successFunction = requestSucceeded;
reqParams.failFunction = requestFailed;
```

```
httpLib.request(reqParams);
```

После сбоя этого запроса вы можете повторить его, изменив продолжительность ожидания, или же попробовать иное решение из текущего рецепта (статический

резервный вызов, динамический резервный вызов, очередь запросов или прекращение обработки).

Статический резервный вызов

Можно изменить функциональность запросов, включив в нее попытки вызова альтернативного хоста. Для этого нужно указать его расположение в коллекции запросов и внести в метод запроса код для смены хостов и совершения дополнительных попыток:

```
var reqParams = {} // Параметры запроса
reqParams.host = "https://api.example.com"
reqParams.url = "/users/q1w2e3";
reqParams.body = "mork=mamund&name=Mike Morkelsen";
reqParams.method = "PUT";
reqParams.waitMS = 300;
reqParams.retryAttempts = 3;
reqParams.successFunction = requestSucceeded;
reqParams.failFunction = requestFailed;
reqParams.alternateHost = "https://alternate-api.example.com";

httpLib.request(reqParams);
```

После сбоя этого решения можете попробовать другие варианты.

Динамический резервный вызов

Это решение чуть сложнее, так как здесь у вас нет фиксированного альтернативного хоста и его нужно искать. Подробнее о реализации этой техники читайте в рецепте 5.16.

Внесение запросов в очередь

Если повторы и резервные вызовы не выполняются, можете поставить запрос в очередь и повторить его позднее. Такое решение можно добавить в виде одной из опций в локальную реализацию запроса:

```
var reqParams = {} // Параметры запроса
reqParams.host = "https://api.example.com"
reqParams.url = "/users/q1w2e3";
reqParams.body = "mork=mamund&name=Mike Morkelsen";
reqParams.method = "PUT";
reqParams.waitMS = 300;
reqParams.retryAttempts = 3;
reqParams.successFunction = requestSucceeded;
reqParams.failFunction = requestFailed;
reqParams.queuingFunction = queueRequest;
reqParams.alternateHost = "https://alternate-api.example.com";

httpLib.request(reqParams);
```

При этом потребуются возвращать потребителю API HTTP-код **202 Accepted** вместе с телом, описывающим дополнительную информацию. Подробнее читайте в рецепте 7.15.

Прекращение обработки

В определенный момент вам придется признать неудачу и прекратить пытаться выполнить запрос. В этом случае нужно вернуть соответствующий ответ (код ошибки **5xx**) вместе с телом, указывающим на невозможность выполнения. Поскольку этот запрос может быть одним из серии запросов потребителя к API, необходимо включить в него любую информацию, которая может ему потребоваться для выполнения дополнительных откатов или других запросов в серии. Подробнее читайте в рецепте 7.16.

Обсуждение

В этой ситуации хорошо то, что первые несколько приведенных решений (автоматические повторы, статические резервные вызовы и динамические резервные вызовы) можно реализовать без согласования с клиентом API, так как они являются скрытыми деталями реализации.

Добавление же запросов в очередь требует основательной координации с потребителем API, так как для этого не только нужно, чтобы он был готов к получению ответа **202 Accepted**, но и может потребоваться уведомить другие сервисы. Рассмотрим в качестве примера случай, когда наш сервис обрабатывает платежные операции и какой-то зависимый сервис оказывается недоступным. Возврат вызывающему ответа **202 Accepted** может вызвать на его стороне путаницу. Возможно, он уже скорректировал складские остатки и запланировал отправку приобретенных товаров. Теперь же ему потребуются решить, будет ли он откатывать эти действия, оставит все как есть и не станет сообщать об этом вызывающему или предложит ему вариант подождать либо отменить эти действия.



Не пытайтесь делать повторы для неидемпотентных HTTP-методов вроде **POST** или **PATCH**. Используйте эти меры только для **GET**, **HEAD**, **PUT** и **DELETE**. Бывает, что операции **POST** можно интерпретировать как сбои, даже если они завершились успешно, например, когда ответ целевого сервиса **201 Created** не достигает потребителя API, который совершил вызов. Подробнее об этом сценарии читайте в рецепте 5.15. В случае же **PUT** последствия ложного сбоя с гораздо меньшей вероятностью приведут к повреждению или дублированию данных в целевом сервисе.

За исключением варианта с прекращением обработки, все упомянутые решения подразумевают некую дополнительную работу на стороне API. Для получения

реальной пользы эти дополнительные действия (повторы, резервные вызовы, очередь) нужно реализовывать локально.

Не допустите ошибки, реализовав эти меры как общие внешние сервисы, вызываемые вашим API. Таким образом вы и их превратите в зависимости, возможно, фатальные. Если код вашего API не сможет связаться с другими внутренними сервисами для выполнения работы, у него также может не получиться связаться и с внешними запасными решениями.

И хотя обычно прибегают к использованию одного из этих механизмов для любого HTTP-ответа с кодом 5xx, вы можете решить ограничить число сценариев, в которых они задействуются. Например, хорошим кандидатом для их применения будет ответ с кодом **408 Request Timeout**.

В будущем могут быть добавлены новые варианты HTTP-ответов уровня 5xx, поэтому не всегда стоит прибегать к этим решениям для *любого* ответа с кодом 5xx. Я рекомендую рассмотреть в качестве кандидатов для повторов, резервных вызовов и очередей HTTP-ответы с кодами **500**, **502**, **503** и **504**. Для других кодов уровня 5xx, возможно, будет лучше сразу использовать вариант с отказом от дальнейших попыток.

Эти решения не требуют согласия на них внутреннего (отказавшего) сервиса. Например, ваш интерфейс может реализовать повторы, резервные вызовы или внесение вызовов в очередь, не вовлекая в этот процесс сервис, который не смог своевременно ответить.

При реализации повторов следует ознакомиться с документацией целевого сервиса (которому вы собираетесь отправлять повторные вызовы), чтобы не запрограммировать интерфейс так, что его вызовы будут приводить к отказу в обслуживании. Проследите за тем, чтобы не перегружать внутренние сервисы слишком частой отправкой повторов.



Стоит вести историю всех запросов, обработанных вашим сервисом, особенно в случаях наподобие описанных в текущем рецепте, когда он выполняет дополнительную работу по сохранению, перенаправлению и обработке запросов потребителя API.

При реализации резервных решений вам может потребоваться обеспечить дополнительное управление запросами/ответами, если вы планируете отслеживать эти действия, например вести историю выполненных для потребителя API операций. Настраивая в качестве запасного решения использование других сервисов, вы делаете исполнителя обработки *переменным* — не все запросы могут быть обработаны одним и тем же сервисом. Если эта деталь имеет значение, потребуется настроить интерфейс сервиса для отслеживания этих запросов и, возможно, предоставить дополнительные варианты их отслеживания и управления ими.

Читайте также

- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.9. Повышение производительности с помощью директив кэширования.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.
- Рецепт 7.17. Поддержка локальной отмены или отката.

5.17. Использование семантических посредников для доступа к несовместимым сервисам

В сердце рецептов, приводимых в этой книге, лежит идея поддержки устойчивых клиентов API за счет реализации интерфейсов сервисов на основе гипермедиа. Но бывают случаи, когда сервисы, которые нужно раскрыть в сети, не поддерживают средства гипермедиа. Таков, например, FTP-сервис загрузки файлов (file transfer protocol — протокол передачи файлов). Иногда у нас нет возможности изменить подобные сервисы, так как ими управляет третья сторона, находящаяся вне IT-экосистемы.

Задача

Как можно безопасно сделать несовместимые сервисы доступными в нашей RESTful-экосистеме? Когда будет правильным организовать локальный прокси-сервис, обертывающий другой сервис, или задействовать удаленные сторонние сервисы? Что нужно для создания стабильных, надежных посредников для сервисов, которые мы не разрабатывали и не обслуживаем?

Решение

Бывает, что нам нужна функциональность существующего сервиса, но в него не заложена поддержка адаптируемых и развиваемых интерфейсов RESTful-экосистемы. В таких ситуациях нужно создавать локальные промежуточные сервисы, которые будут совместимы с интерфейсами и выступят посредниками между несовместимыми сервисами и остальной частью экосистемы.

Как правило, для этого следует проектировать *нужный* интерфейс сервиса, а затем писать внутри оболочки API собственный локальный код, который будет переводить RESTful-запросы в форму, понятную не-RESTful-сервисам. Эти посредники также должны переводить ответы несовместимых сервисов в представления, понятные RESTful-ресурсам.

Иногда единственной проблемой сервиса является то, что он не работает с нужным семантическим словарем или не поддерживает нужные медиатипы для обмена сообщениями. В таких случаях несложно ввести в цепочку посредника, который

будет заниматься семантическим переводом, скрывая от потребителей API несовместимый сервис.

Решение с созданием посредника имеет смысл, когда вам нужно предоставить функциональность, заключенную в несовместимые сервисы (см. раздел «Общие принципы масштабируемых сервисов в веб-среде» главы 1), и невозможно или нецелесообразно встроить в эти сервисы поддержку RESTful-принципов.

Существуют три вида прокси-сервисов, которые мы разберем в этом рецепте: прокси корпоративного уровня (enterprise-level proxy, ELP), индивидуальный прокси (custom one-off proxy, COP) и прокси семантических профилей (semantic profile proxy, SPP).

Прокси корпоративного уровня

В некоторых случаях у вас будет возможность создать ELP, выполняющий алгоритмический перевод между сервисами, не соответствующими и соответствующими RESTful-принципам. Хорошим примером будет решение IBM CICS External Call Interface (EXCI, внешний интерфейс вызова CICS, <https://oreil.ly/OG0ph>). Оно отлично подходит, когда ваша организация приложила много усилий к разработке специализированной платформы сервисов (например, системы бухучета), которая никогда не реализовывалась в виде набора RESTful-сервисов.

Подход с ELP хорош, когда нужно перевести устоявшуюся коллекцию связанных сервисов или один монолит, предоставляющий широкую функциональность из одного источника.

Индивидуальный прокси

Более инкрементным подходом будет создание COP для перевода существующей функциональности в интерфейс, соответствующий принципам RWA. Например, вам может потребоваться добавить RESTful-интерфейс в существующий FTP-сервис для загрузки файлов. Подход с COP уменьшает количество усилий, необходимых для решения проблемы совместимости, к тому же он проще в плане изменения и масштабирования.

Используйте COP, когда у вас есть однородная комбинация сервисов, которые нужно согласовать, или когда вы ожидаете, что подобное потребуется для небольшого набора сервисов.

Прокси семантического профиля

Иногда единственной преградой для интероперабельности внешнего сервиса является его неумение говорить на языке, понятном другим частям вашей системы. Например, вы можете зависеть от сервисов, которые обмениваются сообщениями только в формате `text/csv`, а не в гипермедиаформате вроде SIREN или Collection + JSON. Либо ответы могут быть выполнены не в предпочтительном для вашей организации словаре (например, FHIR для медицинских данных или

BIAN для банковской информации). В таких случаях у вас может существовать возможность настроить SPP для нормализации обмена данными.

Реализации SPP могут представлять простое преобразование XML в SIREN или сложный процесс добавления гипермедиаформ в набор операций обмена сообщениями, изначально созданных с поддержкой только CSV-файлов. Поэтому иногда сложно оценить уровень усилий, необходимых для проектирования и реализации решения на основе SPP. Вы можете использовать подход в стиле ELP для налаживания стандартизированных преобразований сообщений. Например, вам может быть доступно написание посредника на языке XLST, который будет выполнять перевод между Collection + JSON и XML.

Независимо от применяемого подхода (SPP, COP или ELP) все прокси сервисов должны:

- создавать семантический профиль, определяющий используемую область (см. рецепт 5.7);
- публиковать документ с определениями API, описывающий функциональность прокси (см. рецепт 5.9);
- писать код, реализующий все внешние действия с помощью внутренних сервисов, то есть их функций (см. рецепт 5.4).

По сути, написание прокси представляет собой задачу по проектированию, определению и реализации нового интерфейса сервиса. Ключевым отличием здесь является то, что уже существует рабочий сервис, опирающийся на другую, несовместимую реализацию.

Пример

Все модели прокси (ELP, COP и SPP) требуют немного различающихся подходов к реализации.

Пример COP

Вот пример с реализацией RESTful-прокси для FTP-сервиса загрузки файлов. Код описывает интерфейс внутреннего сервиса, который нужно обернуть, а также реализацию RESTful-интерфейса:

```
// Внешнее действие загрузки по HTTP
function httpUpload(file) {
    var uploader = new httpService();
    var file = uploader.read();
    return file;
}

// Клиентская служба FTP
function ftpUpload(file) {
    var client = new ftpService();
```

```
    var results = client.put(file);
    return results;
}

// Прокси-функция для загрузки файлов
function proxyUpload(file) {
    var results = null;;
    var file = httpUpload(file);
    if(file) {
        results = ftpUpload(file)
    }
    return results;
}
```

Вот пример RESTful-интерфейса для загрузки файлов (на HTML):

```
**** REQUEST ****
GET /upload-file/ HTTP/1.1
Accept: text/html
...

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: text/html
...

<form method="post" action="https://api.example.org/uploads/"
  enctype="multipart/form-data" >
  <input type="file" name="file" value="daily-batch.txt" />
  <input type="submit" value="Upload" />
</form>

**** REQUEST ****
POST /uploads/ HTTP/1.1
Accept: text/html
...

Content-Disposition: form-data; name="file";
...

**** RESPONSE
HTTP/1.1 200 OK
Content-Type: text/html
....

<p>File has been uploaded</p>
```

Пример ELP

В случае прокси корпоративного уровня его можно рассмотреть как шлюз перевода, аналогичный тем, что использовались в примере с Wireless Application Protocol (WAP). Другими примерами алгоритмических прокси являются механизмы

преобразования XML-сообщений в форматы JSON, переводчики естественного языка и даже переводчики протоколов вроде того, что показан в примере с прокси FTP ↔ HTTP или с преобразованием WAP ↔ HTTP, о котором шла речь ранее.

Пример SPP

Простые SPP, например переводчики форматов, обычно реализуются легко, если ответы от внутреннего сервиса последовательны, а форматы вывода при этом несильно адаптированы к индивидуальным потребностям. Например, у вас может быть XSLT-реализация, которая последовательно преобразует XML в Collection + JSON:

```
results = convert(xmlDocument, xsltCollectionJSON);
```

Однако вам может потребоваться внедрить в ответы еще и средства гипермедиа. Такое решение прекрасно работает, если внутренний сервис предоставляет интерфейс состояния, например модель в стиле CRUD. В этом случае можно просто прописать в скрипте преобразования стандартные операции Create — Read — Update — Delete.

Наконец, если вам нужно преобразовать словарь внутреннего сервиса, например `first Name` → `givenName` и т. д., то потребуются чуть больше усилий. Как правило, сложно использовать язык преобразования вместе с XSLT. Вместо этого вам, скорее всего, придется писать дополнительный код преобразования, выполняющийся отдельно от любых переводов формата.

Обсуждение

Реализация COP подходит, когда функциональность, которую вы хотите предоставить, ограничена небольшим набором статических операций. ELP нужно резервировать для крупномасштабных задач по преобразованию существующих решений в более современный доступный набор внешних действий. Я работал со множеством ELP-проектов, и часть из них требовали намного бóльших усилий, чем ожидалось. При этом почти все они жили лишь до тех пор, пока не появлялась новая, более эффективная технология.

Прокси сервисов редко поддерживают быстрое масштабирование. Чаще всего работа по переводу между двумя интерфейсами — это кропотливый процесс, состоящий из манипуляций со строками и переключений протоколов. Поэтому прокси-переводчики должны ограничиваться такими частями системы, которые не требуют большого объема передачи данных или малой задержки. Обычно для проксирования сервисов используются реализации на основе очередей.



Не стоит приступать к созданию ELP, не убедившись в достаточном объеме ресурсов. И речь идет не только о деньгах, но и о времени, которое потребуется для завершения первого релиза и его дальнейшего обслуживания.

При реализации ELP-переводчика файлы семантического профиля и определения API документируют семантику на уровне шлюза, а не на уровне предметной информации, передаваемой через него. Таким образом реализация ELP оказывается очень близкой к проектированию, определению и реализации нового протокола обмена сообщениями или полноценного фреймворка программирования.

Как правило, прокси-переводчики протоколов (FT↔HTTP и т. д.) реализуются относительно легко, потому что протоколы уровня приложения являются зрелыми и имеют стабильные спецификации. При этом существует немало «кроличьих нор» в вопросах метаданных (HTTP-заголовков, обработки кодов состояния FTP и т. п.), но чаще всего их можно сгладить с помощью стандартных ответов.

И напротив, успешная реализация SPP зачастую оказывается наиболее сложной. Нередко плохо прописанные словари, двусмысленность терминов и давно забытые в организации детали, закопанные в локальных реализациях сервисов, превращают создание SPP в утомительную задачу. Чем меньше словарь (например, он ограничен предметной областью), тем с большей вероятностью вы добьетесь успеха.

Я встречал два вида успешных прокси сервисов. Первый — это тот, который просто работает довольно хорошо и лишь до того времени, пока не появится более стабильное решение, например новый продукт, причем чем раньше оно появится, тем лучше. Вторая успешная реализация прокси — это такая, которая существует долгие годы, и никто понятия не имеет, как конкретно она работает. Ни один из ее проектировщиков или программистов уже недоступен, и только глупец попытается изменить хоть что-то в этом загадочном, но стабильном прокси.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.9. Публикация документов определений сервисов.
- Рецепт 6.11. Расширение удаленных хранилищ данных.

ГЛАВА 6

Распределенные данные

Чтобы отказаться от привычки опираться на данные, первым делом нужно перестать проектировать системы как коллекции сервисов данных и делать это с целью расширения бизнес-возможностей.

Ираклий Надареишвили, JPMorgan Chase

Эта глава посвящена рецептам интерфейсов сервисов, ориентированных на данные. Эти интерфейсы должны соответствовать принципам, описанным в главе 5, а также иметь некоторые дополнительные детали, которые сопутствуют обязанности хранить данные и управлять ими. Эти детали включают в себя обеспечение целостности данных, сокрытие внутренних моделей данных и технологии реализации, а также обработку всевозможных сбоев сети без вреда для существующих данных. Все это стало особенно важным сегодня, когда данные могут с легкостью путешествовать по миру и сталкиваться с различными законами, такими как General Data Protection Regulation (GDPR, общий регламент ЕС по защите персональных данных, <https://gdpr-info.eu/>) и пр.

Не всем сервисам нужно управлять своими данными, но большинство несут определенную ответственность за их поддержку. Проблема сервисов на основе данных в том, что обычно они поддерживают постоянные данные (persistent data), то есть такие, которые должны продолжать существовать и быть доступными для других сервисов, даже когда сервис отключается. В некоторых случаях перед интерфейсом сервиса стоит задача комбинирования локально управляемых данных с данными из внешних сервисов. Это ставит под вопрос целостность и надежность, поскольку целевой интерфейс для выполнения запрошенной работы теперь должен опираться на *другие* сервисы данных. Особенно актуально это в случае записи — чем больше сервисов участвует в действии, тем вероятнее ошибка и тем сложнее устранение проблем.

Поддержка распределенных данных в среде гипермедиа (рис. 6.1) означает создание ответов с данными в виде сообщений и возврат средств гипермедиа для извещения о возможных действиях с возвращенными данными. Это также означает

предоставление общего языка запросов для извлечения информации (information retrieval query language, IRQL), не зависящего от внутренней технологии запроса данных, такой как SQL, GraphQL и т. д. Редактирование данных в сети означает также поддержку идемпотентности для всех изменений с целью повышения вероятности успешной записи. Наконец, использование распределенных данных в веб-среде ведет к необходимости поддерживать возможность изменения и расширения модели данных бэкенда без нарушения работы интерфейса сервиса.

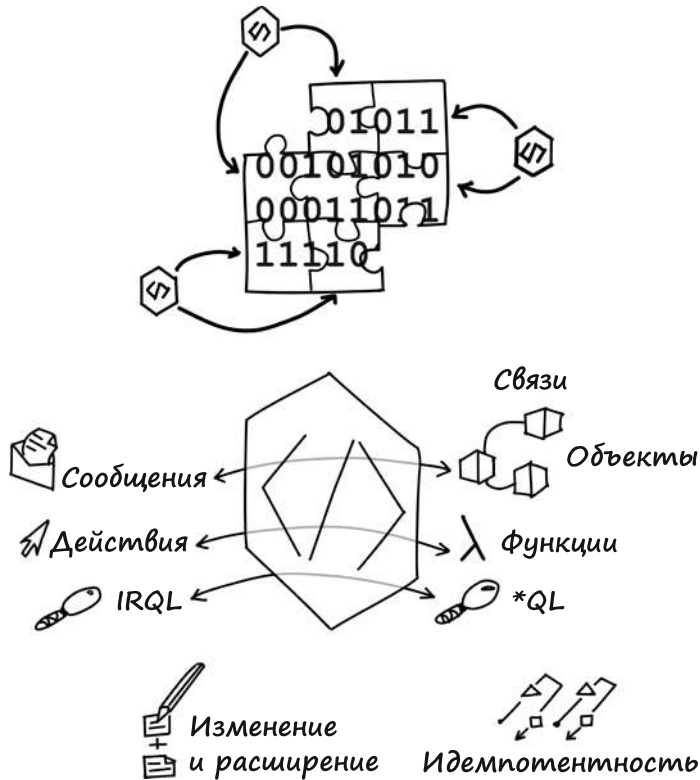


Рис. 6.1. Рецепты для работы с гипермедиаданными

ПРАВИЛО МИНИМАЛЬНЫХ ВОЗМОЖНОСТЕЙ

Рецепты для этой главы отбирались с целью подчеркнуть типичные сложности, показать общие принципы и предоставить рекомендации по реализации API, следующих правилу минимальных возможностей Тима Бернерса-Ли (<https://oreil.ly/o9L8r>): «Чем меньше возможностей у языка, тем больше вы можете делать с хранящимися при его помощи данными».

Сервисы данных, несомненно, сложны. Приведенные здесь решения опираются на использование наименее мощных языков данных, чтобы поддерживать максимум возможностей для работы с хранящимися при их помощи данными.



Дополнительная история о проектировании и реализации сервисов данных описана в разделе «Поддержка распределенных данных» главы 2.

6.1. Соккрытие внутренних деталей хранилища данных

Технологии хранения данных в течение многих десятилетий сильно менялись. В то же время проектировщикам интерфейсов зачастую нужно создавать API, которые позволяют взаимодействовать с сервисами данных, написанными с использованием технологии и возможностей, предназначенных для локального доступа, а не распределенной сети. Руководящим принципом при создании сервисов данных будут соккрытие их внутренней технологии за интерфейсом и представление таких API, которые говорят на языке их потребителя, а не на языке внутренней технологии хранения данных.

Задача

Как лучше всего обеспечить, чтобы API для сервисов данных не допускали утечки используемых сервисом внутреннего хранилища и технологии запросов? Как ограждать потребителей API от периодических изменений внутренних технологий хранения данных? Когда есть смысл раскрыть синтаксис текущей технологии хранения, а когда разумно начать применять для интерфейсов сервисов более обобщенные языки запросов, управления данными и хранения?

Решение

В целом стоит *скрывать* от потребителей API технологию хранения данных. Они не должны знать, используете вы решение на базе SQL, GraphQL или простое файловое хранилище. В идеале нужно проектировать интерфейсы так, чтобы можно было менять внутреннее хранилище данных без вреда для потребителей API. Лучшим способом в этом случае будет обеспечить, чтобы интерфейс сервиса акцентировался на поставленной задаче (например, `updateCustomer` или `findUnpaidInvoices`), а не опирался на язык данных (например, `writeToDB(customerObject)` или `queryData('invoices where balance>0')`) во внешнем интерфейсе.

При добавлении функциональности обработки данных в API лучше всего раскрывать специализированные действия в виде внешнего интерфейса, оставляя выбранную технологию скрытой среди внутренних деталей реализации.

ПЛАНИРУЕТЕ СОЗДАТЬ НОВЫЙ СЕРВИС ОБРАБОТКИ ДАННЫХ?

Исключением из этого правила станут проектирование и реализация самой системы управления данными. Если ваша цель — создать очередной клон GraphQL или доработать технологию Apache Lucene, тогда вам нужно проектировать и реализовывать собственную платформу данных, включая языки для хранения, запросов и управления данными. Это серьезный труд, но определенно стоящий. Однако проектирование механизма обработки данных — это не то же самое, что поддержка сервисов данных для интерфейса пользовательского управления.

Пример

Рассмотрим случай, когда сервису нужно поддерживать обновление существующих объектов в нескольких коллекциях, например `customers`, `salesReps`, `products` и т. д. Предположим также, что наш внутренний сервис раскрывает обобщенный метод (`update(object)`), в котором объект является одним из упомянутых ранее.

В классических реализациях для HTTP можно передать форму, поддерживающую существующие записи:

```
**** REQUEST ****
GET /customers/q1w2e3 HTTP/1.1
Accept: application/vnd.siren+json
...

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.siren+json
ETag: "w\p0o9i8u7"
...
```

```
"class": ["customer"],
"properties": {
  "id": "q1w2e3r4",
  "companyName": "BigCo, Inc.",
  ...
},
"actions": [
  {
    "name": "update", "type": "application/x-www-form-urlencoded",
    "method": "PUT", "href": "http://api.example.org/customers/q1w2e3r4",
    "fields": [
      { "name": "id", "value": "q1w2e3r4" },
      { "name": "companyName", "value": "BigCo, Inc." },
      ...
    ]
  }
]
```

В этом примере вы не знаете, какая технология обработки данных задействуется для реализации обновления, — так и должно быть. Например, в первом релизе это могла быть база данных SQLite, а теперь хранилище данных реализуется с помощью GraphQL. Хорошо, что это изменение в платформе данных можно выполнить без негативного влияния на клиентов, использующих API.

Раскрыть операции записи данных (создание, обновление, удаление), не допуская утечки внутренней технологии, довольно легко. Сложность возникает, когда дело доходит до безопасного выражения запросов в виде внешних действий. Слишком часто можно допустить оплошность и раскрыть в интерфейсе сервиса обобщенный механизм запросов. Это ускорит первый релиз, но создаст сложности в дальнейшем, когда изменится внутренняя технология или схема таблиц/связей. Кроме того, взгляд *поставщика* данных не всегда совпадает со взглядом их *потребителя*, интересы которого желательно считать приоритетными.

Раскрытие внутреннего языка запросов будет плохой идеей. В таком случае вы вскоре начнете тратить время на управление этим языком вместо управления предметной областью, соответствующей интерфейсу сервиса. По возможности стоит раскрывать простые ссылки (и, возможно, формы), описывающие запрос. Такой подход соответствует упомянутому в начале главы правилу минимальных возможностей.

Предложим в качестве примера случай, когда интерфейс сервиса должен поддерживать поиск всех *customers* с остатком задолженности, срок которой превышает 30, 60 и 90 дней. Вот вариант реализации:

```
**** REQUEST ****
GET / HTTP/1.1
Host: api.customers.org
Accept: application/vnd.collection+json
...

{"collection": {
  "title": "Customers",
  "links" : [...]
  "items" : [...]
  "queries" : [
    {"name": "unpaid30", "href": "..."},
    {"name": "unpaid60", "href": "..."},
    {"name": "unpaid90", "href": "..."}
  ]
}}
```

Заметьте, в интерфейсе сервиса ничто не указывает на технологию хранения или запроса данных. А вот еще одно возможное, но чуть более сложное решение:

```
**** REQUEST ****
GET / HTTP/1.1
Host: api.customers.org
```

```

Accept: application/vnd.collection+json
...

{"collection": {
  "title": "Customers",
  "links" : [...]
  "items" : [...]
  "queries" : [
    {"name": "unpaid", "href": "...", "data" : [
      {"name": "days", "value": "30", "required": "true"}
    ]
  ]
}
]
}}

```



Добавление в API необходимых параметров подразумевает дополнительные сложности при его дальнейшем изменении. Если внутренний сервис начнет вместо значения запроса `days` использовать значение `months`, интерфейс нужно будет изменить. Здесь есть два варианта: сделать так, чтобы он продолжил поддерживать `days` и выполнял локальное преобразование, или внести обратно несовместимое критическое изменение. По возможности стоит избегать необходимых параметров.

Необходимый параметр можно сделать необязательным, если интерфейс на случай отсутствия значения в запросе будет гарантировать предоставление значения по умолчанию, например 30. Это упростит интерфейс запроса, но увеличит реализацию.

Обсуждение

Скрыть технологию обработки данных обычно несложно. Достаточно использовать HTTP-протокол для передачи сообщения (например, `customer`, `product` и т. д.) с HTTP-методом. Подробнее о записи данных читайте в рецепте 6.2.

А вот скрыть технологию запроса данных бывает непросто. В конечном счете можно легко раскрыть любые применяемые сервисом технологии запроса, особенно если API является единственным слоем между потребителем и самим хранилищем данных. Рассмотрим безумство, происходящее в следующем коде API:

```

function sqlExecute(connectionString, sqlStatement) {
  var sql = sqlConnection(connectionString);
  var results = sql.query(sqlStatement);
  return results;
}

```

А вот ему в пару HTTP-интерфейс (он ужасен):

```

POST /data/ HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json

```

Content-Type: application/x-www-form-urlencoded

...

```
sqlConnection=user=mork,pw=m04k,server=db1,database=products&
sqlStatement=select%20*%20from%20products
```

Радует то, что проектировщик без особых усилий получит полностью функциональный интерфейс обработки данных. Недостаток же в том, что здесь мы имеем серьезные проблемы с безопасностью и возможностью развития. Не делайте так!

Бывают случаи, когда вам нужно реализовать API, опирающийся на независимый язык запросов вроде Lucene или какого-то иного IRQL. В таких сценариях раскрытие языка запросов несет намного меньше рисков. Подробнее о безопасной реализации специфичной для языка поддержки запросов читайте в рецепте 6.7.

Читайте также

- Рецепт 3.2. Обеспечение будущей совместимости с помощью структурированных медиатипов.
- Рецепт 5.2. Предотвращение утечек внутренних моделей.
- Рецепт 6.3. Соккрытие связей данных для внешних действий.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.

6.2. Делаем все изменения идемпотентными

Одной из важных задач сервисов данных и их интерфейсов является обеспечение целостности. При использовании HTTP в веб-среде это бывает непросто, особенно когда сеть работает медленно или оказывается недоступной. В настоящем рецепте описывается, как повысить надежность операций записи данных по HTTP.

Задача

Иногда операции записи данных по HTTP не выполняются или из-за сбоев в сети, или из-за несогласованности самих записываемых данных. Как лучше всего повысить надежность операций записи на расстоянии при использовании HTTP?

Решение

Самым простым способом повысить надежность записи данных в веб-среде будет ее выполнение только с помощью HTTP-метода PUT. Не применяйте для записи по HTTP методы PATCH или POST. Как уже говорилось в других рецептах,

PUT идемпотентен — он построен так, чтобы возвращать одни и те же результаты даже при многократном повторении. Ни POST, ни PATCH этого не могут.

PATCH — ЭТО НЕ ПАТЧ

В реестре HTTP-методов PATCH определен как неидемпотентный (<https://oreil.ly/k8MJJD>). Однако в его полной спецификации указано, что «запрос PATCH можно отправить идемпотентным образом». Несмотря на это наблюдение, я все равно рекомендую использовать для записи по HTTP только PUT, так как нахожу реализацию PATCH более сложной.

HTTP-метод PUT (<https://oreil.ly/qdr0A>) проектировался с учетом того, чтобы «состояние целевого ресурса создавалось или заменялось состоянием, определенным представлением». Поэтому для *создания* новых записей данных и *обновления* существующих я применяю PUT.



Подробнее о том, как для создания записей вместо POST использовать PUT, читайте в рецепте 5.15.

Интерфейсы сервисов всегда должны делать HTTP-действия PUT *условными* запросами. Для этого при создании нового ресурса данных с помощью PUT необходимо отправлять заголовок `If-None-Match: *`. Это гарантирует, что запись будет создана, только если по задействованному в запросе URL еще нет ресурса.

При использовании PUT для обновления существующего ресурса данных клиент API должен отправлять заголовок `If-Match: "..."`, установив его значение на значение заголовка `Etag`, полученное при чтении обновляемой записи. Это гарантирует, что PUT будет выполнен, только когда теги сущностей (`Etag` в ответе и `If-Match` в запросе) окажутся одинаковыми.

Включая значения тегов существующей записи (или символ подстановки `*`), вы можете гарантировать, что HTTP-запрос будет выполнен только при соответствии условий. Использование тегов подобным образом также делает возможной повторную отправку одного запроса несколько раз без страха перезаписать обновление, выполненное другим клиентом API. В случае применения PUT для создания записей повторная отправка записи не приведет к случайному переопределению новой записи. Наконец, когда потребитель API получает первое сообщение о сбое (или вообще не получает ответа), он может спокойно повторить запрос на обновление, не боясь перезаписать чью-то работу.

Поскольку метод PUT идемпотентен, его можно безопасно повторять, что особенно удобно в M2M-сценариях, когда человек не сможет обнаружить и устранить проблемы, вызвавшие невыполнение запросов.

Пример

Далее приведено несколько примеров использования метода PUT для создания и обновления существующей записи ресурса.

Использование HTTP-метода PUT для создания нового ресурса

Вот короткий пример применения PUT для создания нового ресурса данных:

```
**** REQUEST ****
PUT /tasks/q1w2e3r4 HTTP/1.1
Host: api.example.org
Content-Type: application/x-www-form-urlencoded
If-None-Match: *
Accept: application/vnd.hal+json
...
```

```
id=q1w2e3r4&title=Meet%20with%Dr.%Bison
```

```
**** RESPONSE ****
HTTP/1.1 201 Created
ETag: "w/p0o9i8u7"
Location: http://api.example.org/tasks/q1w2e3r4
```

```
**** REQUEST ****
GET /tasks/q1w2e3r4 HTTP/1.1
Host: api.example.org
ETag: "w/p0o9i8u7"
Accept: application/vnd.hal+json
```

```
**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.hal+json
ETag: "w/p0o9i8u7"
...
```

```
{
  "_links": {...},
  "id": "q1w2e3r4",
  "title": "Meet with Dr. Bison",
  "dateCreated": "2022-09-21"
}
```

Использование HTTP-метода PUT для обновления существующего ресурса

Заметьте, что в первый ответ (201 Created) был включен ETag. Это предоставляет клиентскому приложению нужное значение для ETag, применяемого в следующем запросе GET.

А теперь предположим, что клиент API хочет обновить ту же запись task:

```
**** REQUEST ****
PUT /tasks/q1w2e3r4 HTTP/1.1
Host: api.example.org
```

```

Content-Type: application/json
Accept: application/vnd.hal+json
ETag: "w\p0o9i8u7"
...

{ "id": "q1w2e3r4", "title": "Meet with Dr. Bison at 16:00"}

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.hal+json
ETag: "w\y6t5r4e3"
...

{
  "_links": {...},
  "id": "q1w2e3r4",
  "title": "Meet with Dr. Bison at 16:00",
  "dateCreated": "2022-09-21"
}

```

Обратите внимание на измененное значение **ETag**, которое вернулось в обновленном ответе. Оно отражает новую версию представления ресурса задачи (task).



Теги сущностей (заголовки **ETag**) уникальны для каждого представления ресурса. Например, **ETag** для ресурса `person`, возвращаемый в виде HTML-документа, будет отличаться от **ETag** для того же `person`, возвращенного в виде документа JSON.

Обработка провалившегося обновления при использовании HTTP-метода PUT

Наконец, разберем случай, когда клиентское приложение пытается повторно обновить ресурс, но на этот раз он уже обновлен другим клиентом. В результате обновление окажется невыполненным и потребует повтора:

```

**** REQUEST ****
PUT /tasks/q1w2e3r4 HTTP/1.1
Host: api.example.org
Content-Type: application/json
Accept: application/vnd.hal+json
ETag: "w\p0o9i8u7"
...

{ "id": "q1w2e3r4", "title": "Meet with Dr. Bison at 16:30"}

**** RESPONSE ****
HTTP/1.1 412 Conflict
Content-Type: application/problem+json

{
  "type": "https://api.example.org/probs/lost-update",
  "title": "The resource has already been updated",

```

```
"detail": "The title properties do not match",
"instance": "http://api.example.org/tasks/q1w2e3r4",
}
```

После возвращения 412 Conflict клиентское приложение API должно сразу же отправить запрос GET для получения обновленной записи:

```
**** REQUEST ****
GET /q1w2e3r4 HTTP/1.1
Host: api.example.org
Accept: application/vnd.hal+json
ETag: "w/o9i8u7y6"
...
{
  "_links": {...},
  "id": "q1w2e3r4",
  "title": "Meet with Dr. Bison at downtown office",
  "dateCreated": "2022-09-21"
}
```

Получив обновленную запись, клиент API может повторно отправить запрос на изменение:

```
**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.hal+json
ETag: "w/o9i8u7y6"
...
{
  "_links": {...},
  "id": "q1w2e3r4",
  "title": "Meet with Dr. Bison at the downtown office at 16:00",
  "dateCreated": "2022-09-21"
}
```

В последнем примере ошибка была вызвана более ранним обновлением со стороны другого клиентского приложения. Такая ситуация зачастую называется проблемой потерянного обновления (<https://www.w3.org/1999/04/Editing>). Для ее исправления нужно выполнить запрос GET к существующей записи и затем изменить *именно ее*, прежде чем повторять запрос PUT.

Обработка сетевых сбоев

Еще одним возможным сценарием является сбой сети. В этом случае либо обновление не достигает сервера, либо ответ сервера не доходит до клиента. Можно рассматривать это как проблему потерянного ответа (<https://blog.container-solutions.com/why-i-stopped-using-post>). Исправляется она проще, если не сказать надежнее. Клиент API может просто повторно отправить запрос в надежде, что

на этот раз сервер окажется доступным. Если повторная попытка тоже провалится, клиенту нужно занести в журнал запись об ошибке и перестать тревожить целевой сервер.

Обсуждение

Реализация всех действий как идемпотентных (с помощью PUT) не исключает вероятности сбоев. Зато использование PUT упрощает обработку и исправление сбоев, произошедших в веб-среде.

Применение PUT для создания записей ресурсов означает, что клиентское приложение должно предоставить уникальный идентификатор ресурса, например `/tasks/{id}`. Надежный способ сделать это — программирование клиента на генерацию UUID или применение в качестве идентификатора высокоточной временной метки. Есть подробные руководства по безопасной и эффективной реализации обоих случаев. Подробнее читайте в рецепте 5.15.

ОБРАБОТКА ОТВЕТА 410 GONE

Может случиться так, что клиент API попытается использовать метод PUT для обновления записи, которая была удалена другим потребителем API через HTTP-запрос DELETE. В этом случае клиент должен будет получить ответ 410 Gone и зарегистрировать это состояние для последующего изучения.

Интерфейсы сервисов данных всегда должны возвращать в ответах ETag. Это значение — лучший способ для поставщиков и потребителей API обеспечить целостность ресурса при совершении любых запросов записи. Важно, чтобы ориентированные на данные интерфейсы сервисов *отклоняли* любую попытку изменить состояние ресурса (через PUT, POST, PATCH и DELETE), которая не включает тег сущности (If-None-Match для создания и If-Match для обновления и удаления).



Я акцентировал внимание на заголовках If-None-Match и If-Match, но спецификация HTTP определяет также заголовки If-Modified-Since и If-Unmodified-Since, опирающиеся на временные метки. Я продолжу рекомендовать использование заголовков, основанных на тегах сущности, а не на временных метках, но имейте в виду, что допустимы оба подхода.

Бывают случаи, когда обновление не выполняется и решить эту проблему простым машинным способом никак нельзя. Тогда потребитель API должен внести в журнал запись или отправить текстовое/электронное сообщение, проинформировав о сбое и попросив человека разобраться с проблемой.

Читайте также

- Рецепт 3.6. Проектирование последовательных операций записи с помощью идемпотентных действий.
- Рецепт 4.8. Каждому важному элементу ответа нужен идентификатор.
- Рецепт 5.15. Повышение надежности с помощью идемпотентной операции создания.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.

6.3. Соккрытие связей данных для внешних действий

Следуя принципу сокрытия технологии данных от интерфейса сервиса, важно скрывать и все реализуемые ею связи моделей данных. Текущий рецепт помогает сохранять отношения данных скрытыми от API, продолжая при этом поддерживать их в среде выполнения.

Задача

Как лучше всего скрывать связи моделей данных (например, `person` в виде одной или нескольких адресных записей) от интерфейса сервиса? Есть ли способ поддерживать связи с бэкендом без раскрытия находящихся за интерфейсом моделей данных или технологии их обработки?

Решение

Наиболее безопасным подходом к записи данных, включающих связи между различными сущностями, будет выражение свойств этих связанных данных в едином представлении. Например, в случае с `person` и `address` одна операция записи будет содержать все поля, необходимые для обеих сущностей. При этом либо код API, либо код стоящего за ним сервиса *при необходимости* будет разделять сообщение на множество операций записи.

Стоит также включить в ответ на запись указатель, позволяющий клиенту добавить дополнительную связанную сущность. Например, ответ на успешную операцию записи новой сущности `person` и `address` может включать ссылку или форму для добавления еще одного адреса в сохраненную модель.

Пример

На рис. 6.2 показан пример схемы связей между записями `person` и `address` во вложенной модели данных интерфейса `person`.

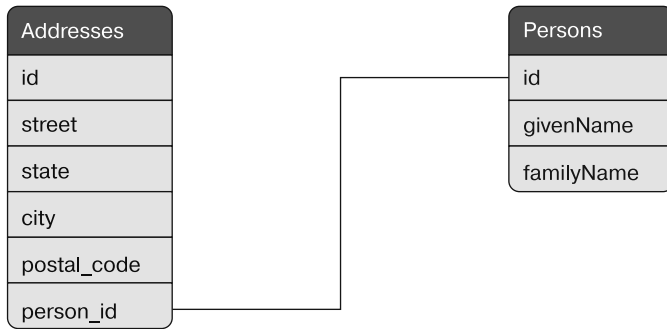


Рис. 6.2. Схема связей person — address

А вот шаблон Collection + JSON из опубликованного API:

```

{"collection": {
  "title": "Persons",
  "links": [...],
  "items": [...],
  "template": {
    "data": [
      {"name": "id", "value": "q1w2e3r4", "type": "person"},
      {"name": "givenName", "value": "Mork", "type": "person"},
      {"name": "familyName", "value": "Markelson", "type": "person"},
      {"name": "street", "value": "123 Main", "type": "address"},
      {"name": "city", "value": "Byteville", "type": "address"},
      {"name": "state", "value": "Maryland", "type": "address"},
      {"name": "postal_code", "value": "12345-6789", "type": "address"}
    ]
  }
}}
  
```

В этом примере в одно сообщение записи включены свойства данных и **person**, и **address**. Сюда была добавлена небольшая деталь реализации, а именно свойство **type**, подсказывающее, как данные могут быть организованы в хранилище. Это делается по желанию.

Код API может превратить этот процесс в две операции записи:

```

var message = http.request.body.toJSON();
var person = personFilter(message);
var address = addressFilter(message);
address.person_id = person.id;

Promise.all([writePerson(person), writeAddress(address)]).then(...);
  
```

В таком случае операции записи как родительской сущности **person**, так и связанной с ней дочерней **address** отправляются в одном HTTP-запросе, оставляя

обработку всего остального коду API. При успешном выполнении этого запроса ответ может включать ссылку для добавления дополнительных дочерних записей:

```
{
  "collection": {
    "title": "Persons and Addresses",
    "links": [
      {
        "rel": "person collection",
        "href": "...",
        "prompt": "List persons"
      },
      {
        "rel": "address create",
        "href": "...",
        "prompt": "Add another address"
      }
    ],
    "items": [...],
    "queries": [...],
    "template": {...}
  }
}
```



Еще один способ сокрытия связей внутренних моделей данных заключается в использовании рецепта «Текущая работа» для инкрементального сбора всей связанной информации (`person`, `addresses`, `online-contacts` и т. д.) в виде серии HTTP-запросов записи. Затем выполняется операция отправки для передачи всей собранной коллекции в виде заключительного HTTP-запроса. Подробнее читайте в рецепте 7.10.

Ключом к применению этого рецепта является моделирование операций интерфейса независимо от любых внутренних моделей данных. Это даст проектировщикам интерфейса свободу корректировать детали API и/или хранилища в будущих релизах, не вызывая критических изменений.

Обсуждение

Несмотря на то что приведенный здесь пример показывает одну связь (`person` и `address`), этот рецепт работает для любого числа сущностных связей. Далее показан один запрос записи, который можно смоделировать как три коллекции из хранилища (`company`, `contact` и `salesRep`):

```
<form name="createEntry" method="PUT" action="...">
  <input name="companyId" value="p0o9i8" class="company" />
  <input name="companyName" value="BigCo, Inc" class="company" />
  <input name="contactId" value="y6t5r4" class="contact" />
  <input name="contactName" value="Mork Markleson" class="contact" />
  <input name="salesRepId" value="w2e3r4" class="salesRep" />
  <input type="submit" />
</form>
```

Заметьте, что последнее свойство данных в этом сообщении записи является ссылкой только на `salesRepId` и ни на какие другие свойства `salesRep`. В данном

случае это ссылка на существующий ресурс `salesRep`. Ожидается, что слои сервиса и/или интерфейса будут знать, как действовать в этой ситуации. Модель данных (например, механизм SQL) может нуждаться только в поле ссылки, как здесь показано. А если рассматривать модель интерфейса документа, например механизм на основе файлов, может потребоваться сначала считывать данные `salesRep` из хранилища и затем включать в итоговый сохраняемый документ.



Желательно журналировать все сообщение, которое было отправлено клиентским приложением. Это особенно полезно, если клиенты просят откатить или отменить операцию записи.

В предыдущем примере использован паттерн с подсказками (`class="company"`). Это удобный способ предоставить слою API или сервису бэкенда дополнительные метаданные о том, какой вид группировки данных ожидает клиент. Но у него есть и свои пределы. До тех пор, пока группировка обеспечивается стороной, отвечающей за сбор свойств данных, все работает прекрасно. Но этот процесс подразумевает своеобразную утечку.



Важно, чтобы клиентские приложения не слишком привязывались к какому-либо подсказкам хранилища данных, включаемым в ответы. Дело в том, что моделирование хранилища в дальнейшем может измениться и эти подсказки в лучшем случае станут просто некорректными, а в худшем — опасными.

Как правило, M2M-взаимодействия могут обрабатывать значительные полезные нагрузки (которые включают несколько сущностных связей) лучше, чем человеко-машинные. Людям трудно удерживать в уме большие объемы данных, и очень длинные формы ввода будут создавать для пользователей неудобство. Если вы проектируете межмашинный интерфейс, то можете смело применить показанный здесь подход с плоской моделью. Если же создаете интерфейс для взаимодействия человека и машины, то лучше подумайте об использовании рецепта 7.10, посвященного паттерну «текущая работа».

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 6.11. Расширение удаленных хранилищ данных.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.

6.4. Использование HTTP URL-адресов для поддержки запросов типа Contains и AND

В протокол HTTP встроен простой язык получения информации (IRQL). С помощью строки запроса URL-адреса можно реализовать поддержку простого, но эффективного инструмента поиска, использующего предикат contains (содержит) и условие поиска AND (И). Текущий рецепт показывает, как быстро и легко добавлять в интерфейсы сервисов надежную поддержку поиска.

Задача

Как проще всего добавить поддержку IRQL в HTTP API? Какие поисковые операторы, условия и предикаты требуются для большинства HTTP-запросов?

Решение

В качестве самого простого инструмента поиска для HTTP-интерфейса можно использовать паттерн строки запроса URL для пар «имя/значение» (?name=value). В этом случае при реализации поддержки поисковых запросов нужно задействовать оператор = (равенства), разделяющий пару «имя/значение», но не в качестве строгой проверки равенства, а для проверки того, содержится или находится ли что-либо в чем-то.

Предположим, у вас в хранилище есть две записи:

ID	NAME	CITY
q1w2e3	Mork	Middletown
e3r4t5	Mick	Morganville

Реализуя = в роли проверки содержания чего-либо, можно создать следующие запросы:

- ?ID=q1w2e3 — возвращает первую запись;
- ?ID=e3 — возвращает обе строки.

В первом запросе механизм поиска возвращает те же результаты, как если бы оператор = был реализован в качестве проверки равенства. А вот второй запрос показывает, что этот оператор запрограммирован возвращать любую строку, в которой заданное значение (e3) содержится в указанном поле (ID).

Можно также использовать в HTTP-запросах зарезервированный символ & для реализации условия поиска AND (И). Это означает, что ID=3e&NAME=Mi вернет только вторую строку из предыдущей таблицы.



В спецификации URI (<https://oreil.ly/POh1U>) указано, что фрагмент, относящийся к запросу, чувствителен к регистру (например, запрос `X=m` не равнозначен `x=M`). Тем не менее в большинстве реализаций, которые я встречал, этот фрагмент был сделан нечувствительным к регистру. Я рекомендую создавать чувствительные к регистру механизмы поиска только при крайней необходимости, делая эту деталь реализации легко находимой, чтобы исключить путаницу и негодование пользователей вашего API.

Реализация поддержки HTTP-поиска с помощью пар «имя/значение», а также операции `=`, проверяющей содержание, и символа `&`, выражающего условие «И», — это самый простой IRQL, который можно создать для HTTP API. Причем по своему опыту могу сказать, что это единственный IRQL, который вам понадобится. Подробнее про синтаксис запроса URI читайте в спецификации IETF (<https://oreil.ly/IG9mx>).

Пример

Большинство форматов гипермедиа поддерживают ту или иную версию форм запросов. Вот ряд фрагментов представлений, использующих свойства данных из предыдущей таблицы.

HTML-поддержка IRQL с помощью FORM:

```
<form method="GET" action="http://api.example.org/persons/" rel="search">
  <input name="ID" value="e3" />
  <input name="NAME" value="" />
  <input name="CITY" value="Mo" />
  <input type="submit" />
</form>
```

Этот код сериализуется в виде следующего HTTP-запроса:

```
GET /persons/?ID=e3&NAME=&CITY=Mo HTTP/1.1
Host: api.example.org
...
```

Заметьте, что все три поля формы преобразовались в пары «имя/значение», включенные в URL, даже поле `NAME` без поискового значения. Не забудьте учесть это при реализации интерфейса. Например, вы можете не передавать это имя сервису поиска, если не уверены, что он игнорирует пустые значения.

А вот как тот же запрос выглядит при использовании формата Collection + JSON:

```
{"collection": {
  "title": "Persons",
  "links": [...],
  "item": [...].
```

```

"queries": [
  {"name": "search", "href": "http://api.example.org/persons/",
   "data": [
     {"name": "ID", "value": "e3"},
     {"name": "NAME", "value": ""},
     {"name": "CITY", "value": "Mo"}
   ]
  },
],
"template" : {...}
}}

```

Продемонстрированный пример с Collection + JSON приведет к тому же HTTP-запросу GET, который был показан ранее.

И последний пример. Медиатип HAL не поддерживает встроенные формы, но допускает встроенные шаблоны URI. Вот пример того же запроса в виде шаблона ссылок HAL:

```

{
  "_links": {
    "search": {"href": "http://api.example.org/persons {?ID,NAME,CITY}"}
  }
}

```

Заметьте, что, хотя HAL и допускает шаблоны URI, эти шаблоны не поддерживают встроенные значения.

Обсуждение

Технически любое строковое значение в URL-адресе между ? и # (или конец URL) считается запросом. Вы можете смело вставлять туда любое содержимое. Тем не менее если создадите собственный синтаксис запросов, то должны обеспечить, чтобы клиенты API поняли его, прежде чем попытаются создать и отправить запрос. Я не рекомендую создавать новый формат строки запросов.

КРЕАТИВНЫЕ ЗАПРОСЫ

Существует множество креативных способов использовать строку HTTP-запроса для поддержки функциональности IRQL. Я здесь придерживаюсь простейшего, поскольку обычно его достаточно. При этом он один из самых понятных для клиентских приложений. Можно реализовывать и другие операторы (неравенства, меньше, больше и т. д.), как и функциональность вроде BETWEEN (между), LIKE (подобно) и т. д. Тем не менее в IETF нет стандарта для выражения таких URI, и если они вам действительно нужны, то можете перейти на другие технологии запросов вроде Lucene, SQL и пр. Подробнее об этом читайте в рецепте 6.7.

Можно многократно включить одно и то же `name` в пару «имя/значение» в одном запросе:

```
?city=northville&city=southland
```

Не забудьте учесть это, когда будете реализовывать поддержку запросов в своем API.

Типичным антипаттерном для реализации запросов IRQL по HTTP будет включение одного свойства строки запроса URI, например `?q=` или `?query=`, сопровождаемого индивидуально настраиваемым запросом данных. Обычно это выглядит так:

```
GET /?query=select * from users where id='q1w2e3r4'
```

А иногда значение запроса закодировано в URL:

```
GET /?query=select%20%2A%20from%20users%20where%20id%3D%27q1w2e3r4%27
```

В обоих случаях вы, по сути, тайком проводите запрос данных внутри URL, что обычно нежелательно, поскольку в конечном счете многое может пойти не так.

Запрос слишком длинный

Запрос получается настолько длинным, что какой-нибудь сервер или встретившийся на пути шлюз может его обрезать.

Запрос содержит зарезервированные символы

Содержимое запроса может включать зарезервированный символ URI (<https://oreil.ly/NTNnw>), который сделает его недействительным.

Кодирование в URL вносит ошибки

Кодирование запроса может привести к тому, что стоящий за API сервис неправильно его поймет и вернет неожиданный результат.

Запрос создает проблему безопасности

Если вы используете точный запрос данных, например инструкцию SQL, то создаете возможность для других эксплуатировать потенциальную дыру в безопасности, что может привести к краже или повреждению данных.

Запрос вносит тесное связывание

Раскрывая прямой запрос, привязанный к модели хранилища данных (например, SQL, Qdata и т. п.), вы тесно связываете эту модель с API. В итоге, например, изменение этой модели (добавление/удаление/переименование полей) может нарушить API.

Читайте также

- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 4.10. Поддержка ссылок и форм для сервисов без поддержки гипермедиа.
- Рецепт 6.5. Возвращение метаданных в ответах на запросы.

- Рецепт 6.6. Возврат на запросы данных HTTP-кода состояния 200 или 400.
- Рецепт 6.12. Установка лимита на объемные ответы.
- Рецепт 7.14. Оптимизация запросов с помощью сохраненных повторов.

6.5. Возвращение метаданных в ответах на запросы

Будет хорошей практикой возвращать с HTTP-ответом на запрос данных не просто *результаты*, но и информацию, которая поможет потребителям API лучше оценить качество ответов и определить, нужно ли изменять запрос и отправлять его повторно для получения нужных результатов.

Задача

Как лучше всего представить результаты IRQL? Какие типы метаданных должны рассматриваться при возврате результатов HTTP-запроса, связанного с данными? Как эти метаданные должны быть представлены в ответе?

Решение

Для большинства запросов данных недостаточно просто вернуть нужные данные. Бывает важно сопроводить их метаданными запроса, чтобы сделавшее запрос клиентское приложение могло лучше понять качество и полезность возвращенных данных. В текущем разделе мы разберем набор значений метаданных, которые нужно рассматривать на предмет включения в представление ответа на HTTP-запрос (рис. 6.3).

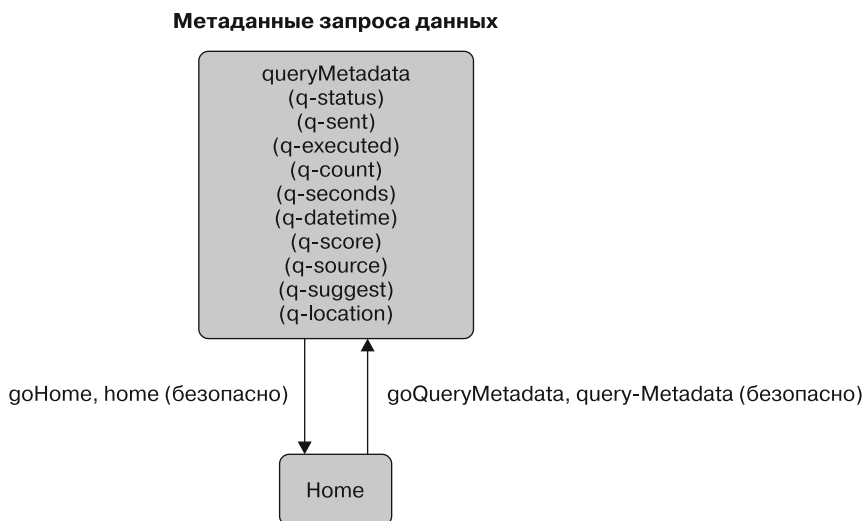


Рис. 6.3. Рецепт для предоставления метаданных запроса

Способ отправки этих значений метаданных зависит от количества свойств, которые вы хотите вернуть. Если возвращается небольшой набор значений, можете включить их в тело ответа. К примеру, если их всего два, включите эти значения в сам ответ в виде дополнительных свойств данных. Если же присутствует несколько значений метаданных, верните HTTP-ссылку (в теле или в заголовке), указывающую на ресурс, который содержит все метаданные запроса. Можете также реализовать комбинацию этих двух методов. Подробнее о возвращении описанных далее значений читайте в разделе «Пример».

Статус запроса (q-status)

Это простое описание статуса запроса. Как правило, ответом здесь будет **Completed successfully** («Успешно выполнен»). Если же запрос был неправильно сформирован, превышено время ожидания или он иным образом задержался, эта строка может содержать описание произошедшего вместе с предполагаемыми изменениями, которые нужно внести в запрос для его успешного завершения. Имейте в виду, что этот ответ отличен от статуса **error** («ошибка») при безуспешном HTTP-запросе (HTTP-ответ с кодом **400**). Подробнее читайте в рецепте 6.6.

Запрос отправлен (q-sent)

Обычно это копия строки HTTP-запроса или тела HTTP-ответа, отправляемая клиентским приложением интерфейсу сервиса. С ее помощью можно напомнить человеку об отправленном запросе, а также помочь машине убедиться в правильности считывания и отправки запроса интерфейсом сервиса.

Фактический запрос выполнен (q-executed)

В некоторых случаях интерфейс сервиса может изменять информацию запроса, чтобы отправить его в хранилище данных. Например, простой HTTP-запрос пары «имя/значение» может быть преобразован в SQL-инструкцию **SELECT**... Это свойство содержит итоговый измененный запрос. Имейте в виду, что произошедшее изменение может вызвать проблемы безопасности (подробнее — в разделе «Обсуждение»).

Количество возвращенных записей (q-returned)

Количество записей, которые вернулись в ответе. Это может быть часть от общего числа записей, соответствующая критерию запроса (см. **q-count**). Например, общее число найденных записей может составлять 11 000, а число возвращенных в ответе — 1000.

Оценочный размер итогового набора (q-count)

Это значение отражает приблизительное общее число выбранных в запросе записей. Например, запрос может приводить к выбору тысяч записей, но приславшему запрос клиенту возвращаются только первые 100. Если примерное количество довольно велико, это может указывать на излишнюю обобщенность запроса и необходимость его корректировки (подробнее — в пункте «Предложения для исправления запроса»).

Длительность выполнения запроса (q-seconds)

Это общее время (в секундах), которое потребовалось сервису для выполнения запроса. Оно может пригодиться при диагностике. С его помощью можно выяснить, например, не работает ли сервис слишком медленно или оказался ли этот конкретный запрос затратным в плане ресурсов.

Дата/время выполнения запроса (q-datetime)

Всегда нужно возвращать зафиксированную метку даты/времени, когда запрос был выполнен, особенно если ответ будет кэшироваться для последующего обзора.

Релевантность результатов (q-score)

Нелишним будет передать любое указание на релевантность результатов запроса, которые могут вернуться от механизма его выполнения. Например, Lucene поддерживает возврат `score` (показателя) запроса (<https://oreil.ly/Hqswl>) (обычно на уровне документа). Если присутствует некое описание релевантности запроса, стоит вернуть его в метаданных ответа.

Источники данных (q-source)

В некоторых случаях будет нелишним вернуть источники данных, использованные при выполнении запроса. Это поможет тем, кто хочет скорректировать запрос для улучшения результатов. Имейте в виду, что это может вызвать проблемы с безопасностью и создать излишнюю связанность. Подробнее читайте в разделе «Обсуждение».

Предложения для исправления запроса (q-suggest)

Если на запрос возвращается слишком много записей или не возвращается ни одной, в ответе можно указать варианты решения, чтобы помочь клиентскому приложению добиться более качественных результатов. Такими решениями могут быть включение или исключение определенных полей, добавление/удаление диапазонов и т. д.

URL для повтора (q-location)

Если запрос сохраняется в интерфейсе или внутреннем сервисе для последующего применения, можно сгенерировать уникальный URL, который упростит его повтор. Это может быть простой возврат тех же результатов или повтор запроса с использованием тех же (сохраненных) параметров.

Вам не нужно включать в метаданные запроса все эти поля. Выберите актуальные для конкретного случая. Например, если запрос предполагает возврат только одной записи (`?id=q1w2e3r4`), можете включить только метаданные даты/времени.

Пример

Вернуть метаданные запроса можно в одном из трех мест:

- в коллекции заголовков HTTP-ответа;
- в теле HTTP-ответа;
- в ссылке на отдельный HTTP-ресурс, содержащий все метаданные запроса.

Эти способы можно комбинировать.

Вот пример передачи метаданных запроса в коллекции HTTP-заголовков:

```
**** RESPONSE ***
HTTP/1.1 /persons/?id=q1w2e3r4 200 OK
Q-Status: successful
Q-DateTime: 2024-12-12:00:12:0012TZ
...
```

Можно также вернуть их в теле HTTP-ответа:

```
**** RESPONSE ***
HTTP/1.1 /persons/?id=q1 200 OK
Content-Type: application/vnd.collection+json
...

{"collection": {
  "title": "Person",
  "metadata": [
    {"name": "q-sent", "value": "?id=q1"},
    {"name": "q-datetime", "value": "2024-12-12:00:12:0012TZ"},
    {"name": "q-status", "value": "result set too large, query canceled"},
    {"name": "q-seconds", "value": "120"},
    {"name": "q-count", "value": "10000+"},
    {"name": "q-suggest",
      "value": "reduce return set with additional query parameters"},
  ]
  ...
}}
```

Если метаданных много или вы хотите сократить количество информации для ответа, включите ссылку на нужные метаданные в заголовок или тело представления ответа:

```
HTTP/1.1 /persons/?id=q1 200 OK
Content-Type: text/html
Link: <http://api.example.org/queries/u7y6t5r4>;rel="query-metadata"
...
<html>
```

```

<title>Person</title>
<meta name="query-metadata" href="http://api.example.org/queries/u7y6t5r4" />
...
<body>
  <a href="http://api.example.org/queries/u7y6t5r4">Query Metadata</a>
  ...
</body>
</html>

```

В этом примере метаданные запроса возвращаются, когда клиентское приложение переходит по ссылке `query-metadata` (<http://api.example.org/queries/u7y6t5r4>). Вот пример ответа `query-metadata` в формате HAL:

```

{
  "_links": {
    "self": {"href": "http://api.example.org/queries/u7y6t5r4"},
  },
  "q-status": "success",
  "q-sent": "?state=MN",
  "q-executed": "SELECT * from persons where contains(state, 'MN')",
  "q-count": "750",
  "q-seconds": ".5",
  "q-datetime": "2024-12-12:12:12:12",
  "q-score": "100%",
  "q-source": "bigco.persons",
  "q-suggest": "none",
  "q-location": "http://api.example.org/queries/t5y6r4u7"
}

```

Естественно, можно совмещать и сопоставлять эти варианты, включая часть основных данных в заголовки (например, `Q-DateTime`), часть — в тело (например, `q-status`, `q-count`, `q-suggest` и т. д.), а всю информацию — в отдельный ресурс `query-metadata` (через HTTP-ссылку).

Обсуждение

Возврат метаданных запроса имеет как положительные, так и отрицательные стороны. Слишком много метаданных — например, фактически выполненный запрос и источники данных — могут нести слишком много информации о сервисах и используемой технологии обработки данных, создавая сложности для обеспечения безопасности и внося вероятность тесного связывания. В то же время полное отсутствие метаданных может привести к неэффективным запросам, проблемам с ответами на недопустимые или неверно сформированные запросы и в целом усложняет для потребителей API создание качественных запросов. Потребуется осторожность и опыт, чтобы найти баланс в каждом уникальном случае.

В конечном счете за защиту от неправильных или вредоносных запросов отвечает именно скрытый за интерфейсом сервис. Тем не менее интерфейс зачастую может принять на себя часть этой ответственности. Напомню, что именно *интерфейс*

определяет границы HTTP-ответов. Вы можете и должны следовать задокументированным ограничениям для ответов (количество записей, время выполнения и т. д.). Подробнее эту тему раскрывает рецепт 6.12.

ЭТО СТАНДАРТНЫЕ СВОЙСТВА МЕТАДАННЫХ?

В этом рецепте описано множество свойств метаданных запроса (`q-status` и пр.). Они не стандартизованы и не являются зарегистрированными значениями заголовков или ссылочных связей. Эти значения я долгие годы использовал в проектах, над которыми работал, и считал их удобными. Возможно, вы найдете какие-то значения, которые будут удобны вам. Можете свободно их применять (после подбора подходящего документирования согласно рецепту 3.4) и делиться ими с сообществом.

Свойство метаданных `q-location` предназначено для хранения быстрой ссылки на результаты запроса. Его можно использовать в качестве способа вспомнить ранее выполненный (и сохраненный) запрос или повторно выполнить его с сохраненными параметрами для получения обновленных результатов.



Сопровождайтесь желанием загружать метаданные запроса, содержащие внутреннюю информацию сервиса (например, отладочную, статистику производительности и пр.). Подобными данными не следует делиться на уровне HTTP с другими сервисами и клиентскими приложениями. Ограничьте предоставляемые в `query-metadata` значения теми, которые непосредственно связаны с нуждами потребителя API, а связанные с внутренними процессами сервиса пусть остаются тайной.

Поддержка ссылки на ресурс `query-metadata` будет к месту, если у вас много метаданных запроса или запрос используется часто (как для доступа к старому результату, так и для генерации нового). В случаях, когда вы применяете простой HTTP-запрос в форме «имя/значение» для возврата одной записи, этот дополнительный ресурс особой пользы не принесет.

Читайте также

- Рецепт 5.8. Поддержка общих словарей в стандартных форматах.
- Рецепт 5.10. Публикация метаданных API.
- Рецепт 5.12. Стандартизация отчетов об ошибках.
- Рецепт 6.6. Возврат на запросы данных HTTP-кода состояния 200 или 400.
- Рецепт 7.7. Предоставление ресурса прогресса, отражающего выполнение рабочих потоков.

6.6. Возврат на запросы данных HTTP-кода состояния 200 или 400

При реализации API для работы с данными бывают случаи, когда тело ответа не содержит записей. В таких ситуациях важно вернуть правильный HTTP-код состояния — тот, который точно пояснит причину отсутствия записей в ответе.

Задача

Какой HTTP-код состояния нужно использовать при возврате HTTP-ответа на запрос данных, если коллекция ответа пуста? Должен ли ответ всегда быть **200 OK**? А может, это всегда должен быть код **404** или какой-то другой из разряда **4xx**? Как решить, какой класс кодов состояния нужно вернуть и как лучше всего обрабатывать пустые ответы на HTTP-запросы данных?

Решение

Как и любой HTTP-запрос, HTTP-код является важным элементом указания состояния возвращаемого ответа. Это правило касается и случаев запроса данных. Если запрос был правильно отформатирован и сервер смог его спарсить и обработать, HTTP-код состояния должен относиться к классу **2xx**. Однако если сервер не смог выполнить, понять или обработать запрос из-за ошибок в самом запросе или в сервисе, который должен был его выполнить, HTTP-код должен быть либо **4xx** (проблема на стороне клиента), либо **5xx** (проблема на стороне сервиса).

Возможны некоторые вариации этого общего правила. Подробнее — в списке далее.

200 OK

Если запрос был правильно выстроен и должен был вернуть набор записей, отвечающих критерию фильтра (например, `/persons/?status=pending`), а в результате его выполнения вернулась пустая коллекция (например, сервис не смог найти соответствующие записи), нужно сопроводить ответ HTTP-кодом **200 OK**.

404 Not Found

Если запрос должен был вернуть один существующий ресурс, например `/persons/?id=q1w2e3`, а его URL-адрес указывает на ресурс, которого *не существует*, тогда API должен вернуть HTTP-код **404 Not Found** с описанием подробностей.

4xx Bad Request

Если клиентское приложение выполнило недействительный запрос (например, указало неправильный URL-адрес, ошибочно сформировало его и т. д.), тогда API должен вернуть ответ **400 Invalid Request** (описав в нем, как исправить проблему).

5xx Server Error

Если запрос клиента был корректен, но внутренний сервис не смог его выполнить из-за проблемы на стороне сервера (например, не удалось связаться

с хранилищем данных, произошел сбой сети и т. п.), тогда API должен вернуть код состояния из разряда 5xx с подробностями.



Применение строк запроса URL для возврата одного ресурса является слабым шаблоном дизайна, которого следует избегать. Вместо использования `persons/? id=q1w2e3` для возврата существующего HTTP-ресурса лучше задействовать `/persons/q1w2e3`.

Естественно, не всегда мы можем выбирать структуру URL. Нужно научиться работать и с URL, спроектированными другими для сервисов, которые мы не контролируем.

Как правило, при возврате пустой коллекции ответ на любой правильный запрос нужно сопровождать кодом **200 OK**. Коды 4xx и 5xx следует оставить для случаев, когда клиент или сервис столкнется с ошибкой. Запрос одного ресурса, возвращающий код состояния **404**, — исключение из этого правила.

Пример

Существуют четыре типичных случая, когда нужно определить подходящий код состояния при пустых ответах на запрос данных.

Возврат 200 OK, когда к получению пустой коллекции приводит критерий фильтра

При возврате в ответе на HTTP-запрос данных пустой коллекции нужно использовать код состояния **200 OK**, сопроводив его метаданными запроса (подробнее — в рецепте 6.5):

```
**** REQUEST ****
GET /persons/?status=pending HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json
...

**** RESPONSE ***
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
...

{"collection": {
  "title": "Persons",
  "metadata": [
    {"name": "q-status", "value": "success",
    "name": "q-sent", "value": "?status=pending",
    "name": "q-count", "value": "0"
  ],
  "items": []
}}
```

Обратите внимание на метаданные запроса (см. рецепт 6.5) в теле ответа. Добавление такой информации в ответ может прояснить, почему вернулась пустая коллекция.

Возврат кода 404, когда строка запроса указывает на несуществующий ресурс

При использовании строки запроса URL-адреса для нахождения существующего ресурса нужно возвращать HTTP-код состояния **404 Not Found**, если фактически запрошенного ресурса не существует:

```
**** REQUEST ****
GET /persons/?id=q1w2e3 HTTP/1.1
Accept: text/html
...

**** RESPONSE ****
HTTP/1.1 404 Not Found
Content-Type: text/html
...

<html>
  <title>Not Found</title>
  <body>
    <h1>Not Found</h1>
    <div class="error">
      Unable to locate the requested resource
      <span class="q-sent">?id=q1w2e3</span>
    </div>
  </body>
</html>
```

Напомню, что такое использование кода состояния **404** ограничено случаями, когда URL и запрос *спроектированы* так, чтобы возвращать один ресурс, а не коллекцию строк данных или ресурсов.

Возврат 4xx при отправке клиентским приложением недействительного запроса

В случаях, когда клиентское приложение отправило недействительный запрос, нужно возвращать подходящий код состояния из серии 4xx, сопровождая его пояснением о том, как клиент может исправить проблему:

```
**** REQUEST ****
GET /persons/?hatsize=13 HTTP/1.1
Accept: application/vnd.collection+json
...

**** RESPONSE ****
HTTP/1.1 400 Bad Request
Content-Type: application/vnd.collection+json
...
```

```
{
  "collection": {
    "title": "Persons List",
    "error": {
      "title": "Data Filter Error",
      "message": "The data property 'hatsize' does not exist"
    }
  }
}
```

Заметьте, что в этом примере запрос на уровне HTTP сформирован правильно. Проблема в том, что сервис данных не может его обработать, поскольку указанные в нем данные недействительны. Естественно, бывают случаи, когда недействителен сам HTTP-запрос, что приводит к ответу с кодом 4xx.

Возврат 5xx, когда сервис не может выполнить действительный запрос

Случается, что клиент отправляет валидный запрос, но сервис не может его правильно выполнить. Так происходит, например, из-за того, что интерфейс не может получить доступ к исходным данным, обработка внутреннего запроса занимает слишком много времени и т. п. В таких случаях API должен возвращать соответствующий код 5xx вместе с информацией о своем состоянии:

```
**** REQUEST ****
GET /persons/?status=active HTTP/1.1
Accept: application/vnd.hal+json
...

**** RESPONSE ****
HTTP/1.1 504 Gateway Timeout
Content-Type: application/problem+json
...

{
  "type": "https://api.example.org/problems/time-out",
  "title": "Data query timed out.",
  "detail": "The remote data storage took too long to reply.",
  "instance": "/persons/?status=active",
  "q-status": "failed",
  "q-seconds": "120",
  "q-suggest": "Try again later."
}
```

Обратите внимание на использование медиатипа описания проблемы (см. рецепт 5.12) вместе с метаданными запроса (см. рецепт 6.5) для повышения качества возвращаемой клиентскому приложению информации.

Обсуждение

Когда нужно решить, какой код вернуть в ответе на запрос данных — 200 или 4xx, обязательно учитывайте то, возврат чего подразумевается в запросе — *коллекции* или *одного ресурса*. В первом случае пустой ответ должен сопровождаться

кодом 200 OK. Если же речь идет о возврате одного ресурса, то пустой ответ означает, что этот ресурс *not found* (не был найден).



Возврат вашим интерфейсом ответов с кодами 4xx и 5xx может привести к раскрытию внутренних деталей сети или сервисов. При создании HTTP-ответов на сообщения всегда будьте внимательны, чтобы не привести лишних деталей того, где и как хранятся ваши данные.

В некоторых случаях API может вызывать другие интерфейсы сервисов, которые не следуют этому рецепту. Например, сервис может возвращать код 404, когда на запрос коллекции возвращается ноль записей. По возможности не делайте так в отношении своих клиентов API. Лучше измените возвращаемый им ответ в соответствии с правилом 200 OK. Так вы получите более последовательный и надежный интерфейс.

Читайте также

- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.
- Рецепт 5.12. Стандартизация отчетов об ошибках.
- Рецепт 6.5. Возврат метаданных в ответах на запросы.
- Рецепт 6.7. Использование медиатипов для запросов данных.
- Рецепт 7.14. Оптимизация запросов с помощью сохраненных повторов.

6.7. Использование медиатипов для запросов данных

Реализация поддержки очереди данных в интерфейсе вашего сервиса может оказаться непростой задачей, так как зачастую нужно выбирать и раскрывать в рамках интерфейса одну технологию обработки данных: SQL, GraphQL, Lucene и т. д. Это идет вразрез с идеей создания слабосвязанных API, скрывающих внутренние детали реализации.

Одной из альтернатив станет создание собственного локального языка обработки данных, не привязанного к одной технологии. Это будет подразумевать не только разработку целого языка, но и его обслуживание, что также может привести к тесному связыванию. И хотя поначалу это кажется простым, при больших масштабах такой подход окажется непрактичным.

Настоящий рецепт предлагает способ отделения технологии обработки данных от интерфейса с сохранением преимуществ, присущих мощным языкам запросов данных.

Задача

Как лучше всего реализовать поддержку языка запроса данных в интерфейсе сервиса? Нужно ли выбирать один из существующих — SQL, GraphQL, Lucene и т. п.? Или же лучше изобрести собственный, который не будет привязан к одной технологии бэкенда? Как наладить поддержку более чем одного языка запросов в одном интерфейсе сервиса? Есть ли способ поддерживать популярные языки запросов, не привязывая API к одному сервису бэкенда? Что произойдет, когда этот сервис в дальнейшем изменится, например переедет с SQL на GraphQL?

Решение

Порой сложно найти баланс между желанием получить слабосвязанные реализации API и широкими возможностями известных языков запросов вроде SQL или GraphQL. Но есть простое решение, которое я считаю полезным, хотя сам встречал его применение довольно редко. Оно заключается в реализации поддержки языка запросов данных по HTTP в виде *медиати́па*. Я уже писал о потенциале использования зарегистрированных медиатипов (RMT) (см. рецепт 3.1) и семантических документов профилей (SPD) (см. рецепт 3.4), а также о преимуществе поддержки согласования содержимого (см. рецепт 5.6). Эти же принципы можно применить и к поддержке согласуемых языков запросов данных.

Первым шагом будет внедрение политики применения для языков запроса данных определений медиатипов. Это уже было проделано для формата SQL в спецификации RFC 6922 (<https://oreil.ly/MFkwv>). Данная спецификация документирует медиатип `application/sql` в качестве способа отправки SQL-запросов по HTTP (см. раздел «Пример»).

И хотя официально эта техника не зарегистрирована, я использовал ее для поддержки других языков запросов в своих API, включая:

- `application/prs.solr+json` для Solr (<https://oreil.ly/IE1CE>);
- `application/prs.odata+json` для OData (<https://oreil.ly/uiQ5e>);
- `application/prs.graphql+json` для GraphQL (<https://oreil.ly/3dN9f>).



На момент написания книги в реестре IANA нет зарегистрированных медиатипов для Solr, Odata и GraphQL. Я использую незарегистрированный идентификатор медиатипа, который содержит префикс `prs.` (означает личную регистрацию, <https://oreil.ly/YMMQQ>). Рекомендую следить за реестром медиатипов IANA, чтобы узнать, когда эти и другие языки официально зарегистрируют идентификаторы своих медиатипов.

Ключевым моментом этого рецепта является применение для отправки запроса данных тел HTTP-запросов. Например, клиент API может составить валидный запрос с помощью обозначенного языка (SQL, OData и т. д.). Это сообщение станет

телом HTTP-запроса POST или PUT (см. рецепт 5.15), отправленного интерфейсу сервиса. API получает запрос, затем по необходимости преобразует его в форму, пригодную для внутреннего механизма обработки данных, и выполняет запрос, возвращая результаты вызывающему.



Использование PUT/POST для отправки запросов данных означает, что можно легко сохранять и заменять одни и те же запросы в будущем. Как это сделать, показано в рецепте 7.14.

Преобразуя запросы данных в запросы медиатипов, вы также задействуете преимущества техники согласования HTTP-содержимого (см. рецепт 5.6). Можно включить поддерживаемые медиатипы запросов данных в метаданные сервиса (см. рецепт 5.10) и указать их строки в ответах на запросы клиентов, содержащие их предпочтения (см. рецепт 5.5).

Пример

Вот пример обмена сообщениями при отправке запроса данных с помощью SQL:

**** REQUEST ****

PUT /person/queries/q1w2e3r4 HTTP/1.1

Host: api.example.org

If-None-Match: *

Content-Type: application/sql

Accept: text/html

...

SELECT id,name,city FROM persons where city LIKE '%ville%'

**** RESPONSE ****

HTTP/1.1 301 Moved Permanently

Location: http://api.example.org/person/results/p0o9i8u7

**** REQUEST ****

GET person/queries/p0o9i8u7 HTTP/1.1

Accept: text/html

**** RESPONSE ****

HTTP/1.1 200 OK

Content-Type: text/html

ETag: "w/o9p0i8y6"

...

<html>

<title>Persons</title>

<body>

<div class=query-metadata>

success

SELECT id,name,city FROM persons where city LIKE '%ville%'


```

    </div>
    <div class="results">
        ...
    </div>
</body>
</html>

```

Обратите внимание на то, что запрос был отправлен с помощью типа содержимого `application/sql`, чтобы сообщить интерфейсу сервиса, какой механизм для него применить. Здесь мы также видим использование метаданных (см. рецепт 6.5) в ответе. Подробнее о создании ресурсов с помощью метода PUT, а не POST читайте в рецепте 5.15.

ДВА РЕСУРСА ЛУЧШЕ ОДНОГО

Во всех примерах этого рецепта запрос PUT, применяемый для создания нового ресурса (`/person/queries/q1w2e3r4`), создает ресурс *запроса данных*, а затем задействует его для создания ресурса *результатов данных* (`person/results/p0o9i8u7`). Это два отдельных ресурса. Один содержит запрос, а второй — результаты. Ресурс с результатами можно вызывать для возврата статичных результатов запроса данных. Ресурс запроса же можно использовать для повторного выполнения запроса в отношении источника данных и получения нового набора результатов (и, возможно, нового URL). Подробнее об этом читайте в рецепте 7.14.

Если интерфейс это поддерживает, тот же запрос можно создать с помощью Solr:

```

**** REQUEST ****
PUT /person/queries/q1w2e3r4 HTTP/1.1
Host: api.example.org
If-None-Match: *
Content-Type: application/prs.solr
Accept: text/html
...

```

```

fl=id name city
city:ville

```

Или с помощью OData:

```

**** REQUEST ****
PUT /person/queries/q1w2e3r4 HTTP/1.1
Host: api.example.org
If-None-Match: *
Content-Type: application/prs.odata
Accept: text/html
...

```

```

$select=id, name, city
$filter=contains(city,'ville')

```

В OData действуют собственные правила относительно создания запросов для отправки с помощью методов `POST` и `GET`. При этом нужно отметить, что в версии OData 4.1 правила сильно изменились. Я работаю с этим форматом (который чуть проще читать, чем текущую версию) и в зависимости от того, какая версия OData используется в бэкенде, переписываю запрос, чтобы он был с ней совместим. Следуя этому или аналогичному подходу, можно сохранить интерфейс сервиса стабильным, даже когда механизм запросов бэкенда со временем меняется.



Есть очень полезная статья Джона Гатого (John Gathogo, <https://oreil.ly/sBPaa>), в которой он рассказывает об использовании OData по HTTP.

В качестве еще одного примера приведу тот же запрос, составленный для механизма запросов GraphQL:

```
**** REQUEST ****
PUT /person/queries/q1w2e3r4 HTTP/1.1
Host: api.example.org
If-None-Match: *
Content-Type: application/prs.graphql
Accept: text/html
...

{
  person(city: {regex: "/ville/"}) {
    id
    name
    city
  }
}
```

Как и механизм запросов OData, GraphQL имеет собственные правила для отправки запросов данных с помощью HTTP-методов `POST` и `GET`. Рекомендую ознакомиться с документацией и при необходимости добавить этап переписывания для перевода сообщения `application/prs.graphql` в приемлемую для GraphQL форму.

Об этом рецепте нужно помнить то, что ваша цель — создать стабильный обособленный способ отправки запросов данных, опирающихся на определенную технологию.

Обсуждение

Передача запросов данных посредством медиатипа подразумевает инкапсуляцию технологии обработки данных в строго типизированный формат сообщений. Это делает возможной реализацию нескольких языков запросов данных для одного хранилища без внесения изменений в интерфейс. То есть вы можете реализовать

поддержку `application/sql` при первом релизе API и, если потребуется, безопасно добавить поддержку для Solr или GraphQL в будущем.

Стоит повторить, что в приведенном рецепте для создания ресурсов используется PUT, а не POST. Тем не менее внутреннему сервису может потребоваться применить POST в отношении, например, запроса GraphQL или OData. Или он может задействовать GET, к примеру, для Solr. Метод PUT служит только для нашего интерфейса сервиса и не зависит ни от какого HTTP-запроса, выполняемого к сервисам бэкенда.

Может показаться, что лучше просто напрямую следовать определяемым механизмом обработки данных соглашениям HTTP, а *не вводить* дополнительную реализацию медиатипа для поддержки запросов данных. Это нормально при условии, что вы не планируете изменять этот механизм в будущем или вносить критические изменения, которые будут в нем появляться. Несмотря на то что применение этого рецепта предполагает некоторые вложения авансом (определение строки медиатипа, установку конечной точки HTTP-методов PUT/POST, реализацию механизма переписывания и т. д.), он оправдывает себя в будущем.

Язык запросов SPARQL (<https://oreil.ly/Jp3Mr>) был создан для использования по HTTP. Поскольку в нем уже есть собственный набор связанных медиатипов, поддержка SPARQL отлично вписывается в этот рецепт.

Стоит отметить, что нет правила, которое бы требовало, чтобы язык запросов SQL применялся только к хранилищу на базе SQL. Можно без проблем реализовать поддержку SQL-запросов после преобразования всех SQL-баз данных в файловое хранилище. Ключом здесь будет рассмотрение языка запросов как отдельного слоя реализации интерфейса. Таким образом, если потребуется, вы можете изменить хранилище бэкенда, не изменяя интерфейс сервиса.

Постарайтесь не угодить в ловушку проектирования собственного языка запросов данных. Если только вы не планируете создавать уникальный механизм обработки данных, лучше использовать этот паттерн с медиатипом для налаживания интероперабельности между сервисами данных и языками запросов. Это обеспечит множество возможностей без внесения нового языка, на поддержку которого у вас может впоследствии не хватить ресурсов.

Читайте также

- Рецепт 3.1. Обеспечение интероперабельности с помощью зарегистрированных медиатипов.
- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.6. Управление форматами представления в среде выполнения.
- Рецепт 5.6. Поддержка согласования HTTP-содержимого.
- Рецепт 5.10. Публикация метаданных API.
- Рецепт 7.14. Оптимизация запросов с помощью сохраненных повторов.

6.8. Игнорирование неизвестных полей данных

Иногда в передаваемых сообщениях больше свойств данных, чем ожидалось или требуется. В таких случаях стоит просто проигнорировать эти дополнительные свойства и продолжить обрабатывать лишь интересующие вас части сообщения.

Задача

Бывают случаи, когда передаваемые между сервисами сообщения данных содержат больше свойств, чем нужно потребителю API. Например, сервис, спроектированный для управления адресами электронной почты, может общаться с сервисом данных, возвращающим полные адреса физической и электронной доставки. Как лучше всего получающему сервису обеспечить целостность данных, когда он считывает/записывает только те свойства, с которыми знаком? Возвращает ли в этом случае потребляющий сервис только измененные свойства или же все поля, включая те, которых не понимает?

Решение

При чтении и последующем обновлении записи данных из другого сервиса всегда нужно передавать *полную* запись, даже когда она содержит поля, которые вы не редактировали и/или не понимаете. Это актуально, когда вы обновляете и возвращаете запись сами, а также когда пересылаете ее другому сервису для обработки и поддержки обновлений от пересылающего сервиса.

Обычно нежелательно исключать данные из входящего сообщения, если вы (или какой-то другой сервис, из которого передаются данные) также поддерживаете *запись* этих данных обратно в источник. Например, попытка записать частичные значения в хранилище данных может привести к ситуациям, когда эти обновленные значения окажутся несовместимыми с неотредактированными. В результате сохраненные данные утрачивают целостность или запись отклоняется. Оба исхода снижают надежность сервиса.

В сердце текущего решения лежит так называемое правило «просто игнорируй». Суть в том, чтобы при получении сообщения игнорировать те его части, которых вы не понимаете. Работайте с теми, которые ваш сервис может понять, и в случае обновления возвращайте весь блок данных.

Пример

Рассмотрим случай, показанный на рис. 6.4, где создается интерфейс, поддерживающий чтение/запись адресов электронной почты пользователей (`emailUpdater`).

Оказывается, ваш интерфейс получает данные от другого сервиса (`addressManager`), который содержит массу информации о зарегистрированных за пользователями адресах почты и реальной доставки (рис. 6.5).

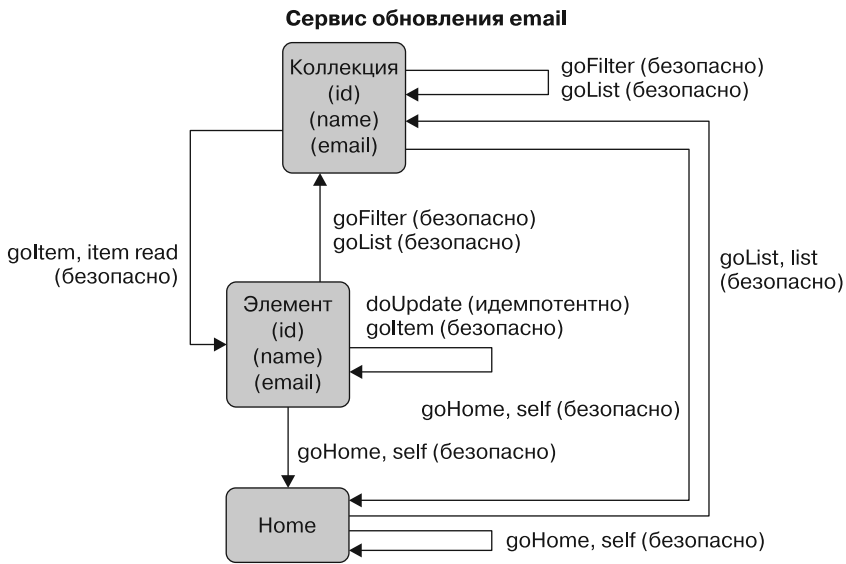


Рис. 6.4. Механизм обновления email

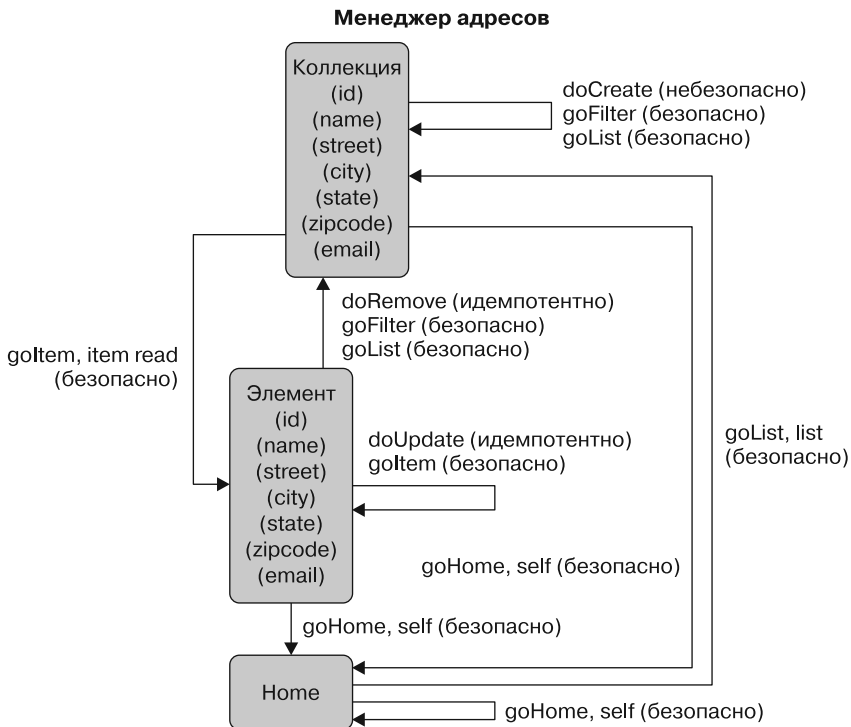


Рис. 6.5. Документ ALPS для API менеджера адресов

При запросе `emailUpdater` получает от вызывающего API идентификатор записи (GET `http://api.emailupdater.org/q1w2e3r4`) и использует его для вызова `addressManager`, который возвращает следующее сообщение:

```
**** REQUEST ****
GET /q1w2e3r4 HTTP/1.1
Host: api.addressManager.org
Accept: application/vnd.collection+json
...

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Content-Length: XX
ETag: "w\u7y6t5r4"

{"collection": {
  "title": "Address Manager",
  "links": [...],
  "items": [
    {"href": "http://api.addressmanager.org/q1w2e3r4",
      "rel": "address",
      "data": [
        {"name": "id", "value": "q1w2e3r4"},
        {"name": "name", "value": "Mork Mickelson"},
        {"name": "street", "value": "123 Main"},
        {"name": "city", "value": "Byteville"},
        {"name": "state", "value": "MD"},
        {"name": "zipCode", "value": "12345"},
        {"name": "email", "value": "mork@example.org"}
      ]
    }
  ],
  "template": {...}
}}
```

Далее `emailUpdater` изменяет электронную почту в соответствии с запросом и возвращает *полную* запись адреса в `addressManager`:

```
**** REQUEST ****
PUT /q1w2e3r4 HTTP/1.1
Host: api.addressManager.org
Accept: application/vnd.collection+json
Content-Type: application/vnd.collection+json
If-Match: "w\u7y6t5r4"
...

{"template": {
  "data": [
    {"name": "id", "value": "q1w2e3r4"},

```



```

    {"name": "name", "value": "Mork Mickelson"},
    {"name": "street", "value": "123 Main"},
    {"name": "city", "value": "Byteville"},
    {"name": "state", "value": "MD"},
    {"name": "zipCode", "value": "12345"},
    {"name": "email", "value": "mickle@example.org"}
  ]
}

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Content-Length: XX
ETag: "w\i8u7y6t5"

{"collection": {
  "title" : "Address Manager",
  "links" : [...],
  "items" : [
    {"href": "http://api.addressmanager.org/q1w2e3r4",
     "rel" : "address",
     "data" : [
       {"name": "id", "value": "q1w2e3r4"},
       {"name": "name", "value": "Mork Mickelson"},
       {"name": "street", "value": "123 Main"},
       {"name": "city", "value": "Byteville"},
       {"name": "state", "value": "MD"},
       {"name": "zipCode", "value": "12345"},
       {"name": "email", "value": "mickle@example.org"}
     ]
    }
  ],
  "template": {...}
}}
```

Наконец, представим, что когда-то в будущем `addressManager` добавит в свои сообщения новые свойства данных, например `telephone` и `sms`. В этом случае `emailUpdater` продолжит выполнять ту же задачу по изменению одного известного ему поля и просто передавать обратно *все* остальные. Таким образом, скрытый за интерфейсом `emailUpdater` сервис не придется изменять при каждом добавлении нового поля сервисом `addressManager`.

Обсуждение

Игнорирование элементов сообщения — это удобный способ повышения устойчивости сервисов и API, которые взаимодействуют с другими интерфейсами. Работая с интерфейсами сервисов, которые не контролируете, вы не можете быть уверены в том, что происходит за кадром. Вам известно лишь то, что обещает интерфейс.

Возможно, со временем этот интерфейс изменится, и если при этом он сохранит обещания, на которые вы полагаетесь, то добавление/удаление полей или действий вне области ваших потребностей не будет играть для вас роли.

ПРАВИЛО «ПРОСТО ИГНОРИРУЙ»

Этот паттерн передачи любых непонятных полей вместо их удаления является ключевым элементом при поддержке будущей совместимости сервисов и интерфейсов. Сама веб-среда тоже создавалась с учетом этого. Хорошее описание правила «просто игнорируй» можно найти в документе *Extending and Versioning Languages Part 1* на сайте W3C (<https://oreil.ly/cLnAV>).

Этот рецепт зависит от возможности сервиса распознавать и использовать в ответах гипермедиаформы. Как и все интерфейсы сервисов, сервисы данных должны возвращать сообщения с элементами гипермедиа (см. рецепты 3.5 и 4.9), включающие инструкцию о том, как вызывающий может выполнить дополнительные действия. Если вы применяете для возвращения формат без поддержки гипермедиа (HAL, CSV, простой JSON и т. п.), тогда нужно возвращать указатель на другой документ, который содержит описания действий, например HAL-FORMS, XML/JSON Schema, ALPS и т. д.

Этот рецепт тесно связан с рецептом 6.13, посвященным посредникам сервисов. Фактически его можно рассматривать как рекомендацию для потребителей API, которые могут взаимодействовать с интерфейсом, выступающим в качестве посредника для передачи данных. Естественно, ваш сервис никогда реально не *узнает*, общается ли он с посредником. Ему будет известно лишь то, что он взаимодействует с интерфейсом сервиса, который выполняет обещанные действия.

Читайте также

- Рецепт 3.3. Предоставление информации о предметной области через публикуемые словари.
- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.11. Проверка свойств данных в среде выполнения.
- Рецепт 4.13. Использование запросов документов для проверки входящих сообщений.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 6.5. Возвращение метаданных в ответах на запросы.
- Рецепт 6.13. Использование промежуточных сервисов для обмена данными.

6.9. Повышение производительности с помощью директив кэширования

В протокол HTTP встроена богатая модель кэширования. Сервисам обработки данных она позволяет без особых усилий повысить производительность и снизить задержку.

Задача

Когда сервис зависит от удаленных источников данных, это ведет к снижению его быстродействия из-за медлительных сетей и их сбоев. Как повысить быстродействие своего сервиса, даже когда он зависит от удаленных сервисов данных, которые вы не контролируете? Как использовать модель кэширования HTTP на стороне сервиса и клиента для повышения надежности и доступности сервисов в среде выполнения?

Решение

Когда интерфейс вашего сервиса для получения данных опирается на другие, удаленные сервисы, для повышения его быстродействия проще всего задействовать предлагаемые HTTP модели кэширования. Это означает, что поставщики сервиса должны отмечать все ответы с помощью метаданных кэширования, чтобы потребители могли понимать, какова длительность жизни ответа. Кроме того, это означает, что потребители сервисов должны проверять и учитывать метаданные кэширования для повышения качества ответов API.

ДЕРЖИТЕ ДАННЫЕ РЯДОМ

Кэши можно хранить на машине поставщика, сторонней машине сети доставки содержимого (Content Delivery Network, CDN) или даже машине потребителя API. Чем ближе кэш к клиенту, тем выше скорость вашего сервиса.



Чтобы лучше разобраться в предлагаемой HTTP модели кэширования, почитайте соответствующую спецификацию в документе RFC 7234 (<https://oreil.ly/E6Z6E>).

В этом рецепте я расскажу только о метаданных модели кэширования для HTTP 1.1 и выше. Директивы кэширования также разбросаны по коллекции

HTTP-заголовков, которая используется для сервисов HTTP 1.0: `Age`, `Expires` и `Pragma`. Подробнее читайте в спецификации HTTP.



Весь список директив кэширования для HTTP можно найти в их реестре (<https://oreil.ly/qk3ab>).

Метаданные кэширования ответа для поставщиков сервисов

Поставщики сервисов могут дополнять свои ответы метаданными кэширования. Как правило, это означает указание области кэширования (`public` или `private`), максимальную длительность хранения ответа (`max-age`) и способ обработки ответов (`no-store`, `no-cache`, `must-revalidate` и т. д.). Это прекрасно работает для ответов, которые изменяются редко, — например, статических списков поиска или письменной документации, которая корректируется нечасто.



Если вы потребляете API сервиса данных, который не использует директивы кэширования, попробуйте убедить его разработчиков все же добавить в ответы такую функциональность. Это окажется особенно эффективным, если вы находитесь в корпоративной среде компании, где сервисы данных разрабатываются и поддерживаются другой командой.

В случаях, когда ответ содержит информацию, которая может меняться часто, рекомендуется применять директивы, которые укажут потребителям API на то, что перед повтором кэшированных ответов нужно свериться с исходным сервером. Поставщики интерфейсов сервисов могут возвращать `ETag` вместе с `Cache-Control: must-revalidate`, сообщая потребителям API, что они должны создавать условные запросы (`GET` или `PUT/DELETE`), возвращая значение `ETag` в заголовке `If-Match`.

ИММУТАБЕЛЬНОЕ КЭШИРОВАНИЕ

Директива кэширования под названием `immutable` помогает потребителям API лучше понять, что ответ имеет продолжительный срок действия. Обычно это используется для подресурсов наподобие сопровождающих новостные тексты фото. Сервисы данных могут применять эту директиву, возвращая долгоживущие ответы вроде статических списков, изображений товаров и т. д. Подробнее о ней можно почитать в документе RFC 8246 (<https://oreil.ly/Nw1k9>).

Этот паттерн также отлично подходит, если поставщики сервисов ожидают, что потребители API будут отправлять обновления (`PUT/POST/PATCH/DELETE`) для одного и того же ресурса, и хотят исключить выполнение этого действия, если его версия на сервере уже была обновлена.

Кэширование метаданных для потребителей API

При отправке запросов в интерфейс сервиса потребители API могут использовать метаданные кэширования для определения типа ответа, который потребитель готов принять. Например, включение в запрос API директивы `Cache-Control: max-age=600, min-fresh=300` сообщит поставщику о желании потребителя обеспечить, чтобы ответ был не старше десяти минут (`max-age=600`) и имел не менее пяти минут жизни в запасе (`min-fresh=300`).

Подобные директивы кэширования актуальны, когда потребитель API хочет получить ответ, который можно будет какое-то время хранить, уменьшив текущую частоту обновлений на стороне клиента.

Потребители API также могут использовать в запросе директиву `Cache-Control: no-cache`, чтобы заставить интерфейс сервиса отправлять свежий ответ. Это происходит, когда потребитель API хочет получить последнюю версию представления ресурса, чтобы, например, его отредактировать.

Клиенты также могут применять директивы `Cache-Control: max-stale, stale-if-error`, сообщая интерфейсам сервисов, что можно вернуть несвежую копию ресурса, если доступна только она. Это хорошо работает, когда потребитель API хочет выполнить запрос, подразумевающий только чтение (без редактирования), и предпочтет получить старый ответ вместо кода состояния 4xx или 5xx.

Пример

Далее приведены примеры применения метаданных кэширования к ответам поставщика и запросам клиента. Среди них также есть расширенный пример, разбирающий сценарий, в котором клиент и сервер используют метаданные кэширования для повышения отзывчивости и качества взаимодействия сообщениями.

Пример метаданных кэширования в ответе поставщика

Вот простой ответ с добавлением метаданных кэширования:

```
**** REQUEST ****
GET /provinces/list HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json
...

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Content-Length: NN
Date: Tue, 15 Nov 2022 08:12:31 GMT
Cache-Control: public, max-age=600
...
```

Указанные метаданные кэширования сообщают потребителям API, что ответ может быть кэширован любым прокси (**public**) и его можно хранить (для последующего воспроизведения) вплоть до десяти минут после получения.

Пример далее показывает, как интерфейс сервиса может предоставить метаданные кэширования, которые с помощью заголовка **ETag** и директивы **must-revalidate** инструктируют потребителей API о необходимости создания условных запросов (**GET** или **PUT/DELETE**):

```
GET /user/q1w2e3r4 HTTP/1.1
Host: api.example.org
Accept: application/vnd.siren+json
...

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.siren+json
Content-Length: NN
ETag: "w/p0o9i8u7y6t5"
Date: Tue, 15 Apr 2022 11:12:13 GMT
Cache-Control: public, max-age=300, must-revalidate, stale-if-error
...
```

В этом случае потребителям API сообщается, что они могут хранить текущий ответ в течение пяти минут. Кроме того, если эту кэшированную копию запросит потребитель API, то сначала ему нужно будет свериться с сервером бэкенда для проверки того, совпадает ли версия на сервере с версией в кэше (при помощи заголовка **ETag**). Обратите внимание на использование директивы **stale-if-error**, которая сообщает держателю кэша, что в случае невыполнения запроса на валидацию (например, из-за ошибки сети) можно вернуть эту копию ответа, даже если к этому моменту период ее «свежести» истечет.

Метаданные кэширования в запросе потребителя

Далее показан пример запроса API, в котором потребитель хочет обеспечить, чтобы ответ был не старше десяти минут (**max-age=600**) и имел в запасе не менее пяти минут жизни (**min-fresh=300**):

```
**** REQUEST ****
GET /users/list HTTP/1.1
Host: api.example.org
Accept: application/vnd.hal+json
Cache-Control: max-age=600, min-fresh=300
```

Потребители API также могут использовать в запросе директивы кэширования, чтобы принудить интерфейсы сервисов отправлять свежий ответ:

```
**** REQUEST ****
GET /users/q1w2e3r4 HTTP/1.1
Host: api.example.org
Accept: application/vnd.hal+json
Cache-Control: no-cache
```

Вот пример, в котором потребитель API сообщает интерфейсу сервиса, что готов принять устаревшую копию ресурса, если на данный момент доступна только она:

```
**** REQUEST ****
GET /users/q1w2e3r4 HTTP/1.1
Host: api.example.org
Accept: application/vnd.hal+json
Cache-Control: max-stale, stale-if-error
```

Расширенный пример метаданных кэширования

Когда интерфейсы сервисов возвращают в HTTP-ответах метаданные кэширования, важно, чтобы потребитель API учитывал эти директивы и использовал их должным образом. Например, поставщики API могут указывать, что ответ можно локально кэшировать в течение десяти минут после получения (`Cache-Control: max-age=600`). Когда это актуально, потребитель API должен сохранять копию ответа и при необходимости воспроизводить его в установленный промежуток времени.

Рассмотрим случай, когда один сервис отслеживает данные `customer`, а другой — данные `order`. Ваш сервис возвращает совокупность этих данных в виде сервиса только для чтения `customer-order`. Как потребитель API `customer` и `order`, вы должны учесть возвращаемые этими сервисами метаданные кэширования.

Вот запрос одной записи из сервиса `customer`:

```
**** REQUEST ****
GET /customer/e3r4t5y6 HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json
Cache-control: no-cache

**** RESPONSE ****
HTTP/1.1 200 OK
Host: api.example.org
Content-Type: application/vnd.collection+json
Cache-Control: private, max-age=600, must-revalidate
ETag: "w/i8u7y6t5r4er3"
```

А вот запрос соответствующего содержимого из сервиса `orders`:

```
**** REQUEST ****
GET /orders/filter?customer=e3r4t5y6 HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json
Cache-control: max-stale

**** RESPONSE ****
HTTP/1.1 200 OK
Host: api.example.org
Content-Type: application/vnd.collection+json
Cache-Control: private, max-age=1800
ETag: "w/u7y6t5r4e3w2"
```

Заметьте, что в первом случае у сервиса `customer` запрашивается свежая копия ресурса (`Cache-Control: no-cache`). А в случае списка связанных записей (сервис `order`) запрос гласит, что потребитель готов принять устаревший ответ, если доступен только он (`Cache-Control: max-stale`). Это типичная схема. Отдельные записи (которые можно обновить) зачастую должны быть свежими копиями, а для связанных списков допускается меньшая свежесть. Тем не менее, когда нужен один ресурс из сервиса `order`, для его получения мудро будет отметить запрос директивой `Cache-Control: no-cache`.

Обсуждение

Будет хорошей практикой для всех сервисов сопровождать ответы метаданными кэширования. Это особенно важно в случае сервисов данных, когда ответы (по сути, записи данных) наверняка будут часто использоваться повторно.

Скорость ожидаемого изменения, например частота изменения записи, будет хорошим ориентиром для установки длительности кэширования ресурса. Если внутренние данные изменяются редко, как, например, список штатов или провинций, названий улиц и т. д., то их период кэширования может быть относительно длинным и составлять часы или дни. Если же данные более изменчивы, как, скажем, содержимое корзины покупателя, тогда продолжительность кэширования должна быть очень короткой, возможно, всего несколько секунд.

Директиву `no-cache` клиенты API должны задействовать осторожно. По сути, она исключает кэш, что может дать дополнительную нагрузку на интерфейсы сервисов. Если клиент ожидает *редактирования* ресурса, разумно использовать директиву `no-cache`, чтобы получить последнюю копию данных. В противном случае он должен просто позволить поставщику отправить предустановленный ответ (кэшированный, если такой доступен).

ЗАГЛОВОК VARY

В этом рецепте не был затронут важный, связанный с кэшированием заголовок `Vary` (<https://oreil.ly/vZtk4>). С его помощью можно указать, какие HTTP-элементы, помимо URL-адреса, хоста и метода, должны учитываться при кэшировании ответа. Например, `vary: Authorization` указывает, что ответ можно повторить, только если значения `Authorization` в последующих запросах совпадают. Еще один распространенный вариант `Vary` включает `accept-language`, `content-type` и другие согласуемые элементы.

Поставщики должны учитывать все это при сопровождении ответа директивами `Cache-Control`, чтобы не допустить повтора ошибочного представления.

В случае долгоживущих данных (статические списки, стабильные документы и т. п.) потребители API могут использовать кэширование для создания

собственной локальной копии регулярно применяемых записей. Это своеобразный паттерн приблизительной репликации. Подробнее читайте в рецепте 7.14.

Читайте также

- Рецепт 4.15. Сохранение состояния.
- Рецепт 5.14. Повышение скорости обработки с помощью клиентских идентификаторов.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 7.14. Оптимизация запросов с помощью сохраненных повторов.

6.10. Изменение моделей данных в промышленной среде

Однажды вам может потребоваться обновить модель данных, которую ваш сервис использует для создания ресурсов. В таких случаях нужно придумать способ ее изменения без вреда для существующих потребителей.

Задача

Как можно безопасно обновить модель данных сервиса после его релиза в промышленную среду? Какие детали модели можно изменить без вреда для потребителей API? Что произойдет, если вы обновите модель в новом релизе, а затем понадобится откатить это изменение?

Решение

Наиболее эффективным подходом для реализации изменений данных в промышленной среде является изначальное проектирование их хранилища с поддержкой такой возможности. Это означает, что изменения в итоге не «прикручиваются», а фактически оказываются частью изначального проекта.

Хорошим способом добавления возможности изменения моделей в хранилище данных будет использование для моделирования двухуровневого подхода, основанного на *явных и неявных свойствах данных*. Явные поля — это часть модели или схемы (например, `{ "familyName" : "Quarkus" }`), а неявные выступают как часть связанной с моделью коллекции имен — значений (например, `{ "nvp": [{ "name": "familyName", "value": "Quarkus" }] }`). При создании свойства в формате массива или коллекции, которое может содержать произвольное число элементов имен и значений, можно встроить возможность добавления в модель данных новых свойств без изменения самой модели или схемы.

Ключевым преимуществом этого подхода является то, что новые свойства можно добавлять в коллекцию имен — значений без обновления схемы для всех потребителей API. Это также означает, что если вы выпустите новую версию API сервиса, включающую дополнительные поля данных, а затем потребуется откатить этот релиз, то модель данных менять не придется и вы не потеряете все данные, собранные в новых полях пар имен — значений. Затем, когда вы исправите все баги и создадите новый релиз, сможете задействовать ранее собранные данные. По сути, вы создаете паттерн моделирования данных, обеспечивающий прямую и обратную совместимость.

Пример

Как правило, локальные хранилища данных моделируются как строго типизированный объект данных. Например, вот объект хранилища `person`:

```
{
  "givenName": "John",
  "familyName": "Doe",
  "age": 21
}
```

Сообщение `person` можно определить с помощью следующего документа схемы:

```
{
  "$id": "https://api.example.org/person.schema.json",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Person",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "givenName": {
      "type": "string",
      "description": "The person's first name."
    },
    "familyName": {
      "type": "string",
      "description": "The person's last name."
    },
    "age": {
      "description": "Age in years. Must be equal to or greater than zero.",
      "type": "integer",
      "minimum": 0
    }
  }
}
```



Во всех примерах JSON Schema, используемых в этом паттерне, значение `additionalProperties` установлено на `false`. То же по умолчанию касается и схем XML.

Добавление нового свойства

Теперь предположим, что мы хотим добавить в объект `person` новое свойство — `middleName`. Если добавим в схему новое поле, то столкнемся с риском нарушить работу существующих потребителей API. Нам нужно обеспечить выпуск обновлений для всех потребляющих сервисов одновременно. Это будет возможно для тех сервисов, в которых всех потребителей и поставщиков API обслуживает одна команда (или две тесно сотрудничающие команды), но даже такое решение реализовать непросто. Вместо этого для добавления нового свойства в объект `person` мы используем паттерн с парами «имя — значение»:

```
{
  "givenName": "John",
  "familyName": "Doe",
  "age": 23,
  "nvp" : [
    {"name" : "middleName", "value" : "Seymore"}
  ]
}
```

А вот новая схема для этой модели:

```
{
  "$id": "https://example.com/person.schema.json",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Person",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "givenName": {
      "type": "string",
      "description": "The person's first name."
    },
    "familyName": {
      "type": "string",
      "description": "The person's last name."
    },
    "age": {
      "description": "Age in years which must be equal to or greater than zero.",
      "type": "integer",
      "minimum": 0
    },
    "nvp" : {
      "type" : "array",
      "items" : { "$ref": "#/$defs/nvp" },
      "description" : "List of name/value pairs.",
    }
  },
  "$defs": {
    "nvp": {
      "type": "object",
      "required": [ "name", "value" ],

```

```

    "properties": {
      "name": {
        "type": "string",
        "description": "The name of the property."
      },
      "value": {
        "type": ["number", "string", "boolean", "object", "array", "null"],
        "description": "The value of the property."
      }
    }
  }
}
}
}

```

Теперь вместо строго типизированной схемы для своей модели данных используем умеренно типизированный подход. Любые свойства, которые мы хотим добавить в модель, можно заменить в массиве `prv`, и итоговый объект все равно будет проходить валидацию на соответствие опубликованной схеме.

Универсальная функция

Двухуровневый подход может несколько усложнить для сервисов обнаружение нужного свойства. В этом случае вопрос стоит так: «Я ищу поле `middleName`. Оно явное или неявное?» Вы можете скрыть двойственную природу модели данных с помощью специальной функции, которая будет выполнять автоматический поиск в двух местах. Вот пример ее реализации на JavaScript:

```

// Вернуть одно свойство, неважно, явное или неявное
// args = {name:n,message:m,nameValuePair:p}
function find(args) {
  var a = args || {};
  var n = a.name || "";
  var m = (a.message || local.m) || {};
  var p = (a.nameValuePair || local.p) || "nvp";
  var r = undefined;

  if(m==={} || n=="") {
    r=undefined;
  }
  else {
    if(m.hasOwnProperty(n)) {
      r=m[n];
    }
    else {
      if(m.hasOwnProperty(p)) {
        try {
          r = m[p].filter(function(i) {return i.name===n})[0].value;
        }
        catch {
          r = undefined;
        }
      }
    }
  }
}

```

```

    }
  }
  return r;
}

```

Теперь можно использовать один и тот же вызов функции вне зависимости от того, явное свойство или неявное:

```

console.log(find({name:"givenName", message:person}));
console.log(find({name:"middleName", message:person}));

```

Эта неявная модель подходит и для сложных значений:

```

{
  "givenName": "John",
  "familyName": "Doe",
  "age": 21,
  "nvp" : [
    {"name" : "hatsize", "value" : null},
    {"name" : "middleName", "value" : "Seymore"},
    {"name" : "nicknames", "value" : ["J", "JJ", "Johnboy", "Jack"]},
    {"name" : "address", "value": {"street": "123 main", "city": "Byteville",
      "state": "MD", "zip": "12345"}}
  ]
};

```

Поддержка нескольких версий схемы

У этого паттерна есть еще одно преимущество при использовании универсальной функции, как было показано ранее. Поскольку теперь неважно, где в сообщении находится свойство, одно и то же клиентское приложение можно запрограммировать на поддержку нескольких экземпляров схем сообщения `person`.

Например, ваш сервис может обновлять модель данных, делая поле `middleName` явным:

```

{
  "givenName": "John",
  "middleName": "Seymore",
  "familyName": "Doe",
  "age": 21
};

```

При этом та же функция `find` будет работать ожидаемым образом:

```

console.log(find({name:"middleName", message:person}));

```

Откат модели

Наконец, в случаях, когда вам нужно вернуться к прежней модели/схеме, можно переместить явные свойства в одну модель и выразить их как неявные в другой с минимальными побочными эффектами.

Обсуждение

Этот паттерн создан в первую очередь с целью упростить для сервисов безопасное изменение внутренних моделей данных. Не рекомендуется раскрывать эти модели напрямую через API сервиса. Вместо этого следует представлять ресурсы с помощью грамотно определенного медиатипа (подробнее — в рецепте 5.3).

Может показаться хорошей идеей просто добавлять новые явные поля в объекты JSON и использовать штатное поведение схем JSON, заключающееся в игнорировании дополнительных свойств при валидации объектов. Однако такой подход создаст проблемы. Во-первых, поведение JSON Schema и XML Schema различается. По умолчанию XML отклоняет сообщения с неизвестными элементами. Во-вторых, опасно предполагать, что значение `additionalProperties` в схемах JSON всегда будет установлено как `true`. Команда разработчиков некоторых потребителей данных может написать собственную схему JSON, которая установит это значение на `true`, и начать отклонять сообщения с неизвестными свойствами. Наконец, внедряя модель «явные/неявные», вы позволяете потребителям API понять, что были добавлены некоторые поля. Это дает им возможность проигнорировать такие поля и предоставляет понятный путь для обнаружения новых свойств в будущем.

Если в итоге вы будете поддерживать в среде выполнения несколько схем моделей (например, некоторые ранее неявные свойства будут преобразованы в явные), обязательно укажите ссылку на корректную ревизию схем JSON или XML, поскольку клиентские приложения могут опираться на эти схемы при выполнении (см. рецепт 4.7).

Текущий рецепт не объясняет, как *сохранять* эти явные и неявные свойства данных, поскольку детали будут меняться в зависимости от используемого формата данных. Документ JSON можно структурировать, чтобы он соответствовал предоставленным примерам объектов. Однако в случаях, когда данные хранятся с помощью таблиц SQL, можно создать таблицу для размещения явных полей и затем поместить неявные свойства в отдельную таблицу (`nameValuePairs`), где каждая строка будет содержать поле индекса, указывающее на явную строку, а также имя и значение (рис. 6.6).



Получение неизвестных неявных полей может нести риски для безопасности. Важно, чтобы все сервисы, которые попытаются считать/записать данные, проверяли содержимое имен и значений, чтобы оградить себя от любой сторонней попытки поместить в хранилище данных опасные значения.

Нужно помнить, что сервисы, которые поддерживают запись данных с помощью этого рецепта, должны знать, какие поля являются явными, а какие — неявными.

Простым решением здесь будет использование списка явных полей в качестве ориентира при внесении записи. Если вдруг появится поле, которое в нем не перечислено, можно предположить, что это неявное поле, которое нужно добавить в коллекцию `nvp`. У вас также может быть возможность задействовать для направления операции записи локальный документ схемы.

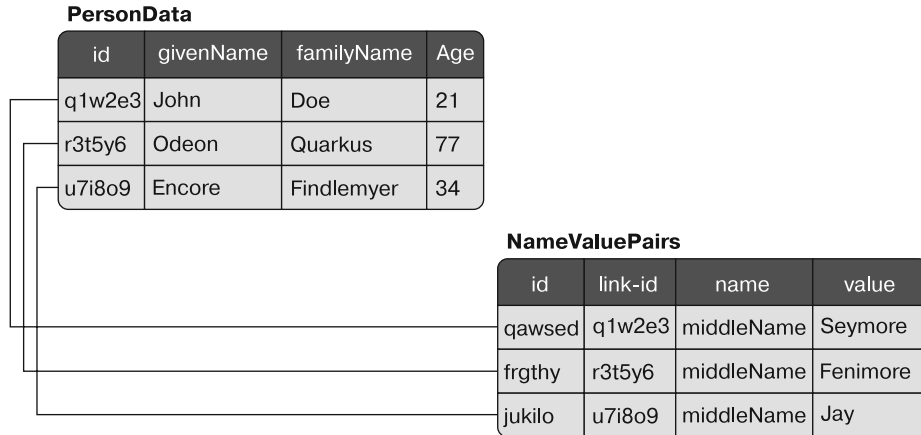


Рис. 6.6. Явная и неявная таблицы

Читайте также

- Рецепт 3.10. Проектирование с учетом возможности расширения сообщений.
- Рецепт 3.11. Проектирование с учетом возможности изменения интерфейсов.
- Рецепт 4.7. Использование документов схем в качестве источника метаданных сообщений.
- Рецепт 4.14. Валидация входящих данных.
- Рецепт 5.2. Предотвращение утечек внутренних моделей.
- Рецепт 6.1. Соккрытие внутренних деталей хранилища данных.
- Рецепт 6.8. Игнорирование неизвестных полей данных.

6.11. Расширение удаленных хранилищ данных

Бывают случаи, когда вы хотите использовать данные из другого сервиса, несмотря на то что в нем содержатся не все нужные свойства. В подобных ситуациях можно рассмотреть *расширение* этого удаленного источника данных собственным локальным хранилищем свойств с помощью ассоциативного ключа.

Задача

Как безопасно и эффективно расширить существующее хранилище данных, которое вы не контролируете, дополнительными значениями их свойств? Когда это будет хорошей идеей? Как поддерживать целостность данных между удаленным и локальным хранилищами?

Решение

Когда вы хотите расширить существующее удаленное хранилище данных дополнительными значениями, можете организовать локальное хранилище, которое будет содержать дополнительные значения и ассоциативный ключ, связывающий эти два хранилища.

Пример

Для начала реализуем пример удаленного и локального хранилищ данных, после чего наладим их связь.

Предположим, вы используете удаленное хранилище, которое отслеживает `user Accounts`. В нем есть свойство `uniqueId` и коллекция других свойств, таких как `givenName`, `familyName`, `email` и пр.:

```
**** REQUEST ****
GET /users/q1w2e3r4 HTTP/1.1
Host: user-accounts.example.org
Accept: application/vnd.collection+json
...

**** RESPONSE ****
HTTP/1.1. 200 OK
Host : user-accounts.example.org
Content-Type: application/vnd.collection+json
Content-Length: XXX

{ "collection" :
  {
    "version" : "1.0",
    "href" : "http://user-accounts.example.org/users/q1w2e3r4",

    "links" : [...],
    "items" : [
      {
        "href" : "http://user-accounts.example.org/users/q1w2e3r4",
        "data" : [
          { "name": "uniqueId", "value": "q1w2e3r4" },
          { "name": "givenName", "value": "Marquis" },
```



```

        {"name": "familyName", "value": "Quarkus"},
        {"name": "email", "value": "user@example.org"}
    ]
}
]
}
}

```

Предположим также, что вы создаете сервис, который отслеживает историю запросов от пользователей вашего торгового сайта. Вы хотите отслеживать такие элементы, как `visitorId`, `pageUrl`, `dateTime`, `dwelTime` и пр.:

```

**** REQUEST ****
GET /history/p0o9i8u7/1 HTTP/1.1
Host: shopping.example.org
Accept: application/vnd.collection+json
...

**** RESPONSE ****
HTTP/1.1. 200 OK
Host : shopping.example.org
Content-Type: application/vnd.collection+json
Content-Length: XXX

{ "collection" :
  {
    "version" : "1.0",
    "href" : "http://shopping.example.org/history/p0o9i8u7/1",

    "links" : [...],
    "items" : [
      {
        "href" : "http://shopping.example.org/history/p0o9i8u7/1",
        "data" : [
          {"name": "historyId", "value": "p0o9i8u7"},
          {"name": "pagerUrl", "value": "..."},
          {"name": "dwelTime", "value": "30000ms"},
          {"name": "dateTime", "value": "20230203T141529Z"}
        ]
      }
    ]
  }
}
}

```

Наконец, вам нужна возможность поделиться этой историей запросов с пользователями по их требованию. По сути, нужно совместить свойства данных `userAccount` и `requestHistory`.

Прямым способом для этого будут применение в качестве ассоциативного ключа URL-адреса среды выполнения, принадлежащего удаленному хранилищу данных

(<http://user-accounts.example.org/users/q1w2e3r4>), и добавление этого ключа в локальную запись:

```
**** REQUEST ****
GET /history/p0o9i8u7/1 HTTP/1.1
Host: shopping.example.org
Accept: application/vnd.collection+json
...

**** RESPONSE ****
HTTP/1.1. 200 OK
Host : shopping.example.org
Content-Type: application/vnd.collection+json
Content-Length: XXX

{ "collection" :
  {
    "version" : "1.0",
    "href" : "http://shopping.example.org/history/p0o9i8u7/1",

    "links" : [...],
    "items" : [
      {
        "href" : "http://shopping.example.org/history/p0o9i8u7/1",
        "data" : [
          {"name": "historyId", "value": "p0o9i8u7"},
          {"name": "pagerUrl", "value": "..."},
          {"name": "dwellTime", "value": "30000ms"},
          {"name": "dateTime", "value": "20230203T141529Z"},
          {"name": "associativeKey",
            "value": "http://user-accounts.example.org/users/q1w2e3r4"}
        ]
      }
    ]
  }
}
```

Теперь, когда вы решите создать ресурс, включающий локально хранящиеся данные и поля из удаленного источника, для получения нужных данных потребуется просто использовать **associativeKey** в локальном хранилище.

Обсуждение

Может оказаться, что удаленный сервис не будет отвечать на вашу попытку получить связанный ресурс. В таком случае следует рассмотреть использование локальной копии (см. рецепт 6.9).

Возможно также, что удаленный сервис работает, но не может найти запрашиваемый ресурс. Например, он мог быть удален. В таком случае сервис должен либо предоставить взамен предустановленные данные, либо вернуть ответ 4xx,

сообщающий вызывающему приложению, что информация теперь недоступна. Если при отсутствии удаленного ресурса связанные с ним локальные данные теряют смысл, можете их удалить.



Возврат сервисом кода 404 еще не означает, что данные были удалены. Они могут просто временно отсутствовать. Не спешите удалять локальные данные, если удаленный сервис присылает ответ 4xx. Если же он отвечает сообщением `410 Gone`, то можете быть уверены, что это состояние постоянно.

Нежелательно создавать свойства локальных данных с теми же именами, что и у свойств в удаленном хранилище. Например, если удаленный ресурс содержит поле `email`, в котором указан личный электронный адрес пользователя (`marquis@example.org`), и вы хотите сохранить вместо него рабочий адрес (`work@example.org`), не нужно создавать локальное свойство `email` с требуемым значением. Вместо этого создайте локальное свойство с уникальным именем (`workEmail`) и используйте его для хранения нужного значения.

Даже если локальный сервис выводит один ресурс, представляющий комбинацию локальных и удаленных свойств, нежелательно пытаться обеспечить для этой общей записи целостность данных. Например, вы можете захотеть внедрить правило, согласно которому любой локальный ресурс, содержащий свойство `email`, *должен* содержать свойство `fullName`. Если одно из этих свойств является частью удаленного ресурса (`email`), а другое — частью локального (`fullName`), вы не сможете быть уверенными в том, что обе записи будут применять одни и те же правила.

А КАК ЖЕ МОЯ СХЕМА?!

В случаях, когда интерфейсы сервисов возвращают ответы, полученные из нескольких источников, применение схем для валидации сообщений начинает вызывать сложности, особенно при использовании сервиса хранилища с поддержкой выбора полей, например OData, GraphQL и т. д. Подробнее о применении схем для API читайте в рецептах 4.12 и 4.13.

Редко стоит использовать локальный ресурс для хранения копий значений свойств удаленного ресурса. Значения удаленных свойств могут меняться, а ваш локальный сервис не будет об этом знать. Подробнее читайте в рецептах 6.9 и 6.13.

Со временем в вашем локальном хранилище наверняка появятся ресурсы с нарушенными ассоциациями, когда `URL associativeKey` будет возвращать код состояния 4xx или 5xx. При необходимости (например, для восстановления пространства)

можете создать задачу или скрипт, который будет обходить локальное хранилище и очищать записи, утерявшие удаленные ассоциации. Тем не менее часто важно сохранять локальные данные (для полноты истории и из прочих соображений), в этом случае локальный сервис должен просто игнорировать ассоциацию и/или заменять ее значения локальными предустановками (например, No Longer Available и т. п.).

Читайте также

- Рецепт 3.10. Проектирование с учетом возможности расширения сообщений.
- Рецепт 3.11. Проектирование с учетом возможности изменения интерфейсов.
- Рецепт 4.11. Проверка свойств данных в среде выполнения.
- Рецепт 4.12. Использование документов схем для валидации исходящих сообщений.
- Рецепт 4.14. Валидация входящих данных.
- Рецепт 6.1. Сокрытие внутренних деталей хранилища данных.
- Рецепт 6.9. Повышение производительности с помощью директив кэширования.

6.12. Установка лимита на объемные ответы

Для сервисов данных, которые получают большие объемы трафика и/или передают значительные объемы контента, ограничение размера ответов может стать эффективным способом реализовать масштабную поддержку клиентов API. И напротив, отсутствие регулирования размера и/или количества данных, возвращаемых при выполнении запросов к крупным наборам данных, запросто приведет к перегрузке вашего сервиса и, если набор данных критический, негативно повлияет на зависимые сервисы.

Задача

Как сделать так, чтобы ответы на запросы сервисов данных продолжали выполняться даже при увеличении размера набора данных? Какие стандартные средства существуют для управления размером возвращаемых коллекций данных? Когда важно устанавливать ограничения на размер ответов, возвращаемых от крупных наборов данных?

Решение

Лучшим способом обеспечить, чтобы ваш интерфейс для сервисов данных не испытывал проблем с быстродействием при увеличении размера набора данных, будет ограничение числа записей, возвращаемых в HTTP-ответах. Для этого нужно установить и должным образом задокументировать *предустановленное* максимальное

число записей для любых запросов данных. И если бэкенд-сервис еще не задал ограничения на размер коллекций ответов, вам нужно самим определить пределы для интерфейса, находящегося между клиентом API и внутренним сервисом.



Текущий рецепт дает возможность ограничить размер ответов на запросы к внутренним хранилищам данных. При этом можно использовать рецепт перемещения по страницам (см. рецепт 7.11) для повышения интерактивности управления ответами при запросах к большим наборам данных.

Большинство механизмов обработки данных позволяют ограничить количество записей, возвращаемых в одном ответе. Например, OData содержит директиву `$top` и параметр `maxpage size`. GraphQL поддерживает использование `first:nn`. В SQL есть директивы `TOP` и `LIMIT`, а в Lucene — параметр запроса `rows`. Так что можете задействовать эти возможности при создании вашим интерфейсом запроса к внутреннему сервису данных.



В запросах к хранилищам данных *всегда* нужно указывать максимально допустимое число записей ответа, а также заменять указанные клиентом предельные значения, если они выходят за установленный вами диапазон (например, слишком большие). Отправка запросов данных без указания максимального количества записей в ответе может негативно сказаться на быстроте и даже безопасности.

Вы можете ограничить размер ответа частью запроса, выполняемого клиентом API, иными словами, позволить клиентам определять это значение. Можно также установить предел в интерфейсе сервиса *до* отправки запроса (по сути, отредактировать запрос клиента до его отправки). Кроме того, можно (хотя потребуются больше работы) поддерживать оба этих варианта, то есть позволить клиентам API устанавливать предельный размер ответов и сделать так, чтобы после отправки запроса в интерфейс сервиса код искал этот размер и там. Если предельного значения не существует, код будет устанавливать базовое значение. Если же предельное значение установлено, но находится вне допустимых границ (например, это 10 000 000), код интерфейса сервиса изменит его нужным образом.

В случаях, когда внутренний источник данных не поддерживает установку максимального числа записей, потребуется указать собственные пределы для размера ответа на уровне интерфейса сервиса. Это означает принятие возвращаемой коллекции от внутреннего источника данных и затем при необходимости ее усечение до максимально допустимого числа записей.



Независимо от того, задаете вы пределы с помощью реального запроса к хранилищу данных или же усечением возвращаемой коллекции, всегда нужно указывать на предельное значение в метаданных запроса (см. рецепт 6.5), возвращаемых вызывающему.

Пример

Установить ограничение на ответы от интерфейсов сервисов данных можно двумя способами.

DirectLimit

Включить директиву ограничения в запрос, отправляемый хранилищу данных.

TruncatedLimit

Обрезать возвращаемую коллекцию данных на уровне интерфейса сервиса.

Далее приведены примеры обоих методов.

Прямая установка ограничения

Для установки прямых ограничений нужно просто позволить клиентским приложениям API отправлять с каждым запросом данных нужные директивы. Например, вот запрос, применяющий механизм поиска Solr. Обратите внимание на использование установки `limit=100`:

```
**** REQUEST ****
GET /persons/search/?q=%22Bob%22&rows=100 HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Q-Status=success
Q-Returned=100
Q-Count=10000
...
```

В этом примере клиентским приложением интерфейсу сервиса был отправлен параметр запроса `rows=100`. Но бывают случаи, когда клиент не отправляет предельное значение. Тогда ответственность по отправке этого значения в хранилище данных ложится на интерфейс сервиса. Вот пример с использованием механизма OData:

```
**** REQUEST ****
GET /persons/search/?$search=%22Bob%22 HTTP/1.1
Host: api.example.org
Accept: application/vnd.collection+json

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Q-Status=success
```

```

Q-Sent=$search=%22Bob%22
Q-Executed=$search=%22Bob%22$top=100
Q-Returned=100
Q-Count=10000
...

```

Обратите внимание на использование значений метаданных запроса `q-sent` и `q-executed` (см. рецепт 6.5). В случаях, когда интерфейс сервиса изменяет запрос клиента до его отправки в хранилище данных, эти два свойства метаданных должны быть возвращены в виде HTTP-заголовков или в теле ответа.

Реализация пределов путем усеечения

Бывают случаи, когда максимальный размер ответа на запрос данных должен сообщить интерфейс сервиса. Иногда это максимальное значение жестко прописывается в хранилище данных, а иногда в нем не установлено никаких ограничений. В обоих случаях важно, чтобы интерфейс сервиса защитил себя от возможного возврата слишком больших наборов данных, что бы это ни значило для отправившего запрос клиента. Обычно это подразумевает добавление в код API поддержки *усечения* возвращаемой из хранилища коллекции данных до ее отправки клиентскому приложению.

Реализация усеечения набора данных на уровне интерфейса сервиса будет различаться в зависимости от используемого языка программирования, возвращаемого формата и прочих факторов. Грубым решением станет простой обход возвращаемой коллекции с сохранением максимально допустимого к возвращению числа записей:

```

function executeQuery(dataStoreAddress, dataQuery) {
  var ix=0;
  var maxLimit=100;
  var responseCollection = [];
  var dataCollection = httpRequest(dataStoreAddress, dataQuery);
  for (let item of dataCollection) {
    if(ix>maxLimit) {
      break;
    } else {
      responseCollection.push(item);
    }
    ix++;
  }
  return responseCollection;
}

```



В некоторых случаях у вас может быть возможность сохранить всю коллекцию и реализовать локальную постраничную навигацию по сохраненной коллекции данных. Подробнее — в рецепте 7.11.

После того как интерфейс сервиса создаст собственную локальную копию коллекции, ограниченную максимально допустимым числом записей, API может вернуть результаты, показанные в примере с прямой установкой предельного размера ответа:

```
**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Q-Status=truncated
Q-Returned=100
Q-Count=10000
...
```

Обратите внимание на использование свойства метаданных `q-status=truncated`, помогающего клиенту API понять значение результатов.

Обсуждение

Наиболее безопасным подходом к реализации пределов для ответов на запросы данных будет их установка в коде API с помощью параметров внутреннего языка запросов. Если вы позволите устанавливать эти пределы клиентским приложениям, *всегда* проверяйте, чтобы они не выходили за приемлемые рамки, например не составляли –100 или 1 000 000.

Использование подхода с усечением ответа имеет свои недостатки. Например, в этом случае вы не контролируете способ, которым крупный набор данных может вернуться к вам от внутреннего источника данных. Даже если ваш API ограничивает коллекции ответов до 100 записей, все равно может возникнуть потребность ждать, пока внутренний (неограниченный) источник данных найдет и вернет 100 000 записей, прежде чем вы сможете применить внутреннюю процедуру усечения. В случае очень больших наборов возвращаемых данных производительность API при этом будет страдать.

ПРЕДЕЛЫ ОТВЕТОВ — ЭТО НЕ РАЗМЕРЫ СТРАНИЦ

Важно не путать пределы запросов, о которых шла речь ранее, с размерами страниц (рассматриваются в рецепте 7.11). Например, вы можете установить предел запроса к API на 1000, в то время как клиентское приложение устанавливает размер страницы на 100. В этом случае клиент будет ожидать в любом единичном ответе не более 100 записей.

При каждом изменении предела ответов или использовании их усечения нужно сообщать эту информацию обратно запрашивающему приложению, желательно с помощью метаданных запроса (см. рецепт 6.5).

Читайте также

- Рецепт 4.10. Поддержка ссылок и форм для сервисов без поддержки гипермедиа.
- Рецепт 4.15. Сохранение состояния.
- Рецепт 5.11. Поддержка мониторинга состояния сервисов.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 6.5. Возвращение метаданных в ответах на запросы.
- Рецепт 6.6. Возврат на запросы данных HTTP-кода состояния 200 или 400.
- Рецепт 7.11. Внедрение стандартной навигации по спискам.
- Рецепт 7.15. Синхронный повтор незавершенной работы с ответом 202 Accepted.

6.13. Использование промежуточных сервисов для обмена данными

Бывает, что сервисам нужно отредактировать подмножество полей в сохраненной записи, например данные `address` в записи `person`. В таких случаях важно обеспечить, чтобы эти изменения не привели к тому, что данные в полной записи станут недействительными. В то же время вышестоящие клиенты не должны знать детали нижестоящих сервисов. Обеспечить это поможет рецепт, описывающий использование промежуточных сервисов.

Задача

Как сохранить целостность хранилища данных, когда другие сервисы хотят отредактировать лишь часть находящей в нем записи? Как сделать так, чтобы при обновлении частичного набора полей данных в записи, например полей `address` в записи `person`, в полном их наборе не возникли конфликты? А если были обнаружены внутренние несогласованности в данных, то какой ответ нужно вернуть сервису, пытающемуся применить недопустимое обновление?

Решение

Хорошим правилом здесь будет *всегда* передавать между сервисами полные записи данных. Это самый простой и надежный способ обеспечить целостность хранящейся в записи информации. Даже если есть сервис, который хочет отредактировать лишь часть записи, например часть `address` записи `person`, он должен получить полную запись, изменить интересующую его часть и вернуть всю запись сервису данных для ее валидации и сохранения.

Когда потребляющий сервис является посредником между двумя сторонами, например сервисом `address`, обменивающимся данными с сервисом `person`, он все равно должен обмениваться с сервисом `person` полными записями. Реализовать это можно так: считать запись `person`, извлечь из нее `address` для отправки, зафиксировать запись `person`, а после получения обновленной информации `address` загрузить ее в запись и отправить последнюю обратно в сервис `person`. Естественно, сервис `address` должен быть готов к сбою записи в сервис `person` и при необходимости сообщить об этом потребителю `address`.

ОДНО УПИРАЕТСЯ В ДРУГОЕ

Пример с `person` и `address` — это напоминание о том, что потребители API никогда не могут быть уверены, общаются они с промежуточным сервисом или с сервисом хранилища. Можно даже представить еще один сервис под названием `telephone`, позволяющий потребителям обновлять значение номера телефона в записи `address`. Теперь у нас есть уже три уровня: `telephone`, `address` и `person`. Как мы знаем, сервис `person` в будущем может измениться с хранилища на промежуточный. Тогда при обмене данными получится уже *четыре* уровня.

Сервисы, созданные для редактирования подмножества данных (`address` для `person`), могут быть ограничены функциональностью обновления и не поддерживать операции создания или удаления. Например, единственным способом создания дополнительного `address` или удаления существующего `address` для `person` станет использование сервиса `person` напрямую.

Обратите внимание на то, что стоит отслеживать метаданные обновления при обмене подмножеством записей. Подробнее читайте в рецепте 6.5.

Пример

На рис. 6.7 показана диаграмма последовательностей, которая дает полноценную картину взаимодействия трех сервисов в нашем примере. Естественно, каждый участник (`apiClient`, `addressAPI` и `personAPI`) знает лишь о своих непосредственных соседях. Например, `apiClient` знаком только с `addressAPI`, `personAPI` — только с `addressAPI`, а `addressAPI` — с ними обоими.

Далее приведен пример взаимодействия сервиса `address` с сервисом `person` с целью поддержки обновления части `address` в `person`.

Сначала сервис `address` получает запрос записи:

```
**** REQUEST ****
GET /q1w2e3r4 HTTP/1.1
Host: api.address.org
Accept: application/vnd.collection+json
```

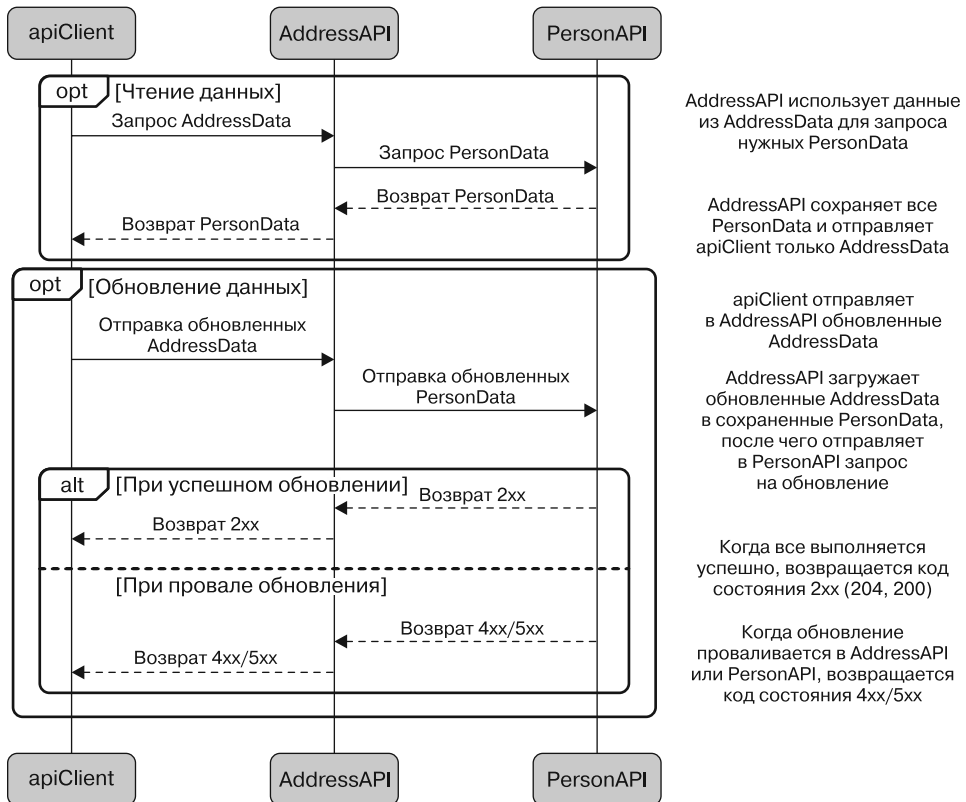


Рис. 6.7. Рецепт передачи данных через промежуточный сервис

Затем сервису `address` нужно связаться с сервисом `person` для получения записи:

```

**** REQUEST ****
GET /q1w2e3r4 HTTP/1.1
Host: api.person.org
Accept: application/vnd.collection+json

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Content-Length: nn
ETag: "w\p0o9i8u7y6"
...

{"collection" : {
  "title": "Person Service",
  "links": [...],
  "items": [
    { "href": "https://api.person.org/q1w2e3r4",

```

```

    "data": [
      {"name": "id", "value": "q1w2e3r4"},
      {"name": "fullName", "value": "Mork Markleson"},
      {"name": "streetAddress", "value": "123 Main St"},
      {"name": "cityTown", "value": "Byteville"},
      {"name": "stateProvince", "value": "MD"},
      {"name": "postalCode", "value": "12345"}
    ]
  }
}
}}

```

Теперь сервис `address` может сохранить копию записи `person` локально (не забудьте также сохранить заголовки ответов) и затем ответить вызывающему подмножеством полей `address`:

```

**** RESPONSE ****
HTTP/1.1 200 OK
Content-Type: application/vnd.collection+json
Content-Length: nnn
ETag: "w\y6t5r4e3w2"

{ "collection": {
  "title": "Address Service",
  "links": [...],
  "items": [
    { "href": "https://api.address.org/q1w2e3r4",
      "data": [
        {"name": "id", "value": "q1w2e3r4"},
        {"name": "street", "value": "123 Main St"},
        {"name": "municipality", "value": "Byteville"},
        {"name": "region", "value": "MD"},
        {"name": "zipCode", "value": "12345"}
      ]
    }
  ],
  "template": {
    "rel": "edit",
    "data": [
      {"name": "id", "value": "q1w2e3r4"},
      {"name": "street", "value": "123 Main St"},
      {"name": "municipality", "value": "Byteville"},
      {"name": "region", "value": "MD"},
      {"name": "zipCode", "value": "12345"}
    ]
  }
}
}}

```

Здесь нужно обратить внимание на пару нюансов. Во-первых, сервис `address` сгенерировал собственное значение `ETag`, соответствующее представлению `address`. Его можно использовать в случае, когда обновить эту запись попытаются более одной стороны (подробнее о проблеме потерянного обновления читайте в рецепте 6.2).



В этом примере сервисы `address` и `person` задействуют одинаковое значение `id`, в том числе в URL-адресе. Так бывает не всегда, и этот нюанс нужно иметь в виду. Работая с промежуточным сервисом, вам может потребоваться отслеживать два разных идентификатора и два отдельных URL.

Во-вторых, обратите внимание на то, что поле `names`, используемое сервисом `address`, необязательно должно совпадать с аналогичными полями из сервиса `person`. Будет удобно, если они окажутся *одинаковыми*, но зачастую мы этот момент не контролируем и для передачи данных от одной стороны другой требуется этап перевода.

Далее сервис `address` может подождать, пока кто-то не вернет запись для обновления:

```
**** REQUEST ****
PUT /q1w2e3r4 HTTP/1.1
Host: api.address.org
Content-Type: application/vnd.collection+json
Content-Length: nn
If-Match: "w\y6t5r4e3w2"
...

"template": {
  "data": [
    {"name": "id", "value": "q1w2e3r4"},
    {"name": "street", "value": "123 Main St, Apt 3G"},
    {"name": "municipality", "value": "Byteville"},
    {"name": "region", "value": "MD"},
    {"name": "zipCode", "value": "12345-6789"}
  ]
}
```

Затем сервис `address` должен загрузить эти данные из вызывающего в собственную локальную копию записи `person` и передать их в сервис `person`:

```
**** REQUEST ****
PUT /q1w2e3r4 HTTP/1.1
Host: api.person.org
Content-Type: application/vnd.collection+json
Content-Length: nn
If-Match: "w\p0o9i8u7y6"
...

"template": {
  "data": [
    {"name": "id", "value": "q1w2e3r4"},
    {"name": "fullName", "value": "Mork Markleson"},
    {"name": "streetAddress", "value": "123 Main St, Apt 3G"},
    {"name": "cityTown", "value": "Byteville"},
    {"name": "stateProvince", "value": "MD"},
  ]
}
```

```

    {"name": "postalCode", "value": "12345-6789"}
  ]
}

```

**** RESPONSE ****

HTTP/1.1 204 No Content

Если все пройдет гладко, сервис `person` вернет сервису `address` ответ 2xx, который проделает то же самое в отношении того сервиса, который его вызвал.

Если же при записи в сервис `person` возникнет проблема, сервису `address` будет возвращен соответствующий код состояния, который он передаст далее сервису, его вызвавшему:

**** RESPONSE ****

HTTP/1.1 412 Precondition Failed

Content-Type: application/vnd.collection+json

Content-Length: nn

```

{"collection": {
  "title": "Person Service",
  "links": [...],
  "error": {
    "title": "Unable to update record",
    "code": "http://api.person.org/reasons/precondition",
    "message": "The record you are trying to update has been changed.",
    "data" [{"name": "id", "value": "q1w2e3r4"}]
  }
}}

```

Вам может потребоваться изменить информацию `error` в зависимости от типа ошибки и места, где она произошла (сервис хранилища, промежуточный сервис).

Обсуждение

Принцип промежуточного интерфейса данных важен лишь для интерфейса, который выступает посредником между двумя сервисами, как в предыдущем примере с сервисом `address`. Потребителям промежуточных сервисов не нужно знать о том, что они взаимодействуют с посредником. То же самое касается любого интерфейса сервиса, находящегося *за* промежуточным.



В качестве альтернативы этому рецепту с посредником сервисы могут игнорировать неизвестные для них поля данных (см. рецепт 6.8).

Главное в этом рецепте — обеспечить, чтобы промежуточные сервисы отслеживали связь между нижестоящей (исходной) и вышестоящей (раскрытой) записями.

Для этого нужно всегда сохранять полную копию нижестоящей записи, включая поля метаданных вроде ETag, а также другую сопутствующую информацию о целостности ресурса. Лучшим способом для этого будет сохранить весь HTTP-ответ, а не только его тело. Подробнее читайте в рецепте 6.5.

В качестве детали реализации для промежуточного сервиса всегда стоит использовать этап перевода, посвященный обработке любых различий между списками свойств данных интерфейса посредника и данных исходного сервиса:

```
function fetchPerson(url,propertyMap) {
  var personRecord = person.read(url);
  var addressRecord = mapProperties(personRecord, propertyMap)
  return addressRecord;
}
```

Этап `mapProperties(personRecord, propertyMap)` отвечает за получение ожидаемых полей из исходной (нижестоящей) записи и построение валидной раскрываемой (вышестоящей) записи. В некоторых случаях это будет не просто переименование свойств данных по принципу «один к одному» (`person.givenName = +address.firstName`). Иногда поля совмещаются или разделяются между интерфейсами:

```
var address.fullName = person.familyName+ " , "+ person.givenName
```

СЕГОДНЯ — ИСТОЧНИК, А ЗАВТРА — ПОСРЕДНИК

Поскольку мы не контролируем все сервисы в веб-среде, то не можем всегда знать, с кем именно связан интерфейс — с сервисом-источником данных или же сервисом-посредником. По факту сервис может изначально выступать источником данных, а затем превратиться в посредника. С позиции интерфейса это неважно. Интерфейсы должны продолжать давать обещания в отношении обмена сообщениями независимо от деталей реализации стоящих за ними сервисов.

Промежуточные сервисы должны быть готовы к случаям, когда плохо управляемый нижестоящий источник изменит свой ответ, например поля данных `add/remove/rename`. Предотвратить это нельзя, но можно смягчить проблему, если всегда инспектировать входящие нижестоящие записи на предмет содержания в них обещанных вышестоящим интерфейсом свойств данных. Если это окажется не так, промежуточный сервис должен возвращать ответ `502 Bad Gate way`, указывая в его теле на проблему:

```
**** REQUEST ****
GET /q1w2e32r4 HTTP/1.1
Host api.address.org
Accept: application/vnd.collection+json
...
```

**** RESPONSE ****

HTTP/1.1 502 Bad Gateway

Content-Type: application/vnd.collection+json

Content-Length: nn

...

```
{
  "collection": {
    "links": [...],
    "error": {
      "title": "Invalid Response",
      "code": "SRC-077",
      "message": "Data source is missing the [givenName] data property"
    }
  }
}
```

Обратите внимание на то, что в этом примере в отчет о состоянии включены только детали, связанные непосредственно с текущим интерфейсом обмена — сервисом `client` и `address`. Здесь нет упоминания нижестоящего источника — сервиса `person`.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 3.10. Проектирование с учетом возможности расширения сообщений.
- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.17. Использование семантических посредников для доступа к несовместимым сервисам.
- Рецепт 6.1. Соккрытие внутренних деталей хранилища данных.
- Рецепт 7.20. Использование посредников рабочих потоков для включения несовместимых сервисов.

Рабочий поток на основе гипермедиа

Продуктивность никогда не является случайностью. Она всегда становится результатом стремления к совершенству, грамотного планирования и сосредоточенных усилий.

Пол Дж. Мейер

Для многих из нас конечной целью создания API сервисов является получение гибких и надежных *рабочих потоков*, уникальным образом комбинирующих существующие сервисы для решения новых задач. В действительности эта цель входит в ключевой принцип, рассматриваемый в разделе «Общие принципы масштабируемых сервисов в веб-среде» главы 1. Процесс связывания сервисов между собой не особо сложен, когда эти сервисы изначально созданы с учетом взаимодействия. Тем не менее включение в процесс тех из них, которые не создавались как группа, ничего друг о друге не знают и не понимают создаваемого решения, представляет собой более серьезную задачу. Этот второй подход зависит от важных деталей реализации, которые делают возможными ряд действий, таких как передача состояния решения, прохождение нескольких этапов для достижения цели и отслеживание/отображение этого прогресса. Всякий раз, когда вы совмещаете интерфейсы сервисов, вероятно возникновение ошибок выполнения, которые могут потребовать повтора ряда шагов, их отмены и понимания того, когда и что из этого необходимо.

Зачастую мы создаем свои сервисы как связанную коллекцию. Если одна и та же команда разрабатывает более одного сервиса, у этих сервисов неизбежно возникают скрытые зависимости, которые сложно распутать. При этом удержание их связи под контролем требует дисциплины и решительности.

Сложность в том, что проектирование и поддержка успешного рабочего потока в веб-среде требуют дополнительных усилий. Зато при условии реализации каждого отдельного сервиса с простым API, а также установки общей модели обработки

и отслеживания, поддерживаемой всеми сервисами, можно создавать устойчивую, гибкую и развиваемую среду рабочего потока, которую клиенты смогут спокойно использовать для решения новых интересных задач.



Подробнее о важных элементах грамотно спроектированных и реализованных потоков в веб-среде читайте раздел «Увеличение расширяемости с помощью рабочего потока на основе гипермедиа» главы 2.

Поддержка потока на основе гипермедиа (рис. 7.1) подразумевает акцент на общепринятых паттернах, таких как навигация по страницам, текущая работа, вытеснение последних использованных (most recently used, MRU) и т. д. Это также означает поддержку обобщенного рабочего потока на основе гипермедиа посредством задач и заданий вкупе с рядом техник оптимизации. Наконец, есть второй уровень действий, нацеленных на управление самим процессом рабочего потока. Состоит он из четырех элементов: паттернов, обобщенного потока, оптимизации и управления, которые составляют здоровую систему рабочего потока на основе гипермедиа.

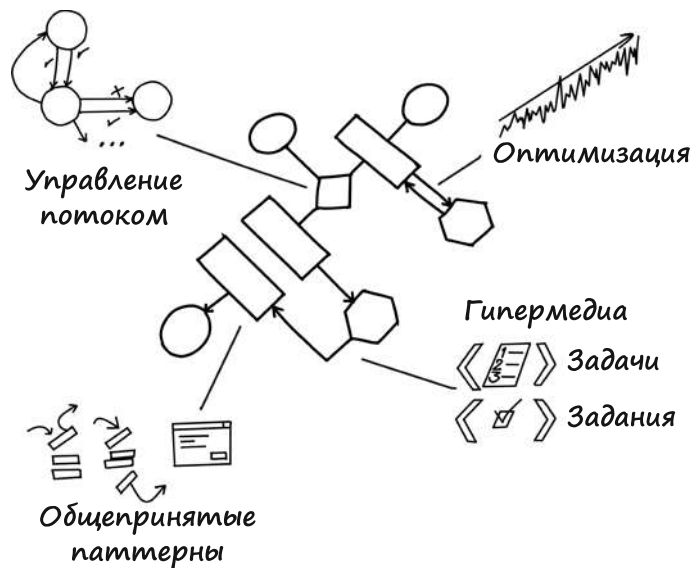


Рис. 7.1. Рецепты рабочего потока на основе гипермедиа

Эта глава содержит два вида рецептов. Первый — это набор типичных рабочих потоков вроде навигации по спискам, паттернов «текущая работа», «обработка форм» и аналогичных (см. рецепты 7.8–7.14). Эти образцы паттернов можно использовать в качестве ориентира при создании собственных семантических профилей (см. рецепт 3.4) для интерфейсов сервисов.

Второй набор приведенных здесь рецептов описывает общий язык взаимодействия рабочих потоков, который можно применять для включения выборочных независимых сервисов и отслеживания прогресса таких мультисервисных решений. Это намного более абстрактный набор паттернов, который охватывает все, начиная со встраивания в рабочий поток поддержки сервисов и заканчивая фактической реализацией сервисов, совместимых с рабочим потоком.

Наконец, здесь вы найдете несколько записей, описывающих общие принципы и паттерны координации таких сервисов и управления ими. К ним относятся обработка ошибок, откатов и даже ряд внутренних паттернов для масштабирования и проксирования других интерфейсов сервисов.



Вы увидите, что многие из рецептов этой главы зависят от рецептов из предыдущей части книги или ссылаются на них. Если вы те главы пропустили, самое время к ним вернуться, чтобы понять истоки некоторых рассматриваемых здесь рецептов, посвященных рабочим потокам.

7.1. Проектирование сервисов, совместимых с рабочими потоками

Общая цель многих проектировщиков интерфейсов — возможность соединять несколько сервисов для формирования рабочего потока. Зачастую такие сервисы называются *компоуемыми*. Текущий рецепт описывает набор базовых решений проектирования, которые должен иметь каждый совместимый с рабочим потоком (компоуемый) сервис.

Задача

Как спроектировать сервис, пригодный для использования в рабочем потоке? Какие ключевые возможности характерны для любого интерфейса компоуемого сервиса? Как безопасно, не затратно и легко реализовать рабочий поток с помощью компоуемых сервисов?

Решение

Ключом к проектированию совместимых с рабочими потоками сервисов выступают последовательный набор действий и простота обмена данными о состоянии между сервисами. Эти элементы — общие действия и общее состояние — лежат в основе стабильных компоуемых интерфейсов сервисов.

Помимо общего набора действий и состояния, совместимые с рабочими потоками сервисы поддерживают общие идентификаторы задачи (в виде `correlation-id`) и каждого задания в рамках нее (в виде `request-id`).

Действия рабочих потоков

В сервисах на основе гипермедиа действия выражаются во входных формах. Каждая форма описывает все, что необходимо для завершения действия: URL-адрес, HTTP-метод, поддерживаемые медиатипы и весь набор входных значений.



Подробнее о грамотном проектировании форм читайте в рецепте 7.12.

И хотя действия могут быть как весьма конкретными (например, `onboardCustomer`, `computeSalesTax` и т. д.), так и обобщенными (`writeRecord`, `filterData` и т. п.), они всегда *должны* быть полноценно описаны с включением всех входных значений и HTTP-метаданных (подробнее — в разделе «Пример»).

В список действий также входят следующие.

Execute (выполнение)

Фактическая работа, которую нужно проделать (например, `applySalesTax`).

Repeat (повтор)

Повтор работы идемпотентным способом (например, `applySalesTax` можно спокойно повторить, получив те же ожидаемые результаты).

Revert (откат)

Возможность отменить завершенное действие (например, `revertSalesTax`).

Continue (продолжение)

Возможность приостановить рабочий поток и затем спустя какое-то время продолжить с того же места.

Rerun (перезапуск)

Возможность выполнить серию задач рабочего потока с начала, повторив все шаги, даже если этот рабочий поток уже выполнялся ранее.



Для всех совместимых с рабочим потоком сервисов желательно реализовывать поддержку всех этих действий, даже если какие-то из них пока ничего не делают, например нет поддержки отката. В таком случае их поддержку будет проще добавить в будущем.

Для любого из этих действий *должны* присутствовать соответствующий URL-адрес и FORM. Первые три действия применяются к каждому шагу (здесь называются *заданиям*) в рабочем потоке. Последние же два относятся ко всем заданиям (называются *задачей*).

Общее состояние

Вот еще два действия, необходимых для интерфейсов, совместимых с рабочим потоком сервисов.

ReadState

Возможность загружать сопутствующие свойства состояния для использования каждым *заданием* в *задаче*.

WriteState

Возможность сохранять сопутствующие свойства состояния для применения другими *заданиями* в одной *задаче*.



Подробнее об обработке общего состояния в рабочих потоках HTTP читайте в рецепте 7.2.

Оба действия должны иметь связанный с ними URL-адрес, указывающий на общий HTTP-ресурс. Этот URL ресурса должен передаваться каждому *заданию* в рабочем потоке. Сервисы сами определяют, как им распознавать представленные в ресурсе состояния значения и при необходимости обновлять их или добавлять новые. Зачастую свойства в ресурсе используются для заполнения FORM, предоставляемых действиями компонуемого сервиса.



Пример обобщенного механизма рабочего потока, реализованного с помощью языка управления задачами, есть в рецепте 7.6.

Идентификатор корреляции для заданий и задач

Каждым задаче и заданию в рамках рабочего потока нужен идентификатор, обычно UUID или некое другое глобально-уникальное значение. В качестве идентификатора корреляции следует использовать `jobID`. Его можно передавать в HTTP-заголовке `correlation-id`. В качестве идентификатора запроса следует использовать значение `taskID`. Его можно передавать в HTTP-заголовке `request-id`. Как и `correlation-id`, `request-id` нужно включать как в HTTP-запрос, так и в HTTP-ответ.



Заголовки `correlation-id` и `requestid` не являются стандартными зарегистрированными заголовками. В вашей организации могут использовать другие их значения или даже другой способ отслеживания задач и заданий рабочего потока. Как бы то ни было, ключом к успеху здесь будут последовательность и хорошее документирование.

Совместимые с рабочими потоками сервисы для каждого задания в задаче поддерживают как заголовок `correlation-id`, так и заголовок `request-id`.

Пример

Рассмотрим компонентный сервис, вычисляющий НДС для товаров в покупательской корзине. Этот сервис вместе с другими совместимыми с рабочим потоком сервисами можно использовать как задание в рамках задачи по обработке оплаты заказа на торговом сайте. Далее приведен вымышленный пример на предметно-ориентированном языке, показывающий, как это может выглядеть:

```
*** Checkout Job
***
READ sharedState WITH urlState
EXECUTE shoppingCartService->checkoutForm WITH sharedSTATE
IF-NOT-OK EXIT
EXECUTE salesTaxService->applyTaxesForm WITH sharedSTATE
IF-NOT-OK EXECUTE shoppingCartService->revertCheckoutForm WITH sharedSTATE
STORE sharedState WITH urlState
EXIT
***
*** End Job
```

В этом примере `sharedState` загружается в клиентское приложение. Затем это приложение пытается выполнить `shoppingCartService → checkoutForm` и `salesTaxService → applyTaxesForm`, после чего сохраняет итоговое `sharedState`. Обратите внимание на поддержку `shoppingCartService → revertCheckoutForm`, добавленную на случай сбоя при вычислении НДС.

Обсуждение

Желательно, чтобы любой интерфейс сервиса, предлагающий действие, которое записывает данные в общее состояние, предоставлял способ отменить это действие. Это может быть простое восстановление старых значений в ресурсе общего состояния (как в примере `applySalesTax`) или запуск отдельной задачи для отмены серии шагов, относящихся к действию, которое нужно откатить. Подробнее об обработке связанных задач читайте в рецепте 7.6.

Желательно, чтобы интерфейсы сервисов каждого задания предоставляли вызовы для повтора и отката, даже если действие «Выполнить» еще не вызывалось. Например, вызовы для отката *не должны* приводить к ошибке, пока проблема не возникнет в самом действии отката. Это означает, что сервисы должны знать, было ли ранее осуществлено действие «Выполнить». Если это так, вызов отката потребует некоторой обработки. Если же действия «Выполнить» еще не было, вызов отката должен просто вернуть `200 OK` без какой-либо обработки. Можете

представить откат как действие DELETE. Удаление уже удаленного ресурса не создаст никаких проблем, значит, ответ 200 OK будет вполне приемлемым.

ОТКАТИТЬ ВСЕ

Если какое-то задание провалится или превысит порог ожидания (например, достигнет `taskMaxTTL`), *задача* окажется недействительной и будет вызвано действие «Отмена». В результате должен производиться откат всех заданий этой задачи. Такой механизм существенно упрощает реализацию межсервисных откатов или отмен (см. рецепт 7.17).

Повтор действия должен легко обрабатываться, поскольку все действия сервисов должны быть идемпотентными (см. рецепт 3.6). Если у вас возникнут сложности с безопасным повтором действия, значит, проблема наверняка в неидемпотентности.

Поддержка передачи информации о состоянии означает, что любая задача имеет документ состояний, который передается между ее внутренними заданиями. Может показаться более удачной идея задействовать для обмена этой информацией запись базы данных, хотя это не так. Для передачи данных о состоянии должен использоваться отдельный HTTP-ресурс, поддерживающий идемпотентные обновления и, возможно, изменяемые медиатипы представления. Это единственный набор свойств, которые должны передаваться между сервисами в задаче. Подробнее об этом рассказывается в рецепте 3.7.

Если в задаче присутствует множество шагов, часть из которых может требовать много времени, и эта задача по какой-то причине остановила выполнение (например, возникла проблема с одним из заданий, которое должен исправить человек), тогда весьма полезной окажется возможность продолжить выполнение с места остановки. Такой механизм экономит время и вычислительные ресурсы.

Поддержка действия «Повтор» пригождается, когда вы хотите принудительно перезапустить задачу, содержащую несколько заданий. Например, вы заметите, что выполнение задачи дало не те результаты, которых вы ожидали. В этом случае можете попробовать исправить информацию о состоянии и заново выполнить завершенную задачу, чтобы получить желаемые результаты.

И хотя здесь об этом не говорилось, вы также можете создавать *шаблоны* задач, в которых присутствуют одинаковые задания, но изменяется содержимое ресурса общего состояния. Это позволяет быстро определять и выполнять задачи, исходя из параметров конфигурации (посредством ресурса общего состояния). Если вы планируете много подобной работы, то посмотрите рецепт 7.6 — там приводится пример обобщенного RESTful-языка управления задачами, который существенно упрощает определение и выполнение задач.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 6.13. Использование промежуточных сервисов для обмена данными.
- Рецепт 7.2. Поддержка общего состояния в рабочих потоках.
- Рецепт 7.6. Поддержка RESTful-языка управления задачами.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.
- Рецепт 7.17. Поддержка локальной отмены или отката.

7.2. Поддержка общего состояния в рабочих потоках

Совместимые с рабочими потоками сервисы должны уметь обмениваться между собой свойствами состояния. Текущий рецепт содержит простой и эффективный способ обмена информацией о состоянии, не требующий передавать данные предметной области или их модели.

Задача

Как проектировать сервисы, чтобы они могли обмениваться данными, не создавая излишне тесного связывания с моделями данных или информацией предметной области? Как будет выглядеть минимальный дизайн механизма обмена состояниями?

Решение

Самый простой способ обмена данными между сервисами — использование *ресурса общего состояния*. Это отдельный HTTP-ресурс, который содержит все данные, связанные с задачей рабочего потока. Каждый совместимый с рабочим потоком сервис, отвечающий за задание в рамках задачи, должен иметь возможность прочесть этот ресурс общего состояния и в некоторых случаях выполнить запись в него.

Как правило, ресурс общего состояния применяется сервисами в качестве источника для заполнения HTTP FORM, описывающих действие (например, `onboardNewUser`). Иногда в этот отдельный ресурс записываются результаты действия (например, `applySalesTax`).

После завершения задачи этот ресурс нужно отправить в архив для возможного использования в будущем.



Ресурс общего состояния — это не то же самое, что ресурс прогресса рабочего потока (см. рецепт 7.7). При реализации надежной системы рабочего потока вам потребуются оба.

Простым способом создания ресурсов общего состояния является использование уникального идентификатора (обычно идентификатора задачи) в качестве его публичного URL-адреса:

```
<link rel="sharedState" href="http://api.example.org/shared-state/u7y6t5r4e3 />
```

Этот URL нужно передавать с каждым HTTP-запросом и ответом, связанными с выполняемой задачей.

Пример

URL-адрес ресурса общего состояния можно передавать в коллекции HTTP-заголовков:

```
**** REQUEST ****
GET /onboarding/job1
Accept: application/vnd.collection+json
Link: <http://api.example.org/state/q1w2e3r4t5>;rel="sharedState"
```

```
**** RESPONSE ****
200 OK
Content-Type: application/vnd.collection+json
Length: XX
```

```
{"collection" : {
  {"title": "Pricing Update Job q1w2e3r4t5",
   "links" : [...],
   "items" : [...]}
}}
```

Или в теле сообщения:

```
**** REQUEST ****
GET /onboarding/job1
Accept: application/vnd.collection+json

**** RESPONSE ****
200 OK
Content-Type: application/vnd.collection+json
Length: XX
```

```
{ "collection" : {  
  { "title": "Pricing Update Job q1w2e3r4t5",  
    "links" : [  
      { "rel": "sharedState", "href": "http://api.example.org/state/q1w2e3r4t5" }  
    ],  
    "items" : [...] }  
}
```

Ресурс общего состояния можно «зарядить» данными до начала выполнения задачи. К примеру, если вы выполняете задачу по обновлению цен выборочного набора элементов в хранилище, то можете загрузить в этот ресурс массив идентификаторов продуктов вместе с информацией о повышении цен:

```
{ "collection" : {  
  "title" : "Shared State for Job ID q1w2e3r4t5",  
  "links" : [  
    { "rel": "read, sharedState",  
      "href": "http://api.example.org/shared-state/q1w2e3r4t5" },  
    { "rel": "create, sharedState",  
      "href": "http://api.example.org/shared-state/q1w2e3r4t5/form" },  
    { "rel": "filter, sharedState",  
      "href": "http://api.example.org/shared-state/q1w2e3r4t5/search" }  
  ],  
  "items" : [  
    { "id": "pid-00122356",  
      "href": "http://api.example.org/shared-state/q1w2e3r4t5#pid-00122356",  
      "data": [  
        { "name": "currentPrice", "value": "10" },  
        { "name": "updatedPrice", "value": "11" },  
        { "name": "state", "value": "pending" },  
        { "name": "text", "value": "" }  
      ]  
    },  
    { ... }  
  ]  
}
```

После загрузки документа задачу можно запускать. Вот соответствующий псевдокод:

```
FOR-EACH item in http://api.example.org/shared-state/q1w2e3r4t5 updatePrice(item)  
END-EACH
```

В этом случае считывается `item` и функция (`updatePrice`) использует его данные для поиска продукта в каталоге, после чего (если `currentPrice` согласуется с указанной в документе общего состояния) применяется `updatedPrice`. Затем документ общего состояния изменяется, отражая состояние работы (например, с `pending` на `completed`), и в `text` передаются все необходимые примечания, например «применено успешно» или «`currentPrice` не соответствует» и т. д.

Каждый раз, когда задача завершает цикл и изменяет документ общего состояния, она должна записывать его обновленную версию по тому же URL-адресу, используя идемпотентный HTTP-метод PUT. Таким образом будет наиболее точно зафиксировано состояние выполняемой работы. Этот подход упрощает поддержку ресурса прогресса (см. рецепт 7.7) и помогает при диагностике любых попутно возникших проблем (см. рецепт 7.18).

После завершения работы итоговый ресурс общего состояния можно заархивировать для последующего обращения.

Обсуждение

В предыдущем примере свойства данных `currentPrice`, `updatedPrice`, `status` и `text` должны являться семантическими идентификаторами, утвержденными заранее. Обычно это обрабатывается с помощью общего словаря (см. рецепт 3.3).

Обратите внимание, что в этом примере задание было описано с помощью идемпотентного языка. Например, целевое свойство изменения выражается в виде `updatedPrice`, а не `percentIncrease`. Это существенно упрощает поддержку повтора и отката изменений.

Вы можете решить, что для ресурса общего состояния достаточно простого документа с парами «имя — значение». Такой вариант может сработать, но гораздо лучше включить множество метаданных, чтобы сервисам было проще отслеживать, какую работу нужно проделать, а какая ее часть уже завершена. По возможности делайте ресурсы общего состояния полноценными HTTP-ресурсами, а не просто файлами с сериализованными данными.

Вам нередко потребуется использовать документ общего состояния в качестве источника для заполнения входной формы данных, получаемой от другого сервиса. Проще всего для этого написать функцию *сопоставления*, которая будет получать форму и данные об общем состоянии и автоматически применять их к форме.

```
var sharedState = httpRead("http://api.example.org/state/q1w2e3r4t5");
var inputForm = httpRead("http://api.example.org/users/onboardingform");
var completedForm = propertyMap(sharedState, inputForm);
var results = httpExecute(completedForm);
```

Одно из преимуществ этого рецепта общего состояния в том, что со временем вы сможете добавлять в документ новые данные состояния, не обновляя существующие сервисы, которые на него опираются. Каждый сервис может использовать содержимое ресурса состояния нужным ему образом. Это ослабляет связанность между сервисом и состоянием, обеспечивая реализацию с бóльшим упором на конфигурацию.

Читайте также

- Рецепт 3.7. Обеспечение интероперабельности с помощью передачи состояния между сервисами.
- Рецепт 4.15. Сохранение состояния.
- Рецепт 5.14. Повышение скорости обработки с помощью клиентских идентификаторов.
- Рецепт 6.11. Расширение удаленных хранилищ данных.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.
- Рецепт 7.13. Использование наблюдения состояния для реализации рабочего потока под управлением клиента.

7.3. Описание рабочего потока через код

В случаях, когда для получения единого решения нужно задействовать несколько сервисов, вам потребуется способ описать список этих сервисов и схему их взаимодействия. К этому есть разные подходы. В текущем рецепте показан самый простой, подразумевающий использование исходного кода, скрытого за интерфейсом сервиса.

Задача

Как проще всего сформировать решение с применением нескольких независимых интерфейсов сервисов при помощи исходного кода компонентов сервиса? Когда есть смысл использовать исходный код, а не другой, более абстрактный и гибкий подход?

Решение

Когда у вас есть довольно простой рабочий поток и/или такой, который не будет со временем меняться и не поддерживает возможность индивидуальной настройки, обычно разумнее всего описать его с помощью исходного кода внутри одного интерфейса сервиса. Это означает создание интерфейса, который содержит все возможности нужного решения, и затем уже за этим интерфейсом использование исходного кода для включения и реализации других удаленных сервисов, нужных для выполнения работы.

Таким образом вы получите простой и стабильный API для приложений потребителей, а также обеспечите максимальную гибкость проекта и сборки в ходе работы за этим стабильным интерфейсом. Со временем задействованные сервисы могут изменять свои API или вы можете заменить один из них (например, сервис `taxComputation`) другим (например, `newTaxComputation`). При условии

неизменности внешнего интерфейса вы можете вносить значительные изменения во внутреннюю реализацию, не нарушая обещания, данного потребителям решения. Подробнее — в рецепте 5.2.

Пример

Рассмотрим организацию, которой нужно реализовать рабочий поток адаптации новых сотрудников. Также предположим, что несколько подразделений этой организации независимо друг от друга предоставляют онлайн-сервисы, которые позволяют их командам выполнять свои обязанности. Это означает, что у вас есть набор полезных интерфейсов сервисов, которые можно использовать для составления решения в исходном коде.

Одно из решений с исходным кодом, собирающее необходимые входные данные и затем вызывающее набор внешних сервисов для выполнения этой работы, может выглядеть так:

```
(TK check code=ma)
// Предоставление экранов ввода для сбора важных данных
var stateData = collectInputs(personalData, backgroundCheck,
    healthInsurance, salaryDetails);

// Параллельное выполнение работы
Promise.all([apiPersonalData(stateData), apiBackgroundCheck(stateData),
    apiHealthInsurance(stateData), apiSalaryDetails(stateData)])
    .then((results) => {
        emitSuccessResponse(results);
    }).catch(error => { emitFailedResponse(results);
});
```

В этом примере собираются необходимые входные данные, после чего выполняется ряд параллельных вызовов к внешним интерфейсам сервисов для обработки каждого задания в рабочем потоке. Затем успешные результаты возвращаются вызывающему с помощью метода `emitSuccessResponse(results)` или, если возникли проблемы с сетью или обработкой, посредством метода `emitFailedResponse(results)`.

Внутренний код можно выразить через HTTP таким образом:

```
*** REQUEST ***
GET /onboarding/jobs/
Accept: text/html
Link: <http://webapicookbook.com/profiles/jobs>;rel=profile
...

*** RESPONSE ***
200 OK
Content-Type: text/html
Link: <http://webapicookbook.com/profiles/jobs>;rel=profile
...
```

```

<html>
  ...
  <h1>Onboard New Employee</h1>
  <form name="submit-job" action="/onboarding/q1w2e3r4" method="put">
  ...
  </form>
</html>

*** REQUEST ***
PUT /onboarding/jobs/q1w2e3r4
Content-Type: application/x-www-form-urlencoded
Accept: text/html
Link: <http://webapicookbook.com/profiles/jobs>;rel=profile

jobID=q1w2e3r4&...

*** RESPONSE ***
202 Accepted
Content-Type: text/html
Link: <http://webapicookbook.com/profiles/jobs>;rel=profile

<html>
  ...
  <h1>Job q1w2e3r4</h1>
  <div name="progress">
    <span id="jobStatus">pending</span>
    ...i
    <a href="..." rel="refresh">Check Job Status</a>
  </div>
</html>

```

В этом обмене по HTTP потребитель API запрашивает ресурс, который предоставляет входные FORM для описания задачи по адаптации новых сотрудников. Затем, после заполнения, FORM отправляется по указанному в форме URL-адресу, где будет выполняться задача. Обратите внимание на то, что механизм рабочего потока в процессе выполнения задачи возвращает код состояния **202 Accepted**.

Потребитель API может использовать URL-адрес **refresh** для отслеживания статуса задачи, и в конечном счете ответом будет либо **200 OK**, либо код состояния **4xx/5xx**, указывающий на ее завершение.

Обсуждение

Заметьте, что в этом примере используется JavaScript-конструкция `promise.all()`. Она предполагает, что все сервисы могут выполняться параллельно и не в строгой последовательности. Если вам нужно реализовать рабочий поток, имеющий один или несколько последовательных шагов, то нужно рассматривать каждый фиксированный шаг как *отдельный* элемент потока (подробнее — в рецепте 7.1).

Задачу по сбору входных данных можно обработать с помощью рецепта 7.10 и/или 7.12.

При этом возможны коварные ошибки. В некоторых случаях вам может потребоваться реализовать поддержку опции *отката* для отмены завершенной работы. Подробнее о том, как добавить в рабочий поток поддержку отмены, читайте в рецепте 7.17.

Несмотря на то что использование исходного кода — это стандартный способ решения проблем с сетевыми рабочими потоками, у него есть свои ограничения. Например, любые изменения рабочего потока с большой вероятностью потребуют дополнительных этапов проектирования, программирования, тестирования и развертывания. В зависимости от доступности ресурсов (например, денег и времени) изменения могут стать затратными узкими местами в рабочем потоке.

И даже когда у вас достаточно ресурсов, применение исходного кода в качестве языка рабочего потока может привести к появлению множества специализированных решений, которые не получится легко повторно использовать (например, `onboardEmployee`, `onboardPartTimeEmployee`, `onboardTemporaryWorker` и т. д.). Если вам требуется персональная настройка существующих решений и/или вы хотите создавать более надежные рабочие потоки, ознакомьтесь с рецептами 7.4 и 7.5. Более сложное, но и более гибкое решение описано в рецепте 7.6.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.11. Проверка свойств данных в среде выполнения.
- Рецепт 4.15. Сохранение состояния.
- Рецепт 5.2. Предотвращение утечек внутренних моделей.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 6.7. Использование медиатипов для запросов данных.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.
- Рецепт 7.4. Описание рабочего потока через DSL.
- Рецепт 7.5. Описание рабочего потока через документы.

7.4. Описание рабочего потока через DSL

При написании множества рабочих потоков использование подхода с исходным кодом может не обеспечить достаточного масштабирования. В таких случаях лучше применить для проектирования, программирования, тестирования и развертывания ваших решений подходящий к рабочему потоку предметно-ориентированный язык (*domain specific language*, DSL).

Задача

Как выглядит DSL, понимающий рабочий поток, и когда есть смысл использовать его вместо нативных реализаций кода? Какие преимущества и проблемы для вашей организации несет реализация рабочих потоков с помощью DSL?

Решение

Как говорилось в рецепте 7.3, применение нативного исходного кода в качестве языка рабочего потока отлично подходит, когда речь идет о довольно простых решениях, которые не требуют особой индивидуальной настройки и/или редко изменяются. Однако бывают случаи, когда рабочий поток должен быть более гибким или вам нужно поддерживать в организации множество рабочих потоков. В таких сценариях обычно стоит реализовать среду с поддержкой обобщенных процессов рабочего потока, которая снизит сложность создания, тестирования и развертывания их в промышленной среде.

Пример

Вот пример обработки заказа и оплаты продовольственных товаров с помощью HyperCLI (https://twitter.com/hyper_cli) — DSL с поддержкой гипермедиа, который я написал специально для этой книги:

```
# Завершение оплаты товаров

# Загрузка конфигурации задачи
CONFIG LOAD grocery.config
grocery.exitOnError = true

# Загрузка общего состояния
REQUEST WITH-URL $$sharedstateURL$$
STACK PUSH WITH-RESPONSE ##sharedState#

# Получение обновления корзины
REQUEST WITH-URL $$scartURL$$
REQUEST WITH-FORM cartCheckout WITH-STACK
STACK PUSH WITH-RESPONSE ##sharedState##

# Получение деталей доставки
REQUEST WITH-URL $$shippingURL$$
REQUEST WITH-FORM bookShipping WITH-STACK
STACK PUSH WITH-RESPONSE ##sharedState##

# Проведение платежа
REQUEST WITH-URL $$paymentURL$$
```



```
REQUEST WITH-FORM settlePayment WITH-STACK  
STACK PUSH WITH-RESPONSE ##sharedState##
```

```
# Обновление общего состояния  
REQUEST WITH-URL $$sharedStateURL$$  
REQUEST WITH-FORM updateState WITH-STACK
```

```
# Завершение задачи
```

В этом примере выполняются три задания: `cartCheckout`, `bookShipping` и `settlePayment`. В начале и конце скрипта первым делом загружается общее состояние, после чего оно обновляется тремя заданиями и снова сохраняется. Обратите внимание также на файл `grocery.config`, загружаемый в начале. Он содержит различные URL, необходимые для выполнения задачи.

DSL с поддержкой гипермедиа вроде показанного в этом примере может упростить создание рабочих потоков, поскольку многие детали (например, обработка HTTP-запросов, обнаружение и заполнение форм в ответе и т. д.) обрабатываются языком напрямую. Если бы вы использовали код (см. рецепт 7.3), то вам потребовалось бы делать это самостоятельно (возможно, с помощью библиотеки обработки общего кода).

В качестве альтернативы то же самое можно проделать с помощью скрипта оболочки MS-DOS, Linux `bash` и т. п. Для этого потребуется чуть больше работы, но данный подход довольно эффективен. Вам придется задействовать множество утилит вроде `cURL` для обработки HTTP, `xslt` и/или `jq` для парсинга XML и JSON соответственно, а также использовать много `grep`, `awk` и прочих инструментов для управления строками.

Обсуждение

DSL могут упростить и обезопасить создание скриптов рабочего потока, так как ограниченность этих языков ведет к ограничению возможных действий. Это тоже может не нравиться программистам, которые привыкли к полноценно функциональным языкам программирования вроде Java, NodeJS и пр.

Использование скриптов оболочки и утилит повышает гибкость и расширяет возможности в сравнении со специализированным DSL вроде того, что был показан в примере. Но нужно помнить, что расширение возможностей ведет к увеличению ответственности. Создать безопасный и эффективный скрипт оболочки сложнее, и в этом случае вы с большей вероятностью внесете в него баги.

Независимо от того, задействуете вы специальный DSL или же собираете собственный скрипт с помощью оболочки, выражение рабочего потока в виде сценария может сократить общее время, необходимое на программирование, тестирование и развертывание рабочих потоков.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 4.11. Проверка свойств данных в среде выполнения.
- Рецепт 4.15. Сохранение состояния.
- Рецепт 5.14. Повышение скорости обработки с помощью клиентских идентификаторов.
- Рецепт 6.7. Использование медиатипов для запросов данных.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.
- Рецепт 7.3. Описание рабочего потока через код.
- Рецепт 7.5. Описание рабочего потока через документы.

7.5. Описание рабочего потока через документы

Наиболее надежным и устойчивым способом описания рабочего потока сервисов выступает использование декларативных документов. В текущем рецепте будет показано, как может выглядеть декларативный язык рабочих потоков, основанный на документах.

Задача

Нам нужно выразить рабочий поток декларативным образом — так, чтобы не включать пошаговый план выполнения, а описать всю необходимую работу и позволить потребителям документов самим решать, как ее выполнить. Как будет выглядеть подобный документ? Какие элементы он должен содержать и каковы пределы возможностей данного подхода при реализации рабочего потока в среде гипермедиа?

Решение

Лучшим способом описания рабочего потока декларативным образом является создание словаря (см. рецепт 3.3), отражающего свойства и действия, необходимые для выполнения всех требований совместимого с рабочим потоком сервиса, как описано в рецепте 7.1. Это означает поддержку заданий и задач, а также общего состояния. Такой документ должен упрощать для потребителей и производителей использование заголовков `correlation-id` и `request-id` для отслеживания управления выполняемой работой.



Подробнее о том, как описывать и реализовывать декларативный рабочий поток, читайте в рецепте 7.6.

Декларативные документы рабочего потока включают все свойства и действия (см. пример), необходимые для выполнения работы. Обычно такие документы имеют вид HTTP-ресурсов (например, `http://api.example.org/jobs/q1w2e3r4`). Кроме того, это могут быть сообщения в очереди (см. рецепт 7.19). Такая возможность особенно полезна при высокой частоте или большом количестве обрабатываемых задач, а также в случае прерывистости их потока, когда возникают узкие места или длительные периоды ожидания.

Пример

Далее показан образец документа, описывающий задачу рабочего потока в соответствии с рекомендациями рецепта 7.1 (в виде HTML-представления):

```
<html>
  <head>
    <title>Shopping Checkout Workflow</title>
  </head>
  <body>
    <h1>Shopping Checkout Workflow</h1>
    <div class="job">
      <span class="jobID">q1w2e3r4t5</span>
      <span class="jobStatus">working</span>
      <span class="jobDateCreated">2023-02-23:14:00:00</span>
      <span class="jobDateUpdated">2023-02-23:14:00:30</span>
      <span class="jobMaxTTL">300</span>
      <span class="jobDescription">...</span>
      <a href="..." rel="jobURL">Job</a>
      <a href="..." rel="jobSharedStateURL">Shared State</a>
      <a href="..." rel="jobProgressURL">Shared State</a>
      <a href="..." rel="jobSuccessURL">Success</a>
      <a href="..." rel="jobFailedURL">Failed</a>
      <a href="..." rel="jobContinueURL">Continue</a>
      <a href="..." rel="jobRestartURL">Restart</a>
      <a href="..." rel="jobCancelURL">Cancel</a>
    <div class="tasks">
      <div class="task">
        <span class="taskID">p0o9i8u7y6</span>
        <span class="taskStatus">completed</span>
        <span class="taskDateCreated">2023-02-23:14:00:00</span>
        <span class="taskDateUpdated">2023-02-23:14:00:30</span>
        <span class="taskMaxTTL">60</span>
        <span class="taskDescription">ComputeTaxes</span>
        <span class="taskMessage">Success (200)</span>
        <a href="..." rel="taskURL">Task</a>
        <a href="..." rel="taskStartURL">Start</a>
        <a href="..." rel="taskRollbackURL">Rollback</a>
        <a href="..." rel="taskRerunURL">Rerun</a>
        <a href="..." rel="taskCancelURL">Cancel</a>
      </div>
      ...
    </div>
  </body>
</html>
```

```
<div class="task">
  <span class="taskId">u8y7t6r5e4</span>
  <span class="taskStatus">pending</span>
  <span class="taskDateCreated">2023-02-23:14:00:00</span>
  <span class="taskDateUpdated">2023-02-23:14:00:30</span>
  <span class="taskMaxTTL">60</span>
  <span class="taskDescription">Schedule Shipping</span>
  <a href="..." rel="taskURL">Task</a>
  <a href="..." rel="taskStartURL">Start</a>
  <a href="..." rel="taskRollbackURL">Rollback</a>
  <a href="..." rel="taskRerunURL">Rerun</a>
  <a href="..." rel="taskCancelURL">Cancel</a>
</div>
</div>
</div>
</body>
</html>
```

Этот документ/словарь может использоваться *приложением-обработчиком* рабочего потока для реализации всех деталей, связанных с выполнением и мониторингом задачи. Подробнее об обработке каждого элемента действия читайте в рецепте 7.6.

Обсуждение

Преимуществом применения в рабочих потоках декларативных документов является то, что таким образом вы отделяете детали описания от обрабатывающего кода. Это позволяет часто корректировать код и постепенно улучшать его без изменения формата или содержимого документа.

Рабочие потоки на основе документов довольно эффективны в контексте интерфейсов совместимых с ними сервисов. Если вы пытаетесь включить в процесс несовместимые сервисы, то наверняка лучше использовать подход с DSL (см. рецепт 7.4) или кодом (см. рецепт 7.3).

Следите за тем, чтобы не внести в документы рабочих потоков императивное пошаговое описание работы. Такие элементы, как ветвление (`if..then..else`), проверка переменных (`when $state.value=="13"`) и прочие аналогичные конструкции, превращают декларативный документ в императивный язык программирования. Эти связанные с принятием решений элементы следует размещать внутри *заданий* (используемого сервиса), а не в документе *задачи*. Если же вы считаете, что вам нужен именно такой уровень программирования, то стоит применить подход с DSL (см. рецепт 7.4) или исходным кодом (см. рецепт 7.3).

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.

- Рецепт 3.9. Проектирование с учетом возможной отмены действий.
- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 4.15. Сохранение состояния.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.
- Рецепт 7.3. Описание рабочего потока через код.
- Рецепт 7.5. Описание рабочего потока через документы.

7.6. Поддержка RESTful-языка управления задачами

Одна из характерных сложностей при создании мультисервисных рабочих потоков в веб-среде связана с использованием последовательного и надежного языка для их описания и реализации. Текущий рецепт приводит базовый набор действий и метаданных для оркестрации независимых рабочих потоков в открытой веб-среде.

Задача

При создании масштабных рабочих потоков полезно бывает реализовывать обобщенный механизм, поддерживающий управление множеством задач и заданий. Что необходимо для поддержки языка управления задачами (job control language, JCL) и как можно уверенно совмещать независимые интерфейсы сервисов, работающие в разных уголках веб-среды, и управлять ими?

Решение

По мере повышения активности использования рабочего потока вам потребуется последовательный способ описания его задач, управления ими и их выполнения. Я уже рассказывал об основах компонуемых, совместимых с рабочим потоком сервисов (см. рецепт 7.1), и здесь приведу надежный JCL для веб-среды.



В течение многих лет я использовал различные вариации этого рецепта создания RESTful JCL, подстроенные под некие локальные нужды. Здесь же привожу обобщенный рецепт, в котором объединено множество функциональных возможностей, которые успешно применял в прошлом. Можете смело корректировать его в соответствии со своими потребностями.

СТОП, А КАК ЖЕ IPAAS?

Определенно, существуют и другие платформы рабочих потоков, которые вы можете задействовать для достижения необходимой интероперабельности. Здесь ввиду ограничения объема повествования я решил сосредоточиться на решении с применением гипермедиа, поскольку встречаю его довольно редко, хотя сам с успехом применял его в прошлом. Большинство других подходов приводят к жесткому прописыванию связей и общему использованию параметров, притом что подход на основе REST позволяет достичь тех же результатов без создания тесно связанной реализации.

Этот рецепт познакомит вас с базовым набором действий и метаданных для координирования независимых сервисов в веб-среде. Он также содержит действия для управления записями задач (их вывода, фильтрации, добавления, обновления и удаления). На рис. 7.2 показан управляющий рабочий поток в стиле RESTful.

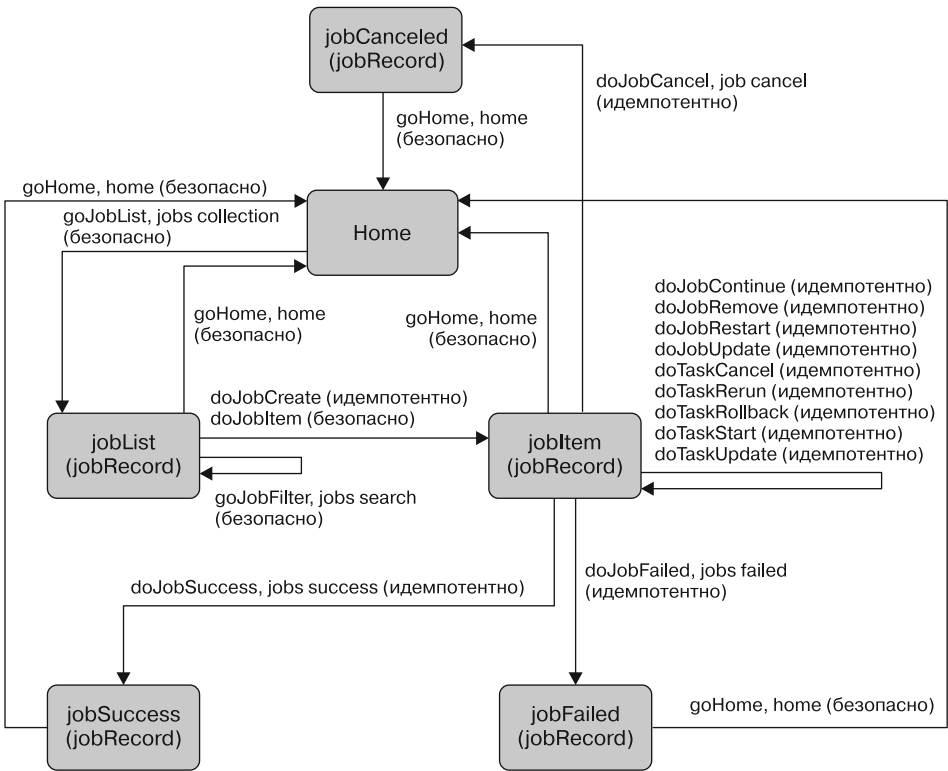


Рис. 7.2. Управляющий рабочий поток в стиле RESTful

Выполнение рабочего потока с помощью RESTful JCL

При определении и выполнении записей рабочего потока задействуются два ключевых элемента.

Задания

Задание — это одно действие в рамках рабочего потока (`computeTaxes`, `DoCredit-Check`, `Update CustomerAccounts` и т. д.). Задание поддерживает выполнение (`taskStartURL`), повтор (`taskRerunURL`), откат (`taskRollbackURL`) и отмену (`taskCancelURL`).

Задачи

Каждая задача содержит одно или несколько заданий. Набор заданий в каждой задаче *должен* допускать параллельное выполнение. Задача поддерживает перезапуск (`jobRestartURL`), продолжение (`jobContinueURL`) и отмену (`jobCancelURL`). У нее также есть URL для вызова при успешном завершении (`jobSuccessURL`) и неудаче (`jobFailedURL`).

Для выполнения задачи ее документ нужно отправить в механизм рабочего потока. После получения этого документа механизм в процессе обработки заданий будет возвращать код состояния **202 Accepted** (см. рецепт 7.15), а в конечном счете — ответ о завершении с кодом **200 OK** или **4xx/5xx**. После выполнения задачи следует активировать соответствующую ссылку (`jobSuccessURL` или `jobFailedURL`), а ее документ *можно* отправить в архив для использования в будущем или повторного выполнения.

Подходящие механизмы рабочего потока *должны* быть подготовлены для обработки `jobCancelURL`, `jobContinueURL` и `jobRestartURL`. При этом они должны поддерживать свойства `taskStartURL`, `taskRollbackURL`, `taskRerunURL` и `taskCancelURL`, а также отслеживание или обновление прогресса посредством `taskStatus` и `jobStatus`. *Желательно*, чтобы эти механизмы мониторили значения `jobMaxTTL` и `taskMaxTTL`, отменяя задачу в случае превышения максимально допустимого времени выполнения.

Задания, которые должны быть выполнены в строгой последовательности, *необходимо* разделять по отдельным документам задач, которые можно будет выполнять в заданном порядке при успешном завершении каждого.

Управление записями RESTful JCL

Помимо реализации *выполнения* рабочих потоков, вам нужно поддерживать создание, фильтрацию и удаление их документов. В текущем рецепте приводится ряд действий, предназначенных для управления рабочими потоками.

jobList

Возвращает список записей задач.

jobFilter

Выполняет поиск записей задач.

jobRead

Возвращает одну запись задачи.

jobCreate

Добавляет в коллекцию новую запись задачи.

jobUpdate

Изменяет запись задачи.

jobRemove

Удаляет запись задачи.

Обратите внимание на то, что этот список не включает добавление, редактирование и удаление заданий. В этом случае минимальным управляемым ресурсом в системе выступает `jobItem`. При желании можно реализовать дополнительный набор действий (по сути, применив рецепт 7.11) на уровне заданий. Но по своему опыту могу сказать, что это пригождается редко.

Общее состояние, идентификаторы корреляции и прогресс

Здесь не было показано внедрение поддержки передачи состояний между заданиями. Подробности ее реализации описаны в рецепте 7.2.

Еще одним важным элементом выступает обмен `jobID` и `taskID` через заголовки в среде выполнения. Как описывалось в рецепте 7.1, любой совместимый с рабочим потоком сервис *должен* передавать `jobID` (в виде заголовка `Correlation-ID`) и `taskID` (в виде заголовка `Request-ID`) при каждом действии в рамках задачи.

Совместимые с рабочим потоком сервисы этого рецепта должны поддерживать отчет о прогрессе (см. рецепт 7.7) и, возможно, паттерн «Призыв на помощь» (см. рецепт 7.18).

Обсуждение

RESTful JCL работает, только если все сервисы, которые вы включаете в качестве заданий, следуют правилам, изложенным в рецепте 7.1. При этом хорошо то, что вы можете реализовать совместимые с рабочим потоком сервисы, не поддерживая полноценный язык управления задачами. RESTful JCL идет уже *после* создания таких сервисов. Если вы планируете реализацию систем рабочих потоков, сначала уделите внимание совместимым интерфейсам сервисов, а затем переходите к изучению вариантов JCL.

Сервисы, отвечающие за создание рабочего потока, *можно* отделить от сервисов, реализующих его выполнение. Редактор и механизм выполнения необязательно должны размещаться в одном интерфейсе сервиса. Процессы редактирования и выполнения обычно лучше разделять, поскольку они могут осуществляться в собственном индивидуальном темпе.

ВЫ УВЕРЕНЫ, ЧТО ХОТИТЕ ПОЙТИ ЭТИМ ПУТЕМ?

Реализовать общую систему управления задачами вроде описанной здесь непросто. У этого рецепта есть альтернативы в виде готового ПО и SAAS-продуктов (Software-as-Service — программное обеспечение как услуга). Решение создать собственный JCL не следует принимать поспешно. Если вам нужна поддержка нескольких платформ хостинга, включая собственные локальные серверы, то этот рецепт окажется полезным. Если же вы размещаете сервисы на одной платформе, например AWS, Azure, Google и т. д., то использование предоставляемого ею инструментария для управления рабочим потоком, пожалуй, будет более удачной идеей.

Суть этого рецепта в том, что вам нужно создавать рабочие потоки, задействующие интерфейсы сервисов, которые вы не создавали и не контролируете (см. приложение А). Даже если вы работаете с сервисами, разработанными другими командами вашей организации, RESTful JCL может облегчить этот процесс за счет повторного использования существующих сервисов, применяемых другими отделами компании.



Этот рецепт отлично подойдет, когда вы придерживаетесь описания рабочего потока в виде документов (см. рецепт 7.5). Его реализация окажется гораздо сложнее, если вы выражаете рабочие потоки через код (см. рецепт 7.3) или DSL (см. рецепт 7.4).

Читайте также

- Рецепт 3.8. Проектирование с учетом возможности повторения действий.
- Рецепт 3.9. Проектирование с учетом возможной отмены действий.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 4.15. Сохранения состояния.
- Рецепт 5.7. Публикация полноценных словарей для клиентов.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочими потоками.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.
- Рецепт 7.17. Поддержка локальной отмены или отката.
- Рецепт 7.18. Призыв на помощь.

7.7. Предоставление ресурса прогресса, отражающего выполнение рабочих потоков

При выполнении и мониторинге задач рабочего потока желательно предоставлять отчет о прогрессе, чтобы потребители и производители API могли отслеживать процесс и возможные проблемы. В текущем рецепте приводится пример ресурса прогресса, который вы можете использовать в своих реализациях рабочих потоков.

Задача

Как мне отслеживать длительно выполняющиеся задачи рабочих потоков? Где можно просмотреть детали транзакций и узнать, не произошло ли ошибок? Что нужно для создания общего ресурса отслеживания прогресса рабочего потока, который будет работать независимо от того, выражаете вы этот рабочий поток с помощью кода (см. рецепт 7.3), DSL (см. рецепт 7.4) или документов (см. рецепт 7.5)?

Решение

Как и во многих других решениях из этой книги, вы зачастую можете решить свою проблему с помощью HTTP-ресурса метаданных (см. рецепты 5.5, 5.11 и 5.12). В данном случае мы создадим ресурс отслеживания прогресса рабочего потока, который будет содержать все данные о выполнении, связанные с задачей рабочего потока и ее заданиями (подробнее о задачах и заданиях читайте рецепты 7.1 и 7.6).

У любой задачи рабочего потока должен быть ресурс для отслеживания ее прогресса. Этот ресурс можно привязать к задаче с помощью `jobID`, который зачастую совпадает со значением `correlation-id`, хранящимся в HTTP-заголовке. Этот ресурс может выступать сборной точкой для любых отчетов о прогрессе или иных аспектах отслеживания задачи. Вам следует мониторить метаданные каждой задачи и каждого ее задания по мере их выполнения. На рис. 7.3 показан прогресс рабочего потока.

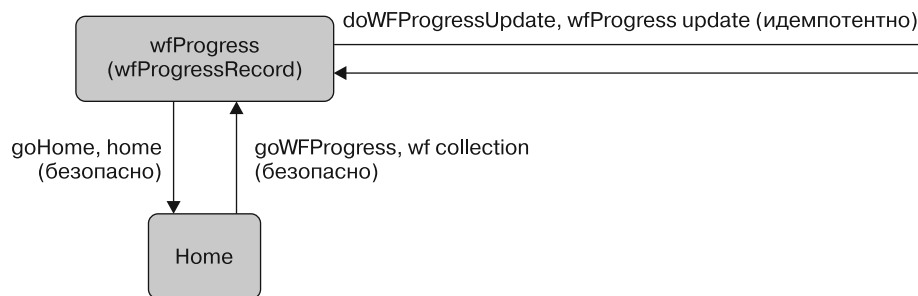


Рис. 7.3. Прогресс рабочего потока

Важные метаданные задачи.

jobID

Уникальный идентификатор для ресурса задачи (зачастую это `correlation-id`).

jobURL

URL документа ресурса задачи.

jobDescription

Человекочитаемый текст, описывающий задачу.

jobStatus

Указывает статус задачи (в ожидании, выполняется, завершена или провалена).

jobDateCreated

Указывает дату/время (UTC) создания ресурса задачи.

jobDateUpdated

Указывает последние дату/время (UTC) изменения ресурса задачи.

jobMaxTTL

Указывает максимальное количество времени (в секундах), в течение которого ресурс считается валидным.

Важные метаданные задания.

taskID

Уникальный идентификатор задания (зачастую `request-id`).

taskURL

URL ресурса задания.

taskDescription

Человекочитаемый текст, описывающий задание.

taskStatus

Указывает статус задания (в ожидании, выполняется, завершено или провалено).

taskStartDateTime

Указывает дату/время (UTC) начала обработки задания.

taskStopDateTime

Указывает дату/время (UTC) прекращения обработки задания.

taskMaxTTL

Указывает максимальное количество времени (в секундах), в течение которого задание считается валидным.

taskMessage

Человекочитаемый текст, содержащий все связанные с заданием сообщения.

Последний элемент каждого задания (`taskMessage`) может содержать строки отчета о прогрессе, например все попытки его завершения. Для большинства нужд мониторинга достаточно будет записи текстовых строк в поле `taskMessage`. Еще один полезный способ регистрации этой информации — включение деталей HTTP-запроса (URL, заголовков и тела) и HTTP-ответа (кода состояния, заголовков и тела).



Если вы включите в ресурс прогресса HTTP-детали, то можете случайно создать утечку, которая будет угрожать безопасности. Обеспечьте, чтобы эту часть ресурса прогресса могли читать только должным образом авторизованные пользователи.

Ресурс прогресса можно отформатировать для отображения любых стандартных медиатипов (HTML, Collection + JSON, HAL, SIREN, UBER и т. д.). При отображении данных прогресса следует также включать свежую ссылку и данные кэширования, сообщая потребителям API о свежести ресурса.

Пример

Вот пример ресурса прогресса в формате Collection + JSON:

```
**** REQUEST ****
GET /jobs/q1w2e3r4t5;progress
Accept: application/vnd.collection+json
...

**** RESPONSE ****
200 OK
Content-type: application/vnd.collection+json
Link: <<http://api.example.org/jobs/q1w2e3r4t5;progress>>; rel="refresh"
Cache-control: max-age 10000, public

{"collection": {
  "title": "Job q1w2e3r4t5 Progress",
  "links": [
    {"rel": "refresh", "href": "http://api.example.org/jobs/q1w2e3r4t5;progress"}
  ],
  "items": [
    {"id": "q1w2e3r4t5",
     "type": "job",
     "data": [
       {"name": "jobID", "value": "q1w2e3r4t5"},
       {"name": "jobURL", "value": "http://api.example.org/jobs/q1w2e3r4t5"},
       {"name": "jobDescription", "value": "Employee Onboarding Job"},
       {"name": "jobStatus", "value": "working"},
       {"name": "jobDateCreated", "value": "2024-05-01:13:12:11"},
       {"name": "jobDateUpdated", "value": "2024-05-01:13:15:14"},
       {"name": "jobMaxTTL", "value": "30000"}
     ]
    }
  ]
},
},
```

```

{"id": "u8y7t6r5",
  "type": "task",
  "data": [...]}
,
{"id": "5r4e3w2q",
  "type": "task",
  "data": [...]}
,
{"id": "p0o9i8u7",
  "type": "task",
  "data": [
    {"name": "taskID", "value": "p0o9i8u7"},
    {"name": "taskURL", "value": "http://api.example.org/jobs/q1w2e3r4t5#p0o9i8u7"},
    {"name": "taskDescription", "value": "Credit Check"},
    {"name": "taskStatus", "value": "completed"},
    {"name": "taskStartDateTime", "value": "2024-05-01:13:13:11"},
    {"name": "taskStopDateTime", "value": "2024-05-01:13:14:11"},
    {"name": "taskMaxTTL", "value": "5000"},
    {"name": "taskMessage", "value": "completed successfully."}
  ]
}
]
}}

```

Обратите внимание, что в этом примере предоставляется только базовая информация о прогрессе каждого задания, например **"completed successfully"**. Это снижает вероятность утечки личных данных и интеллектуальной собственности, которая может привести к нарушению безопасности.

После завершения задачи эти записи прогресса следует отправить в архив, где при необходимости их можно будет просмотреть.

Обсуждение

Ресурс прогресса рабочих потоков преследует одну простую цель — предоставить дополнительную обратную связь вызывающим API потребителям по мере выполнения задач и содержащихся в них заданий. Поэтому желательно не усложнять такие ресурсы. Не пытайтесь внести в них слишком много данных и ни в коем случае не включайте внутреннюю или приватную программную информацию, так как это может создать угрозу для безопасности.

Ресурсы прогресса *не следует* использовать в качестве журнала трассировки ваших сервисов, и они не должны включать какие-либо внутренние значения и/или отладочную информацию. Если вам нужны эти детали, задействуйте средства, доступные внутри вашей платформы, не раскрывая эту информацию через HTTP.

У вас может возникнуть желание включить в отражающий прогресс документ несколько ссылок, которые позволят вызывающему перезапустить задание, выполнить откат или задействовать другие возможности управления рабочим потоком.

Так делать нежелательно. Если вы используете для описания рабочего потока код (см. рецепт 7.3) или DSL (см. рецепт 7.4), то эти дополнительные возможности могут вызвать конфликт и путаницу в среде выполнения.

Читайте также

- Рецепт 3.3. Предоставление информации о предметной области через публикуемые словари.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 5.5. Оповещение клиента о поддерживаемых настройках.
- Рецепт 5.10. Публикация метаданных API.
- Рецепт 6.5. Возвращение метаданных в ответах на запросы.
- Рецепт 7.14. Оптимизация запросов с помощью сохраненных повторов.

7.8. Возвращение всех сопутствующих действий

Бывают случаи, когда список возможных действий довольно велик. Иногда один ответ может вызвать более десятка различных переходов состояния с помощью ссылок или форм. Возвращение всех этих форм с заполненными предустановленными значениями для каждого запроса может оказаться очень затратным и повлиять на время отклика сервиса.

Вместо возврата всех возможных переходов состояния на каждый запрос можно создать отдельный ресурс ответа, который будет делать это независимо от текущего рабочего потока или состояния приложения. Это уменьшит размер полезной нагрузки ответа, ускорит время отклика и стандартизирует повторяемый способ получения свежего списка доступных переходов.

Задача

В некоторых приложениях возможное число переходов состояния (или ссылок) в любой отдельный момент может оказываться довольно большим. Типичные взаимодействия рабочего потока вроде страничного перебора списка (см. рецепт 7.11) могут допускать выполнение в один момент времени множества различных действий, таких как `top`, `next`, `previous`, `last`, `item`, `create`, `refresh` и `self`. Сюда не входят переходы в другие коллекции или связанные действия (`list-users`, `list-companies`, `list-reports` и т. д.).

Перечисление всех этих вариантов в каждом ответе может стать утомительным процессом и значительно увеличить полезную нагрузку. В этом случае необходим

стандартизированный способ запроса всех возможных переходов в качестве отдельного шага, например перехода, чтобы уменьшить размер полезной нагрузки, не снижая объем ценной информации.

Решение

Простым способом решить проблему с излишним числом ссылок в ответе будет предоставление отдельного ресурса, всегда содержащего все возможные действия в приложении на конкретный момент времени. Это можно сделать с помощью значения ссылочной связи **related** системы IANA (<https://oreil.ly/IQuev>). В этом случае интерфейс сервиса можно спроектировать так, чтобы он предоставлял важные ссылки в теле полезной нагрузки, а все другие возможные ссылки — в виде отдельного ресурса (подробнее в примере далее).

Преимущество этого подхода в том, что стоящий за таким интерфейсом сервис при сборке содержимого ресурса **related** может учитывать различные контекстные переменные. Сюда входит состояние данных сервиса (доступны ли они в данный момент?), состояние обработки приложения (не находится ли сервис в режиме обслуживания?), состояние клиентского приложения (находится ли клиент в процессе выполнения набора действий?) и даже вызывающий субъект (является ли этот авторизованный пользователь админом?).

На рис. 7.4 показана простая схема ссылки **related**, которую можно добавлять в каждый ответ. Подробнее — в разделе «Пример».

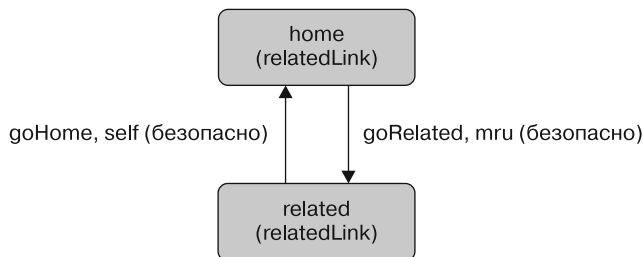


Рис. 7.4. Рабочий поток с переходом **related**

Пример

Добавляя в ответ ссылку **related**, вы можете спрятать дополнительные переходы, которые не хотите раскрывать в базовом ответе. В этом примере в ответы всегда добавляется переход **self**. Есть также ссылка **related**, которую клиенты могут использовать для получения более полного списка всех возможных действий:

```

{
  "collection" : {
    "links": [

```

```

    {
      "name": "self",
      "rel" : "home self",
      "href": "https://api.example.org/"
    },
    {"$comment" : "link that points to the list of related actions"},
    {
      "name": "related",
      "rel": "collection",
      "href": "https://api.example.org/related"
    },
    {
      "name": "list",
      "rel": "collection",
      "href": "https://api.example.org/list"
    }
  ],
  "items": [ ... ],
  "queries": [ ... ],
  "template": { ... }
}

```



Утвержденный управлением IANA медиатип `text/uri-list` — это прекрасный формат для возврата списков сопутствующих ссылок.

Когда клиентское приложение переходит по ссылке **related**, в ответе возвращается полный список доступных действий в текущем контексте вызова:

```

{
  "collection" : {
    "links": [
      {
        "name": "self",
        "rel" : "related",
        "href": "https://api.example.org/"
      },
      {"$comment" : "list of related available actions"},
      {
        "name": "list",
        "rel": "collection",
        "href": "https://api.example.org/list"
      },
      {
        "name": "filter",
        "rel": "collection",
        "href": "https://api.example.org/filter"
      }
    ]
  }
}

```



```

    {
      "name": "create",
      "rel": "collection",
      "href": "https://api.example.org/create-form"
    },
    {
      "name": "needs-approval",
      "rel": "collection",
      "href": "https://api.example.org/list?status=needs-approval"
    },
    {
      "name": "home",
      "rel": "home",
      "href": "https://api.example.org/"
    }
  ],
  "items": [ ... ],
  "queries": [ ... ],
  "template": { ... }
}
}

```

Стоит отметить, что в этих двух примерах ссылочная связь `list` присутствует в обоих списках. Нет такого правила, которое требовало бы указания ссылки только в одном месте. Кроме того, содержимое ресурса `related` со временем может меняться, и список ссылок будет отражать текущее состояние сервиса, клиентов и пользовательский контекст.

Когда интерфейсы сервисов поддерживают паттерн `related`, клиентские приложения можно легко запрограммировать на поиск ссылки или формы, которой изначально не было в ответе интерфейса сервиса, например:

```

// Получение ответа по корневому URL и выполнение расчета
var output = request(apiRootUrl);
var link = output.findLink("checkout");
if (link === null) {
  link = output.findLink("related") {
    if(link !== null) {
      link output.findLink("checkout");
    }
  }
}
if(link !== null) {
  runCheckout(output, stateVariables);
}
else {
  runUnableToCheckout(output, stateVariables);
}

```

При желании можно написать также код потребителя API, скрывающий использование `related`. В таком коде достаточно будет прописать `output.fin`

`dLink("related");`, чтобы он сначала выполнял поиск в локальном ответе и при необходимости задействовал ссылку `related`.

Обсуждение

Этот рецепт аналогичен рецепту 7.9 в том, что помогает оптимизировать для клиентских приложений поиск ссылок. Люди очень хорошо справляются с нахождением ссылок и созданием собственных, чего нельзя сказать о машинах. Текущий рецепт позволяет компенсировать недостатки машин, не прибегая к помощи искусственного интеллекта.

Его разумно использовать, когда в любой отдельный момент возможно множество разных переходов. Проектировщики интерфейсов могут просто создавать нужный набор ответов, не включая все возможные переходы в каждый ответ. Такой подход уменьшает беспорядок и расхождения в отношении того, какие ссылки и формы нужно включать в ответы.

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.7. Использование медиатипов для запроса данных.
- Рецепт 7.9. Возвращение последних использованных ресурсов.
- Рецепт 7.14. Оптимизация запросов с помощью сохраненных повторов.

7.9. Возвращение последних использованных ресурсов

Вы можете повысить удобство для потребителей API включением в ответы списка последних использованных ссылок/форм. Это поможет программистам лучше понять типичные схемы применения вашего интерфейса и упростит доступ к наиболее актуальным ссылкам, которые им нужны.

Задача

Разработчикам и проектировщикам бывает сложно договориться о том, какие ссылки и формы должны возвращаться в ответах с поддержкой гипермедиа. Проектировщики обычно пытаются создать интерфейс, который будет и удобен, и полезен. Как правило, это означает исключение некоторых деталей. Разработчики же и архитекторы зачастую стремятся к полноте и готовы в каждый ответ включить

все возможные опции (см. рецепт 7.8). Как правильно сбалансировать эти два подхода с сохранением в ответах полезных ссылок? Что произойдет, если самые нужные пользователям ссылки со временем будут изменяться?

Решение

Этот рецепт тесно связан с рецептом 7.8, в котором предлагается способ выносить (*stash*) множество сопутствующих ссылок в отдельный документ, чтобы не раздувать ответы, но при этом предоставлять доступ ко всем возможным действиям. Тем не менее если зайти слишком далеко, то большинство ответов с гипермедиа будут содержать всего одну ссылку (*related*) и клиентам для реализации своих задач потребуется совершать дополнительные HTTP-запросы. Не самая удачная идея. Вместо этого стоит отслеживать последние использованные (*most recently used*, MRU) ссылки на ресурсы и включать их в ответы.

ИСТОРИЯ MRU

Подход с идентификацией и раскрытием последних примененных возможностей приложения стал популярным еще в графических интерфейсах пользователей в 1980–1990-х годах. На фоне увеличения числа опций меню возникла идея по улучшению интерфейса за счет сокращения их списка, отображаемого для пользователя. Внедрение паттерна MRU позволило предоставлять последние редактированные документы, задействованные команды и даже запрошенные приложения. Этот принцип использовался также для кэширования сервисов с целью отслеживания последних запрошенных ресурсов.

Этот рецепт можно применять совместно с рецептом 7.8, включая дополнительные ссылки на часто применяемые ответы. При таком подходе интерфейс сервиса может отслеживать часто запрашиваемые ресурсы и включать в ответы ссылки на них. Например, если клиентские приложения обычно запрашивают ресурс *list*, а затем ресурс *add*, то интерфейс сервиса может отследить этот паттерн и автоматически включать ссылку на ресурс *add* в ответ с ресурсом *list*.

Такое отслеживание часто используемых ресурсов можно реализовать на уровне приложения или пользователя. Это позволит настроить список MRU для контекста вызова, включая права доступа авторизованного пользователя.

Налаживание отслеживания MRU для контекста (на уровне сервиса или потребителя) требует определенной работы, но особой сложности не представляет. Для этого нужен лишь список последних запрошенных ресурсов. По мере запроса каждого ресурса его ссылка помещается в список (URL и количество). Когда приходит время представления ответа с ресурсом, интерфейс сервиса может обратиться к списку MRU и при необходимости после фильтрации контекста на основе прав доступа вставить в ответ последние использованные ссылки.

Возврат MRU уменьшит зависимость от рецепта 7.8 и поможет новым разработчикам узнать, какие действия в сервисе наиболее востребованы. Список MRU обычно имеет вид стандартных ссылок в ответе (рис. 7.5). Но вы также можете создать ресурс `mru`, аналогичный ресурсу `related` из рецепта 7.8.

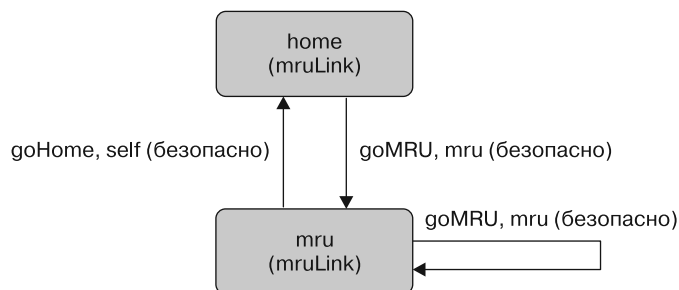


Рис. 7.5. Рабочий поток MRU

Пример

Далее приведен пример ответа (в формате Collection + JSON), где представлены и ссылка MRU ("`name`": "`mru`"), и несколько последних использованных ссылок, в которых `rel` установлена на `mru`:

```

{
  "collection" : {
    "links": [
      {
        "name": "self",
        "rel": "home self",
        "href": "https://api.example.org/"
      },
      {"$comment": "link that points to the MRU list"},
      {
        "name": "mru",
        "rel": "collection",
        "href": "https://api.example.org/mru"
      },
      {"$comment": "some MRU links in the response"},
      {
        "name": "list",
        "rel": "mru collection",
        "href": "https://api.example.org/list"
      },
      {
        "name": "pending",
        "rel": "mru collection",
        "href": "https://api.example.org/list?status=pending"
      }
    ]
  }
}
```

```
{
  "name": "home",
  "rel": "mru home",
  "href": "https://api.example.org/"
},
"items": [ ... ],
"queries": [ ... ],
"template": { ... }
}
```

Сервисы, реализованные с помощью рецепта MRU, должны возвращать в каждом ответе как минимум ссылку MRU. Это будет напоминать клиентам, куда они могут перейти для получения дополнительных деталей. Можно также возвращать в ответах первые несколько MRU (см. пример). Эти возвращенные MRU должны отражать текущий пользовательский контекст (например, последние задействованные авторизованным в данный момент пользователем ссылки), а также контекст сервиса (например, последние использованные ссылки, валидные в текущий момент). На этот момент актуальными могут оказаться не все последние задействованные ссылки. Например, может отсутствовать возможность запросить список клиентов в период обслуживания сервисом базы данных.

Обсуждение

Отслеживание последних использованных URL можно реализовать с помощью простого стека по типу «первым вошел, первым вышел» (first in, first out, FIFO) фиксированной длины (я в большинстве случаев применяю глубину от трех до пяти элементов). После завершения запроса можно добавить значение URL в стек и при необходимости отбрасывать нижние URL, когда список становится слишком большим. Тогда вы сможете просто возвращать этот список URL в ответе.

Не пытайтесь запомнить слишком много информации о прошлых запросах. Например, нежелательно включать информацию заголовка или пытаться понять цель запроса с помощью других метаданных. Просто сохраняйте ссылки и возвращайте их.

Применение паттерна MRU прекрасно поможет потребителям API понять, какие вызовы к вашему сервису совершаются чаще всего. Этот паттерн можно рассматривать как вспомогательную возможность для неопытных пользователей. В качестве его реализации можно предоставлять опцию переключения, позволяющую потребителям включать или отключать MRU в ответах. Подробнее о ее реализации читайте в рецепте 5.5.

Рецепт 5.5 можно применять также для включения переменной, отражающей размер списка MRU в каждом ответе. Как правило, возвращения трех — пяти последних ссылок вполне достаточно, так что можно смело задействовать это количество в качестве предустановленного.

Будет хорошей идеей предоставлять значения MRU в виде простых ссылок, а не форм. Это сократит количество работы, которую серверу нужно проделать для их включения, а также содержимое в ответах.

При отображении MRU в ответе можно повысить удобство для потребителей API, определив их список с помощью значений стандартных ссылочных связей ("mru"), блока внутри ответа (`<div id="mru">...</div>`) или другого метода, типичного для формата ответа.

Нежелательно возвращать значения MRU в HTTP-заголовках, используя элемент `link`. Список MRU может становиться большим, и лучше отображать его в теле сообщения. В качестве альтернативы можно включать один элемент `link`, указывающий на отдельный список MRU:

```
**** REQUEST ****
GET /users/list HTTP/1.1
Host: api.company.org
Accept: application/vnd.siren
Authentication: BASIC o9eir8tu5ry6362yeir...

**** RESPONSE ****
200 OK HTTP/1.1
Content-Type: application/vnd.siren
Link: <https://company.org/users/mru>;rel="mru"

...

```

Если ваш сервис поддерживает идентификацию пользователей, то можете встроить в него поддержку отслеживания MRU по отдельному пользователю. Это принесет в список ответов дополнительную возможность индивидуальной настройки.

Даже если вы не хотите отслеживать именно часто используемые URL, можете применить рецепт MRU для возврата URL, которые с наибольшей вероятностью окажутся полезными, например домашнего URL, списка коллекций URL и т. д.

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.7. Использование медиатипов для запросов данных.
- Рецепт 7.8. Возвращение всех сопутствующих действий.
- Рецепт 7.14. Оптимизация запросов с помощью сохраненных повторов.

7.10. Паттерн «текущая работа» с сохранением состояния

Иногда вам нужно собрать от потребителей API большой объем данных, так что одного ресурса FORM недостаточно. В таких случаях поможет набор сопутствующих шагов для сбора и оценки данных. Этот рецепт познакомит вас с паттерном «текущая работа», используемым для сбора данных и отправки итоговых результатов.

Задача

Как собрать большой объем данных из небольших связанных частей и отправить собранный результат за один шаг? Что будет подразумевать сбор данных, когда несколько человек заполняют один и тот же ресурс? Что, если процесс сбора, предположительно, займет много времени из-за ожидания данных от других сторон?

Решение

Рецепт «текущая работа» (work-in-progress, WIP) — прекрасный способ собрать большой объем данных в течение некоторого отрезка времени, после чего проанализировать их и отправить на обработку. Этот подход позволяет определить долгосрочный рабочий документ, а затем посредством набора из одного или нескольких взаимодействий продолжить собирать входные данные, пока не накопится их объем, достаточный для отправки сервису на обработку.

Этот рецепт можно использовать также для управления несколькими потоками документов на лету. К примеру, вам может потребоваться запустить два или три документа WIP для выполнения задачи вроде одобрения кредита на покупку.

Вот список действий в рецепте WIP.

`listWIP`

Переход к ресурсу списка WIP.

`filterWIP`

Получение отфильтрованного списка записей WIP.

`createWIP`

Создание нового ресурса WIP.

`readWIP`

Получение одного ресурса WIP.

`updateWIP`

Обновление одного ресурса WIP.

`cancelWIP`

Отмена существующего ресурса WIP.

shareWIP

Передача/сохранение ресурса WIP.

submitWIP

Отправка полноценного ресурса WIP (для финального принятия).

Первые два действия (`listWIP` и `filterWIP`) помогают управлять списком документов WIP. Следующие (`createWIP`, `read WIP`, `updateWIP` и `cancelWIP`) работают с одним документом WIP. `shareWIP` позволяет отправлять документ другому пользователю или машине для обработки, а `submitWIP` отвечает за отправку на обработку завершённого документа WIP.



Как правило, этот рецепт работает совместно с рецептом 7.12, который можно использовать для сбора входных данных.

На рис. 7.6 показан рабочий поток WIP.

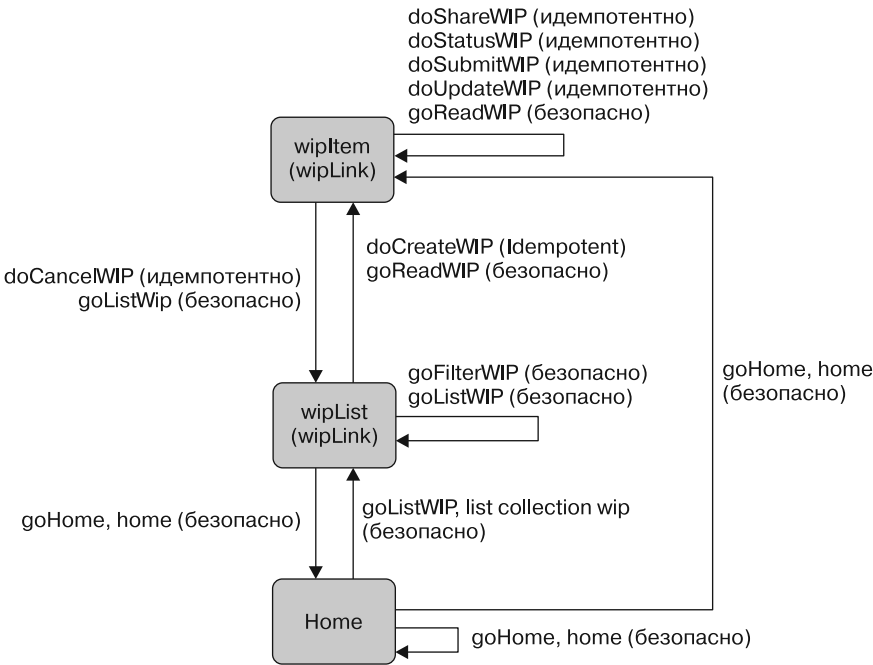


Рис. 7.6. Рабочий поток WIP

Возьмем в качестве примера адаптацию новых сотрудников. Для этого может потребоваться множество записей данных от множества сторон, возможно, придется ожидать подтверждения некоторых других входных данных, которые могут

задерживаться на один или два дня. Для обработки всего этого можно использовать документ WIP. Вам может потребоваться собрать данные о сотрудниках, информацию с собеседований из отдела кадров, результаты проверки их биографий от третьей стороны и детали для заключения контракта о найме. Вы можете использовать рецепт WIP, чтобы запустить в работу документ, который будет содержать все эти входные данные и предоставлять их различным сторонам внутри организации для редактирования. В итоге кто-то обладающий необходимыми полномочиями сможет отправить подготовленные данные для финальной обработки.

Пример

Вот пример реализации сценария с адаптацией новых сотрудников:

**** REQUEST ****

GET /onboarding/

Accept: application/vnd.siren+json

**** RESPONSE ****

200 OK

Content-Type: application/vnd.siren+json

```
{
  "title" : "Onboarding",
  "links" : [
    {"rel" : ["listwip"], "href" : "/onboarding/wip/"}
    ...
  ]
}
```

Переход по ссылке `listwip` вернет список из всех текущих обрабатываемых документов WIP:

**** REQUEST ****

GET /onboarding/wip

Accept: application/vnd.siren+json

**** RESPONSE ****

200 OK

Content-Type: application/vnd.siren+json

```
{
  "title" : "Onboarding WIP List",
  "links" : [
    {"rel" : ["self", "listwip"], "href" : "/onboarding/wip/"},
    {"rel" : ["home"], "href" : "/onboarding/"},
    ...
  ],
  "actions" : [
    {
      "name" : "createwip",
      "method" : "PUT",
      "href" : "/onboarding/wip/p0o9i8", "fields" : [
        {"name" : "dateCreated", "value" : "2024-MAY-30"}],
    }
  ]
}
```

```

        {"name" : "wipowner", "value" : "Mandy Mayberry"}
    ]
}
...
]
}

```

Форма `createwip` может собирать основные данные вроде даты/времени создания документа и создавшего его пользователя, после чего предоставлять через интерфейс инструкции для сбора данных.

Выглядеть это может так:

```

**** REQUEST ****
GET /onboarding/wip/p009i8
Accept: application/vnd.siren+json

**** RESPONSE ****
200 OK
Content-Type: application/vnd.siren+json

{
  "title" : "Onboarding Mick Mickelsen",
  "links" : [
    {"rel" : ["self", "listwip"], "href" : "/onboarding/wip/"},
    {"rel" : ["home"], "href" : "/onboarding/"},
    {"rel" : ["employee"], "href" : "/onboarding/wip/p009i8/employee"},
    {"rel" : ["interviews"], "href" : "/onboarding/wip/p009i8/interviews"},
    {"rel" : ["background"], "href" : "/onboarding/wip/p009i8/background"},
    {"rel" : ["contract"], "href" : "/onboarding/wip/p009i8/contract"},
    ...
  ],
  "actions" : [
    {
      "name" : "cancelwip",
      "method" : "DELETE",
      "href" : "/onboarding/wip/p009i8",
    },
    {
      "name" : "updatewip",
      "method" : "PUT",
      "href" : "/onboarding/wip/p009i8", "fields" : [
        {"name" : "status", "value" : "working"},
        ...
      ]
    },
    {
      "name" : "submitwip",
      "method" : "PUT",
      "href" : "/onboarding/wip/p009i8", "fields" : [
        {"name" : "status", "value" : "submitted"}
      ]
    },
    ...
  ]
}

```

Обратите внимание на то, что тело ответа содержит нативные действия WIP (`cancel`, `submit`, `update`), а раздел `link` документа включает несколько вторичных шагов для завершения процесса. Как правило, каждый шаг (`employee`, `interviews` и пр.) содержит форму для входных данных, непосредственно поддерживающую действие `updateWIP`. Таким образом, каждый шаг выражается в виде ресурса, который обновляет общее состояние этого документа WIP.



Текущий рецепт имеет множество вариаций и способов реализации. Обязательно ознакомьтесь с разделом «Обсуждение», где приводятся рекомендации для его эффективного применения.

В некоторых случаях часть работы приходится поручать третьей стороне, что может подразумевать также передачу документа WIP. Для этого существует действие `shareWIP`. В зависимости от того, как работает ваша реализация, можете просто изменить свойство «владелец» документа WIP, чтобы он отображался в списке задач нового пользователя. Либо можно сгенерировать электронное письмо с просьбой перейти по ссылке для заполнения необходимых данных. Или даже создать новый ресурс для другой системы (например, рабочее задание) и использовать дополнительный рабочий поток для встраивания новых данных в этот документ WIP.

В конечном счете кто-то должен выбрать действие `cancelWIP` или `submitWIP`. После этого документ можно будет поставить в очередь на обработку (см. рецепт 7.15) или отменить.

Обсуждение

Рецепт WIP имеет множество вариаций, а на рис. 7.6 приводится не так уж много пояснений. В моей версии минимум элементов, и вам потребуется расширить этот рецепт другими, о которых говорилось ранее.

Желательно не мудрить с иерархией полей в документе WIP. Слишком много зависимых полей или таких, которые необходимо заполнять совместно, может превратить рабочий поток WIP в запутанный процесс с сохранением состояния, который будет сложно понимать, прогнозировать или отлаживать. Вместо этого сделайте так, чтобы в отношении каждого действия `updateWIP` сервис принимал *любую* комбинацию входных данных или только одну одновременно. Можно использовать действие `submitWIP` в качестве сигнала о том, что потребитель API закончил запись и ожидает ревью входных данных.

Это бывает непросто, но вы можете связать между собой документы WIP, чтобы создать многоэтапный рабочий поток, избежав лишних сложностей. При использовании предыдущего примера ссылки на каждый шаг (`employee`, `interviews` и т. д.) могут указывать на другой документ WIP, например `interviewsWIP`, который, в свою очередь, будет указывать на следующий и т. д. Таким образом, вы можете воспользоваться рецептом WIP, чтобы реализовать систему изменчивого рабочего потока, предназначенную для человека.



Если вы хотите создавать более запутанные рабочие потоки под управлением машин, прочитайте рецепты 7.6 и 7.15 — там приводятся подходы, которые лучше годятся для автоматизированных многоэтапных потоков.

Стоит отправлять в архив все завершенные или даже отмененные документы WIP, чтобы иметь возможность обращаться к ним в будущем. Вы даже можете добавить поддержку повторного использования этих документов. Рекомендации по реализации паттерна повторного применения описаны в рецепте 7.14.

Модель WIP предоставляет прекрасный способ реализовать, например, систему тикетов для клиентской службы, поддержки и т. д. Вы можете сделать каждый набор данных очередным ресурсом для входной информации, а также автоматизировать и отображать процесс передачи WIP (через `shareWIP`) на основе состояния документа, например до сих пор недостающих полей. Обычно это означает прописывание в сервисе определенной логики проверки для каждой отправки данных (через `updateWIP`):

```
var wipDocument = parseUpdateRequestBody();

if(wipDocument.onboardingStatus = "needs background check") {
  wipDocument.assignedTo = "Moiria";
  writeStoredItem(wipDocument);
}
...
```

Вы можете использовать точки данных в каждом документе WIP, чтобы облегчить фильтрацию с помощью `filterWIP` любых ожидающих документов. Например, можете добавить поля метаданных наподобие `assignedTo`, `dueDate` и т. д. и даже автоматически отфильтровывать полный список на их основе. Это позволит вам создавать для потребителей API индивидуально настроенные списки задач.

Простой реализацией этого рецепта будет возвращение одного документа WIP со всеми запрошенными полями, возможно, с десятками единиц входных данных. В случаях машинной обработки это не проблема, особенно если вы используете для сбора данных рецепт 7.12. Для человеческого UI можно даже применить подход с одним документом, если оформить входные данные так, чтобы получить своеобразный табличный интерфейс (рис. 7.7):

```
<html>
<head><title>Onboarding Mick Mickelsen</title></head>
<body>

  <div id="wip-actions">
    <a rel="cancelWIP" href="...">Cancel</a>
    <a rel="updateWIP" href="...">Update</a>
    <a rel="submitWIP" href="...">Submit</a>
  </div>
  <div id="wip-inputs">
    <form name="employee" ... >
```

```

        <submit name="updateWIP" />
        <input name="field1" ... />
        <input name="field20" ... />
    </form>
    <form name="interviews">
        <submit name="updateWIP" />
        <input name="field21" ... />
        <input name="field37" ... />
    </form>
    <!-- здесь будут другие формы -->
</div>
</body>

```

В этом примере каждый набор собранных входных данных обернуты в HTML-элемент FORM, который в любой момент можно отправить в механизм WIP. На рис. 7.7 показано, как это решение может выглядеть для людей.

The image shows three overlapping web forms. The topmost form is titled 'Employee Data' and contains fields for 'Given Name' (Mick), 'Family Name' (Mickelsen), 'Email' (mick@example.com), and 'SMS/Phone' (123-456-7890). Below these fields is a 'Save' button. The middle form is titled 'Assignment Data' and contains fields for 'District' (9), 'Building' (51), 'Team' (MNU), 'Leader' (Wikus VanDeMerv), and 'Desk' (109.17). It also has a 'Save' button. The bottommost form is titled 'Submit' and contains the text 'Submit this work-in-progress document for processing.' with a 'Yes' button and a 'Submit' button. Each form has a header bar with tabs for 'Employee Data', 'Assignment Data', 'Cancel', and 'Submit'.

Рис. 7.7. HTML-интерфейс на основе паттерна WIP для использования человеком

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.4. Использование HTTP URL-адресов для поддержки запросов типа Contains и AND.
- Рецепт 7.11. Внедрение стандартной навигации по спискам.
- Рецепт 7.12. Поддержка частичной отправки форм.
- Рецепт 7.14. Оптимизация запросов с помощью сохраненных повторов.

7.11. Внедрение стандартной навигации по спискам

Большинство интерфейсов сервисов должны поддерживать возможность навигации по спискам — вывода следующей страницы, выбора конкретного элемента и т. д. Текущий рецепт содержит простой паттерн, который можно применить к любому открытому списку ресурсов.

Задача

Как лучше всего реализовать поддержку перемещения по страницам больших списков данных?

Решение

Хорошим способом добавления в интерфейс сервиса возможности навигации по страницам будет предоставление фиксированного набора действий, которые можно применить к любому списку ресурсов. Этот набор действий должен включать не только действия для перемещения по списку (первый, предыдущий, следующий и последний), но и действия для выбора конкретного элемента списка и даже выход из самого процесса навигации.

Вот список действий, которые должен поддерживать модуль навигации по страницам.

List (список)

Переход к списку (обычно к началу).

First (первая)

Переход на первую страницу списка.

Previous (предыдущая)

Переход на предыдущую страницу списка.

Next (следующая)

Переход на следующую страницу списка.

Last (последняя)

Переход на последнюю страницу списка.

Select (выбор)

Выбор элемента из списка.

Exit (выход)

Выход из режима навигации по списку.

Home (домой)

Переход к ресурсу Home (выход из списка).

Типичным подходом будет предоставление в документе ссылки с `rel="list"` или `rel="collection"`, которая будет начинать процесс навигации. Тогда возвращаемый ресурс будет содержать ссылки для выполнения других действий в предыдущем списке, включая возможность его покинуть через `rel="exit"` и/или `rel="home"`. Вам необязательно поддерживать и то и другое, но бывают случаи, когда нужно очистить состояние и использовать `rel="exit"`.

Действие `rel="select"` показывает, что потребители API могут задействовать эту ссылку для перехода к одному элементу ресурса. Это удобно, когда не все элементы списка поддерживают одиночное чтение, например в случае списка значений конфигурации.

Этот рецепт можно расширить рецептом 6.4, добавив поддержку создания фильтруемых списков. Можно также использовать рецепт 7.12 для реализации поддержки редактирования списка или добавления в него ресурсов. На рис. 7.8 показан рабочий поток навигации по спискам.

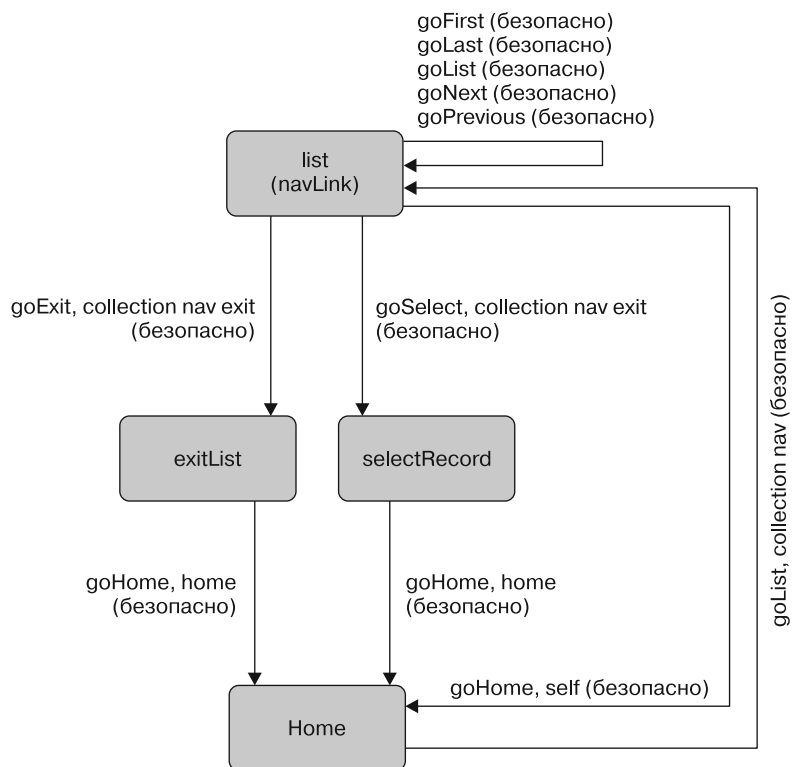


Рис. 7.8. Рабочий поток навигации по спискам

Пример

Как правило, вы будете встречать ресурсы, предлагающие возможность перемещения по списку:

```
{ "collection" : {
  "title" : "Manage Customers",
  "links" : [
    { "rel" : "home self", "href" : "/customers/" },
    { "rel" : "list collection", "href" : "/customers/list" }
  ]
}}
```

Активация ссылки `list` возвращает ресурс `list` с сопутствующими действиями:

```
{ "collection" : {
  "title" : "Customer List",
  "links" : [
    { "rel" : "self list collection", "href" : "/customers/list" },
    { "rel" : "home", "href" : "/customers/" },
    { "rel" : "first", "href" : "/customers/list/q1w2" },
    { "rel" : "next", "href" : "/customers/list/t5y6" },
    { "rel" : "exit", "href" : "/customers/p0o9" }
  ],
  "items" : [
    ...
  ]
}}
```

В данном примере можно предположить, что это первая страница списка. Обратите внимание на то, что ссылка `previous` здесь не предлагается, поскольку невозможно перейти назад от начала списка. Нет здесь и ссылки `last`. Так бывает в случаях, когда список очень большой (содержит тысячи записей) и нет простого способа вычислить последнюю страницу всей коллекции.

Вы заметите, что значения `href` для `first` и `next` не имеют семантического значения, например `page1`, `page2` и т. д. Нет потребности предоставлять строки ссылок, которые человек может легко прочитать. Подробнее о значениях ссылок для навигации читайте в разделе «Обсуждение».

Ссылка `exit` может указывать на временный ресурс, который просит подтверждения выхода из режима навигации и, возможно, согласия с потерей некоторых собранных данных. Это типичный случай, когда создание списка оказывается затратным, и выход будет означать аннулирование всей проделанной работы. Ссылка `exit` также может указывать на ресурс, который позволяет потребителю API сохранять, собирать или иным образом использовать полученные данные о состоянии, например выводить список собранных записей, последнюю считанную запись и т. д.

Вот пример ресурса, который может возвращаться после достижения конца списка:

```
{ "collection" : {
  "title" : "Customer List",
```



```

"links" : [
  {"rel" : "self list collection", "href" : "/customers/list/ht5r"},
  {"rel" : "home", "href" : "/customers/"},
  {"rel" : "first", "href" : "/customers/list/q1w2"},
  {"rel" : "previous", "href" : "/customers/list/t5y6"},
  {"rel" : "exit", "href" : "/customers/p0o9"}
],
"items" : [
  ...
]
}}

```

Обратите внимание, что здесь отсутствуют ссылки на действия `next` и `last`, поскольку они неуместны.

Наконец, если навигация поддерживает выбор или просмотр отдельных ресурсов из списка, можно использовать идентификатор ссылок `select`:

```

{"collection" : {
  "title" : "Customer List",
  "links" : [
    {"rel" : "self list collection", "href" : "/customers/list/ht5r"},
    {"rel" : "home", "href" : "/customers/"},
    {"rel" : "first", "href" : "/customers/list/q1w2"},
    {"rel" : "previous", "href" : "/customers/list/t5y6"},
    {"rel" : "next", "href" : "/customers/list/r4e3"},
    {"rel" : "exit", "href" : "/customers/p0o9"}
  ],
  "items" : [
    { "href" : "/customers/aq1sw2de3",
      "data" : [
        ...
      ]
    },
    "links" : [
      {"rel" : "select item", "href" : "/customers/aq1sw2de3"}
    ],
    ...
  ]
}}

```

Обсуждение

Поддержка списков поначалу выглядит простой затеей, но это верно только при их небольших размерах. По мере увеличения списков процесс становится все сложнее. Будет правильным всегда предполагать, что вы поддерживаете список из 1000 записей, и реализовать навигацию соответствующим образом. Не думайте, что будет легко получать доступ к содержимому списка, фильтровать или отображать его. Стремитесь сократить усилия, необходимые для реагирования на запросы к спискам, и вы избежите множества проблем.

Вполне типично желание предоставлять метаданные, позволяющие потребителям API управлять размером каждой страницы данных (например, 50, 100 и т. д.).

Для реализации этого можно опираться на указанные клиентом предпочтения (см. рецепт 5.5). Кроме того, всегда нужно устанавливать базовое значение размера страницы. Разумная величина будет различаться в зависимости от полноты возвращаемого в списке ресурса. Как правило, удачным началом будет 50.

При возвращении ресурсов в списке не нужно включать их целиком. Зачастую будет достаточно просто вернуть основную информацию вроде сводных данных, часто используемых полей и ссылки на подробное представление (с помощью ссылки `select`).

Помните, что генерация ресурса `list` бывает затратной. Например, перечисление всех компьютеров сети, подключенных к серверу, может оказаться длительным процессом. В таких случаях можно создавать частичные списки — этого будет достаточно для заполнения одной страницы, а остальные страницы вычислять, только если потребитель API сделает очередной запрос.

Может показаться хорошей идеей наладить поддержку навигации, которая позволит потребителям API переходить, допустим, на страницу 11 в списке. Но такой запрос может оказаться затратным. Он требует от вас заранее знать содержимое всего списка. Например, если вы перечисляете всех пользователей чата, то их число будет постоянно меняться. Даже когда список фиксированный, при его значительной величине (тысячи элементов) это может привести к трате ценных ресурсов среды выполнения в попытке перейти на страницу 133 списка из 600 страниц.

Можно расширить поддержку навигации по спискам, добавив возможность поиска (см. рецепт 6.4). Тем не менее бывает сложно перемещаться через фильтруемый список данных. Если вы предоставите такую опцию, то будет разумно создать один снимок состояния отфильтрованной коллекции, сохранить его в памяти и использовать этот список в качестве источника для навигации. Естественно, тогда вам, прежде чем предоставлять первую страницу данных, сначала потребуется создать их полную отфильтрованную коллекцию.

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.6. Возврат на запросы данных HTTP-кода состояния 200 или 400.
- Рецепт 6.7. Использование медиатипов для запросов данных.
- Рецепт 7.9. Возвращение последних использованных ресурсов.
- Рецепт 7.13. Использование наблюдения состояния для реализации рабочего потока под управлением клиента.

7.12. Поддержка частичной отправки форм

Несмотря на то что формы ввода являются ключевым элементом любой системы на основе гипермедиа, наладить по ним навигацию бывает непросто, особенно для машин или автоматизированных процессов. Данный рецепт предоставляет простой стандартный способ обработки входящих форм, который уменьшает вероятность отклонения входных данных в среде выполнения. Вдохновил меня на его создание пионер в сфере проектирования пользовательского опыта Дональд Норман (Donald Norman), который сказал: «Рассматривайте каждое действие пользователя как попытку шагнуть в правильном направлении».

Задача

У пользователей возникают трудности с заполнением форм ввода, особенно когда пользователем выступает *машина*, а не человек. Такие возможности, как зависимые раскрывающиеся списки, подробная валидация на клиентской стороне и просто длинный список ввода, повышают вероятность того, что сервер отклонит присланную форму. Как спроектировать и реализовать формы ввода, которые будут удобны для навигации и повысят вероятность успешного заполнения? Как сделать формы ввода более понятными для машин?

Решение

Вы можете значительно улучшить формы ввода с помощью рецепта их частичного заполнения (Partial Form Submit, PFS). Смысл PFS в том, что на сервер можно отправить даже не полностью заполненные формы. Это позволяет клиентскому приложению заполнять только часть формы, сохранять эту информацию на сервере и затем постепенно дополнять ее. После внесения всех данных клиентское приложение может выполнить итоговую отправку, сообщив сервису, что форма заполнена и готова к любой последующей обработке.



Этот рецепт связан с рецептом 7.10 (текущая работа), но не поддерживает длительные, например занимающие часы или дни, многостраничные процессы ввода.

Рецепт PFS особенно полезен для управляемых машинами клиентских приложений, поскольку они могут отправлять по одному полю за раз с просьбой к серверу проверить и сохранить этот ввод. Такой подход значительно упрощает проверку ошибок для машинных клиентов. Реализация PFS означает возвращение формы ввода с указанием входных данных и вариантов отправки, включая следующие.

`partialSubmit`

Отправка текущей формы и сохранение введенных в нее данных. Она еще не закончена.

`resetSubmit`

Очистка присутствующих входных данных, чтобы можно было начать заново.

`refreshSubmit`

Возврат текущего состояния формы, включая все введенные данные.

`cancelSubmit`

Полная отмена процесса ввода и очистка всех введенных данных.

`finalSubmit`

Клиентское приложение закончило заполнять данные. Отправка формы на финальную обработку.

По мере отправки каждого фрагмента формы посредством `partialSubmit` сервис может выполнять проверку валидности присланного содержимого. Если все в порядке, сервер должен сохранять присланные данные и возвращать клиентскому приложению обновленную форму, содержащую уже введенные значения и оставшиеся пустыми поля для ввода. Этот процесс продолжается до тех пор, пока клиентское приложение не выберет `cancelSubmit` или `finalSubmit`. В любой момент оно может использовать также действие `resetSubmit` для сброса формы и возвращения к началу процесса.

На рис. 7.9 показан рабочий поток PFS.

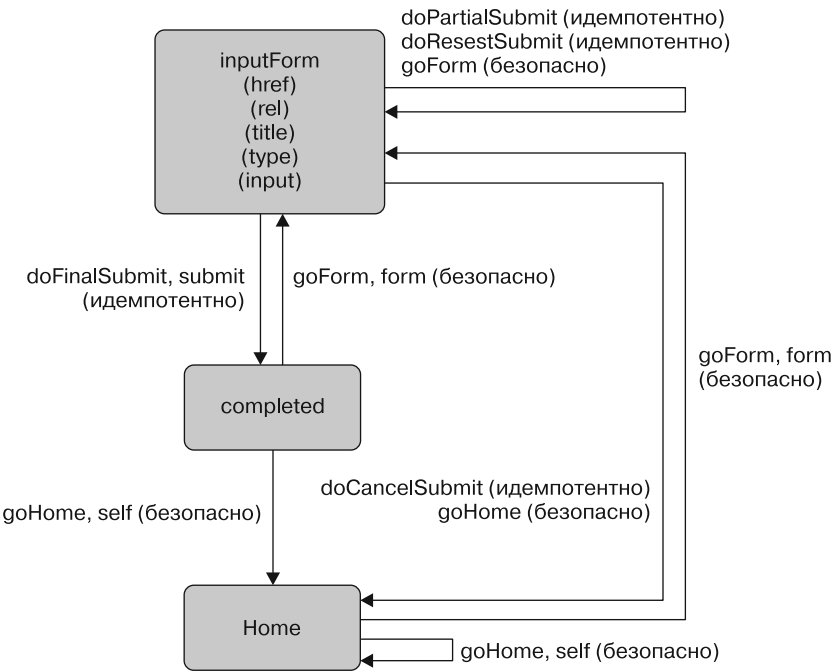


Рис. 7.9. Рабочий поток отправки частично заполненной формы

Пример

Рассмотрим следующую HTML-форму ввода:

```
<form name="search-users" action="/users/filter" method="GET">
  <select name="type">
    <option>customer</option>
    <option>prospect</option>
  </select>
  <select name="region">
    <option>west</option>
    <option>south</option>
  </select>
  <input type="text" name="name" />
  <input type="text" name="salesRep" />
  <input type="submit" name="submit" value="partialSubmit" />
  <input type="submit" name="submit" value="refreshSubmit" />
  <input type="submit" name="submit" value="resetSubmit" />
  <input type="submit" name="submit" value="cancelSubmit" />
  <input type="submit" name="submit" value="finalSubmit" />
</form>
```

Теперь предположим, что написали автономного бота, который ищет все записи, где `type` установлен на `customers`, а значение `salesRep` содержит `Mork`. Это будет простой бот, заполняющий по одному значению за раз и в завершение выполняющий действие `finalSubmit`. Итак, наш бот будет совершать следующие три запроса:

```
GET /users/filter?type=customer&submit=partialSubmit
GET /users/filter/?salesRep=Mork&submit=partialSubmit
GET /users/filter/?submit=finalSubmit
```

Поскольку каждый ввод отправляется отдельно, интерфейс сервиса может выполнить его валидацию и предоставить обратную связь. Например, если бот в качестве первого ввода попытается отправить `type=friend`, сервис может ответить ему кодом состояния `400`, чтобы клиент исправил проблему и повторил попытку, используя `refreshSubmit` для перезагрузки формы.

Обратите внимание: этот рецепт не предполагает, что для выполнения действия `finalSubmit` необходимо заполнить *все* поля ввода. Кроме того, после отправки `finalSubmit` интерфейс сервиса все равно может выполнить валидацию, которая может привести к возврату кода состояния `400` с требованием внести исправления.

Обсуждение

Основной смысл этого рецепта в том, чтобы упростить для любых пользователей отправку данных на сервер. С помощью отделения просьбы «сохранить мои данные» от просьбы «обработать эту форму» можно упростить взаимодействие, а также исправление допущенных в форме ошибок. Такой подход прекрасно себя оправдывает при использовании для отправки данных автоматизированного клиента.

Текущий рецепт фактически усложняет создание форм ввода, требующих «валидации зависимого ввода». Например, будет сложно реализовать поддержку правила, утверждающего: «Если вы выберете `customer` в `inputA`, то *не должны* заполнять значение для `input8` и т. д.». В этом случае вы можете написать для сервера код, который будет использовать действие `finalSubmit` для валидации этих зависимостей и возвращать сообщения об ошибках с подсказками по решению проблемы. По возможности следует избегать таких случаев, поскольку их разрешение для автоматизированных клиентов может оказаться весьма сложным.

Поскольку сервер каждый раз возвращает форму, в которой заполнено все больше полей, клиентское приложение также может вносить в эти поля изменения. Это вполне нормально и особенно полезно для пользователей-людей, поскольку они могут обнаружить ошибки в ранее введенных данных.

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 4.10. Поддержка ссылок и форм для сервисов без поддержки гипермедиа.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.7. Использование медиатипов для запросов данных.
- Рецепт 7.9. Возвращение последних использованных ресурсов.
- Рецепт 7.10. Паттерн «текущая работа» с сохранением состояния.
- Рецепт 7.15. Синхронный повтор незавершенной работы с ответом 202 Accepted.

7.13. Использование наблюдения состояния для реализации рабочего потока под управлением клиента

Несмотря на то что большинство рецептов, посвященных сервисам рабочих потоков, строятся вокруг примеров, в которых *сервис* понимает конечную цель с помощью кода (см. рецепт 7.3), DSL (см. рецепт 7.4) или документа (см. рецепт 7.5), иногда цель потребителя API серверу неизвестна. В таких ситуациях можно использовать текущий рецепт для создания рабочего потока, в котором конечная цель будет известна только клиенту, а сервисы будут предоставлять информацию, помогающую ему определить момент ее достижения.

Задача

Бывают случаи, когда клиентскому приложению известно нужное конечное состояние, а сервисам — нет. Как в таких случаях спроектировать рабочий поток, чтобы он отвечал потребностям клиента (потребителя API)? Когда такая ориентированная на клиента цель окажется применима? Какие нужно учитывать нюансы, если поручить управление рабочим потоком клиентскому приложению?

Решение

Впервые решение задачи с управляемым клиентом рабочим потоком было упомянуто в рецепте 4.16, когда речь шла об определенных конечных целях (Defined End Goals, DEG) и целевых состояниях (Defined State Goals, DSG). И текущий рецепт посвящен тому, как сделать возможными DSG. Основным элементом здесь выступает обеспечение поддержки интерфейсом сервиса выбранных клиентом переменных состояния. Это подразумевает реализацию способа, которым клиентские приложения смогут *сообщать сервисам*, какие данные нужно возвращать.

А КАК ЖЕ GRAPHQL?!

Протокол обработки данных GraphQL (<https://graphql.org>) — прекрасный пример платформы, которая поддерживает запросы, управляемые клиентами. Я не буду здесь использовать GraphQL, но, если этот инструмент уже есть у вас в стеке технологий, можете воспользоваться текущим рецептом, опираясь на его платформу.

Что такое наблюдение за состоянием

Рассмотрим пример, в котором клиентское приложение (**temp03**) должно мониторить показания датчиков температуры в большом здании. Это приложение знает допустимый температурный диапазон для, скажем, третьего этажа. Оно также знает, что нужно мониторить датчики температуры на этом этаже и управлять термостатом на этом участке. То есть клиент должен получать обратную связь по каждому **thermo-sensor** на этаже и иметь доступ к управлению **thermostat**.

А теперь представим еще одно приложение (**light03**), которое должно мониторить движение по тому же самому этажу и включать/выключать освещение в зависимости от присутствия в помещениях движущихся людей. Этому приложению необходим доступ ко всем **motion-sensors** и **light-switches** третьего этажа.

Наконец, предположим, что есть единый интерфейс сервиса управления зданием, поддерживающий доступ ко всем датчикам и исполнительным устройствам в нем. Именно с этим сервисом взаимодействуют `temp03` и `light03` для выполнения своей работы. Идеальным решением будет включение в каждый запрос клиента только тех свойств данных, которые ему нужно отслеживать, и возвращение только тех элементов действий (FORMS), к которым каждое приложение хочет иметь доступ.

Клиентским приложениям нужен способ сообщать сервису, какую информацию он хочет видеть в каждом HTTP-ответе. Все это сводится к тому, что клиент сообщает интерфейсу сервиса значения состояния, за которыми он будет наблюдать. Отсюда и название рецепта — *наблюдение за состоянием*.

Можно ли за этим наблюдать

Чтобы все это сработало, интерфейс сервиса должен уметь сообщить потребителю API, какие конкретно свойства данных доступны для наблюдения, то есть какие свойства можно выбрать для включения в HTTP-ответы. По сути, это вариация рецепта навигации по спискам (см. рецепт 7.11). И поскольку мы об этом уже говорили, я не стану повторять детали. Для наших целей предположим, что список доступных для наблюдения элементов включает все датчики движения, термодатчики, выключатели света и термостаты в здании. Также упростим задачу и допустим, что идентифицированный субъект нашего приложения имеет доступ только к элементу, расположенному на третьем этаже здания. Это означает, что вызов `watchList` вернет доступные элементы.

Теперь нам нужна возможность сообщить сервису, какие конкретно элементы из списка каждое клиентское приложение хочет видеть.

Я буду следить за датчиками движения на третьем этаже

Клиентское приложение может обозначить элементы, которые хочет отслеживать, четырьмя способами. Каждый из них имеет как преимущества, так и недостатки.

Строка запроса

Простейший способ передать список наблюдаемых элементов — включение их в строку HTTP-запроса. Преимущество данного подхода в том, что он хорошо известен и легко реализуется. Недостатком является то, что список может оказаться довольно большим, усложняя парсинг URL промежуточными инстанциями.

HTTP-заголовки Prefer

В RFC 7240 определен заголовок `Prefer` (<https://oreil.ly/Fu5xQ>), который позволяет клиентам передавать метаданные о предпочтениях клиента в отношении

того, как сервис должен изменять тело HTTP-ответа. Преимущество этого решения в том, что оно удерживает информацию вне URL и использует установленный стандарт. Недостаток же его заключается в отсутствии на данный момент свойства (см. HTTP Preferences, <https://oreil.ly/GW99A>), которое можно было бы применить для списка наблюдаемых элементов. Тем не менее вы можете подать заявку на его определение или задействовать этот заголовок без регистрации свойства.

Ресурс наблюдения за состоянием

Еще одно решение подразумевает определение ресурса наблюдения за состоянием, который будет ассоциироваться с запросами клиента. В этом случае для размещения списка элементов, которые должен возвращать сервис, можно использовать отдельный ресурс. Указатель на него можно передавать через заголовок Link с идентификатором `rel="watch"`. Преимущество этого решения в его понятности и простоте, а недостаток — в том, что это *отдельный* ресурс, который в среде выполнения должны учитывать и потребитель API, и его поставщик.

Тело запроса данных

Чтобы исключить проблемы с длинными URL, наблюдаемые элементы состояния можно передавать в теле запроса. Так работает GraphQL. Преимущество в том, что для этого есть рабочая технология, а недостаток — в необходимости использования небезопасного HTTP-метода (PUT или POST), что усложняет кэширование, повторы и создает прочие проблемы.

По ряду причин я рекомендую решение с применением отдельного ресурса. Оно окажется наиболее совместимым с подходящим применением протокола HTTP и исключит проблемы, связанные с длинными списками свойств данных. Кроме того, оно позволит клиентам устанавливать свои предпочтения в самом начале сеанса рабочего потока и не повторять их отправку, пока не придет время изменять список.

Свойства наблюдения за состоянием, ресурсы и действия

Поддержка паттерна наблюдения за состоянием включает такие свойства, как ресурсы и действия. К свойствам наблюдения за состоянием относятся следующие.

watchElementID

Уникальный идентификатор наблюдаемого элемента.

watchElementURL

URL наблюдаемого элемента.

watchElementName

Текстовое имя наблюдаемого элемента.

`watchElementValue`

Текущее значение наблюдаемого элемента.

`watchElementTag`

Фильтрующий тег наблюдаемого элемента.

Далее перечислены ресурсы состояния.

`watchListResource`

Содержит список всех доступных для наблюдения элементов.

`watchResource`

Представляет список *выбранных* наблюдаемых элементов.

А также действия по наблюдению за состоянием.

`dowatchClear`

Небезопасное действие по очистке всех выбранных наблюдаемых элементов.

`dowatchCreate`

Небезопасное действие по созданию нового списка из выбранных наблюдаемых элементов.

`dowatchUpdate`

Небезопасное действие обновления текущего списка выбранных наблюдаемых элементов.

`gowatchList`

Безопасное действие для возврата списка доступных наблюдаемых элементов.

`gowatchResource`

Безопасное действие для возврата списка выбранных наблюдаемых элементов.

Типичное взаимодействие при наблюдении за состоянием

При использовании решения на основе отдельного ресурса процесс выбора клиентом наблюдаемых элементов состояния и уведомления об этом выборе сервера будет выглядеть так.

1. Клиент находит ссылку `rel=watch` сервиса (см. рецепт 7.8).
2. Клиент выбирает элементы, за которыми хочет наблюдать.
3. Клиент использует форму `updatewatchList` и отправляет список наблюдаемых элементов интерфейсу сервиса с помощью уникального URL, сгенерированного *самим клиентом API*.
4. Теперь при каждом запросе к сервису, например `listStates`, в ответ будет включаться статус каждого элемента.
5. Это продолжается, пока клиент не решит обновить или очистить `watchList`.

Наглядно этот процесс отражен на рис. 7.10 и описан в разделе «Пример».

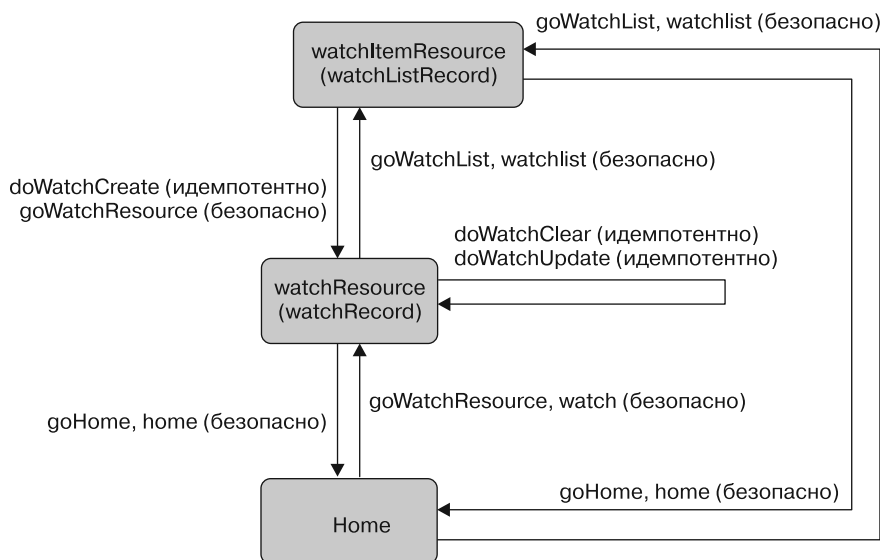


Рис. 7.10. Рабочий поток наблюдения за состоянием

Пример

Интерфейс сервиса, поддерживающий паттерн наблюдения за состоянием, может сообщить об этом с помощью HTTP-заголовка `Link` и/или ссылки в теле ответа:

**** REQUEST ****

GET /building

Host: api.example.org

Accept: application/vnd.siren+json

...

**** RESPONSE ****

200 OK

Content-Type: application/vnd.siren+json

Link: <<http://api.example.org/building/elements/list>>; rel="watchList"

Link: <<http://api.example.org/building/selected;t6r5e4ed>>; rel="watchSelected"

```

{"class": ["building"],
 "links": [
  {"rel": ["self", "home"],
   "href": "/building"},
  {"rel": ["watchList", "collection"],
   "href": "/building/elements/list"},
  {"rel": ["watchSelected", "collection"],
   "href": "/building//building/selected;t6r5e4"}
 ],
}
```

```

    "properties": {...},
    "entities": {...}
}

```

Обратите внимание на то, что в ответе есть две ссылки, связанные с паттерном наблюдения состояния. Первая, `rel="watchList"`, представляет список доступных элементов.



Важно, чтобы каждый потребитель API имел собственный ассоциированный с ним ресурс `rel=watchSelected`. Генерация этого URL чаще всего является обязанностью самого потребителя (напомню, что рабочим потоком управляет именно он). Подробнее читайте в разделе «Обсуждение».

Клиенты могут использовать ее для просмотра и выбора интересующих их элементов. Вторая ссылка, `rel=watchSelected`, представляет список выбранных элементов, возвращаемых с каждым ответом:

```

**** REQUEST ****
GET /building/elements/list
Host: api.example.org
Accept: application/vnd.siren+json

**** RESPONSE ****
200 OK
Content-Type: application/vnd.siren+json
Link: <<http://api.example.org/building/elements/list>>; rel="watchList"
Link: <<http://api.example.org/building/selected;t6r5e4ed>>; rel="watchSelected"

```

```

{"class": ["building"],
 "links" : [
  {"rel" : ["home"],
   "href": "/building"},
  {"rel" : ["self", "watchList", "collection"],
   "href": "/building/elements/list"},
  {"rel" : ["watchSelected", "collection"],
   "href": "/building/elements/selected"}
 ],
 "entities" : [
  {"class": ["watch", "element"], "properties": {
    "id": "q1w2e3r4",
    "href": "/building/elements/list/q1w2e3r4",
    "type": "thermo-sensor",
    "name": "ts-01",
    "location": "Third floor"
  }},
  ...
  {"class": ["watch", "element"],
   "properties": {
    "id": "u8y7t6r5",
    "href": "/building/elements/list/u8y7t6r5",

```

```

        "type": "thermostat",
        "name": "stat-0301",
        "location": "Third floor"
    }
  ],
  "actions": [
    {
      "name": "select-elements",
      "title": "Select Elements",
      "method": "PUT",
      "href": "http://api.example.org/building//building/selected;t6r5e4",
      "type": "text/plain",
      "fields": [
        { "name": "elements", "type": "value": "q1w2e3r4,u8y7t6r5" }
      ]
    }
  ]
}

```

В этом примере клиент отправляет в список `watchSelected` два элемента: `q1w2e3r4` и `u8y7t6r5`. Теперь каждый ответ от связанного с ним ресурса (`/building/`) будет включать актуальный статус этих элементов:

```

GET /building
Host: api.example.org
Accept: application/vnd.siren+json
...

```

```

**** RESPONSE ****
200 OK

```

```

Content-Type: application/vnd.siren+json
Link: <<http://api.example.org/building/elements/list>>; rel="watchList"
Link: <<http://api.example.org/building/selected;t6r5e4ed>>; rel="watchSelected"

```

```

{"class": ["building"],
 "links": [
   {"rel": ["self", "home"],
    "href": "/building"},
   {"rel": ["watchList", "collection"],
    "href": "/building/elements/list"},
   {"rel": ["watchSelected", "collection"],
    "href": "/building/elements/selected"}
 ],
 "properties": {...},
 "entities": [
   {"class": ["thermo-sensor"], "id": "q1w2e3r4", "name": "ts-01", "temp": "-3C"},
   {"class": ["thermostat"], "id": "u8y7t6r5", "name": "stat-0301", "state": "off"},
   ...
 ],
 "actions": [...]
}

```

Обратите внимание на то, что два этих выбранных элемента были включены в ответ ресурса `/building`. Можно также получить просто список выбранных элементов:

```
GET /building//building/selected;t6r5e4
Host: api.example.org
Accept: application/vnd.siren+json
...

**** RESPONSE ****
200 OK
Content-Type: application/vnd.siren+json
Link: <<http://api.example.org/building/elements/list>>; rel="watchList"
Link: <<http://api.example.org/building/selected;t6r5e4ed>>; rel="watchSelected"

{"class": ["watchSelected"],
 "links": [
  {"rel" : ["home"],
   "href": "/building"},
  {"rel" : ["watchList", "collection"],
   "href": "/building/elements/list"},
  {"rel" : ["home", "watchSelected", "collection"],
   "href": "/building/selected;t6r5e4"}
 ],
 "properties" {...},
 "entities": [
  {"class":["thermo-sensor"],"id":"q1w2e3r4","name":"ts-01","temp": "-3C"},
  {"class":["thermostat"],"id":"u8y7t6r5","name":"stat-0301","state": "off"},
  ...
 ],
 "actions": [
  {"name": "updateWatchList", "method": "PUT", ...},
  {"name": "clearWatchList", "method": "DELETE", ...}
 ]
}
```

В этом примере были возвращены не только выбранные элементы со своими текущими значениями, но и две формы. Действие `updateWatchList` можно применить для изменения списка наблюдения, а действие `clearWatchList` — для быстрой очистки всех элементов из него.

Используя пример с двумя потребителями API, один из которых наблюдает за температурой, а второй — за движением и освещением, можно было построить и другой сценарий.

Обсуждение

Представленный паттерн наблюдения за состоянием отлично работает, когда потребителю API для принятия решения или выполнения действия нужно оценить

небольшой набор доступных данных. Мониторинг определенных датчиков (температурных, движения, жидкостных и т. д.) — хороший пример использования паттерна наблюдения за состоянием.



Этот рецепт — один из самых сложных во всем наборе. Он требует, чтобы созданный ресурс использовался в среде выполнения клиентом и сервером, а конкретно: сервером на основе предоставленного клиентом ввода с уникальным ID. Обе стороны должны помнить его URL при обмене HTTP-запросами и ответами.

Поскольку паттерн наблюдения за состоянием опирается на два ключевых HTTP-ресурса (список доступных для наблюдения элементов и список наблюдаемых элементов для каждого клиента API), необходима предельная ясность при создании уникального URL для выбранных клиентом элементов. Подробности этого URL здесь неважны, но вам нужно обеспечить, чтобы клиент мог сгенерировать уникальный ID для собственного `watchResource` (см. рецепт 5.14), *запомнить* его и передавать в сервис при необходимости. Последний этап обычно означает, что сервис должен возвращать URL с каждым ответом соответствующему клиенту (в HTTP-заголовках и/или теле).

В показанных ранее примерах в форме `updateWatchList` для отправки разграниченного запятыми списка наблюдаемых элементов используется `text/plain`. Можно использовать и другие форматы сериализации, но потребуется сделать так, чтобы клиентские приложения заранее знали, как парсить ответы с наблюдаемыми свойствами.

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 4.14. Валидация входящих данных.
- Рецепт 4.15. Сохранение состояния.
- Рецепт 4.16. Наличие цели.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.5. Возвращение метаданных в ответах на запросы.
- Рецепт 7.7. Предоставление ресурса прогресса, отражающего выполнение рабочих потоков.
- Рецепт 7.15. Синхронный повтор незавершенной работы с ответом 202 Accepted.

7.14. Оптимизация запросов с помощью сохраненных повторов

Этот рецепт предлагает стандартизированный способ создания сохраненных запросов, управления ими и их выполнения для систем, которые активно используют их вариации, настроенные под индивидуальные потребности.



Возможно, вы ожидали встретить этот рецепт в главе 6. Но поскольку он подразумевает согласованную работу нескольких ресурсов, я решил привести его здесь, в главе, посвященной рабочим потокам.

Задача

Некоторые запросы оказываются сложными и/или затратными для выполнения. Порой HTTP-запросы требуют множества фильтрующих параметров, так как создают очень длинные строки. Поскольку на большинстве HTTP-серверов установлен фиксированный лимит для длины URL, комплексные запросы могут быть ошибочно интерпретированы как атака на безопасность. Что можно предпринять, чтобы сделать сохранение и *воспроизведение* таких комплексных запросов простыми, безопасными и дешевыми? Как можно более эффективно управлять списком собранных запросов, в том числе предоставляя пользователям или машинам простую возможность создания и последующей публикации этих запросов для применения другими?

Решение

Для того чтобы сделать комплексные запросы безопасными и простыми в применении, первым делом нужно реализовать сам запрос в виде ресурса. Это означает выражение его деталей в виде HTTP-тела и последующее использование этого тела для создания нового *ресурса запроса*, который можно будет при необходимости выполнять непосредственно. Это будет уже второй шаг — задействование URL ресурса запроса для его выполнения и возврата результатов.

НЕ ВСЕ URL РАВНЫ

В этом рецепте кроется небольшая хитрость. Потребители API должны знать URL для *выполнения* запроса, а иногда им может потребоваться знать также URL для его *редактирования*. Обработка этих моментов описана в подразделе «Пример».

По сути, таким образом вы создаете коллекцию ресурсов запросов, которой в том числе нужно управлять. В данном рецепте используется навигация по списку (см. рецепт 7.11) для реализации вывода, фильтрации, редактирования и удаления ресурсов запросов.

Наконец, второстепенным элементом в этом рецепте будет действие `shareQuery`. Оно позволяет реализовать отправку ресурса запроса кому-то другому для использования и/или установить владельца этого ресурса для дальнейшего управления им.

С сохраненными запросами ассоциируется коллекция метаданных. Их (и другие) можно применять для управления создаваемыми и воспроизводимыми запросами и их отслеживания.

identifier

Уникальный идентификатор сохраненного запроса.

URL

URL этого отложенного документа ответа.

resourceQuery

Полный (оригинальный) HTTP-запрос.

cacheTTL

Как долго (в миллисекундах) ответ должен кэшироваться клиентом.

cachingDirectives

Дополнительные директивы кэширования запроса.

description

Человекочитаемый текст, описывающий сохраненный запрос.

owner

Идентификатор (URL или текст) субъекта, контролирующего этот ресурс запроса.

tags

Строка свободной формы с разделенными пробелом значениями. Используется для фильтрации или иной группировки сохраненных запросов.

dateCreated

Указывает дату/время (UTC) создания этого ресурса воспроизведения.

dateUpdated

Указывает дату/время (UTC), когда этот ресурс воспроизведения был изменен.

dateLastRun

Указывает последнюю дату/время (UTC), когда этот сохраненный ресурс был выполнен.

Свойство `resourceQuery` содержит всю строку запроса, передаваемую в сервис при создании ресурса. Его тоже можно обновлять. Свойства `cacheTTL` и `caching-Directives` могут содержать детали, отражающие вид метаданных кэширования, которые должны возвращаться с ответом на запрос. Свойства `owner` и `tags` удобны для отслеживания деталей управления вроде ответственного за ресурс и способов группировки или фильтрации ресурса в коллекции.

На рис. 7.11 показан рабочий поток воспроизведения сохраненного запроса.

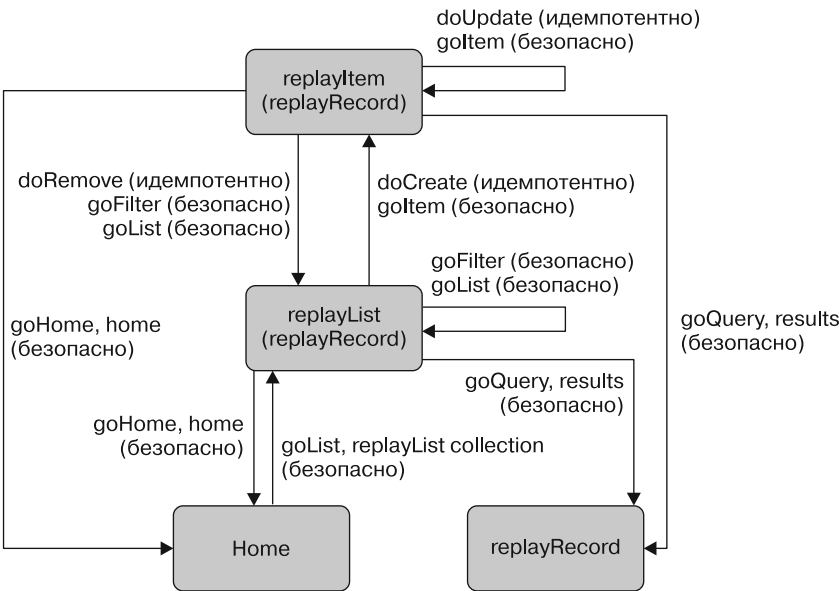


Рис. 7.11. Рабочий поток воспроизведения сохраненного запроса

Пример

Этот рецепт разделен на две части. Первая охватывает создание и воспроизведение ресурсов запросов, а вторая — управление списком сохраненных ресурсов запросов.

Создание и воспроизведение ресурсов запросов

Первое действие создает ресурс запроса и сохраняет его на сервере. Выглядит это так:

```
**** REQUEST ****
PUT /queries/q1w2e3r4
Content-Type: application/x-www-form-urlencoded
Accepted: application/vnd.collection+json
```

```
Length: NN
If-Not-Match: "*"

```

```
type=customer&region=south&name=Andrews&balance-is-great-then=10000
&past-due-days-is-great-than=90&salesrep=Mork

```

```
**** RESPONSE ****

```

```
200 OK

```

```
Content-Type: application/vnd.collection+json

```

```
Length: NN

```

```
{
  "collection" : {
    "title" : "Query Results",
    "metadata" : [
      {
        "name": "q-sent", "value": "type=customer&region=south&name=Andrews& +
        balance-is-great-then=10000&past-due-days-is-great-than=90&salesrep=Mork"},
      {
        "name": "q-datetime", "value": "2024-12-12:00:12:0012TZ"},
      {
        "name": "q-status", "value": "sucess"},
      {
        "name": "q-seconds", "value": "120"},
      {
        "name": "q-count", "value": "100"},
    ],
    "links" : [...],
    "items" : [...]
  }
}
```

Как видите, потребитель API отправил комплексный запрос в виде PUT для создания нового ресурса (подробнее читайте в рецепте 5.15). При отсутствии проблем сервис создает новый ресурс запроса, выполняет его и возвращает результаты. Вы также заметите, что сервер вернул в ответе метаданные запроса (см. рецепт 6.5). Это необязательно, но будет особенно полезным при использовании данного рецепта для выполнения запросов.

Теперь, когда запрос сохранен на сервере, потребители API могут вызывать его ресурс напрямую для воспроизведения:

```
**** REQUEST ****

```

```
GET /queries/q1w2e3r4

```

```
Accept: application/vnd.collection+json

```

```
...

```

```
**** RESPONSE ****

```

```
200 OK

```

```
Cache-Control: public, max-age: 3600

```

```
ETag: "w/ki8ju7hy6gt5"

```

```
...

```

```
{
  "collection" : {
    "title" : "Query Results",
    "metadata" : [...],
    "links" : [...],
    "items" : [...]
  }
}
```

Здесь потребителю API для возвращения результатов просто нужно отправить HTTP-запрос GET (не PUT). В нашем случае сервис воспроизвел сохраненный запрос и вернул обновленные результаты. Обратите также внимание на использование заголовка `Cache-Control` для передачи потребителю директив о том, как долго можно хранить запрос локально. Подробнее читайте в рецепте 6.9.

Управление сохраненными ресурсами запросов

Создание и воспроизведение сохраненных запросов — лишь часть этого рецепта. Вторая часть подразумевает управление коллекцией сохраненных запросов. Это означает предоставление действий `list`, `filter`, `update` и `remove` из рецепта 7.11. В текущем рецепте я дополнительно ввел действие `share`.

Хорошим способом предоставления указанных управляющих элементов будет включение ссылок в ответы на запросы. Их можно отправлять в теле медиатипа:

```
{ "collection" : {
  "title" : "Query Results",
  "metadata" : [...],
  "links" : [
    { "rel": "list", "href": "http://api.company.org/queries/list" },
    { "rel": "filter", "href": "http://api.company.org/queries/filter" },
    { "rel": "update", "href": "http://api.company.org/queries/q1w2e3r4#update" },
    { "rel": "share", "href": "http://api.company.org/queries/q1w2e3r4#share" }
  ]
  "items" : [...]
}
```

Ссылки также можно включить в коллекцию HTTP-заголовков. В примере далее текущий пользователь может только передать этот запрос, не имея возможности редактировать или удалять его, как и просматривать весь список:

```
**** RESPONSE ****
200 OK
Content-Type: application/vnd.collection+json
Link: <<https://api.company.org/queries/q1w2e3r4#share>>; rel=share
```

Я не буду подробно описывать здесь действия `list`, `filter`, `update` и `remove`. Подробнее о них можно почитать в рецепте 7.11. Но кое-что скажу об URL, показанном в предыдущем примере. При создании URL для этого рецепта возникают определенные сложности. Нам нужен URL для *выполнения* запроса, а также URL для управления им, например чтения, редактирования, удаления и передачи.

И здесь есть несколько вариантов. Лично я сообщаю сервису о желаемом действии путем изменения URL ресурса запроса. Это может не нравиться тем, кто не любит встречать в URL HTTP-глаголы. Если хотите, можете использовать для управления ресурсом запроса иной URL (например, `https://api.company.org/`

queries/list/q1w2e3r4), нежели для его выполнения (<https://api.company.org/queries/q1w2e3r4>).

Конкретный URL, который вы увидите, зависит от вас. Просто помните, что для реализации текущего рецепта нужно будет отслеживать их в среде выполнения и сообщать потребителю API о различиях.

Обсуждение

Одним из ярких преимуществ реализации этого рецепта является решение проблемы с очень длинными строками запроса. Я часто встречаю применение HTTP-метода POST для поддержки больших строк. И хотя этот подход работает, он лишает вас преимущества кэширования при использовании POST. Решение с созданием и последующим выполнением ресурса запроса работает, потому что вы можете вернуть URL нового ресурса и выполнить для него простой запрос GET. Таким образом взамен вы получите все варианты кэширования.

Данный рецепт реально оправдывает себя, когда вы *воспроизводите* запросы позднее. Если не хотите этого делать, то можете пропустить этап реализации управления ресурсами, реализовав только этап создания и выполнения посредством PUT/POST. В таких случаях можно сохранять созданные ресурсы короткое время, после чего просто их удалять. Последующие попытки вызова этих временных URL будут давать ответы с HTTP-кодом ошибки 401.

Здесь были приведены метаданные owner, но сказать о них нечего, разве что описать применение этой функциональности. Как правило, она связана с идентификаторами пользователей или машин и сопутствующими вопросами безопасности. Подробное описание всего этого выходит за рамки тематики книги.

Передача запросов позволяет упростить для людей повторное использование имеющихся ресурсов. Я часто делаю все воспроизводимые запросы публичными и включаю интерфейс, который позволяет пользователям или машинам выбирать один из них или несколько для собственного применения. Если вы это сделаете, то потребуются определиться с тем, кто может редактировать эти повторно используемые запросы. При этом вам также может потребоваться реализовать действие сору, чтобы люди могли делать для себя копию и редактировать ее без изменения исходного запроса.

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 3.8. Проектирование с учетом возможности повторения действий.

- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 4.14. Валидация входящих данных.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 6.4. Использование HTTP URL-адресов для поддержки запросов типа Contains и AND.
- Рецепт 6.9. Повышение производительности с помощью директив кэширования.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.

7.15. Синхронный повтор незавершенной работы с ответом 202 Accepted

Порой для выполнения нужной работы может потребоваться некоторое время. Для таких случаев HTTP, основанный на синхронной передаче запросов/ответов, предоставляет ответ с кодом состояния **202 Accepted**. Этот код можно использовать для создания паттерна, позволяющего откладывать ответы, разрешение которых может потребовать нескольких часов или даже дней.

Задача

Иногда заранее известно, что на выполнение запроса к ресурсу уйдет много времени — больше нескольких секунд. В таких ситуациях вам нужно реализовать паттерн «отложенный ответ», который сообщит клиенту, что на выполнение потребуется время, и предоставит достаточно метаданных, чтобы клиент понял, как ему поступить с этой задержкой. Что же необходимо для реализации и применения этого паттерна? Какие метаданные нужно передавать клиенту? Какие опции рабочего потока при этом должны поддерживаться?

Решение

В случаях, когда ответ на HTTP-запрос наверняка задержится (займет более нескольких секунд), можно использовать паттерн отложенного ответа. Реализуется он путем возвращения от сервиса кода состояния **202 Accepted** вместе с метаданными (в теле и/или заголовках), информирующими клиентское приложение о статусе запроса, и вариантами дальнейших действий наподобие `refresh` или `cancel`.

Изначальный ответ можно вернуть еще до начала выполнения запрошенной работы, например сохранения записи, вычисления регрессии и т. д. Типичный вариант тела ответа и возможных заголовков приводится в разделе «Пример».

Тело отложенного ответа может включать следующие метаданные.

identifier

Уникальный идентификатор отложенного ответа.

acceptedURL

URL документа отложенного ответа.

completedURL

URL ресурса, отражающего выполненное состояние.

failedURL

URL ресурса, отражающего неудачное состояние.

description

Человекочитаемый текст, описывающий документ.

refresh

Значение (в миллисекундах), представляющее интервал обновления.

percentCompleted

Значение, отражающее степень завершения (0–100).

status

Указывает состояние документа (в ожидании, в работе, выполнен или провален).

dateCreated

Указывает дату/время (UTC) создания отложенного ответа.

dateUpdated

Указывает последнюю дату/время (UTC) изменения отложенного ответа.

dateEstimated

Указывает дату/время (UTC), когда отложенный запрос должен быть выполнен.

Кроме того, могут встречаться стандартные действия.

goAccepted

Переход к ресурсу отложенного ответа (безопасно).

doCancel

Запрос на обновление ожидающей задачи с помощью DELETE (идемпотентно).

goCompleted

Переход к ресурсу выполненного состояния (безопасно).

goFailed

Переход к ресурсу неудачного состояния (безопасно).

goHome

Переход к ресурсу Home (безопасно).

Некоторые из этих метаданных можно вернуть также с помощью заголовков Link. На рис. 7.12 показан рабочий поток возврата HTTP-кода 202 Accepted.

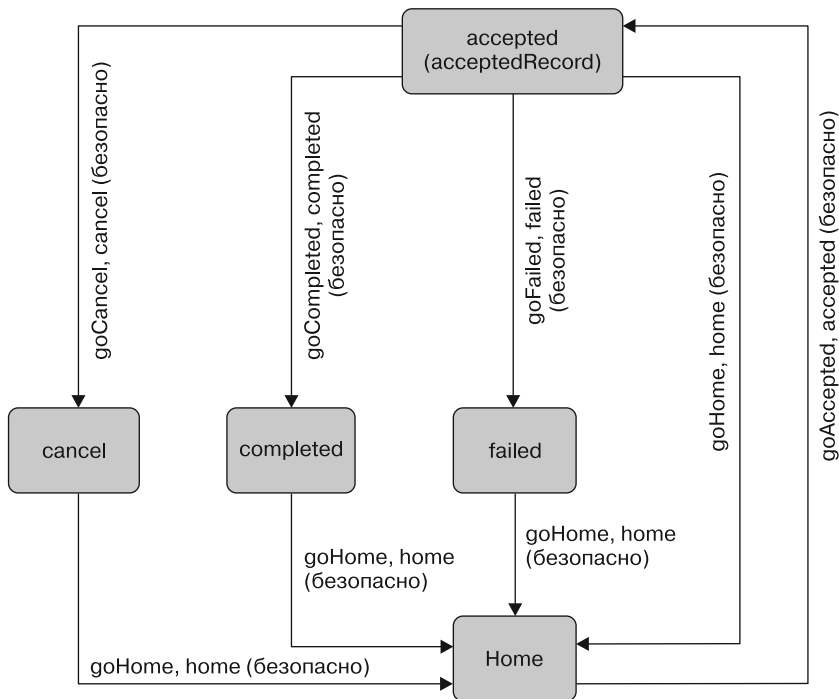


Рис. 7.12. Рабочий поток 202 Accepted

Пример

Далее показан типичный пример паттерна отложенного ответа.

Начинается все с того, что потребитель API отправляет запрос интерфейсу сервиса, который начинает работу (или ставит ее в очередь) и возвращает ответ 202 Accepted:

```

**** REQUEST
PUT /services/compute-results HTTP/1.1
Content-Type: application/vnd.collection+json
...

{"collection": {...}}

```



```

**** RESPONSE
202 Accepted HTTP/1.1
Content-Type: application/vnd.collection+json
Link: <http://api.example.org/services/compute-results/q1w2e3>;rel=self;
refresh=60000
Link: <http://api.example.org/services/cancel-form/q1w2e3>;rel=cancel

{"collection":
{
  "links": [
    {"rel":"self", "name":"acceptedURL",
     "href":"/services/compute-results/q1w2e3"}
  ],
  "items": [
    {
      "href": "/services/compute-results/q1w2e3",
      "data" : [
        {"name":"identifier", "value":"q1w2e3"},
        {"name":"description",
         "value":"Compute net present value for property"},
        {"name":"refresh", "value":"60000"},
        {"name":"percentCompleted", "value":"10"},
        {"name":"status", "value":"working"},
        {"name":"dateCreated", "value":"2024-02-01:22:11:00"},
        {"name":"dateUpdated", "value":"2024-02-01:22:11:00"},
        {"name":"dateEstimated", "value":"2024-02-01:22:15:00"}
      ]
    }
  ]
}
}
}

```

Затем потребитель API использует полученные в ответе метаданные для периодического обновления DR-Doc (документ отложенного ответа), отмечая состояние работы:

```

**** REQUEST
GET /services/compute-results/q1w2e3 HTTP/1.1
Accepted: application/vnd.collection+json
...

**** RESPONSE
200 OK HTTP/1.1
Content-Type: application/vnd.collection+json
Link: <http://api.example.org/services/compute-results/q1w2e3>;rel=self;
refresh=60000
Link: <http://api.example.org/services/cancel-form/q1w2e3>;rel=cancel

{"collection":
{
  "links": [
    {"rel":"self", "name":"acceptedURL",
     "href":"/services/compute-results/q1w2e3"}
  ],

```

```

"items": [
  {
    "href": "/services/compute-results/q1w2e3",
    "data" : [
      {"name": "identifier", "value": "q1w2e3"},
      {"name": "description",
        "value": "Compute net present value for property"},
      {"name": "refresh", "value": "60000"},
      {"name": "percentCompleted", "value": "75"},
      {"name": "status", "value": "working"},
      {"name": "dateCreated", "value": "2024-02-01:22:11:00"},
      {"name": "dateUpdated", "value": "2024-02-01:22:14:00"},
      {"name": "dateEstimated", "value": "2024-02-01:22:15:00"}
    ]
  }
]
}
}
}

```

Когда Dr-Doc достигает статуса `completed` или `failed`, потребитель API переходит по соответствующей ссылке:

```

**** REQUEST
GET /services/compute-results/q1w2e3 HTTP/1.1
Accepted: application/vnd.collection+json
...

**** RESPONSE
200 OK HTTP/1.1
Content-Type: application/vnd.collection+json
Link: <http://api.example.org/services/npv/p009i8u7y6>;rel=completed
Link: <http://api.example.org/services/compute-results/q1w2e3>;rel=self;
refresh=60000

```

```

{"collection":
{
  "links": [
    {"rel": "self", "name": "acceptedURL",
      "href": "/services/compute-results/q1w2e3"}
  ],
  "items": [
    {
      "href": "/services/compute-results/q1w2e3",
      "data" : [
        {"name": "identifier", "value": "q1w2e3"},
        {"name": "description",
          "value": "Compute net present value for property"},
        {"name": "percentCompleted", "value": "100"},
        {"name": "status", "value": "completed"},
        {"name": "dateCreated", "value": "2024-02-01:22:11:00"},
        {"name": "dateUpdated", "value": "2024-02-01:22:16:00"},
        {"name": "dateEstimated", "value": "2024-02-01:22:15:00"}
      ]
    }
  ]
}

```

```
    "links": [  
      {"rel": "completed", "href": "/services/npv/p0o9i8u7y6"}  
    ]  
  }  
]  
}  
}
```

Наконец, если в ответе такая возможность присутствует, потребитель API может использовать действие `cancel` для отмены запроса.

Обсуждение

Ключом к успешной реализации отложенного ответа является добавление в промежуточные ответы всех возможных метаданных. Самые важные — интервал обновления `refresh` и значение `status`. Нелишними будут и значения `dateEstimated` и `percentCompleted`.

Большинство потребителей API не смогут действенно отреагировать на полученный от интерфейса сервиса ответ `202 Accepted`. Поэтому важно заранее сообщить им, какие действия могут привести к возврату этого кода состояния. Сделать это нужно с помощью документации интерфейса.

Не все DR (отложенные ответы) поддерживают возможность отмены запроса. Если внутренняя работа ограничена обработкой длительных вычислений (как в нашем примере с чистой приведенной стоимостью) или генерацией длинного отчета, тогда поддержку отмены добавить несложно. Однако если запрос подразумевает изменение состояния где-либо (создание записи, удаление или добавление данных для одной или нескольких существующих записей и т. д.), сделать это будет сложнее. Подробнее о реализации отмены для RESTful веб-сервисов читайте в рецепте 7.17.

В зависимости от формата представления, используемого в отложенных ответах, действие отмены можно выразить в виде встроенной FORM или ссылки, а также в виде ссылки на FORM. При реализации ответов обязательно соблюдайте соглашения о выражении действий гипермедиа в теле ответа.

Может возникнуть соблазн включить в HTTP-заголовки множество метаданных отложенного ответа, но в этом случае лучше свести применение заголовков к минимуму. В особенности это касается случаев, когда вы хотите передать временную информацию вроде `percentCompleted`, `dateUpdated`, `status` и т. д. Включение в заголовки изменчивых данных усложняет кэширование.

Фактические значения URL, используемые в отложенных ответах (`acceptedURL`, `completedURL`, `failedURL` или `cancelURL`), здесь неважны. Не имеет значения, если, например, `acceptedURL` совпадет с `completedURL`.

Стоит хотя бы какое-то время хранить документацию выполненных отложенных ответов. Она может пригодиться для составления отчетов, аудита, внутренней отладки и других нужд.

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 3.7. Обеспечение интероперабельности с помощью передачи состояния между сервисами.
- Рецепт 4.6. Управление форматами представления в среде выполнения.
- Рецепт 4.9. Использование средств гипермедиа в ответе.
- Рецепт 5.4. Выражение внутренних функций в виде внешних действий.
- Рецепт 5.10. Публикация метаданных API.
- Рецепт 6.5. Возвращение метаданных в ответах на запросы.
- Рецепт 7.10. Паттерн «текущая работа» с сохранением состояния.

7.16. Краткосрочные исправления с помощью автоматических повторов

Бывают ситуации, когда HTTP-запрос не выполняется из-за проблем в сети. Зачастую они оказываются временными и разрешаются сами собой буквально за несколько секунд. Текущий рецепт предлагает структурированный способ встраивания в интерфейс сервиса автоматических повторов, которые позволят какое-то время повторять несработавший вызов, прежде чем возвращать HTTP-код ошибки 5xx.

Задача

Иногда выполнение HTTP-запроса может прерываться временными неполадками в сети или отключениями. Как реализовать автоматический повтор запросов в нестабильных условиях работы? Как обеспечить, чтобы долгосрочные сбои (не временные) не блокировали полностью выполнение запросов к сервису?

Решение

При организации автоматического повтора запросов первым делом нужно обеспечить идемпотентность (<https://oreil.ly/nuhwE>) самого запроса. В HTTP методы GET, PUT и DELETE разработаны как идемпотентные. Обычно этого достаточно, но не всегда. Подробнее читайте в разделе «Обсуждение».



Текущий рецепт, как и рецепт 7.19, посвящен внутренним деталям программирования сервера. Все присутствующие здесь элементы следует рассматривать соответственно. Эта информация не относится к такой, которую легко найти в Интернете.

Также вам нужно проанализировать изначальный ответ на запрос, чтобы определить, необходим ли ему повтор. Например, если ответ содержит код состояния 4xx (то есть проблема на стороне клиента), то повтор этого запроса без изменений не приведет к успеху. Если же ошибка отражается кодом состояния 5xx, вполне можно попробовать повторить тот же самый запрос. Наконец, если ошибка произойдет на уровне локального соединения, допустим, из-за разрыва сетевого подключения, тогда также есть смысл попробовать повторить запрос.



Если вы хотите узнать больше о временных проблемах в сети и их решении, рекомендую статью *Transient Fault Handling* (<https://oreil.ly/al0cp>) с портала документации Microsoft Azure.

Когда дело касается решения временных сетевых проблем, есть несколько вариантов действий, одни из которых лучше других. Далее в порядке снижения предпочтительности приведены наиболее актуальные решения с комментариями.

Экспоненциальная задержка (Exponential backoff, EBO)

Это лучшее решение. Например, при возникновении сбоя сети можно подождать 2 с и повторить запрос. Если он снова провалится, подождать уже 4 с и затем 16 с. Пытаться повторить один и тот же запрос более трех раз не рекомендуется (подробнее — в разделе «Обсуждение»).

Инкрементальная задержка (Incremental backoff, IBO)

Можно установить для попыток повтора инкрементируемое значение задержки, например 4 с. Это будет означать, что сначала вы ожидаете 4 с, затем 8 с, потом 16 с, после чего прекращаете попытки.

Регулярные повторы (Regular interval retries, RIR)

Добиться успеха можно путем повтора попытки через одинаковые промежутки времени, например каждые 3 с. Один из недостатков этого подхода в том, что, если одновременно и с одинаковой частотой повторы будут выполнять несколько экземпляров сервиса, это может создать больше проблем, чем вы предполагаете.

Мгновенный повтор (Immediate retry, IR)

Если разрыв связи оказывается очень коротким, есть смысл попробовать повторить запрос сразу же. Это решение может сработать, если проблема возникла в шлюзе или каком-то другом кластере, использующем несколько машин для перенаправления одного запроса. Если вы задействуете это решение, не забудьте повторить его больше двух раз, поскольку иначе состояние сети может просто не успеть измениться.

Рандомизированный повтор (Randomized retry, RR)

Рандомизированные повторы могут исключить узкие места, создаваемые IR и другими решениями. В этом случае можете сгенерировать случайное время ожидания (не более 15 с) и сделать несколько повторов. Недостаток данного

подхода в том, что он может случайно сгенерировать длительные промежутки ожидания, например выполнить все три попытки через 10 с, тогда как до восстановления работы сети по факту было бы достаточно подождать всего пару секунд.

Выбрав подходящий вариант, вы можете настроить параметры повторов в ресурсе конфигурации (см. рецепт 5.5). Такой ресурс должен присутствовать для каждого интерфейса сервиса (или задачи рабочего потока, см. рецепт 7.1). Конкретные метаданные в конфигурации будут зависеть от выбранного метода (подробнее — в разделе «Пример»), но в целом список важных свойств для полноценного блока `retry` будет выглядеть так.

Method

Поддерживаемый вариант повтора (EBO, IBO, RIR, IR или RR).

Max-Retries

Максимальное количество повторов запроса.

Max-Wait-Seconds

Максимальное количество секунд ожидания попытки повтора.

Starting-Value-Seconds

Количество секунд до первой попытки повтора.

Increment-Value-Seconds

Количество секунд до каждого инкрементируемого повтора.

Если попытки повтора провалятся, сервис должен сообщить последний код состояния и полученные детали запроса.



Помните, что целевой сервис может воспринять ваши повторы как атаку типа «отказ в обслуживании» (подробнее — в разделе «Обсуждение»).

Вам также следует зафиксировать свои попытки в локальном журнале для последующего отчета. Эту информацию не нужно сообщать в записи о прогрессе рабочего потока (см. рецепт 7.7), поскольку она относится к внутренним деталям, видеть которые должны только разработчики сервиса.

Пример

Далее приведен пример свойств метаданных, которые можно использовать при записи опций повтора в файл конфигурации или другой источник (см. рецепт 5.5). Вот пример конфигурации повтора в формате XML:

```
<!-- Для метода EBO -->
<retries>
  <method>EBO</method>
```

```
<starting-value-seconds>2</starting-value-seconds>
<max-retries>3</max-retries>
</retries>

<!-- Для метода IBO -->
<retries>
  <method>RIR</method>
  <increment-value-seconds>3</increment-value-seconds>
  <max-retries>3</max-retries>
</retries>

<!-- Для метода RIR -->
<retries>
  <method>RIR</method>
  <starting-value-seconds>2</starting-value-seconds>
  <increment-value-seconds>3</increment-value-seconds>
  <max-retries>3</max-retries>
</retries>

<!-- Для метода IR -->
<retries>
  <method>IR</method>
  <max-retries>3</max-retries>
</retries>

<!-- Для метода RR -->
<retries>
  <method>RR</method>
  <max-retries>3</max-retries>
  <max-wait-seconds>10</max-wait-seconds>
</retries>
```

Показанный здесь блок `retries` можно добавить в имеющийся документ предпочтений клиента (см. рецепт 5.5) или другой источник конфигурации, который использует сервис. Проследите, чтобы случайно не выдать эту информацию неавторизованным сторонам.

Обсуждение

Этот рецепт нацелен на *сокрытие* временных сетевых проблем и попытку решить их без прерывания потока трафика. У него есть недостатки. Например, вы можете иметь дело с регулярным сбоем связи и не замечать его, пока не просмотрите локальные журналы сервиса. По этой причине желательно почаще заглядывать в них.

Важно помнить, что многократный повтор одного и того же запроса *может* быть распознан целевым сервисом как вредоносное поведение, например атака «отказ в обслуживании». Сюда относится как отправка в точности одного запроса, так и выступление в роли источника множества повторяющихся запросов к различным, связанным друг с другом URL. Старайтесь быть сдержанными при настройке автоматических повторов и делайте все возможное, чтобы ваши действия показывали вас как порядочного участника сети.

При журналировании повторов обязательно записывайте их как предупреждения, а не как ошибки. Только когда все повторы провалятся, можете зафиксировать последнюю попытку как ошибку. Это упростит последующее чтение и обработку журналов, когда вы будете искать в них, например, только повторы.

Если отчет вашего журнала покажет, что попытки повторов относятся к одним и тем же сервисам, подумайте, как можно исправить это подключение (если это возможно) или заменить сервис чем-то более надежным.

Подход ЕВО окажется удобен для М2М-взаимодействий. Им не помешает растущее время задержки между повторами. Если вы запрограммируете человеко-машинный интерфейс, например UI вместо API, то более эффективным подходом будет IR и более раннее прекращение попыток.

Помните, что любые задержки при выполнении запросов могут усугубляться длинным списком последовательных действий. Например, если у вас через один плохо работающий шлюз проходят три последовательных запроса, а максимальное время ожидания составляет 10 с для каждой из трех попыток, то эти три запроса могут в общей сложности создать задержку в 90 с. Единственным способом избежать этого будет использование совместимых с рабочим потоком сервисов (см. рецепт 7.1), поддерживающих опции `jobMaxTTL` и `taskMaxTTL`.

В случае же применения повторов к набору параллельных действий максимальная задержка может быть ограничена 30 с, что тоже очень долго.

Читайте также

- Рецепт 3.5. Выражение действий в среде выполнения с помощью встроенных средств гипермедиа.
- Рецепт 3.8. Проектирование с учетом возможности повторения действий.
- Рецепт 5.12. Стандартизация отчетов об ошибках.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 7.7. Предоставление ресурса прогресса, отражающего выполнение рабочих потоков.
- Рецепт 7.17. Поддержка локальной отмены или отката.

7.17. Поддержка локальной отмены или отката

По возможности интерфейс сервиса должен предоставлять потребителям опцию отмены для отката последнего действия. Это может быть отмена обновления данных, удаление только что созданного ресурса или откат предыдущего процесса. При этом существует ряд сложностей, включая вероятность того, что сам внутренний сервис не будет предоставлять такую функциональность. Тем не менее это не означает, что API не сможет ее реализовать.

Задача

Предоставление возможности отмены последнего действия в интерфейсе (добавления, редактирования, изменения и т. д.) — хорошая практика. Тем не менее так делают не все сервисы (код, находящийся за API). Как реализовать ее даже в случаях, когда сервисы такую функциональность не поддерживают? При каких условиях возможность отмены считается желательной, а при каких — нет?



Поддержка локальной отмены — это своеобразный паттерн посредника API. Более обобщенный подход к его реализации описан в рецепте 5.17.

Решение

Как правило, любой сервис оказывается более удобным, если в той или иной степени позволяет отменить выполненные действия. Возможность отмены последнего действия предполагается для всех интерфейсов, предназначенных для людей, вполне актуальна она и для используемых машинами.

Когда возможность отмены предоставляет сам сервис, ее обязательно нужно выразить в виде внешнего действия интерфейса (см. рецепт 5.4). В таких случаях воплотить эту функциональность легко — достаточно просто реализовать ее в API. Тем не менее иногда внутренний сервис не предоставляет функциональность отмены. Но и в таких ситуациях по возможности желательно реализовывать ее в API, создавая своеобразную имитацию, взаимосвязанную с внутренним сервисом (или сервисами).

Прямая отмена

В простых сервисах по типу CRUD отмена реализуется довольно просто. Для этого нужно создать действие сервиса, например `updateCustomer`, вести в индексе локального хранилища историю выполнения допускающих отмену действий по их идентификаторам (`contextId`) и предоставить собственное внешнее действие, например `undoUpdateCustomer`, получающее `contextId`.

При реализации локальной функциональности отмены стоит установить интервал времени, в течение которого можно выполнять откаты. Например, вы можете позволять отменить действие при условии, что в течение 5 с после его завершения будет выполнен соответствующий запрос отмены. Так вы снизите вероятность того, что отмена некоего давнего действия негативно повлияет на другие части системы.

Работая с простыми сервисами в стиле CRUD, можно использовать подход с непосредственной отменой. Если же изменения происходят в нескольких внутренних сервисах или вы не уверены в последствиях реализации прямой отмены действия, можно вместо этого задействовать решение с отложенной отменой.

Поддержка отложенной отмены

Менее опасным способом реализации функциональности отмены будет простое откладывание запрошенного изменения. Например, можно создать действие `updateCustomer` и при выполнении ставить его на ожидание на какое-то время, скажем 5 с. Если за этот промежуток субъект, вызывающий API, поймет, что совершил ошибку, и отправит сообщение `undoUpdateCustomer`, то первый запрос можно будет проигнорировать. В этом случае внутренний (-е) сервис (-ы) не увидит изначальное действие `updateCustomer` и отмена этой невыполненной транзакции не приведет к инвалидации какой-либо другой части системы.

Недостаток же этого подхода в том, что все операции, для которых возможна отмена, будут выполняться с задержкой. Поэтому желательно использовать его только для тех, которые могут поддерживать отложенное выполнение, не нарушая обязательств по быстродействию. Хорошими кандидатами для отложенной отмены станут, например, ночные пакетные обновления, паттерны, работающие посредством очередей, и т. д.

Пример

Есть два возможных шаблона для реализации отмены в интерфейсе сервиса — прямой и отложенный.

Прямая отмена

При прямом подходе мы рассматриваем сервисы CRUD, в которых отмена присутствует в виде внутренней функции:

```
// код внутреннего сервиса
function internal_actions(action, id, object) {
  var rt = null;
  switch(action) {
    case "create":
      rt = create(id, object);
      break;
    case "read":
      rt = read(id);
      break;
    case "update":
      rt = update(id, object);
      break;
    case "delete":
      rt = remove(id);
      break;
  }
}
```

Одним из способов реализации отмены в API будет добавление действия, которое выглядит так:

```
// внешние действия в коде API
function createAction(id, object, context) {
  var rt = null;
  if(writeLogAction(id, object, context)) {
    rt = internal_actions("create", id, object);
  }
  return rt;
}

function undoCreateAction(id, context) {
  var rt = null;
  rt = readLogAction(id, context);
  if(isUndoable(rt)) {
    rt = internal_actions("remove", id);
  }
  else {
    rt = errNotUndoable(id, context);
  }
  return rt;
}
```

Отмена посредством отложенного выполнения

Реализация отката посредством отложенного выполнения выглядит несколько иначе. Эту опцию можно воплотить путем постановки небезопасной операции (создания, обновления или удаления) в очередь на выполнение с задержкой. Затем, если это потребуется, можно будет отменить внесенное изменение *до* его фактической отправки в сервис бэкенда.

В примере далее приводится внутренняя операция, вносящая изменения в три источника: `customers`, `orders` и `products`. Вот упрощенная версия того, как это может выглядеть в коде:

```
// внешнее действие
function scheduleUpdate(inputProperties) {
  // изменение свойств в трех источниках:
  var customers = {...};
  var orders = {...};
  var products = {...};
  context = setTimeout(
    function(){updateSources(customers, orders, products);}, 5000
  );
  return context;
}
```

```
// внешнее действие
function cancelUpdate(context) {
    clearTimeout(context);
}

// внутренняя функция
function updateSources(customers, orders, products) {
    var rt = null;
    // параллельное обновление всех трех источников
    rt = updateWith(customers, orders, products);
    return rt;
}
```

Процесс HTTP-обмена для реализации этого случая может выглядеть так:

```
**** REQUEST ****
PUT /update HTTP/1.1
Host: api.example.org
Content-Type: x-www.form-urlencoded
Accept: text/html
...

customerId=q1w2e3&orderId=p0o9i8&productId=5t6y7u&...

**** RESPONSE ****
HTTP/1.1 202 Accepted
Content-Type text/html
...

<p>Order Updated. You can cancel this order in the next five seconds.</p>
<form name="rollback" method="put" action="http://api.example.org/update/r4t5">
  <input type="hidden" name="contextId" value="r4t5" />
  <input type="submit" value="Undo Order Update" />
</form>
```

Обратите внимание на то, что здесь в качестве ответа используется код состояния **202 Accepted**. Поддержка отмены посредством отложенного выполнения необязательна, но может оказаться полезной для клиентских приложений API, поскольку явно указывает на задержку при обмене сообщениями. Также можно просто вернуть код состояния **200 OK** или **201 Created**, продолжая поддерживать паттерн отмены действий.

Обсуждение

В случае паттерна прямой отмены нужно обеспечить, чтобы текущий контекст выполнения (например, авторизованный пользователь) имел право на выполнение всех внутренних действий, необходимых для отмены. Например, если у авторизованного пользователя нет прав на выполнение **DELETE**, он не должен иметь доступа к функциональности отмены.

Даже если код внутреннего сервиса или сервисов поддерживает простые действия CRUD, но при этом использует небезопасные операции (добавление, изменение или удаление) в отношении более чем одного внутреннего сервиса, например `updateCustomer` и `updateSalesLog`, следует рассмотреть реализацию локальной отмены отложенным методом.



Целью этого рецепта является реализация функциональности отмены внутри интерфейса сервиса. Подробнее о выполнении отмены в отношении нескольких сервисов читайте в рецепте 7.17.

Может показаться, что реализация собственной функциональности отмены вызовет проблемы с внутренними сервисами. Это не так, если ваш код API будет соответствовать представленной этим сервисом функциональности. Иными словами, если сервис поддерживает операцию удаления, не должно иметь значения, используется ли она в отношении записи, сделанной кем-то три недели назад, или той, которую создал потребитель вашего API 5 с назад.

Одним из недостатков отложенного подхода будет добавление в интерфейс сервиса состояний ожидания. Пара секунд могут не показаться чем-то серьезным, но их число может быстро сложиться в более значимые задержки. Представьте, что добавляете трехсекундное ожидание в API, который вызывает три внутренних сервиса. А теперь представьте, что каждый из этих сервисов тоже имеет трехсекундную задержку, и т. д. Прежде чем вы это осознаете, в рабочем потоке может возникнуть пауза выполнения продолжительностью 15–30 с. И это вполне нормально, если речь идет о ночных пакетных обновлениях, но станет серьезной проблемой, если эти API используются для реализации мгновенных обновлений на платформе биржевой торговли.

Читайте также

- Рецепт 3.6. Проектирование последовательных операций записи с помощью идемпотентных действий.
- Рецепт 3.9. Проектирование с учетом возможной отмены действий.
- Рецепт 4.3. Программирование отказоустойчивых клиентов с помощью реализаций на основе сообщений.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 5.17. Использование семантических посредников для доступа к несогласованным сервисам.
- Рецепт 7.16. Краткосрочные исправления с помощью автоматических повторов.
- Рецепт 7.18. Призыв на помощь.

7.18. Призыв на помощь

Бывает, что проблема рабочего потока не решается легко в среде выполнения машинным способом. В таких случаях стоит просто попросить помощи, отправив уведомление человеку, который может решить эту проблему, и затем, если такое возможно, перезапустить рабочий поток либо полностью его отменить.

Задача

Что делать, когда в рабочем потоке возникла ошибка и нет простого способа ее исправить? Какая минимальная информация необходима для отправки просьбы о помощи?

Решение

Не исключены ситуации, когда ваш старательно созданный запрос рабочего потока не выполнится. Иногда эту проблему можно исправить с помощью автоматического повтора (см. рецепт 7.16), но такое решение сработает не всегда. Когда это происходит, лучше просто позвать на помощь специалиста, который решит проблему.

Например, ошибка сети может приводить к сбою запросов, и в данный момент никак не получается это исправить. В таком случае обработчик рабочего потока должен отправить уведомление (обычно электронное письмо или SMS) специалисту, отвечающему за решение подобных проблем.

Уведомление должно включать следующее (или указывать на эти сведения):

- описание рабочего потока (см. рецепты 7.3–7.5);
- ресурс, отражающий прогресс рабочего потока (см. рецепт 7.7);
- общее состояние рабочего потока (см. рецепт 7.1);
- любую другую актуальную информацию, такую как отчеты об ошибках, трейсы и пр.

Передать ее можно либо в виде прикрепленных к письму ZIP-файлов, либо в виде указателей (URL) на соответствующие документы (при условии, что у специалиста есть к ним доступ).

На основе этого ответственный сможет попробовать решить проблему и, если она решаема, продолжить либо перезапустить запрос рабочего потока. Если ни один из этих вариантов невозможен, он просто отменит задачу целиком.

Произошедший случай нужно зафиксировать как инцидент и внести в регулярно просматриваемую коллекцию. Эта коллекция инцидентов может послужить отправной точкой для исправления бага или перепроектирования с целью снижения частоты запросов о помощи в будущем.

Пример

Первым действием этого рецепта является включение ответственной стороны в описание задачи рабочего потока. В случае потоков на основе документов (см. рецепт 7.5) это будет выглядеть так (читайте раздел **Contact** HTML-версии документа рабочего потока):

```
<html>
  <head>
    <title>Shopping Checkout Workflow</title>
  </head>
  <body>
    <h1>Shopping Checkout Workflow</h1>
    <div class="job">
      <span class="jobID">q1w2e3r4t5</span>
      <span class="jobStatus">working</span>
      ...
      <div class="contact">
        <span name="person">Mook Maundy</span>
        <span name="email">mook@example.org</span>
        <span name="sms">1234567890</span>
        <span name="voice">9086574321</span>
      </div>
      <div class="tasks">
        <div class="task">
          <span class="taskID">p0o9i8u7y6</span>
          <span class="taskStatus">completed</span>
          ...
        </div>
        ...
      </div>
    </div>
  </body>
</html>
```

Заметьте, что блок **contact** содержит свойства **name**, **email**, **voice** и **sms**. При этом в нем могут присутствовать значения не всех из них, но как минимум должны быть указаны **email**, **voice** или **sms**. Эти значения могут быть групповыми (например, **support@example.org**) или представлять общую линию техподдержки (например, **1-800-PLS-HELP**).

Если возникает препятствующая выполнению ошибка (такая, которую нельзя решить автоматически), система посредством **email** или **sms** должна отправлять уведомление с актуальными данными или URL, указывающим на них. Например, подобная ошибка может автоматически генерировать следующее сообщение об инциденте, отправляемое на email-адрес в виде приложения к сообщению:

```
{"collection": {
  "title": "CFH Incident Report q1w2e3r4",
  "links": [
    {"rel": "job", "href": "..."},
    {"rel": "progress", "href": "..."},
  ]
}
```

```

    {"rel": "sharedState", "href": "..."},
    {"rel": "errorReport", "href": "..."}
  ]
}}
```

Это пример машиночитаемой версии, но те же данные можно отправить и в понятном для человека виде.

Обсуждение

Реализация этого рецепта является важной частью здорового рабочего потока. В среде выполнения может случиться всякое, поэтому очень важно иметь возможность уведомить человека, способного решить проблему.



Несмотря на то что здесь это не упоминалось, будет нелишним установить для уведомлений рабочего потока определенные коды приоритета. Эта задача выходит за рамки простого проектирования, и ее лучше поручать существующей платформе обработки инцидентов.

При использовании документов рабочего потока (см. рецепт 7.5) генерировать призыв на помощь довольно легко. Если же для описания рабочего потока применяется DSL или код, эта задача усложняется. В случаях, когда рабочий поток выражается через исходный код (см. рецепт 7.3), у вас наверняка не будет возможности поделиться этим кодом и для передачи в отчете потребуется сгенерировать трассировочную или иную внутреннюю информацию.



Проследите, чтобы записи о призыве на помощь не раскрывали публично какую-либо частную или проприетарную информацию, а также связанные с безопасностью данные, которые кто-то может использовать.

Вывод механизма призыва на помощь можно публиковать на информационной панели, характеризуя общее состояние системы. Это поможет отслеживать наиболее частые инциденты и устранять вызывающие их баги или применять обновления. Кроме того, это позволит понять систематичность инцидентов, например, происходят ли они в определенное время рабочего дня. Наконец, вы можете отслеживать время, которое затрачивается на их разрешение, для оценки эффективности реагирования и корректировки работы команд специалистов.

В зависимости от действующей рабочей среды у вас может существовать возможность использовать систему информирования об ошибках для автоматической генерации приоритетного тикета, запускающего собственный поток уведомления и перенаправления. Это необязательно, но наверняка станет более простым решением по сравнению с поддержкой собственного внутреннего процесса уведомления посредством email/sms/voice.

Если вы генерируете собственные уведомления, то для отслеживания инцидентов и управления ими вам наверняка нужно настроить ресурс `list` (см. рецепт 7.11)

и, возможно, даже интерфейс текущей работы (см. рецепт 7.10). В этот момент вы оказываетесь в опасной близости от реализации собственной платформы поддержки рабочего потока, и для создания программы призыва на помощь вам, возможно, потребуется изучить какие-то готовые решения.

Читайте также

- Рецепт 3.3. Предоставление информации о предметной области через публикуемые словари.
- Рецепт 4.11. Проверка свойств данных в среде выполнения.
- Рецепт 5.16. Предоставление зависимым сервисам запасного варианта выполнения.
- Рецепт 6.5. Возвращение метаданных в ответах на запросы.
- Рецепт 7.7. Предоставление ресурса прогресса, отражающего выполнение рабочих потоков.
- Рецепт 7.17. Поддержка локальной отмены или отката.

7.19. Масштабирование рабочего потока с помощью очередей и кластеров

Одной из сложностей реализации рабочего потока в веб-среде является вероятность возникновения высокого трафика — спорадически или постоянно. Рецепт посвящен внутренним техникам программирования, с помощью которых можно более эффективно обрабатывать случаи, когда объем запросов вызывает задержку.

Задача

Как обеспечить, чтобы тяжелый трафик не приводил к падению сервисов рабочего потока или их ненадежному функционированию? Что можно предпринять для упрощения обработки больших объемов трафика и/или многослойных рабочих потоков, не вынуждая потребителей интерфейса сервиса менять свое поведение?

Решение

Если вас беспокоит то, что сервисы рабочего потока могут оказаться перегруженными большим объемом трафика или длительно выполняющимися задачами, можете использовать в интерфейсе две техники программирования, которые позволят повысить его надежность и масштабируемость, — очереди и кластеры.



Текущий рецепт, как и рецепт 7.16, посвящен внутреннему программированию сервиса, и все его элементы нужно рассматривать соответственно. Эта информация не относится к той, которую легко найти в Интернете.

Масштабирование трафика запросов с помощью очередей

Добавив очередь сообщений между интерфейсом сервиса и самим сервисом, можно ускорить *принятие* запроса, даже если в итоге для начала его *обработки* потребуется какое-то время (рис. 7.13).

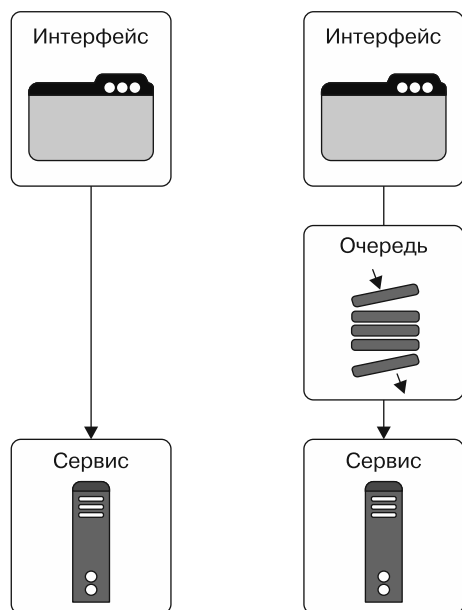


Рис. 7.13. Очереди сообщений помогают увеличить объем запросов

Эта техника особенно легко реализуется, если вы оформляете запросы рабочего потока в виде документов (см. рецепт 7.5). Добавляя между интерфейсом и сервисом очередь, можно по прибытии запроса быстро возвращать ответ **202 Accepted** и затем вносить запрос в долгосрочную очередь. Далее сервис сможет считывать из этой очереди сообщения и обрабатывать их по мере доступности.

Если ваши сообщения превышают установленный лимит времени, достигая `jobMaxTTL` или `taskMaxTTL` до момента своей успешной обработки, вам может потребоваться добавить в рабочий поток дополнительные обработчики для чтения очередей.

Обработка запросов с помощью кластеров

Если в рамках рабочего потока не получается обработать все задачи, скорее всего, в него нужно добавить дополнительные обработчики. Реализовать это проще всего, создав кластер машин (рис. 7.14). Большинство платформ, включая все облачные, предоставляют возможность, которая позволяет объявить для интерфейса сервиса

единый IP-адрес и разместить за ним несколько машин. В результате любая машина этого кластера сможет получать запросы рабочего потока, в том числе считывая их из очереди.

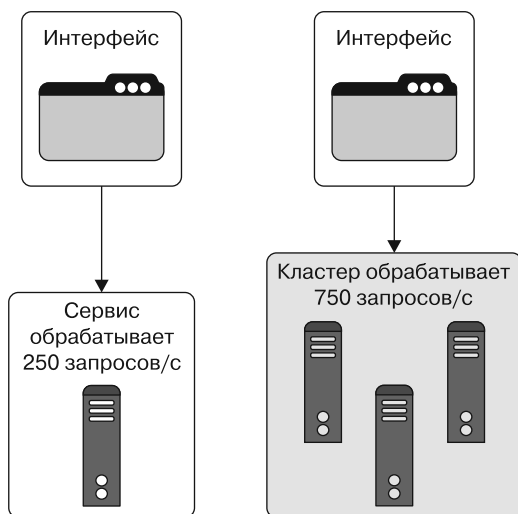


Рис. 7.14. Кластеры помогают увеличить объем обработки



Подробное рассмотрение настройки кластеров машин и управления ими выходит за рамки тематики книги. Прекрасным примером решения по управлению кластерами является Kubernetes. Все ведущие облачные платформы (AWS, Google, Heroku, Microsoft и пр.) тоже предлагают поддержку кластеризации.

Использование кластеров машин тоже позволяет защититься от периодических сбоев отдельных из них. Если одна машина выйдет из строя, остальные смогут взять на себя ее часть работы, пока она не будет перезагружена или заменена.

Хорошо то, что и очереди, и кластеры можно внедрять, не нарушая работу публичного интерфейса сервиса.

Обсуждение

Существует множество решений для реализации очередей за рубежом интерфейса. Простейшим является создание на жестком диске каталога, например `/requests/`, и запись всех запросов в него в виде файлов. Вдобавок к этому нужно создать код, который будет мониторить каталог и считывать находящиеся в нем файлы. Недостаток этого решения в том, что каталог зачастую будет привязан к конкретной машине, усложняя реализацию общих очередей для кластеров машин.

Все ведущие облачные провайдеры предлагают возможность автоматического масштабирования. Это позволяет устанавливать параметры, управляющие масштабом роста объема трафика, автоматически подключать новые машины для обработки дополнительных запросов и затем отключать их по мере уменьшения объема работы. Такое решение порой может оказаться хлопотным в настройке и затратным в обслуживании, но зато позволит успешно справиться с краткосрочным скачком трафика.

В случае как очередей, так и кластеров важно, чтобы сообщения были максимально описательными, общие состояния существовали как независимые ресурсы, а сервисы могли работать параллельно. Все эти возможности относятся к сервисам, совместимым с рабочими потоками, о которых шла речь в рецепте 7.1.

Читайте также

- Рецепт 3.7. Обеспечение интероперабельности с помощью передачи состояния между сервисами.
- Рецепт 4.11. Проверка свойств данных в среде выполнения.
- Рецепт 6.1. Скрытие внутренних деталей хранилища данных.
- Рецепт 6.9. Повышение производительности с помощью директив кэширования.
- Рецепт 7.7. Предоставление ресурса прогресса, отражающего выполнение рабочих потоков.
- Рецепт 7.15. Синхронный повтор незавершенной работы с ответом 202 Accepted.

7.20. Использование посредников рабочих потоков для включения несовместимых сервисов

Бывает, что нужно задействовать в рабочем потоке интерфейс несовместимого сервиса. Текущий рецепт покажет, как это сделать безопасно и когда так делать *не нужно*.

Задача

Что делать, когда у вас есть интерфейс сервиса, несовместимый с рабочим потоком, в который его нужно включить в виде задания? В каких случаях это можно сделать безопасно и когда так делать не рекомендуется?

Решение

Самый простой способ включения интерфейса сервиса в рабочий поток, с которым он несовместим, — его обертывание в совместимого *посредника*. Этот посредник сможет предоставить все аспекты совместимого сервиса (см. рецепт 7.1), включая

все действия, поддержку общего состояния, а также передачу идентификаторов корреляции и запросов.

Посредник рабочего потока должен поддерживать все важные действия, перечисленные в рецепте 7.1, включая:

- выполнение (Execute);
- повтор (Repeat);
- отмену (Revert).

Как правило, действие «выполнение» можно использовать для указания на целевой сервис и выполнение нужной работы. При этом наверняка потребуется запрограммировать процедуры, которые будут считывать содержимое из соответствующего ресурса общего состояния и записывать в него.

Реализация идемпотентного действия повтора и отмены иногда оказывается сложнее. В отношении сервисов только для чтения вроде проверяющих почтовый код по списку или сервисов только для вычислений наподобие вычисляющих налоги с продаж нет смысла реализовывать возможность отмены и можно легко повторять эти действия желаемое количество раз.

Обратите внимание на то, что если целевой сервис меняет состояние на стороне сервера, например корректирует сохраненные данные, ресурс и т. д., то нужно будет обеспечить поддержку как повтора, так и отмены. Если же интерфейс сервиса уже это делает, можно просто связать с целевым сервисом эти опосредованные действия. В противном случае потребуется выяснить, можно ли запрограммировать посредника так, чтобы он обеспечивал необходимую поддержку.

Например, для реализации отмены создания ресурса вы можете закладывать в посредника внутреннюю информацию создаваемых им ресурсов и выполнять по запросу отмену для удаления определенного из них. Причем нужно будет обеспечить, чтобы ресурс, который вы хотите удалить, не вызвал сопутствующего изменения состояния (не изменил другие ресурсы вроде рабочего задания, записи журнала и т. п.). Это может оказаться почти невозможным, если вы или ваша команда не владеете интерфейсом целевого сервиса.

ЗДЕСЬ ОБИТАЮТ ДРАКОНЫ!

По названным здесь причинам вам следует опосредовать лишь сервисы только для чтения. Исключением из этого правила будут случаи, когда вы управляете несовместимым сервисом и уверены, что можете безопасно реализовать в его отношении действий отмены и повтора.

Пример

Предположим, у вас есть несовместимый сервис `computeVAT`, который вы хотите включить в рабочий поток магазина. Для этого потребуется создать совместимый с рабочим потоком промежуточный интерфейс (назовем его `workflowVAT`), который будет поддерживать следующие возможности.

`readSharedState`

Загружает общее состояние в локальную память, готовую предоставить любые данные, необходимые для выполнения действий сервиса.

`inputVATData`

Предоставляет форму для вызова операции выполнения, которая по возможности заполняется свойствами из коллекции `sharedState`.

`executeVATComputation`

Вызывает ключевую функциональность целевого сервиса, например `computeVAT`, используя предыдущую форму, заполненную данными из коллекции `sharedState`.

`writeSharedState`

Записывает результаты выполнения в коллекцию `sharedState`.

`repeatVATComputation`

Повторяет процесс получения данных из коллекции `sharedState`, заполняя форму `inputVATData` и выполняя действие `executeVATComputation`. Это приводит также к вызову действия `writeSharedState` для обновления коллекции `sharedState`.

`RevertVATComputation`

Удаляет любые результаты (действия `executeVATComputation`) из коллекции `sharedState`, по сути, возвращая ее к состоянию *до* любого выполнения, и выполняет `writeSharedState` для внесения действия отмены в коллекцию.

В течение всего этого процесса HTTP-интерфейс для опосредованных действий поддерживает свойства `correlation-id` (идентификатор задачи) и `request-id` (идентификатор задания), которые передаются в сообщениях запросов и ответов.

Вы можете заметить, что посредник выполняет немало тяжелой работы, чтобы сделать возможным использование сервиса `computeVAT`. И ее оказывается так много, что может оказаться разумнее заменить этот сервис таким, который будет совместим с рабочим потоком. Это будет верное решение.

Иными словами, применяйте рецепт по созданию посредников рабочих потоков, только когда не можете заменить несовместимый сервис совместимым.

Обсуждение

Внедрение посредников для рабочих потоков может снизить надежность системы. Вы добавляете в цепочку дополнительный сервис, создавая лишнюю точку, где вероятны обрывы сети и сбои сервиса на стороне посредника или целевого сервиса. Обязательно вносите в свою реализацию необходимые исправления (см. рецепт 5.16).

Особенно рискованной будет попытка опосредовать небезопасный несовместимый сервис, который записывает данные, если вы не контролируете целевой сервис. Например, вы не знаете, что происходит за представленным вам интерфейсом. Есть ли там другие сервисы? Есть ли там цепочка зависимых операций записи, которую не видно? Даже если вы уверены в текущих действиях целевого сервиса, вы не можете знать, что произойдет в случае его обновления или замены владельцем.

Если у вас есть несовместимый сервис, то может возникнуть потребность использовать описанный паттерн посредника в качестве временного решения, которое позволит реализовать нужный рабочий поток на время, пока вы не замените несовместимые сервисы теми, которые отвечают всем требованиям, изложенным в рецепте 7.1. Это типичный паттерн цифровой трансформации, который применяют многие организации. Можно сравнить его с подходом «имитируй, пока не реализуешь». Подробнее об этом сказано в разделе «Трансформация существующих сервисов» главы 8.

Читайте также

- Рецепт 3.4. Описание предметной области с помощью семантических профилей.
- Рецепт 3.6. Проектирование последовательных операций записи с помощью идемпотентных действий.
- Рецепт 4.4. Программирование эффективных клиентов, понимающих профили словарей.
- Рецепт 5.13. Облегчение поиска сервисов с помощью реестра сервисов среды выполнения.
- Рецепт 5.17. Использование семантических посредников для доступа к несовместимым сервисам.
- Рецепт 6.13. Использование промежуточных сервисов для обмена данными.
- Рецепт 7.1. Проектирование сервисов, совместимых с рабочим потоком.
- Рецепт 7.15. Синхронный повтор незавершенной работы с ответом 202 Accepted.
- Рецепт 7.17. Поддержка локальной отмены или отката.

ГЛАВА 8

Подведение итогов

Каждый день — это путешествие, а само путешествие — это жизнь.

Мацуо Басё (Matsuo Basho)

И вот мы подходим к заключительному этапу нашего путешествия — его завершению. Немало материала изучено, и сейчас есть смысл уделить время заключительному пересмотру и рассуждениям.

Свое путешествие мы начали со знакомства (см. главу 1) с основными идеями книги, в том числе обозначив концепцию RESTful Web API и разобрав то, как в нее вписывается модель гипермедиа. Помимо этого, в главе 1 мы определили некоторые *общие принципы масштабируемых сервисов в веб-среде* (см. одноименный раздел главы 1), чтобы проще было ориентироваться при выборе и создании рецептов для книги.

Я также вкратце изложил предысторию (см. главу 2) появления некоторых ключевых технологий и принципов проектирования, которые легли в основу веб-среды и вдохновили меня на разработку приведенных здесь рецептов. Надеюсь, этот исторический экскурс помог вам лучше понять контекст различных подходов к их проектированию и реализации.

Естественно, основное содержание книги пришлось на часть II, в которой и разместились все рецепты.

В завершение же я дал ссылки на дополнительные полезные материалы, а также поделился размышлениями на тему возможных дальнейших шагов.

Использование рецептов

Приведенные в книге рецепты охватывают очень обширное поле, включая проектирование (глава 3), клиентов (глава 4), сервисы (глава 5), данные (глава 6) и рабочий поток (глава 7). И хотя все они имеют единую структуру, некоторые определенно являются более сложными. Несмотря на то что порядок глав был

специально продуман, он может не совпадать с той последовательностью, в которой вам и вашей команде будет удобнее применять рецепты к интерфейсам сервисов, публикуемым или потребляемым внутри вашей организации.

Кроме того, некоторые из этих рецептов несут преимущественно рекомендательный характер (например, рецепт 3.8), а другие являются подробными руководствами по реализации (например, рецепт 7.6). Некоторые сильно зависят от сотрудничества между потребителями и производителями API (например, рецепт 7.13). А есть и такие, для реализации которых не требуется прямое взаимодействие между клиентом и сервером на этапе проектирования или сборки (например, рецепт 5.2). Выяснение того, какие рецепты можно применять независимо, а какие потребуют тесного сотрудничества между всеми сторонами, поможет понять, какие из них обеспечат максимальную отдачу от затраченного времени и вложенных ресурсов.

Стоит отметить, что одним из преимуществ использования этих рецептов является то, что в них тем или иным типичным вещам даны единые имена. Эти рецепты являются своеобразным языком, с помощью которого мы можем говорить о наших вариантах проектирования. По своему опыту могу сказать, что еще до того, как познакомить клиентов с этими рецептами, я вижу примеры применения последних в некоторых из их интерфейсов сервисов. Часто бывает, что команды, даже зная, например, стандартный способ перебора страниц списков, не делятся этим рецептом с другими и не придумывают для него единого имени, которое применялось бы во всей компании. Порой простое использование *имен* этих рецептов может привести в ваши реализации API дополнительную согласованность.

Сначала проект

Мой опыт показывает, что вы можете использовать рецепты в качестве отправной точки для любых случаев, когда нужно создавать интерфейсы сервисов с нуля. Такой подход сработает независимо от того, используете вы API для доступа к существующим сервисам или же создаете код интерфейса и сервиса одновременно. И преимуществом здесь будет то, что за счет рецептов проектирования вы сможете формировать у потребителей и создателей API правильные ожидания, исключая необходимость подгонять обновленные правила интерфейса под уже устоявшиеся сервисы, которые им не соответствуют.

Но даже когда вы начинаете создание сервисов с нуля, важно не поддаваться соблазну воплотить все сразу. Совершенствуйте проекты своих API небольшими шагами. Например, одним из удачных начальных этапов будет установка стандарта, который ограничит новые API конкретным числом форматов представления (см. рецепт 3.1). Внедрение таких возможностей, как повторяемость, отменяемость, расширяемость и изменяемость, станет более сложной задачей, поскольку они требуют, чтобы и потребители, и производители понимали последствия применения рецепта в среде выполнения.

Клиенты и серверы, объединяйтесь!

Главы, посвященные рецептам клиентов (глава 4) и сервисов (глава 5), были написаны как самостоятельные. Тем не менее вы наверняка найдете между ними определенные пересечения и взаимосвязи. Не очень интересно создавать интерфейсы сервисов, если не будет клиентского приложения для их потребления. Поэтому, даже если ваша нынешняя роль в команде заключается в том, чтобы сосредотачиваться только на одной из этих двух взаимодействующих сторон, желательно прочесть обе главы, чтобы получить более полную картину.

Одним из ключевых посылов рецептов клиентов является необходимость предположить определенный уровень независимости от какого-либо конкретного сервиса. Вместо проектирования и создания невольных клиентов, умеющих работать только с одним интерфейсом сервиса, я постарался включить в книгу рецепты, которые помогут создавать клиентские приложения, способные с легкостью обходить несколько сервисов для решения одной задачи (см. рецепты 4.15 и 4.16). И даже если вы все равно будете создавать привязанные к сервисам клиентские приложения, сохранение духа свободы в ваших фронтенд-приложениях повысит стабильность и гибкость базы кода.

Аналогично глава 5 содержит множество рецептов, подчеркивающих важность публикации метаданных о самом интерфейсе сервиса. Предпочтения клиентов (см. рецепт 5.5), файлы определений сервисов (см. рецепт 5.9), документы метаданных API (см. рецепт 5.10) и списки словарей (см. рецепт 5.7) — все это примеры важности предоставления метаданных интерфейса сервиса. И даже рецепты, посвященные ошибкам (см. рецепт 5.12), состоянию (см. рецепт 5.11) и согласованию содержимого (см. рецепт 5.6), показывают, что дополнительная информация о самих запросах к сервису поможет при создании надежных интерфейсов.

Начинать с данных сложно

Я знаю, что многие организации значительную часть своей архитектуры основывают на существующих моделях данных. В некоторых случаях это модели, которые прорабатывались не одно десятилетие и оказываются невероятно мощным способом выражения ключевых бизнес-правил и практик. Для таких компаний применение основанных на данных рецептов из главы 6 может представлять определенные сложности. Используемый в этой книге подход к данным предполагает, что ваши сервисы имеют собственное закрытое хранилище данных и взаимодействие с сервисами реализуется передачей свойств в среде выполнения (например, рецепт 7.2), а не обращением нескольких сервисов к одним и тем же таблицам базы данных. В таких случаях наиболее полезными рецептами обработки данных окажутся те,

которые описывают способы представления запросов (см. рецепт 6.4) и ответов (см. рецепт 6.5), а также посвященные сокрытию моделей данных (см. рецепт 6.1) и кэшированию результатов ответа (см. рецепт 6.9).

Тем не менее, как я уже писал, если вы проектируете и реализуете новые сервисы хранения данных, то в главе 6 есть множество рецептов, которые подскажут, какие использовать технологии, поисковые библиотеки и как управлять целостностью.

Следование потоку

Глава 7 была посвящена программированию рабочих потоков. В ней мы объединили многие рецепты из предыдущих глав для решения задач с помощью коллекции из нескольких сервисов. В этой главе используется особый подход, отличающийся от применяемого во многих известных мне книгах по данной тематике. Вместо того чтобы акцентироваться на противопоставлении оркестрации и хореографии, приведенные в ней рецепты активно опираются на возможности гипермедиа, рассмотренные ранее, создавая более декларативный, общедоступный и распределенный способ включения сервисов в рабочий поток.

Поскольку я показываю иной подход, реализовать эти рецепты с помощью типичных платформ рабочих потоков может оказаться сложно. Тем не менее мой опыт показывает, что по мере выработки навыка *мыслить в рамках гипермедиа* (см. одноименный раздел главы 2) эти рецепты станут менее туманными и более простыми для перевода в рабочий код.

Также стоит отметить, что для простоты усвоения я сгруппировал рецепты, посвященные рабочим потокам. Рецепты 7.1–7.5 дают общее представление об основах надежной системы рабочего потока. В рецептах 7.8–7.13 рассматриваются простые решения для реализации типичных взаимодействий наподобие навигации по спискам, отправки форм, взаимодействий при «выполняемой работе» и пр. Есть и другие рецепты, акцентирующие на конкретных сложностях определения реализаций рабочих потоков и управления ими, но одним из самых важных является рецепт 7.6, который демонстрирует расширяемую основу обобщенного языка управления задачами в RESTful-системах, позволяя решить практически любую проблему рабочего потока.

Как и во всех других главах, при реализации рецептов, посвященных рабочему потоку, лучше всего начинать не спеша и по возможности искать простые решения. Можете смело пропускать рецепты, которые кажутся вам слишком сложными. При этом ничто не мешает вам изменять их и даже создавать собственные, которые будут лучше соответствовать потребностям вашей организации.

Трансформация существующих сервисов

Всякий раз, проходя трансформацию, совершайте наименьшее, что дает максимальный результат, и затем повторяйте это снова и снова.

Адриан Кокрофт (Adrian Cockcroft), бывший технический директор Netflix Cloud

Коллекция рецептов вместе с вводными материалами из глав 1 и 2 обеспечивает твердую основу для трансформации существующих сервисов в соответствии с описанным здесь подходом на основе гипермедиа. Но простое изменение интерфейсов сервисов без какого-либо ясного плана или предположений об отдаче от его проведения может оказаться не только неприятным, но и опасным процессом.

Как и любая другая работа по трансформации, она приносит плоды, только если изменение существующей системы несет очевидные преимущества. Лучшей стратегией будет не пытаться втиснуть множество важных корректировок в один релиз, а постепенно вносить небольшие независимые изменения.

Вы можете использовать рецепты из книги для определения одной задачи (например, «Как повысить быстродействие запросов данных?»), а затем вносить в сервисы или API одно или несколько изменений, которые приведут к измеримым улучшениям. Акцент на внесении изменений, которые приносят компании измеримые улучшения, дает своеобразный накопительный эффект. По мере того как каждое изменение приводит к повышению быстродействия и стабильности работы в среде выполнения, попутно снижая потребность в ресурсах при обновлениях, этот процесс придает вашей работе импульс и упрощает получение поддержки для реализации других инициатив по улучшению внутри организации. Хорошая работа способствует еще большему ее улучшению.

Эта книга содержит немало рецептов, призванных упростить процесс трансформации. Они называются переходными (это рецепты 5.17, 6.13 и 7.20) и нацелены на то, чтобы сделать возможным (в некоторых случаях) *обертывание* имеющихся сервисов в тонкий промежуточный интерфейс, который более точно соответствует другим рецептам книги. Это дает возможность модернизировать некоторые существующие сервисы, обеспечить возможность персонализированной настройки для не принадлежащих вам внешних сервисов и привнести в экосистему API согласованность. Причем все это не потребует переписывать код или заменять существующие сервисы, которые по-прежнему являются целесообразными и функциональными. Как говорилось в этих промежуточных рецептах, их использование имеет некоторые недостатки, поэтому при реализации нужно быть осторожным.

По мере того как ваша компания будет применять все больше таких рецептов к существующим и новым API, вы сможете заметить изменения в нетехнических областях, включая динамику работы команд и повышение эффективности труда отдельных специалистов. Поскольку выбранные рецепты создавались для того,

чтобы обеспечить поддержку стандартного набора общих принципов (см. раздел «Общие принципы масштабируемых сервисов в веб-среде» главы 1), они также могут помочь командам освободиться от ограниченности каскадной модели разработки и повысить уровень креативности в организации. Создавая и публикуя сервисы, которые позволяют командам работать в собственном темпе и безопасно изучать креативные решения, не беспокоясь о нарушении работы других частей экосистемы, ваша компания может начать проявлять некоторые из достоинств, заложенных в рецепты книги.

Если вы участвуете в согласованном начинании по трансформации технических элементов IT-департаментов, будьте готовы к изменениям в культуре компании.

Дополнительные ресурсы

Эта книга стала большим проектом, и, честно говоря, на ее страницы попало не все, что было запланировано. Время, ресурсы и приоритеты порой могут работать против автора, вынуждая исключать какие-то материалы. Кроме того, есть сопутствующие проекты вроде образцов, онлайн-документации для рецептов, видео и других элементов, которые существуют вне книги. Поэтому я решил привести ссылки на дополнительные материалы, которые вы можете использовать для обогащения своего опыта ее прочтения.

Вот ресурсы, к которым вы можете обратиться теперь, когда книга выпущена.

WebAPICookbook.com

Это базовая страница для книги и всех связанных с ней материалов. Если вы ищете дополнительные ресурсы, указатели на новую информацию или просто хотите проверить состояние проекта книги, начните с этого сайта.

@WebAPICookbook

Это аккаунт X (бывший Twitter, <http://twitter.com/webapicookbook>), связанный с книгой. Подпишитесь на него, чтобы не пропустить уведомления об обновлении содержания, добавлении новых ресурсов, презентаций и пр.

Видео

Я планирую выпустить серию видео, посвященных некоторым из материалов книги, а также интервью с проектировщиками, архитекторами и программистами API. Размещаться эти ролики будут на моем YouTube-канале (<http://youtube.com/mamund>), также вы можете отслеживать их появление на странице аккаунта X.

Дополнительные рецепты

Я постепенно буду размещать онлайн дополнительные рецепты, которые не попали в эту редакцию книги. Следите за подробностями на моей странице в X, а также заглядывайте в связанный с книгой репозиторий GitHub (<http://github.com/webapicookbook>).

Каталог ALPS

Для многих рецептов книги существует обширный онлайн-каталог (<http://webapicookbook.github.io/alpsdocuments>) документов семантики профилей прикладного уровня (Application-Level Profile Semantics, ALPS). В нем вы найдете дополнительные документы ALPS, а также образцы реализаций рецептов.

Дополнительный контент

Со временем могут появиться дополнительные материалы, включая презентации, подкасты и т. п. Я буду стараться поддерживать актуальный список и этих ресурсов.

Наконец, если вы будете размещать что-либо онлайн или увидите где-нибудь ссылки на коллекцию рецептов, дайте мне знать через аккаунт X, и я добавлю их в свой список.

Дальнейшие шаги

Итак, теперь, когда вы добрались до конца этой части путешествия, что делать дальше? Надеюсь, вы уже в некоторой степени научились мыслить категориями гипермедиа и поняли, как применять эти рецепты в собственных проектах API. Вы можете даже придумывать собственные рецепты для решения важных задач, возникающих в вашем сообществе. Теперь самое время начинать применять изученные идеи в своей работе. Это лучший способ понять, какие из пройденных рецептов действительно вам подойдут, а какие нужно разработать и задокументировать.

Помните руководящий принцип книги, и это станет еще одним способом продумывания следующих шагов. Как вы и ваша команда можете задействовать глобальный охват веб-среды для решения задач, о которых вы еще не думали, для людей, с которыми не встречались? Даже если вы не используете рецепты непосредственно, следование этому принципу позволяет устанавливать связь с профессиональными и основанными на личных интересах сообществами и несколько упрощает жизнь.

Надеюсь, эта книга вдохновила вас на то, чтобы креативно подходить к тому, как можно использовать средства гипермедиа и Интернет для решения задач новым, творческим способом. Никто не знает, что ждет нас в будущем, но, как сказал Алан Кей (повторю):

«Лучший способ предсказать будущее — изобрести его».

Каким бы путем вы ни пошли, я желаю вам удачи!

Приложения

ПРИЛОЖЕНИЕ А

Руководящие принципы RESTful Web API

Основополагающий принцип рассмотренной коллекции рецептов звучит так:

«Использовать глобальный охват для решения задач, о которых вы никогда не думали, для людей, с которыми никогда не встречались».

Сопутствующие принципы представлены на рис. А.1 и описаны далее.

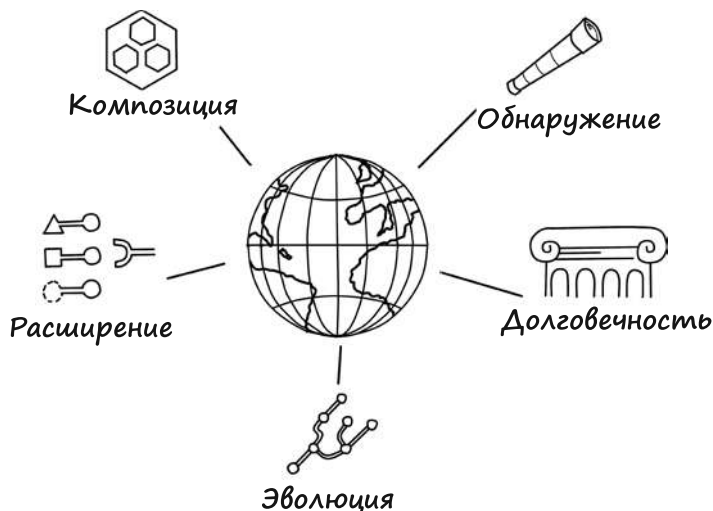


Рис. А.1. Принципы RESTful Web API

Обнаружение

Хорошие рецепты повышают глобальный охват, то есть обеспечивают возможность делиться нашими решениями, находить и использовать решения других.

Расширение

Хорошие рецепты позволяют другим применять грамотно построенные сервисы такими способами, которых мы не предполагали.

Композиция

Хорошие рецепты позволяют незнакомцам (сервисам и/или людям) безопасно и успешно взаимодействовать друг с другом для решения одной задачи.

Эволюция

Хорошие рецепты повышают долговечность и обеспечивают возможность независимой эволюции на протяжении десятилетий.

Долговечность

Хорошие рецепты признают, что нет ничего постоянного и со временем будут происходить изменения.

ПРИЛОЖЕНИЕ Б

Дополнительные материалы

Далее приведен список полезных материалов для чтения, которые лично я держу на своей виртуальной книжной полке. Большинство из них были упомянуты на страницах книги.

- *Alexander C.* A Pattern Language. New York: Oxford University Press, 1977.
- *Alexander C.* The Timeless Way of Building. New York: Oxford University Press, 1979.
- *Amundsen M.* Building Hypermedia Clients with HTML5 and Node. Sebastapol, CA: O'Reilly Media, Inc., 2011.
- *Amundsen M.* Design and Build Great Web APIs. Raleigh, NC: Pragmatic Bookshelf, 2020.
- *Amundsen M.* RESTful Web Clients. Sebastapol, CA: O'Reilly Media, Inc., 2017.
- *Amundsen M.* What Is Serverless? Sebastapol, CA: O'Reilly Media, Inc., 2020.
- *Amundsen M., Gillig D.* API Strategy for Decision-Makers. Sebastapol, CA: O'Reilly Media, Inc., 2022.
- *Bardini T.* Bootstrapping. Stanford, CA: Stanford University Press, 2000.
- *Bass L. et al.* Software Architecture in Practice. 4th ed. Boston: Addison-Wesley, 2021.
- *Berners-Lee T., Fishetti M.* Weaving the Web. New York: Harper-Collins, 2008.
- *Evans E.* Domain-Driven Design. Sebastapol, CA: O'Reilly Media, Inc., 2003.
- *Gibson J.J.* The Ecological Approach to Visual Perception. New York: Psychology Press, 1979.
- *Gibson J.J.* The Senses Considered as Perceptual Systems. Boston: Houghton Mifflin, 1966.
- *Hall W. et al.* Rethinking Hypermedia. New York: Springer US, 1996.
- *Higginbotham J.* Principles of Web API Design. Boston: Addison-Wesley, 2021.
- *Hohpe G., Woolf B.* Enterprise Integration Patterns. Boston: Addison-Wesley, 2003.

- *Lauret A.* The Design of Web APIs. Shelter Island, NY: Manning Publications, 2019.
- *Meadows D.* Thinking in Systems. White River Junction, VT: Chelsea Green Publishing, 2008.
- *Medjaoui M. et al.* Continuous API Management. 1st ed. Sebastapol, CA: O'Reilly Media, Inc., 2018.
- *Medjaoui M. et al.* Continuous API Management. 2nd ed. Sebastapol, CA: O'Reilly Media, Inc., 2021.
- *Nadareishvili I. et al.* Microservice Architecture. Sebastapol, CA: O'Reilly Media, Inc., 2016.
- *Nadareishvili I., Mitra R.* Microservices: Up and Running. Sebastapol, CA: O'Reilly Media, Inc., 2020.
- *Nelson T.* Computer Lib / Dream Machines. Tempus Books, 1974.
- *Newman S.* Building Microservices. Sebastapol, CA: O'Reilly Media, Inc., 2021.
- *Norman D.* The Design of Everyday Things. New York, NY: Basic Books, 2013.
- *Richardson C.* Microservices Patterns. Shelter Island: Manning Publications, 2018.
- *Richardson L. et al.* RESTful Web APIs. Sebastapol, CA: O'Reilly Media, Inc., 2013.
- *Rosenfeld L. et al.* Information Architecture. 4th ed. Sebastapol, CA: O'Reilly Media, Inc., 2015.
- *Russell S., Norvig P.* Artificial Intelligence. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1995.
- *Shiflett C.* HTTP Developer's Handbook. Indianapolis, IN: Sams Publishing, 2003.
- *Taylor R. N. et al.* Software Architecture. Hoboken, NJ: John Wiley & Sons, 2008.
- *Wolff E.* Microservices. Boston: Addison-Wesley, 2016.
- *Wright A.* Cataloging the World. New York: Oxford University Press, 2014.
- *Yourdon E., Larry E.* Constantine. Structured Design. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1975.

ПРИЛОЖЕНИЕ В

Сопутствующие стандарты

По всей книге я ссылался на документы стандартов. Большинство из них открытые, но есть и проприетарные или частные работы. Далее привожу полный список всех упомянутых.

Приемлемые зарегистрированные медиатипы для RESTful Web API

Этот раздел содержит актуальный на момент выхода оригинала книги перечень медиатипов, которые следует иметь в виду при создании собственных RWA. Большинство из них уже зарегистрированы Администрацией адресного пространства Интернета (IANA, <https://oreil.ly/EEAst>), некоторые еще проходят этот процесс, а часть только была создана.



К моменту прочтения вами этой книги могут быть добавлены новые медиатипы, а некоторые из перечисленных здесь — исключены. Так что желательно следить за изменениями в их реестре.

Я перечислил только те форматы, с которыми имел дело в прошлом. Здесь наверняка отсутствуют какие-то известные вам или, наоборот, есть незнакомые. Просто сосредоточьтесь на тех, которые вам больше всего подойдут, и можете смело изменять их список в соответствии с собственным опытом и нуждами. Подробнее о структурированных и неструктурированных типах читайте в рецепте 3.2.

Структурированные медиатипы

- HTML (<https://www.w3.org/TR/html52>);
- Collection + JSON (<http://amundsen.com/media-types/collection>);
- UBER (<http://uberhypermedia.com/>);

- HAL (https://en.wikipedia.org/wiki/Hypertext_Application_Language);
- SIREN (<https://github.com/kevinswiber/siren>);
- HAL-FORMS (<https://rwcbook.github.io/hal-forms>);
- Atom (<https://datatracker.ietf.org/doc/html/rfc4287>).

Неструктурированные медиатипы

- XML (<https://www.w3.org/standards/xml>);
- JSON (<https://www.rfc-editor.org/rfc/rfc8259.html>);
- CSV (<https://www.rfc-editor.org/rfc/rfc4180.html>);
- JSON-LD (<https://json-ld.org/spec>).

Незарегистрированные медиатипы

- YAML (<https://yaml.org/>);
- JSON API (<https://jsonapi.org/>).

Гипермедиаформаты неравнозначны. Например, формат Hypertext Application Language (HAL) стандартизирует ссылочные элементы, но не формы или тела данных. Формат Structured Interface for Representing Entities (SIREN) стандартизирует ссылки и формы, но не тела данных. HTML, Collection + JSON и Uniform Basis for Exchanging Representations (UBER) стандартизируют все три элемента сообщений. Я создал несколько форматов полизаполнения гипермедиа для SIREN (Profile Object Description) и HAL (HAL-FORMS), чтобы восполнить пробелы, но они могут усложнить реализации.

Форматы определения API

Далее приведен список распространенных форматов определения API (в алфавитном порядке), используемых на момент написания книги. Вы можете применять эти форматы для определения конкретных деталей API ваших сервисов (URI, методов запросов, кодов ответов, тел сообщений, метаданных и т. д.). Некоторые из них созданы для классических HTTP-взаимодействий «запрос/ответ», другие служат для асинхронного обмена сообщениями. При реализации API своих сервисов вы наверняка примените более одного из этих форматов:

- AsyncAPI (<https://oreil.ly/MFPnV>);
- OpenAPI (<https://oreil.ly/udjzo>);
- Protobuf (<https://oreil.ly/g55LO>);
- RAML (<https://oreil.ly/nvdyY>);

- Schema Definition Language (практикум, <https://oreil.ly/frT9a>);
- WSDL (<https://oreil.ly/UkG7a>);
- WADL (<https://oreil.ly/HmqDW>).



К моменту прочтения вами книги приведенные здесь версии могут устареть, поэтому обязательно проверяйте версии и даты выхода форматов, которые хотите поддерживать.

Форматы документов семантических профилей

На момент написания книги существуют два распространенных SPD, а также устаревший формат XMDP, который мне все еще иногда попадает:

- Dublin Core Application Profiles (DCAP, <https://oreil.ly/BRF93>);
- Application-Level Profile Semantics (ALPS, <http://alps.io/spec>);
- XHTML Meta Data Profiles (XMDP, <http://gmpg.org/xmdp>).

Сопутствующие форматы гипермедиа

Далее перечислены форматы, призванные восполнить пробелы в некоторых форматах гипермедиа (POD и HAL-FORMS), а также выразить другие связанные с гипермедиа метаданные на этапе проектирования или сборки (WESTL):

- Profiles for Object Description (POD, <http://rwcbook.github.io/pod-spec>);
- HAL-FORMS Media Type (<http://rwcbook.github.io/hal-forms>);
- Web Service Transition Language (WESTL, <http://rwcbook.github.io/wstl-spec>).

Использование HyperCLI

Утилита `hyper` — это простая оболочка/REPL в стиле командной строки для взаимодействия с онлайн-сервисами/API. Являясь полноценным HTTP-клиентом, `hyper` особенно эффективно проявляет себя в работе с гипермедиа-сервисами, включая Collection + JSON (<https://oreil.ly/LCAu2>), SIREN (<https://oreil.ly/z3DCK>) и HAL (<https://oreil.ly/SU7Jy>). В будущем планируется добавление в этот инструмент поддержки PRAG + JSON (<https://oreil.ly/1wB73>), MASH + JSON (<https://oreil.ly/MhYdr>) и, возможно, UBER (<https://oreil.ly/iNBAJ>). Посредством плагинов можно добавить поддержку и других медиатипов.

Вместе с HTTP и командами, поддерживающими медиатипы, `hyper` предоставляет удобную функциональность наподобие команд оболочки, управления файлами конфигурации и стека LIFO для обработки локальных переменных памяти.



Обучающие материалы, примеры, исходный код и последний релиз инструмента HyperCLI доступны на странице HyperCLI (<https://oreil.ly/RKhJ8>).

Hello, Hyper!

Вот простейший скрипт HyperLANG:

```
GOTO https://company-atk.herokuapp.com
```

Выполняется он так.

1. Установите последнюю версию HyperCLI:

```
$> npm install -g @mamund/hyper
```
2. Запустите HyperCLI REPL:

```
$> hyper
```
3. Введите следующую команду и нажмите **Enter**:

```
> GOTO https://company-atk.herokuapp.com
```

Для запуска сервиса может потребоваться несколько секунд, но в итоге вы должны увидеть следующий ответ:

```
STATUS 200
https://company-atk.herokuapp.com
application/forms+json; charset=utf-8
```

Вы также можете просматривать детали запросов/ответов с помощью этих команд:

```
SHOW REQUEST
SHOW RESPONSE
SHOW METADATA
SHOW ALL
SHOW URL
SHOW STATUS
SHOW CONTENT-TYPE
SHOW HEADERS
```

Наконец, можно собрать все команды HyperLANG в текстовый файл и подключить его к HyperCLI так:

```
$> hyper <hello-hyper.script
```

Поздравляю! Вы только что создали свою первую программу HyperLANG.

Прочая информация

Вот некоторые дополнительные подробности о проекте HyperCLI.

Примеры

Много рабочих примеров можно найти в каталоге `/scripts/` репозитория GitHub.

Поддержка плагинов

Вы можете создать собственный плагин для `hyper` и поместить его в каталог `/plugins/` проекта. В результате он автоматически загрузится в среде выполнения. Подробнее об этом читайте в разделе *Plug-In Authoring* репозитория GitHub.

Поддержка пакетов NPM

Можно скачивать и отслеживать последние обновления пакета HyperCLI из npm (<https://oreil.ly/uRvov>).

Исходный код

Исходный код этой утилиты можно найти в каталоге `/src/` репозитория GitHub.

Команды HyperCLI

В табл. Г.1 приводятся ключевые слова команд HyperCLI (на момент выхода оригинала книги).



Обязательно ознакомьтесь с последней документацией в репозитории GitHub.

Таблица Г.1. Команды HyperCLI с описанием

#	Комментарий
VERSION	Возвращает информацию о версии
EXIT STOP	Приостановка и выход с кодом 0
EXIT-ERR	Приостановка и выход с кодом 1
EXIT-IF	Приостановка и выход с кодом 1 при встрече простого условия
.. INVALID-URL <url #\$>	Возвращает TRUE, если строка не является валидным URL
.. STACK-EMPTY	Возвращает TRUE, если внутренний стек пуст
CLEAR	Очистка консоли
SHELL	Поддержка простой оболочки (bash/dos)
.. LS DIR [folder/path]	
PLUGINS	Возвращает список загруженных внешних плагинов
CONFIG (READ)	Возвращает NVP сохраненных данных конфигурации
.. FILE LOAD [filename]	Загружает файл конфигурации (по умолчанию hyper.config)
.. SAVE WRITE [filename]	Загружает файл конфигурации (по умолчанию hyper.config)
.. SET <{ n:v,... }>	Запись в общий файл конфигурации
.. CLEAR	Удаляет все настройки
.. RESET	Сброс к настройкам по умолчанию
.. REMOVE <string>	Удаляет указанный элемент
STACK	Стек LIFO объектов JSON
.. CLEAR FLUSH	Удаляет все элементы из стека
.. PEEK	Показывает верхний объект JSON в стеке
.. PUSH <{ n:v,... }>	Добавляет новый объект JSON в стек
.. PUSH WITH-RESPONSE	Добавляет в стек новый элемент из вершины стека ответов
.. PUSH WITH-PATH <path-string \$#>	Добавляет в стек элемент, который является результатом JSONPATH

Продолжение ➤

Таблица Г.1 (продолжение)

#	Комментарий
.. EXPAND-ARRAY	Расширяет массив на вершине стека до <i>n</i> элементов стека
.. POP	Удаляет из стека верхний элемент
.. LEN LENGTH	Возвращает глубину стека
.. SET <{"n":"v"...}>	Обновляет объект JSON на вершине стека
.. LOAD FILE [filename]	Считывает один объект JSON с диска в стек (по умолчанию в hyper.stack)
.. SAVE WRITE [filename]	Записывает верхний элемент стека на диск (по умолчанию из hyper.stack)
.. DUMP [filename]	Записывает весь стек на диск (по умолчанию в hyper.dump)
.. FILL [filename]	Заменяет текущий стек содержимым из файла на диске (по умолчанию из hyper.dump)
OAUTH OAuth 2.0 support	
.. LOAD [filename]	Загружает файл конфигурации OAuth (по умолчанию oauth.env)
.. SAVE [filename]	Сохраняет файл конфигурации OAuth (по умолчанию oauth.env)
.. DEFINE <string> <{ n:v,... }>	Создает запись в конфигурации OAuth
.. UPDATE <string> <{ n:v,... }>	Изменяет настройки в конфигурации OAuth
.. GENERATE GEN <string>	Генерирует токен доступа с использованием данных конфигурации
.. REMOVE <string>	Удаляет указанный элемент
ACTIVATE CALL GOTO GO	Создает HTTP-запрос
.. WITH-URL <url \$#>	Использование URL для создания запроса
.. WITH-REL <string \$#>	Использование значения HREF для связанного элемента in-doc
.. WITH-ID <string \$#>	Использование значения HREF для связанного элемента in-doc
.. WITH-NAME <string \$#>	Использование значения HREF для связанного элемента in-doc
.. WITH-PATH <json-path-string \$#>	Использование в качестве URL значения из результата JSONPath
.. WITH-ACCEPT <string \$#>	Непосредственная установка заголовка accept
.. WITH-HEADERS <{ n:v,... } \$#>	Запрос заголовков
.. WITH-OAUTH <string \$#>	Установка заголовка авторизации с использованием указанного токена OAUTH
.. WITH-BASIC <string \$#>	Установка заголовка авторизации с использованием указанной конфигурации BASIC
.. WITH-QUERY <{ n:v,... } \$#>	Запрос аргументов строки в виде JSON NVPs
.. WITH-BODY <name=value&... \$#>	Для POST/PUT/PATCH (по умолчанию app/form-urlencoded)
.. WITH-METHOD <string \$#>	Для установки HTTP-метода (по умолчанию GET)
.. WITH-ENCODING <media-type \$#>	Для установки индивидуально настроенной кодировки POST/PUT/PATCH
.. WITH-FORMAT	Установка заголовка accept со значением конфигурации

#	Комментарий
.. WITH-PROFILE	Установка заголовка профиля link со значением конфигурации
.. WITH-FORM <name> \${#}>	Использование метаданных указанной формы (URL, METHOD, ENCODING, FIELDS) для построения HTTP-запроса (только для SIREN)
.. WITH-STACK	Использование верхнего элемента стека в виде набора переменных для других операций (например, для заполнения форм, передачи значений строки запроса, заголовков и т. д.)
.. WITH-DATA <name=value&... \${#}> DISPLAY SHOW	Заполнение форм подходящими данными Показ сохраненного ответа (с вершины стека LIFO)
.. ALL	Возвращает полное взаимодействие (запрос, метаданные ответа, тело ответа)
.. REQUEST	Возвращает информацию о запросе (URL, метода, строки запроса, тела и заголовков)
.. METADATA META	Возвращает метаданные ответа (URL, статуса и заголовков)
.. RESPONSE PEEK	Возвращает тело ответа с вершины стека
.. URL HREF	Возвращает фактический URL ответа
.. STATUS STATUS-CODE	Возвращает HTTP-статус ответа
.. CONTENT-TYPE	Возвращает тип HTTP-содержимого ответа
.. HEADERS	Возвращает всю коллекцию HTTP-заголовков ответа
.. POP	Удаляет ответ из стека
.. CLEAR FLUSH	Удаляет все ответы из стека
.. LEN LENGTH	Возвращает длину сохраненного стека
.. PATH <JSONPath XMLPath \${#}>	Возвращает результаты запроса (XML или JSON) из ответа с вершины стека
.. JPATH <JSONPath \${#}>	Возвращает результаты запроса JSONPath из ответа с вершины стека
.. XPATH <XMLPath \${#}>	Возвращает результаты запроса XPath из ответа с вершины стека
CJ	Возвращает строго типизированную версию ответа из вершины стека (vnd.collection+json)
.. METADATA	Возвращает массив метаданных из ответа Collection + JSON
.. LINKS	Возвращает массив ссылок из ответа Collection + JSON
.. ITEMS	Возвращает массив элементов из ответа Collection + JSON
.. QUERIES	Возвращает массив запросов из ответа Collection + JSON
.. TEMPLATE	Возвращает коллекцию шаблонов из ответа Collection + JSON
.. ERROR ERRORS	Возвращает объект error из ответа Collection + JSON
.. RELATED	Возвращает объект related из ответа Collection + JSON
.. ID NAME REL TAG <string \${#}>	Возвращает один узел
.. IDS RELS NAMES TAGS	Возвращает простой список

Таблица Г.1 (продолжение)

#	Комментарий
.. PATH <JSONPath \$#>	Возвращает результаты запроса JSONPath из ответа Collection + JSON
HAL	Возвращает строго типизированную версию ответа из вершины стека (vnd.hal+json)
.. LINKSHAL	Возвращает массив ссылок из ответа
.. EMBEDDED	Возвращает массив элементов из ответа HAL
.. ID REL KEY NAME TAG <string \$#>	Возвращает один узел
.. IDS RELS KEYSTAGS	Возвращает простой список
.. PATH <JSONPath \$#>	Возвращает результаты запроса JSONPath из ответа HAL
SIREN	Возвращает строго типизированную версию ответа из вершины стека (vnd.siren+json)
.. LINKS	Возвращает массив ссылок из ответа SIREN
.. ACTIONS FORMS	Возвращает массив действий из ответа SIREN
.. ENTITIES	Возвращает массив сущностей из ответа SIREN
.. PROPERTIES	Возвращает массив свойств из ответа SIREN
.. TAG CLASS <string \$#>	Возвращает узлы, связанные со значением CLASS
.. ID ENTITY <string \$#>	Возвращает сущность, связанную с ID
.. REL LINK <string \$#>	Возвращает ссылку, связанную с REL
.. NAME FORM ACTION <string \$#>	Возвращает действие, связанное с NAME
.. IDS RELS NAMES FORMS TAGS CLASSES	Возвращает простой список
.. PATH <JSONPath \$#>	Возвращает результаты запроса JSONPath из ответа SIREN
WSTL	Возвращает строго типизированную версию ответа из вершины стека (vnd.wstl+json)
.. TITLE	Возвращает строку title из ответа WSTL
.. ACTIONS	Возвращает массив действий из ответа WSTL
.. DATA	Возвращает массив сущностей из ответа WSTL
.. RELATED	Возвращает объект related из ответа WSTL
.. CONTENT	Возвращает объект content из ответа WSTL
.. ID REL NAME FORM TAG TARGET <string \$#>	Возвращает один узел
.. IDS RELS NAMES FORMS TAGS TARGETS	Возвращает простой список
.. PATH <JSONPath \$#>	Возвращает результаты запроса JSONPath из ответа WSTL
FJ	Возвращает строго типизированную версию ответа JSON + FORMS из вершины стека (forms+json)
.. METADATA	Возвращает из ответа массив метаданных
.. LINKS	Возвращает из ответа массив ссылок

#	Комментарий
.. ITEMS	Возвращает из ответа массив элементов
.. ID <string \$#>	Возвращает элемент (метаданные, ссылку, элемент), связанный с ID
.. TAG <string \$#>	Возвращает совпадающие узлы
.. REL <string \$#>	Возвращает ссылку, связанную с REL
.. NAME <string \$#>	Возвращает элемент (метаданные, ссылку, свойство), связанный с NAME
.. IDS NAMES RELS FORMS TAGS	Возвращает простой список
.. PATH <JSONPath \$#>	Возвращает из ответа результаты запроса JSONPath
MASH	Возвращает строго типизированную версию ответа из вершины стека (vnd.mash+json)
.. METADATA	Возвращает из ответа массив метаданных
.. LINKS	Возвращает из ответа массив ссылок
.. ITEMS	Возвращает из ответа массив элементов
.. TAG <string \$#>	Возвращает совпадающие узлы
.. ID <string \$#>	Возвращает элемент (метаданные, ссылку, объект), связанный с ID
.. REL <string \$#>	Возвращает ссылку, связанную с REL
.. NAME <string \$#>	Возвращает элемент (метаданные, ссылку, свойство), связанный с NAME
.. IDS NAMES RELS FORMS TAGS	Возвращает простой список
.. PATH <JSONPath \$#>	Возвращает из ответа SIREN результаты запроса JSONPath
PRAG	Возвращает строго типизированную версию ответа из вершины стека (vnd.prag+json)
.. METADATA	Возвращает массив метаданных из ответа PRAG
.. LINKS	Возвращает массив ссылок из ответа PRAG
.. ITEMS	Возвращает массив элементов из ответа PRAG
.. ID <string \$#>	Возвращает элемент (метаданные, ссылку, объект), связанный с ID
.. TAG <string \$#>	Возвращает совпадающие узлы
.. REL <string \$#>	Возвращает ссылку, связанную с REL
.. NAME <string \$#>	Возвращает элемент (метаданные, ссылку, свойство), связанный с NAME
.. IDS NAMES RELS FORMS TAGS	Возвращает простой список
.. PATH <JSONPath \$#>	Возвращает из ответа SIREN результаты запроса JSONPath

Об авторе

Широко известный автор и спикер **Майк Амундсен** консультирует организации по всему миру на тему сетевой инфраструктуры, веб-разработки, а также взаимосвязей между технологиями и обществом. Он работает с компаниями различных масштабов, помогая им извлекать пользу из возможностей, предоставляемых API, микросервисами и цифровой трансформацией, актуальных как для потребителей, так и для корпораций.

Иллюстрация на обложке

На обложке изображен мунго Грандидье (*Galidictis grandidieri*), названный в честь французского натуралиста и путешественника Альфреда Грандидье, который в течение 1860-х годов несколько раз посещал Мадагаскар. На острове обнаружены шесть видов мангустов. Конкретно мунго Грандидье обитает в очень небольшой удаленной субтропической области рядом с национальным парком Циманампе-цоа на юго-западе Мадагаскара, куда исследователям попасть довольно сложно.

Мунго Грандидье, также известный как гигантский широкополосый мангуст, имеет длину от 32 до 40 см, хвост от 28 до 30 см и восемь темных полос на светло-коричневом мехе. Чтобы избежать палящего зноя, царящего днем в местах его обитания, этот вид ведет ночной образ жизни. Весь спектр диеты этих животных неизвестен, но они точно едят овощи. Мы также не знаем среднюю продолжительность жизни этих мангустов, но было документально зафиксировано, что размножаются они круглый год в моногамных парах.

Мунго Грандидье — вымирающий вид, причем исчезновение угрожает и самой среде их обитания. Многие из животных с обложек книг издательства O'Reilly вскоре могут исчезнуть, но все они важны для нашего мира.

Иллюстрацию для обложки выполнила Карен Монтгомери на основе старинной гравюры из музея *Histoire Naturelle*.

КРОК

СОЗДАЕМ НАСТОЯЩЕЕ,
ИНТЕГРИРУЕМ БУДУЩЕЕ



croc.ru

КРОК — технологический партнер с комплексной экспертизой в области построения и развития инфраструктуры, внедрения информационных систем, разработки программных решений и сервисной поддержки.

Центры компетенций КРОК фокусируются на ключевых отраслевых кластерах — промышленность, финансовый сектор, розничные продажи, муниципальное управление, спорт и культура.

Ежегодно сотни проектов КРОК становятся системообразующими для экономики и социально-культурной сферы.

